

Scenario-Based SQL Library — 500 Scenarios

SQL Interview Prep — 2026 Edition · Learn & Excel

This is the round-2/3 scenario file of the pack. 7 scenario categories — debug, optimize, design schema, what-will-this-return, index decisions, transaction conflicts, data quality investigations. Each scenario follows the same format: tags · question · 30-second answer · step-by-step thought process · edge cases · what NOT to say · common follow-up.

Table of Categories

- Category 1, Part A — Debug This Query: Logic Bugs
- Category 1, Part B — Debug This Query: NULL & Type & Cardinality Bugs
- Category 1, Part C — Debug This Query: Aggregation, Window Function & Recency Bugs
- Category 2, Part A — Optimize This Query: Index-Based Fixes
- Category 2, Part B — Optimize This Query: Query Rewrites
- Category 2, Part C — Optimize This Query: Plan-Driven & Infra Fixes
- Category 3, Part A — Design This Schema: OLTP Patterns
- Category 3, Part B — Design This Schema: Analytics & Warehousing Patterns
- Category 4, Part A — What Will This Return: NULL, Type & Ordering
- Category 4, Part B — What Will This Return: Joins, Windows & Aggregation Edges
- Category 5, Part A — Index Design Decisions: Given This Workload
- Category 5, Part B — Why Is My Index Not Being Used? Plus Maintenance
- Category 6, Part A — Transaction Conflict: Deadlocks & Lock Ordering
- Category 6, Part B — Isolation Level Fixes & Anomaly Scenarios
- Category 7, Part A — Data Quality: Duplicate Investigations
- Category 7, Part B — Data Quality: Missing Rows & Counts Mismatch
- Category 7, Part C — Data Quality: Drift & Audit Trail Gaps
- Category 7, Part D — Data Quality: Timezone, Encoding, Locale & NULL Issues

Category 1, Part A — Debug This Query: Logic Bugs

This is the classic interview opener — the interviewer slides a query across the screen (or pastes it into the shared editor) and says "this is returning wrong results, find the bug." No table sizes, no execution plan, just SQL and the symptom. Part A focuses on **logic bugs** — queries that run cleanly without errors but produce the wrong output: missing rows, extra rows, duplicated rows, or rows in the wrong order. The bugs here are about how SQL semantics differ from how the candidate read the query in their head. Parts B and C cover NULL/coercion bugs and aggregation/window bugs respectively. For each scenario below, the structure is the same: scenario setup, the buggy SQL, expected vs actual, a "thinking out loud" walkthrough that mirrors how you should narrate your debugging on a live screen-share, the diagnosis, the fix, and the common follow-up the interviewer will ask next.

Scenario1 · LEFT JOIN Silently Turned Into INNER JOIN

Tags: Difficulty: Easy · Asked at: Flipkart, Razorpay, Swiggy, Zomato · Dialect: vendor-agnostic

The scenario:

You're asked to list every user along with their total order count. Users who have never placed an order should still appear with a count of zero. The query below runs fine but the analytics team says brand-new signups are missing from the report.

```
SELECT u.user_id, u.name, COUNT(o.order_id) AS total_orders
FROM users u
LEFT JOIN orders o ON o.user_id = u.user_id
WHERE o.status = 'completed'
GROUP BY u.user_id, u.name;
```

Expected: All users, with `total_orders = 0` for users with no completed orders. **Actual:** Users with zero completed orders are missing entirely from the result.

Thinking out loud (what to say while you stare at the screen): 1. "It's a *LEFT JOIN*, so all users from the left side should survive — let me re-read why they're not." 2. "The *WHERE* clause references `o.status` — for users with no orders, `o.status` is *NULL* after the *LEFT JOIN*." 3. "`NULL = 'completed'` is unknown, which *WHERE* treats as false, so those rows get dropped after the join." 4. "That effectively converts my *LEFT JOIN* back into an *INNER JOIN*."

The diagnosis: The *LEFT JOIN* preserves users without orders by filling order columns with *NULL*. The *WHERE* clause then evaluates `NULL = 'completed'`, which is *UNKNOWN*, which *WHERE* treats the same as false. Those preserved rows are filtered out, and you end up with *INNER JOIN* semantics.

The fix:

```
SELECT u.user_id, u.name,  
       COUNT(o.order_id) FILTER (WHERE o.status = 'completed') AS total_orders  
FROM users u  
LEFT JOIN orders o ON o.user_id = u.user_id  
GROUP BY u.user_id, u.name;
```

Or, move the status predicate into the join condition:

```
SELECT u.user_id, u.name, COUNT(o.order_id) AS total_orders  
FROM users u  
LEFT JOIN orders o  
  ON o.user_id = u.user_id  
  AND o.status = 'completed'  
GROUP BY u.user_id, u.name;
```

Why this works: Predicates in the ON clause are evaluated before the outer join decides whether to preserve the left-side row, so non-matching users still survive with NULL on the right.

What NOT to say: - "I'll just add `OR o.status IS NULL` to the WHERE" — that works by accident and breaks if you ever add another status filter. - "LEFT JOIN doesn't work in this database" — it does; the bug is in your filter placement.

Common follow-up: *"What if I also need to filter to orders placed in the last 30 days? Where does that predicate go?"*

Scenario2 · Inner Join Where Outer Was Needed

Tags: Difficulty: Easy · Asked at: PhonePe, Paytm, Meesho · Dialect: vendor-agnostic

The scenario:

A merchant dashboard should show every merchant alongside their last payout amount. Merchants who have never been paid out should show as zero. The report is silently dropping new merchants who haven't seen a payout cycle yet.

```
SELECT m.merchant_id, m.business_name, p.amount AS last_payout  
FROM merchants m  
JOIN payouts p ON p.merchant_id = m.merchant_id  
WHERE p.payout_date = (  
  SELECT MAX(payout_date) FROM payouts WHERE merchant_id = m.merchant_id  
);
```

Expected: Every merchant in the result, with NULL or zero for those without payouts. **Actual:** Merchants with zero payouts are absent.

Thinking out loud: 1. " **JOIN** without **LEFT** means **INNER** — a merchant with no row in payouts is gone immediately." 2. "Even if I correlate the subquery, the outer join is the wrong one." 3. "I need to preserve all merchants and let payouts be optional."

The diagnosis: The plain **JOIN** is **INNER**. New merchants have zero rows in **payouts** and therefore zero rows in the join output. The correlated subquery is a red herring — it filters the matched payout row, but the row had to exist first.

The fix:

```
SELECT m.merchant_id, m.business_name, COALESCE(p.amount, 0) AS last_payout
FROM merchants m
LEFT JOIN payouts p
  ON p.merchant_id = m.merchant_id
 AND p.payout_date = (
   SELECT MAX(payout_date) FROM payouts WHERE merchant_id = m.merchant_id
 );
```

Why this works: **LEFT JOIN** preserves every merchant; the correlated subquery still picks the latest payout when one exists; **COALESCE** shows zero when none does.

What NOT to say: - "I'll **UNION** the missing merchants" — wasteful and easy to get wrong. - "Just add **WHERE p.merchant_id IS NOT NULL OR TRUE** " — meaningless noise.

Common follow-up: "Rewrite this without a correlated subquery — use a window function or **DISTINCT ON**."

Scenario3 · Duplicate Rows From One-to-Many Join

Tags: Difficulty: Easy · Asked at: Razorpay, Cred, Zerodha · Dialect: vendor-agnostic

The scenario:

You're computing revenue per order. Each order has multiple line items, and the join is producing rows that, when summed, inflate revenue 3-4x. The CFO is asking pointed questions.

```
SELECT SUM(o.order_total) AS revenue
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.created_at >= '2026-01-01';
```

Expected: Sum of distinct order totals for the period. **Actual:** Each order's total is summed once per line item, inflating revenue.

Thinking out loud: 1. "`SUM(o.order_total)` — *order_total is on the orders table, but the FROM has a 1-to-many join.*" 2. "*For an order with 4 items, that order_total shows up 4 times in the joined row set.*" 3. "*Either remove the join (if I don't need item data), or sum line items instead, or de-dupe before summing.*"

The diagnosis: The join multiplies each order row by its number of items. Summing the order-level total across multiplied rows inflates the result. Classic fan-out bug.

The fix: If you don't need item data at all:

```
SELECT SUM(order_total) AS revenue
FROM orders
WHERE created_at >= '2026-01-01';
```

If you need it for another part of the query:

```
SELECT SUM(oi.quantity * oi.unit_price) AS revenue
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.created_at >= '2026-01-01';
```

Why this works: Either we drop the join that caused the duplication, or we sum at the line-item grain where each row genuinely contributes once.

What NOT to say: - "I'll just divide by the average items per order" — terrifying. - "Use SELECT DISTINCT" — DISTINCT on aggregated rows doesn't fix this and hides the real issue.

Common follow-up: "*Now compute revenue and item count in a single query.*"

Scenario4 · ORDER BY With LIMIT, Non-Deterministic Order

Tags: Difficulty: Easy · Asked at: Swiggy, Zomato, Dunzo · Dialect: vendor-agnostic

The scenario:

"Top 10 most recent orders" page is showing a different set of 10 orders every time the dashboard reloads, even though no new orders are coming in during the test window.

```
SELECT order_id, user_id, created_at, status
FROM orders
ORDER BY created_at DESC
LIMIT 10;
```

Expected: A stable set of 10 most recent orders. **Actual:** Same `created_at` value on many orders; the 10 returned shuffle between requests.

Thinking out loud: 1. "ORDER BY is there, so the order key is deterministic — but if many rows share the same `created_at`, ties are broken arbitrarily." 2. "If a hundred orders share the same second, any 10 of them might surface." 3. "I need a tiebreaker, ideally something unique."

The diagnosis: `created_at` likely has duplicates (same second, or same default-precision timestamp). When the ORDER BY key has ties, SQL is free to return any ordering of the tied rows, and that ordering can change across executions.

The fix:

```
SELECT order_id, user_id, created_at, status
FROM orders
ORDER BY created_at DESC, order_id DESC
LIMIT 10;
```

Why this works: Adding a unique tiebreaker (`order_id` is the PK) makes the ordering total, so the same 10 rows surface every time.

What NOT to say: - "Pagination is broken in our database" — your ORDER BY is broken. - "Add `ORDER BY 1, 2, 3, 4`" — random columns don't guarantee uniqueness either.

Common follow-up: "What if I want pages 2, 3, 4 of this list and the data is being written to continuously?"

Scenario5 · LIMIT Without ORDER BY

Tags: Difficulty: Easy · Asked at: Ola, Uber India, Rapido · Dialect: vendor-agnostic

The scenario:

A new dev wrote "give me 100 sample users for a CSV export". The query works, but the same 100 users don't come back twice in a row, and they're not even close to a random sample — they all seem to be from one city.

```
SELECT user_id, name, email, city
FROM users
LIMIT 100;
```

Expected: A consistent or at least clearly-defined set of 100 users. **Actual:** Set varies between runs; heavily biased toward whatever rows the storage engine returns first.

Thinking out loud: 1. "LIMIT without ORDER BY returns any 100 rows the planner finds first." 2. "That depends on physical storage order, current index choice, parallel workers — none of it is stable." 3. "If the user wants 'first 100 by signup', say so in SQL. If they want random, say that in SQL."

The diagnosis: LIMIT does not imply any order. The engine returns whichever rows are cheapest to surface — often the first physical page, which clusters by insertion or by some index. The result is unstable and biased.

The fix: For deterministic "first 100 by signup":

```
SELECT user_id, name, email, city
FROM users
ORDER BY signup_date, user_id
LIMIT 100;
```

For a random sample (Postgres):

```
SELECT user_id, name, email, city
FROM users TABLESAMPLE BERNOULLI (1)
LIMIT 100;
```

Why this works: The first version defines exactly which 100 rows; the second uses an actual sampling mechanism instead of relying on physical order.

What NOT to say: - "Add ORDER BY RAND()" — works on small tables, melts your CPU on large ones. - "LIMIT is broken" — LIMIT is fine; the missing ORDER BY is on you.

Common follow-up: "How would you take a reproducible 1% sample week after week, with the same users each time?"

Scenario6 · DISTINCT Applied Too Late

Tags: Difficulty: Medium · Asked at: Cred, Razorpay, Jupiter · Dialect: vendor-agnostic

The scenario:

A "unique customers who paid today" tile is slow and returning a too-high number. The query uses DISTINCT but is being run against the joined transactions-and-payments view.

```
SELECT DISTINCT t.user_id, u.name, u.email
FROM transactions t
JOIN payments p ON p.transaction_id = t.transaction_id
JOIN users u ON u.user_id = t.user_id
WHERE t.txn_date = CURRENT_DATE;
```

Expected: Unique paying customers. **Actual:** Count is plausible at first glance but reconciliation against payments-only shows it's slightly too high; query is also slow because the joins fan out before the de-duplication.

Thinking out loud: 1. "DISTINCT is at the top — it dedupes the final result set, which is correct for the rows we project." 2. "But the joins multiply rows before the dedupe, so we're doing extra work and the join with users could double-fire if there are dirty user rows." 3. "Better to dedupe earlier — at the transactions level — and join users afterward."

The diagnosis: DISTINCT works on the final projected rows. If the join with `payments` produces multiple payment rows per transaction (partial payments, retries), the `user_id` is repeated, then de-duped. If there's any dirty data in `users` (duplicate emails for same `user_id`), DISTINCT can hide it or, if you include such a duplicate column, expose it. Pushing the dedupe earlier is cheaper and clearer.

The fix:

```
WITH paying_users AS (
  SELECT DISTINCT t.user_id
  FROM transactions t
  JOIN payments p ON p.transaction_id = t.transaction_id
  WHERE t.txn_date = CURRENT_DATE
)
SELECT pu.user_id, u.name, u.email
FROM paying_users pu
JOIN users u ON u.user_id = pu.user_id;
```

Why this works: We dedupe at the smallest grain (`user_id` only), then enrich. The final join with `users` happens against a tidy unique set.

What NOT to say: - "DISTINCT is always bad" — it isn't, it just needs to be applied at the right grain. - "Add `GROUP BY` everywhere" — that hides the issue rather than fixing it.

Common follow-up: "What if `users` itself has duplicate rows per `user_id` — how would your query behave?"

Scenario7 · GROUP BY Missing A Column

Tags: Difficulty: Medium · Asked at: Flipkart, Myntra, Nykaa · Dialect: Postgres / SQL Server (MySQL behaves differently)

The scenario:

A report "orders per city per status" works on the dev's MySQL laptop but blows up on the prod Postgres database with an error about non-aggregated columns. The dev says "it works for me."

```
SELECT u.city, o.status, COUNT(*) AS order_count
FROM orders o
JOIN users u ON u.user_id = o.user_id
GROUP BY u.city;
```

Expected: Per-city per-status counts. **Actual** (Postgres): `ERROR: column "o.status" must appear in the GROUP BY clause`. (MySQL with relaxed mode): runs but returns one arbitrary status per city.

Thinking out loud: 1. "I'm selecting `o.status` but only grouping by `u.city` — every selected non-aggregate column must be in GROUP BY." 2. "MySQL with `ONLY_FULL_GROUP_BY` off picks a random status per group, which is why it 'worked' on the laptop." 3. "Add `o.status` to GROUP BY."

The diagnosis: Standard SQL (and Postgres/SQL Server by default) requires every non-aggregate column in the SELECT list to be in GROUP BY. MySQL historically allowed it and returned arbitrary values, which is almost always a silent bug.

The fix:

```
SELECT u.city, o.status, COUNT(*) AS order_count
FROM orders o
JOIN users u ON u.user_id = o.user_id
GROUP BY u.city, o.status;
```

Why this works: Now each output row corresponds to exactly one (city, status) pair, and the count is meaningful.

What NOT to say: - "Turn off `ONLY_FULL_GROUP_BY` in prod" — that hides the bug; don't do it. - "Use `ANY_VALUE(o.status)`" — only valid when you genuinely don't care which status is picked, which is almost never.

Common follow-up: "What if I want city totals plus per-status breakdown in one query? GROUPING SETS?"

Scenario8 · WHERE Used Where HAVING Is Required

Tags: Difficulty: Easy · Asked at: Razorpay, PhonePe, Paytm · Dialect: vendor-agnostic

The scenario:

A query meant to list cities with more than 100 orders is throwing "column COUNT(*) does not exist" or "aggregate functions are not allowed in WHERE".

```
SELECT u.city, COUNT(*) AS order_count
FROM orders o
JOIN users u ON u.user_id = o.user_id
WHERE COUNT(*) > 100
GROUP BY u.city;
```

Expected: Cities with more than 100 orders. **Actual:** Syntax/semantic error about aggregates in WHERE.

Thinking out loud: 1. "WHERE filters input rows before grouping. COUNT() is only defined after grouping." 2. "The filter on an aggregate has to be in HAVING."*

The diagnosis: WHERE runs row-by-row before aggregation; aggregates aren't available yet. HAVING runs after GROUP BY and is allowed to filter on aggregate results.

The fix:

```
SELECT u.city, COUNT(*) AS order_count
FROM orders o
JOIN users u ON u.user_id = o.user_id
GROUP BY u.city
HAVING COUNT(*) > 100;
```

Why this works: HAVING evaluates after the aggregation, so `COUNT(*)` is defined and can be compared.

What NOT to say: - "Wrap it in a subquery" — works but unnecessarily indirect for this case. - "WHERE and HAVING are interchangeable" — no, they really aren't.

Common follow-up: "When would you use a subquery anyway, despite HAVING being legal?"

Scenario9 · HAVING Used Where WHERE Is Better

Tags: Difficulty: Medium · Asked at: Flipkart, Meesho, Snapdeal · Dialect: vendor-agnostic

The scenario:

Query is correct but painfully slow. It aggregates the entire orders table by city and then filters out cities outside India. The filter on country should have happened earlier.

```
SELECT u.city, COUNT(*) AS order_count
FROM orders o
JOIN users u ON u.user_id = o.user_id
GROUP BY u.city, u.country
HAVING u.country = 'IN' AND COUNT(*) > 50;
```

Expected: Cities in India with more than 50 orders. **Actual:** Result is right but the query scans and groups every row globally, then discards non-IN groups.

Thinking out loud: 1. "`u.country = 'IN'` is a non-aggregate predicate — it doesn't depend on COUNT." 2. "In HAVING, it runs after the giant aggregation. Moving it to WHERE shrinks the input dramatically." 3. "COUNT() > 50 must stay in HAVING — that one is genuinely an aggregate."*

The diagnosis: HAVING is meant for aggregate filters. Non-aggregate predicates belong in WHERE so they shrink the input before grouping happens.

The fix:

```
SELECT u.city, COUNT(*) AS order_count
FROM orders o
JOIN users u ON u.user_id = o.user_id
WHERE u.country = 'IN'
GROUP BY u.city
HAVING COUNT(*) > 50;
```

Why this works: WHERE narrows the input, the aggregation runs on a much smaller set, and HAVING only handles the aggregate predicate.

What NOT to say: - "It's fine, HAVING does the same thing" — semantically yes, but the planner can't always push it down. - "Add an index on city" — useful, but not the bug.

Common follow-up: "Show me the execution plan before and after the change."

Scenario10 · ON Vs WHERE In Outer Joins

Tags: Difficulty: Medium · Asked at: Cred, Jupiter, Niyo · Dialect: vendor-agnostic

The scenario:

Two versions of a query return different numbers. The candidate insists they're equivalent because "filter is filter." The interviewer asks them to walk through why a particular customer disappeared.

```
-- Version A
SELECT u.user_id, b.booking_id
FROM users u
LEFT JOIN bookings b
  ON b.user_id = u.user_id
WHERE b.status = 'confirmed';

-- Version B
SELECT u.user_id, b.booking_id
FROM users u
LEFT JOIN bookings b
  ON b.user_id = u.user_id
AND b.status = 'confirmed';
```

Expected: One of these preserves all users; the other doesn't. **Actual:** Version A behaves like an INNER JOIN; Version B preserves users with no confirmed booking (and NULL booking_id for them).

Thinking out loud: 1. "In a LEFT JOIN, predicates in ON apply before the outer-join decision." 2. "Predicates in WHERE apply after the join, so they can filter out the preserved NULL-padded rows." 3. "Version A's WHERE on `b.status` drops the NULL rows — losing users with no confirmed booking."

The diagnosis: For outer joins, ON and WHERE are not interchangeable. ON shapes the match; WHERE shapes the final filter. Filtering the right-side table in WHERE silently converts the outer join into an inner one.

The fix: Use the appropriate semantics. Version B keeps all users. If the goal is to keep only users with a confirmed booking, an INNER JOIN is more honest:

```
SELECT u.user_id, b.booking_id
FROM users u
JOIN bookings b
  ON b.user_id = u.user_id
AND b.status = 'confirmed';
```

Why this works: You're now expressing the intent explicitly — "I want users who have a confirmed booking" — rather than writing LEFT JOIN you didn't mean.

What NOT to say: - "ON and WHERE are the same thing" — for outer joins, they're materially different. - "Just use WHERE for everything, it's simpler" — simpler-looking, harder bug.

Common follow-up: "Now write me a query that returns every user and the count of their confirmed bookings, including users with zero."

Scenario11 · Self-Join Missing The Exclusion

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Zerodha · Dialect: vendor-agnostic

The scenario:

Finding pairs of transactions on the same user, same amount, within a 5-minute window — meant to flag possible duplicates. Output is dominated by rows where each transaction is paired with itself.

```
SELECT a.txn_id AS a_id, b.txn_id AS b_id, a.user_id, a.amount
FROM transactions a
JOIN transactions b
  ON a.user_id = b.user_id
 AND a.amount = b.amount
 AND ABS(EXTRACT(EPOCH FROM (a.created_at - b.created_at))) < 300;
```

Expected: Pairs of distinct transactions. **Actual:** Every transaction is paired with itself (a=a), and each genuine pair appears twice (a-b and b-a).

Thinking out loud: 1. "Self-join — every row matches itself trivially because all conditions are equal to themselves." 2. "And every pair (a, b) shows up as (b, a) too." 3. "Add `a.txn_id < b.txn_id` — kills both birds."

The diagnosis: A self-join joins the table with itself, so every row trivially matches itself when the join columns are equal to themselves. Symmetric pairs (a-b and b-a) also both appear unless you impose an ordering predicate.

The fix:

```
SELECT a.txn_id AS a_id, b.txn_id AS b_id, a.user_id, a.amount
FROM transactions a
JOIN transactions b
  ON a.user_id = b.user_id
 AND a.amount = b.amount
 AND ABS(EXTRACT(EPOCH FROM (a.created_at - b.created_at))) < 300
 AND a.txn_id < b.txn_id;
```

Why this works: `a.txn_id < b.txn_id` excludes the self-match (where they're equal) and the duplicate orientation (a-b is kept, b-a dropped).

What NOT to say: - "Use `a.txn_id != b.txn_id`" — still gives you the duplicate orientation. - "Add DISTINCT" — wasteful and doesn't kill self-matches cleanly.

Common follow-up: "What if you need to find triples instead of pairs?"

Scenario12 · Missing Join Condition Produces Cartesian Product

Tags: Difficulty: Easy · Asked at: any startup with junior devs · Dialect: vendor-agnostic

The scenario:

"Pull all orders along with the user's mobile number." The query runs for ages and returns billions of rows on a tiny dev dataset.

```
SELECT o.order_id, u.mobile, o.amount
FROM orders o, users u
WHERE o.amount > 0;
```

Expected: One row per order. **Actual:** Every order × every user. Cartesian product.

Thinking out loud: 1. "Comma syntax cross-joins by default — there's no ON condition." 2. "The only predicate is on amount; nothing links orders to their users." 3. "Need to add the join condition on user_id."

The diagnosis: Comma in the FROM clause is a CROSS JOIN. Without a predicate linking the two tables, you get the full cartesian product. The amount filter doesn't help — it just narrows the cartesian rows.

The fix:

```
SELECT o.order_id, u.mobile, o.amount
FROM orders o
JOIN users u ON u.user_id = o.user_id
WHERE o.amount > 0;
```

Why this works: Explicit `JOIN ... ON ...` makes the linkage obvious and forces you to think about it. Each order matches exactly one user.

What NOT to say: - "The database is slow today" — no, you wrote a cross join. - "Add `LIMIT 1000`" — masks the bug and gives wrong results.

Common follow-up: *"Why do we still see this bug in 2026? Should the linter catch it?"*

Scenario13 · Implicit Cross Join With Three+ Tables

Tags: Difficulty: Medium · Asked at: Infosys, Wipro, TCS Digital · Dialect: vendor-agnostic

The scenario:

Old-style comma joins across three tables. The query *almost* works because two of the three are linked, but the third sneaks in as a cross join.

```
SELECT o.order_id, u.name, p.payment_method
FROM orders o, users u, payments p
WHERE o.user_id = u.user_id;
```

Expected: One row per order with the user's name and the payment method. **Actual:** Each order-user row is multiplied by every row in `payments`.

Thinking out loud: 1. "Three tables, comma-joined. Two are linked via the `user_id` predicate. Payments has no join predicate." 2. "That makes payments a cross join against the other two." 3. "Need `p.order_id = o.order_id` (or whichever link exists)."

The diagnosis: With comma joins, *every* pair of tables must have its linking predicate in WHERE. Miss one, and you get a partial cartesian. With explicit ANSI joins, the structure forces you to write the linkage table-by-table.

The fix:

```
SELECT o.order_id, u.name, p.payment_method
FROM orders o
JOIN users    u ON u.user_id = o.user_id
JOIN payments p ON p.order_id = o.order_id;
```

Why this works: ANSI JOIN syntax binds each predicate to a specific table pair, so it's structurally impossible to forget one without a syntax error or obvious gap.

What NOT to say: - "Comma joins are equivalent to JOIN ON" — they are, but only when you remember every predicate. - "Old syntax is fine" — fine for one table, dangerous for many.

Common follow-up: "Convert this codebase from comma joins to ANSI joins — how would you scope the change?"

Scenario14 · Scalar Subquery Returning Multiple Rows

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Niyo · Dialect: vendor-agnostic

The scenario:

A query that ran fine last quarter just started failing in prod with "more than one row returned by a subquery used as an expression." Test data had one row per user; prod has more than one for some users.

```
SELECT u.user_id,  
       u.name,  
       (SELECT amount FROM payments WHERE user_id = u.user_id) AS latest_payment  
FROM users u;
```

Expected: One value per user. **Actual:** Error when a user has more than one payment.

Thinking out loud: 1. "Scalar subquery — must return at most one row." 2. " `payments` can have many rows per user, especially in prod." 3. "Need to either aggregate (MAX, SUM) or order-and-limit to one row."

The diagnosis: A subquery in the SELECT list must return at most one row and one column. With multiple payments per user, the engine doesn't know which one to pick and raises an error.

The fix: For latest payment by date (Postgres):

```
SELECT u.user_id,  
       u.name,  
       (SELECT amount  
        FROM payments  
        WHERE user_id = u.user_id  
        ORDER BY paid_at DESC  
        LIMIT 1) AS latest_payment  
FROM users u;
```

For total payments:


```
SELECT u.user_id,  
       u.name,  
       (SELECT COALESCE(SUM(amount), 0)  
        FROM payments WHERE user_id = u.user_id) AS total_payments  
FROM users u;
```

Why this works: Either we collapse multiple rows into one with an aggregate, or we pick exactly one with ORDER BY + LIMIT.

What NOT to say: - "Add DISTINCT inside the subquery" — only works when the amounts happen to be identical. - "It was fine in dev, must be a database bug" — your dev data was too thin.

Common follow-up: *"Rewrite this with a LEFT JOIN and a window function instead of a correlated subquery."*

Scenario15 · IN Subquery Contains NULLs

Tags: Difficulty: Medium · Asked at: Flipkart, Walmart Labs India, Myntra · Dialect: vendor-agnostic

The scenario:

A query "users who have placed at least one order" is mysteriously returning zero rows in one environment. Same query, same data shape, fewer matches than expected.

```
SELECT user_id, name  
FROM users  
WHERE user_id IN (SELECT user_id FROM orders);
```

Expected: All users who appear in orders. **Actual:** In one environment, returns no rows.

Thinking out loud: 1. "IN works fine even with NULLs if there's at least one matching non-NULL row." 2. "But if `orders.user_id` has any NULL, and the candidate value doesn't match any non-NULL row, IN returns UNKNOWN, which WHERE treats as false." 3. "For positive IN it's usually okay — the bug is more famous for NOT IN. But there can still be edge cases here."

The diagnosis: Positive `IN` mostly behaves intuitively even with NULL values in the subquery because any true match short-circuits the comparison. The classic failure mode is when the outer value is itself NULL, in which case `NULL IN (...)` is UNKNOWN and the row is dropped. If the report is looking at "users with NULL user_id" (data-quality investigation), this scenario surfaces.

The fix:

```
SELECT u.user_id, u.name
FROM users u
WHERE EXISTS (
    SELECT 1 FROM orders o WHERE o.user_id = u.user_id
);
```

Why this works: EXISTS evaluates a clean boolean per row and is not confused by NULL columns in the inner query.

What NOT to say: - "Just `WHERE user_id IS NOT NULL` " — fixes the symptom, not the habit. - "IN doesn't work in this database" — it does, but its NULL semantics bite.

Common follow-up: *"Now show me the NOT IN version of this bug, which is much worse."*

Scenario16 · NOT IN With Subquery Containing NULL

Tags: Difficulty: Hard · Asked at: Cred, Razorpay, Jupiter, Niyo · Dialect: vendor-agnostic

The scenario:

"Users who have *not* placed any order" returns zero rows. The team is staring at it because they know for a fact there are inactive users in the table.

```
SELECT user_id, name
FROM users
WHERE user_id NOT IN (SELECT user_id FROM orders);
```

Expected: Users with no orders. **Actual:** Empty result. Always.

Thinking out loud: 1. " `NOT IN` against a subquery — if the subquery has even one NULL, every NOT IN comparison turns into UNKNOWN." 2. "WHERE drops UNKNOWN rows, so we get nothing." 3. "Three-valued logic strikes again. NOT EXISTS doesn't have this problem."

The diagnosis: `x NOT IN (a, b, NULL)` becomes `x != a AND x != b AND x != NULL`. The last comparison is UNKNOWN. AND with UNKNOWN is at best UNKNOWN, never true. So the row is dropped. One NULL in the subquery's projected column wipes out the entire result.

The fix: Use NOT EXISTS:

```
SELECT u.user_id, u.name
FROM users u
WHERE NOT EXISTS (
  SELECT 1 FROM orders o WHERE o.user_id = u.user_id
);
```

Or filter NULLs out of the subquery, but NOT EXISTS is safer:

```
SELECT user_id, name
FROM users
WHERE user_id NOT IN (
  SELECT user_id FROM orders WHERE user_id IS NOT NULL
);
```

Why this works: NOT EXISTS uses per-row existence checks, which are clean booleans. The second form removes the NULLs before NOT IN sees them.

What NOT to say: - "NOT IN is just like NOT EXISTS" — they differ exactly when NULLs are involved, and that's the bug. - "Add **IS NOT NULL** to the outer WHERE" — wrong side; the NULLs are inside.

Common follow-up: *"Why does NOT EXISTS not have this problem semantically?"*

Scenario17 · EXISTS Vs IN Semantic Mismatch

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Paytm · Dialect: vendor-agnostic

The scenario:

A dev rewrote an **IN** query as **EXISTS** for "performance" and now the results differ. They swear the queries are equivalent.

```
-- Original
SELECT user_id, name
FROM users
WHERE user_id IN (
    SELECT u2.user_id
    FROM users u2
    JOIN orders o ON o.user_id = u2.user_id
    WHERE o.amount > 5000
);

-- Rewrite
SELECT user_id, name
FROM users u
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.amount > 5000
);
```

Expected: Users with at least one order above 5000. **Actual:** The rewrite returns *all* users, because the EXISTS subquery is uncorrelated.

Thinking out loud: 1. "EXISTS works on a correlated subquery — it has to reference the outer row." 2. "The rewrite has no `o.user_id = u.user_id` link; it just asks 'does any high-value order exist anywhere?'" 3. "If even one exists, EXISTS is true for every outer row."

The diagnosis: EXISTS without a correlation predicate is equivalent to "is the subquery non-empty?" — a global yes/no, not a per-user yes/no.

The fix:

```
SELECT u.user_id, u.name
FROM users u
WHERE EXISTS (
    SELECT 1 FROM orders o
    WHERE o.user_id = u.user_id AND o.amount > 5000
);
```

Why this works: The correlation `o.user_id = u.user_id` makes the EXISTS check per-user.

What NOT to say: - "EXISTS is always faster than IN" — sometimes; not always; not the point here. - "Add DISTINCT to the outer query" — doesn't fix anything.

Common follow-up: "When would you actually prefer IN over EXISTS, and vice versa?"

Scenario18 · CASE Without ELSE Silently Returns NULL

Tags: Difficulty: Easy · Asked at: any analytics interview · Dialect: vendor-agnostic

The scenario:

A status-bucketing query is showing NULL for some orders. The dev expected the unhandled cases to fall into a default like 'other'.

```
SELECT order_id,  
       CASE  
         WHEN status = 'paid'      THEN 'success'  
         WHEN status = 'refunded' THEN 'reversed'  
         WHEN status = 'failed'   THEN 'failure'  
       END AS status_group  
FROM orders;
```

Expected: Every order has a status_group. **Actual:** Orders with status like 'pending' or 'partial_refund' get NULL.

Thinking out loud: 1. "CASE without ELSE returns NULL when nothing matches." 2. "I never wrote a catch-all bucket." 3. "Need an explicit ELSE clause."

The diagnosis: A CASE expression returns NULL for any input that doesn't match any WHEN branch. There's no implicit "else default" — you have to write one.

The fix:

```
SELECT order_id,  
       CASE  
         WHEN status = 'paid'      THEN 'success'  
         WHEN status = 'refunded' THEN 'reversed'  
         WHEN status = 'failed'   THEN 'failure'  
         ELSE 'other'  
       END AS status_group  
FROM orders;
```

Why this works: The ELSE branch guarantees a non-NULL value for every row, so downstream code doesn't have to special-case NULL.

What NOT to say: - "I'll wrap it in COALESCE" — works but hides the intent; ELSE is more obvious. - "There shouldn't be any other status values" — there always are, eventually.

Common follow-up: "How would you flag rows that hit the ELSE branch so the data team can investigate?"

Scenario19 · BETWEEN Endpoints Confusion

Tags: Difficulty: Easy · Asked at: Wipro, Infosys, Capgemini · Dialect: vendor-agnostic

The scenario:

A candidate inserted "BETWEEN 18 AND 25" for age, expecting 19-24. Output includes 18 and 25, leading to confusion downstream.

```
SELECT user_id, name, age
FROM users
WHERE age BETWEEN 18 AND 25;
```

Expected (per dev's mental model): 19-24. **Actual:** 18, 19, 20, 21, 22, 23, 24, 25.

Thinking out loud: 1. "BETWEEN is inclusive on both ends in SQL." 2. "If I want strict bounds, I need to use `>` and `<` explicitly."

The diagnosis: SQL's `BETWEEN x AND y` is shorthand for `>= x AND <= y` — both endpoints included. This is a constant source of off-by-one bugs because mathematical "between" is often read as exclusive.

The fix:

```
SELECT user_id, name, age
FROM users
WHERE age > 18 AND age < 25;
```

Or, if 18 was the actual lower bound intent:

```
SELECT user_id, name, age
FROM users
WHERE age >= 18 AND age < 26;
```

Why this works: Explicit operators leave no ambiguity about endpoint behavior.

What NOT to say: - "BETWEEN should be exclusive, this is a SQL bug" — it's defined behavior. - "Use NOT BETWEEN to invert" — fine for inverting, not relevant here.

Common follow-up: "Now do the same for a date range — what's the gotcha there?"

Scenario20 · Date Range Off-By-Day With Timestamps

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy · Dialect: vendor-agnostic

The scenario:

A "March transactions" report shows 31-day numbers that don't match Finance's reconciliation. Finance says we're including one extra day on the end, or missing one at the start, depending on which month.

```
SELECT SUM(amount) AS march_total
FROM transactions
WHERE created_at BETWEEN '2026-03-01' AND '2026-03-31';
```

Expected: All March transactions. **Actual:** Misses transactions on 31 March after midnight (because `'2026-03-31'` is implicitly `'2026-03-31 00:00:00'`).

Thinking out loud: 1. "`created_at` is a timestamp. `'2026-03-31'` casts to `2026-03-31 00:00:00`." 2. "*BETWEEN is inclusive but only up to that instant — anything after 00:00:00 on 31 March is excluded.*" 3. "*Need to use a half-open interval up to 1 April.*"

The diagnosis: Comparing a timestamp column to a date literal anchors the comparison at midnight of that date. Inclusive BETWEEN misses everything that happens *during* the last day. The robust pattern is a half-open interval `>= start AND < next_day`.

The fix:

```
SELECT SUM(amount) AS march_total
FROM transactions
WHERE created_at >= '2026-03-01'
      AND created_at < '2026-04-01';
```

Why this works: Half-open intervals scale to any precision (date, timestamp, timestamptz) without subtle inclusion bugs.

What NOT to say: - "Use BETWEEN '2026-03-01' AND '2026-03-31 23:59:59'" — misses anything in the last second, and `.999999` precision games are fragile. - "Cast `created_at` to DATE" — works but kills index usage on `created_at`.

Common follow-up: "*Now do this in the user's local timezone instead of UTC.*"

Scenario21 · Week Starts Monday Vs Sunday

Tags: Difficulty: Medium · Asked at: Cred, Swiggy, Dunzo · Dialect: Postgres / MySQL differ

The scenario:

A weekly cohort report buckets transactions by week. Numbers look off by one day at the boundary, depending on the database.

```
SELECT DATE_TRUNC('week', created_at) AS wk, SUM(amount)
FROM transactions
GROUP BY DATE_TRUNC('week', created_at);
```

Expected (Finance): weeks start Sunday in their model. **Actual** (Postgres): weeks start Monday.

Thinking out loud: 1. "Postgres `DATE_TRUNC('week', ...)` follows ISO 8601 — weeks start Monday." 2. "MySQL's `WEEK()` defaults to Sunday-start in many configurations." 3. "If Finance reports against Sunday-start weeks, I need to align."

The diagnosis: Different databases (and different settings within MySQL) use different week-start conventions. ISO 8601 says Monday; US convention often uses Sunday.

The fix (Postgres, simulate Sunday-start):

```
SELECT DATE_TRUNC('week', created_at + INTERVAL '1 day') - INTERVAL '1 day' AS wk_start,
       SUM(amount)
FROM transactions
GROUP BY DATE_TRUNC('week', created_at + INTERVAL '1 day');
```

Or document and standardise on ISO Monday-start and educate Finance, which is honestly cleaner.

Why this works: Adding a day before truncation shifts the week boundary by one day, then subtracting it back gives a Sunday-anchored bucket.

What NOT to say: - "Postgres is buggy" — it's just opinionated. - "Use `EXTRACT(DOW, ...)` and group on that" — doesn't bucket by week, only by day-of-week.

Common follow-up: "How would you make this configurable per tenant?"

Scenario22 · Timezone Mismatch: UTC In DB, IST In Business

Tags: Difficulty: Hard · Asked at: Razorpay, PhonePe, Cred · Dialect: Postgres

The scenario:

A daily revenue dashboard shows the wrong day's numbers between midnight IST and 5:30am IST. Sales recorded "today in India" land in "yesterday in UTC", or vice versa.

```
SELECT DATE(created_at) AS dt, SUM(amount) AS revenue
FROM transactions
GROUP BY DATE(created_at);
```

Expected: Daily totals in IST. **Actual:** Daily totals in UTC, which puts post-midnight-IST transactions onto the previous day.

Thinking out loud: 1. " `created_at` is `timestampz` stored in UTC. `DATE()` casts to UTC date." 2. "India is UTC+5:30. Anything created between 18:30 UTC and 23:59 UTC is 'tomorrow' in IST." 3. "Need to convert to IST before truncating."

The diagnosis: By default, casting a UTC-stored timestamp to date uses UTC. Without an explicit timezone conversion, your day boundary is in UTC, not in the user's or business's timezone.

The fix (Postgres):

```
SELECT (created_at AT TIME ZONE 'Asia/Kolkata')::date AS dt_ist,
       SUM(amount) AS revenue
FROM transactions
GROUP BY (created_at AT TIME ZONE 'Asia/Kolkata')::date;
```

Why this works: `AT TIME ZONE 'Asia/Kolkata'` shifts the stored timestamp into IST wall-clock before we truncate to a date, so the day boundaries align with the business calendar.

What NOT to say: - "Just add 5.5 hours to `created_at`" — works around DST issues for India today, but fragile. - "Store everything in IST instead" — terrible idea for multi-region apps.

Common follow-up: "What if you have customers in Singapore (UTC+8) and Dubai (UTC+4) and need per-customer-local daily totals?"

Scenario23 · Boolean Column In WHERE Without Explicit Compare

Tags: Difficulty: Easy · Asked at: any · Dialect: Postgres vs SQL Server differ

The scenario:

A migration moves a query from Postgres to SQL Server. It refuses to run with "An expression of non-boolean type specified in a context where a condition is expected."

```
SELECT user_id, name
FROM users
WHERE is_active;
```

Expected: Active users. **Actual** (SQL Server): error. (Postgres): works fine.

Thinking out loud: 1. "Postgres has a real boolean type; `is_active` alone is a valid condition." 2. "SQL Server doesn't have a native boolean — `BIT` columns need explicit comparison." 3. "Write it as `is_active = 1` (or whatever truthy value the schema uses)."

The diagnosis: Boolean handling is not uniform across SQL dialects. Postgres allows a boolean column as a WHERE condition directly; SQL Server requires an explicit equality against the BIT representation. MySQL allows it because BOOLEAN is an alias for TINYINT.

The fix (portable):

```
SELECT user_id, name
FROM users
WHERE is_active = TRUE;    -- Postgres, MySQL
-- or
WHERE is_active = 1;      -- SQL Server (BIT)
```

Why this works: Explicit comparison removes the ambiguity and works consistently across dialects (with the value adjusted per dialect).

What NOT to say: - "SQL Server is broken" — it's just stricter. - "Cast it to boolean" — SQL Server still won't accept a non-boolean expression in a condition context.

Common follow-up: "How would you keep this portable across Postgres and SQL Server in the same codebase?"

Scenario24 · LIKE Without A Wildcard

Tags: Difficulty: Easy · Asked at: any · Dialect: vendor-agnostic

The scenario:

A search field uses LIKE but is behaving as exact match. "ravi" returns one Ravi instead of all the Ravis.

```
SELECT user_id, name
FROM users
WHERE name LIKE 'ravi';
```

Expected: Names starting with or containing 'ravi'. **Actual:** Only exact match 'ravi' (case-sensitive too on some collations).

Thinking out loud: 1. "LIKE without % or _ is functionally =." 2. "Need 'ravi%' for prefix, '%ravi%' for substring, '%ravi' for suffix." 3. "And consider case-insensitivity — ILIKE on Postgres, LOWER() everywhere else."

The diagnosis: LIKE only filters by pattern; without wildcards there's no pattern to match, so it behaves as equality. Add the wildcard you actually meant.

The fix:

```
-- Substring search, case-insensitive (Postgres)
SELECT user_id, name
FROM users
WHERE name ILIKE '%ravi%';

-- Portable
SELECT user_id, name
FROM users
WHERE LOWER(name) LIKE '%ravi%';
```

Why this works: The wildcard enables substring matching; ILIKE or LOWER() handles case folding.

What NOT to say: - "Replace LIKE with =" — that's what it's already doing, badly. - "Add wildcards on both sides always" — kills index usage; only do it when needed.

Common follow-up: "How would you make this search fast on a 10-million-row table?"

Scenario25 · Operator Precedence: AND Vs OR

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Swiggy · Dialect: vendor-agnostic

The scenario:

A query "high-value orders in Mumbai or Bangalore" is returning every Mumbai order regardless of value, plus only the high-value Bangalore ones.

```
SELECT order_id, city, amount
FROM orders
WHERE city = 'Mumbai' OR city = 'Bangalore' AND amount > 10000;
```

Expected: High-value orders in Mumbai or Bangalore. **Actual:** All Mumbai orders + only Bangalore orders above 10000.

Thinking out loud: 1. "AND binds tighter than OR." 2. "That parses as `city='Mumbai' OR (city='Bangalore' AND amount>10000)` ." 3. "I need explicit parentheses around the OR group."

The diagnosis: SQL's operator precedence puts AND above OR. Without parentheses, OR's branches don't share predicates the way the dev intended.

The fix:

```
SELECT order_id, city, amount
FROM orders
WHERE (city = 'Mumbai' OR city = 'Bangalore')
      AND amount > 10000;
```

Or, cleaner:

```
SELECT order_id, city, amount
FROM orders
WHERE city IN ('Mumbai', 'Bangalore')
      AND amount > 10000;
```

Why this works: Parentheses force the OR to be evaluated first; IN makes the intent obvious and avoids the trap entirely.

What NOT to say: - "SQL evaluates left-to-right" — no, it respects precedence. - "Always wrap everything in parentheses" — fine instinct, but clean grouping is better.

Common follow-up: "Rewrite this with a CASE to bucket the cities and amounts."

Scenario26 · String Concatenation With NULL

Tags: Difficulty: Easy · Asked at: any · Dialect: Postgres/MySQL/SQL Server differ

The scenario:

A "full name" column for a CRM export is mysteriously empty for users with no middle name. The dev used `||` (or `+` in SQL Server) and didn't handle NULL.

```
-- Postgres
SELECT user_id, first_name || ' ' || middle_name || ' ' || last_name AS full_name
FROM users;
```

Expected: 'Ravi Kumar Sharma', or 'Ravi Sharma' if no middle name. **Actual:** NULL for any user with a missing middle name (or first or last).

Thinking out loud: 1. "`||` propagates NULL — anything concatenated with NULL is NULL in standard SQL." 2. "Need `COALESCE` per field, or use `CONCAT` (which treats NULL as empty in Postgres and MySQL), or `CONCAT_WS` for cleaner separators."

The diagnosis: The `||` and `+` concatenation operators (and `CONCAT` in some dialects) treat NULL as a poison value — any NULL operand makes the whole expression NULL. `CONCAT` in Postgres and MySQL treats NULL as empty string. `CONCAT_WS` is even cleaner because it skips NULLs entirely.

The fix:

```
-- Postgres / MySQL
SELECT user_id,
       CONCAT_WS(' ', first_name, middle_name, last_name) AS full_name
FROM users;

-- SQL Server (uses + and ISNULL)
SELECT user_id,
       CONCAT_WS(' ', first_name, middle_name, last_name) AS full_name
FROM users;
```

Why this works: `CONCAT_WS` ignores NULLs and uses the separator only between actual values, producing clean names regardless of which middle fields are missing.

What NOT to say: - "NULL is empty string, my fault for forgetting" — NULL is *not* empty string; that's the bug. - "Wrap each field in `COALESCE(field, '')` and use `||`" — works, but you'll get double-spaces when middle_name is missing.

Common follow-up: "What if last_name is also nullable and you want it formatted as 'Last, First'?"

Scenario27 · DELETE / UPDATE Without WHERE

Tags: Difficulty: Easy (to spot), Hard (to recover from) · Asked at: every senior screen · Dialect: vendor-agnostic

The scenario:

A migration script intended to mark inactive users ran without a WHERE clause, and every user in the table now has `is_active = false`. The interviewer wants to know (a) how this could happen, (b) how to prevent it, and (c) what's the recovery story.

```
-- The catastrophic version
UPDATE users SET is_active = false;

-- The intended version
UPDATE users SET is_active = false WHERE last_login_at < CURRENT_DATE - INTERVAL '180 days';
```

Expected: Inactive users flagged. **Actual:** Every user is now flagged inactive.

Thinking out loud: 1. "UPDATE without WHERE updates every row. DELETE without WHERE deletes every row." 2. "The query is syntactically valid; the engine has no way to know intent." 3. "Recovery depends on backups, point-in-time recovery, or transaction logs — and prevention is far cheaper than recovery."

The diagnosis: SQL statements update or delete the rows that match the WHERE clause. Omit WHERE and the predicate is implicitly true, which matches every row. This is the most common cause of "I just nuked prod" stories.

The fix (prevention, not retroactive): 1. Always write the SELECT first, verify the row set, then convert to UPDATE/DELETE. 2. Wrap DDL/DML in an explicit transaction so you can ROLLBACK before COMMIT. 3. Configure the client to require WHERE clauses (MySQL Workbench's `safe_update` mode; psql custom scripts; ORM guardrails). 4. Use `LIMIT` with UPDATE/DELETE (MySQL/Postgres extension) as a belt-and-braces during ad-hoc fixes. 5. Reviewable migrations — never run ad-hoc DML in prod consoles.

```
BEGIN;
SELECT count(*) FROM users WHERE last_login_at < CURRENT_DATE - INTERVAL '180 days';
-- verify the count matches expectation
UPDATE users SET is_active = false WHERE last_login_at < CURRENT_DATE - INTERVAL '180 days';
-- inspect a sample
SELECT user_id, is_active FROM users WHERE last_login_at < CURRENT_DATE - INTERVAL '180 days' LIMIT 10;
COMMIT; -- or ROLLBACK if anything looks wrong
```

Why this works: The transaction makes the operation reversible until COMMIT; the SELECT-first habit verifies the predicate before destructive action; the safe-update setting in clients adds an extra net.

What NOT to say: - "I'd just restore from backup" — recovery is the last resort; if the WAL isn't archived you'll lose data; *and* you'll lose every transaction between backup and disaster. - "It's the database's fault for allowing this" — every relational DB allows this by design.

Common follow-up: *"Walk me through your prod-database access policy at your last job — who had write access, what review existed, what guardrails were in place?"*

Category 1, Part B — Debug This Query: NULL & Type & Cardinality Bugs

Part B focuses on the trickiest bug class — NULL semantics, implicit type coercion, and join cardinality surprises. Where logic bugs are obvious once you see them (a wrong operator, a missing GROUP BY clause), these need closer reading. The query parses, runs, returns a result, and the result looks plausible. Then someone reconciles it against another system and the gap shows up. Each of the 27 scenarios below is a real shape that has shipped to production at an Indian product company — the kind round-2 interviewers ask precisely because they separate candidates who can read SQL from candidates who can debug it.

Scenario28 · The signup count that drops half the users

Asked at: Razorpay · Meesho · **Difficulty:** Easy · **Topic:** NULL bug · COUNT(col) vs COUNT(*)

The scenario: A growth analyst pings you on Slack — the daily signup dashboard shows 12,400 new users on Monday but the product team's own onboarding log shows 22,800. Half the users are vanishing. The query feeding the dashboard is this:

```
SELECT signup_date, COUNT(referral_code) AS signups
FROM users
WHERE signup_date = '2026-05-25'
GROUP BY signup_date;
```

Expected vs Actual: Expected 22,800 signups. Got 12,400.

Thinking out loud: 1. "COUNT with a column name only counts non-NULL values of that column — not rows. That is the standard behaviour across every dialect." 2. " `referral_code` is almost certainly nullable — most users sign up without a referral. So we are counting only referred users, not all signups." 3. "22,800 minus 12,400 is 10,400, which is exactly the kind of number you'd get if roughly 45% of users come in organic with NULL referral_code." 4. "The query author probably wrote

`COUNT(referral_code)` thinking it meant 'count this group' — a very common mistake when you started SQL with `COUNT(*)` and got told 'use a column name, it is more efficient'."

Diagnosis: `COUNT(expression)` skips NULL values of that expression. To count rows regardless of NULL, use `COUNT(*)`. To count distinct non-NULL referrals, use `COUNT(DISTINCT referral_code)`. The query as written is asking "how many signups had a non-NULL referral_code", not "how many signups".

Fix SQL:

```
SELECT signup_date, COUNT(*) AS signups
FROM users
WHERE signup_date = '2026-05-25'
GROUP BY signup_date;
```

Why this works: `COUNT(*)` counts rows — it does not look at any specific column, so NULL values are irrelevant. This is also the path most query planners optimise hardest because it can be answered from the index without touching the heap rows.

What NOT to say: - "COUNT(*) is slower than COUNT(column)" — false on every modern engine; often the opposite. - "Add ISNULL around referral_code" — that just maps NULL to a sentinel, still doesn't restore the semantics if the sentinel matches a real value.

Common follow-up: "When would `COUNT(column)` actually be the right call?" — when you genuinely want the count of non-NULL values, e.g. how many users have set a profile picture.

Scenario29 · The average order value that ignores the freebies

Asked at: Flipkart · Zepto · **Difficulty:** Easy · **Topic:** NULL bug · AVG excludes NULLs

The scenario: Pricing team is debating whether to retire the free-trial SKU. They asked for the average order value last month. Your query returns ₹847. The finance team's report says ₹612. Same time window, same orders table.

```
SELECT AVG(order_value) AS avg_order_value
FROM orders
WHERE order_date BETWEEN '2026-04-01' AND '2026-04-30';
```

Expected vs Actual: Expected ₹612 (matches finance). Got ₹847.

Thinking out loud: 1. "AVG ignores NULLs the same way COUNT does. So if free-trial orders have `order_value = NULL`, they drop out of the numerator AND the denominator." 2. "Finance is treating free orders as ₹0 in the average, which lowers the mean. Our query is averaging only paid orders." 3. "Both numbers are technically correct — but they answer different questions. The ambiguity sits in NULL vs 0."

4. "For 'business-level AOV' the free orders should be ₹0, not absent. We have to decide which interpretation the dashboard owner wants."

Diagnosis: `AVG(order_value)` computes `SUM(non-null values) / COUNT(non-null values)`. NULL rows are silently dropped from both. If business defines AOV as "total revenue divided by total orders including freebies", you have to coerce NULLs to 0.

Fix SQL (assuming finance's definition is the business truth):

```
SELECT AVG(COALESCE(order_value, 0)) AS avg_order_value
FROM orders
WHERE order_date BETWEEN '2026-04-01' AND '2026-04-30';
```

Why this works: COALESCE converts NULL to 0 before AVG sees the value. Now free orders count in the denominator and contribute 0 to the numerator — matching finance's calculation. Even cleaner: `SUM(order_value) / COUNT(*)`, which makes the intent explicit.

What NOT to say: - "AVG is broken" — it is doing exactly what the spec says. - "We should filter out free orders" — that changes the business question entirely.

Common follow-up: "What if order_value is sometimes NULL because the row is mid-checkout, not because it's a freebie?" — then you have a data quality problem, not an aggregate problem, and COALESCE will mask it.

Scenario30 · The SUM that returned NULL instead of zero

Asked at: Cred · PhonePe · **Difficulty:** Easy · **Topic:** NULL bug · SUM over all-NULL set

The scenario: A merchant onboarding flow shows "Lifetime refunds: NULL" on the dashboard instead of "₹0" for new merchants. The frontend now crashes because it tries to render NULL as a currency string.

```
SELECT merchant_id, SUM(refund_amount) AS lifetime_refunds
FROM refunds
WHERE merchant_id = 8821
GROUP BY merchant_id;
```

Expected vs Actual: Expected ₹0 for a merchant with no refunds. Got NULL — or worse, no row at all if the merchant has zero refunds.

Thinking out loud: 1. "SUM over an empty or all-NULL set returns NULL across every major dialect — not 0. That's the SQL standard." 2. "If the merchant has no rows in the refunds table at all, the GROUP BY produces zero rows. The frontend sees a missing row, not NULL." 3. "If the merchant has rows but every refund_amount is NULL, SUM returns NULL. Two different bug paths, same symptom." 4. "Right fix

has two parts: COALESCE the SUM, and make sure we return a row even when there are no refunds. That needs an outer join from the merchants table."

Diagnosis: Two NULL traps stacked. SUM of an empty/NULL set yields NULL; missing GROUP key produces no row at all. Frontend treats both as errors.

Fix SQL:

```
SELECT m.merchant_id,  
       COALESCE(SUM(r.refund_amount), 0) AS lifetime_refunds  
FROM merchants m  
LEFT JOIN refunds r ON r.merchant_id = m.merchant_id  
WHERE m.merchant_id = 8821  
GROUP BY m.merchant_id;
```

Why this works: LEFT JOIN guarantees a row for the merchant even if no refunds exist. COALESCE turns the NULL SUM into 0. Frontend gets a clean ₹0.

What NOT to say: - "SUM should return 0 by default" — argue with the standard, not the interviewer. - "Add IF EXISTS" — that's procedural thinking, not SQL.

Common follow-up: "How would you do this if your engine doesn't have COALESCE?" — ISNULL (SQL Server), NVL (Oracle), or IFNULL (MySQL). Same semantics, different name.

Scenario31 · The dedup that didn't dedup

Asked at: Swiggy · Myntra · **Difficulty:** Medium · **Topic:** NULL bug · NULL = NULL is NULL

The scenario: You're cleaning up a customer table that was loaded twice. You write a "find duplicates" query joining the table to itself on the natural key. It returns far fewer dupes than the count of suspicious rows. Manual inspection shows obvious duplicates the query missed.

```
SELECT a.customer_id, b.customer_id  
FROM customers a  
JOIN customers b  
  ON a.email = b.email  
  AND a.phone = b.phone  
  AND a.dob   = b.dob  
WHERE a.customer_id < b.customer_id;
```

Expected vs Actual: Expected ~14,000 duplicate pairs (count of suspicious rows). Got ~6,200.

Thinking out loud: 1. "Duplicates we are missing all share one trait — they have NULL in at least one of email, phone or dob." 2. "In SQL, NULL = NULL is not TRUE, it is NULL. A join predicate that evaluates to NULL is treated as FALSE for the purpose of including the row." 3. "So any duplicate where

both rows have NULL phone is silently dropped from the join." 4. "Fix is `IS NOT DISTINCT FROM` in Postgres, or a manual `(a.x = b.x OR (a.x IS NULL AND b.x IS NULL))` in dialects that lack it."

Diagnosis: SQL uses three-valued logic. `NULL = NULL` evaluates to UNKNOWN, which join predicates treat as "no match". Genuine duplicates that share NULL phone/dob/email never pair up.

Fix SQL (Postgres):

```
SELECT a.customer_id, b.customer_id
FROM customers a
JOIN customers b
  ON a.email IS NOT DISTINCT FROM b.email
 AND a.phone IS NOT DISTINCT FROM b.phone
 AND a.dob   IS NOT DISTINCT FROM b.dob
WHERE a.customer_id < b.customer_id;
```

Why this works: `IS NOT DISTINCT FROM` is NULL-safe equality. NULL matches NULL, and any value matches itself. Standard SQL since SQL:2003, supported in Postgres natively, and MySQL has the equivalent `<=>` operator.

What NOT to say: - "Just put NOT NULL on the columns" — fine for new data, doesn't fix the dirty rows you already have. - "Use COALESCE to map NULL to empty string" — works but breaks down when empty string is itself a valid value (it usually is for phone).

Common follow-up: "In MySQL, what's the operator?" — `<=>`, the NULL-safe equal. Same behaviour, ugly syntax.

Scenario32 · The "!=" filter that hides half the bugs

Asked at: Zerodha · Groww · **Difficulty:** Medium · **Topic:** NULL bug · `!=` excludes NULL rows

The scenario: Compliance team asks for "all orders where the order status is not CANCELLED" — they want to audit everything that wasn't cancelled. The query returns 41,000 rows. The orders table has 60,000 rows and only 7,000 are CANCELLED. They expected ~53,000.

```
SELECT order_id, customer_id, status
FROM orders
WHERE status != 'CANCELLED';
```

Expected vs Actual: Expected ~53,000 orders. Got 41,000. The missing 12,000 have `status = NULL` — orders that are mid-checkout and haven't been assigned a status yet.

Thinking out loud: 1. " `status != 'CANCELLED'` evaluates to NULL when status IS NULL. Three-valued logic again — NULL in a WHERE clause is treated as FALSE, so those rows drop out." 2. "Compliance asked for 'not cancelled', which in English includes 'no status yet'. SQL doesn't agree." 3. "Fix is to be explicit — `status IS DISTINCT FROM 'CANCELLED'` in Postgres, or `(status != 'CANCELLED' OR status IS NULL)` in plain SQL." 4. "This is one of those bugs you only catch when you reconcile against a row count. The query 'looks right' in isolation."

Diagnosis: Comparison with NULL using `=`, `!=`, `<`, `>` always yields NULL, which the WHERE clause treats as FALSE. Rows with NULL `status` are dropped silently.

Fix SQL:

```
SELECT order_id, customer_id, status
FROM orders
WHERE status IS DISTINCT FROM 'CANCELLED';
```

Or vendor-neutral:

```
WHERE status != 'CANCELLED' OR status IS NULL
```

Why this works: `IS DISTINCT FROM` is NULL-safe, so NULL is considered "distinct from" any non-NULL value. The vendor-neutral form is explicit about the NULL branch.

What NOT to say: - "The data is wrong, fix the data" — sometimes true, but the query has to be defensible regardless. - "NOT IN is safer than !=" — opposite is true; NOT IN with a NULL in the list returns nothing.

Common follow-up: "What's the bug with `WHERE status NOT IN ('CANCELLED', NULL)`?" — returns zero rows. NOT IN with a NULL is equivalent to FALSE for every row.

Scenario33 · The login check that won't recognise NULL passwords

Asked at: ICICI Digital · Paytm · **Difficulty:** Easy · **Topic:** NULL bug · `= NULL` never matches

The scenario: A migration script needs to find legacy accounts that have no password set (they signed up via OTP only). The query returns zero rows even though the DBA confirmed there are ~80,000 such accounts.

```
SELECT user_id, signup_method
FROM users
WHERE password_hash = NULL;
```

Expected vs Actual: Expected ~80,000 rows. Got 0.

Thinking out loud: 1. " `col = NULL` evaluates to `NULL`, never `TRUE`. `WHERE` drops every row." 2. "The right check is `password_hash IS NULL`. Different operator entirely." 3. "Worse — some dialects with `ANSI_NULLS OFF` (old SQL Server compat mode) will let `= NULL` work as `IS NULL`. So this exact query might 'work' on one environment and silently fail on another." 4. "Always use `IS NULL` / `IS NOT NULL`. Never compare to `NULL` with `=`."

Diagnosis: `=` is the equality operator; `NULL` is not a value, it's a marker for "unknown". Asking "is unknown equal to unknown" returns unknown, not true.

Fix SQL:

```
SELECT user_id, signup_method
FROM users
WHERE password_hash IS NULL;
```

Why this works: `IS NULL` is the explicit `NULL`-test operator. It returns `TRUE` only when the column is `NULL`, `FALSE` otherwise — never `UNKNOWN`.

What NOT to say: - "It worked on my old SQL Server" — yes, with `ANSI_NULLS OFF`, which is deprecated and unsafe. - "Use `COALESCE(password_hash, 'NONE') = 'NONE'`" — works but is allergic to sentinel collisions and harder to index.

Common follow-up: "Can you index `WHERE password_hash IS NULL` efficiently?" — yes, with a partial index in Postgres: `CREATE INDEX ON users (user_id) WHERE password_hash IS NULL;` .

Scenario34 · The foreign key that doesn't enforce anything

Asked at: HDFC · Razorpay · **Difficulty:** Medium · **Topic:** `NULL` bug · `NULL` in FK column

The scenario: A new junior dev claims "our orders.customer_id has a foreign key to customers, so every order belongs to a customer". They run a query joining orders to customers, expecting to see every order. Some orders are missing.

```
SELECT o.order_id, c.customer_name
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.order_date = '2026-05-20';
```

Expected vs Actual: Expected every order from 2026-05-20 (1,820 rows). Got 1,790 rows. The missing 30 are "guest checkout" orders where `customer_id` is `NULL`.

Thinking out loud: 1. "Foreign key constraints in SQL allow `NULL` by default — the FK only enforces 'if not null, must match'. Empty cells are fine." 2. "Inner join on a `NULL` customer_id never matches anything

in customers — guests drop out." 3. "Fix is a *LEFT JOIN* if guest orders are expected, or a *NOT NULL* constraint on *customer_id* if guests should not exist." 4. "The 'every order has a customer' belief is a constraint claim, not a query result. The constraint as written is `(customer_id IS NULL) OR (customer_id IN customers)`, which is weaker."

Diagnosis: A FOREIGN KEY constraint does not imply NOT NULL. Rows with NULL in the FK column satisfy the constraint trivially. Inner joins drop them.

Fix SQL (if guests are legitimate):

```
SELECT o.order_id, COALESCE(c.customer_name, 'Guest') AS customer_name
FROM orders o
LEFT JOIN customers c ON o.customer_id = c.customer_id
WHERE o.order_date = '2026-05-20';
```

If guests should never exist:

```
ALTER TABLE orders ALTER COLUMN customer_id SET NOT NULL;
```

Why this works: LEFT JOIN preserves orders with NULL `customer_id`; COALESCE labels them. The schema fix prevents the bug at source.

What NOT to say: - "Foreign keys mean the value always exists" — only if NOT NULL is also set. - "Just add a default 0 for guest" — creates a fake row in customers, breaks reporting.

Common follow-up: "What if you put NOT NULL on *customer_id* and a row is already NULL?" — `ALTER` fails. Have to backfill first.

Scenario35 · The unique index that allowed three NULLs

Asked at: Salesforce · Microsoft India · **Difficulty:** Medium · **Topic:** NULL bug · UNIQUE allows multiple NULLs (dialect-specific)

The scenario: A migration team is moving customer data from SQL Server to Postgres. The Postgres side has a UNIQUE index on `email`. They successfully load a dataset that has 12,000 rows with NULL email. The same load on SQL Server fails with a unique constraint violation on the second NULL.

```
CREATE TABLE customers (  
    customer_id BIGINT PRIMARY KEY,  
    email VARCHAR(255) UNIQUE  
);  
-- Insert 12,000 rows with email = NULL.  
-- Works fine on Postgres + MySQL + Oracle.  
-- Fails on SQL Server after the second NULL.
```

Expected vs Actual: Engineer expected UNIQUE behaviour to be consistent across engines. It is not.

Thinking out loud: 1. "This is one of the most famous dialect divergences. Most engines treat NULL as 'unknown', so two NULLs are not 'equal' and don't violate uniqueness." 2. "SQL Server is the odd one out — its default unique index treats NULL as a value and allows only one NULL row. Filtered indexes can change this." 3. "Postgres 15+ added the `NULLS NOT DISTINCT` clause to flip the behaviour the other way. So you can configure it either side." 4. "For migration scripts, the safe move is to make `email NOT NULL` with a sentinel — or to be explicit about the desired NULL semantics."

Diagnosis: SQL standard says NULLs are not equal, so a UNIQUE index should permit multiple NULLs. Postgres / MySQL / Oracle / SQLite follow the standard. SQL Server historically deviates — only one NULL allowed per unique index.

Fix SQL (Postgres-style — explicit):

```
-- Standard behaviour (multiple NULLs allowed)  
CREATE UNIQUE INDEX customers_email_uniq ON customers (email);  
  
-- Strict behaviour (only one NULL row)  
CREATE UNIQUE INDEX customers_email_uniq ON customers (email) NULLS NOT DISTINCT;
```

For SQL Server, a filtered index gets the standard behaviour:

```
CREATE UNIQUE INDEX customers_email_uniq  
ON customers (email)  
WHERE email IS NOT NULL;
```

Why this works: Explicit NULL handling at index creation time makes the semantics match across engines.

What NOT to say: - "Postgres is broken" — it follows the standard; SQL Server doesn't. - "Just don't allow NULL emails" — sometimes that's the right call, but you have to know it's a choice.

Common follow-up: "What's the right design — allow NULL emails or force a sentinel?" — usually allow NULL, because email is genuinely optional for many users.

Scenario36 · The dedup join that needed IS DISTINCT FROM

Asked at: Atlassian · Postman · **Difficulty:** Medium · **Topic:** NULL bug · NULL-safe equality fix

The scenario: A data engineer is auditing a CDC pipeline that compares "current snapshot" against "previous snapshot" to detect changes. The pipeline reports zero changes for some rows, but manual inspection shows the rows were updated — the change was specifically that a nullable column went from NULL to 'something' or vice versa.

```
SELECT cur.id
FROM current_snapshot cur
JOIN previous_snapshot prev ON cur.id = prev.id
WHERE cur.middle_name = prev.middle_name
      AND cur.address_line2 = prev.address_line2;
-- Pipeline says: "These rows are unchanged."
-- Reality: middle_name was NULL in prev, now 'Kumar' in cur.
```

Expected vs Actual: Expected to detect "no change" when columns truly match. Actually flagging "no change" any time a NULL flip happens, because `NULL = anything` is NULL (treated as false), and the row falls out of the "unchanged" set — but the bigger sibling query that flags "changed" rows uses the same `=` operator and also misses them. Both halves of the diff lose these rows.

Thinking out loud: 1. "Plain equality on nullable columns is broken in both directions — `=` says 'not equal' when one side is NULL, even if the other side is also NULL." 2. "The right operator for CDC diffs is `IS DISTINCT FROM` / `IS NOT DISTINCT FROM`. It treats `NULL = NULL` as 'same' and `NULL = value` as 'different'." 3. "Without it, you have to write `(a = b OR (a IS NULL AND b IS NULL))` for every nullable column — verbose and easy to forget." 4. "This pattern shows up in any data-quality / replication / diff-detection scenario where nullable columns flip."

Diagnosis: `=` is not NULL-safe. CDC pipelines that diff snapshots with nullable columns must use `IS DISTINCT FROM` / `IS NOT DISTINCT FROM` to avoid silent miscounts.

Fix SQL:

```
-- Rows that are genuinely unchanged across all monitored columns:
SELECT cur.id
FROM current_snapshot cur
JOIN previous_snapshot prev ON cur.id = prev.id
WHERE cur.middle_name IS NOT DISTINCT FROM prev.middle_name
      AND cur.address_line2 IS NOT DISTINCT FROM prev.address_line2;
```

Why this works: `IS NOT DISTINCT FROM` is NULL-safe. `(NULL, NULL) → same`. `(NULL, 'Kumar') → distinct`. `('A', 'A') → same`.

What NOT to say: - "Just COALESCE everything to empty string" — collisions with real empty strings. - "It's an edge case" — for nullable columns, NULL flips are the common case in CDC.

Common follow-up: "What's the equivalent operator in MySQL?" — `<=>` for NULL-safe equal; for "distinct from" you negate: `NOT (a <=> b)`.

Scenario37 · The COALESCE that doesn't exist on this engine

Asked at: Oracle India · TCS BFSI · **Difficulty:** Easy · **Topic:** Dialect mismatch · COALESCE vs ISNULL vs NVL

The scenario: A team is porting a Postgres reporting query into an Oracle environment. The query parses fine on Postgres but throws `ORA-00904: invalid identifier: ISNULL` on Oracle. The dev tries `IFNULL` and gets a different error. They start guessing.

```
-- Original (Postgres-leaning):  
SELECT customer_id, ISNULL(loyalty_points, 0) AS points  
FROM customers;
```

Expected vs Actual: Expected the same NULL-default function name everywhere. It does not exist.

Thinking out loud: 1. "NULL-default functions are one of the worst dialect tangles in SQL. Every engine ships its own name." 2. "COALESCE is the standard, supported across Postgres, MySQL, Oracle, SQL Server, BigQuery, Snowflake. Always pick COALESCE for portability." 3. "ISNULL is SQL Server. IFNULL is MySQL. NVL is Oracle. NVL2 is also Oracle (3-arg form). All do roughly the same thing." 4. "Bonus trap: SQL Server's ISNULL takes 2 args; Postgres has no ISNULL function (it has the IS NULL predicate, different thing)."

Diagnosis: Vendor-specific names for "return first non-NULL argument" don't port. COALESCE is the standard.

Fix SQL (portable):

```
SELECT customer_id, COALESCE(loyalty_points, 0) AS points  
FROM customers;
```

Why this works: COALESCE is in the SQL standard and supported by every major engine — Postgres, MySQL, Oracle, SQL Server, SQLite, BigQuery, Snowflake, Redshift.

What NOT to say: - "ISNULL works everywhere" — only on SQL Server and as a different operator in Postgres (`expr IS NULL` returns boolean). - "Use a CASE statement, more portable" — actually less so, since CASE is portable but so is COALESCE, and CASE is verbose.

Common follow-up: "What's the difference between COALESCE and NVL in Oracle?" — NVL is 2-arg only and forces type coercion of both args; COALESCE short-circuits and is N-arg.

Scenario38 · The NULLIF that erased the wrong rows

Asked at: Walmart Labs · Target India · **Difficulty:** Medium · **Topic:** NULL bug · NULLIF direction

The scenario: A report wants to treat `discount_pct = 0` rows as "no discount" (NULL) so they're excluded from AVG. The query author writes:

```
SELECT product_id, AVG(NULLIF(discount_pct, 0)) AS avg_real_discount
FROM line_items
GROUP BY product_id;
```

A reviewer says "this looks backwards — NULLIF returns NULL when the column IS NULL, right?" Confusion follows.

Expected vs Actual: Both devs are unsure which direction NULLIF goes. The query is actually correct, but the discussion reveals a common mental model bug. The reviewer wants to use NULLIF to replace NULLs with a sentinel — which is the wrong function entirely.

Thinking out loud: 1. "NULLIF(a, b) returns NULL when a equals b; otherwise returns a. It is the OPPOSITE of what people often expect from the name." 2. "NULLIF(discount_pct, 0) means 'treat 0 as NULL'. That is what the report wants — exclude zero-discount rows from AVG." 3. "What the reviewer is thinking of is COALESCE, which fills in NULLs. NULLIF creates NULLs." 4. "The mnemonic: NULLIF(a, b) = `NULL if a = b`. Read it as a conditional, not a fallback."

Diagnosis: The query is correct; the reviewer's mental model of NULLIF is wrong. NULLIF creates NULL when two values match; COALESCE replaces NULL with a default. They are inverses.

Fix SQL (it is already correct, but spelled out):

```
-- "Treat zero as missing" — for AVG that should ignore zero discounts:
SELECT product_id, AVG(NULLIF(discount_pct, 0)) AS avg_real_discount
FROM line_items
GROUP BY product_id;

-- "Fill missing with zero" — for SUM that needs zero-default:
SELECT product_id, SUM(COALESCE(discount_pct, 0)) AS total_discount
FROM line_items
GROUP BY product_id;
```

Why this works: NULLIF and COALESCE are complementary. NULLIF is for masking a known sentinel; COALESCE is for filling in missing values.

What NOT to say: - "NULLIF is for replacing NULLs" — exactly backwards. - "We need both NULLIF and IFNULL on every column" — not without a reason; default to COALESCE.

Common follow-up: "What does NULLIF(NULL, NULL) return?" — NULL. Because NULL is never equal to NULL, the comparison fails, and NULLIF returns the first arg, which is NULL.

Scenario39 · The CASE that drops UNKNOWN cases

Asked at: Adobe India · Intuit · **Difficulty:** Medium · **Topic:** NULL bug · Three-valued logic in CASE

The scenario: A loyalty programme classifies customers into Gold / Silver / Bronze based on `lifetime_value`. The categorisation query returns three buckets but a separate count shows 4,200 customers that don't appear in any bucket.

```
SELECT customer_id,
       CASE
         WHEN lifetime_value >= 100000 THEN 'Gold'
         WHEN lifetime_value >= 25000  THEN 'Silver'
         WHEN lifetime_value >= 0      THEN 'Bronze'
       END AS tier
FROM customers;
```

Expected vs Actual: Expected every customer to land in one tier. 4,200 rows return NULL tier — they have `lifetime_value IS NULL` (never made a purchase).

Thinking out loud: 1. "CASE evaluates conditions top-down. The first one that's TRUE wins." 2. "If `lifetime_value IS NULL`, then `NULL >= 100000` returns NULL, not TRUE. Same for the other two branches." 3. "With no TRUE branch and no ELSE, CASE returns NULL. So nulls leak through." 4. "Fix is an ELSE clause to catch them — usually 'Unranked' or 'New'."

Diagnosis: Three-valued logic. NULL >= anything is NULL, not TRUE. CASE with no matching WHEN and no ELSE returns NULL.

Fix SQL:

```
SELECT customer_id,
       CASE
         WHEN lifetime_value IS NULL THEN 'New'
         WHEN lifetime_value >= 100000 THEN 'Gold'
         WHEN lifetime_value >= 25000  THEN 'Silver'
         WHEN lifetime_value >= 0      THEN 'Bronze'
         ELSE 'Other'
       END AS tier
FROM customers;
```

Why this works: Explicit NULL check first; explicit ELSE catches negative or unexpected values. Every row gets a non-NULL label.

What NOT to say: - "Add WHERE lifetime_value IS NOT NULL" — drops the customers entirely, which is worse for the report. - "CASE is non-deterministic with NULL" — it is deterministic; it's just that NULL falls through.

Common follow-up: "Why does *ELSE* matter even if you think you've covered every case?" — defensive coding. ELSE catches the bugs you didn't anticipate (negatives, NaN in some engines, unexpected NULL).

Scenario40 · The ORDER BY that puts NULLs in the wrong place

Asked at: Snowflake India · Databricks · **Difficulty:** Easy · **Topic:** NULL bug · NULLS FIRST/LAST default differs

The scenario: A dashboard built on Postgres got ported to Oracle. The "latest signups" list shows new users sorted by signup date desc. On MySQL it showed real users at the top. On Oracle the top 50 are all "NULL signup date" placeholder rows.

```
SELECT customer_id, signup_date
FROM customers
ORDER BY signup_date DESC
LIMIT 50;
```

Expected vs Actual: Same query, two engines, different sort. Postgres puts NULLs last by default with DESC; Oracle puts NULLs first by default with DESC.

Thinking out loud: 1. "ORDER BY's NULL handling default varies by engine. Postgres treats NULLs as 'larger than anything', so DESC sorts them first. Oracle does the same. MySQL treats NULLs as 'smaller'." 2. "Wait — Postgres puts NULLs first with DESC, last with ASC. MySQL is the opposite. SQL Server is yet another flavour." 3. "Always be explicit with NULLS FIRST or NULLS LAST when sorting nullable columns. Don't trust the default." 4. "For 'latest signups', we want real dates first and NULLs last — so NULLS LAST."

Diagnosis: NULL ordering default is dialect-specific. The portable fix is to specify **NULLS FIRST** or **NULLS LAST** explicitly.

Fix SQL:

```
SELECT customer_id, signup_date
FROM customers
ORDER BY signup_date DESC NULLS LAST
LIMIT 50;
```

Why this works: **NULLS LAST** forces NULL rows to the bottom regardless of dialect default. Supported by Postgres, Oracle, SQLite. MySQL doesn't support the keyword but **ORDER BY signup_date IS NULL, signup_date DESC** achieves the same result.

What NOT to say: - "NULLS LAST works everywhere" — MySQL needs a different syntax. - "Just exclude NULL rows" — changes the semantic; the dashboard might still want to count them.

Common follow-up: "What's the MySQL workaround?" — `ORDER BY signup_date IS NULL ASC, signup_date DESC`. The first key sorts NULLs to the end (since NULL IS NULL = 1, non-NULL IS NULL = 0).

Scenario41 · The integer join key that became a string

Asked at: ThoughtWorks · Infosys · **Difficulty:** Medium · **Topic:** Type bug · Implicit cast int → varchar disables index

The scenario: A reporting query that used to run in 80ms now takes 14 seconds after a schema migration. The migration moved `customer_id` from INT to VARCHAR in the new `customer_events` table (to support multi-tenant IDs). The join in the report still works but is dog slow.

```
SELECT c.customer_name, e.event_type, e.event_time
FROM customers c
JOIN customer_events e ON c.customer_id = e.customer_id
WHERE c.region = 'IN';
-- customers.customer_id is INT
-- customer_events.customer_id is now VARCHAR(64)
```

Expected vs Actual: Expected the existing index on `customer_events.customer_id` (varchar) to handle the join. Actual plan shows a full scan of `customer_events` with implicit cast on every row.

Thinking out loud: 1. "When the join column types differ, SQL coerces one side to the other. MySQL and SQL Server typically coerce silently (column side cast to varchar or int — which kills the index). Postgres actually errors out (`operator does not exist: integer = character varying`) unless an explicit cast is provided." 2. "That cast happens per row of the table being scanned. If the cast is on the indexed side, the index becomes unusable for the join." 3. "Postgres explain plan will literally show `(customer_id)::integer = c.customer_id` — that `::integer` wraps the indexed column." 4. "Fix is to cast on the unindexed side, or align the column types at the schema level."

Diagnosis: Implicit type coercion in join predicates often disables index usage because the indexed expression is wrapped in a cast that the planner can't match to the raw index.

Fix SQL (short-term):

```
SELECT c.customer_name, e.event_type, e.event_time
FROM customers c
JOIN customer_events e ON CAST(c.customer_id AS VARCHAR(64)) = e.customer_id
WHERE c.region = 'IN';
```

Long-term fix is to align column types — either both INT or both VARCHAR — and rebuild the index.

Why this works: Casting the smaller, unindexed side (`customers.customer_id`) preserves the index on the larger side (`customer_events.customer_id`).

What NOT to say: - "Implicit casts are free, the planner handles it" — they are not free; they force per-row evaluation and disable index seeks. - "Add a function-based index" — sometimes works but is a workaround for a schema bug.

Common follow-up: "How would you detect this without staring at the plan?" — look for **Seq Scan** (Postgres) or **Full Table Scan** (Oracle/MySQL) on a column that has an index.

Scenario42 · The varchar filter that triggers a full scan

Asked at: Goldman Sachs · Morgan Stanley · **Difficulty:** Medium · **Topic:** Type bug · `varchar` → `int` implicit cast

The scenario: A user-facing report filters on `account_number = 12345` where `account_number` is `VARCHAR(20)`. The query is slow despite a B-tree index on `account_number`. Engineer claims "the index isn't being used".

```
SELECT *
FROM accounts
WHERE account_number = 12345;
-- account_number is VARCHAR(20), indexed
```

Expected vs Actual: Expected index seek (sub-ms). Got full table scan (8s).

Thinking out loud: 1. "`account_number` is varchar but the literal is an int. SQL has to coerce one side. Standard rule: string is promoted to int." 2. "That means every row gets `account_number::int = 12345`, which kills the index." 3. "Also dangerous — any row where `account_number` can't parse as int (e.g. 'A12345') will throw at runtime in some engines, or silently drop in others." 4. "Fix is to quote the literal: `account_number = '12345'`. Index seek restored."

Diagnosis: Implicit `varchar` → `int` coercion. Engine wraps the indexed column in a cast, disabling the index. Also a correctness risk if any rows fail to parse.

Fix SQL:

```
SELECT *
FROM accounts
WHERE account_number = '12345';
```

Why this works: String literal matches string column directly. No cast, index is usable, sub-ms seek.

What NOT to say: - "The engine should know what we mean" — engines do exactly what the spec says.
- "Wrap with TO_NUMBER" — opposite of what you want; that forces the cast.

Common follow-up: *"Why does the cast go varchar → int and not int → varchar?"* — SQL's implicit conversion rules pick the higher-precedence type, and numeric outranks character. You get a slow scan; the alternative would be wrong results.

Scenario43 · The SUM that overflowed an INT

Asked at: Tata 1mg · Pharmeasy · **Difficulty:** Medium · **Topic:** Type bug · Numeric overflow on aggregate

The scenario: A revenue report that ran cleanly for two years started returning negative numbers in January 2026 after a strong holiday quarter. The dev checks the query — it's a plain SUM over a transactions table. There are no negative transactions.

```
SELECT SUM(amount_paise) AS total_revenue_paise
FROM transactions
WHERE quarter = '2025-Q4';
-- amount_paise is INT (4 bytes, signed)
-- Q4 total exceeded 2^31 - 1 = 2.147 billion paise = ₹21.47 crore
```

Expected vs Actual: Expected ~₹35 crore total revenue. Got a negative number — classic 32-bit signed integer overflow.

Thinking out loud: 1. *"INT in most engines is 4 bytes, signed — range about ± 2.1 billion. Summing many positive INTs can exceed that range."* 2. *"In paise, ₹21.47 crore is the overflow point. We just crossed it."* 3. *"Postgres throws an integer overflow exception. MySQL silently wraps to negative. SQL Server's behaviour depends on ARITHABORT setting."* 4. *"Fix: cast to BIGINT before SUM, or use SUM(amount_paise * 1.0) to promote to NUMERIC. BIGINT is cleaner."*

Diagnosis: SUM of INT stays INT in many engines (MySQL especially). Aggregate can overflow the INT range. Some engines warn / throw, some silently wrap.

Fix SQL:

```
SELECT SUM(CAST(amount_paise AS BIGINT)) AS total_revenue_paise
FROM transactions
WHERE quarter = '2025-Q4';
```

Why this works: Each value is promoted to 8-byte signed integer (range $\pm 9.2 \times 10^{18}$) before the running total accumulates. No realistic financial sum reaches that range.

What NOT to say: - "SUM auto-promotes to BIGINT" — true on Postgres for SUM(int) → bigint, but not universally; MySQL keeps the type. - "We should never store paise as INT" — long-term yes, but the bug is in the query right now.

Common follow-up: "How would you store currency safely going forward?" — DECIMAL(19,4) or BIGINT for paise, never FLOAT. And cast on aggregate to be defensive.

Scenario44 · The division that lost the decimal

Asked at: Zoho · Freshworks · **Difficulty:** Easy · **Topic:** Type bug · Integer division precision loss

The scenario: A success-rate report computes `successful_orders / total_orders`. The result is always 0 or 1, never a fraction. Dev expected percentages like 0.87 or 0.92.

```
SELECT product_id,  
       SUM(CASE WHEN status = 'SUCCESS' THEN 1 ELSE 0 END) / COUNT(*) AS success_rate  
FROM orders  
GROUP BY product_id;
```

Expected vs Actual: Expected 0.0 to 1.0 floats. Got 0 or 1 (integer truncation).

Thinking out loud: 1. "Both numerator and denominator are integers. Integer / integer in most engines returns integer with truncation." 2. "In Postgres, MySQL, SQL Server — integer division drops the decimal. Only when one side is float/numeric does the result include decimals." 3. "Fix: multiply numerator by 1.0, or cast one side to NUMERIC." 4. "A common trap because the SQL parses fine and returns plausible-looking values. Test on a known product first."

Diagnosis: Integer division. To get a fractional result, one operand must be a non-integer type.

Fix SQL:

```
SELECT product_id,  
       SUM(CASE WHEN status = 'SUCCESS' THEN 1 ELSE 0 END) * 1.0 / COUNT(*) AS success_rate  
FROM orders  
GROUP BY product_id;
```

Or explicit cast:

```
CAST(SUM(...) AS NUMERIC) / COUNT(*) AS success_rate
```

Why this works: Multiplying by 1.0 promotes the numerator to NUMERIC; the division then returns NUMERIC with the decimals intact.

What NOT to say: - "Just round to 2 decimals" — there are no decimals to round; the truncation already happened. - "Use AVG of a CASE expression" — works but obscures the intent; the multiplication-by-1.0 trick is the canonical fix.

Common follow-up: "What's the difference between NUMERIC, DECIMAL and FLOAT for this?" — NUMERIC/DECIMAL are exact; FLOAT is binary approximation and inappropriate for money or rates that get audited.

Scenario45 · The DATE column that won't match the TIMESTAMP

Asked at: Airbnb India · Booking.com · **Difficulty:** Medium · **Topic:** Type bug · DATE vs TIMESTAMP mismatch

The scenario: A check-in report joins `bookings.check_in_date` (DATE) to `events.event_ts` (TIMESTAMP) on the assumption that they represent the same day. It returns zero matched rows even though the data clearly aligns.

```
SELECT b.booking_id, e.event_type
FROM bookings b
JOIN events e ON b.check_in_date = e.event_ts;
-- check_in_date is DATE: 2026-05-25
-- event_ts is TIMESTAMP: 2026-05-25 14:32:11
```

Expected vs Actual: Expected matches on same calendar day. Got zero rows.

Thinking out loud: 1. "DATE has no time component, TIMESTAMP does. When SQL compares them, DATE gets promoted to TIMESTAMP at 00:00:00." 2. "So `2026-05-25` becomes `2026-05-25 00:00:00`. An event at `2026-05-25 14:32:11` is not equal to that. No match." 3. "Fix is to truncate the TIMESTAMP side to DATE before comparing — `DATE(e.event_ts)` in MySQL/Postgres, `CAST(e.event_ts AS DATE)` portable." 4. "Beware of timezone — `DATE(event_ts)` uses the session timezone, which may not be IST."

Diagnosis: Comparing DATE and TIMESTAMP promotes DATE to midnight TIMESTAMP. Real-time events never match midnight exactly.

Fix SQL:

```
SELECT b.booking_id, e.event_type
FROM bookings b
JOIN events e ON b.check_in_date = CAST(e.event_ts AS DATE);
```

Why this works: Both sides are now DATE. Calendar-day matching works. Trade-off: the cast on `event_ts` will disable any index on that column unless you have a functional index on `CAST(event_ts AS DATE)`.

What NOT to say: - "DATE and TIMESTAMP are interchangeable" — they are not; only the date part overlaps. - "Concatenate the date with ' 00:00:00 ' " — same wrong result, more typing.

Common follow-up: "How would you index this join efficiently?" — `CREATE INDEX ON events ((CAST(event_ts AS DATE)))`; — a functional index on the cast expression.

Scenario46 · The timezone bug that shifted Diwali sales to Thursday

Asked at: Nykaa · Tata CLiQ · **Difficulty:** Hard · **Topic:** Type bug · Timezone arithmetic

The scenario: A festival-sales report shows the Diwali campaign starting on Thursday instead of Wednesday. Marketing insists they pushed it live on Wednesday at 8pm IST. The SQL uses naive `TIMESTAMP` (no timezone).

```
SELECT campaign_id, COUNT(*) AS clicks
FROM clicks
WHERE click_ts >= '2026-11-04 00:00:00'
      AND click_ts < '2026-11-05 00:00:00'
      AND campaign_id = 'DIWALI-2026';
-- Server is UTC. Click events are stored in UTC TIMESTAMPS.
-- Marketing pushed at 2026-11-04 20:00:00 IST = 14:30:00 UTC.
-- The first hour of clicks (until midnight IST) bleeds into the next UTC day.
```

Expected vs Actual: Expected Wednesday Nov 4 to show 8pm onwards. Wednesday looks empty; clicks appear on Thursday.

Thinking out loud: 1. "Click timestamps are stored UTC. The filter uses literal '2026-11-04 00:00:00' which the engine interprets as UTC (server tz). So the bucket is Nov 4 UTC = Nov 4 5:30am IST onwards. Marketing started at 8pm IST = 14:30 UTC, which is in the Nov 4 bucket — but ends at Nov 5 18:29:59 IST." 2. "The 'Wednesday in India' window the marketing team means is `2026-11-04 00:00:00 IST` to `2026-11-05 00:00:00 IST` = `2026-11-03 18:30:00 UTC` to `2026-11-04 18:30:00 UTC`. The query filters a different window." 3. "Fix is to convert IST literals to UTC explicitly, or to use `TIMESTAMPTZ` everywhere and let the engine handle the conversion." 4. "For India this is a 5h30 offset bug — half a day's data lands in the wrong bucket if your filter window doesn't account for it."

Diagnosis: Naive timestamps + UTC-stored data + IST-meaning filter literals. The arithmetic looks right but applies to the wrong day boundaries.

Fix SQL (Postgres with `TIMESTAMPTZ`):

```
SELECT campaign_id, COUNT(*) AS clicks
FROM clicks
WHERE click_ts >= TIMESTAMP '2026-11-04 00:00:00' AT TIME ZONE 'Asia/Kolkata'
      AND click_ts < TIMESTAMP '2026-11-05 00:00:00' AT TIME ZONE 'Asia/Kolkata'
      AND campaign_id = 'DIWALI-2026';
```

Why this works: Filter literals are explicitly anchored in IST and converted to UTC for comparison against the UTC-stored column. Day boundaries align with India's calendar day.

What NOT to say: - "Just add 5.5 hours to the literal" — works once, breaks for DST transitions in other countries, and breaks if the server timezone changes. - "Store in IST" — creates the opposite problem for users in other timezones; UTC is the right storage choice.

Common follow-up: "Why use *TIMESTAMPTZ* over *TIMESTAMP* for new tables?" — *TIMESTAMPTZ* stores a UTC instant and lets each session render in its own timezone; *TIMESTAMP* is naive and prone to exactly this class of bug.

Scenario47 · The CHAR column that won't match the VARCHAR

Asked at: Bank of America · IBM India · **Difficulty:** Medium · **Topic:** Type bug · CHAR padding

The scenario: An old core-banking table uses `CHAR(10)` for IFSC codes. A newer microservice uses `VARCHAR(20)`. A join between the two returns surprisingly few matches in some dialects, even though the IFSC values are correct in both.

```
SELECT a.account_no, b.branch_name
FROM accounts a
JOIN branches b ON a.ifsc_code = b.ifsc_code;
-- accounts.ifsc_code is CHAR(10), padded with spaces: 'HDFC0001234'
-- branches.ifsc_code is VARCHAR(20), no padding:      'HDFC0001234'
```

Expected vs Actual: Some IFSC matches drop, depending on dialect. Behaviour is dialect-specific and dangerous.

Thinking out loud: 1. "CHAR(n) is fixed-width — values shorter than n are padded with trailing spaces. VARCHAR stores the exact length." 2. "In Postgres, comparison of CHAR(10) with VARCHAR strips the CHAR padding silently — usually you get a match." 3. "In MySQL, the default behaviour also pads. In SQL Server, it depends on the comparison context and ANSI_PADDING setting." 4. "The unsafe bet is to assume any dialect handles this transparently. Trim explicitly when joining CHAR with VARCHAR."

Diagnosis: CHAR padding semantics vary by engine. Some strip on comparison, some don't. Cross-engine portability requires explicit trimming.

Fix SQL:

```
SELECT a.account_no, b.branch_name
FROM accounts a
JOIN branches b ON RTRIM(a.ifsc_code) = b.ifsc_code;
```

Or convert the schema:

```
ALTER TABLE accounts ALTER COLUMN ifsc_code TYPE VARCHAR(11);
```

Why this works: `RTRIM` strips trailing spaces explicitly, making the join behave identically across engines. Schema-level fix removes the trap entirely.

What NOT to say: - "VARCHAR is always better than CHAR" — for fixed-length codes (IFSC, country codes), CHAR can be marginally more efficient if used consistently. - "Comparison ignores trailing spaces by SQL standard" — true for the LIKE operator but not always for `=` across engines.

Common follow-up: "What's the difference between `CHAR(10)` and `VARCHAR(10)` storage in Postgres?" — Postgres internally stores both as varlena; the only difference is the padding semantics, not storage.

Scenario48 · The case-sensitive join that misses 'Mumbai' vs 'mumbai'

Asked at: Make My Trip · Goibibo · **Difficulty:** Medium · **Topic:** Type bug · Collation mismatch

The scenario: A reporting query joins `searches.destination_city` to `cities.city_name`. The result set is much smaller than expected. Manual check shows that the source data has cities in mixed case ('Mumbai', 'mumbai', 'MUMBAI') while the lookup table is title case ('Mumbai').

```
SELECT s.search_id, c.country
FROM searches s
JOIN cities c ON s.destination_city = c.city_name;
-- Postgres default collation is case-sensitive.
-- Joins on 'mumbai' = 'Mumbai' return false.
```

Expected vs Actual: Expected all searches to resolve. ~30% don't, because of case mismatch.

Thinking out loud: 1. "Postgres by default is case-sensitive. 'mumbai' $\neq$ 'Mumbai' at the byte level." 2. "MySQL with default `utf8mb4_0900_ai_ci` collation is case-insensitive — 'mumbai' = 'Mumbai' = TRUE. SQL Server depends on the column collation." 3. "Fix: normalise both sides with `LOWER`, or change the column collation, or fix the source data." 4. "`LOWER()` on the join key disables the index on that key — so for hot joins, a case-insensitive collation or generated lowercase column is better."

Diagnosis: Collation determines case sensitivity at the column or expression level. Default behaviours differ across engines and even across MySQL versions.

Fix SQL:

```
SELECT s.search_id, c.country
FROM searches s
JOIN cities c ON LOWER(s.destination_city) = LOWER(c.city_name);
```

Better long-term: add a `city_name_lower` generated column with an index.

Why this works: LOWER normalises case on both sides before comparison. Index trade-off can be resolved with a functional index on `LOWER(city_name)`.

What NOT to say: - "Postgres is case-insensitive by default" — it is not; default is case-sensitive byte comparison. - "ILIKE will fix it" — ILIKE is for the LIKE operator, not equality joins.

Common follow-up: "How would you make a Postgres column case-insensitive at the schema level?" — use the `citext` extension and `CREATE TABLE cities (city_name CITEXT)`.

Scenario49 · The emoji in the customer name that breaks the join

Asked at: WhatsApp India · ShareChat · **Difficulty:** Hard · **Topic:** Type bug · Encoding mismatch (utf8 vs utf8mb4 in MySQL)

The scenario: A MySQL-based customer service tool joins WhatsApp display names to internal user IDs. Users with emojis in their display name (very common — heart, flag, etc.) fail to resolve. The join column is declared `VARCHAR(255)` but the database is on the legacy MySQL `utf8` charset (which is actually 3-byte UTF-8, not full UTF-8).

```
SELECT u.user_id, w.whatsapp_handle
FROM users u
JOIN whatsapp_profiles w ON u.display_name = w.display_name;
-- users.display_name is utf8 (3-byte) — emojis stored as '???' or lost on insert.
-- whatsapp_profiles.display_name is utf8mb4 (4-byte) — emojis stored correctly.
-- Join compares mangled vs correct bytes. No match for emoji-containing names.
```

Expected vs Actual: Expected every user to match. Emoji-bearing names drop out (often ~15% of consumer userbase).

Thinking out loud: 1. "MySQL's `utf8` charset is the historical bug — it supports only the basic multilingual plane (3-byte UTF-8). `utf8mb4` is real, full UTF-8 with 4-byte support, which is required for emojis." 2. "On insert into the utf8 column, emojis get silently replaced or the insert errors depending on strict mode. Either way, the stored value differs from the original." 3. "Even if both sides were correct, joining utf8 to utf8mb4 with default collations can throw 'illegal mix of collations' errors." 4. "Fix: convert the column to utf8mb4 + utf8mb4_0900_ai_ci collation. Postgres avoids this by being utf8-only from day one."

Diagnosis: MySQL's legacy `utf8` is 3-byte; modern UTF-8 is 4-byte (`utf8mb4`). Cross-column joins on mixed charsets fail silently or throw.

Fix SQL:

```
ALTER TABLE users
  MODIFY display_name VARCHAR(255)
  CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;

-- Then re-ingest the affected rows from source to recover the dropped emoji bytes.
```

Why this works: utf8mb4 supports the full Unicode space including emojis. After migration, both columns use the same charset and collation; joins work cleanly.

What NOT to say: - "MySQL utf8 is real UTF-8" — it is not; it is a historical 3-byte truncation. - "Just strip emojis before storage" — destroys display intent, fails on flag/skin-tone modifiers.

Common follow-up: "What's the database-level fix?" — set

`character_set_database = utf8mb4` and `collation_database = utf8mb4_0900_ai_ci` so all new tables default correctly.

Scenario50 · The double-counted GMV from a one-to-many join

Asked at: Flipkart · Tata Neu · **Difficulty:** Medium · **Topic:** Cardinality bug · SUM inflated by join

The scenario: A weekly business review shows GMV at ₹84 crore. Finance reconciliation says it should be ₹42 crore. Exactly double. The query joins `orders` to `order_items` to compute per-category GMV — but rolls up to total without thinking about cardinality.

```
SELECT SUM(o.order_total) AS total_gmv
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.order_date BETWEEN '2026-05-19' AND '2026-05-25';
```

Expected vs Actual: Expected ₹42 crore (sum of order totals). Got ₹84 crore — every order has on average 2 items, and the join duplicates `order_total` by the number of items.

Thinking out loud: 1. "Joining orders to order_items multiplies rows. A 3-item order produces 3 rows in the joined set. SUM(o.order_total) then sums the order total 3 times." 2. "The query author probably wanted per-category breakdown and forgot that summing parent-level metrics across a one-to-many join double-counts." 3. "Fix depends on what they want — for total GMV, drop the join. For per-category GMV, sum item-level amounts, not order_total." 4. "This is the most common 'looks right but is doubled' bug in SQL reporting. Always check the join cardinality before summing."

Diagnosis: Joining a parent table to a child on one-to-many duplicates the parent's columns. Summing the parent's columns across the joined set multiplies by the child count.

Fix SQL (for total GMV):

```
SELECT SUM(o.order_total) AS total_gmv
FROM orders o
WHERE o.order_date BETWEEN '2026-05-19' AND '2026-05-25';
```

For per-category GMV, sum item-level prices:

```
SELECT oi.category, SUM(oi.line_amount) AS gmv
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.order_date BETWEEN '2026-05-19' AND '2026-05-25'
GROUP BY oi.category;
```

Why this works: For totals, the parent table alone has every order once. For per-child breakdowns, you sum child-level amounts. Don't mix scopes.

What NOT to say: - "Use DISTINCT" — `SELECT SUM(DISTINCT order_total)` is wrong because two different orders might genuinely have the same total. - "JOIN bug, switch to LEFT JOIN" — doesn't change the duplication.

Common follow-up: "How would you debug this without recomputing manually?" — `SELECT order_id, COUNT(*) FROM (joined query) GROUP BY order_id HAVING COUNT(*) > 1;` — confirms duplication.

Scenario51 · The many-to-many that exploded to 12 million rows

Asked at: LinkedIn India · Naukri · **Difficulty:** Medium · **Topic:** Cardinality bug · M:N row explosion

The scenario: A recruiter analytics query joins `candidates` to `applications` to `interviews` and the result set is 12 million rows from 50,000 candidates. The query takes 4 minutes and then crashes the dashboard. Each candidate has on average 8 applications and 6 interviews — $50,000 \times 8 \times 6 \approx 2.4$ million, but they're seeing 12 million.

```
SELECT c.candidate_id, c.name, a.role, i.interview_round
FROM candidates c
JOIN applications a ON a.candidate_id = c.candidate_id
JOIN interviews i ON i.candidate_id = c.candidate_id;
-- All three joins on candidate_id only.
-- The applications join produces 50k × 8 = 400k rows.
-- The interviews join cross-products with applications, producing 400k × 6 = 2.4M.
-- Actual observed is higher because the average is heavy-tailed.
```

Expected vs Actual: Expected one row per interview (~300k). Got 12M — a Cartesian-style explosion because applications × interviews is a cross-product per candidate.

Thinking out loud: 1. "Joining candidates → applications gives one row per application. Joining that to interviews on candidate_id gives every (application, interview) pair for the same candidate." 2. "That is a cross-product within each candidate. If candidate A has 8 applications and 6 interviews, the result contains 48 rows for that candidate — even though interviews are not tied to specific applications in this join." 3. "The right join key is

`interviews.application_id = applications.application_id`, assuming interviews are scheduled per application. With that, you get one row per interview." 4. "Without it, the engine has no choice but to enumerate every pair."

Diagnosis: Joining three tables only on the common ancestor key creates a many-to-many Cartesian within each group. Need the secondary FK to constrain the join.

Fix SQL:

```
SELECT c.candidate_id, c.name, a.role, i.interview_round
FROM candidates c
JOIN applications a ON a.candidate_id = c.candidate_id
JOIN interviews i ON i.application_id = a.application_id;
```

Why this works: `interviews.application_id` ties each interview to a specific application. The join is now one-per-interview, not all-pairs-per-candidate.

What NOT to say: - "Use SELECT DISTINCT to dedup" — masks the bug, doesn't fix it, and is expensive on 12M rows. - "Add LIMIT 1000" — hides the explosion, lies about the data.

Common follow-up: "What if interviews don't have an application_id?" — then your schema can't answer "which application is this interview for" — you have a data model problem, not a query problem.

Scenario52 · The LEFT JOIN to a subquery that duplicates parents

Asked at: Razorpay · Stripe India · **Difficulty:** Hard · **Topic:** Cardinality bug · Subquery returns multiple rows per outer row

The scenario: A "latest payment" report uses a LEFT JOIN to a derived table that's supposed to return one payment per customer. But the report shows some customers appearing multiple times.

```
SELECT c.customer_id, c.name, p.payment_id, p.payment_ts
FROM customers c
LEFT JOIN (
    SELECT customer_id, payment_id, payment_ts
    FROM payments
    WHERE status = 'SUCCESS'
) p ON p.customer_id = c.customer_id;
```


Expected vs Actual: Expected one row per customer, with their latest payment (or NULL if none). Got many rows per customer — one per successful payment.

Thinking out loud: 1. "The subquery returns every successful payment per customer, not the latest. The LEFT JOIN then expands to one row per payment." 2. "The author probably thought 'latest' was implied by some ordering. SQL has no implicit ordering; you have to explicitly pick the latest." 3. "Fix: use a window function (ROW_NUMBER) to pick the latest per customer, then filter to row 1. Or use DISTINCT ON in Postgres." 4. "Or — if the goal is 'show the latest', use a correlated subquery with (SELECT MAX(payment_ts) FROM payments WHERE customer_id = c.customer_id) ."

Diagnosis: A "latest per group" intent expressed without a deduplication step. The subquery returns N payments per customer; the LEFT JOIN duplicates the customer N times.

Fix SQL (Postgres DISTINCT ON):

```
SELECT c.customer_id, c.name, p.payment_id, p.payment_ts
FROM customers c
LEFT JOIN (
    SELECT DISTINCT ON (customer_id) customer_id, payment_id, payment_ts
    FROM payments
    WHERE status = 'SUCCESS'
    ORDER BY customer_id, payment_ts DESC
) p ON p.customer_id = c.customer_id;
```

Portable form (any dialect):

```
LEFT JOIN (
    SELECT customer_id, payment_id, payment_ts,
           ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY payment_ts DESC) AS rn
    FROM payments
    WHERE status = 'SUCCESS'
) p ON p.customer_id = c.customer_id AND p.rn = 1
```

Why this works: The deduplication happens inside the subquery so each customer has exactly one payment row (or zero), and the LEFT JOIN preserves the one-row-per-customer cardinality.

What NOT to say: - "Use MAX(payment_ts) GROUP BY customer_id" — gives you the timestamp but not the payment_id of that latest payment. - "Add LIMIT 1 to the subquery" — that limits the entire subquery to 1 row globally, not 1 per customer.

Common follow-up: "What if you wanted the top 3 payments per customer instead of just the latest?" — same ROW_NUMBER pattern with WHERE rn <= 3 .

Scenario53 · The wrong join key that returned zero rows

Asked at: Stripe India · Pine Labs · **Difficulty:** Easy · **Topic:** Cardinality bug · Wrong join key

The scenario: A merchant reconciliation query joins `transactions` to `merchants`. It used to work; after a schema migration, it returns zero rows.

```
SELECT t.txn_id, t.amount, m.merchant_name
FROM transactions t
JOIN merchants m ON t.user_id = m.user_id;
-- After migration:
-- transactions.user_id is the END USER who paid.
-- merchants.user_id no longer exists; the new field is merchants.owner_user_id.
-- Old query used merchants.user_id = the merchant's owner_user_id.
-- The correct relationship is transactions.merchant_id = merchants.merchant_id.
```

Expected vs Actual: Expected every transaction to resolve to a merchant. Got zero rows or an "unknown column" error depending on engine.

Thinking out loud: 1. "The join key is wrong. We are matching the paying user's ID to the merchant's owner user ID. Different things." 2. "Pre-migration, the schema happened to have both fields named `user_id`, so the join 'worked' by accident — until business added a check that revealed it was joining wrong rows the whole time." 3. "Post-migration, the column was renamed and the query now errors. Lucky — the migration exposed the bug." 4. "The right join key is `transactions.merchant_id = merchants.merchant_id`."

Diagnosis: Wrong join key. Two columns happened to share a name (`user_id`) but represent different entities. The join silently produced incorrect results until a rename forced the bug into the open.

Fix SQL:

```
SELECT t.txn_id, t.amount, m.merchant_name
FROM transactions t
JOIN merchants m ON t.merchant_id = m.merchant_id;
```

Why this works: Joining on the proper foreign key that actually models the "transaction belongs to merchant" relationship.

What NOT to say: - "Restore the old column name as a backwards-compatible alias" — preserves the bug, papers over the rename. - "Join on multiple keys for safety" — confuses cardinality further.

Common follow-up: "How would you catch this kind of bug in code review?" — name your foreign key columns by the entity they reference (`merchant_id` , not `user_id`), and avoid generic names like `user_id` that can mean different roles in different tables.

Scenario54 · The self-join that counted every friend pair twice

Asked at: Bumble India · ShareChat · **Difficulty:** Medium · **Topic:** Cardinality bug · Self-join without ordering predicate

The scenario: A "mutual friends" analysis self-joins the `friendships` table to find pairs of users who are friends. The output count is exactly double the expected (each pair appears as (A, B) and (B, A)).

```
SELECT f1.user_id AS user_a, f2.user_id AS user_b
FROM friendships f1
JOIN friendships f2 ON f1.friend_id = f2.user_id
                    AND f1.user_id = f2.friend_id;
-- friendships table stores (user_id, friend_id) – symmetric, both directions.
-- The self-join finds reciprocal pairs but counts each pair twice.
```

Expected vs Actual: Expected N pairs. Got 2N rows — each pair appears as (A, B) and (B, A).

Thinking out loud: 1. "Classic self-join symmetry bug. If (A, B) qualifies, so does (B, A) under the same predicate." 2. "Standard fix is to add `WHERE f1.user_id < f2.user_id` — only one ordering of each pair survives." 3. "This is the same pattern as 'find duplicate emails' or 'find users in same city' — any predicate that's symmetric across two copies of the same table produces doubled output." 4. "The `<` predicate also conveniently excludes self-matches where `user_id = friend_id`."

Diagnosis: Symmetric self-join without an ordering predicate. Both orderings of each pair satisfy the join, doubling the row count.

Fix SQL:

```
SELECT f1.user_id AS user_a, f2.user_id AS user_b
FROM friendships f1
JOIN friendships f2 ON f1.friend_id = f2.user_id
                    AND f1.user_id = f2.friend_id
WHERE f1.user_id < f2.user_id;
```

Why this works: `f1.user_id < f2.user_id` keeps only the (A, B) ordering where A's ID is smaller. (B, A) where B > A is rejected. Each pair appears once.

What NOT to say: - "SELECT DISTINCT" — works on (a, b) tuples but if you compute counts later, you still need ordering. - "Use UNION instead of JOIN" — different shape; doesn't address the symmetry.

Common follow-up: "What if you want self-friendships (user is friends with self) included?" — change `<` to `<=`. That keeps (A, A) once.

Category 1, Part C — Debug This Query: Aggregation, Window Function & Recency Bugs

Part C wraps up the Debug category by tackling the bugs that show up most often in product company SQL rounds — aggregations that silently return wrong numbers because of NULL handling or join inflation, window functions that fail because the candidate forgot about frames or evaluation order, and the classic "give me the latest record per user" question that has at least four wrong answers and two clean ones. Interviewers love these because the query "runs" — it just returns garbage, and only a candidate who has actually been bitten in production will spot it.

Scenario55 · GROUP BY Missing a Non-Aggregated Column

Tags: aggregation, group-by, sql-standard, postgres-strict, mysql-lenient

Scenario: A junior engineer at a Pune-based edtech firm writes a query to show the latest enrolment per course. It runs on the staging MySQL box, returns plausible numbers, and ships. A week later the product team complains the dashboard shows random instructor names against each course.

SQL:

```
SELECT
  course_id,
  instructor_name,
  MAX(enrolled_at) AS latest_enrolment
FROM enrolments
GROUP BY course_id;
```

Expected vs Actual: Expected one row per course with the correct instructor. Actual: instructor_name is picked arbitrarily — sometimes the right one, sometimes a stale row.

Thinking out loud: 1. The query groups by `course_id` but selects `instructor_name` which isn't in the GROUP BY and isn't aggregated. 2. ANSI SQL says this is illegal. MySQL with `ONLY_FULL_GROUP_BY` off will accept it and return *any* value from the group — undefined behaviour. 3. Postgres would have rejected this at parse time with `column "instructor_name" must appear in the GROUP BY clause`. 4. Even if instructor and course are 1:1, the engine doesn't know that — it returns the first row it happens to see.

Diagnosis: Non-deterministic column selection because of lenient MySQL mode. The query is functionally dependent in business logic but not declared as such to the engine.

Fix:

```
SELECT course_id, instructor_name, MAX(enrolled_at) AS latest_enrolment
FROM enrolments
GROUP BY course_id, instructor_name;
```

Why this works: Adding `instructor_name` to GROUP BY makes the engine partition by both columns, which is safe when each course truly has one instructor. If a course can have multiple instructors, you instead want a window function or a join-back pattern (covered in Scenario14).

What NOT to say: "MySQL just picks randomly so add ORDER BY." ORDER BY does not fix selection within a group — it only orders the final result set.

Common follow-up: "What if the same course has had two instructors over time?" Then GROUP BY both, or use ROW_NUMBER partitioned by course_id ordered by enrolled_at DESC and filter to rn=1 to get the latest instructor.

Scenario56 · HAVING vs WHERE Confusion

Tags: having, where, aggregation, evaluation-order

Scenario: A Bengaluru fintech analyst writes a query to find merchants who processed more than ₹10 lakh in successful transactions in April. The query returns merchants who never had any successful April transactions at all.

SQL:

```
SELECT merchant_id, SUM(amount) AS total
FROM transactions
GROUP BY merchant_id
HAVING status = 'success'
      AND txn_date >= '2026-04-01'
      AND SUM(amount) > 1000000;
```

Expected vs Actual: Expected merchants whose successful April sums cross ₹10L. Actual: merchants whose *total* (across all statuses and dates) crosses ₹10L, filtered weirdly because `status = 'success'` in HAVING is treated against an arbitrary row.

Thinking out loud: 1. HAVING runs after GROUP BY on aggregate-aware predicates. Row-level filters like `status = 'success'` belong in WHERE. 2. In some engines `HAVING status = 'success'` either throws or evaluates against an undefined row in the group. 3. The SUM is over *all* transactions for the merchant, not just successful April ones, because the row filter was never applied. 4. The fix is to move row predicates to WHERE and keep only the aggregate predicate in HAVING.

Diagnosis: Row-level filters placed in HAVING are either ignored, rejected, or applied at the wrong stage.

Fix:

```
SELECT merchant_id, SUM(amount) AS total
FROM transactions
WHERE status = 'success'
      AND txn_date >= '2026-04-01'
GROUP BY merchant_id
HAVING SUM(amount) > 1000000;
```

Why this works: WHERE filters rows *before* aggregation, so SUM only adds successful April amounts. HAVING then filters the aggregated groups.

What NOT to say: "HAVING and WHERE are interchangeable for performance." They are not interchangeable for correctness, and putting row filters in HAVING is always slower because the engine aggregates rows it should have discarded.

Common follow-up: "When would you put a non-aggregate in HAVING?" Almost never. The one valid case is when you've already grouped and the column is a GROUP BY key — but even then WHERE is clearer.

Scenario57 · COUNT(*) vs COUNT(column) NULL Handling

Tags: count, nulls, aggregation

Scenario: A Hyderabad SaaS team tracks support tickets. PM asks "how many tickets did we close last week with a resolution note?" Engineer writes `COUNT(*)` and gets 4,200. Manual sample suggests it should be closer to 3,100.

SQL:

```
SELECT COUNT(*) AS resolved_with_note
FROM tickets
WHERE status = 'closed'
      AND closed_at >= CURRENT_DATE - INTERVAL '7 days';
```

Expected vs Actual: Expected count of closed tickets *that have a non-null resolution_note*. Actual: count of all closed tickets, including those with NULL notes.

Thinking out loud: 1. `COUNT(*)` counts rows regardless of NULL. 2. `COUNT(resolution_note)` counts rows where that column is NOT NULL. 3. The question asked for "with a resolution note" — that's a NULL filter, not a row count. 4. Either switch to `COUNT(resolution_note)` or add `AND resolution_note IS NOT NULL` to WHERE.

Diagnosis: COUNT(*) ignores NULL semantics; the analyst conflated "row exists" with "column has a value".

Fix:

```
SELECT COUNT(resolution_note) AS resolved_with_note
FROM tickets
WHERE status = 'closed'
AND closed_at >= CURRENT_DATE - INTERVAL '7 days';
```

Why this works: COUNT(column) silently skips NULLs, which is exactly the semantic we want here.

What NOT to say: "COUNT(*) and COUNT(1) are different." They are identical in every modern engine.

Common follow-up: "What if the column is sometimes an empty string?" Empty strings are not NULL — COUNT(col) would still count them. You'd need `COUNT(NULLIF(resolution_note, ''))` or a WHERE filter for trimmed length.

Scenario58 · COUNT(DISTINCT) on a 200-Million-Row Table

Tags: count-distinct, performance, approximate-functions

Scenario: A Mumbai-based OTT platform runs `COUNT(DISTINCT user_id)` over a 200M-row event table to estimate weekly active users. The query takes 14 minutes and locks up the BI warehouse during morning standup hours.

SQL:

```
SELECT COUNT(DISTINCT user_id) AS wau
FROM events
WHERE event_date >= CURRENT_DATE - INTERVAL '7 days';
```

Expected vs Actual: Expected sub-minute response for a dashboard. Actual: 14 minutes, blocks other queries.

Thinking out loud: 1. COUNT(DISTINCT) needs to hold every unique value in memory or spill to disk — it doesn't scale. 2. Most modern warehouses (BigQuery, Snowflake, Redshift, DuckDB) have an approximate variant — APPROX_COUNT_DISTINCT or APPROX_DISTINCT — with 1-2% error and a fraction of the cost. 3. For an internal dashboard, 1% error on WAU is invisible. 4. If we truly need exact, partition the query by date and roll up via HyperLogLog sketches.

Diagnosis: Using exact distinct when approximate suffices — a classic warehouse cost trap.

Fix (dialect-dependent):

```
-- BigQuery / Snowflake
SELECT APPROX_COUNT_DISTINCT(user_id) AS wau
FROM events
WHERE event_date >= CURRENT_DATE - INTERVAL '7 days';

-- Postgres (no native; use the hll extension or pg_stat_sampling)
-- Redshift
SELECT APPROXIMATE COUNT(DISTINCT user_id) AS wau FROM events WHERE ...;
```

Why this works: HyperLogLog sketches give cardinality estimates with bounded error in constant memory.

What NOT to say: "Just add an index on user_id." Indexes don't help COUNT(DISTINCT) materially because the engine still has to scan every distinct value.

Common follow-up: "What if finance needs an exact count for monthly reporting?" Run the exact query offline on the weekend or maintain a daily distinct sketch that you UNION.

Scenario59 · SUM(CASE WHEN ...) Returns 0 vs NULL

Tags: conditional-aggregation, nulls, coalesce

Scenario: A Delhi-based D2C brand wants the count of refunds per order. Analyst joins orders with refunds and aggregates. Orders with no refunds show 0 in one query and NULL in another, and downstream filtering breaks.

SQL:

```
SELECT
  o.order_id,
  SUM(CASE WHEN r.refund_id IS NOT NULL THEN 1 ELSE 0 END) AS refund_count
FROM orders o
LEFT JOIN refunds r ON r.order_id = o.order_id
GROUP BY o.order_id;
```

Expected vs Actual: Expected 0 for unrefunded orders. Actual: 0 — which is fine here, but the *neighbouring* query that did `SUM(r.amount)` returned NULL for unrefunded orders and broke a `WHERE refund_total > 0` filter elsewhere.

Thinking out loud: 1. SUM over an empty set returns NULL in SQL standard, not 0. 2. SUM(CASE WHEN ... THEN 1 ELSE 0 END) returns 0 because the ELSE 0 makes it a non-empty sum when there's any row — and the LEFT JOIN produces at least one row (with r.* as NULL). 3. SUM(r.amount) returns NULL because every joined row has NULL for amount. 4. The fix is COALESCE(SUM(...), 0) when you want 0 for empty groups.

Diagnosis: SQL's SUM semantics return NULL for "no rows", and LEFT JOIN preserves the outer row but with NULL columns. Without ELSE or COALESCE, you get NULL.

Fix:

```
SELECT
  o.order_id,
  COALESCE(SUM(r.amount), 0) AS refund_total,
  COUNT(r.refund_id) AS refund_count
FROM orders o
LEFT JOIN refunds r ON r.order_id = o.order_id
GROUP BY o.order_id;
```

Why this works: COALESCE turns NULL totals into 0. COUNT(r.refund_id) handles the row-count side cleanly because COUNT(col) skips NULLs.

What NOT to say: "Use ISNULL." ISNULL is SQL Server only — COALESCE is portable.

Common follow-up: "Why not just inner join?" Inner join would drop orders with no refunds entirely, which we need for the dashboard's "refund rate per order" metric.

Scenario60 · AVG Over Filtered Subset — Wrong Denominator

Tags: avg, denominator, filtering, semantics

Scenario: A Chennai-based food delivery startup reports "average delivery time for late orders" as 47 minutes. The CEO asks why the same metric drops to 32 minutes when computed with a different filter clause. Both queries claim to use the same data.

SQL (the wrong one):

```
SELECT AVG(CASE WHEN delivered_at > eta THEN delivery_minutes END) AS avg_late_time
FROM orders;
```

Expected vs Actual: Expected the average over late orders only. Actual: SQL's AVG ignores NULL, so the function looks correct — but the analyst expected the denominator to be *total orders*, not *late orders*, leading to confusion when this number was compared with a SUM/COUNT version that used the wrong denominator.

Thinking out loud: 1. AVG(CASE WHEN cond THEN val END) gives average over rows where cond is true — NULLs are skipped. 2. SUM(CASE WHEN cond THEN val ELSE 0 END) / COUNT() *gives average over all rows** — different denominator. 3. The two formulas answer different questions. Pick the one matching the business definition. 4. Always spell out the denominator in code review.

Diagnosis: Ambiguous "average" — average of late delivery times, vs. expected delivery time across the whole order base.

Fix (explicit):

```
-- Average over late orders only
SELECT
  AVG(delivery_minutes) AS avg_late_time
FROM orders
WHERE delivered_at > eta;

-- Average lateness contribution over all orders
SELECT
  SUM(CASE WHEN delivered_at > eta THEN delivery_minutes - eta_minutes ELSE 0 END) * 1.0 / COUNT(*)
```

Why this works: Each form spells out exactly which rows are in the numerator and denominator.

What NOT to say: "AVG always handles NULL correctly." Correctly by SQL standards, yes — but that's often the opposite of what the business meant.

Common follow-up: "How would you compute weighted average?" `SUM(value * weight) / SUM(weight)` — and watch for SUM(weight) being zero.

Scenario61 · Median via AVG — Statistically Wrong

Tags: median, percentile, statistical-functions

Scenario: A Gurgaon-based jobs platform reports "median salary offered" but the dashboard shows ₹14.7L when the recruitment team insists the true median is closer to ₹9L. The query is being defended in a review meeting.

SQL:

```
SELECT AVG(salary) AS median_salary
FROM offers
WHERE offered_at >= '2026-04-01';
```

Expected vs Actual: Expected the middle value (50th percentile). Actual: the arithmetic mean — heavily skewed by a few ₹80L+ exec offers.

Thinking out loud: 1. AVG is mean, not median. The analyst named the column wrong. 2. True median in SQL needs PERCENTILE_CONT(0.5) or PERCENTILE_DISC(0.5) — ordered-set aggregates. 3. Postgres and Oracle support PERCENTILE_CONT WITHIN GROUP. BigQuery has PERCENTILE_CONT as a

window/analytic. MySQL 8 has no native — needs window simulation. 4. Salary data is right-skewed — mean will always overshoot median.

Diagnosis: Mean used as a proxy for median on skewed data — guaranteed to mislead.

Fix:

```
-- Postgres / Oracle
SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary) AS median_salary
FROM offers
WHERE offered_at >= '2026-04-01';

-- BigQuery
SELECT APPROX_QUANTILES(salary, 100)[OFFSET(50)] AS median_salary
FROM offers
WHERE offered_at >= '2026-04-01';

-- MySQL 8 (simulation)
WITH ranked AS (
  SELECT salary, ROW_NUMBER() OVER (ORDER BY salary) AS rn, COUNT(*) OVER () AS cnt
  FROM offers WHERE offered_at >= '2026-04-01'
)
SELECT AVG(salary) AS median_salary
FROM ranked
WHERE rn IN (FLOOR((cnt+1)/2), CEIL((cnt+1)/2));
```

Why this works: PERCENTILE_CONT interpolates the true median; the MySQL simulation picks the middle one or two ordered rows and averages.

What NOT to say: "Mean and median are basically the same for large samples." Only on symmetric distributions — salary is never symmetric.

Common follow-up: "What about p95 latency?" Same pattern —

`PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY latency_ms)` .

Scenario62 · LAST_VALUE With Default Frame

Tags: window-functions, frames, last-value

Scenario: A Bengaluru analyst writes a window query to fetch the last login timestamp per user across their session history, expecting each row to show the user's most recent login. Every row shows that row's own login instead.

SQL:

```
SELECT
  user_id,
  login_at,
  LAST_VALUE(login_at) OVER (PARTITION BY user_id ORDER BY login_at) AS last_login
FROM sessions;
```

Expected vs Actual: Expected each row's `last_login` to equal the user's final `login_at`. Actual: `last_login = login_at` on every row.

Thinking out loud: 1. The default frame in a window with ORDER BY is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. 2. `LAST_VALUE` on that frame returns the current row — because the frame ends at current row. 3. The fix is to extend the frame to `UNBOUNDED FOLLOWING`. 4. Alternative: use `FIRST_VALUE` with DESC ordering.

Diagnosis: Default window frame stops at current row, so `LAST_VALUE` never sees future rows in the partition.

Fix:

```
SELECT
  user_id,
  login_at,
  LAST_VALUE(login_at) OVER (
    PARTITION BY user_id
    ORDER BY login_at
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
  ) AS last_login
FROM sessions;

-- Or simpler:
SELECT
  user_id,
  login_at,
  MAX(login_at) OVER (PARTITION BY user_id) AS last_login
FROM sessions;
```

Why this works: Either explicit unbounded frame, or `MAX` (which doesn't need a frame because it's a partition-level aggregate).

What NOT to say: "LAST_VALUE is just buggy in this engine." It's working as specified — the frame default is the trap.

Common follow-up: "When would you actually want LAST_VALUE?" Almost never in interview problems — `MAX/FIRST_VALUE-with-DESC` or `LAG/LEAD` cover most needs more clearly.

Scenario63 · Filtering on a Window Function Result

Tags: window-functions, where-clause, evaluation-order, cte

Scenario: An Indore-based logistics analyst tries to fetch only the latest event per shipment using ROW_NUMBER. The query throws "column rn does not exist" or returns all rows depending on the engine.

SQL (broken):

```
SELECT
  shipment_id,
  event_at,
  status,
  ROW_NUMBER() OVER (PARTITION BY shipment_id ORDER BY event_at DESC) AS rn
FROM shipment_events
WHERE rn = 1;
```

Expected vs Actual: Expected one latest event per shipment. Actual: error (most engines) or unfiltered output.

Thinking out loud: 1. WHERE is evaluated *before* SELECT, so the alias `rn` doesn't exist yet at the WHERE phase. 2. Window functions are computed during/after SELECT, not before WHERE. 3. The fix is to wrap in a CTE or subquery, then filter on the outer level. 4. QUALIFY (BigQuery, Snowflake, Teradata, DuckDB) is the cleanest variant.

Diagnosis: Logical query processing order — WHERE runs before window functions are evaluated.

Fix:

```
-- Portable
WITH ranked AS (
  SELECT
    shipment_id, event_at, status,
    ROW_NUMBER() OVER (PARTITION BY shipment_id ORDER BY event_at DESC) AS rn
  FROM shipment_events
)
SELECT shipment_id, event_at, status
FROM ranked
WHERE rn = 1;

-- Snowflake / BigQuery / DuckDB
SELECT shipment_id, event_at, status
FROM shipment_events
QUALIFY ROW_NUMBER() OVER (PARTITION BY shipment_id ORDER BY event_at DESC) = 1;
```

Why this works: Wrapping isolates the window evaluation; QUALIFY is syntactic sugar that lets the engine push the filter at the right phase.

What NOT to say: "Just use HAVING instead of WHERE." HAVING doesn't apply to window functions either — both run before window phase.

Common follow-up: "When does the engine evaluate window functions exactly?" After GROUP BY/HAVING but before final SELECT / ORDER BY / LIMIT.

Scenario64 · ROW_NUMBER vs RANK vs DENSE_RANK — Wrong Pick

Tags: window-functions, row-number, rank, dense-rank, ties

Scenario: A Bengaluru gaming company runs a "top 10 scorers this week" leaderboard. Six players tie at rank 1 with the same high score, but the leaderboard shows only one of them and ranks the next four players as 2, 3, 4, 5.

SQL:

```
SELECT player_id, score,  
       ROW_NUMBER() OVER (ORDER BY score DESC) AS leaderboard_rank  
FROM weekly_scores;
```

Expected vs Actual: Expected ties to share rank 1, then next player at rank 2 (DENSE_RANK) or rank 7 (RANK). Actual: arbitrary rank 1-6 across the six tied players, and rank 7 onward looks normal — but a tied player at #6 got pushed off the top-10 unfairly.

Thinking out loud: 1. ROW_NUMBER always gives 1,2,3 with no ties. 2. RANK gives 1,1,1,1,1,1,7,8,9 — gaps after ties. 3. DENSE_RANK gives 1,1,1,1,1,2,3,4 — no gaps. 4. For a leaderboard where ties should genuinely share the rank, RANK is the typical pick; DENSE_RANK if you want gapless.

Diagnosis: ROW_NUMBER is non-deterministic on ties unless tiebreakers are added; wrong function for a leaderboard.

Fix:

```
SELECT player_id, score,  
       RANK() OVER (ORDER BY score DESC) AS leaderboard_rank  
FROM weekly_scores;
```

Why this works: RANK lets ties share the same rank, which is the universally understood leaderboard semantic.

What NOT to say: "RANK and DENSE_RANK are interchangeable." They are not — the gap behaviour matters for "top N with ties".

Common follow-up: "How do you pick top N with ties guaranteed included?" Use RANK() (or DENSE_RANK) in a CTE then filter `WHERE rank <= 10` — this returns more than 10 rows if ties exist at position 10.

Scenario65 · LAG Returning NULL for First Row in Partition

Tags: lag, lead, defaults, window-functions

Scenario: A Hyderabad e-commerce analyst computes the time gap between consecutive orders per customer. The first order per customer shows NULL gap, which is correct — but downstream `WHERE gap_days > 30` filtering drops all first-order rows silently, biasing churn analysis.

SQL:

```
SELECT
  customer_id, order_id, order_at,
  EXTRACT(DAY FROM order_at - LAG(order_at) OVER (PARTITION BY customer_id ORDER BY order_at)) AS gap_days
FROM orders;
```

Expected vs Actual: Expected first-order rows to either be included or explicitly excluded with reasoning. Actual: silent NULL → silent drop on downstream filter.

Thinking out loud: 1. LAG defaults to NULL for the first row per partition because there's no prior row. 2. LAG has a third argument: `LAG(col, 1, default_value)` — useful when you want a sentinel. 3. NULL > 30 is NULL, which is treated as false in WHERE — so first-order customers vanish from any "gap" filter. 4. Either set a default (e.g., the customer signup date) or handle NULL explicitly.

Diagnosis: Default NULL for first row in partition flows silently into filter and biases the result.

Fix:

```
SELECT
  customer_id, order_id, order_at,
  EXTRACT(DAY FROM order_at - LAG(order_at, 1, signup_at) OVER (PARTITION BY customer_id ORDER BY order_at)) AS gap_days
FROM orders o
JOIN customers c USING (customer_id);

-- Or be explicit downstream
WHERE gap_days IS NULL OR gap_days > 30
```

Why this works: Either substituting signup_at as the previous-row default makes the first gap meaningful, or being explicit about NULL handling prevents silent drops.

What NOT to say: "LAG returns 0 for missing rows." It returns NULL — always.

Common follow-up: "What if you need the next gap instead?" Use LEAD with the same default-value pattern.

Scenario66 · Running Total Broken Because No ORDER BY in OVER

Tags: running-total, cumulative-sum, window-functions

Scenario: A Pune fintech runs a daily running total of payments received per branch. The numbers are random and inconsistent across reruns of the same query.

SQL:

```
SELECT
  branch_id, payment_date, amount,
  SUM(amount) OVER (PARTITION BY branch_id) AS running_total
FROM payments;
```

Expected vs Actual: Expected cumulative sum up to each row's date. Actual: every row shows the branch's *grand total* — same value on every row.

Thinking out loud: 1. SUM OVER (PARTITION BY x) with no ORDER BY is a partition-level aggregate — same value on every row in the partition. 2. To get cumulative, add `ORDER BY payment_date` inside OVER. 3. The default frame with ORDER BY is `ROWS UNBOUNDED PRECEDING AND CURRENT ROW` — exactly what running total needs. 4. Without ORDER BY, the frame defaults to the entire partition.

Diagnosis: Missing ORDER BY inside OVER turns "running" into "total".

Fix:

```
SELECT
  branch_id, payment_date, amount,
  SUM(amount) OVER (
    PARTITION BY branch_id
    ORDER BY payment_date
    ROWS UNBOUNDED PRECEDING
  ) AS running_total
FROM payments;
```

Why this works: ORDER BY inside OVER establishes the cumulative direction; the explicit frame removes ambiguity around RANGE vs ROWS.

What NOT to say: "Just use a self-join with `<=` ." Self-joins for running totals are $O(N^2)$ and inexcusable when window functions exist.

Common follow-up: "What if there are ties in payment_date?" RANGE and ROWS behave differently with ties — RANGE includes all tied rows in the same frame edge, ROWS does not. Use ROWS for strict per-row cumulative.

Scenario67 · Multiple Unnamed Windows in Same Query

Tags: window-functions, named-window, performance

Scenario: A Bengaluru analytics team's nightly report has six window functions, each with the same `PARTITION BY user_id ORDER BY event_at` clause. Query runs for 28 minutes. The plan shows six separate Sort operations.

SQL (problematic):

```
SELECT
  user_id, event_at,
  ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY event_at) AS rn,
  LAG(event_at) OVER (PARTITION BY user_id ORDER BY event_at) AS prev_evt,
  LEAD(event_at) OVER (PARTITION BY user_id ORDER BY event_at) AS next_evt,
  FIRST_VALUE(page) OVER (PARTITION BY user_id ORDER BY event_at) AS landing,
  LAST_VALUE(page) OVER (PARTITION BY user_id ORDER BY event_at ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS last_page,
  COUNT(*) OVER (PARTITION BY user_id ORDER BY event_at) AS events_so_far
FROM events;
```

Expected vs Actual: Expected a single sort. Actual: planner sorted six times because each OVER is structurally distinct in the parser even if semantically identical.

Thinking out loud: 1. Most engines can reuse a sort when OVER clauses are *textually identical* — but the LAST_VALUE one differs (explicit frame), breaking reuse. 2. Named windows via WINDOW clause make sharing explicit and let the planner reuse the sort. 3. Splitting the LAST_VALUE into its own pass might be unavoidable but the other five can share. 4. Always check the explain plan for sort count.

Diagnosis: Window reuse depends on textual identity; minor differences split the sort.

Fix:

```
SELECT
  user_id, event_at,
  ROW_NUMBER() OVER w AS rn,
  LAG(event_at) OVER w AS prev_evt,
  LEAD(event_at) OVER w AS next_evt,
  FIRST_VALUE(page) OVER w AS landing,
  LAST_VALUE(page) OVER w_full AS exit_page,
  COUNT(*) OVER w AS events_so_far
FROM events
WINDOW
  w AS (PARTITION BY user_id ORDER BY event_at),
  w_full AS (PARTITION BY user_id ORDER BY event_at ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)
```

Why this works: Named windows are explicit reuse contracts; engines that respect WINDOW clauses sort once per named window.

What NOT to say: "Window functions always share the sort." They share only when the OVER clause is byte-identical (in many engines) or named identically.

Common follow-up: "Does Snowflake/BigQuery support WINDOW clauses?" Both do; MySQL 8 does; Postgres does; some older engines don't. Check the docs for your target.

Scenario68 · Latest Record Per Group via MAX + JOIN — Duplicates on Ties

Tags: latest-per-group, join, ties, duplicates

Scenario: A Delhi-based jobs platform wants the latest application per candidate. The query joins applications with `MAX(applied_at)` per candidate. Some candidates show up twice — they applied to two roles at the exact same millisecond.

SQL:

```
SELECT a.*
FROM applications a
JOIN (
  SELECT candidate_id, MAX(applied_at) AS max_t
  FROM applications
  GROUP BY candidate_id
) m ON a.candidate_id = m.candidate_id AND a.applied_at = m.max_t;
```

Expected vs Actual: Expected one row per candidate. Actual: candidates with multi-application ties at the same timestamp returned twice.

Thinking out loud: 1. MAX + self-join is the classic "latest per group" pattern. 2. It breaks on ties because both rows with max timestamp match the join. 3. To tie-break, you need a secondary ordering

— but MAX can't express that. 4. The cleaner fix is ROW_NUMBER (Scenario15) or DISTINCT ON (Scenario16).

Diagnosis: MAX + JOIN pattern has no tiebreaker, so any tie at the max value duplicates the row.

Fix (with tiebreaker):

```
-- Tiebreak by application_id (or any unique secondary)
SELECT a.*
FROM applications a
JOIN (
  SELECT candidate_id, MAX(applied_at) AS max_t
  FROM applications
  GROUP BY candidate_id
) m ON a.candidate_id = m.candidate_id AND a.applied_at = m.max_t
QUALIFY ROW_NUMBER() OVER (PARTITION BY a.candidate_id ORDER BY a.application_id DESC) = 1;
```

But Scenario15's pattern is cleaner.

Why this works: We've added a deterministic tiebreaker. Still messier than ROW_NUMBER though.

What NOT to say: "Use SELECT DISTINCT at the end." DISTINCT on `a.*` works only if columns differ — and with truly identical rows we still won't deduplicate something we want.

Common follow-up: "Why is this pattern still taught?" Historical — pre-window-function SQL had no better option. Modern code shouldn't use it.

Scenario69 · Latest Record Per Group via ROW_NUMBER — The Canonical Fix

Tags: latest-per-group, row-number, canonical-pattern

Scenario: After hitting the MAX+JOIN duplication issue (Scenario14), the same team rewrites with ROW_NUMBER. Result is clean and one-row-per-candidate.

SQL:

```
WITH ranked AS (
  SELECT a.*,
    ROW_NUMBER() OVER (PARTITION BY candidate_id ORDER BY applied_at DESC, application_id DESC) AS rn
  FROM applications a
)
SELECT *
FROM ranked
WHERE rn = 1;
```

Expected vs Actual: Expected one row per candidate. Actual: exactly that, every time, even with timestamp ties.

Thinking out loud: 1. ROW_NUMBER gives a unique sequence per partition — no ties. 2. The ORDER BY inside OVER controls which row gets rn=1. 3. Adding a secondary key (application_id DESC) guarantees determinism on ties. 4. Filtering rn = 1 in outer query picks exactly one per group.

Diagnosis: This is the fix to Scenario14 — the same problem solved cleanly.

Fix: As shown — and the pattern is so common it deserves a permanent name in your team's style guide.

Why this works: Window-based ranking is partition-aware, tie-deterministic when you supply a secondary key, and runs in a single pass over the table.

What NOT to say: "ROW_NUMBER is slower than MAX." It is usually the same speed or faster — both require a sort by partition key.

Common follow-up: "What if I need the top-3 per candidate, not just top-1?" Same pattern, filter `WHERE rn <= 3`.

Scenario70 · DISTINCT ON (Postgres) — Clean Alternative

Tags: distinct-on, postgres, latest-per-group, dialect

Scenario: A Postgres-shop analyst at a Mumbai-based brokerage uses the ROW_NUMBER pattern and a senior reviewer asks why they didn't use `DISTINCT ON`. The reviewer claims it's faster and shorter.

SQL:

```
SELECT DISTINCT ON (candidate_id) *  
FROM applications  
ORDER BY candidate_id, applied_at DESC, application_id DESC;
```

Expected vs Actual: Expected one row per candidate, latest application. Actual: exactly that — and the query is half the lines of the ROW_NUMBER version.

Thinking out loud: 1. DISTINCT ON is Postgres-only syntax. The columns in DISTINCT ON define the group; the ORDER BY decides which row wins. 2. ORDER BY must start with the DISTINCT ON columns, then add the ordering for picking the winner. 3. It's not faster than ROW_NUMBER in all cases — Postgres can use the same index either way — but the SQL is shorter and reads better. 4. Not portable to MySQL, SQL Server, BigQuery.

Diagnosis: This isn't a bug — it's a dialect comparison. Pick based on portability needs.

Fix: Already shown. If portability matters, stick with ROW_NUMBER. If you're Postgres-only, DISTINCT ON is idiomatic.

Why this works: DISTINCT ON is purpose-built syntactic sugar for the "latest per group" question.

What NOT to say: "DISTINCT ON is always faster than window functions." Same plan in most cases; readability is the real win.

Common follow-up: "What's the BigQuery / Snowflake equivalent?" QUALIFY with ROW_NUMBER — Scenario9's pattern.

Scenario71 · Aggregating After JOIN Inflates Totals

Tags: join-multiplication, aggregation, line-items

Scenario: A Chennai-based retail chain reports a sudden 4x jump in revenue. Analyst is summing order totals after joining with line items.

SQL:

```
SELECT SUM(o.total_amount) AS revenue
FROM orders o
JOIN order_items i ON i.order_id = o.order_id
WHERE o.order_date = '2026-05-01';
```

Expected vs Actual: Expected total revenue for May 1st. Actual: revenue inflated by average items-per-order (4x in this case).

Thinking out loud: 1. The JOIN to order_items multiplies each order row by its line-item count. 2. SUM(o.total_amount) then counts each order's total once per line item. 3. The fix is to aggregate before joining, or to sum at the order grain without joining. 4. Classic "fan-out" join-multiplication bug.

Diagnosis: Joining a 1:N table inflates 1-side aggregates by the N count.

Fix (multiple options):

```
-- Option 1: Don't join if you don't need item details
SELECT SUM(total_amount) AS revenue
FROM orders
WHERE order_date = '2026-05-01';

-- Option 2: Aggregate before joining if you do need items
SELECT o.order_id, o.total_amount, agg.item_count
FROM orders o
JOIN (
    SELECT order_id, COUNT(*) AS item_count FROM order_items GROUP BY order_id
) agg ON agg.order_id = o.order_id
WHERE o.order_date = '2026-05-01';

-- Option 3: Sum the line items directly
SELECT SUM(i.quantity * i.unit_price) AS revenue
FROM order_items i
JOIN orders o USING (order_id)
WHERE o.order_date = '2026-05-01';
```

Why this works: Each option ensures we sum at the correct grain — order-level or item-level, never mixed.

What NOT to say: "Just SELECT DISTINCT before SUM." DISTINCT on order_id-amount might work but breaks if two orders have the same amount.

Common follow-up: "What if you need both order count and item count in one query?" Two subqueries joined back, or window-COUNT, but never naively SUM after JOIN.

Scenario72 · Aggregate of Aggregate — Needs Subquery

Tags: aggregate-of-aggregate, subquery, window-function

Scenario: A Bengaluru SaaS team wants "average number of orders per customer" — a single number. Engineer tries `AVG(COUNT(order_id))` and the engine throws.

SQL (broken):

```
SELECT AVG(COUNT(order_id))
FROM orders
GROUP BY customer_id;
```

Expected vs Actual: Expected a single average. Actual: most engines reject nested aggregates without a window or subquery.

Thinking out loud: 1. You can't nest two aggregate functions directly — SQL needs you to first aggregate, then aggregate again. 2. Subquery is the standard fix: GROUP BY customer_id inside, AVG

outside. 3. Window function alternative: COUNT per customer using OVER, then AVG. 4. Some engines allow `AVG(COUNT(...)) OVER ()` as a workaround — non-portable.

Diagnosis: Aggregates can't be nested without an intermediate query level.

Fix:

```
-- Standard subquery
SELECT AVG(order_count) AS avg_orders_per_customer
FROM (
  SELECT customer_id, COUNT(order_id) AS order_count
  FROM orders
  GROUP BY customer_id
) t;
```

Why this works: The inner query produces one row per customer with an order count; the outer averages those counts.

What NOT to say: "Use a window function for everything." Window functions don't collapse rows — you'd still need a final aggregation step.

Common follow-up: "What if you want it per region?" Wrap the per-customer count, then GROUP BY region in the outer query.

Scenario73 · COUNT in Correlated Subquery — Slow

Tags: correlated-subquery, performance, join-rewrite

Scenario: A Hyderabad-based analytics team's nightly job has a query that times out at 4 hours. The query counts, for each product, how many orders contain that product.

SQL (slow):

```
SELECT
  p.product_id,
  p.name,
  (SELECT COUNT(*) FROM order_items i WHERE i.product_id = p.product_id) AS order_count
FROM products p;
```

Expected vs Actual: Expected completion in minutes. Actual: 4-hour timeout because the subquery executes once per product row (50k products × full scan).

Thinking out loud: 1. Correlated subqueries that touch a large fact table are $O(\text{rows_outer} \times \text{cost_inner})$. 2. Rewrite as a join + GROUP BY for a single scan. 3. Modern optimisers sometimes flatten correlated subqueries — but don't rely on it. 4. LEFT JOIN preserves products with zero orders.

Diagnosis: Correlated subquery forces per-row re-execution.

Fix:

```
SELECT
  p.product_id, p.name,
  COALESCE(agg.order_count, 0) AS order_count
FROM products p
LEFT JOIN (
  SELECT product_id, COUNT(*) AS order_count
  FROM order_items
  GROUP BY product_id
) agg ON agg.product_id = p.product_id;
```

Why this works: One pass over order_items aggregates everything; a hash join then attaches counts to products.

What NOT to say: "Add an index on product_id." The index helps the correlated subquery only marginally — the real win is single-pass aggregation.

Common follow-up: "When is a correlated subquery acceptable?" When the outer is small (a dozen rows) or when it's an EXISTS / NOT EXISTS that short-circuits.

Scenario74 · GROUP BY on Expression — Index Issue

Tags: group-by, expression, function-on-column, sargability

Scenario: A Pune-based logistics analyst groups shipments by month using `MONTH(shipment_date)`. The query scans the full table even though there's an index on shipment_date.

SQL:

```
SELECT
  MONTH(shipment_date) AS month,
  COUNT(*) AS shipments
FROM shipments
WHERE YEAR(shipment_date) = 2026
GROUP BY MONTH(shipment_date);
```

Expected vs Actual: Expected index seek on shipment_date. Actual: full table scan because applying a function to the indexed column makes it non-sargable.

Thinking out loud: 1. Function on an indexed column kills index usage on most engines. 2. Rewrite the filter to a range that uses the raw column. 3. GROUP BY on expression is fine — it's the WHERE that hurts. 4. Some engines (Postgres) support expression indexes if grouping is frequent.

Diagnosis: Function-wrapped column predicates can't use B-tree indexes.

Fix:

```
SELECT
  DATE_TRUNC('month', shipment_date) AS month,
  COUNT(*) AS shipments
FROM shipments
WHERE shipment_date >= '2026-01-01' AND shipment_date < '2027-01-01'
GROUP BY DATE_TRUNC('month', shipment_date);
```

Why this works: Range predicate on the raw column uses the index; DATE_TRUNC in SELECT/GROUP BY is computed after seek.

What NOT to say: "Indexes aren't useful for GROUP BY anyway." Index-organised reads massively help when filtering before grouping.

Common follow-up: "What if I really need MONTH() in the WHERE?" Create an expression index:

`CREATE INDEX ix_ship_month ON shipments((MONTH(shipment_date)))` — Postgres syntax, slightly different on each engine.

Scenario75 · DATE_TRUNC Missing Timezone

Tags: date-trunc, timezone, aggregation, edge-of-day

Scenario: A Mumbai-based fintech reports daily transaction volumes. Volumes look about 5% lower than expected for some Indian customers, with late-night transactions appearing on the *previous* day.

SQL:

```
SELECT
  DATE_TRUNC('day', created_at) AS txn_day,
  COUNT(*) AS txns
FROM transactions
GROUP BY 1;
```

Expected vs Actual: Expected IST-day grouping. Actual: UTC-day grouping because `created_at` is stored as UTC and DATE_TRUNC operates without conversion.

Thinking out loud: 1. UTC and IST differ by 5h30m. A 2am IST transaction is 8:30pm UTC the previous day. 2. DATE_TRUNC on a UTC timestamp gives a UTC day. 3. Convert to IST before truncating:

`DATE_TRUNC('day', created_at AT TIME ZONE 'Asia/Kolkata')` in Postgres. 4. Other engines: BigQuery has `DATE(created_at, 'Asia/Kolkata')`; Snowflake has `CONVERT_TIMEZONE`.

Diagnosis: Truncating without converting timezone aligns days to UTC, not the user's local boundary.

Fix:

```
-- Postgres
SELECT
  DATE_TRUNC('day', created_at AT TIME ZONE 'Asia/Kolkata') AS txn_day,
  COUNT(*) AS txns
FROM transactions
GROUP BY 1;

-- BigQuery
SELECT
  DATE(created_at, 'Asia/Kolkata') AS txn_day,
  COUNT(*) AS txns
FROM transactions
GROUP BY 1;

-- Snowflake
SELECT
  DATE_TRUNC('DAY', CONVERT_TIMEZONE('UTC', 'Asia/Kolkata', created_at)) AS txn_day,
  COUNT(*) AS txns
FROM transactions
GROUP BY 1;
```

Why this works: Each engine has a way to anchor the truncation to the target timezone's day boundary.

What NOT to say: "Just store everything in IST." That breaks when you scale to multiple timezones — keep storage in UTC, convert at query time.

Common follow-up: "What about daylight saving?" India doesn't observe DST, so IST is fine. But for global businesses, prefer named timezones over fixed offsets.

Scenario76 · Active Users — At Least One Event vs Total Events

Tags: active-users, distinct-vs-count, semantics

Scenario: A Bengaluru SaaS team reports 8.4 lakh "active users this month" but their MAU on the investor pitch deck is 2.1 lakh. Both numbers come from the same database.

SQL (wrong):

```
SELECT COUNT(*) AS active_users
FROM events
WHERE event_date >= '2026-05-01';
```

Expected vs Actual: Expected count of *distinct users* with at least one event. Actual: count of events, which is much larger because heavy users contribute many rows.

Thinking out loud: 1. "Active users" means distinct users; the analyst counted events. 2. COUNT(DISTINCT user_id) gives the right number. 3. If the table is huge, approximate variants are fine (Scenario4). 4. Always sanity-check that $\text{COUNT(*)} \leq \text{COUNT(DISTINCT user_id)}$ — and they should differ a lot for engaged products.

Diagnosis: Confused "active user count" with "event count".

Fix:

```
SELECT COUNT(DISTINCT user_id) AS active_users
FROM events
WHERE event_date >= '2026-05-01';
```

Why this works: DISTINCT collapses repeat events from the same user into one count.

What NOT to say: "It's the same as long as users are roughly equal in activity." They're never equal — power users dominate event volume.

Common follow-up: "How do you do retention then?" Need user-cohort tables — distinct users per cohort × event-window, joined and counted carefully.

Scenario77 · Cumulative DISTINCT Users — No Native Window

Tags: cumulative-distinct, window-functions, workaround

Scenario: A Chennai-based growth team wants "cumulative unique users by day" — Day 1: users who used the app on Day 1. Day 2: users who used the app on Day 1 or Day 2. And so on. They try `COUNT(DISTINCT user_id) OVER (ORDER BY day)` and get an error.

SQL (doesn't work):

```
SELECT
  day,
  COUNT(DISTINCT user_id) OVER (ORDER BY day ROWS UNBOUNDED PRECEDING) AS cumulative_users
FROM daily_active;
```

Expected vs Actual: Expected cumulative distinct count. Actual: most engines reject COUNT(DISTINCT) inside OVER.

Thinking out loud: 1. SQL standard doesn't define COUNT(DISTINCT) as a window function. Some engines (BigQuery, Postgres 16+) support it with caveats. 2. Workaround: find the *first day* each user

appeared (MIN per user), then count users whose first day $\leq N$. 3. Equivalent to "cumulative new users" — the cumulative distinct equals the running sum of new users. 4. This is the standard pattern.

Diagnosis: COUNT(DISTINCT) is not generally supported in OVER clauses.

Fix:

```
WITH first_seen AS (  
    SELECT user_id, MIN(event_date) AS first_day  
    FROM events  
    GROUP BY user_id  
) ,  
new_per_day AS (  
    SELECT first_day AS day, COUNT(*) AS new_users  
    FROM first_seen  
    GROUP BY first_day  
)  
SELECT  
    day,  
    SUM(new_users) OVER (ORDER BY day ROWS UNBOUNDED PRECEDING) AS cumulative_users  
FROM new_per_day;
```

Why this works: Each user contributes exactly once (their first appearance); the running sum gives cumulative distinct.

What NOT to say: "Cross-join every day with users and count." That's $O(\text{days} \times \text{users})$ which explodes on real data.

Common follow-up: "What if I want cumulative distinct over a sliding window, not cumulative-forever?" Use HyperLogLog sketches per day, then UNION-merge across the window.

Scenario78 · Mode / Most Frequent Value

Tags: mode, most-frequent, group-by

Scenario: An Indore-based analyst is asked for the "most common payment method per merchant" in an interview. They try `MODE(payment_method)` and the engine throws.

SQL (engine-dependent):

```
SELECT  
    merchant_id,  
    MODE() WITHIN GROUP (ORDER BY payment_method) AS top_method  
FROM transactions  
GROUP BY merchant_id;
```

Expected vs Actual: Expected most-frequent method per merchant. Actual: works in Postgres and Oracle; throws in MySQL, BigQuery, Snowflake.

Thinking out loud: 1. MODE WITHIN GROUP is Postgres/Oracle. Not standard. 2. The portable pattern is GROUP BY + COUNT + ROW_NUMBER. 3. Tiebreaker logic must be explicit if two methods tie. 4. APPROX_TOP_K (BigQuery, Snowflake) gives an approximate version.

Diagnosis: MODE is dialect-specific; portable code needs the window pattern.

Fix:

```
WITH counts AS (  
    SELECT merchant_id, payment_method, COUNT(*) AS c  
    FROM transactions  
    GROUP BY merchant_id, payment_method  
)  
,  
ranked AS (  
    SELECT *,  
        ROW_NUMBER() OVER (PARTITION BY merchant_id ORDER BY c DESC, payment_method) AS rn  
    FROM counts  
)  
SELECT merchant_id, payment_method AS top_method  
FROM ranked  
WHERE rn = 1;
```

Why this works: Count occurrences per (merchant, method), rank within merchant by count, pick rank 1.

What NOT to say: "Use MAX(COUNT(*))." That gives the count value, not the method — and joining back to retrieve the method is more code than the window pattern.

Common follow-up: "What about top 3 methods per merchant?" Same pattern, filter

`WHERE rn <= 3`.

Scenario79 · Sessionization Gone Wrong — Wrong Gap Threshold

Tags: sessionization, sessions, gap-analysis, window-functions

Scenario: A Mumbai content platform defines a "session" as page views within 30 minutes of the previous view. The sessionization query gives suspiciously long sessions (some lasting 14 hours).

SQL:

```

WITH gaps AS (
  SELECT
    user_id, page_view_at,
    EXTRACT(EPOCH FROM page_view_at - LAG(page_view_at) OVER (PARTITION BY user_id ORDER BY page_v
  FROM page_views
),
flagged AS (
  SELECT *,
    CASE WHEN gap_sec > 1800 OR gap_sec IS NULL THEN 1 ELSE 0 END AS is_new_session
  FROM gaps
)
SELECT user_id, page_view_at,
  SUM(is_new_session) OVER (PARTITION BY user_id ORDER BY page_view_at) AS session_num
FROM flagged;

```

Expected vs Actual: Expected new session after 30-minute gap. Actual: code looks right but engineers misread the threshold — they wrote 18000 (5 hours) instead of 1800 (30 min) in production rollout.

Thinking out loud: 1. The pattern is correct: LAG for previous timestamp, flag new sessions on gap > threshold, running sum. 2. Bug is the threshold value — 18000 seconds (5 hours), not 1800 (30 minutes). 3. Always express thresholds via named constants or CTEs: `WITH config AS (SELECT 1800 AS session_gap_sec)`. 4. Worth defining in code review.

Diagnosis: Magic number off by a factor of 10 — classic.

Fix:

```

WITH config AS (SELECT 30 * 60 AS session_gap_sec), -- 30 minutes
gaps AS (
  SELECT
    user_id, page_view_at,
    EXTRACT(EPOCH FROM page_view_at - LAG(page_view_at) OVER (PARTITION BY user_id ORDER BY page_v
  FROM page_views
),
flagged AS (
  SELECT g.*,
    CASE WHEN g.gap_sec > c.session_gap_sec OR g.gap_sec IS NULL THEN 1 ELSE 0 END AS is_new_sessi
  FROM gaps g CROSS JOIN config c
)
SELECT user_id, page_view_at,
  SUM(is_new_session) OVER (PARTITION BY user_id ORDER BY page_view_at) AS session_num
FROM flagged;

```

Why this works: Named threshold via CTE avoids magic-number bugs and makes the intent reviewable.

What NOT to say: "Sessionization is too complex for SQL." It is one of SQL's strongest use cases when window functions are used well.

Common follow-up: "How would you compute session duration?" `MAX(page_view_at) - MIN(page_view_at)` per (user_id, session_num).

Scenario80 · Recency Aggregation — Off-by-One Sliding Window

Tags: sliding-window, last-n-days, off-by-one, recency

Scenario: A Bengaluru-based ride-hailing app reports "rides in the last 7 days" but the count includes 8 days when reviewed against business records. The analyst's filter looks reasonable.

SQL:

```
SELECT COUNT(*) AS rides_last_7
FROM rides
WHERE ride_at >= CURRENT_DATE - 7
      AND ride_at <= CURRENT_DATE;
```

Expected vs Actual: Expected 7 days of rides. Actual: 8 days because the inclusive range spans `today - 7` through `today`, which is 8 distinct calendar days.

Thinking out loud: 1. `CURRENT_DATE - 7` is exactly 7 days before today. 2. Range from `today - 7` to `today` is 8 days inclusive (today-7, today-6, ..., today). 3. For "last 7 days including today", you want `>= today - 6` or `> today - 7`. 4. Also worry about timestamps vs dates — `ride_at` may be a timestamp and the cast matters.

Diagnosis: Off-by-one in inclusive date ranges — most candidates hit this once.

Fix:

```
-- Last 7 days including today (today, today-1, ..., today-6)
SELECT COUNT(*) AS rides_last_7
FROM rides
WHERE ride_at >= CURRENT_DATE - INTERVAL '6 days'
      AND ride_at < CURRENT_DATE + INTERVAL '1 day';

-- Or, explicit windowed: last 7 complete days, excluding today
SELECT COUNT(*) AS rides_last_7
FROM rides
WHERE ride_at >= CURRENT_DATE - INTERVAL '7 days'
      AND ride_at < CURRENT_DATE;
```

Why this works: Explicit half-open intervals `[start, end)` are the safest way to avoid off-by-one — and `< CURRENT_DATE + 1 day` handles the timestamp-vs-date trap.

What NOT to say: "Just use BETWEEN." BETWEEN is inclusive on both ends, which is exactly the trap — and worse with timestamps where `BETWEEN '2026-05-19' AND '2026-05-26'` excludes the afternoon of May 26.

Common follow-up: "What about timezone for 'today'?" `CURRENT_DATE` returns server-default timezone — use `(NOW() AT TIME ZONE 'Asia/Kolkata')::date` for IST-correct day.

Category 2, Part A — Optimize This Query: Index-Based Fixes

"This query works but takes 8 minutes" is the most universal interview prompt in the optimization category — every product company in Bengaluru, Hyderabad, Pune and the GCC belt opens the second round with some version of it. Part A of this category covers the 27 scenarios where the right answer is the right index — adding a missing one, reordering an existing composite, switching from B-tree to GIN or BRIN, dropping a redundant cousin, or building a partial index for a hot subset. Parts B and C go further: B handles query rewrites (CTEs, EXISTS vs IN, window-function replacements), and C handles plan-driven fixes that need EXPLAIN ANALYZE before you touch anything. Read them as a trilogy — interviewers love jumping between the three.

Scenario81 · Add Missing Single-Column Index — The Basic Case

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Zoho · Dialect: ANSI / All

Scenario setup: A support engineer at a logistics startup says "our admin dashboard takes 12 seconds to load a single user's order history." The orders table has 40 million rows. The query is straightforward — fetch all orders for one `user_id`. There's a primary key on `id`, but no index on `user_id`.

```
SELECT id, status, amount, created_at
FROM orders
WHERE user_id = 1487291
ORDER BY created_at DESC;
```

Current perf vs target: 12s full table scan reading 40M rows; target is sub-100ms for a UI call.

Thinking out loud: 1. The WHERE clause filters on a column with no index — that forces a sequential scan of every row in the orders table. Even on a fast SSD with cold cache, that's 12 seconds at this volume; on a warm cache it's still 4-5 seconds of CPU walking pages. 2. `user_id` is high cardinality (one row per user is rare, most have 5-50 orders), so an index will narrow the scan from 40M rows to roughly 20-50 rows. The index does the heavy filtering, the engine just visits the matching heap pages. 3. Confirm the access pattern is read-heavy and that `user_id` doesn't change after insert — both true for `user_id` on orders, so the write-amplification cost of an index is negligible. The index pages will be appended at the `user_id` leaf each time a new order is placed; no page splits in unpredictable places. 4.

Before recommending the index, ask: is this the only query that filters by `user_id`, or do we also filter by `user_id` + `status` combined? If combined, a composite is better — covered in Scenario2. Don't add a single-column index that will be made redundant by next sprint's composite.

The diagnosis: Missing index on `orders.user_id` — every dashboard load triggers a full table scan of 40M rows.

The fix:

```
CREATE INDEX idx_orders_user_id ON orders(user_id);  
-- Postgres: prefer CREATE INDEX CONCURRENTLY on a live table to avoid table locks  
CREATE INDEX CONCURRENTLY idx_orders_user_id ON orders(user_id);
```

Why this works: The database can now do a single B-tree lookup to find the leaf pages for that `user_id`, then follow pointers to the actual heap rows. Cost drops from $O(N)$ sequential scan to $O(\log N)$ tree traversal plus a small number of random heap fetches. For 40M rows, the tree depth is roughly 4 levels — five page reads to find any `user_id`. The `ORDER BY` on `created_at` still requires a sort, but on 20 rows that sort is essentially free — it happens in memory in microseconds. Scenario8 covers when the sort itself becomes the bottleneck and needs to be absorbed into the index.

What NOT to say: "Add an index on every column in the `WHERE` clause" — that's how you end up with 30 single-column indexes on one table and a write performance catastrophe. Index decisions are query-pattern decisions, not column-level decisions. Also avoid saying "an index always speeds up reads" — on tiny tables it's slower than a full scan because the planner has to descend the tree.

Common follow-up: "*What's the write penalty?*" — Roughly 10-15% slower `INSERT/UPDATE` on `user_id`, because the index has to be maintained on every row insert and on any `UPDATE` that touches `user_id` (rare). For a read-heavy admin path on an insert-heavy table, this is still net positive. If writes are the bottleneck, batch your inserts and use unlogged or partitioned tables to amortize the index-maintenance cost.

Scenario82 · Composite Index — Order Matters (Leftmost-Prefix)

Tags: Difficulty: Medium · Asked at: Flipkart, Myntra, Swiggy, GCC banks · Dialect: ANSI / MySQL / Postgres

Scenario setup: A reporting query pulls all "delivered" orders for a specific user in a date range. There's an index on (`status`, `user_id`, `created_at`) already, but the query still does a full scan. The team is confused — they "have an index" but it's not being used.

```
SELECT id, amount, created_at
FROM orders
WHERE user_id = 1487291
      AND status = 'delivered'
      AND created_at >= '2026-01-01';
```

Current perf vs target: 9s scan despite existing 3-column index; target sub-200ms.

Thinking out loud: 1. Composite indexes are sorted by the leading column first, then the next, then the next — think of it like a phone book sorted by surname, then first name, then street. The index on (status, user_id, created_at) is sorted by status first, so the tree only narrows by status before any other dimension. 2. The query filters on user_id alongside status — but if the leading column doesn't carry weight, the optimizer either does a partial scan or skips the index entirely. With 4 status values, the leading column narrows the search to only 25% of the table; user_id then has to filter linearly within that. 3. The leftmost-prefix rule (sometimes called the "left-to-right rule"): an index on (A, B, C) is useful for queries filtering on A, or A+B, or A+B+C — never B alone, never C alone, never A+C without B. The optimizer can use any prefix that begins at the left edge. 4. The fix is reordering: lead with the most selective equality column. user_id is most selective (millions of values), status next (4 values), created_at is a range so it must come last. Equality columns first, then range columns last is the standard composite-index recipe.

The diagnosis: Composite index column order is wrong — leading column is low-cardinality status (4 values), not the high-cardinality user_id (millions of values) the query actually filters on. The index can't seek efficiently to a user_id without first scanning the status slice.

The fix:

```
DROP INDEX idx_orders_status_user_created;
CREATE INDEX idx_orders_user_status_created
  ON orders(user_id, status, created_at);
```

Why this works: Now the index can seek to the exact (user_id, status) prefix in one descent, then walk forward through created_at in already-sorted order. The range filter on created_at piggybacks on the sort order — no extra scan needed because the rows are already ordered by created_at within each (user_id, status) group. From a tree-depth perspective, lookup goes from "find status slice, scan for user_id within it" to "descend directly to user_id, descend to status, walk created_at."

What NOT to say: "The optimizer should figure it out" — it can't. The B-tree structure is physical; you can't skip leading columns without scanning every value of them. Some engines call this trick "loose index scan" or "skip scan" — Oracle has it, MySQL 8 has limited support, Postgres doesn't — but it's slower than a properly-ordered composite in every benchmark.

Common follow-up: "Why not put created_at first since it's most selective?" — Range columns belong last. A range on the leading column stops the index from being useful for any equality filter after it, because the tree fans out over the range instead of pinning a single value. Equality columns first preserve

the ability to seek down to a precise point in the tree; range columns at the end let you walk a contiguous slice from that point.

Scenario83 · Reorder Columns in Existing Composite for New Query Pattern

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Cred, Walmart Labs · Dialect: ANSI / All

Scenario setup: A payments table has a composite index on (merchant_id, captured_at) that serves the merchant dashboard well. A new analytics query was added last sprint that filters only by captured_at across all merchants — and it's not using the index. The team wants to add a second index but is worried about write cost.

```
SELECT merchant_id, SUM(amount) AS daily_volume
FROM payments
WHERE captured_at >= '2026-05-01'
GROUP BY merchant_id;
```

Current perf vs target: 45s scan of 200M-row payments table; target sub-5s.

Thinking out loud: 1. The existing index leads with merchant_id, so it's useless for a query that doesn't filter by merchant_id — exactly the leftmost-prefix problem from Scenario2 in reverse. The analytics query has no merchant_id in its WHERE, so the optimizer falls back to a sequential scan of all 200M rows. 2. Option A: add a second index on (captured_at). This doubles the write penalty on this hot table — every payment insert now updates two indexes instead of one. 3. Option B: see if the existing index can be reordered to (captured_at, merchant_id). But then the merchant dashboard query breaks because it expects merchant_id as the leading column for its per-merchant lookups. Reordering helps one query at the cost of the other. 4. The right move is two indexes — but only if both query patterns are truly hot. Profile both before adding. If the analytics query runs once a day at 2 AM, you might tolerate the slow scan and skip the second index. If it runs every five minutes from a dashboard, the second index is justified.

The diagnosis: Existing composite cannot serve a query that skips its leading column. Need either a second index or a redesign of the access pattern. The choice depends on query frequency and write budget.

The fix:

```
CREATE INDEX idx_payments_captured_at_merchant
ON payments(captured_at, merchant_id);
```

Why this works: The new index leads with captured_at (the range filter), so the engine can seek to May 1, 2026 and walk forward sequentially through the leaf pages until the range ends. Including merchant_id as the second column lets it do an index-only scan for the GROUP BY without touching the heap — the

planner reads just the index leaves, groups by merchant_id, and sums amount from the same pages. Covered more in Scenario4 on covering indexes.

What NOT to say: "Indexes are free, just add one for every query" — they cost write throughput, storage, and cache pressure. The honest answer is "two indexes for two access patterns, and we'll monitor write latency for the next week to confirm the trade-off is worth it." Also avoid "we'll add a materialized view" without checking whether it's truly cheaper than the index — materialized views have their own refresh overhead.

Common follow-up: "*What if writes get slower?*" — Drop the lower-value index. On a payments table, captured_at is essentially append-only, so index-maintenance cost is low — the page splits happen at the tail end of the B-tree and don't fragment older pages. Monitor with `pg_stat_user_indexes` to confirm the new index is actually being scanned; if it's not, drop it after a week.

Scenario84 · Covering Index — INCLUDE / Index-Only Scan

Tags: Difficulty: Medium · Asked at: Microsoft, Postgres-heavy startups, Atlassian · Dialect: Postgres 11+ / SQL Server / MySQL 8

Scenario setup: A KYC verification page reads a small slice of columns from a wide users table — just email, kyc_status, and last_login_at — for users in a specific city. There's an index on city, but the query still does heap lookups for every matching row, and the page takes 4s.

```
SELECT email, kyc_status, last_login_at
FROM users
WHERE city = 'Bengaluru'
      AND kyc_status = 'verified';
```

Current perf vs target: 4s with heap fetches for 200k rows; target sub-500ms.

Thinking out loud: 1. The current index on city does the filtering but only returns row pointers — the engine still has to visit the heap (the actual table pages) to read email, kyc_status, and last_login_at. For 200k matching rows, that's 200k random I/O operations against the heap, each potentially a cache miss. 2. If we include all three columns in the index itself, the engine can answer the query entirely from the index — no heap visit at all. Postgres calls this an "index-only scan"; SQL Server calls it a "covering index"; same concept, different vocabulary. 3. The trick: use INCLUDE for the non-key columns so they don't bloat the B-tree key, but are still available for the result. INCLUDE columns are stored in leaf pages only, meaning they don't affect tree depth or fanout — they just ride along. 4. Sanity-check: are these columns mostly stable? last_login_at updates often, so including it means every login triggers an index update — that may swing the trade-off. If last_login_at gets updated 1000 times per second across the user base, the index churn might cost more than the read savings. Profile both sides.

The diagnosis: Index returns row pointers; engine then does 200k random heap reads to fetch the actual column values. A covering index eliminates the heap visit entirely by stuffing the needed columns into the index leaves.

The fix:

```
-- Postgres 11+ / SQL Server
CREATE INDEX idx_users_city_kyc_covering
  ON users(city, kyc_status)
  INCLUDE (email, last_login_at);

-- For MySQL 8, add columns to the index key (no INCLUDE)
CREATE INDEX idx_users_city_kyc_email
  ON users(city, kyc_status, email, last_login_at);
```

Why this works: The leaf pages of the index now contain everything the query needs — city and kyc_status for filtering, email and last_login_at for output. The planner sees this and uses an "index-only scan" — zero random I/O against the heap. For a wide table where the heap row is 2KB and the index entry is 200 bytes, you read 10x less data. In cache-cold scenarios the difference is even more dramatic because random heap reads thrash the buffer pool.

What NOT to say: "Just add all columns to the index key" — that bloats the B-tree, increases write cost, and slows down lookups by reducing fanout. INCLUDE columns sit in leaf pages only, which is the whole point. Also avoid suggesting "make every index covering" — coverage is per-query; an index that covers query A may force heap visits for query B, and you can't cover everything.

Common follow-up: *"What if MySQL doesn't have INCLUDE?"* — MySQL 8 has it via separate functional indexes; older MySQL forces you to add the columns to the index key itself, which bloats the B-tree. Calling it out shows you know the dialect difference. The Postgres + SQL Server INCLUDE clause is genuinely cleaner because it preserves the small key while still enabling index-only scans.

Scenario85 · Partial Index for Hot Filter Values

Tags: Difficulty: Medium · Asked at: Zerodha, Postgres-heavy fintechs, Stripe-style teams · Dialect: Postgres / SQL Server

Scenario setup: A transactions table has 500M rows total but only ~50k rows at any moment have status = 'pending'. The payment processor polls "pending" rows every second to retry them. A full index on status is bloated and ineffective because 99% of rows are 'success' or 'failed'.

```
SELECT id, amount, retry_count
FROM transactions
WHERE status = 'pending'
ORDER BY created_at
LIMIT 100;
```

Current perf vs target: 800ms per poll with full status index; target sub-50ms.

Thinking out loud: 1. A B-tree on status with 500M entries pointing to 'success' is pointless — the optimizer would skip the index anyway because the cardinality is poor. With only 3-4 status values, each value matches such a huge chunk of the table that a full scan is cheaper than the index seek + heap reads. 2. The hot subset is tiny — only the small slice of 'pending' rows matters for the polling workload. A partial index that excludes the rest is essentially a 50k-row mini-table glued onto the side of the main table. 3. The polling query reads only pending rows, so a partial index `WHERE status = 'pending'` covers 100% of its workload while skipping 99% of the data. The index can include other columns (like `created_at`) sorted in poll-order for free. 4. Bonus: the partial index is small enough to live in RAM, so polls become memory-resident reads — a huge latency win and a near-zero cache-eviction penalty for the rest of the database.

The diagnosis: Full index on status is bloated and selectivity is poor for the only column value the application actually queries. A partial index for the hot value is dramatically smaller and faster, and is automatically maintained as rows enter and exit the 'pending' state.

The fix:

```
-- Postgres / SQL Server with filtered index syntax
CREATE INDEX idx_transactions_pending
  ON transactions(created_at)
  WHERE status = 'pending';
```

Why this works: The index now contains only ~50k entries — about 1MB on disk vs the 5GB the full status index would consume. Every poll hits a fully-cached index and walks the first 100 entries in `created_at` order. As rows transition out of 'pending' (via an UPDATE setting status to 'success' or 'failed'), the database automatically removes them from the partial index — no manual maintenance, no stale rows accumulating. The optimizer matches the query predicate `WHERE status = 'pending'` against the index predicate directly and picks it without prompting.

What NOT to say: "MySQL has partial indexes too" — it doesn't, in the same WHERE-clause-on-the-index sense. MySQL's `KEY (col(N))` is a different feature (indexing a prefix of a string column). If asked in a MySQL shop, propose an expression index, a separate hot-queue table, or a generated-column workaround. Calling MySQL's prefix-key feature a "partial index" in an interview is a credibility hit.

Common follow-up: "What if 'pending' grows to 5M rows?" — Still fine; the index stays proportional to the hot set. The real risk is if 'pending' becomes 50% of the table, at which point a partial index has no

advantage over a regular one — the predicate filters nothing meaningful. Another follow-up: "*What about INSERT cost?*" — Negligible, because rows enter the partial index only when status starts as 'pending', and exit when status changes. Most inserts probably already start as 'pending', so it's one index update either way.

Scenario86 · Expression / Functional Index for WHERE UPPER(col)

Tags: Difficulty: Medium · Asked at: Oracle shops, BFSI GCCs, Postgres startups · Dialect: Postgres / Oracle / MySQL 8

Scenario setup: A legacy user-lookup query uppercases the email before comparing, because the column has mixed case from years of inconsistent data entry. There's an index on email, but it's never used — the optimizer can't match an index on `email` against a predicate on `UPPER(email)`.

```
SELECT id, name, signup_at
FROM users
WHERE UPPER(email) = 'RAVI@EXAMPLE.IN';
```

Current perf vs target: 6s full scan of 20M users; target sub-50ms.

Thinking out loud: 1. Wrapping a column in a function disables any plain index on the underlying column — the optimizer has no way to know `UPPER(x)` is monotonic or even deterministic, so it can't translate `UPPER(email) = 'X'` into a B-tree seek against the raw email column. 2. Two paths: rewrite the query to compare without the function (problematic if the data really is mixed case and you can't trust normalization), or build an index on the expression itself. The expression-index path doesn't require any application change. 3. The expression index is the cleaner fix because it preserves the existing query and handles every variation of case the column might contain. The engine evaluates `UPPER(email)` once at insert time and stores the result in the index. 4. Side note: a better long-term fix is data cleanup — normalize email to lowercase at write time and add a CHECK constraint enforcing it — but that's a multi-week ticket touching the write path. The expression index is a 30-second deployment that buys you time.

The diagnosis: WHERE clause wraps the indexed column in `UPPER()`, making the plain index unusable. Need an index on the exact expression the query uses so the optimizer can match the predicate against the indexed expression.

The fix:

```
-- Postgres / Oracle
CREATE INDEX idx_users_email_upper ON users(UPPER(email));

-- MySQL 8+ via functional index (note the double parens for expression)
CREATE INDEX idx_users_email_upper ON users((UPPER(email)));
```


Why this works: The B-tree now stores the uppercased form, and the optimizer can match the query predicate `UPPER(email) = 'X'` directly against the index key. No function call per row, no scan — single seek down the tree, single heap fetch. The expression is computed once at write time and cached in the index leaves. Every read amortizes the one-time write cost across millions of lookups.

What NOT to say: "Just add `LOWER()` in the query and an index on `LOWER(email)`" — same problem if other queries still call `UPPER()`. You need the index to match the predicate the application actually writes. Also avoid "the optimizer should constant-fold `UPPER`" — it can fold constants, not column expressions, because the engine has no proof that `UPPER` is a pure function on every value in the column.

Common follow-up: "*What about `citext` in Postgres?*" — Cleaner option for new schemas: the `citext` type does case-insensitive comparison natively, no expression index needed, and the equality operator just works. But you can't retrofit it on a 20M-row column without a rewrite, a column-type change, and an index rebuild — usually not worth the migration risk on a working production table. The expression index is the practical answer for existing schemas.

Scenario87 · Index on FK Column — The One Everyone Forgets

Tags: Difficulty: Easy · Asked at: Every product company · Dialect: MySQL / Postgres / SQL Server

Scenario setup: A teammate complains that deleting a user takes 30 seconds because of the orders table's foreign key. The orders table has 80M rows. There's no index on `orders.user_id`, only the FK constraint declaration. (MySQL InnoDB auto-creates one, but Postgres does not — and the team is on Postgres.)

```
DELETE FROM users WHERE id = 99284;  
-- Cascades into orders.user_id check
```

Current perf vs target: 30s delete (full orders scan to check FK); target sub-100ms.

Thinking out loud: 1. FK constraint just defines the relationship — it doesn't auto-create an index in Postgres or SQL Server. MySQL InnoDB does auto-create one, which causes everyone to assume it's universal — and then they get burned the first time they touch a Postgres database. 2. Every `DELETE` on the parent triggers a scan of the child to verify no orphans exist (or to cascade the delete). Without an index on the FK column, that's a full table scan of the child for every parent-row delete. 3. Same issue on `UPDATE` if the PK changes (rare in modern schemas, but happens in legacy systems where the natural key was originally used). Same scan cost. 4. Adding the index is essentially mandatory for any FK column on a child table that's larger than a few thousand rows. The write-cost penalty is real but tiny compared to the delete-cost penalty without the index.

The diagnosis: Foreign key column has no index — every parent-row delete or PK update triggers a full scan of the child table to check referential integrity. This is one of the most common silent performance bugs in Postgres and SQL Server schemas.

The fix:

```
CREATE INDEX idx_orders_user_id ON orders(user_id);  
-- On a live large table, use CONCURRENTLY in Postgres to avoid table locks  
CREATE INDEX CONCURRENTLY idx_orders_user_id ON orders(user_id);
```

Why this works: The DELETE on users(id) now uses an index seek into orders to find dependent rows in $O(\log N)$ instead of $O(N)$. Cascading deletes (if configured) become proportional to the number of child rows that actually reference the parent, not the entire child table. For a parent with 5 children, the scan reads 5 index entries plus 5 heap rows — milliseconds, not minutes.

What NOT to say: "FK constraints are bad for performance, drop them" — that's how data integrity dies. The constraint is fine; the missing index is the bug. Also avoid "Postgres should auto-create the index" — that's a design choice (Oracle and Postgres deliberately don't, MySQL does); the database isn't broken, the schema author missed a step.

Common follow-up: "How do I find all FKs missing indexes?" — In Postgres, join `pg_constraint` against `pg_index` and filter for FK constraints whose columns aren't covered by any index. Most DBAs have a saved query for this and run it before every schema migration. SQL Server has `sys.foreign_keys` joined to `sys.indexes` for the same audit; MySQL exposes the relationship in `information_schema.KEY_COLUMN_USAGE` but the InnoDB auto-index usually makes this audit unnecessary there.

Scenario88 · Index for ORDER BY + LIMIT Pattern

Tags: Difficulty: Medium · Asked at: Swiggy, Zomato, Dunzo, Razorpay · Dialect: ANSI / All

Scenario setup: A "recent orders" widget on the user profile page fetches the latest 10 orders for a user. The table has an index on `user_id`, and the query filters by `user_id` and sorts by `created_at` DESC with LIMIT 10. But the engine reads all of that user's orders (could be thousands), sorts them, then takes 10 — slow for power users.

```
SELECT id, amount, status, created_at  
FROM orders  
WHERE user_id = 1487291  
ORDER BY created_at DESC  
LIMIT 10;
```

Current perf vs target: 1.2s for a heavy user (8k orders); target sub-50ms.

Thinking out loud: 1. The index on `user_id` finds the relevant rows in $O(\log N)$, but then everything inside that user's slice has to be sorted by `created_at` — that's an explicit sort step on whatever number of orders the user has. For a power user with 8,000 orders, that sort dominates the query time. 2. If the

index includes `created_at` as the second column, the rows are already in `created_at` order within each `user_id`, so the sort vanishes — just walk backwards from the latest entry. The index does both jobs at once. 3. The `LIMIT 10` then becomes a top-of-tree walk: descend to the `user_id`, jump to the last entry in that user's slice, take 10 going backwards, stop. Constant time regardless of how many orders the user has — 10 orders or 10 million orders is the same wall-clock latency. 4. Important: the index must order `created_at` the same direction the query asks for (or the engine must support backwards scan, which most do, but with sometimes-worse cost estimates). Building the index `DESC` removes the ambiguity.

The diagnosis: Sort is happening after the index lookup because the index doesn't preserve the `ORDER BY` direction. A composite index covering both filter and sort eliminates the sort entirely.

The fix:

```
CREATE INDEX idx_orders_user_created
ON orders(user_id, created_at DESC);
```

Why this works: The index physically stores rows grouped by `user_id`, and within each group ordered by `created_at` descending. The query descends to `user_id 1487291`, walks 10 entries from the head of that user's slice, and returns — no sort, no extra work. This is the single most common composite-index pattern in real-world product code: filter column first, sort column second, with sort direction matching the query. Almost every "recent X for user Y" widget uses exactly this shape.

What NOT to say: "Add a second index on `created_at` alone" — that doesn't help, because the filter on `user_id` would force the optimizer to choose between the two indexes; it can only use one for the bulk of the work. Index-merge plans exist but are rarely chosen and rarely faster than a properly-designed composite.

Common follow-up: "What if the user has only 5 orders?" — Same plan, even faster. The optimizer doesn't care about the row count under the `user_id`; the descent cost is $O(\log N)$ regardless. "What if the user has 5 million orders?" — Still constant time for the first 10, because `LIMIT` short-circuits the index walk. The composite shape scales gracefully across the full range of per-user activity.

Scenario89 · Index for `ORDER BY DESC + LIMIT` — Direction Matters

Tags: Difficulty: Medium · Asked at: PhonePe, Cred, fintech GCCs · Dialect: Postgres / MySQL 8

Scenario setup: A transaction-history API returns the latest 50 transactions across all users, sorted by `created_at DESC`. The team has an index on `created_at ASC`. The plan shows a sort step costing 80% of total time.

```
SELECT id, user_id, amount, created_at
FROM transactions
ORDER BY created_at DESC
LIMIT 50;
```

Current perf vs target: 2.1s with explicit sort; target sub-100ms.

Thinking out loud: 1. An index on `created_at` ASC can sometimes be walked backwards by the engine — Postgres supports this for all B-trees, MySQL supports it from 8.0 — but in older MySQL versions and in certain plan shapes, the optimizer will fall back to a full scan + sort because the cost estimate for the backwards walk comes out wrong. 2. Building the index in DESC order eliminates the ambiguity — the engine walks the leaf pages forward and gets the latest rows first. The plan choice is unambiguous and the cost estimate is straightforward. 3. Postgres has supported backwards index scan since forever, but the cost estimator sometimes picks a sequential scan + sort if statistics are stale or if the table is small enough that the planner thinks the full scan is cheaper. Explicit DESC removes the guesswork. 4. MySQL 8 finally supports descending indexes natively — before 8, the DESC keyword in an index definition was silently ignored, which led to years of confusion. If you're still on 5.7, you have to live with the ASC index and hope the backwards-scan plan gets picked.

The diagnosis: Index direction doesn't match query sort direction; engine either does extra work or picks a worse plan. An explicit DESC index aligns physical storage with query intent and removes the ambiguity from the cost estimate.

The fix:

```
-- MySQL 8+, Postgres, SQL Server
CREATE INDEX idx_transactions_created_desc
ON transactions(created_at DESC);
```

Why this works: The leaf pages are now physically ordered with the latest rows first. The LIMIT 50 walks the first 50 entries and stops — no sort step, no full scan. Plan goes from `Sort (cost=high)` to `Index Scan ... LIMIT 50` with predictable cost. On a 200M-row table the difference is the difference between 2 seconds and 2 milliseconds.

What NOT to say: "The optimizer always handles DESC" — true in Postgres, mostly true in modern MySQL, but on MySQL 5.7 the DESC was silently ignored on indexes for years and many production schemas still don't have explicit DESC indexes for that reason. Always test on your target version. Also avoid suggesting "the cost is the same" — backwards scans have slightly higher cost estimates in some planners and may lose to a sort + sequential scan if the table is small.

Common follow-up: "What if I sort by two columns, one ASC one DESC?" — Build a composite index with matching directions: `(col_a ASC, col_b DESC)`. MySQL 8 and Postgres handle this; older MySQL doesn't. Without per-column direction support, you have to pick one direction for the index and the other column always gets a sort step — usually you choose the leading column's direction to match the most common query.

Scenario90 · Index for Sort + Filter Combined

Tags: Difficulty: Medium · Asked at: Flipkart, Myntra, retail GCCs · Dialect: ANSI / All

Scenario setup: A product-listing page filters orders by warehouse_id and sorts by created_at DESC for an inventory dashboard. Indexes exist on warehouse_id (alone) and created_at (alone). The plan picks one, scans, sorts the result — slow at 200ms per page load.

```
SELECT id, sku, qty, created_at
FROM inventory_movements
WHERE warehouse_id = 17
ORDER BY created_at DESC
LIMIT 100;
```

Current perf vs target: 200ms with sort step; target sub-30ms.

Thinking out loud: 1. Two separate indexes can't be combined into a single sorted scan — the engine picks one and pays the cost of sorting (or filtering) the other dimension. The planner might use index-merge in some engines, but the merge step is expensive and rarely competitive with a proper composite. 2. The composite trick: leading column is the equality filter (warehouse_id), next column is the sort key (created_at DESC). Rows within each warehouse are stored already sorted by created_at descending, so the engine just walks the slice in order. 3. This is the same shape as Scenario8 — filter column first, sort column second with matching direction — and it's the pattern that comes up in every list-with-filter UI in production. Master it once and you'll use it weekly. 4. The leftmost column must be the equality (not the range/sort), otherwise the index can't seek directly to it. If you ever see someone put created_at first because "it's more selective," gently educate them on the leftmost-prefix rule.

The diagnosis: Two single-column indexes can't satisfy filter + sort together; the engine picks one and does extra work on the other dimension. A composite with filter-then-sort columns in matching direction does both in one walk.

The fix:

```
CREATE INDEX idx_inv_movements_warehouse_created
ON inventory_movements(warehouse_id, created_at DESC);

-- Optionally drop the now-redundant single-column index
DROP INDEX idx_inv_movements_warehouse;
```

Why this works: The descent finds warehouse_id = 17 in log time, lands on the slice already sorted by created_at DESC, and walks the first 100 entries — zero sort work, zero scan beyond what's returned. The single-column warehouse_id index is subsumed by the composite (the leftmost prefix rule means any query that used the old single-column index now uses the composite for free) — covered in Scenario11. Total scan is bounded by the LIMIT, not by the warehouse size.

What NOT to say: "The optimizer can index-merge" — it can, sometimes, but the merge step is expensive (bitmap-OR or sort-merge of row IDs) and rarely faster than a properly-designed composite. Real-world rule: prefer composites over relying on index-merge plans. Index-merge is a fallback the optimizer uses when nothing better is available; don't design indexes assuming the planner will rescue you with a merge.

Common follow-up: "*What if users sort by amount sometimes?*" — Then you need a second composite on (warehouse_id, amount DESC). Two indexes for two sort patterns is normal; just monitor write cost and drop one if it's not earning its keep. A third sort dimension (e.g., by sku name) usually starts the conversation about whether you need a different read path entirely — a search engine or a materialized denormalized view.

Scenario91 · Remove Redundant Index Subsumed by Composite

Tags: Difficulty: Easy · Asked at: DBA roles, senior backend rounds · Dialect: ANSI / All

Scenario setup: After adding the composite index from Scenario10, an old single-column index on warehouse_id still exists. A write-heavy ETL is now slower than expected. The DBA wants to know whether the single-column index can safely go.

```
-- Existing:
CREATE INDEX idx_inv_movements_warehouse ON inventory_movements(warehouse_id);
CREATE INDEX idx_inv_movements_warehouse_created
  ON inventory_movements(warehouse_id, created_at DESC);
```

Current perf vs target: ETL is 20% slower than before composite was added; target restoration to baseline.

Thinking out loud: 1. A composite index on (A, B) can satisfy any query that a single-column index on A could satisfy — leftmost-prefix rule applies to the read side too. The optimizer treats the composite as if its leftmost slice were a standalone index, so a lookup like `WHERE warehouse_id = 17` descends the composite tree just as efficiently as the standalone index. 2. So the single-column index on warehouse_id is duplicating work — every INSERT, UPDATE, DELETE maintains both indexes, but the composite handles every read query the single-column index could. It's become pure overhead: paying write cost and cache pages without ever being picked for a read plan. 3. Confirm no query specifically benefits from the smaller key size of the single-column index — usually not a concern unless the second column is very wide (multi-KB strings). For normal-sized columns like a timestamp, the savings are imperceptible and the duplicate is genuinely pure waste. 4. Drop the redundant one; ETL latency should drop back to baseline within minutes. Watch `pg_stat_user_indexes.idx_scan` for the composite before and after to confirm the optimizer still picks it for the warehouse-only queries that previously used the dropped index.

The diagnosis: Two indexes overlap — the composite subsumes the single-column index via leftmost-prefix. Both must be maintained on every write, doubling index-write cost for no read benefit. On a write-

heavy ETL ingesting tens of thousands of rows per second, that wasted work compounds quickly and shows up as p99 insert latency.

The fix:

```
DROP INDEX idx_inv_movements_warehouse;  
-- Postgres: use CONCURRENTLY to avoid locking the table during drop  
DROP INDEX CONCURRENTLY idx_inv_movements_warehouse;
```

Why this works: Every write now updates one index instead of two — roughly 50% less index-maintenance work for this table. The composite still satisfies every query that used the dropped index, because (warehouse_id) is the leftmost prefix of (warehouse_id, created_at). ETL throughput bounces back to baseline within minutes; cache pressure on the buffer pool also drops because there's one less hot index competing for pages, which indirectly speeds up unrelated queries and reduces replication lag on standbys.

What NOT to say: "It's fine to keep redundant indexes for safety" — it's not, because every redundant index costs write throughput, storage, and cache budget without ever serving a read. Audit and prune. Also avoid "we'll re-add it if needed" without first checking that the composite truly covers every existing query — drop a real index only after a week of monitoring confirms no plan regressions, captured via `pg_stat_statements` before/after.

Common follow-up: "How do I find redundant indexes?" — Tools like `pg_qualstats`, MySQL's `sys.schema_redundant_indexes` view, or `SHOW INDEX` cross-referenced manually with grep. Most DBAs run this audit quarterly and treat it like a code-review pass. Postgres extensions like `pg_stat_statements` show which indexes are actually used, so you can identify both redundant and unused indexes in one pass. A strong answer mentions audit cadence and rollback plan, not just the detection tool.

Scenario92 · Drop Unused Index — Consuming Write Budget

Tags: Difficulty: Easy · Asked at: DBA / SRE rounds · Dialect: Postgres / MySQL

Scenario setup: A noisy table has 12 indexes accumulated over years. Writes are slowing down on the events table — 50k inserts/second peak. An audit reveals 4 of the 12 indexes have zero scans recorded in the last 30 days.

```
-- Postgres: check pg_stat_user_indexes  
SELECT indexrelname, idx_scan  
FROM pg_stat_user_indexes  
WHERE schemaname = 'public' AND relname = 'events'  
ORDER BY idx_scan;
```

Current perf vs target: 12 indexes, 4 unused, p99 insert latency 30ms; target sub-15ms.

Thinking out loud: 1. Every index has a write cost — the engine maintains B-tree balance, updates leaf pages, sometimes splits pages on insert. Twelve indexes means twelve maintenance operations per row insert, each a small but real synchronous write to the WAL/redo log before the transaction can commit. 2. Indexes with zero scans in 30 days are either genuinely unused, or used by a quarterly batch job that hasn't run inside the window — check the application code, cron schedule, and BI tool's saved queries before dropping. A common gotcha: month-end reconciliation that runs on the 30th when the audit happened on the 28th. 3. Postgres exposes `pg_stat_user_indexes` with `idx_scan` and `idx_tup_read` counters that accumulate since stats reset; MySQL exposes `sys.schema_unused_indexes` from 5.7+ via `performance_schema`. Always check when stats were last reset — a freshly-reset counter looks unused but might just be young. 4. Drop one at a time and monitor — never in bulk in case the audit missed a quarterly job that runs tomorrow. Keep a one-line rollback `CREATE INDEX` command in your runbook so you can restore in 30 seconds if a slow-query alert fires post-drop.

The diagnosis: Unused indexes are still maintained on every write, adding latency without serving any read query. Drop after confirming zero usage across a window long enough to catch your slowest-cadence batch job. On a 50k-inserts/second table, even four unused indexes can swing p99 by 5-15ms because every insert pays the maintenance tax synchronously.

The fix:

```
-- Postgres: stage the drop with INVALIDATE first (Postgres 12+)
-- Then DROP INDEX CONCURRENTLY to avoid locks
DROP INDEX CONCURRENTLY idx_events_unused_thing;
-- Keep the CREATE statement in a runbook for instant rollback if needed
```

Why this works: Removing 4 of 12 indexes from a hot insert path reduces per-row maintenance work by ~33%. p99 latency drops proportionally — usually visible within minutes once WAL volume per transaction shrinks. Freed cache and disk space help the remaining indexes stay hot, and you'll often see secondary wins like faster `VACUUM` cycles and lower replication lag on standbys because there's less index data to ship downstream.

What NOT to say: "Drop unused indexes immediately" — without a 30-day-plus monitoring window, you'll catch the quarterly compliance report on the wrong week and break a CFO's month-end dashboard. Always wait, then drop. Also avoid "if it's not used, it's safe to drop" without confirming `pg_stat_user_indexes` counters haven't been reset recently — a server restart or manual `pg_stat_reset()` zeroes them and makes everything look unused.

Common follow-up: "What about indexes used once a year?" — Keep them, but document why so the next audit doesn't accidentally drop them. The audit threshold is "zero scans in 90 days" for most teams; an annual job will show up in a full-year window. A strong answer says "I'd check `idx_scan` over 90 days plus grep the codebase plus ask the analytics team before dropping anything."

Scenario93 · Replace B-tree with GIN for JSON Path Query (Postgres)

Tags: Difficulty: Hard · Asked at: Postgres-first startups, Cred, Razorpay platform · Dialect: Postgres only

Scenario setup: An events table stores semi-structured payloads in a jsonb column. A new query filters events where `payload->>'event_type' = 'checkout_completed'`. There's a B-tree expression index on that exact path, but a follow-up query needs `payload @> '{"source": "android"}'` containment — and the B-tree can't help.

```
SELECT id, user_id, created_at
FROM events
WHERE payload @> '{"source": "android"}'::jsonb;
```

Current perf vs target: 18s scan of 100M-row events; target sub-2s.

Thinking out loud: 1. B-tree is for exact-match or range queries on a single value — JSON containment is neither, so the existing expression index is useless for `@>`. The expression index on `payload->>'event_type'` only helps when the query uses exactly that path with equality or range. 2. GIN (Generalized Inverted Index) is built for "key/value exists in this document" queries — exactly what `@>` does on jsonb. Internally GIN maps every indexable token (each key, value, or hashed path) to a posting list of row IDs, so containment becomes a posting-list intersection rather than a per-row JSON walk. 3. Postgres has two GIN flavors for jsonb: default (indexes every key + value, supports `?`, `?|`, `?&`, `@>`, `<@`) and `jsonb_path_ops` (indexes only the hashed path, smaller and faster for `@>` but no existence operators). Choice depends on whether you ever need `?` outside containment. 4. For a containment-only workload, `jsonb_path_ops` is the right choice — typically 30% the size of default GIN and faster for `@>` because the posting list is keyed on hashed full paths. Smaller index means more of it fits in cache.

The diagnosis: B-tree can't index unstructured JSON containment. Need a GIN index purpose-built for jsonb operators, with the operator class chosen based on which jsonb operators the application actually uses.

The fix:

```
-- For ?, ?|, ?&, @>, <@ – use default jsonb GIN
CREATE INDEX idx_events_payload_gin ON events USING gin (payload);

-- For @> only – smaller, faster
CREATE INDEX idx_events_payload_pathops
ON events USING gin (payload jsonb_path_ops);
```

Why this works: GIN inverts the document — it stores a mapping from each key/value (or path) back to the row IDs containing it. The containment query `payload @> '{"source": "android"}'` becomes a fast inverted-index lookup: find the posting list for that key/value, return row IDs, then

recheck the jsonb document against the full predicate. Goes from a 100M-row sequential scan that parses jsonb per row to a posting-list lookup returning candidates in milliseconds.

What NOT to say: "GIN works on any column" — it doesn't; it's built for composite values (jsonb, arrays, tsvector) where each row contributes multiple indexable tokens. Don't suggest GIN for a plain integer column where B-tree is dramatically better. Also avoid "GIN is always slower to write" without qualifying — with `fastupdate = on` the cost moves to a pending list that flushes when full, so write latency stays low for bursty inserts.

Common follow-up: "What's the write cost of GIN?" — Significantly higher than B-tree because every key/value pair is indexed, and each insert may touch many posting lists. Use `fastupdate` to batch updates into a pending list that flushes asynchronously; tune `gin_pending_list_limit` if writes become a bottleneck. A senior answer mentions monitoring `pg_stat_user_indexes.idx_blks_*` and watching VACUUM logs to ensure the pending list isn't growing unbounded.

Scenario94 · Replace B-tree with BRIN for Append-Only Large Table (Postgres)

Tags: Difficulty: Hard · Asked at: Data engineering, analytics platforms · Dialect: Postgres only

Scenario setup: A telemetry table holds 2 billion rows of sensor readings, ordered by recorded_at because data only ever appends in time order. A B-tree index on recorded_at is 80GB — bigger than the working set fits in cache. Range queries on recorded_at are slow because the index itself doesn't fit in RAM.

```
SELECT device_id, AVG(temperature)
FROM telemetry
WHERE recorded_at BETWEEN '2026-05-01' AND '2026-05-07'
GROUP BY device_id;
```

Current perf vs target: 45s with cold B-tree; target sub-5s and tiny index.

Thinking out loud: 1. B-tree is overkill for a column naturally correlated with physical row order — every row inserted goes to the end in time order. Maintaining an 80GB tree to map timestamps to row IDs when those row IDs are already essentially in timestamp order is paying full tree-maintenance cost for a non-problem. 2. BRIN (Block Range Index) stores a min/max per block range (default 128 pages = 1MB). For a column where physical order matches logical order, BRIN narrows the scan to just the relevant block ranges — instead of indexing every row, it indexes every chunk, which is wildly cheaper. 3. BRIN size: roughly 1KB per 1GB of table at default `pages_per_range`. For a 200GB table, the BRIN index is 200KB vs 80GB for a B-tree. That's not a typo; the ratio is that extreme because BRIN only records summary statistics, not individual row pointers. 4. The trade-off: BRIN is useless if the column is not physically correlated (e.g., random uuid timestamps, or tables you frequently UPDATE in random places). Always check `pg_stats.correlation` first; correlation near 1.0 or -1.0 means BRIN will work, anything close to 0 means BRIN will scan most of the table.

The diagnosis: B-tree on append-only time-series column is unnecessarily large because the data is already physically ordered. BRIN exploits the correlation for a 1000x smaller index that lives entirely in cache and serves range scans by jumping directly to the relevant heap chunks.

The fix:

```
DROP INDEX idx_telemetry_recorded_at;  
  
CREATE INDEX idx_telemetry_recorded_at_brin  
  ON telemetry USING brin (recorded_at)  
  WITH (pages_per_range = 32);
```

Why this works: BRIN stores one min/max pair per 32-page chunk. The range query for May 1-7 finds the chunks whose min/max overlap that window, then scans just those chunks sequentially from the heap. For an append-only table this might be 0.1% of the total — dramatic I/O reduction with a microscopic index. Smaller `pages_per_range` gives finer-grained pruning at the cost of slightly larger BRIN; 32 is a reasonable starting point you can tune based on workload shape.

What NOT to say: "BRIN is always smaller, use it everywhere" — it's only useful when physical order correlates with the indexed column, which is a strong precondition. On a randomly-distributed column, BRIN scans most of the table because min/max ranges overlap too much. Also avoid "BRIN is just a B-tree alternative" — it's a fundamentally different structure (lossy summary vs precise pointers) with a different cost model.

Common follow-up: *"How do I keep correlation high?"* — On append-only tables it stays high naturally because every new row lands at the physical tail. If you bulk-update old rows (e.g., backfilling a new column), correlation degrades and you'll need `CLUSTER` or `pg_repack` to restore physical order. For high-churn tables, BRIN is a poor choice — every UPDATE that changes the indexed column scrambles the min/max boundaries and summarisation drifts from reality.

Scenario95 · Trigram Index for LIKE '%suffix' Search

Tags: Difficulty: Hard · Asked at: Search-heavy products, Postgres startups · Dialect: Postgres (pg_trgm) / Some MySQL via plugins

Scenario setup: A product-search bar runs `WHERE name ILIKE '%shirt%'`. There's a B-tree on name. The query takes 3 seconds on 5M products because the leading `%` makes the B-tree useless — no index can seek to a prefix that starts with a wildcard.

```
SELECT id, name, price  
FROM products  
WHERE name ILIKE '%shirt%';
```

Current perf vs target: 3s scan; target sub-200ms.

Thinking out loud: 1. B-tree indexes are prefix-sortable, so `LIKE 'shirt%'` works (the engine seeks to 'shirt' and walks the leaf pages) but `LIKE '%shirt%'` does not — the leading wildcard prevents prefix matching because the tree has no way to know which leaf contains 'shirt' as a substring versus a prefix. 2. The `pg_trgm` extension breaks strings into 3-character chunks (trigrams) and indexes them via GIN or GiST. Any substring search becomes "find rows whose trigrams overlap with the query's trigrams" — a posting-list intersection, not a tree descent. The word 'shirt' becomes the trigrams `_sh`, `shi`, `hir`, `irt`, `rt_` after `pg_trgm`'s standard padding. 3. For full-text search there's also `tsvector` with `to_tsvector` and `to_tsquery`, but trigram is the right pick for LIKE-style substring matching where you don't want stemming, stopword filtering, or language-specific tokenisation. Trigram matches the exact byte sequence; full-text matches lexemes. 4. Trigram indexes also accelerate fuzzy match via the `%` similarity operator and the `<->` distance operator, which is a free bonus for "did you mean" features and typo-tolerant search. The same index serves substring LIKE, ILIKE, and similarity queries.

The diagnosis: Leading-wildcard LIKE can't use a B-tree because the tree depends on prefix order. Trigram indexes break strings into 3-char chunks and use GIN/GiST for fast substring lookup — turning containment into an inverted-index lookup over the trigram tokens.

The fix:

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;

CREATE INDEX idx_products_name_trgm
  ON products USING gin (name gin_trgm_ops);
```

Why this works: The query `name ILIKE '%shirt%'` is converted to a trigram lookup for 'shi', 'hir', 'irt' — the GIN index returns row IDs that contain all three trigrams, then the engine confirms each candidate with a final ILIKE check (the "recheck" step). Goes from a 5M-row scan to a few thousand candidate rows in milliseconds. The recheck is necessary because the trigram set can produce false positives (e.g., 'shir' + 't' as separate words), but the candidate set is small enough that the final filter is essentially free.

What NOT to say: "Use full-text search instead" — for short single-word substring matches like product names, trigram is faster and simpler. Full-text shines for natural-language queries (multi-word, ranking, stemming, multi-language), not raw substring containment in a product catalogue. Also avoid "trigrams are always faster than B-tree" — for a pure prefix match like `LIKE 'shirt%'`, a regular B-tree still wins because trigram has extra recheck cost.

Common follow-up: "What about MySQL?" — MySQL doesn't have trigrams natively, so the standard answer is full-text indexes (`FULLTEXT INDEX` with the `MATCH ... AGAINST` operator) for word-boundary search, or a third-party search engine like Elasticsearch / Meilisearch for non-trivial workloads. A senior answer also mentions that for an Indian e-commerce stack on MySQL 8, many teams just push search to OpenSearch or Typesense rather than fight the in-database tooling, because LIKE-with-leading-wildcard at scale really is a search engine's job.

Scenario96 · Index on Generated Column for Case-Insensitive Search

Tags: Difficulty: Medium · Asked at: MySQL shops, e-commerce GCCs · Dialect: MySQL 8 / Postgres

Scenario setup: A vendor lookup uses `WHERE LOWER(vendor_code) = 'in-blr-001'`. A plain index on `vendor_code` doesn't help because of the function call. The team is on MySQL 8 and prefers generated columns over expression indexes for readability.

```
SELECT id, vendor_name, address
FROM vendors
WHERE LOWER(vendor_code) = 'in-blr-001';
```

Current perf vs target: 700ms scan of 500k vendors; target sub-20ms.

Thinking out loud: 1. Same root problem as Scenario6 — function-wrapped column disables the plain index, because the optimizer can't translate `LOWER(vendor_code) = 'X'` into a B-tree seek against the raw column. Without proof that `LOWER` is monotonic and deterministic, the planner scans every row. 2. MySQL 8 supports expression indexes via `((expr))` syntax, but many teams prefer a generated stored column that's plainly visible in the schema, then a normal B-tree on it. The generated column shows up in `DESCRIBE` output; an expression index is invisible unless someone runs `SHOW INDEX`. 3. The trade-off: generated stored column doubles storage for the column (original plus lowercased copy), but is clearer to future maintainers than an expression index. For a 500k-row table the storage cost is trivial — maybe 25MB — and worth the clarity. 4. Generated virtual columns also work and don't take storage, but the index on them is essentially an expression index under the hood. For lookup-heavy workloads, stored is preferred because the value is materialised and read paths are simpler.

The diagnosis: Function call in `WHERE` disables the plain index. Use a generated column (or expression index) to give the optimizer something to seek on — the index stores the pre-computed function result and the optimizer matches the predicate directly.

The fix:

```
-- MySQL 8
ALTER TABLE vendors
  ADD COLUMN vendor_code_lower VARCHAR(50)
    GENERATED ALWAYS AS (LOWER(vendor_code)) STORED;

CREATE INDEX idx_vendors_code_lower ON vendors(vendor_code_lower);

-- Then either rewrite the query or trust the optimizer to use the generated column
```

Why this works: The generated column holds the lowercased version, kept in sync automatically on every `INSERT` and `UPDATE` of `vendor_code` — no application change needed. The B-tree on it serves direct equality queries on `LOWER(vendor_code)` — most MySQL 8 versions recognize the

function-on-source-column expression and use the index automatically. If the optimizer is stubborn, rewrite the WHERE to filter on the generated column directly and the pick becomes unambiguous.

What NOT to say: "Just rewrite the application to store lowercase" — sometimes you can't, because multi-team data ownership or legacy callers prevent a coordinated release, and the generated column gives the same outcome without that coordination. Also avoid "indexes always understand function calls" — they don't, and pretending otherwise is the kind of confident-but-wrong claim that costs you a senior interview.

Common follow-up: "Stored vs virtual generated column?" — Stored consumes disk but is fast to read because the value is materialised at write time; virtual is computed on the fly every read. For indexed lookups, stored is usually right because the index needs the value materialised anyway, and storing once in the table avoids re-computing on every related UPDATE. A strong answer also mentions that stored columns participate in row-format limits (innodb_default_row_format and the 65,535-byte row limit) while virtual columns don't.

Scenario97 · Hash Index for Equality-Only Column (Postgres 10+)

Tags: Difficulty: Hard · Asked at: Postgres-deep teams, infra-leaning roles · Dialect: Postgres 10+ only

Scenario setup: A session-token lookup happens on every API call — `WHERE session_id = 'abc123...'`. The session_id is a 64-char hex string, lookups are equality-only (never range, never sort). A B-tree on session_id is being maintained, but the team wonders if hash would be faster.

```
SELECT user_id, expires_at
FROM sessions
WHERE session_id = 'a8f3e2c4b1...';
```

Current perf vs target: 0.6ms p50 with B-tree; want lower variance under high concurrency.

Thinking out loud: 1. Postgres hash indexes were unreliable pre-10 (no WAL logging, couldn't survive crash, not replicable). Post-10 they're crash-safe and replicable — finally usable in production. The internet still has stale 9.x-era advice saying "never use hash indexes," which trips up candidates who haven't tracked the change. 2. Hash indexes are smaller than B-tree for long keys (constant-size hash buckets vs full sortable keys) and have O(1) lookup vs O(log N). For a 64-char hex string, B-tree stores ~64 bytes per entry while the hash bucket is just the hash plus row pointer — dramatically smaller index and shorter traversal. 3. The catch: hash supports only `=`, never `<`, `>`, `BETWEEN`, `ORDER BY`. If any query needs those, you need a B-tree anyway, and dropping to hash makes you re-add B-tree later — losing the benefit. Confirm the access pattern is genuinely equality-only before switching. 4. For a strict equality lookup on a long opaque string, hash wins on size and lookup cost. Postgres 14+ benchmarks show ~30% smaller index and ~10-15% faster lookups under contention, with the gap widening as keys get longer because B-tree key comparisons get more expensive while hash comparisons stay constant.

The diagnosis: B-tree is overkill for equality-only lookups on long opaque strings. Hash indexes are smaller and have $O(1)$ lookup — ideal for session tokens, API keys, or any opaque identifier never sorted or range-queried.

The fix:

```
DROP INDEX idx_sessions_session_id;  
  
CREATE INDEX idx_sessions_session_id_hash  
ON sessions USING hash (session_id);
```

Why this works: Hash buckets give constant-time lookup regardless of table size — no tree descent. The index is smaller because it stores buckets keyed by a 4-byte hash instead of full sortable keys. For a session-lookup hot path running millions of times per second, the savings add up — both in p99 latency (less variance under contention because no tree depth) and in cache pressure (smaller index means more stays resident).

What NOT to say: "Hash indexes are deprecated" — they were unreliable pre-Postgres 10, but are fully supported now and recommended for the right workload. If the interviewer says "I heard they're unsafe," correct politely with the version history; it's a chance to demonstrate you read release notes. Also avoid "hash is always faster than B-tree" — for short integer keys the gap is negligible and B-tree's range support outweighs any micro-benefit.

Common follow-up: "What about MySQL hash indexes?" — MySQL has them only on MEMORY tables and some NDB cluster configs. For InnoDB you're stuck with B-tree, although InnoDB maintains an "adaptive hash index" automatically for hot keys behind the scenes — different mechanism but similar lookup-speed benefit for equality. Calling out the dialect difference signals depth, especially since most candidates assume MySQL has parity with Postgres on index types.

Scenario98 · Partial Index for `is_active = true` Hot Path

Tags: Difficulty: Medium · Asked at: SaaS startups, multi-tenant platforms · Dialect: Postgres / SQL Server

Scenario setup: A multi-tenant SaaS has a tenants table with 80k rows, of which only 12k are `is_active = true` — paying customers. Almost every query filters by `is_active = true AND tenant_id = X`. A composite index on `(is_active, tenant_id)` works but is bloated with inactive rows.

```
SELECT id, plan_tier, billing_email  
FROM tenants  
WHERE is_active = TRUE  
AND tenant_id = 99284;
```

Current perf vs target: 8ms p95 with composite; want smaller index and lower cache footprint.

Thinking out loud: 1. The `is_active = true` rows are a small minority (12k of 80k, roughly 15%) but account for ~99% of query volume because inactive tenants don't generate dashboard traffic. The other 85% is dead weight in the index — pages in the B-tree that never match at read time but still cost write-maintenance work and cache space. 2. A partial index `WHERE is_active = TRUE` shrinks the index to 12k entries — fits comfortably in cache, often in a single megabyte of RAM. Once it's cache-resident, every lookup becomes a memory-only operation with no disk I/O. 3. Bonus: the index doesn't need `is_active` as a column, because that constraint is implicit in the WHERE-clause predicate. So the partial index is just on `tenant_id`, making the B-tree key as narrow as possible and maximising fanout. 4. The optimizer matches the query WHERE against the index WHERE — both must be identical or the query predicate must be a provable subset, or the index is skipped. So `WHERE is_active = TRUE AND tenant_id = X` matches; `WHERE is_active = TRUE OR tenant_id = X` does not.

The diagnosis: Composite index includes inactive rows that are never queried. A partial index covering only active rows is smaller, hotter, and serves the same queries — with zero loss of correctness because the predicate excluding inactive rows is encoded into the index definition itself.

The fix:

```
DROP INDEX idx_tenants_active_tenant;

CREATE INDEX idx_tenants_active_partial
  ON tenants(tenant_id)
  WHERE is_active = TRUE;
```

Why this works: The partial index is ~1/7 the size of the full composite. It lives entirely in `shared_buffers` / buffer pool, often comfortably under a megabyte. Lookups become memory-resident hash-like operations because the tree is small enough that every level is hot. The `is_active` filter is implicit in the index definition, so no extra check at read time and no extra column in the B-tree key. Write cost also drops because rows in the inactive 85% never touch the index — only active-row INSERTs and UPDATEs trigger maintenance.

What NOT to say: "Partial indexes are too restrictive" — they're the right tool when the hot subset is small and stable. For dynamic predicates (like `WHERE created_at > NOW() - INTERVAL '7 days'`), partial indexes are a bad fit because the predicate window slides over time and yesterday's matching rows become today's non-matching rows without any UPDATE happening. Use partial only for stable predicates that don't drift.

Common follow-up: "What if a tenant is reactivated?" — Row enters the partial index automatically on the UPDATE that flips `is_active` to TRUE; deactivation removes it. Maintenance is transparent. A strong answer also notes the partial index works hand-in-hand with the leftmost-prefix rule from Scenario11 — if other queries filter by `tenant_id` without the `is_active` predicate, you may need a separate full index and accept the duplication.

Scenario99 · Index for ORDER BY with NULLS LAST

Tags: Difficulty: Medium · Asked at: Postgres-leaning teams, reporting platforms · Dialect: Postgres / Oracle / MySQL 8

Scenario setup: A "users who haven't logged in" report sorts users by `last_login_at` ascending with NULLS LAST so never-logged-in users appear at the bottom. There's a plain index on `last_login_at` ASC. The query plan shows an explicit sort step despite the index, because the default NULLS ordering differs from what the query requests.

```
SELECT id, email, last_login_at
FROM users
ORDER BY last_login_at ASC NULLS LAST
LIMIT 100;
```

Current perf vs target: 4s with sort step on 30M users; target sub-100ms.

Thinking out loud: 1. Postgres defaults: ASC → NULLS LAST, DESC → NULLS FIRST. Oracle defaults: ASC → NULLS LAST, DESC → NULLS LAST. They differ — and the index must match the query's request, otherwise the engine has to physically re-order the result. This is exactly the kind of dialect detail that catches teams porting reports between databases. 2. If the index was built without an explicit NULLS clause, Postgres uses NULLS LAST for ASC — which matches in this case. But if the index was built DESC, NULLS would default FIRST, mismatching `ORDER BY ... DESC NULLS LAST` and forcing a sort. Many production schemas have indexes that "look right" but mismatch on NULLS ordering. 3. The fix is to declare NULLS ordering explicitly in both index and query, so they match and the sort step disappears. Explicit beats implicit here — defaults differ across versions and dialects, but explicit syntax is always portable. 4. MySQL 8 finally supports NULLS FIRST / NULLS LAST syntax; before 8, NULL ordering was fixed (NULLs sort low, so ASC puts NULLs first, DESC puts NULLs last) and you had to use `IS NULL` tricks like `ORDER BY field IS NULL, field`. The trick worked but disabled the index for ordering, costing a sort.

The diagnosis: Index NULLS ordering differs from query NULLS ordering; the engine adds a sort step to physically rearrange the result. Align them explicitly so the index leaves are already in the exact order the query wants — no sort needed.

The fix:

```
CREATE INDEX idx_users_last_login_nullslast
ON users(last_login_at ASC NULLS LAST);
```

Why this works: The leaf pages are physically ordered with NULLs at the end — exactly the order the query wants. The LIMIT 100 walks the first 100 non-NULL entries and stops; no sort, no scan beyond the LIMIT. On a 30M-row users table where most have a `last_login_at` value and only a fraction are NULL, the index serves the query as an ordered stream and the planner short-circuits after the LIMIT.

What NOT to say: "Just rewrite the query to push NULLs out" — you can hack it with

`COALESCE(last_login_at, '9999-12-31')` and ORDER BY that expression, but it's fragile (what if a real timestamp ever exceeds your sentinel?) and the expression sort still disables the index. The declarative NULLS LAST is cleaner, portable, and obvious to the next reader. Sentinel-value hacks are a credibility hit in senior interviews — they show willingness to encode assumptions in data.

Common follow-up: "What about MySQL 5.7?" — No native NULLS LAST. Common workaround:

`ORDER BY (last_login_at IS NULL), last_login_at` — but it disables index ordering for the second clause, costing a sort over the whole result set. A strong answer adds "we'd push the team toward MySQL 8 specifically for this kind of ORDER BY semantics, because the cost of working around it adds up across many reports."

Scenario100 · Index Leading Column with Wrong Cardinality

Tags: Difficulty: Medium · Asked at: Senior backend, query-tuning rounds · Dialect: ANSI / All

Scenario setup: A composite index on (status, country_code, created_at) was built years ago. status has 4 values; country_code has 200; created_at is a timestamp. Almost every query hits status = 'completed' AND country_code = 'IN' — and is slow because the leading column has terrible cardinality.

```
SELECT id, amount, created_at
FROM orders
WHERE status = 'completed'
      AND country_code = 'IN'
      AND created_at >= '2026-01-01';
```

Current perf vs target: 6s with poor index selectivity; target sub-500ms.

Thinking out loud: 1. The composite leads with status (4 values), so the first index descent only narrows to 1/4 of the table — still 50M rows on a 200M-row table. That's not a useful narrowing; the index has to walk an enormous slice before any secondary filter can engage, and the planner often gives up and falls back to a sequential scan. 2. Then within that subset, country_code narrows to 1/200 — but the index has to walk a large chunk before it can use country_code, because country_code is the second column and the engine can only seek to it after fixing the leading status value. The 50M-row first-level scan is the bottleneck. 3. Better leading column: country_code (200 values) or even created_at (effectively unique) — but range columns can't lead a composite well for equality+range workloads, so country_code wins. The classic recipe is: lead with the equality column that has the highest cardinality, follow with other equality columns by descending cardinality, end with the range column. 4. Rebuild with country_code first, then status (still useful for filtering after the country narrows), then created_at last for the range walk. This matches the standard composite-index recipe and gives each level of the descent maximum selectivity.

The diagnosis: Leading column has poor cardinality, so the initial seek doesn't narrow the search space enough. Reorder columns so the most-selective equality leads — selectivity-first composite design is the rule of thumb for any equality+equality+range workload.

The fix:

```
DROP INDEX idx_orders_status_country_created;  
  
CREATE INDEX idx_orders_country_status_created  
ON orders(country_code, status, created_at);
```

Why this works: The descent narrows by 1/200 immediately (country = IN), then by 1/4 (status = completed), then walks created_at as a sorted range. Each step is dramatically more selective than before — the search space shrinks from 200M rows to 1M after the first descent, then to 250k after the second, then narrowed further by the date range. Engine reads far fewer index pages and far fewer heap rows, and the optimizer's cost estimate makes index-scan obviously cheaper than sequential-scan so the plan choice is stable.

What NOT to say: "Selectivity doesn't matter for composites" — it absolutely does, because the leftmost column determines how big a slice the index has to scan before secondary filters kick in. A composite led by a 4-value column is essentially a sequential-scan-with-extra-steps for most queries. Also avoid "always lead with the column most queries filter on" without the selectivity check — if the most-common filter is on a low-cardinality column, you need either a partial index or a different composite shape.

Common follow-up: *"What if status is sometimes queried alone?"* — Then you need an additional index leading with status, or you accept the slower plan for that less-frequent query. Trade-offs are explicit and worth measuring: count how often each query shape runs, multiply by its current latency, and compare against the write cost of a second index. A senior answer talks numbers, not just "it depends."

Scenario101 · Index on Date Column for Range Filter

Tags: Difficulty: Easy · Asked at: Every analytics-touching role · Dialect: ANSI / All

Scenario setup: A monthly revenue report sums payments for a date range. The payments table has 200M rows. There's no index on captured_at. The report takes 90 seconds.

```
SELECT SUM(amount) AS monthly_total  
FROM payments  
WHERE captured_at >= '2026-05-01'  
AND captured_at < '2026-06-01';
```

Current perf vs target: 90s full scan; target sub-10s.

Thinking out loud: 1. Range filter on a column with no index = full table scan, regardless of how small the range is. A query targeting one day out of three years still reads all 200M rows, because without an index the engine can't skip ahead — it has to inspect every row's captured_at value. 2. B-tree on

captured_at lets the engine seek to May 1 and walk sequentially until June 1 — pages read proportional to the range, not the table. Tree descent finds the starting leaf in log time, then leaf-page traversal is a contiguous walk in the index's physical order. 3. For a month of payments out of years of history, that's maybe 5% of the table — 20x less work in pages touched, and even less CPU because there's no per-row predicate evaluation. Just walk and emit. 4. Bonus: if SUM is the only aggregation and the column set is small, consider a covering index (Scenario4) to skip heap reads entirely — `INCLUDE (amount)` makes index leaves carry the sum input and the query becomes index-only. For aggregation-heavy reports on a single numeric column, often a 5-10x further speed-up.

The diagnosis: No index on the range column forces a full scan of the entire 200M-row table to find rows in a one-month window. A simple B-tree on captured_at narrows the scan to the requested date range — index seek to find the start, then sequential leaf-page walk to the end.

The fix:

```
CREATE INDEX idx_payments_captured_at ON payments(captured_at);  
-- For an aggregation hot path, consider INCLUDE (amount) for index-only scan  
CREATE INDEX idx_payments_captured_at_amt ON payments(captured_at) INCLUDE (amount);
```

Why this works: B-tree finds May 1 in $O(\log N)$, then walks leaves sequentially until June 1 — pages proportional to (range / total). 90 seconds becomes 4-5 seconds with the basic index, mostly heap I/O for matching rows; with the covering variant the heap visit disappears and you're looking at maybe 1-2 seconds because leaves carry the amount values directly. The range walk is also a sequential disk pattern, dramatically faster than the random access B-tree typically requires.

What NOT to say: "Add an index on every date column just in case" — only if queried with range filters or order-by; an index on a never-read date column is pure write tax. Date columns only written and never queried (like soft-delete timestamps checked only by a nightly cleanup) often don't need indexes. Also avoid "B-tree always beats sequential scan" — for queries selecting more than ~10-20% of the table, sequential scan wins because random I/O for heap lookups loses.

Common follow-up: "Why not partition by date?" — Partitioning helps at 1B+ row scale where even indexed scans touch too much data, or where you need cheap retention drops (drop partition = millisecond DDL, delete = hours). For 200M rows, an index is enough; partitioning adds operational complexity. The right answer is usually "index now, partition when the table outgrows the buffer pool or when retention deletes become painful."

Scenario102 · Index on date_trunc Expression for Daily Aggregation

Tags: Difficulty: Medium · Asked at: Postgres analytics teams · Dialect: Postgres (date_trunc) / Most engines via expression index

Scenario setup: A daily metrics dashboard groups events by day with `date_trunc('day', created_at)`. There's an index on `created_at`, but the GROUP BY runs a sort step over millions of rows. The dashboard takes 22 seconds to render.

```
SELECT date_trunc('day', created_at) AS day,
       COUNT(*) AS event_count
FROM events
WHERE created_at >= '2026-05-01'
GROUP BY 1
ORDER BY 1;
```

Current perf vs target: 22s with sort; target sub-3s.

Thinking out loud: 1. The index on `created_at` supports the range filter (find rows where `created_at` >= May 1), but the GROUP BY on `date_trunc` requires a separate sort step to bucket the trimmed timestamps. The raw index is sorted by full timestamp, not by day boundary, so the engine materialises the full filtered set and sorts or hashes it for the GROUP BY. 2. An expression index on `date_trunc('day', created_at)` physically stores the day-bucketed values, so the GROUP BY can walk the sorted index without a sort step. The leaves are already ordered by day, which is exactly the grouping key. 3. Trade-off: the expression index doesn't help the original range filter on raw `created_at`, so you may end up wanting both — or restructure the query to filter and group on the same expression. The cleanest fix is to push `date_trunc` into the WHERE as well, so the index serves both filter and group. 4. For really hot daily aggregates, the better long-term move is a materialized view refreshed nightly — the dashboard then queries a 30-row summary table instead of aggregating millions of rows per page load. But the expression index is the first quick win that doesn't require refresh-schedule operational changes.

The diagnosis: GROUP BY on an expression requires sort because the underlying index sorts raw values, not bucketed ones. An expression index on the bucketed value eliminates the sort and turns the aggregation into a streaming walk.

The fix:

```
CREATE INDEX idx_events_created_day
ON events (date_trunc('day', created_at));

-- And rewrite the WHERE to use the same expression for clarity
SELECT date_trunc('day', created_at) AS day, COUNT(*)
FROM events
WHERE date_trunc('day', created_at) >= '2026-05-01'
GROUP BY 1 ORDER BY 1;
```

Why this works: Index leaves are already sorted by day, so GROUP BY 1 streams through buckets in order — no sort, no hash aggregation, no temp file spill. COUNT(*) increments per bucket as the engine walks the leaves, emitting one row per day as soon as the bucket boundary is crossed. For 31 days of data over millions of events, the output is at most 31 rows produced from a contiguous index walk — wall-clock time is dominated by index leaf I/O, not aggregation.

What NOT to say: "Just cache the result in the application" — sometimes right, but the question is about the query and caching is a separate layer that shouldn't excuse a quadratic database query. Application caching complements query tuning, not substitutes for it. Also avoid "expression indexes are always better than B-tree" — for queries filtering on raw timestamps (not bucketed), the raw B-tree wins; the expression index is workload-specific.

Common follow-up: *"What about partitioning by day?"* — At scale (multi-billion rows), partition pruning beats expression indexes because the planner skips entire partitions without touching their data. At hundreds of millions, the expression index wins on simplicity — no partition maintenance, no schema-change coordination, just a single CREATE INDEX. A senior answer notes the right move evolves: index now, partition when the table hits buffer-pool ceiling or when retention drops become the cheap way to delete old data.

Scenario103 · Foreign Key Without Index → Slow Cascade DELETE

Tags: Difficulty: Medium · Asked at: Senior backend, data-platform roles · Dialect: Postgres / MySQL InnoDB

Scenario setup: A GDPR-style "delete user" request takes 8 minutes per user. The users table cascades to orders, payments, sessions, events, addresses, devices, refunds — seven child tables. Five of the seven have no index on user_id.

```
DELETE FROM users WHERE id = 99284;  
-- ON DELETE CASCADE fires on 7 child tables
```

Current perf vs target: 8 minutes per user delete; target sub-5 seconds.

Thinking out loud: 1. Postgres doesn't auto-index FK columns; the developer has to add the index explicitly. Same for SQL Server. MySQL InnoDB does auto-create one, but only for the constraint's referencing column — and many teams assume the universal default is MySQL's behaviour, which causes silent slow-delete bugs on Postgres for years. 2. Without the index, every cascade-delete on a child table is a full table scan to find matching child rows. The constraint enforcement adds a "find rows where user_id = X" query for each child, executed inside the delete transaction, and without an index that's a full scan. 3. For seven child tables, that's seven full scans of potentially huge tables — easily 8 minutes of wall time even at 100MB/s scan speed, plus extra lock contention because the delete holds row-level locks on every parent and child row it touches. GDPR / data-protection workflows specifically hit this pattern hard. 4. The fix is one index per child FK column. Add them concurrently to avoid downtime — `CREATE INDEX CONCURRENTLY` in Postgres takes longer wall time but doesn't block writes on the table, which matters when the child tables are receiving live traffic.

The diagnosis: Multiple child tables have FK columns with no index, so cascade deletes scan each child table fully. Adding FK indexes is the standard fix and should be treated as a schema-completeness requirement, not an optimisation.

The fix:

```
CREATE INDEX CONCURRENTLY idx_orders_user_id    ON orders(user_id);
CREATE INDEX CONCURRENTLY idx_payments_user_id  ON payments(user_id);
CREATE INDEX CONCURRENTLY idx_sessions_user_id  ON sessions(user_id);
CREATE INDEX CONCURRENTLY idx_events_user_id    ON events(user_id);
CREATE INDEX CONCURRENTLY idx_addresses_user_id ON addresses(user_id);
-- repeat for every cascading FK
```

Why this works: Each cascade now uses an index seek into the child table to find dependent rows — $O(\log N)$ per child instead of $O(N)$. For seven child tables, total delete time drops from 8 minutes to a few seconds because the constraint check becomes a quick seek-and-walk rather than a full table read. Lock contention also drops dramatically because the engine holds locks only on actual referenced rows, not on every row it had to scan to find them.

What NOT to say: "Just disable cascade and delete manually" — pushes the problem to the application layer, risks orphan rows when the application crashes mid-cascade, and breaks the database's ability to enforce referential integrity. The constraint is correct; the missing indexes are the bug. Also avoid "cascades are slow, avoid them" — properly indexed cascades are fast and the alternative (manual deletes) is error-prone in production code.

Common follow-up: "How do I audit this across the schema?" — Run a query joining `pg_constraint` against `pg_index` to find FK columns without matching indexes. Most teams keep a saved query for this and run it before any compliance review or major schema migration. A strong answer also mentions automating the check in CI — a schema-lint job that fails the build if any new FK is added without a covering index — so the bug class doesn't recur.

Scenario104 · Multi-Column Index for IN + Range Filter Combo

Tags: Difficulty: Medium · Asked at: Marketplace platforms, reporting tools · Dialect: ANSI / All

Scenario setup: A merchant dashboard shows orders across a list of warehouses for a date range — `warehouse_id IN (...) AND created_at BETWEEN ...`. There's an index on `warehouse_id` alone. The plan does an index lookup for each warehouse, then sorts the union by `created_at` — slow when the IN list has 50+ values.

```
SELECT id, amount, warehouse_id, created_at
FROM orders
WHERE warehouse_id IN (17, 18, 19, /* ... 50 warehouses ... */)
  AND created_at BETWEEN '2026-05-01' AND '2026-05-31'
ORDER BY created_at;
```

Current perf vs target: 3s with multiple lookups + sort; target sub-300ms.

Thinking out loud: 1. IN with N values is essentially N separate equality lookups OR'd together. Each lookup uses the warehouse_id index, returning a slice for that warehouse — then the engine UNIONS them all and applies remaining filters and sorts. For a 50-warehouse IN list, that's 50 separate index seeks plus a merge. 2. The created_at range filter has to be applied per row from the unioned set, and the final ORDER BY adds a sort over the merged-and-filtered result. Each step is independent work the engine can't fuse with the lookup when the index doesn't carry created_at. 3. A composite on (warehouse_id, created_at) lets each per-warehouse lookup land in an already-sorted slice for that date range — narrower scan per warehouse, and the union can be merge-sorted by created_at without a separate sort step. The plan becomes "for each warehouse, walk the May 1 - May 31 slice in order, merge the streams." 4. The optimizer may still need help — sometimes a UNION ALL of explicit per-warehouse queries with explicit ORDER BY clauses is faster than IN because the planner has clearer signal about each branch's row estimates. Worth A/B testing on production-sized data.

The diagnosis: IN + range + sort is three workloads stacked on one index. A composite covering filter + sort gives each per-value lookup a pre-sorted slice, so the engine stream-merges results in the desired order instead of materialising and sorting them.

The fix:

```
CREATE INDEX idx_orders_warehouse_created
ON orders(warehouse_id, created_at);
```

Why this works: For each warehouse in the IN list, the engine seeks to (warehouse_id, '2026-05-01') and walks sequentially to '2026-05-31'. The N per-warehouse slices are already sorted by created_at, so merging them is a streaming merge — no full sort, no temp file spill. Often 10x faster than two single-column indexes because each per-value lookup pays log-N cost once and then walks a contiguous slice already in the desired output order.

What NOT to say: "Indexes don't help with IN clauses" — they absolutely do; the engine runs one lookup per value and the optimizer chooses between index-scan-per-value and a bitmap-OR plan that intersects per-value posting lists. If the IN list is small (<100), per-value lookups work fine; if huge (>10k), consider a temp table join because per-value descent overhead dominates. Avoid blanket "always use IN" or "never use IN" — it's a sizing question.

Common follow-up: "What if IN list has 10,000 values?" — Switch to a JOIN against a values list (`FROM (VALUES (17), (18), ...)`) or a temp table. The optimizer often picks a hash join, which beats N lookups at high N because per-warehouse fixed cost is amortised across the hash build. A senior answer notes that for truly large IN lists, parameter-binding limits in client libraries (libpq's 65k cap) force the join-against-temp-table approach anyway.

Scenario105 · Filtered Index (SQL Server) for Sparse Hot Subset

Tags: Difficulty: Hard · Asked at: SQL Server / .NET shops, GCC enterprise teams · Dialect: SQL Server

Scenario setup: A SQL Server claims-processing system has a Claims table with 200M rows. The processing queue is `WHERE status = 'pending_review' AND assigned_to IS NULL` — typically 5,000 rows. A regular index on status is too coarse and includes hundreds of millions of irrelevant rows.

```
SELECT TOP 100 claim_id, claimant_name, filed_at
FROM Claims
WHERE status = 'pending_review' AND assigned_to IS NULL
ORDER BY filed_at;
```

Current perf vs target: 1.4s with regular composite; target sub-50ms with smaller cache footprint.

Thinking out loud: 1. SQL Server's filtered index is the equivalent of Postgres's partial index — same idea (an index containing only rows matching a predicate), different syntax (`WHERE` on `CREATE INDEX`). The mental model is identical; only the keyword and a few restrictions differ. 2. The hot subset is tiny (5k rows out of 200M), so a filtered index covering only those rows is a 5k-entry mini-index — fits in cache, served entirely from RAM. The cache-benefit-to-write-overhead ratio is overwhelmingly positive when the hot subset is this small. 3. Bonus: include `filed_at` as the index key so the `ORDER BY` is satisfied by index order — no explicit sort. The TOP 100 becomes a head-of-tree walk that stops after 100 rows regardless of underlying queue size. 4. Filtered indexes must use SARGable predicates (no functions, no implicit conversions, no `OR` across unrelated columns); `IS NULL` is allowed and is a common queue-style predicate. SQL Server's restrictions here are slightly tighter than Postgres's, but cover most real-world queue patterns.

The diagnosis: Regular index on status includes hundreds of millions of non-pending rows that the queue query never touches. A filtered index restricted to the queue subset is dramatically smaller and hotter, and the optimizer matches the query predicate against the filter predicate so the index is picked automatically.

The fix:

```
CREATE INDEX ix_claims_pending_unassigned
ON Claims (filed_at)
WHERE status = 'pending_review' AND assigned_to IS NULL;
```

Why this works: The index has ~5k entries, sized at perhaps 200KB total — small enough to live entirely in buffer pool RAM. The query becomes a tiny B-tree walk against a fully-cached index, returning the first 100 rows by `filed_at` without ever touching the heap (if `SELECT` columns are included) or with one heap fetch per row otherwise. Plan goes from `Index Seek + Filter + Sort` to `Index Seek + Top` — measurably visible in the execution plan.

What NOT to say: "SQL Server doesn't support partial indexes" — it does, under the name "filtered index," and knowing the dialect name matters when working with a SQL Server team that's been hearing

about Postgres partial indexes from blog posts. Also avoid "filtered indexes are exactly the same as partial indexes" without the caveat that SQL Server has SARGability restrictions Postgres doesn't.

Common follow-up: *"What if the WHERE predicate changes?"* — Filtered indexes are static; if the query definition evolves (e.g., adds a priority column), you must drop and recreate — there's no in-place modification. Versioning the index name (e.g., `ix_claims_pending_v2`) helps coordinate the rebuild across environments and lets you keep both around briefly during transition. A strong answer mentions monitoring `sys.dm_db_index_usage_stats` to confirm the new version gets picked up.

Scenario106 · Avoid Wide Index — Large Key Length Hurts Cache

Tags: Difficulty: Medium · Asked at: Performance-focused senior rounds · Dialect: ANSI / All

Scenario setup: A naive index on (email, name, address, phone, gst_number) was added because a developer thought "more columns = more covering." The index is enormous — 12GB on a 4GB-RAM box. Index lookups are now slow because the index itself doesn't fit in cache, and B-tree fanout is low.

```
-- The bad index:
CREATE INDEX idx_vendors_everything
ON vendors(email, name, address, phone, gst_number);
```

Current perf vs target: 200ms p95 lookups (cache misses); target sub-10ms.

Thinking out loud: 1. B-tree page size is fixed (8KB Postgres, 16KB InnoDB). Wider keys = fewer keys per page = lower fanout = deeper tree = more I/O per lookup. Fanout drop compounds because every level pays the same penalty, so even a single-level depth increase means meaningfully more page reads per lookup. 2. The wide index also evicts other useful pages from buffer pool — cache pressure spreads to unrelated queries because the big index hogs slots that could serve narrower indexes. You'll see cache hit rate drop across the board, not just on this table, after a fat index lands. 3. The fix: narrow the index key to what actually filters (usually 1-3 columns), and use INCLUDE for additional output columns if covering is the goal. INCLUDE columns live in leaf pages only — they don't bloat the key, so upper levels stay narrow and fanout stays high while leaves still carry everything the query needs. 4. The original wasn't covering anything useful; nobody queries by (email, name, address, phone, gst_number) as a tuple, so the wide composite key bloated the tree without enabling any real query. Wide indexes are usually cargo-cult "more columns = more coverage" thinking rather than analysing actual query shapes.

The diagnosis: Wide composite key reduces B-tree fanout, increases tree depth, and bloats cache. Slim the key to what actually filters; use INCLUDE for output-only columns when covering is genuinely needed. The result is a small key + rich leaves design that gives both fast lookup and index-only scan capability.

The fix:

```
DROP INDEX idx_vendors_everything;
```

```
-- Postgres 11+ / SQL Server  
CREATE INDEX idx_vendors_email  
ON vendors(email)  
INCLUDE (name, phone);
```

Why this works: B-tree key is now ~50 bytes instead of 600 bytes. Fanout goes from ~25 keys/page to ~150 keys/page — roughly 6x more keys per page, which translates to one fewer tree level for the same row count. Every lookup does one less page read. Index size shrinks to ~1.5GB, fits in cache, lookup p95 drops to single digits. INCLUDE columns ride along in the leaves so the query still gets `name` and `phone` without a heap fetch, preserving the index-only scan benefit.

What NOT to say: "Bigger indexes always cover more queries" — they cover more queries badly, and "cover" in the index-design sense doesn't mean "include more columns just in case." Targeted slim indexes beat one giant fat index almost always because the planner picks best-fit per query rather than always paying the widest cost. Also avoid "INCLUDE columns are free" — they consume cache space and write maintenance, just not at the same rate as key columns.

Common follow-up: *"What's the max width before it becomes a problem?"* — Rule of thumb: index key over ~200 bytes starts hurting fanout noticeably; over 1KB you're definitely doing it wrong and probably violating Postgres's 2712-byte hard limit anyway. Use INCLUDE for everything that doesn't need to be in the key. A senior answer says "measure don't guess" — pull leaf fanout from `pgstattuple` or `sys.dm_db_index_physical_stats` and compare against page size.

Scenario107 · Index for Self-Join Lookup (Parent-Child)

Tags: Difficulty: Medium · Asked at: Org-hierarchy / CRM platforms · Dialect: ANSI / All

Scenario setup: An employees table self-joins to show every employee with their manager's name. The PK on id is fine for the left side; the join `m.id = e.manager_id` uses the PK on id for the right side. But there's no index on manager_id — and a downstream query "find all reports of manager X" scans the table.

```
-- Slow query: who reports to manager 4587?  
SELECT id, name, designation  
FROM employees  
WHERE manager_id = 4587;
```

Current perf vs target: 1.8s scan of 800k employees; target sub-50ms.

Thinking out loud: 1. The self-join in one direction (employee → their manager) uses the PK and is fast — every employee has at most one manager_id and the PK on id resolves it in $O(\log N)$. The reverse

direction (manager → their reports) needs an index on `manager_id`, because finding every row where `manager_id = some_id` without an index is a full scan. 2. This is a special case of Scenario7 (FK without index), but specifically about self-referential FKs which people forget more often because the parent and child are the same table — the visual reminder "child table needs FK index" doesn't trigger when there's only one table. 3. Bonus: the same index helps recursive CTEs walking down the hierarchy from a root manager — each WITH RECURSIVE iteration does a `manager_id = current_level` lookup, and without the index every level is a full scan, turning a 5-level walk into 5 full table scans. 4. If org-chart queries are super hot, consider a closure table (pre-materialized ancestor-descendant pairs) — but the missing index is the first fix and the closure table is the long-term optimisation. Don't jump to the complex solution before the simple one is in place.

The diagnosis: Self-referential FK column has no index. Queries walking down the hierarchy (manager → reports) full-scan the entire employees table, and recursive CTE traversals multiply that cost by hierarchy depth.

The fix:

```
CREATE INDEX idx_employees_manager_id ON employees(manager_id);
```

Why this works: The lookup `manager_id = 4587` now uses an index seek to find all direct reports in $O(\log N)$ — for an 800k-row table that's roughly 20 page reads instead of a full scan. Recursive CTEs walking down N levels do N seeks instead of N full scans, so a 10-level org-chart traversal becomes 10 quick seeks. Org-chart queries scale to enterprise-sized hierarchies with millions of employees and still return in tens of milliseconds.

What NOT to say: "Self-joins are inherently slow" — they're not; missing indexes make them slow, and with both PK (on `id`) and FK (on `manager_id`) indexed, self-joins are no different in cost from any other equi-join. Also avoid "recursive CTEs are always slow" — they're slow when underlying lookups are slow, and adding the FK index fixes both flat-join and recursive-CTE cases simultaneously.

Common follow-up: *"What about indexing rarely-changing dimension tables?"* — Dimension tables (countries, cities, plan tiers) are read-mostly and tiny; usually indexes on the join and lookup columns are enough, and the table often fits entirely in cache so even a full scan is microseconds. Over-indexing a 200-row dimension table is pointless — full scan is already memory-resident and index lookup overhead can be slower. A strong answer separates "self-referential FK indexes are mandatory" from "dimension-table indexes are optional and workload-dependent."

Category 2, Part B — Optimize This Query: Query Rewrites

Part B is the "rewrite the SQL itself" class — sometimes a query is slow because the planner cannot see a better path through the SQL you have written, even when the indexes, statistics and hardware are all sensible. The interviewer hands you a query that runs in seven seconds and asks you to bring it under

one — and the only thing you are allowed to change is the SQL. Twenty-seven scenarios follow, each one a rewrite a senior engineer at an Indian product company (Razorpay, Swiggy, Flipkart, Zerodha, PhonePe, Cred, Meesho) would reach for inside the first ten minutes of triage. Read the diagnosis, read the rewrite, and read the "Why it speeds up" — that third part is what the interviewer is actually grading.

Scenario108 · IN With Subquery That Should Be EXISTS

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart · Topic: Query rewrite

The scenario: A reporting team complains that the "users who placed an order this week" query takes seven seconds on a 40 million row `users` table. The query uses `IN` with a subquery.

```
SELECT id, email
FROM users
WHERE id IN (
    SELECT user_id FROM orders WHERE created_at >= CURRENT_DATE - 7
);
```

Current performance: 7.1 seconds, full sequential scan on `users`, materialised subquery of 1.8 million `user_id` rows held in memory.

Target performance: Under 400 ms by letting the planner switch to a semi-join on the orders index.

Thinking out loud: 1. "`IN (subquery)` looks innocent, but older planners (MySQL 5.7, Postgres pre-12 in some shapes) materialise the entire inner result before deciding the join order. With 1.8 million `user_id` values, that materialisation alone costs seconds." 2. "`EXISTS` is a row-by-row probe that stops at the first match — semantically identical for non-NULL columns, and the planner is far more likely to pick a nested-loop semi-join using the `orders(user_id, created_at)` composite index." 3. "I would also confirm the index on `orders(created_at, user_id)` exists — without it, no rewrite saves us." 4. "Postgres 12+ usually folds `IN` into a semi-join automatically — so the rewrite matters most on MySQL, older Postgres, and SQL Server with stale statistics."

Diagnosis: The planner is materialising the subquery instead of pushing it as a semi-join. The `EXISTS` form makes the semi-join intent explicit and the planner has a tighter contract to fulfil.

Fix SQL:

```
SELECT u.id, u.email
FROM users u
WHERE EXISTS (
    SELECT 1 FROM orders o
    WHERE o.user_id = u.id
    AND o.created_at >= CURRENT_DATE - 7
);
```

Why it speeds up: `EXISTS` is a short-circuit boolean — the planner picks a semi-join, walks the small `orders` index range (one week of data), and probes `users.id` (primary key). No materialisation, no 1.8M-row hash table, no full scan of users.

What NOT to say: - "`IN` and `EXISTS` are identical, just style." — They are semantically identical for non-NULL data, but the planner often executes them very differently. - "I would add a hint to force a hash join." — Hints are a last resort; an `EXISTS` rewrite is cleaner and portable.

Common follow-up: "What changes if the `user_id` column in `orders` is nullable?" — Then `NOT IN` becomes dangerous (next scenario), but `IN` and `EXISTS` are still equivalent because NULL on the inner side simply does not match.

Scenario109 · NOT IN Trap — Rewrite to NOT EXISTS

Tags: Difficulty: Medium · Asked at: Cred, Zerodha, PhonePe · Topic: Query rewrite, NULL safety

The scenario: The marketing team wants users who have never placed an order. Their query uses `NOT IN` and silently returns zero rows after a recent data migration that introduced NULLs in `orders.user_id`.

```
SELECT id, email
FROM users
WHERE id NOT IN (SELECT user_id FROM orders);
```

Current performance: Returns zero rows (wrong answer) — and was already slow at 12 seconds before that.

Target performance: Correct result in under 600 ms.

Thinking out loud: 1. "`NOT IN` has a three-valued-logic landmine — if even one inner row has a NULL `user_id`, the entire result becomes empty because `x NOT IN (1, 2, NULL)` evaluates to `UNKNOWN` for every `x`." 2. "This is not a perf bug first, it is a correctness bug. The migration introduced NULLs, and now the query lies. I would fix correctness before performance." 3. "`NOT EXISTS` does not suffer this — it explicitly checks for the absence of a matching row using equality, and NULL on the inner side simply does not produce a match." 4. "`NOT EXISTS` also tends to plan as an anti-join, which is index-driven and far faster than the hash-anti-join `NOT IN` typically gets."

Diagnosis: Two problems stacked — `NOT IN` is NULL-unsafe, and it is also slower because the planner cannot guarantee anti-join semantics in the presence of NULLs.

Fix SQL:

```
SELECT u.id, u.email
FROM users u
WHERE NOT EXISTS (
  SELECT 1 FROM orders o WHERE o.user_id = u.id
);
```

Why it speeds up: The planner picks an anti-join using the `orders(user_id)` index, walks each user once, and uses index lookups to confirm absence — no materialisation, no NULL-induced wrong answers, and roughly 20x faster on a 40M-user table.

What NOT to say: - "Just add `WHERE user_id IS NOT NULL` to the inner query and keep `NOT IN` ." — It works, but it leaks implementation detail; future schema changes can re-break it. - "NULL is the same as zero." — Wrong on every database; reveals a deep gap.

Common follow-up: "What is the SQL standard's reasoning for `NOT IN` behaving this way?" — Three-valued logic: NULL means "unknown", and you cannot prove a value is not in a set that contains an unknown.

Scenario110 · Correlated Subquery Rewritten as JOIN

Tags: Difficulty: Medium · Asked at: Flipkart, Meesho, Walmart Labs · Topic: Query rewrite

The scenario: A finance dashboard query attaches each user's latest order amount with a correlated scalar subquery and runs in 18 seconds for 500k users.

```
SELECT u.id, u.email,
  (SELECT amount FROM orders o
   WHERE o.user_id = u.id
   ORDER BY o.created_at DESC LIMIT 1) AS last_order_amount
FROM users u;
```

Current performance: 18 seconds — the subquery fires once per outer row, so 500k subquery executions.

Target performance: Under 1.2 seconds via a single grouped join.

Thinking out loud: 1. "Per-row scalar subqueries are the classic $N+1$ inside a single SQL statement — 500k executions of even a fast 0.03 ms lookup is 15 seconds of cumulative overhead." 2. "I can rewrite it as a single JOIN to a derived table that pre-computes the latest order per user, then read the amount from that join." 3. "On Postgres, `DISTINCT ON` is the cleanest expression. On MySQL or older Postgres, `ROW_NUMBER` with a `WHERE rn = 1` works everywhere." 4. "The planner can now use the `orders(user_id, created_at DESC)` index once, hash-aggregate or sort-merge with users — a single pass instead of half a million."

Diagnosis: The correlated subquery forces a nested-loop pattern where each outer user row triggers a fresh inner execution. Aggregating the inner query once and joining flips this to a single pass.

Fix SQL:

```
WITH latest AS (  
    SELECT DISTINCT ON (user_id) user_id, amount  
    FROM orders  
    ORDER BY user_id, created_at DESC  
)  
SELECT u.id, u.email, l.amount AS last_order_amount  
FROM users u  
LEFT JOIN latest l ON l.user_id = u.id;
```

Why it speeds up: One pass over `orders` produces the latest row per user using the `(user_id, created_at DESC)` index. The outer JOIN is hash- or merge-driven — 500k correlated lookups collapse into a single scan plus a merge.

What NOT to say: - "Correlated subqueries are always bad." — Sometimes they win, especially with very small outer row sets. - "I would add an index on `users.id` ." — It is the primary key; it already exists.

Common follow-up: *"What if the planner already rewrote this internally — would it still help?"* — Postgres 12+ may decorrelate simple cases, but the `DISTINCT ON` form forces the optimal plan and is more readable.

Scenario111 · De-Duping JOIN With a Semi-Join via EXISTS

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, BookMyShow · Topic: Query rewrite, de-duplication

The scenario: A query for "users who have at least one paid transaction" uses an INNER JOIN plus `DISTINCT` , runs in 9 seconds, and produces 2 million duplicate rows that get collapsed at the end.

```
SELECT DISTINCT u.id, u.email  
FROM users u  
INNER JOIN transactions t ON t.user_id = u.id AND t.status = 'paid';
```

Current performance: 9.2 seconds — INNER JOIN explodes the user row to one row per matching transaction, then `DISTINCT` collapses them.

Target performance: Under 800 ms with a semi-join.

Thinking out loud: 1. "`DISTINCT` over a JOIN is the classic smell — we generated rows we did not want and then paid CPU to remove them." 2. "What we actually want is a semi-join: for each user, does at

least one matching transaction exist? Stop at one." 3. " **EXISTS** expresses semi-join intent. The planner can short-circuit at the first matching transaction per user instead of fetching all of them." 4. "This also removes the **DISTINCT** sort/hash, which on 2M rows is the second-biggest cost in the original plan."

Diagnosis: The query is using an INNER JOIN as a back-door semi-join, then patching the cardinality with **DISTINCT**. The planner cannot tell that we only want one match per user, so it fetches them all.

Fix SQL:

```
SELECT u.id, u.email
FROM users u
WHERE EXISTS (
  SELECT 1 FROM transactions t
  WHERE t.user_id = u.id AND t.status = 'paid'
);
```

Why it speeds up: Semi-join short-circuits at the first matching transaction per user, the index on **transactions(user_id, status)** is walked once per user with an early-exit, and the **DISTINCT** step is eliminated entirely. CPU and memory both drop sharply.

What NOT to say: - " **DISTINCT** is free." — It is a sort or a hash, both $O(n \log n)$ or $O(n)$ with memory cost. - "GROUP BY would also work." — True, but it still forces all rows to be produced before grouping; semi-join is strictly better.

Common follow-up: "What if we also want to count transactions per user?" — Then a GROUP BY is correct because the count requires every row; the semi-join trick only applies to existence checks.

Scenario112 · SELECT * Replaced With Column Enumeration

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Cognizant · Topic: Query rewrite, column pruning

The scenario: A list endpoint that returns user IDs and names for a dropdown is fetching 220 KB per row because the **users** table has a **profile_jsonb** column with 200 KB of analytics blob per user, and the query uses **SELECT ***.

```
SELECT * FROM users WHERE city = 'Mumbai';
```

Current performance: 4.5 seconds for 12k Mumbai users — 2.6 GB of data over the wire.

Target performance: Under 200 ms; under 5 MB on the wire.

Thinking out loud: 1. "The endpoint needs only **id** and **name**, but **SELECT *** is fetching the wide **profile_jsonb** blob for every row — that is the entire cost of the query." 2. "This is also a covering-index opportunity — if I enumerate just **id** and **name**, and there is a **(city, name) INCLUDE**

(*id*) index, the heap is never touched." 3. "Beyond raw bandwidth, the JSONB column may be TOASTed in Postgres — decompressing it for every row adds CPU on top of bandwidth." 4. "Column-pruning is a free win for the API layer too — serialisation, network, garbage collection on the app server all drop."

Diagnosis: `SELECT *` pulls every column including a large JSONB blob the API does not need. The cost is dominated by reading and shipping the blob, not by row filtering.

Fix SQL:

```
SELECT id, name
FROM users
WHERE city = 'Mumbai';
```

Why it speeds up: The TOAST/LOB column is never read or de-compressed; the row size shrinks from ~220 KB to ~120 bytes; the network is no longer the bottleneck; and if a covering index exists, the heap is skipped entirely. Two orders of magnitude faster.

What NOT to say: - "`SELECT *` is fine in dev." — It corrodes habits and breaks the moment a wide column is added. - "The ORM does this for me." — Most ORMs default to selecting all columns; you have to teach them otherwise.

Common follow-up: "How would you enforce this across a team?" — Code review checklists, a lint rule banning `SELECT *` in repository layer code, and a CI check on slow-query logs.

Scenario113 · Scalar Subquery in SELECT Rewritten as JOIN

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Flipkart · Topic: Query rewrite

The scenario: An admin order report shows `user_name`, `payment_status` and `last_login` per order using three scalar subqueries in the SELECT list. The query runs in 22 seconds for 80k orders.

```
SELECT o.id,
       (SELECT name FROM users WHERE id = o.user_id) AS user_name,
       (SELECT status FROM payments WHERE order_id = o.id ORDER BY created_at DESC LIMIT 1) AS payment_status,
       (SELECT MAX(created_at) FROM sessions WHERE user_id = o.user_id) AS last_login
FROM orders o
WHERE o.created_at >= CURRENT_DATE - 30;
```

Current performance: 22 seconds — 80k × 3 subqueries = 240k subquery executions.

Target performance: Under 1.5 seconds via three joins.

Thinking out loud: 1. "Three scalar subqueries in SELECT means 3N subquery executions where N is outer rows. At 80k orders, that is a quarter-million round-trips inside the planner." 2. "Two of these are simple lookups — `user_name` is a direct `users.id` join, and `last_login` is a grouped MAX

from sessions. Both can become joins." 3. " `payment_status` (latest payment per order) needs the `DISTINCT ON` or `ROW_NUMBER` pattern — same as Scenario 3." 4. "After rewriting, the planner can hash-join users (small), aggregate sessions once, and merge-join the pre-computed latest payment per order. Three passes instead of 240k."

Diagnosis: SELECT-list scalar subqueries are per-row executions. Each one is an N+1 problem disguised as a single query.

Fix SQL:

```
WITH latest_payment AS (
  SELECT DISTINCT ON (order_id) order_id, status
  FROM payments
  ORDER BY order_id, created_at DESC
),
last_session AS (
  SELECT user_id, MAX(created_at) AS last_login
  FROM sessions
  GROUP BY user_id
)
SELECT o.id, u.name AS user_name,
       lp.status AS payment_status,
       ls.last_login
FROM orders o
LEFT JOIN users u ON u.id = o.user_id
LEFT JOIN latest_payment lp ON lp.order_id = o.id
LEFT JOIN last_session ls ON ls.user_id = o.user_id
WHERE o.created_at >= CURRENT_DATE - 30;
```

Why it speeds up: Each derived table is materialised exactly once. The joins are hash- or merge-based. We have replaced 240k correlated calls with three single passes — typically 15-20x faster on this row volume.

What NOT to say: - "Scalar subqueries in SELECT are sometimes faster." — They are faster only when the outer row count is tiny (under a few hundred); on 80k rows they are catastrophic. - "Just add more indexes." — Indexes do not save you from 240k separate executions; the per-call overhead itself dominates.

Common follow-up: "What if one of the scalar subqueries returns a value used by another?" — Then chain CTEs; CTEs run top-down and the planner can fold them.

Scenario114 · UNION Replaced by UNION ALL When Duplicates Are Impossible

Tags: Difficulty: Easy · Asked at: Infosys, Cognizant, HCL · Topic: Query rewrite

The scenario: A monthly revenue report unions the current month and previous month from two partitioned tables. The query takes 11 seconds and the engineer who wrote it used `UNION`.

```
SELECT order_id, amount FROM orders_2026_04
UNION
SELECT order_id, amount FROM orders_2026_05;
```

Current performance: 11 seconds — the planner runs a full deduplication step on the combined 24 million rows.

Target performance: Under 2 seconds.

Thinking out loud: 1. " `UNION` always deduplicates — it runs a sort or hash over every column in the combined result. On 24M rows that is the entire query cost." 2. " `UNION ALL` skips deduplication and just appends. We need to ask: can a duplicate occur? If the two source tables are partitioned by month, no `order_id` can appear in both — duplication is impossible by design." 3. "This is the most common preventable perf mistake in monthly partitioned reports — `UNION` is the SQL default many people type without thinking." 4. "Cost saving scales linearly with row count and number of unioned tables — on 12 months of partitions, the savings are dramatic."

Diagnosis: `UNION` is doing an expensive deduplication that the schema design has already ruled out.

Fix SQL:

```
SELECT order_id, amount FROM orders_2026_04
UNION ALL
SELECT order_id, amount FROM orders_2026_05;
```

Why it speeds up: No sort/hash for deduplication; rows stream straight through. The planner can also parallelise the two scans independently. Memory pressure drops because the dedup hash table is gone.

What NOT to say: - " `UNION` and `UNION ALL` are interchangeable." — Semantically very different; `UNION` removes duplicates, `UNION ALL` does not. - "Always use `UNION ALL` ." — Only when duplicates are impossible or acceptable; otherwise you change the answer.

Common follow-up: "How would you verify the no-duplicates assumption?" — A one-off `SELECT order_id, COUNT(*) FROM (... UNION ALL ...) GROUP BY order_id HAVING COUNT(*) > 1 LIMIT 1` . If empty, you are safe.

Scenario115 · OR Conditions Split Into UNION ALL for Index Use

Tags: Difficulty: Medium · Asked at: Swiggy, Zomato, BookMyShow · Topic: Query rewrite

The scenario: A customer-support query searches orders by either `phone` or `email` , both individually indexed. The query uses `OR` and runs in 6 seconds because the planner falls back to a sequential scan.

```
SELECT id, status FROM orders
WHERE phone = '+919876543210' OR email = 'rahul@example.com';
```

Current performance: 6.3 seconds — full scan because the planner cannot combine two separate indexes through an `OR`.

Target performance: Under 50 ms.

Thinking out loud: 1. "With `OR` across two columns, many planners cannot use both indexes — they pick the cheaper full scan instead. Postgres can do a `BitmapOr` but only when statistics convince it." 2. "Splitting into two queries unioned together forces each branch to use its own index — the planner now has two clearly-indexed paths to combine." 3. " `UNION ALL` is correct here only if duplicates are impossible or acceptable. Since a single order could plausibly match both predicates, I would consider whether the user wants dedup; for a support lookup, `UNION ALL` followed by a manual eyeball is fine." 4. "If true dedup is required, `UNION` (not `ALL`) is still faster than the full-scan original because each branch is index-driven."

Diagnosis: `OR` across independently-indexed columns disables single-index plans. The planner often picks a sequential scan as a safer-cost choice.

Fix SQL:

```
SELECT id, status FROM orders WHERE phone = '+919876543210'
UNION ALL
SELECT id, status FROM orders WHERE email = 'rahul@example.com'
AND phone IS DISTINCT FROM '+919876543210';
```

Why it speeds up: Each branch is a fast index lookup (one row each). The `IS DISTINCT FROM` guard in the second branch removes the rare double-match. Total cost is roughly two B-tree lookups instead of a 12M-row scan.

What NOT to say: - "Just create a composite index on (phone, email)." — A composite is only useful when both columns are filtered together, not in an `OR`. - "Modern planners handle this." — They sometimes do; the rewrite is portable and guarantees the plan.

Common follow-up: "What if the `OR` has three or four branches?" — Same pattern, three or four unions, each with progressively longer `IS DISTINCT FROM` guards if you want strict dedup.

Scenario116 · Long IN List Rewritten as VALUES or Temp Table

Tags: Difficulty: Medium · Asked at: Flipkart, Meesho, Walmart Labs · Topic: Query rewrite

The scenario: A bulk export job filters orders by 14,000 specific `user_id` s. The query has a 14k-element `IN` list, takes 9 seconds to parse, and then 28 seconds to plan.

```
SELECT id, amount FROM orders WHERE user_id IN (12, 47, 89, ..., 9_998_402);
```

Current performance: 37 seconds — most of it spent in parsing and planning a giant `IN` list.

Target performance: Under 3 seconds.

Thinking out loud: 1. "Long `IN` lists are parsed as flat array literals — the parser and planner both struggle past a few thousand elements. Many databases impose hard limits (Oracle 1000, SQL Server 32k)." 2. "Rewriting as a `VALUES` constructor or a temp table lets the planner treat the list as a real relation — it can hash-join, use statistics, and plan a normal nested-loop or hash-join." 3. "`VALUES` is in-line, no DDL needed, and on Postgres it gives the planner a row-count estimate via `VALUES`'s known cardinality." 4. "For repeated use across queries, a temporary table with an index on `user_id` is faster still."

Diagnosis: The planner is choking on the literal list, not on the actual data fetch. Converting the list into a relational input lets the planner reason about it normally.

Fix SQL:

```
SELECT o.id, o.amount
FROM orders o
JOIN (VALUES (12), (47), (89), ..., (9998402)) AS ids(user_id)
ON o.user_id = ids.user_id;
```

Why it speeds up: Parsing is faster (the `VALUES` form is a single relation, not 14k separate expressions), and the planner picks a hash-join between `ids` (small) and `orders` (large via the `user_id` index). Parse + plan time drops from 37 seconds to about 200 ms.

What NOT to say: - "Just keep adding to the `IN` list." — It will eventually exceed parser or driver buffer limits. - "Use a temporary table for everything." — DDL has its own cost; `VALUES` is enough for one-shot queries.

Common follow-up: "What if the list itself comes from another query?" — Then it is already a subquery; rewrite as `EXISTS` or a `JOIN`, never as `IN`.

Scenario117 · LATERAL Join for Per-Row Top-N

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, PhonePe · Topic: Query rewrite, LATERAL

The scenario: A merchant dashboard needs the three most recent transactions for each of 50k merchants. The current query uses a window function over the full 80M-row `transactions` table and runs in 47 seconds.

```
SELECT * FROM (  
  SELECT t.*, ROW_NUMBER() OVER (PARTITION BY merchant_id ORDER BY created_at DESC) AS rn  
  FROM transactions t  
) ranked  
WHERE rn <= 3;
```

Current performance: 47 seconds — window function partitions across all 80M rows before filtering.

Target performance: Under 4 seconds via LATERAL.

Thinking out loud: 1. "The window function processes every transaction row even though we only want three per merchant — that is 80M rows of useless ranking work." 2. "A LATERAL subquery fires once per merchant, with each invocation returning just the top three rows using the `transactions(merchant_id, created_at DESC)` index. Cost is proportional to $\text{merchants} \times 3$, not to transactions." 3. "LATERAL is the cleanest expression of 'for each row in the outer table, run this correlated query'. The optimiser will use the index and stop at LIMIT 3." 4. " $50k \times 3 = 150k$ rows fetched, versus 80M rows scanned and ranked in the original — roughly 500x less work."

Diagnosis: Window functions process the whole partition before the filter is applied. For top-N per group, a LATERAL join lets the planner use the index and stop early.

Fix SQL:

```
SELECT m.id, t.*  
FROM merchants m  
LEFT JOIN LATERAL (  
  SELECT * FROM transactions  
  WHERE merchant_id = m.id  
  ORDER BY created_at DESC  
  LIMIT 3  
) t ON true;
```

Why it speeds up: One index scan per merchant, stopping at 3 rows. The composite index on `(merchant_id, created_at DESC)` is walked from the top of each merchant's range. Total work scales with $\text{merchants} \times N$, not with table size.

What NOT to say: - "Window functions are always optimal for top-N." — They are convenient but rarely the fastest for small N per group. - "LATERAL is only for Postgres." — Standard SQL since 1999; MySQL 8 and SQL Server support it; only ancient versions miss it.

Common follow-up: "What if N is 1?" — Then `DISTINCT ON` (Postgres) or a correlated subquery is even simpler; LATERAL shines from N=2 upward.

Scenario118 · DISTINCT ON for Latest Row Per Group (Postgres)

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Postman · Topic: Postgres-specific rewrite

The scenario: Build a "current price per product" report from a price-history table on Postgres. The current query joins the table to itself to find the max `effective_date` per product and runs in 14 seconds.

```
SELECT p1.product_id, p1.price, p1.effective_date
FROM price_history p1
INNER JOIN (
  SELECT product_id, MAX(effective_date) AS max_date
  FROM price_history GROUP BY product_id
) p2 ON p1.product_id = p2.product_id AND p1.effective_date = p2.max_date;
```

Current performance: 14 seconds — two passes over the 25M-row table plus a hash-join.

Target performance: Under 1.5 seconds with `DISTINCT ON`.

Thinking out loud: 1. "The self-join to find the latest row per group is the canonical pattern, but Postgres has a clean operator: `DISTINCT ON (product_id) ... ORDER BY product_id, effective_date DESC`." 2. "With the right index `(product_id, effective_date DESC)`, `DISTINCT ON` walks the index and picks the first row per `product_id` group — a single pass." 3. "The self-join version reads the table twice and pays for a hash-join; `DISTINCT ON` reads it once and uses no join at all." 4. "Caveat: `DISTINCT ON` is Postgres-only. For cross-database code, use `ROW_NUMBER` with `WHERE rn = 1` (next scenario)."

Diagnosis: The self-join pattern is a generic answer; Postgres provides a faster, native operator.

Fix SQL:

```
SELECT DISTINCT ON (product_id) product_id, price, effective_date
FROM price_history
ORDER BY product_id, effective_date DESC;
```

Why it speeds up: A single index-ordered scan picks one row per `product_id` group. No join, no two passes, no hash table. The index on `(product_id, effective_date DESC)` is walked in order, and the planner skips to the next `product_id` after taking one row.

What NOT to say: - "`DISTINCT ON` is the same as `DISTINCT`." — Wrong; `DISTINCT ON` keeps the first row per group based on `ORDER BY`, while `DISTINCT` deduplicates whole rows. - "It is non-standard, so I avoid it." — It is Postgres-only but blazingly fast; for Postgres-only systems it is the right tool.

Common follow-up: "What if I want the second-latest row?" — Switch to `ROW_NUMBER` with `WHERE rn = 2`, or LATERAL with `LIMIT 1 OFFSET 1`.

Scenario119 · ROW_NUMBER With rn = 1 for Latest Per Group (Portable)

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Flipkart, Zomato · Topic: Query rewrite

The scenario: Same "latest order per customer" report, but the team runs on MySQL 8 (no `DISTINCT ON`). Current query uses a correlated subquery and takes 31 seconds for 2 million customers.

```
SELECT o.* FROM orders o
WHERE o.created_at = (
    SELECT MAX(created_at) FROM orders WHERE customer_id = o.customer_id
);
```

Current performance: 31 seconds — correlated MAX subquery per customer.

Target performance: Under 3 seconds with `ROW_NUMBER`.

Thinking out loud: 1. "The correlated subquery is a per-row execution; for 2M customers that is 2M MAX calls. Even if each is fast, the overhead is fatal." 2. "`ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY created_at DESC)` ranks each customer's orders in one pass; filtering `WHERE rn = 1` keeps the latest." 3. "Window functions are a single sort over the input, ideally streamed using the index on `(customer_id, created_at DESC)`." 4. "This is the portable rewrite — works on Postgres, MySQL 8, SQL Server, Snowflake, BigQuery. `DISTINCT ON` is Postgres-only; `ROW_NUMBER` is universal."

Diagnosis: Correlated subqueries scale linearly with outer rows; window functions are a single ranking pass.

Fix SQL:

```
WITH ranked AS (
    SELECT o.*,
           ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY created_at DESC) AS rn
    FROM orders o
)
SELECT * FROM ranked WHERE rn = 1;
```

Why it speeds up: One pass with a single sort over `(customer_id, created_at DESC)` ranks all rows. If the index supports the ordering, the sort is free. 2M correlated subquery calls collapse into one streamed scan.

What NOT to say: - "`ROW_NUMBER` is slow." — On large data with the right index, it is much faster than correlated subqueries. - "Just GROUP BY." — `GROUP BY customer_id` would lose the row identity; you need the entire row, not just the max date.

Common follow-up: "What if there are ties on `created_at` ?" — Add a deterministic tie-breaker to the ORDER BY (e.g., `id DESC`) so the result is reproducible across runs.

Scenario120 · Moving Filter Into JOIN ON for Outer Joins

Tags: Difficulty: Medium · Asked at: Swiggy, Flipkart, Walmart Labs · Topic: Query rewrite, semantics

The scenario: A LEFT JOIN of `users` to `orders` with `WHERE orders.status = 'paid'` accidentally turns the outer join into an inner join, dropping 8 million users who have no paid orders. The query also misses the index on `orders(user_id, status)` .

```
SELECT u.id, u.email, o.id AS order_id
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
WHERE o.status = 'paid';
```

Current performance: 5 seconds and wrong answer (acts as INNER JOIN).

Target performance: Correct LEFT JOIN behaviour, under 1.5 seconds.

Thinking out loud: 1. " `WHERE o.status = 'paid'` filters out all NULL rows from the LEFT JOIN — and those NULLs are exactly the users without paid orders. The outer-join semantics are silently broken." 2. "To preserve outer-join semantics, the filter must move into the ON clause — that way, unmatched users are still returned with NULL `order_id` ." 3. "As a bonus, moving the filter into ON often lets the planner use the composite index `(user_id, status)` for the join lookup itself, rather than fetching all matches then filtering." 4. "This is the most common LEFT-JOIN bug I have ever seen in code review — it usually shows up only when reported numbers do not match expectations."

Diagnosis: A WHERE on the right side of a LEFT JOIN converts it to an INNER JOIN. The fix is to push the filter into ON.

Fix SQL:

```
SELECT u.id, u.email, o.id AS order_id
FROM users u
LEFT JOIN orders o
  ON o.user_id = u.id AND o.status = 'paid';
```

Why it speeds up: Correctness aside, the planner can now use the composite index `(user_id, status)` directly during the join probe — no post-join filter pass. Outer rows are preserved as NULLs.

What NOT to say: - "The query was wrong but at least it was fast." — Wrong answers are not fast, they are wrong. - "I would add `OR o.status IS NULL` to the WHERE." — Works but is fragile and obscures intent; the ON-clause fix is the standard idiom.

Common follow-up: "What if I want only users who do not have any paid order?" — That is the anti-join pattern: `LEFT JOIN ... ON ... AND status = 'paid' WHERE o.id IS NULL`.

Scenario121 · WHERE Pushed Past OUTER JOIN

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman · Topic: Query rewrite, predicate pushdown

The scenario: A reporting query LEFT JOINS three tables and filters on the outer table at the end. The optimiser does not push the predicate down past the outer joins, scans every row, and takes 18 seconds.

```
SELECT u.id, u.email, o.id, p.id
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
LEFT JOIN payments p ON p.order_id = o.id
WHERE u.country = 'IN' AND u.created_at >= CURRENT_DATE - 30;
```

Current performance: 18 seconds — predicate on `users` is applied late because the planner is uncertain it is safe to push past the LEFT JOINS.

Target performance: Under 1.2 seconds.

Thinking out loud: 1. "Predicate pushdown lets the planner apply the most selective filter as early as possible — in this case, filter `users` first, then join only the filtered set." 2. "The planner is conservative with LEFT JOINS; it often will not push WHERE predicates that reference the outer side because of NULL semantics. Hoisting the filter into a derived table forces the issue." 3. "By wrapping the filtered `users` in a derived table or CTE, I make the row count explicit before the joins — the planner now joins 50k IN+30-day users to orders, not 40M total users." 4. "This rewrite is also more readable; intent is obvious."

Diagnosis: WHERE clauses on outer-join inputs are sometimes not pushed down due to NULL handling concerns. Manually pre-filtering removes ambiguity.

Fix SQL:

```
WITH recent_in_users AS (
  SELECT id, email FROM users
  WHERE country = 'IN' AND created_at >= CURRENT_DATE - 30
)
SELECT u.id, u.email, o.id, p.id
FROM recent_in_users u
LEFT JOIN orders o ON o.user_id = u.id
LEFT JOIN payments p ON p.order_id = o.id;
```

Why it speeds up: The CTE produces a tiny 50k-row driving set; joins are now driven by a small relation with index lookups on `orders.user_id` and `payments.order_id`. The planner no longer scans 40M users.

What NOT to say: - "Pushdown happens automatically." — Sometimes it does; for LEFT JOINs with multi-table predicates, it often does not. - "CTEs are always slower." — Postgres 12+ inlines them by default; even when materialised, a small result is cheap.

Common follow-up: "How would you confirm pushdown happened?" — Read EXPLAIN and check that the filter is applied on the `users` scan node, not on the join output node.

Scenario122 · Rewriting OR as IN

Tags: Difficulty: Easy · Asked at: Infosys, TCS, Wipro, Capgemini · Topic: Query rewrite

The scenario: A status-filter query lists orders in one of six statuses using `OR` and runs in 4.5 seconds. The planner picks a sequential scan.

```
SELECT id, amount FROM orders
WHERE status = 'paid' OR status = 'shipped' OR status = 'delivered'
      OR status = 'returned' OR status = 'refunded' OR status = 'cancelled';
```

Current performance: 4.5 seconds — `OR` chain confuses the planner.

Target performance: Under 200 ms.

Thinking out loud: 1. "`OR` on the same column with multiple constants is exactly what `IN` was designed for. The planner is better at recognising index-friendly access patterns from `IN` than from chained `OR` s." 2. "With `IN`, the planner sees a known small set and picks an index range scan (or multiple index seeks) on `orders(status)`." 3. "With chained `OR`, some planners still figure it out, but it is fragile and version-dependent." 4. "The readability win is also real — `IN` is the universal idiom for membership."

Diagnosis: Multi-OR on the same column is a syntactic disguise of an `IN` predicate. The planner is more reliable with the explicit form.

Fix SQL:

```
SELECT id, amount FROM orders
WHERE status IN ('paid', 'shipped', 'delivered', 'returned', 'refunded', 'cancelled');
```

Why it speeds up: The planner now plans an index scan that walks the six status buckets directly (or, for low-cardinality columns, picks a single sequential scan with the filter applied during the scan). Either way, it is faster and more predictable than the OR chain.

What NOT to say: - " **OR** and **IN** are identical." — Logically they are; physically the planner often handles them differently. - "Always use **IN** ." — **IN** is for one column; cross-column conditions still need **OR** .

Common follow-up: "What if **status** only has 6 distinct values and 90% of rows are in this set?" — Then a seq scan with predicate evaluation is actually optimal; the index would not help. Always check selectivity.

Scenario123 · Avoid Function on Indexed Column

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Flipkart · Topic: Query rewrite, sargability

The scenario: A query searches for users by email, but wraps the column in **LOWER()** . The B-tree index on **email** is bypassed and the query does a full scan in 6 seconds.

```
SELECT id, name FROM users WHERE LOWER(email) = 'rahul@example.com';
```

Current performance: 6.1 seconds — function wraps the indexed column, index is unusable.

Target performance: Under 30 ms.

Thinking out loud: 1. "Once you wrap an indexed column in a function, the index is dead — the planner cannot use it because the indexed values are **email** , not **LOWER(email)** ." 2. "Two fixes: store the column already-normalised (best long-term), or create a functional/expression index on **LOWER(email)** , or rewrite the query to push the function to the right side of the equality." 3. "If the column is always stored lowercase (validated on insert), the **LOWER()** is redundant and can simply be removed." 4. "A functional index **CREATE INDEX ON users (LOWER(email))** is the second-best option — works without changing the query."

Diagnosis: The function on the left side of **=** makes the predicate non-sargable. Move the function to the right side (which is a constant), or remove it, or build a functional index.

Fix SQL:

```
-- Best: if column is normalised at write time
SELECT id, name FROM users WHERE email = 'rahul@example.com';

-- Otherwise, expression index:
CREATE INDEX users_email_lower_idx ON users (LOWER(email));
-- Original query then uses the new index.
```

Why it speeds up: With the predicate **email = 'constant'** , the planner does a B-tree seek — one or two page reads. The expression index alternative builds a separate B-tree on **LOWER(email)** that the planner can use when the query keeps the **LOWER()** wrapping.

What NOT to say: - "Indexes always work." — They only work for sargable predicates. - "Just scan the table; it is fast." — On 40M rows it is not.

Common follow-up: *"What if email matching needs to be case-insensitive but the column is stored with original case?"* — Functional index is the cleanest portable fix; CITEXT (Postgres) is even cleaner if you can change the column type.

Scenario124 · Avoid Implicit Cast in WHERE

Tags: Difficulty: Medium · Asked at: Swiggy, Zomato, Cred · Topic: Query rewrite, sargability

The scenario: A query filters `phone` (stored as VARCHAR) using an integer literal. The implicit cast forces every row to be cast to integer, the index on `phone` is unused, and the query takes 5 seconds.

```
SELECT id, name FROM users WHERE phone = 9876543210;
```

Current performance: 5.2 seconds — every row's `phone` is cast to integer for the comparison.

Target performance: Under 30 ms.

Thinking out loud: 1. *"The column is VARCHAR; the literal is an integer. The database has to cast one side or the other to make the comparison legal. Many databases cast the column (the wrong side), which kills the index."* 2. *"Wrapping the literal in quotes makes the comparison string-to-string — the index on `phone` is usable directly."* 3. *"This bug often hides because the query returns the right answer (slowly) — it is only flagged when a perf review notices the sequential scan."* 4. *"Lesson: match the literal type to the column type explicitly; do not rely on implicit coercion."*

Diagnosis: Implicit casts on the indexed-column side make the predicate non-sargable. Match types explicitly.

Fix SQL:

```
SELECT id, name FROM users WHERE phone = '9876543210';
```

Why it speeds up: The literal is now the same type as the column, the index is used for a direct B-tree seek, and no per-row cast is performed. Six orders of magnitude less work for a unique-index lookup.

What NOT to say: - "It works, who cares?" — Senior interviewers grade you down for this answer. - "Add an index on `CAST(phone AS BIGINT)` ." — Works, but you are papering over a type-design issue.

Common follow-up: *"What if the column should be numeric?"* — Then the migration is to change the column type, not patch the query. Until then, keep types matched.

Scenario125 · Nested Subquery Refactored Into a CTE for Clarity

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro · Topic: Readability, query rewrite

The scenario: A nested-subquery query is correct but unreadable; future maintainers fear it. A CTE rewrite keeps the same plan but makes intent obvious.

```
SELECT * FROM (
  SELECT u.id, u.email, (
    SELECT COUNT(*) FROM orders o WHERE o.user_id = u.id
  ) AS order_count
  FROM users u
) t WHERE t.order_count > 0;
```

Current performance: 6 seconds — correlated COUNT per user (we will address that in Scenario 24).

Target performance: Same correctness, dramatically better readability; plan parity.

Thinking out loud: 1. "This is primarily a readability rewrite, not a perf one — but readability is a real interview signal, and modern Postgres 12+ usually plans CTEs identically to inlined subqueries." 2. "A CTE names each intermediate step, which makes the query easier to debug, easier to add columns to, and easier to test in isolation." 3. "Older Postgres (pre-12) treated CTEs as optimisation fences — a real perf consideration we will revisit in Scenarios 19 and 20." 4. "For MySQL 8+, SQL Server, and BigQuery, CTEs are inlined and behave like derived tables."

Diagnosis: Nested subqueries hurt human readability without helping performance on modern planners.

Fix SQL:

```
WITH user_order_counts AS (
  SELECT u.id, u.email,
    (SELECT COUNT(*) FROM orders o WHERE o.user_id = u.id) AS order_count
  FROM users u
)
SELECT * FROM user_order_counts WHERE order_count > 0;
```

Why it speeds up: It does not, materially — the win is intent. (To actually speed this up, see Scenario 24 — convert the correlated COUNT to a GROUP BY join.)

What NOT to say: - "CTEs are always faster." — They are usually plan-equivalent; on older Postgres they were slower due to materialisation. - "Nested subqueries are fine; CTEs are noise." — Try maintaining a five-level nested subquery in six months; you will change your mind.

Common follow-up: "Now make this fast — the correlated COUNT is the real bottleneck." — See Scenario 24.

Scenario126 · MATERIALIZED Hint to Force CTE Reuse (Postgres 12+)

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Postman · Topic: Postgres-specific, CTE behaviour

The scenario: A Postgres 14 query references the same expensive CTE three times. The planner inlines it (Postgres 12+ default), causing the expensive query to run three times. Total time: 24 seconds.

```
WITH expensive_aggregation AS (  
    SELECT user_id, SUM(amount) AS total  
    FROM transactions GROUP BY user_id  
)  
SELECT u.id, e.total  
FROM users u  
JOIN expensive_aggregation e ON e.user_id = u.id  
WHERE e.total > 1000  
    OR e.total IN (SELECT total FROM expensive_aggregation WHERE total > 100000)  
    OR u.id IN (SELECT user_id FROM expensive_aggregation WHERE total < 50);
```

Current performance: 24 seconds — the aggregation runs three times.

Target performance: Under 10 seconds with `MATERIALIZED`.

Thinking out loud: 1. "Postgres 12 changed the default — CTEs are now inlined like derived tables. Usually faster, but if the same CTE is referenced N times, inlining causes N evaluations." 2.

"`MATERIALIZED` is the explicit override — Postgres will compute the CTE once, store the result, and reuse it for every reference." 3. "This is exactly the inverse of the older perf complaint where CTEs were always materialised even when reuse did not justify it." 4. "Pre-12 Postgres, all CTEs were materialised by default; we did not need the hint."

Diagnosis: Multiple references to the same expensive CTE are being inlined and re-evaluated. Force materialisation.

Fix SQL:

```
WITH expensive_aggregation AS MATERIALIZED (  
    SELECT user_id, SUM(amount) AS total  
    FROM transactions GROUP BY user_id  
)  
SELECT u.id, e.total  
FROM users u  
JOIN expensive_aggregation e ON e.user_id = u.id  
WHERE e.total > 1000  
    OR e.total IN (SELECT total FROM expensive_aggregation WHERE total > 100000)  
    OR u.id IN (SELECT user_id FROM expensive_aggregation WHERE total < 50);
```

Why it speeds up: The aggregation runs exactly once; results are stored in a temporary structure and reused for the three references. Total CPU drops to roughly 1/3 of the inlined plan.

What NOT to say: - "Inlining is always better." — Not when the CTE is referenced multiple times and is expensive. - "Just paste the subquery three times." — Defeats the purpose of named composition.

Common follow-up: "What if I want the opposite — force inlining?" — See the next scenario, **NOT MATERIALIZED**.

Scenario127 · NOT MATERIALIZED Hint to Let Planner Inline

Tags: Difficulty: Hard · Asked at: Razorpay, Postman, Cred · Topic: Postgres-specific, CTE behaviour

The scenario: A Postgres 11 query (or a Postgres 14 query the team upgraded from 11) has a CTE that filters a large table, then joins. The CTE materialises 50M rows in memory and runs in 33 seconds.

```
WITH recent_transactions AS (  
    SELECT * FROM transactions WHERE created_at >= CURRENT_DATE - 365  
)  
SELECT u.id, u.email, COUNT(*) AS txn_count  
FROM users u  
JOIN recent_transactions r ON r.user_id = u.id  
WHERE u.country = 'IN'  
GROUP BY u.id, u.email;
```

Current performance: 33 seconds on Postgres 11 (forced materialisation), or in Postgres 12+ if **MATERIALIZED** was added by an over-eager engineer.

Target performance: Under 3 seconds via inlining.

Thinking out loud: 1. "Materialising the entire **recent_transactions** set means storing 50M rows in a temp structure before doing any filtering by user country — wasteful." 2. "If the CTE is inlined, the planner can push the **users.country = 'IN'** filter down past the join and pre-filter the transactions scan." 3. "On Postgres 12+, the default is inlining and we do not need a hint. On Postgres 11, the only way to inline is to remove the CTE — or upgrade. The team should upgrade." 4. "If we are on 12+ and someone added **MATERIALIZED**, we want to either remove it or switch to **NOT MATERIALIZED** to make the inline behaviour explicit."

Diagnosis: A CTE that is referenced once and would benefit from predicate pushdown should be inlined, not materialised.

Fix SQL:


```
WITH recent_transactions AS NOT MATERIALIZED (  
    SELECT * FROM transactions WHERE created_at >= CURRENT_DATE - 365  
)  
SELECT u.id, u.email, COUNT(*) AS txn_count  
FROM users u  
JOIN recent_transactions r ON r.user_id = u.id  
WHERE u.country = 'IN'  
GROUP BY u.id, u.email;
```

Why it speeds up: Inlined CTEs let the planner push predicates and combine filters; only Indian users' recent transactions are scanned, not all 50M rows. The materialised temp structure is gone.

What NOT to say: - "Use **MATERIALIZED** for everything to be safe." — Disables the planner's best trick. - "CTEs are always inlined." — Only on Postgres 12+ and only without **MATERIALIZED** .

Common follow-up: "When is **MATERIALIZED** actually the right call?" — Multi-reference CTEs (Scenario 19), recursive CTEs (always materialised), and CTEs whose result is statistically much smaller than the planner believes.

Scenario128 · Window Function in CTE vs Subquery — Planner Behaviour

Tags: Difficulty: Hard · Asked at: Flipkart, Walmart Labs, Postman · Topic: Window functions, query rewrite

The scenario: A query computes a running total per user using a window function in a CTE, then filters to the top day. The CTE materialises 80M rows; the filter is applied at the end. Query takes 41 seconds.

```
WITH daily AS (  
    SELECT user_id, day, amount,  
           SUM(amount) OVER (PARTITION BY user_id ORDER BY day) AS running_total  
    FROM daily_spend  
)  
SELECT * FROM daily WHERE day = CURRENT_DATE - 1;
```

Current performance: 41 seconds — window function processes all rows; filter applied after.

Target performance: Under 3 seconds.

Thinking out loud: 1. "Window functions are computed before WHERE filters that reference other columns — predicates cannot push past a window function partition." 2. "Materialising the CTE forces the entire 80M-row window computation before the WHERE on **day** is applied." 3. "Restructuring as a subquery with the filter inside lets the planner first reduce the input (one day's worth of data), then compute the window on the reduced set." 4. "Be careful: if the window function needs the full history to compute (e.g., a cumulative total), restructuring changes the answer. In that case, pre-aggregate the running total in a daily materialised view."

Diagnosis: Window functions block predicate pushdown across partitions. The CTE materialises an enormous intermediate that the final filter discards.

Fix SQL:

```
-- If "running total as of yesterday" is the goal, pre-aggregate:
WITH cumulative AS (
  SELECT user_id, SUM(amount) AS total_so_far
  FROM daily_spend
  WHERE day <= CURRENT_DATE - 1
  GROUP BY user_id
)
SELECT user_id, total_so_far FROM cumulative;
```

Why it speeds up: GROUP BY does the aggregation directly without a per-row window pass; only relevant days are scanned; no 80M-row intermediate.

What NOT to say: - "Always materialise CTEs with window functions." — Materialisation is exactly what makes this slow. - "Window functions and GROUP BY are interchangeable." — They are not; window keeps row-level detail, GROUP BY collapses it. Pick the one your output requires.

Common follow-up: *"What if I genuinely need the per-row running total preserved for an audit trail?"* — Build a daily materialised view; serve the audit from it. Do not compute it at query time.

Scenario129 · Temp Table Replaced With CTE When Reuse Is Limited

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Cognizant · Topic: Query rewrite

The scenario: A stored procedure CREATES a temp table, INSERTs filtered data, then SELECTs from it once. The DDL and cross-statement overhead costs 4 seconds before the actual query starts.

```
CREATE TEMP TABLE recent_orders AS
SELECT * FROM orders WHERE created_at >= CURRENT_DATE - 7;

SELECT user_id, SUM(amount) FROM recent_orders GROUP BY user_id;
```

Current performance: 4.7 seconds total — 4 seconds is DDL and write overhead.

Target performance: Under 700 ms with a CTE.

Thinking out loud: 1. *"Temp tables have setup cost — CREATE TABLE, INSERT, sometimes statistics collection, and on commit DROP. If the data is referenced only once, all this overhead pays for nothing."* 2. *"A CTE (or even an inlined subquery) achieves the same isolation without the DDL — the optimiser sees both halves and may even push filters."* 3. *"Temp tables make sense when (a) the result is reused 3+ times in subsequent statements, (b) you need an index on the intermediate result, or (c) the planner*

statistics on the live table mislead the optimiser." 4. "For single-use, a CTE is universally faster on every modern database I have measured."

Diagnosis: Temp tables carry DDL overhead that single-use queries never recover.

Fix SQL:

```
WITH recent_orders AS (  
  SELECT * FROM orders WHERE created_at >= CURRENT_DATE - 7  
)  
SELECT user_id, SUM(amount) FROM recent_orders GROUP BY user_id;
```

Why it speeds up: No DDL, no INSERT, no temp-table tracking. The CTE is inlined (Postgres 12+, MySQL 8, BigQuery) or treated as a derived table; the planner picks an index range scan on `orders(created_at)` and aggregates in one pass.

What NOT to say: - "Temp tables are always slower." — They are faster for multi-use intermediates; only single-use is wasteful. - "Temp tables persist across sessions." — They do not; they are session-scoped by default.

Common follow-up: "When would you still pick a temp table?" — Re-use across many follow-up statements, the need for an index on the intermediate, or when the planner makes bad cardinality estimates on the inlined form.

Scenario130 · Aggregate Pushdown — Pre-Aggregate Before Join

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Flipkart · Topic: Query rewrite, aggregate pushdown

The scenario: A query joins `orders` (50M rows) to `users` (40M rows), aggregates orders by user, and returns total spend per user. Joining first then aggregating moves 50M rows through the join; the query takes 26 seconds.

```
SELECT u.id, u.email, SUM(o.amount) AS total_spend  
FROM users u  
JOIN orders o ON o.user_id = u.id  
GROUP BY u.id, u.email;
```

Current performance: 26 seconds — join produces 50M rows, then aggregates down to 40M users.

Target performance: Under 4 seconds.

Thinking out loud: 1. "The join is moving 50M rows through the planner before aggregation. If I pre-aggregate orders per user first (40M result), then join, the join input is much smaller." 2. "This is `aggregate pushdown` — manually performing what some planners do automatically but many do not." 3. "After pre-aggregation, the join is 40M users against 40M user-totals — a balanced hash- or

merge-join, much cheaper than the 40M × 50M shape." 4. *"This rewrite shines when the aggregation reduces row count significantly — multiple orders per user collapsed to one total per user."*

Diagnosis: Joining before aggregating moves more rows than necessary. Pre-aggregate to shrink the join input.

Fix SQL:

```
WITH spend_per_user AS (  
    SELECT user_id, SUM(amount) AS total_spend  
    FROM orders  
    GROUP BY user_id  
)  
SELECT u.id, u.email, s.total_spend  
FROM users u  
JOIN spend_per_user s ON s.user_id = u.id;
```

Why it speeds up: The aggregation is one pass over orders using the `(user_id)` index; the join is then balanced and small. We avoid moving 50M rows through the join machinery.

What NOT to say: - "The planner does this for me." — Sometimes; never count on it for large data. - "Pre-aggregation is always faster." — Only when the aggregation actually reduces row count.

Common follow-up: *"What if some users have no orders?"* — Switch to LEFT JOIN and COALESCE the total to 0.

Scenario131 · Correlated COUNT Converted to GROUP BY JOIN

Tags: Difficulty: Medium · Asked at: Flipkart, Meesho, Cred · Topic: Query rewrite

The scenario: A dashboard query counts orders per user using a correlated subquery in SELECT. For 500k users, the correlated count runs 500k times and takes 39 seconds.

```
SELECT u.id, u.email,  
       (SELECT COUNT(*) FROM orders o WHERE o.user_id = u.id) AS order_count  
FROM users u;
```

Current performance: 39 seconds.

Target performance: Under 2 seconds.

Thinking out loud: 1. *"This is the same N+1 pattern from Scenarios 3 and 6, specialised to COUNT. 500k correlated COUNTs is 500k separate index scans."* 2. *"A single `GROUP BY user_id` over orders produces all counts in one pass; left-join back to users to include zero-count users."* 3. *"`LEFT JOIN` plus `COALESCE(count, 0)` preserves users with zero orders — the correlated subquery returned 0*

for those automatically, and the rewrite must match that behaviour." 4. "Same shape applies to SUM, AVG, MIN, MAX — any correlated aggregate is a candidate for this rewrite."

Diagnosis: Correlated aggregates in SELECT are N+1 query patterns. Aggregate once and join.

Fix SQL:

```
WITH counts AS (  
  SELECT user_id, COUNT(*) AS order_count  
  FROM orders GROUP BY user_id  
)  
SELECT u.id, u.email, COALESCE(c.order_count, 0) AS order_count  
FROM users u  
LEFT JOIN counts c ON c.user_id = u.id;
```

Why it speeds up: One pass over orders aggregates everything; one hash-join applies counts to users. 500k correlated scans collapse into two streamed operations. 20x or better is typical.

What NOT to say: - "Correlated subqueries are sometimes faster." — For tiny outer sets, occasionally; for 500k rows, never. - "Just drop users with zero orders." — Changes the result; the LEFT JOIN preserves them.

Common follow-up: "What if I need three different aggregates — count, sum, max — per user?" — Compute all three in the same GROUP BY in the CTE; the cost is shared.

Scenario132 · UNION ALL of Indexed Paths Instead of OR

Tags: Difficulty: Medium · Asked at: Swiggy, Zomato, BookMyShow · Topic: Query rewrite

The scenario: A query searches messages where the user is either the sender or the recipient. Both columns are indexed individually. The `OR` query falls back to a sequential scan and takes 12 seconds.

```
SELECT id, subject, body  
FROM messages  
WHERE sender_id = 42 OR recipient_id = 42;
```

Current performance: 12 seconds — neither single-column index can be used cleanly with `OR`.

Target performance: Under 80 ms.

Thinking out loud: 1. "This is the same shape as Scenario 8, but with the role of dedup mattering more — a message could have the same user as both sender and recipient (a draft to self) and we may or may not want to dedup." 2. "Splitting into two UNIONed queries lets each branch use its index — `messages(sender_id)` for one, `messages(recipient_id)` for the other." 3. "`UNION ALL` is correct if duplicates are impossible (no self-messages) or acceptable. `UNION` deduplicates at extra cost

— usually negligible for the output volume here." 4. "On Postgres, BitmapOr can sometimes combine the two indexes automatically — but only when statistics align. Manual UNION is more reliable."

Diagnosis: `OR` across independently-indexed columns blocks single-index plans on most planners. Manual UNION forces each branch to use its index.

Fix SQL:

```
SELECT id, subject, body FROM messages WHERE sender_id = 42
UNION
SELECT id, subject, body FROM messages WHERE recipient_id = 42;
```

Why it speeds up: Each branch is a fast indexed lookup, typically returning a few dozen rows; the UNION dedup is over a tiny combined set. 12-second seq scan becomes two B-tree probes.

What NOT to say: - "Create a composite index `(sender_id, recipient_id)` ." — Composite is only useful when both are filtered together, not in an OR. - "Modern planners handle this." — They sometimes do; manual UNION is portable and guarantees the plan.

Common follow-up: "What if I want messages where the user is either sender OR recipient, but ordered by timestamp?" — UNION the two paths, then wrap in an outer ORDER BY with LIMIT; or use a UNION ALL + dedup + sort, depending on data volume.

Scenario133 · LIMIT Pushdown for Early Termination

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Flipkart · Topic: Query rewrite, LIMIT semantics

The scenario: A query fetches the 10 most recent orders for each of 1000 VIP users. The current query joins, sorts the full join result, then LIMITs — 38 seconds for 80M total orders scanned.

```
SELECT o.* FROM orders o
JOIN vip_users v ON v.user_id = o.user_id
ORDER BY o.created_at DESC LIMIT 10;
```

Current performance: 38 seconds — full join, then global sort, then limit.

Target performance: Under 1 second.

Thinking out loud: 1. "The current shape forces the planner to sort $1000 \times 80k = 80M$ rows globally to find the top 10. The LIMIT only saves output, not work." 2. "If the goal is genuinely just the top 10 across all VIP users, push the LIMIT into a per-user subquery — top 10 per user — then take the top 10 overall. The per-user index range scan can terminate early." 3. "Better yet, use LATERAL (Scenario 10) for per-user top-N; then take top 10 of the unioned result." 4. "The general lesson: LIMIT only helps when the

planner can stop work early — that usually requires the data to arrive in the sort order, ideally via an index."

Diagnosis: A global LIMIT after a join with no per-user reduction wastes work. Push the LIMIT down to a per-key subquery.

Fix SQL:

```
WITH top_per_vip AS (  
  SELECT o.*  
  FROM vip_users v  
  CROSS JOIN LATERAL (  
    SELECT * FROM orders WHERE user_id = v.user_id  
    ORDER BY created_at DESC LIMIT 10  
  ) o  
)  
SELECT * FROM top_per_vip  
ORDER BY created_at DESC LIMIT 10;
```

Why it speeds up: Each VIP user contributes at most 10 rows via an index-driven early-exit; the outer LIMIT then sorts at most 10,000 rows. 80M-row sort becomes a 10k-row sort plus 1000 small index lookups.

What NOT to say: - "LIMIT always saves work." — Only when it can terminate the underlying scan early.
- "Just add `LIMIT 10` and hope." — The planner cannot help if the data is not pre-sorted.

Common follow-up: "What if I want top 10 per VIP user, not top 10 overall?" — Drop the outer LIMIT; the LATERAL already produces top 10 per user.

Scenario134 · OFFSET Pagination Replaced With Keyset (Cursor) Pagination

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Flipkart, Zomato · Topic: Query rewrite, pagination

The scenario: An admin panel shows orders, 25 per page, with `ORDER BY created_at DESC LIMIT 25 OFFSET 9975` for page 400. On a 40M-row table the page load takes 14 seconds and gets slower with every page.

```
SELECT id, user_id, amount, created_at  
FROM orders  
ORDER BY created_at DESC  
LIMIT 25 OFFSET 9975;
```

Current performance: 14 seconds on page 400; page 4000 would be effectively unusable.

Target performance: Under 100 ms regardless of page depth.

Thinking out loud: 1. "OFFSET makes the database fetch and discard the first N rows. On page 400 of 25, that is 9,975 rows fetched, sorted, and thrown away before delivering 25." 2. "The cost grows linearly with page depth — page 4,000 means discarding 99,975 rows. OFFSET pagination is unscalable for large datasets." 3. "Keyset pagination uses the last-seen value as the next page's starting point. `WHERE (created_at, id) < (last_created_at, last_id) ORDER BY created_at DESC, id DESC LIMIT 25` is a single index range scan — constant cost regardless of page." 4. "Tradeoff: keyset cannot jump to an arbitrary page number; it only supports `next` and `previous`. For admin tools where most users go page by page, that is fine. For ones with a 'jump to page 400' input, hybrid solutions exist."

Diagnosis: OFFSET pagination is $O(\text{offset})$ per page. Keyset (cursor) pagination is $O(1)$ per page when an index supports the ordering.

Fix SQL:

```
-- Page 1
SELECT id, user_id, amount, created_at
FROM orders
ORDER BY created_at DESC, id DESC
LIMIT 25;

-- Page 2 onward — pass last row's (created_at, id) from previous page
SELECT id, user_id, amount, created_at
FROM orders
WHERE (created_at, id) < ('2026-04-12 10:33:00', 871334)
ORDER BY created_at DESC, id DESC
LIMIT 25;
```

Why it speeds up: The `(created_at, id) < (...)` predicate is sargable against an index on `(created_at DESC, id DESC)` — a single B-tree seek to the cursor position, then read 25 rows. No rows are fetched and discarded. Page 1 and page 4,000 cost identically.

What NOT to say: - "OFFSET is fine if you do not have many pages." — A polite way to say it does not scale; senior interviewers will press on what "many" means. - "Add a covering index and OFFSET will be fast." — A covering index helps the per-row cost but does not change the $O(\text{offset})$ shape.

Common follow-up: "How would you support 'jump to page 400'?" — Either accept the OFFSET cost for direct navigation (rare), or precompute a `(page_number, first_id)` map periodically, or design the UX around continuous scroll. Most modern systems pick the third.

Category 2, Part C — Optimize This Query: Plan-Driven & Infra Fixes

Part C is the "read the plan, change the strategy" layer — the questions that show up the moment an interviewer realises you can write an index and rewrite a subquery, and now wants to know whether you

understand the layer below. Join algorithm choice, statistics, partitioning, materialisation, parallel execution, connection pooling, replicas, sharding — these are the dials senior product co panels and GCC SRE rounds love to twist. Part A covered index and rewrite-level fixes, Part B covered the row-shape and aggregate-shape fixes, and Part C is where the database starts to look less like a query engine and more like a distributed system. Twenty-six scenarios, each one anchored in a realistic Indian production setup, with planner behaviour and dialect call-outs flagged honestly — because the same query will pick a hash join in Postgres, a merge join in SQL Server, and a nested loop in MySQL on the same data.

Scenario135 · Stale Statistics Forcing the Wrong Plan

Asked at: Razorpay · Walmart Labs · ThoughtSpot · **Difficulty:** Medium · **Topic:** Statistics · **Dialect:** Postgres · SQL Server (vendor-agnostic principle)

The interviewer's prompt: *"Your nightly batch job loads 8 crore rows into `orders` every day. The morning report query that ran in 12 seconds yesterday is now taking 9 minutes. Same SQL, same data shape. What's the first thing you check?"*

Thinking out loud (the diagnosis path): 1. "Same SQL, same indexes, same data volume — but a wildly different runtime means the planner picked a different plan. The most common reason for a plan flip after a bulk load is stale statistics. The planner saw row counts from before the load and made a join-order choice that no longer fits." 2. "I'd pull the plan first — `EXPLAIN (ANALYZE, BUFFERS)` in Postgres or the `Actual Execution Plan` in SQL Server — and compare estimated rows vs actual rows on each operator. If estimated says 12,000 and actual says 80,00,000, statistics are stale." 3. "For a bulk-loaded table, autovacuum or auto-update-stats may not have fired yet. After 8 crore rows of churn the sample is meaningless."

The likely root cause: The planner used row estimates from before the nightly load and chose a nested-loop join on what it thought was a small driving table. After the load, that "small" table is now huge, and the nested loop is doing crores of inner-side lookups.

The fix (and the verification step): - Postgres: run `ANALYZE orders;` immediately after the load completes. For a permanent fix, add `ANALYZE` as the last step of the ETL job, not as a background task. - SQL Server: `UPDATE STATISTICS dbo.orders WITH FULLSCAN;` for critical tables; `WITH SAMPLE 30 PERCENT` if FULLSCAN is too slow. - MySQL 8: `ANALYZE TABLE orders;` — same intent. - Verify by re-running the query and confirming the estimated row counts in EXPLAIN now match actual within an order of magnitude.

What NOT to say: - "Just rebuild the indexes — that updates stats too." — Rebuilding indexes only updates index stats in some engines, and it is a far heavier operation than ANALYZE. Wrong tool. - "Stats update automatically, so this can't be the issue." — Autovacuum thresholds are percentage-based; after a fresh load on an empty table they may not trigger until the next day.

Common follow-up: "How would you set up monitoring to catch stale stats before they cause a slow query in production?" — Postgres: `pg_stat_user_tables.last_analyze` vs `n_mod_since_analyze` ; SQL Server: `sys.dm_db_stats_properties` ; alert when modification count crosses 10% of row count.

```
-- Postgres: after the nightly load, always
ANALYZE orders;

-- SQL Server: critical-table refresh
UPDATE STATISTICS dbo.orders WITH FULLSCAN;

-- Diagnostic: which tables have stale stats?
SELECT schemaname, relname, n_live_tup, n_mod_since_analyze, last_analyze
FROM pg_stat_user_tables
WHERE n_mod_since_analyze > n_live_tup * 0.1
ORDER BY n_mod_since_analyze DESC;
```

Scenario136 · Extended Statistics for Correlated Columns

Asked at: PhonePe · Zomato · Atlassian GCC · **Difficulty:** Hard · **Topic:** Statistics · **Dialect:** Postgres (10+) primary · SQL Server has filtered statistics analogue

The interviewer's prompt: "You have a `customers` table with `state` and `pincode`. You filter `WHERE state = 'Maharashtra' AND pincode LIKE '400%'`. The planner estimates 30 rows; actual is 3 lakh. Index is not used, query is slow. Why is the estimate so wrong?"

Thinking out loud (the diagnosis path): 1. "Classic correlated-column problem. The planner multiplies selectivities assuming independence — Maharashtra times pincodes starting 400. But pincodes starting 400 are almost entirely in Maharashtra, so the columns are heavily correlated. The independence assumption breaks." 2. "This underestimate makes the planner think it has 30 rows, so it picks a nested loop or skips the bitmap index in favour of a sequential scan. It actually finds 3 lakh rows and does 3 lakh inner-side lookups — the slow path we see." 3. "The fix is to tell the planner about the correlation. Postgres has extended statistics since version 10 for exactly this case." 4. "I would also confirm the same misestimate is not corrupting downstream join cardinality — a wrong number at the bottom of the plan tree cascades upward into wrong join algorithm choices."

The likely root cause: Per-column statistics assume independence. `state` is roughly 1/30 selective, `pincode LIKE '400%'` is roughly 1/600 selective. Multiplied, that gives ~1/18,000 — hence the 30-row estimate. Reality: `pincode` and `state` are functionally dependent because Indian PIN codes are state-prefixed by design — 4xxxxx is Maharashtra, 5xxxxx is Karnataka/Andhra/Telangana, 7xxxxx is West Bengal, and so on.

The fix (and the verification step): - Postgres 10+: create extended statistics. `sql CREATE STATISTICS customers_state_pin (dependencies, ndistinct) ON state, pincode FROM customers; ANALYZE customers;` The `dependencies` kind tells the planner about

functional dependencies; `ndistinct` improves group-by estimates across the combined columns. - SQL Server: there is no direct equivalent, but filtered statistics on common state values can help — `CREATE STATISTICS s_pin_mh ON customers(pincode) WHERE state = 'Maharashtra';` . - MySQL: no extended-stats mechanism; either rewrite the query to expose only the more selective predicate, denormalise into a precomputed region column, or accept the bad plan and force an index hint. - Verify with `EXPLAIN` that the estimated row count is now within ~2x of actual, and that the planner switches from a sequential scan to a bitmap index scan or index range scan.

What NOT to say: - *"Increase the statistics target so the histogram is finer."* — Helps for single-column skew, not for cross-column correlation. The histogram is per column. - *"Rewrite as `pincode LIKE '400%'` only since it implies state."* — You lose the semantic check, and it does not fix the underlying planner problem for other correlated pairs (date and quarter, city and state, brand and category). - *"Just disable the index — sequential scan with the right filter is fine at this size."* — 3 lakh rows out of millions is exactly where a bitmap index scan wins; abandoning the index is wasteful.

Common follow-up: *"What kinds of extended statistics does Postgres support and when do you use each?"* — `ndistinct` for GROUP BY estimates over multiple columns, `dependencies` for functional dependency between columns (this scenario), `mcv` (Postgres 12+) for most-common-value lists across multiple columns when the data is skewed toward a handful of combinations.

Scenario137 · Sort Spilling to Disk — Raise `work_mem`

Asked at: Razorpay · Cred · Postman · **Difficulty:** Medium · **Topic:** Memory tuning · **Dialect:** Postgres

The interviewer's prompt: *"A report query that sorts 50 lakh rows is taking 3 minutes. EXPLAIN ANALYZE shows `Sort Method: external merge Disk: 480MB`. The box has 64 GB of RAM. What's happening and how do you fix it?"*

Thinking out loud (the diagnosis path): 1. *"`external merge Disk` means the sort did not fit in `work_mem` and spilled to a temp file. Disk sorts are 10 to 50 times slower than in-memory sorts, depending on the storage. So the cure is to let the sort live in memory."* 2. *"`work_mem` is per sort or hash operation, per backend — not per query, not per server. Default is 4 MB. The box has 64 GB so we have plenty of headroom on the machine itself."* 3. *"I would not raise it globally to 1 GB — that multiplies across every concurrent operation. A query with three sorts and two hash joins uses five times `work_mem`. With 200 connections each doing five operations, the math gets scary fast."* 4. *"The right pattern is to raise it for this session, this role, or this query class only — keep the global modest."*

The likely root cause: Default `work_mem = 4MB` is far too small for an analytics workload sorting 480 MB of data — the sort cannot possibly fit, so Postgres spills batches to temp files and merges them, paying disk I/O cost for what should be a CPU-only operation.

The fix (and the verification step): - Per-session for the report: `SET work_mem = '512MB';`
`SELECT ... ORDER BY ...;` - Per-role for the BI user: `ALTER ROLE bi_user SET work_mem = '512MB';` — applies to every session that user opens. - Per-statement scope inside a transaction:

`BEGIN; SET LOCAL work_mem = '512MB'; SELECT ...; COMMIT;` - Globally only after capacity math: $\text{work_mem} \times \text{max_connections} \times \sim 2 (\text{sort} + \text{hash per query})$ must fit in available RAM with headroom for OS cache and shared_buffers. On a 64 GB box with 200 connections, that caps global work_mem at around 30 MB if you want safety. - Verify with `EXPLAIN ANALYZE` — the sort line should now read `Sort Method: quicksort Memory: 480MB` and the query should drop from 3 minutes to under 10 seconds. If it still spills, double the setting.

What NOT to say: - "Set `work_mem` to 4GB globally so this never happens again." — Will blow up the box the moment 50 BI users connect simultaneously and each runs a multi-sort query. - "Disk sorts are fine on SSDs." — Even on NVMe, in-memory sort wins by an order of magnitude; the syscall and copy overhead dominates over raw read latency. - "Use a temp table to break up the sort." — Does not help; the temp table itself needs a sort to populate, and you still spill.

Common follow-up: "How do you decide per-role vs per-statement vs global for `work_mem`?" — Per-statement when the query is one-off and you want isolation; per-role for a known workload class (analyst user, BI tool service account); global only when the workload mix is genuinely uniform — rare in practice.

Scenario138 · Plan Regression After a Postgres Minor Upgrade

Asked at: Swiggy · Flipkart · Atlassian GCC · **Difficulty:** Hard · **Topic:** Plan diagnosis · **Dialect:** Postgres (pg_stat_statements) · SQL Server (Query Store)

The interviewer's prompt: "You bumped Postgres from 15.4 to 15.6 on Saturday. By Monday, two queries that used to take 200 ms are taking 8 seconds. Same data, same indexes. Walk me through the diagnosis."

Thinking out loud (the diagnosis path): 1. "A minor version bump should not change plans in theory, but in practice it sometimes does — planner bug fixes, costing-model tweaks, or a stats refresh during the upgrade can flip a plan sitting at the cost threshold." 2. "I would lean on `pg_stat_statements` to find offending queries by `mean_exec_time` jump, then capture the current plan with `EXPLAIN (ANALYZE, BUFFERS)` and compare to a known-good plan from staging or a pre-upgrade snapshot." 3. "On SQL Server I would do the same with Query Store — it stores plan history per query and you can force the old plan with one click. Postgres makes you work harder; SQL Server gives you a button." 4. "Before changing anything, I want both plans side by side and an explicit understanding of which operator changed. Otherwise you are guessing."

The likely root cause: A statistics refresh during the upgrade re-sampled histograms; or a small costing change tipped the planner from a nested loop to a merge join (or the other way) for a query sitting right at the cost threshold; or a partial index that the new planner version now costs slightly differently and refuses to pick.

The fix (and the verification step): - First identify: `SELECT queryid, mean_exec_time, calls FROM pg_stat_statements ORDER BY mean_exec_time * calls DESC LIMIT 20;` (reset stats first with `pg_stat_statements_reset()` after the upgrade to compare clean windows). -

Postgres: there is no first-class "force this plan" — but you can shape it. Try `ANALYZE` the affected tables first; a stats refresh often realigns the planner. If that does not fix it, install the `pg_hint_plan` extension and pin a join order or operator. Last resort: rewrite the query slightly to nudge the planner — adding a redundant `OFFSET 0` in a subquery, for instance, disables some optimisations. - SQL Server: open Query Store, find the regression, click "Force Plan" on the old one. This is a genuine emergency button — use it during the incident and remove the forcing once the root cause is fixed. - Verify by re-running and capturing the plan; mean execution time should drop back to baseline within a few minutes.

What NOT to say: - *"Roll back the minor version."* — Drastic and rarely necessary; minor versions ship security patches you want and the regression usually has a per-query fix. - *"Just restart Postgres."* — Does not change the plan choice; only clears caches that may briefly help and then re-degrade. - *"Disable autovacuum so stats stop changing."* — Catastrophic for table bloat; addresses a symptom by causing a much worse problem.

Common follow-up: *"Why does SQL Server let you force a plan but Postgres does not?"* — Different philosophy. SQL Server bets on operator stability for OLTP and gives DBAs explicit plan-pinning tooling; Postgres bets that the planner with current stats is usually right and gives you `pg_hint_plan` only as an out-of-tree escape hatch. Neither view is wrong.

Scenario139 · Forcing a Plan via Hint — When It Is Justified

Asked at: Walmart Labs · Goldman Sachs GCC · **Difficulty:** Hard · **Topic:** Hints · **Dialect:** SQL Server (OPTION) · MySQL (USE/FORCE INDEX) · Postgres (`pg_hint_plan` extension)

The interviewer's prompt: *"You are debugging a slow query. The planner picks a hash join, but a nested loop is 10x faster on this data. Should you force the plan with a hint? When is it justified and when is it a code smell?"*

Thinking out loud (the diagnosis path): 1. *"Hints are escape hatches, not first-line tools. If a hint is the only thing between production and an outage, use it — but log a follow-up ticket the same day, because the underlying planner mismatch will bite again on a different query."* 2. *"Legitimate cases are narrow: a known-stable plan for a hot OLTP query where data shape is predictable for years, or a documented workaround for a planner bug the vendor has not patched."* 3. *"Illegitimate cases are far more common in production codebases: hiding a stats problem, hiding a missing index, papering over a correlated-column misestimate, or 'we always did it this way' tribal knowledge nobody dares touch."* 4. *"My default in an interview — investigate root cause first, hint as last resort, always with a comment explaining when to re-evaluate."*

The likely root cause (of the wrong plan, not of the question): A row-count misestimate driving the cost model toward the wrong join algorithm — usually either stale stats or correlated columns that defeat the independence assumption.

The fix (and the verification step): - Step 1: try to fix the root cause first. `ANALYZE`, create extended statistics, add the missing index, rewrite a correlated predicate. Hints are step 5, not step 1. - Step 2: if a

hint is truly needed in the moment: - SQL Server: `OPTION (LOOP JOIN)` or `OPTION (FORCE ORDER, USE PLAN N'...')` for full plan-forcing. - MySQL: `SELECT /*+ JOIN_ORDER(a, b) */ ...` or `USE INDEX (idx_x) / FORCE INDEX (idx_x)`. - Postgres: install `pg_hint_plan` extension, then `/*+ HashJoin(a b) */` or `/*+ NestLoop(a b) Leading((a b)) */`. - Step 3: leave a comment in the SQL explaining why the hint is needed and what would let you remove it — "Stats correlation between region and pincode; remove when extended stats added in #JIRA-1234." Future-you will thank present-you. - Verify post-hint latency matches expectation, and add a monitor that alerts if the same query slows again — that is your signal that the hint has become wrong.

What NOT to say: - "Add a hint everywhere — predictable plans are good." — Hints freeze the plan against data growth; what was right at 1 lakh rows is wrong at 10 crore. The planner adapts; a hint does not. - "Hints are always bad." — Sometimes they are the right tool, and a senior interviewer wants to hear nuance. Black-and-white answers signal lack of production experience. - "Force the plan in Query Store and forget about it." — Forcing without fixing the underlying problem is technical debt with compound interest.

Common follow-up: "How would you know to revisit and remove a hint?" — Track query latency post-hint; when underlying stats or schema changes (column added, partitioning introduced, data grows 10x, new index added), re-test without the hint and confirm the planner now picks well. Some teams put a "remove by" date in the hint comment as a forcing function.

Scenario140 · Parallel Query Enablement

Asked at: Razorpay · PhonePe · Walmart Labs · **Difficulty:** Medium · **Topic:** Parallelism · **Dialect:** Postgres · SQL Server (MAXDOP) · MySQL 8 (limited)

The interviewer's prompt: "Your analytical query scans 5 crore rows and runs serially in 4 minutes. The box has 16 cores. How do you get the planner to use them?"

Thinking out loud (the diagnosis path): 1. "Parallel query is on by default in Postgres 10+ and SQL Server, but the planner only picks it if estimated cost crosses a threshold and a handful of setup parameters are configured to allow it. On managed services like AWS RDS and DigitalOcean, the defaults are sometimes conservative." 2. "In Postgres, the relevant knobs are `max_parallel_workers_per_gather` (how many workers per Gather node), `max_parallel_workers` (total system-wide), `min_parallel_table_scan_size` (when scanning is worth parallelising), and the cost factors `parallel_setup_cost` / `parallel_tuple_cost` that the planner weighs." 3. "I would check `EXPLAIN` first — if there is no `Gather` or `Gather Merge` node, the planner did not even consider parallelism. That points to either a setting blocking it or a query construct that disables it (function safety, locking, CTE materialisation)." 4. "After enabling, I would also watch CPU utilisation during the query — if I see one core pegged and others idle, parallelism is configured but not actually engaging at execution time."

The likely root cause: `max_parallel_workers_per_gather = 0` (default in some managed services), or the table is too small to clear `min_parallel_table_scan_size = 8MB`, or the query uses a parallel-unsafe function — covered in detail in the next scenario.

The fix (and the verification step): - Postgres:

`SET max_parallel_workers_per_gather = 4; SET max_parallel_workers = 8;` then re-EXPLAIN. Lower `parallel_setup_cost` (default 1000) if the planner still rejects parallel. For a 16-core box, the typical sizing is `max_parallel_workers = 8` and `max_worker_processes = 16`. - SQL Server: `OPTION (MAXDOP 8)` on the query, or set instance-level `max degree of parallelism`. Watch the `cost threshold for parallelism` (default 5 — way too low for modern hardware; most DBAs raise it to ~50 so OLTP queries do not get parallelised). - MySQL 8: limited — parallel only for read scans of partitioned tables and InnoDB cluster index reads. For real analytical parallelism you usually need a different engine (Snowflake, BigQuery, ClickHouse). - Verify with EXPLAIN — you should see `Gather` over `Parallel Seq Scan` or `Parallel Index Scan` in Postgres; query time should drop roughly N-fold where N is worker count, until I/O bandwidth becomes the bottleneck (usually around 4-6 workers on a single disk).

What NOT to say: - *"Just set workers to 32 — more is always better."* — Each worker has setup overhead and competes for I/O bandwidth; sweet spot is usually 4-8 for I/O-bound, more for CPU-bound aggregations. - *"Parallel query speeds up OLTP."* — It speeds up scans on large data; OLTP point lookups gain nothing and may pay the parallel-setup cost for no benefit. - *"Increase max_parallel_workers to 64 across the board."* — Causes context-switching storms when many users run analytical queries concurrently; the server thrashes.

Common follow-up: *"What kinds of queries cannot go parallel in Postgres?"* — Queries calling parallel-unsafe functions, queries inside a CTE that the planner materialises non-parallel, queries acquiring row locks (SELECT FOR UPDATE), and write paths (INSERT/UPDATE/DELETE — parallel writes only landed in Postgres 14 for the apply phase).

Scenario141 · Parallel-Safe vs Parallel-Restricted Functions

Asked at: Atlassian GCC · ThoughtSpot · **Difficulty:** Hard · **Topic:** Parallelism · **Dialect:** Postgres primary

The interviewer's prompt: *"You added parallel workers but the planner still runs the query serially. The only oddity is that the query calls a custom function `format_invoice_number(id)`. Why might that disable parallelism?"*

Thinking out loud: 1. *"In Postgres, every function has a parallel safety marker — `parallel safe`, `parallel restricted`, or `parallel unsafe`. The default for user-defined functions is `parallel unsafe`, the most conservative, because Postgres cannot introspect the body and prove safety — it assumes the worst."* 2. *"If the planner sees even one `parallel unsafe` function anywhere in the query, it disables parallelism for the whole plan. A single innocuous helper poisons the whole query."* 3. *"The fix is to mark the function correctly — if it is safe in parallel workers, declare it so. The marker is a promise from the developer to the planner; declare carefully."* 4. *"I would also audit*

nearby UDFs in the same query class — formatters, currency converters, geo-coders. They tend to live together in a `utils` schema and were created with the same wrong default."

The likely root cause: The custom function was created without the `PARALLEL SAFE` clause, so Postgres assumes it might call something that does not work in parallel workers — sequence touches, temp tables, advisory locks, prepared statement state, listen/notify. A formatter that just does string concatenation is almost certainly safe, but you have to say so explicitly.

The fix: - If the function is genuinely pure: `ALTER FUNCTION format_invoice_number(int) PARALLEL SAFE;` — and ideally also mark it `IMMUTABLE` if its output depends only on inputs, which unlocks further planner optimisations like expression-result caching. - If it touches sequences or temp objects but is otherwise read-only: `PARALLEL RESTRICTED` — the function can run in the leader process but not in worker processes, which still lets the planner parallelise scans even if the leader does the formatting at the top. - Re-run EXPLAIN — `Parallel Seq Scan` should now appear under a `Gather` node, and elapsed time should drop to roughly serial-time / worker-count.

What NOT to say: - "Mark every function as `PARALLEL SAFE` to be safe." — That hides bugs; an unsafe function called from a worker can produce wrong results because workers do not share session state with the leader (advisory locks, temp tables, sequence nextval order). - "This only affects Postgres." — True that other engines do not use this exact mechanism, but SQL Server has similar restrictions around scalar UDFs that prevent parallelism (the inline scalar UDFs feature in 2019+ helps), and Oracle has its own pragma for this. - "Recreate the function — defaults will be right now." — The default is still `PARALLEL UNSAFE` in current Postgres; recreating with the same DDL changes nothing.

Common follow-up: "What makes a function genuinely parallel-unsafe?" — Writes (INSERT/UPDATE/DELETE inside the function body), advisory locks (`pg_advisory_lock`), sequence nextval, temp table creation, listen/notify, anything that mutates non-transactional state. A good rule of thumb: if the function would behave differently when called twice in the same transaction from different processes, it is unsafe.

Scenario142 · Partition Pruning — Make Sure the Planner Can Prune

Asked at: Flipkart · Razorpay · Walmart Labs · **Difficulty:** Medium · **Topic:** Partitioning · **Dialect:** Postgres · SQL Server · vendor-agnostic principle

The interviewer's prompt: "You partitioned `orders` by month. The query `WHERE created_at = '2026-04-15'` is still scanning every partition. Why?"

Thinking out loud: 1. "Partition pruning works only when the planner can prove at plan time which partitions are relevant. If the predicate uses a function on the partition key, or compares against a parameter from a non-pruned source, pruning is disabled and every partition is scanned — defeating the whole point." 2. "Common pruning killers: `WHERE DATE(created_at) = '2026-04-15'` (function destroys the constraint-exclusion match), `WHERE created_at = $1` in a prepared statement that planned generically, or a join condition where the partition key is on the inner side of a nested loop." 3. "I

would *EXPLAIN* and check the number of partitions touched — if it shows all 36 monthly partitions in an `Append` node instead of one, pruning is off. The plan literally enumerates each partition, very visual." 4. "Also check whether timezone semantics hide a function — `WHERE created_at AT TIME ZONE 'IST' = '...'` is sneaky because the conversion is a function on the partition key."

The likely root cause: Either a wrapping function (`DATE()` , `CAST()` , `AT TIME ZONE`) on the partition key that prevents Postgres from matching the predicate against the partition boundary expression, or a prepared statement that planned generically before the parameter value was known.

The fix: - Rewrite to expose the raw partition key: `WHERE created_at >= '2026-04-15' AND created_at < '2026-04-16'` instead of `WHERE DATE(created_at) = '2026-04-15'` . The half-open range matches the partition boundary, so constraint exclusion fires. - For prepared statements, Postgres has run-time partition pruning since v11 — confirm `enable_partition_pruning = on` and look for `Subplans Removed: N` in *EXPLAIN ANALYZE*. - For joins, ensure the partition key is on the outer side or partition-wise join is enabled (next scenario). - Verify *EXPLAIN* — only one partition should show in the plan tree.

What NOT to say: - "Partitioning automatically makes queries fast." — Only if predicates align with the partition key; otherwise it adds overhead without read benefit. - "Pruning happens at the storage layer." — It happens at planner (plan-time) or executor (run-time); storage just stores per-partition files. - "Use a global non-partitioned index to fix this." — Postgres does not support global indexes across partitions; even where vendors offer it, maintenance cost during partition swap usually outweighs the gain.

Common follow-up: "Plan-time vs run-time pruning?" — Plan-time happens before execution and is visible in *EXPLAIN* as fewer partitions; run-time happens during execution for parameterised plans and shows up as `Subplans Removed: N` . Pre-v11 Postgres only did plan-time, which made prepared statements painful — run-time pruning was a major fix.

Scenario143 · Partition-Wise Join in Postgres

Asked at: PhonePe · Swiggy · ThoughtSpot · **Difficulty:** Hard · **Topic:** Partitioning · **Dialect:** Postgres (11+)

The interviewer's prompt: "You have `orders` and `order_items` , both partitioned by `order_month` with matching boundaries. Joining them takes forever because Postgres joins the whole tables instead of partition-by-partition. How do you fix this?"

Thinking out loud: 1. "This is exactly what partition-wise join is for — if two tables have matching partition schemes, the planner can join partition-1 with partition-1, partition-2 with partition-2. Without it, the planner unions all partitions of both tables, then joins the two giant unions — losing the partitioning benefit and forcing a hash join over the full data size." 2. "Partition-wise join is off by default in Postgres because it can explode plan time on tables with hundreds of partitions. The planner evaluates join strategies per partition pair, so planning time grows with partition count." 3. "Enable it when the workload benefits — typically BI and reporting workloads joining two large partitioned fact tables along the same

axis." 4. "Confirm partition keys match exactly in name, type, and boundary expressions — Postgres requires structural equivalence. A partition by `order_date` and one by `order_month` will not co-partition even if semantically aligned."

The likely root cause: `enable_partitionwise_join = off` (the default for new clusters), so the planner unions partitions before joining instead of joining partition-by-partition.

The fix: - `SET enable_partitionwise_join = on;` (session or per-role for the BI workload). Setting it cluster-wide is fine if your workload is dominated by partitioned analytical queries and few partitioned tables have hundreds of partitions. - For aggregates over the same partition key: `SET enable_partitionwise_aggregate = on;` . This lets the planner aggregate per-partition before combining, which is a huge win for `GROUP BY` over the partition key. - Confirm partition schemes match exactly — same column, same boundaries, same partitioning strategy (range vs list). A mismatch silently falls back to the un-partitioned join, with no warning. - Verify with EXPLAIN — you should see one `Append` containing multiple per-partition `Hash Join` (or `Nested Loop`) nodes, instead of one big `Hash Join` over two `Append` nodes. Per-partition working sets are smaller, so each join fits in `work_mem` and avoids spilling.

What NOT to say: - "Enable globally — it is free." — It can blow up planning time on tables with many partitions; turn on per workload and verify planning time stays acceptable. - "Works in MySQL and SQL Server the same way." — SQL Server has partition-wise via colocation but the planner heuristics and required matching are different; MySQL does not have true partition-wise join in InnoDB, only partition pruning. - "Just write per-partition queries and UNION them manually." — Defeats the abstraction and creates maintenance nightmare when you add or drop partitions; the planner's job is to do exactly this for you.

Common follow-up: "Why is partition-wise join off by default in Postgres?" — Planning cost grows roughly linearly with partition count because the planner has to consider join algorithms per partition pair, and for short OLTP queries the planning overhead outweighs the execution benefit. A good follow-on is to mention that for analytical workloads with daily or monthly partitions (12-36 of them), the planning cost is trivial relative to query time, so enabling it pays off; for tables with thousands of partitions, you re-evaluate.

Scenario144 · Materialised View for Repeated Heavy Query

Asked at: Razorpay · Cred · Flipkart · **Difficulty:** Medium · **Topic:** Materialisation · **Dialect:** Postgres (MATERIALIZED VIEW) · SQL Server (indexed view) · MySQL (manual table)

The interviewer's prompt: "Your dashboard runs the same 12-second aggregation every page load — `SUM(amount) per merchant per day` over 6 months of orders. 200 dashboard users. How do you stop re-running this every time?"

Thinking out loud: 1. "Textbook materialised view case. The aggregation is expensive (a full 6-month scan of crores of orders), the result is small (a few lakh merchant-day rows), data changes slowly for

historical days, and many readers want the same answer. Compute once, read cheap — at 200 concurrent users you save $200 \times 12 = 40$ minutes of CPU per page-load cycle." 2. "In Postgres I would create a **MATERIALIZED VIEW** and refresh nightly or hourly depending on staleness tolerance. SQL Server has indexed views that refresh on every write — different tradeoff, much heavier on the write path but always fresh. MySQL has no native equivalent; people roll their own with a scheduled INSERT INTO a summary table." 3. "I would also index the view for the dashboard filter — typically **(merchant_id, day)**. Without the index, even though the view is small, dashboards re-scan it on every page hit." 4. "Dashboard code should query the view directly, not be told to choose between view and base table at runtime — keeps the abstraction clean."

The likely root cause: The same expensive aggregate runs on every page load instead of once per refresh window; the database does 200x the work it needs to do.

The fix: - Postgres: ``sql CREATE MATERIALIZED VIEW mv_merchant_daily_revenue AS SELECT merchant_id, DATE(created_at) AS day, SUM(amount) AS revenue FROM orders WHERE created_at >= CURRENT_DATE - INTERVAL '6 months' GROUP BY merchant_id, DATE(created_at);

CREATE INDEX idx_mv_merchant_day ON mv_merchant_daily_revenue(merchant_id, day); `` - Refresh on a schedule via cron (concurrent variant in next scenario). For a 6-month rolling window, also schedule a daily "trim" so the view does not grow forever. - SQL Server: CREATE VIEW ... WITH SCHEMABINDING then a UNIQUE CLUSTERED INDEX makes it indexed; auto-maintained on writes — watch OLTP throughput as every order INSERT now also updates the indexed view. - Verify dashboard query drops from 12 seconds to under 100 ms; peak DB CPU falls dramatically.

What NOT to say: - "A view would solve this." — A regular view is a saved query; runs every time, no caching. The interviewer is testing whether you know the difference. - "Cache it in Redis." — Valid for some shapes but moves truth out of the database, complicates invalidation, and makes joins back to live data harder. - "Add more indexes to the orders table." — Indexes help filter and lookup; they do not eliminate the aggregation cost. The 12 seconds is the GROUP BY scan and sum, not the seek.

Common follow-up: "How do you decide refresh frequency?" — Driven by business tolerance for staleness — financial reports nightly is fine; live operational dashboards may need hourly. You can also layer — hourly view for the last 7 days, daily for the older 6 months — to balance freshness with refresh cost.

Scenario145 · CONCURRENTLY Refresh of a Materialised View

Asked at: Cred · Atlassian GCC · PhonePe · **Difficulty:** Medium · **Topic:** Materialisation · **Dialect:** Postgres

The interviewer's prompt: "Your materialised view refresh takes 4 minutes and the dashboard goes blank during the refresh because the view is exclusively locked. How do you fix it?"

Thinking out loud: 1. "By default, `REFRESH MATERIALIZED VIEW` takes an `ACCESS EXCLUSIVE` lock on the view — no reads, no writes, until the refresh completes. That is why the dashboard goes blank: every dashboard query against the view hangs waiting for the lock, then either times out or returns the moment refresh finishes." 2. "Postgres has `REFRESH MATERIALIZED VIEW CONCURRENTLY` for exactly this — it computes the new version in a side table and then merges deltas (`INSERT` for new keys, `DELETE` for missing keys, `UPDATE` for changed values) into the original storage without blocking readers. Readers continue to see the old version until commit, then atomically see the new version." 3. "The catch is that `CONCURRENTLY` requires a unique index on the view (so Postgres can match old and new rows), and it is slower than the standard refresh — it does a row-by-row diff against the old version instead of a clean truncate-and-insert." 4. "For high-availability dashboards this trade is almost always worth it. The refresh time goes up maybe 50%, but the user-visible blank window goes to zero."

The likely root cause: Standard `REFRESH` takes a long `ACCESS EXCLUSIVE` lock for the entire 4-minute compute window; every reader of the view blocks behind that lock, and dashboards show empty results or time out.

The fix: - Add a unique index on the view first — this is the prerequisite, Postgres will reject `CONCURRENTLY` without it: `sql CREATE UNIQUE INDEX mv_merchant_daily_pk ON mv_merchant_daily_revenue (merchant_id, day);` - Then refresh concurrently: `sql REFRESH MATERIALIZED VIEW CONCURRENTLY mv_merchant_daily_revenue;` - Readers see the old data until the refresh commits, then see the new data atomically — no blank dashboard, no flicker, no error toast. - Verify with `SELECT * FROM pg_locks WHERE relation = 'mv_merchant_daily_revenue'::regclass;` during the refresh — you should see only a `SHARE UPDATE EXCLUSIVE` lock, not the `ACCESS EXCLUSIVE` lock that the standard refresh takes.

What NOT to say: - "Just refresh during off-peak hours." — Fine for daily refresh on a domestic-only product, but does not scale to hourly or live workloads, and even "off-peak" in a 24x7 India product is just less peak. - "Drop and recreate the view." — Same lock-up problem during the recreate, plus you lose downstream grants, dependent views, and any indexes you carefully built — a multi-hour clean-up exercise to recover from. - "Use a regular index instead of unique." — Postgres specifically requires a unique index because the diff algorithm needs to identify rows; a non-unique index will not satisfy `CONCURRENTLY` and the command errors out.

Common follow-up: "Why is `CONCURRENTLY` slower than the standard refresh?" — It does a diff against the old version row-by-row to compute inserts, deletes, and updates, instead of a clean truncate-and-insert. It also has to write WAL for the deltas. A good answer adds that for very high-churn views (most rows change every refresh), the diff overhead can be significant — at that point you might rethink the materialisation strategy or refresh more frequently with smaller deltas.

Scenario146 · VACUUM ANALYZE on Hot Tables

Asked at: Razorpay · Swiggy · Flipkart · **Difficulty:** Medium · **Topic:** Maintenance · **Dialect:** Postgres

The interviewer's prompt: "You have a `payment_attempts` table with 50 lakh inserts and 30 lakh updates per day. Queries on it have slowed steadily over the past month. EXPLAIN shows the table reading way more pages than its row count would suggest. What is happening?"

Thinking out loud: 1. "In Postgres MVCC, an UPDATE does not modify the row in place — it writes a new row version and marks the old one dead. VACUUM scans for dead tuples and marks the space reusable. If autovacuum cannot keep up with churn — common on heavily-updated tables — dead tuples pile up and reads scan extra pages full of garbage to find live rows." 2. "This is table bloat. ANALYZE is the stats-refresh half; VACUUM is the space-reclaim half. VACUUM ANALYZE does both in one pass." 3. "Autovacuum's default trigger is 20% of the table changing (`autovacuum_vacuum_scale_factor = 0.2`). On a hot table with 80 lakh changes per day, 20% is reached in hours, and one pass cannot finish before the next 20% accumulates — permanent bloat." 4. "I would check `pg_stat_user_tables` for `n_dead_tup` vs `n_live_tup` — anything above 20-30% dead is suspicious; above 50% is screaming."

The likely root cause: Autovacuum falling behind on a high-churn table; dead tuples accumulate; reads touch many pages of dead tuples for the same logical row count, blowing up buffer cache miss rates.

The fix: - Tune autovacuum per table — defaults assume modest churn: `sql ALTER TABLE payment_attempts SET ( autovacuum_vacuum_scale_factor = 0.05, -- trigger at 5% churn autovacuum_analyze_scale_factor = 0.02, -- analyze at 2% autovacuum_vacuum_cost_limit = 2000 -- more I/O budget );` - One-off manual cleanup: `VACUUM (ANALYZE, VERBOSE) payment_attempts;` — verbose output tells you how many dead tuples were removed. - For severe bloat where in-place VACUUM cannot reclaim disk, use `pg_repack` (next scenario). - Verify `n_dead_tup` drops close to zero post-vacuum and the steady-state dead ratio sits under 10%.

What NOT to say: - "Run VACUUM FULL nightly." — Takes ACCESS EXCLUSIVE lock, rewrites the entire table — kills the app, and payment_attempts has no nightly window. - "Autovacuum handles everything." — On hot tables it cannot keep up at defaults; saying "it handles itself" misses the diagnosis. - "Disable autovacuum and run VACUUM from cron." — Loses adaptive triggering; a forgotten cron job leads to runaway bloat. Tune autovacuum aggressively instead.

Common follow-up: "VACUUM vs VACUUM FULL vs pg_repack?" — VACUUM reclaims space within the table (marks tuples reusable) without exclusive lock, does not return space to OS; VACUUM FULL rewrites the whole table into a new file with ACCESS EXCLUSIVE lock, releasing space to the filesystem; pg_repack achieves the same end-state as VACUUM FULL but uses a trigger-based shadow table to do almost all the work online, with only a brief swap lock at the end.

Scenario147 · pg_repack for Severe Bloat

Asked at: ThoughtSpot · Atlassian GCC · Cred · **Difficulty:** Hard · **Topic:** Maintenance · **Dialect:** Postgres

The interviewer's prompt: "Your `events` table is 200 GB on disk but only contains 40 GB of live rows. `VACUUM` has run but disk usage has not dropped. The table is heavily read 24x7 — no downtime window. How do you reclaim the space?"

Thinking out loud: 1. "VACUUM marks space reusable but does not return it to the OS. The 160 GB of dead-tuple space is reused by future inserts within the existing file, but disk footprint stays — Postgres only truncates if dead space happens to be at the very end. To shrink the file you need `VACUUM FULL` or `pg_repack`." 2. "VACUUM FULL takes `ACCESS EXCLUSIVE` for the entire rewrite — hours on a 200 GB table, during which no `SELECT`, no `INSERT`, nothing works. Not acceptable for a 24x7 service." 3. "pg_repack is the right tool — builds a fresh compacted copy in the background, captures concurrent changes via triggers into a log table, replays them onto the new copy, then swaps with only a brief `ACCESS EXCLUSIVE` lock at the very end (seconds, not hours)." 4. "I would also identify what caused the bloat — long-running transactions blocking VACUUM cleanup are the usual culprit; if I do not fix that, bloat returns in weeks."

The likely root cause: Long-running transactions (forgotten BI query, stuck replication slot) held an old xmin horizon, which prevented VACUUM from reclaiming dead tuples; once vacuum fell behind, heavy load means VACUUM cannot catch up enough to truncate the file from the end and release space to OS.

The fix: - Install the extension: `CREATE EXTENSION pg_repack;` (superuser; needs to create temporary triggers). - Run from the shell: `pg_repack -d mydb -t events --jobs 4` — `--jobs` parallelises index rebuild. - Mechanism: builds new heap in a side table, copies live rows in batches, replays trigger-captured changes, then swaps. Brief `ACCESS EXCLUSIVE` at the swap (seconds). - Verify `\dt+ events` — drops from 200 GB to ~40 GB plus index overhead. - Investigate the root:

`SELECT pid, xact_start, query FROM pg_stat_activity WHERE state = 'idle in transaction' AND xact_start < now() - interval '1 hour';` and kill or fix the source app.

What NOT to say: - "Just take downtime and run `VACUUM FULL`." — Not acceptable for the workload; a 200 GB rewrite is 30 minutes to several hours of full lockout. - "DELETE all the bloat." — Bloat IS dead tuples; DELETE creates more dead tuples, making it worse. - "Run `CLUSTER` on the table." — CLUSTER also rewrites with `ACCESS EXCLUSIVE`; same blocking problem.

Common follow-up: "What can go wrong with `pg_repack`?" — Long-running transactions block the final swap (it queues behind any concurrent transaction); triggers add overhead during the repack window; index bloat needs `--only-indexes` or `REINDEX CONCURRENTLY`; and `pg_repack` needs 1-2x the live table size as free disk space during operation.

Scenario148 · Forcing Hash Join on Big × Big

Asked at: Walmart Labs · Razorpay · PhonePe · **Difficulty:** Hard · **Topic:** Join algorithms · **Dialect:** Postgres · SQL Server (similar) · MySQL 8 (hash join 8.0.18+)

The interviewer's prompt: "You are joining `orders` (10 crore rows) with `payments` (8 crore rows) on `order_id`. The planner picks a nested loop and the query runs for an hour. What is wrong and how do you fix it?"

Thinking out loud: 1. "Nested loop on big $\times$ big is almost always a planner mistake. Nested loop is right when the outer side is tiny and the inner side has a great index — you do outer-size lookups. With both sides at the crore scale, you are doing crores of inner-side lookups, and even at one millisecond each that is hours." 2. "Hash join is the right operator here — build a hash on the smaller side, probe with the larger. $O(N + M)$ instead of $O(N \times M)$. The cost is `work_mem` to hold the hash; if it does not fit, hash join spills to disk in batches but is still vastly faster than nested loop." 3. "The planner picked nested loop because it misestimated one side as tiny — a filter selectivity miscalculation, stale stats, or correlated columns. Fix the estimate first; force the hash join as a backup if the misestimate is structural." 4. "I would also confirm there is no `LIMIT` near the top of the plan tricking the planner into thinking it can stop early — one of the few cases nested loop is genuinely better on large inputs."

The likely root cause: A stats error or filter the planner over-estimated as highly selective made one side look like a few hundred rows when it is actually crores. The planner then picked nested loop — right algorithm for the row counts it believed, wrong for reality.

The fix: - Step 1 — fix the stats: `ANALYZE orders; ANALYZE payments;`. The gap between estimated and actual rows in EXPLAIN ANALYZE tells you which operator was fooled. - Step 2 — if stats are correct, increase the planner's search: `SET join_collapse_limit = 12;` so it explores more orderings. - Step 3 — last resort, force the operator: - Postgres (pg_hint_plan): `/*+ HashJoin(o p) */`. Add a comment explaining why. - SQL Server: `INNER HASH JOIN o, p ON o.order_id = p.order_id`. - MySQL 8.0.18+: hash join is automatic for equi-joins where no index is usable. - Verify EXPLAIN shows `Hash Join` over two `Seq Scan` or `Index Scan` nodes; runtime drops from hours to single-digit minutes. Check `Hash Batches: 1` — if it spills, raise `work_mem`.

What NOT to say: - "Add an index on `payments.order_id` — it will help the nested loop." — Wrong operator; the index makes nested loop less terrible but still vastly slower than hash join because you still do 10 crore lookups. - "Increase `work_mem` and let it figure itself out." — Only helps if hash join is chosen; if planner sticks with nested loop, memory is wasted. - "Just disable nested loops: `SET enable_nestloop = off`." — Heavy-handed; kicks even OLTP queries to hash or merge join.

Common follow-up: "When is merge join better than hash join?" — When both inputs are already sorted on the join key (from the same index), merge join wins on memory because it streams in $O(1)$ memory — useful over very large inputs or when the next step is sort-aware like a window function.

Scenario149 · Increase shared_buffers / buffer_pool_size

Asked at: Razorpay · Cred · Atlassian GCC · **Difficulty:** Medium · **Topic:** Memory tuning · **Dialect:** Postgres (shared_buffers) · MySQL (innodb_buffer_pool_size) · SQL Server (max server memory)

The interviewer's prompt: "Your active working set is 25 GB. The box has 64 GB of RAM. Postgres `shared_buffers` is at the default 128 MB. Queries are I/O-bound — every read hits disk. What is the right tuning?"

Thinking out loud: 1. "`shared_buffers` is Postgres's own page cache, sitting on top of (and separate from) the OS page cache. At the default 128 MB the database cannot hold any meaningful working set — every query reads from disk or relies on the kernel page cache to bail it out, which is far less efficient than holding pages in the database's own cache." 2. "Postgres conventional wisdom is 25% of RAM for `shared_buffers` — 16 GB on a 64 GB box — leaving room for OS cache, `work_mem` allocations per connection, WAL buffers, and other backend memory. Going higher than 40% rarely helps and often hurts." 3. "MySQL InnoDB is different — the InnoDB buffer pool is the only meaningful cache layer (largely bypasses OS cache via `O_DIRECT`), so the standard is 50-70% of RAM." 4. "Before bumping `shared_buffers` I would confirm the workload is genuinely cache-cold — check `pg_stat_database.blks_hit` vs `blks_read` for the hit ratio."

The likely root cause: `shared_buffers` at default 128 MB; the hot working set of 25 GB has no room in the database's cache, so every read hits OS cache (slower than `shared_buffers` due to syscall and copy) or disk.

The fix: - Postgres: edit `postgresql.conf` — `shared_buffers = 16GB` . Restart required (fixed allocation at process start). Also bump `effective_cache_size = 48GB` so the planner knows how much data is likely cached. - MySQL: `innodb_buffer_pool_size = 48G` (75% of RAM). Dynamic since MySQL 5.7. - SQL Server: `EXEC sp_configure 'max server memory (MB)', 56000;` — leaves ~8 GB for OS and non-buffer-pool consumers. - Verify with `pg_bufferscache` what is actually resident; cache hit ratio (`blks_hit / (blks_hit + blks_read)`) should climb toward 99%.

What NOT to say: - "Set `shared_buffers` to 90% of RAM." — Starves the OS cache, `work_mem`, connection memory, and the kernel; performance gets worse, and the box risks OOM under load. - "More RAM is always better." — Diminishing returns once the working set fits; beyond that, money is better spent on CPU, faster disks, or replicas. - "Restart Postgres nightly so the cache is fresh." — Catastrophically backwards; a warm cache is what makes Postgres fast. Cold cache causes sustained latency until pages re-warm.

Common follow-up: "`shared_buffers` vs OS page cache?" — `shared_buffers` is process-shared inside Postgres with its own clock-sweep eviction; OS cache is unified across all processes with strict LRU. Pages can be in both — a small inefficiency called double buffering, but in practice the two layers diverge enough that having both is a net win for hot pages.

Scenario150 · Connection Pooling with PgBouncer

Asked at: Razorpay · Swiggy · PhonePe · Cred · **Difficulty:** Medium · **Topic:** Infrastructure · **Dialect:** Postgres (PgBouncer) · MySQL (ProxySQL)

The interviewer's prompt: "During peak hours, your service throws **FATAL: sorry, too many clients already**. Postgres is set to **max_connections = 200**, you have 50 app pods each with a connection pool of 20. The math obviously does not work. How do you fix this without raising **max_connections** to 1000?"

Thinking out loud: 1. "Postgres connections are expensive — each forks a backend process with ~10 MB of base memory plus per-connection work_mem. 1000 connections is 10 GB of overhead before the database has done any work, plus a lot of context-switching as the scheduler shuffles between them." 2. "The right answer is a transaction-pooling broker between app pods and Postgres. PgBouncer in **transaction** mode lets 1000 client connections share say 50 server connections, because most clients are idle at any moment — they only need a backend for the duration of a transaction." 3. "App pods talk to PgBouncer; PgBouncer multiplexes onto the actual database. Each Postgres backend handles many client transactions, one at a time, returned to the pool the instant the transaction commits." 4. "Size the pool right — too small and clients wait; too big and you defeat the purpose. Rule of thumb: **default_pool_size** slightly less than Postgres **max_connections** divided by distinct (user, database) pairs."

The likely root cause: 50 pods × 20 connections = 1000 client connection demand against a 200-connection database limit. App pools are sized for per-pod peak; the database is not sized for the sum. The moment ten pods peak simultaneously, the server refuses connections.

The fix: - Deploy PgBouncer in **pool_mode = transaction**. Config: **default_pool_size = 25** **max_client_conn = 2000** **pool_mode = transaction** **reserve_pool_size = 5** - App pods connect to PgBouncer on port 6432 instead of Postgres on 5432; Postgres stays at 200 connections, PgBouncer holds ~50 of those. - MySQL: ProxySQL plays the same role with connection multiplexing. - Verify with **SHOW POOLS;** — **cl_active** high, **sv_active** much lower, few **cl_waiting**. If **cl_waiting** is persistent, raise **default_pool_size** modestly.

What NOT to say: - "Raise max_connections to 2000." — Each connection is a process; more memory on idle backends than on data, and the scheduler thrashes under load. Exactly the wrong direction. - "Use session pooling mode." — Defeats the purpose; one session equals one server connection until disconnect, so 1000 client connections still need 1000 server connections. - "Add more app replicas with smaller pools." — Same total demand distributed differently; the database does not care which pod sent the connection.

Common follow-up: "What features break under transaction pooling?" — Named prepared statements (next transaction may land on a different backend), **SET** without **LOCAL** outside a transaction, advisory locks across transactions, temp tables across transactions, listen/notify. PgBouncer 1.21+ added protocol-level prepared statement support which fixes the most common breakage.

Scenario151 · Read Replicas for Read-Heavy Workloads

Asked at: Razorpay · Flipkart · Cred · **Difficulty:** Medium · **Topic:** Replication · **Dialect:** vendor-agnostic principle · Postgres (streaming replication) · MySQL (async replication) · SQL Server (Always On)

The interviewer's prompt: *"Your primary database is at 90% CPU. 80% of queries are reads — product browsing, search, dashboards. Writes are only 20%. How do you scale this without re-architecting the application?"*

Thinking out loud: 1. *"Read-heavy workload with a single primary is the textbook case for read replicas. Reads route to one or more replicas; writes still go to the primary. Primary CPU drops because it only handles the 20% write workload; total read capacity scales roughly linearly with replica count up to a few replicas."* 2. *"The catch is replication lag — replicas are eventually consistent. With Postgres streaming or MySQL async, a replica lags by a few milliseconds healthy, seconds under load. You cannot do read-after-write from a replica without risk of staleness — the row you just inserted may not be visible yet."* 3. *"The application or a smart proxy needs to route appropriately — read-after-write queries to primary, eventually-consistent reads to replicas. This is the 'application change' part of 'no re-architecture'."* 4. *"I would also want session-stickiness for flows where a user's own writes must be immediately visible — easiest implementation is a 'last write timestamp' per user, and route reads to primary if within the lag window."*

The likely root cause: Single primary serving the entire read load when most reads do not need primary-fresh data; read CPU dominates with nowhere else to go.

The fix: - Spin up streaming replicas (Postgres async streaming via `primary_conninfo` ; MySQL `CHANGE MASTER TO` ; SQL Server Always On with read-intent routing). - Route reads at the app layer (master/replica DataSource in Hibernate/Spring Data) or via a proxy (PgPool-II, ProxySQL, RDS Proxy). - Tag queries that need primary-fresh — "just placed an order, show detail" or "check balance after deposit" — and pin them to primary. Everything else (catalog, search, dashboards, reports) goes to a replica. - Verify primary CPU drops to under 30%; replica lag stays under 1 second via `pg_stat_replication.replay_lag` ; alert if lag exceeds 5 seconds.

What NOT to say: - *"Replicas are perfect copies, route everything to them."* — They lag; reading a just-written row may return pre-write state. Worse, the staleness is invisible — the query succeeds with old data. - *"Just upgrade the primary's CPU."* — Vertical scaling hits a ceiling fast, is expensive (5x cost for 2x compute), and does not help when you eventually need horizontal scaling anyway. - *"Replicas are free, add 10."* — Each replica adds streaming bandwidth load on the primary's WAL plus operational complexity. 2-3 typically saturate the benefit.

Common follow-up: *"How do you decide which reads tolerate replica lag?"* — Reports and dashboards: yes. List of products: usually yes. Reading the row you just inserted: no — route to primary or wait for replication using a "monotonic read" pattern. Some teams use a feature flag per query class to rebalance during incidents.

Scenario152 · Sharding When Vertical Scaling Is Not Enough

Asked at: Flipkart · Swiggy · Cred · PhonePe · **Difficulty:** Hard · **Topic:** Horizontal scaling · **Dialect:** vendor-agnostic principle

The interviewer's prompt: "Your `orders` table is at 50 crore rows, the largest box you can buy is already running it, and writes are saturating the primary. What is the next step?"

Thinking out loud: 1. "Vertical scaling is out — you're on the largest instance. Read replicas do not help because writes are the bottleneck and replicas only scale reads. The honest answer is sharding — split data horizontally across multiple primaries by a shard key, so each shard handles a fraction of total writes." 2. "The killer question is the shard key, answered by workload not convenience. For an e-commerce orders table, `customer_id` is usually right — keeps a customer's data on one shard, lets you do customer-centric queries (order history, returns, refunds) without cross-shard joins. `order_id` itself is wrong because it scatters one customer's orders across all shards." 3. "Sharding is a major lift — application changes, routing layer, rebalancing for hot shards, distributed transactions for cross-shard writes. Make sure simpler options (caching, archival, columnar offload for analytics) are genuinely exhausted first." 4. "I would also push back gently — 'is the entire 50 crore rows hot, or is the last 6 months getting all the writes?' Time-based archival is often more pragmatic than sharding."

The likely root cause: Single primary write bottleneck on a workload that cannot fit vertically — even SSD-backed writes have a ceiling defined by WAL throughput per primary.

The fix: - Pick a shard key driven by query patterns (`customer_id` for customer-centric workloads, `merchant_id` for merchant dashboards, geographic region for residency). The choice is permanent in practice — get it wrong and re-sharding is far worse than choosing well upfront. - Add a routing layer — Citus for Postgres (transparent), Vitess for MySQL (proven at YouTube scale), or app-level sharding with a lookup service. - Migrate in waves — dual-write during cutover (read from old, write to both), backfill, cut reads one query class at a time, then cut writes. - Hardest part is cross-shard queries — analytics, "top 100 customers by revenue" — typically need a separate analytical store (warehouse, ClickHouse, BigQuery) populated by CDC.

What NOT to say: - "Just shard by hash of primary key." — Loses locality; every customer query becomes scatter-gather across all shards, making latency unpredictable. - "NoSQL solves all this." — Different tradeoffs; you trade joins and ACID for partition tolerance, and end up rebuilding joins in the application. - "Start with 256 shards so we never re-shard." — Operational complexity scales with shard count; start with 4-16 and split when needed.

Common follow-up: "Sharding vs partitioning?" — Partitioning splits a table within one database server (same RAM, CPU, disk); sharding splits across multiple servers, each with its own storage and CPU. Partitioning helps with pruning and maintenance; sharding scales beyond a single machine's limits.

Scenario153 · Sequential Scan of Huge Cold Table — Partition or Pre-Aggregate

Asked at: Walmart Labs · ThoughtSpot · Atlassian GCC · **Difficulty:** Medium · **Topic:** Cold storage · **Dialect:** vendor-agnostic

The interviewer's prompt: "You have an `events` table with 5 years of data, 80 crore rows, that gets queried only for monthly reports. Each report sequentially scans the whole table. How do you make this not painful?"

Thinking out loud: 1. "Two angles, often layered. One — partition by month and the planner only scans the relevant partition; that alone cuts an 80-crore scan down to 1-2 crore for a monthly report. Two — for reports that aggregate the same dimensions repeatedly, store pre-aggregated rollups in a separate table." 2. "For 5 years of cold data, a third lever is moving older partitions to cheaper storage — tablespaces on slow disk, or archive to S3 and query via a foreign data wrapper. The 2025 partition lives on NVMe; the 2021 partition on cold HDD or S3." 3. "The right choice depends on query shape. 'Last month' style with raw access — partitioning alone works. 'YTD aggregate by region' — pre-aggregation wins because the output is tiny regardless of input size." 4. "I would also ask the business: do reports need row-level access or just summaries? If summaries, pre-aggregation suffices and cold partitions become audit-only."

The likely root cause: Single monolithic table on hot storage, scanned in full for queries that need only summaries or only a recent slice; the database does crores of pointless reads per report.

The fix: - Partition by month: `PARTITION BY RANGE (created_at)` ; old partitions can live on a slower tablespace via `CREATE TABLESPACE cold_storage LOCATION '/mnt/hdd'` and `ALTER TABLE events_2021_01 SET TABLESPACE cold_storage;` . The planner's partition pruning makes the cold partitions invisible to "last month" queries. - Pre-aggregate via a daily/weekly summary table maintained by the ETL or a materialised view:

`SELECT region, DATE(created_at) AS day, COUNT(*), SUM(amount) FROM events GROUP BY 1, 2;` populated incrementally on append. - Archive truly cold data — older than 2 years — to columnar storage (Parquet on S3) or external object store; access via Postgres `parquet_fdw` or `s3_fdw` if rarely needed. This pushes the inactive 60-70% of data off your primary's expensive SSD. - Verify reports run in seconds against summaries, not minutes against raw events; primary's disk footprint drops dramatically once cold partitions move off.

What NOT to say: - "Add more indexes to the events table." — A 5-year sequential scan does not benefit from indexes when the query touches all rows; indexes help filter to a subset, but if the report aggregates the entire history, the planner correctly chooses a sequential scan. - "Cache the results in the app." — Caching helps for repeated identical queries; analytical queries vary by date range, filter, and grouping, so cache hit ratio will be low. - "Just rebuild the table to compress old rows." — Postgres has TOAST for large values but no native row-level compression for analytical workloads; columnar storage is the right tool, not a clever rewrite.

Common follow-up: "How would you keep the pre-aggregation up to date efficiently?" — Append-only rollup — for each new day, INSERT that day's aggregate into the summary table; never re-aggregate history because history does not change. For corrections (rare), reprocess only the affected day's slice rather than full rebuild.

Scenario154 · Hash Aggregate Spilling — Same Fix Family

Asked at: PhonePe · Cred · ThoughtSpot · **Difficulty:** Medium · **Topic:** Aggregation · **Dialect:** Postgres primary

The interviewer's prompt: "Your `GROUP BY user_id` over a billion-row events table is slow. `EXPLAIN` shows `HashAggregate` with `Batches: 32 Memory: 4 MB Disk: 800MB`. What is happening?"

Thinking out loud: 1. "HashAggregate builds an in-memory hash of group keys to running aggregates. If `work_mem` cannot hold all distinct groups, Postgres spills to disk in batches — `Batches: 32` means 32 disk passes, each reading and writing partial hashes to temp files. Disk I/O dominates the runtime." 2. "The fix mirrors the sort-spill fix — raise `work_mem` for this query so the aggregate fits in memory. Same caveats: per-query or per-role, not global, because `work_mem` multiplies across operations and connections." 3. "Alternatively, if `GROUP BY` columns match an existing index order (`user_id` indexed), the planner can do a sorted GroupAggregate, which streams in $O(1)$ memory regardless of how many distinct groups exist — a completely different algorithm." 4. "I would also confirm group cardinality with `SELECT COUNT(DISTINCT user_id);` — if there are 50 crore distinct users, no amount of `work_mem` will fit; you must use sort-based GroupAggregate or pre-aggregate."

The likely root cause: `work_mem` too small for the cardinality of `user_id` — many crore distinct users with per-group state does not fit in 4 MB; 32 batches is Postgres doing the right thing within its memory budget.

The fix: - Per-query: `SET work_mem = '1GB'; SELECT user_id, COUNT(*) ... GROUP BY user_id;` - If GroupAggregate over an index is feasible, ensure stats are fresh and the planner picks it — check `EXPLAIN` for `GroupAggregate` over `Index Scan`. May need `SET enable_hashagg = off;` to force the planner. - Pre-aggregate via a materialised view if the query is repeated; at billion-row scale, an incremental rollup pays for itself in a single report cycle. - Verify `EXPLAIN` now reads `Memory: 800MB` instead of `Disk: 800MB`, `Batches: 1`; runtime drops from minutes to single-digit seconds.

What NOT to say: - "Use `DISTINCT` instead." — Same spill problem (implemented as hash or sort), often slower than `GROUP BY` because `GROUP BY` can compute the aggregate in the same pass. - "Drop the index — it must be misleading the planner." — Wrong; the index helps the streaming GroupAggregate path. Dropping it forces hash, which is the spill case. - "Add `LIMIT 10` to make it small." — Does not help; HashAggregate must process all input before knowing which 10 groups make the cut; `LIMIT` only saves work at the end.

Common follow-up: "HashAggregate vs GroupAggregate?" — Hash builds a hash table of group keys; works on unsorted input but needs memory proportional to group count. Group needs sorted input and streams in $O(1)$ memory regardless of group count, because it only holds the current group's running aggregate. Choose based on group count and whether a sorted input is available.

Scenario155 · Hot Row Contention — Sharded Counter Pattern

Asked at: Razorpay · Swiggy · Flipkart · **Difficulty:** Hard · **Topic:** Contention · **Dialect:** vendor-agnostic principle

The interviewer's prompt: "You maintain a single row `total_orders_today` that every order increments. During a flash sale, the database starts deadlocking and throughput tanks. What is happening and how do you fix it?"

Thinking out loud: 1. "That single row is a serialisation point. Every order transaction takes a row-level lock on it via the `UPDATE`, so concurrent orders queue behind whoever holds it. Throughput caps at $1 / \text{lock_hold_time}$ — at a typical 5 ms transaction, that is ~200 increments per second regardless of hardware. Deadlocks happen when transactions touch the counter and other rows in inconsistent orders." 2. "The classic fix is the sharded counter — instead of one row, keep N rows (say 64) and each transaction increments a random shard. Total is `SUM(count)` across shards. Writes spread across 64 lock targets, contention drops 64x; reads pay a tiny aggregation cost." 3. "Trade-off: writes scale roughly linearly with shard count; reads pay a microsecond SUM on 64 rows. Acceptable for almost all real workloads — counters, vote tallies, view counts." 4. "I would also push back on whether you really need real-time accuracy — for a 'today's order count' display, a 30-second-stale value via periodic batch often works and avoids contention entirely."

The likely root cause: Row-level lock on the single counter row is the per-transaction bottleneck; under flash-sale concurrency the lock queue grows faster than transactions finish, and deadlocks happen when transactions update the counter alongside other rows in inconsistent orders.

The fix: - Create a sharded counter table:

```
sql CREATE TABLE counter_shards( name TEXT NOT NULL, shard INT NOT NULL, count
BIGINT NOT NULL DEFAULT 0, PRIMARY KEY(name, shard) ); INSERT INTO
counter_shards SELECT 'orders', s, 0 FROM generate_series(0, 63) s; - On write:
pick a random shard (e.g., shard = pg_backend_pid() % 64 ) and UPDATE counter_shards
SET count = count + 1 WHERE name = 'orders' AND shard = ? . - On read: SELECT
SUM(count) FROM counter_shards WHERE name = 'orders'; - Verify write QPS scales
roughly N-fold; deadlock rate drops close to zero.
```

What NOT to say: - "Use `SELECT FOR UPDATE` to serialise cleanly." — Removes deadlock noise but does not solve the throughput cap; transactions still queue. - "Disable transactions for that statement." — Loses durability; rarely acceptable for a payment-adjacent counter, and Postgres autocommit is still a transaction. - "Use Redis `INCR` instead." — Valid for many cases but moves truth out of the database and introduces consistency gaps with the orders table.

Common follow-up: "How do you avoid drift if a shard gets deleted or a server fails mid-increment?" — Idempotent operations (`UPSERT` with `ON CONFLICT DO UPDATE`), tolerate eventual consistency on the SUM, and reconcile against the authoritative source (the orders table itself) hourly or daily.

Scenario156 · Batch vs Row-by-Row Processing

Asked at: Cred · Walmart Labs · Razorpay · **Difficulty:** Medium · **Topic:** Throughput · **Dialect:** vendor-agnostic

The interviewer's prompt: "Your ETL script processes 50 lakh rows by looping in Python and issuing one `UPDATE` per row. It takes 6 hours. How do you make it not take 6 hours?"

Thinking out loud: 1. "Row-by-row processing means 50 lakh round-trips between Python and the database. Each round-trip is at least a few hundred microseconds on a local socket, often a millisecond across a VPC. At 50 lakh $\times$ 1 ms that is over an hour just in network latency, plus per-statement parse, plan, and lock overhead." 2. "The fix is batched processing — write the update in SQL itself, joining against a temp table or a multi-row `VALUES` list, so the database does the loop natively in one statement." 3. "Even within the round-trip model, batching N rows per statement and using prepared statements (parse-plan happens once) cuts wall time an order of magnitude — useful intermediate if you cannot restructure." 4. "I would also confirm we are not committing per row — if the loop does `execute; commit` per row, every commit is an fsync. Batching commits every 1000 rows alone is a large win."

The likely root cause: Network round-trip overhead times 50 lakh rows, plus per-statement parse/plan, plus per-row commit fsync if commits are not batched. Each `UPDATE` is microscopic in the database itself, but the wrapper costs dominate.

The fix: - Option A — load the deltas into a temp table, then one `UPDATE`: `sql CREATE TEMP TABLE updates(id BIGINT, new_status TEXT); COPY updates FROM '/tmp/updates.csv'; UPDATE orders SET status = u.new_status FROM updates u WHERE orders.id = u.id;` The `COPY` is the fastest path to load the deltas; the join-based `UPDATE` is one statement, one parse, one plan, one transaction. - Option B — multi-row `VALUES` in chunks of 1000: `UPDATE orders SET status = v.s FROM (VALUES (1,'paid'),(2,'paid')...) v(id,s) WHERE orders.id = v.id;` Useful when CSV staging is not convenient. - Verify by timing — 6 hours typically drops to under 5 minutes; the `UPDATE` itself is then bound by index maintenance and WAL throughput, which are the only legitimate costs.

What NOT to say: - "Add an index — it will speed up the per-row `UPDATE`." — Per-row `UPDATE` is already $O(\log N)$ with the primary-key index; the bottleneck is round-trips, not lookup time, so the index does not help. - "Parallelise the Python loop with 16 threads." — Adds connection pressure on the database and still pays round-trip per row; you might shave 2-3x off the wall time but you are still hours away from the right answer, which is set-based SQL. - "Use a stored procedure to loop inside the database." — Better than Python looping, but still a row-at-a-time loop; the right pattern is set-based SQL, which is what the database is built for.

Common follow-up: "What about for inserts — does the same advice apply?" — Yes, but for inserts `COPY` is even better than `VALUES` (next scenario). The general principle is: any time you find yourself looping in application code to do per-row data modification, look for a way to express the same logic as one SQL statement against a staged dataset.

Scenario157 · COPY / Bulk Load Instead of Multi-Row INSERT

Asked at: PhonePe · Razorpay · ThoughtSpot · **Difficulty:** Medium · **Topic:** Ingestion · **Dialect:** Postgres (COPY) · MySQL (LOAD DATA INFILE) · SQL Server (BULK INSERT / bcp)

The interviewer's prompt: "Your daily ingest is loading a 2 GB CSV via batched `INSERT INTO ... VALUES` statements and takes 90 minutes. How do you speed it up?"

Thinking out loud: 1. "Multi-row `INSERT` is faster than per-row `INSERT` (one parse per batch instead of one parse per row), but every dialect has a dedicated bulk-load path that bypasses statement parsing, WAL overhead, and a lot of constraint-checking machinery. That is the right tool for bulk ingestion." 2. "Postgres: `COPY ... FROM` — typically 5-10x faster than equivalent `INSERT` for the same data because it uses a streaming protocol that avoids per-row parse and pipelines WAL writes. MySQL: `LOAD DATA INFILE` is the same idea — extremely fast bulk load with minimal per-row overhead. SQL Server: `BULK INSERT` or the `bcp` utility, with `TABLOCK` hint for full-batch optimisation." 3. "For maximum throughput, drop secondary indexes before the load, `COPY` in, then rebuild indexes in one shot — this is faster than incremental index maintenance for large loads because each rebuild can use the bulk sorting path." 4. "I would also consider whether the source CSV can be sorted by primary key before loading, because Postgres physically inserts in CSV order and a sorted-by-PK load gives you a clustered heap, which speeds up subsequent range scans without needing a separate `CLUSTER` step."

The likely root cause: `INSERT` statement overhead (parse, plan, execute, per-row WAL record, per-row constraint check) dominates; the actual write of bytes is a small fraction of total time, so the wrapper costs eat almost all the wall clock.

The fix: - Postgres: `sql COPY orders (order_id, customer_id, amount, created_at) FROM '/var/data/orders_2026-04-15.csv' WITH (FORMAT csv, HEADER true);` - Pre-load: `DROP INDEX idx_orders_user;` and other secondary indexes (optional, do this only if the table will be loaded in one shot — for incremental loads keep indexes live). - Post-load: rebuild indexes (`CREATE INDEX CONCURRENTLY` if the table must stay readable), then run `ANALYZE orders;` so the planner has fresh stats. - Verify: 2 GB load drops from 90 minutes to under 10 minutes; CPU during load is dominated by parse-and-tuple-form rather than statement overhead.

What NOT to say: - "Use an ORM bulk_insert method." — Most ORM bulk methods still issue multi-row `INSERTs` under the hood; they do not switch to `COPY` unless explicitly configured (psycopg `copy_expert`, SQLAlchemy `bulk_insert_mappings` does NOT use `COPY`). Many ORM users discover this the hard way. - "Just turn off transactions." — Postgres still uses WAL inside an implicit transaction; the gain from "no transaction" is mostly imaginary, and you lose atomicity if the load fails partway. - "Pre-compute the SQL in Python and execute one giant `INSERT`." — Parser memory pressure becomes the bottleneck; very long SQL strings hit pathological behaviour in the parser. `COPY` avoids this entirely.

Common follow-up: "Why is `COPY` so much faster than `INSERT`?" — Single statement parse, batched WAL records (one WAL flush per few thousand rows instead of per row), batched constraint checks, a streamlined wire protocol that skips per-row planning, and the absence of per-statement command tag round-trips. Combined, these compound to the 5-10x speedup.

Scenario158 · Async Commit for Non-Critical Writes

Asked at: Cred · ThoughtSpot · Atlassian GCC · **Difficulty:** Hard · **Topic:** Durability · **Dialect:** Postgres (synchronous_commit) · MySQL (innodb_flush_log_at_trx_commit)

The interviewer's prompt: "You have a high-throughput logging table — `user_activity` — that receives 50,000 inserts per second. Disk I/O on the WAL is saturated. The data is useful but losing a few seconds is acceptable. How do you trade durability for throughput?"

Thinking out loud: 1. "Default Postgres behaviour is synchronous commit — the WAL record is fsync'd to disk before COMMIT returns success. That fsync is expensive (microseconds to milliseconds even on NVMe) and serialises commits because each waits for disk acknowledgement before the next can proceed." 2. "For workloads not financially critical — analytics events, app logs, traces — `synchronous_commit = off` lets COMMIT return immediately and WAL flush happens later. You gain throughput at the cost of losing up to roughly $3 \times \text{wal_writer_delay}$ of data if the server crashes between WAL write to OS and flush to disk." 3. "Same idea in MySQL — `innodb_flush_log_at_trx_commit = 2` flushes to OS page cache but not disk on commit; `= 0` flushes once per second." 4. "Critically, the database remains consistent after crash — async commit only loses recent transactions, does not corrupt anything. Very different from `fsync = off`, which can corrupt the database on crash."

The likely root cause: fsync per commit is the I/O bottleneck for high-throughput log ingestion; 50,000 inserts/sec means 50,000 fsyncs/sec, well beyond what a single disk can do.

The fix: - Postgres per-session: `SET synchronous_commit = off;` then run the bulk-insert path. - Per-role: `ALTER ROLE logger SET synchronous_commit = off;` — applies to every session that role opens, no application code change. - Per-statement: `SET LOCAL synchronous_commit = off;` inside the transaction. - MySQL: `SET GLOBAL innodb_flush_log_at_trx_commit = 2;` (or `= 0` for more throughput at slightly more risk). - Verify throughput goes up 5-10x for fsync-bound workloads; understand the recovery contract — a few hundred ms of recent log writes may be lost on power cut.

What NOT to say: - "Turn synchronous_commit off everywhere for speed." — Catastrophic for financial data; must scope per workload (per role/session/statement) so orders and payments still get full durability. - "This is the same as fsync=off." — Critically different. `synchronous_commit = off` keeps the database consistent on crash, only loses last few hundred ms. `fsync = off` may corrupt the database because WAL records may be missing or out of order. Never use fsync = off in production. - "Use a non-transactional table type." — Postgres has no such thing; MyISAM in MySQL loses ACID entirely.

Common follow-up: "What is the data loss window when synchronous_commit is off?" — At most $3 \times \text{wal_writer_delay}$ (default 200 ms, so under a second) on a crash. The database remains consistent; only the most recent unflushed transactions are missing. For systems-level durability you can also use Postgres remote_apply / remote_write modes for sync replication.

Scenario159 · TEMP Table with Index for Multi-Statement Reuse

Asked at: Walmart Labs · ThoughtSpot · Goldman Sachs GCC · **Difficulty:** Medium · **Topic:** Materialisation · **Dialect:** Postgres · SQL Server · MySQL

The interviewer's prompt: "Your report runs five queries in sequence, each one filtering and aggregating the same 50 lakh-row derived set. Each query re-derives it. How do you compute it once and reuse it?"

Thinking out loud: 1. "CTEs are scoped to one statement; the derived set is not reusable across separate statements. The right tool is a TEMP table — create it, fill it once, index it for the downstream access patterns, then run all five queries against it. Derivation cost paid once, not five times." 2. "TEMP tables in Postgres live for the session and are dropped on disconnect; with `ON COMMIT DROP` they survive only until the transaction commits. They have their own statistics (after ANALYZE) and can be indexed like a regular table." 3. "For five queries hitting the same derived set, this is a clear win — 5x derivation collapses to 1x derivation plus 5x cheap lookups." 4. "TEMP tables in Postgres do NOT use shared_buffers — they use per-session `temp_buffers`. For large temp tables, raising `temp_buffers` keeps them hot in memory."

The likely root cause: Repeated derivation of the same intermediate result across five separate statements, each time paying the full filter + aggregate computation. The database has no way to know it could cache the result.

The fix:

```
BEGIN;

CREATE TEMP TABLE tmp_active_users ON COMMIT DROP AS
SELECT user_id, MAX(last_login) AS last_seen, COUNT(*) AS sessions
FROM sessions
WHERE last_login >= CURRENT_DATE - INTERVAL '30 days'
GROUP BY user_id;

CREATE INDEX ON tmp_active_users(user_id);
ANALYZE tmp_active_users; -- so the planner has stats for downstream joins

-- Run the five reports against tmp_active_users
SELECT ...;
SELECT ...;

COMMIT;
```

- Verify report time drops from 5x derivation cost to 1x plus 5x cheap lookups; for a 50 lakh-row set, typically minutes to under a minute total.

What NOT to say: - "Use a CTE and reference it five times." — CTEs do not persist across statements; you would need one massive statement, and pre-v12 Postgres materialised CTEs unconditionally (now inlines by default) — reuse semantics changed across versions. - "Use a materialised view." — Persistent

and global; overkill for session-scoped intermediate data, incurs refresh cost, leaves a stale artifact in the schema. - "Run the five queries in parallel — they share buffer cache." — Buffer cache helps with raw page reads but not with re-running GROUP BY; aggregate work is still done five times.

Common follow-up: "Lifetime of a TEMP table?" — Until end of session (default), or end of transaction with `ON COMMIT DROP`, or with `ON COMMIT DELETE ROWS` (table persists, contents wiped on commit). Other sessions cannot see it — each session has its own private namespace, which is exactly why TEMP tables scale: no cross-session contention.

Scenario160 · Diagnosing Plan Regression Across Releases — End-to-End

Asked at: Razorpay · Cred · Goldman Sachs GCC · Atlassian GCC · **Difficulty:** Hard · **Topic:** Plan-driven diagnosis · **Dialect:** Postgres (`pg_stat_statements` + `auto_explain`) · SQL Server (Query Store)

The interviewer's prompt: "After a feature release on Saturday, three queries that used to run in under a second are now taking 30 seconds intermittently. Same SQL, same schema. Walk me through your diagnosis end-to-end."

Thinking out loud: 1. "Intermittent slowness with the same SQL strongly suggests plan flips — the planner is picking a bad plan some of the time, driven by parameter values or stats freshness. The SQL text is identical, so the variable is what the planner does with it at any given moment." 2. "Capture data first. In Postgres: enable `auto_explain` with `auto_explain.log_min_duration = 1000` and `log_analyze = on` to log full actual-row plans for slow runs. Compare with a fast plan from staging." 3. "In SQL Server: open Query Store and look at the regressed-queries report — it lists queries whose plan has changed and shows old vs new plans side by side, with wait-stats breakdown so you can see whether it is CPU, I/O, or lock-wait dominated." 4. "Common causes: parameter sniffing (planner cached a plan for one parameter set, now reused with a very different one), stale stats after a release-time backfill, a new index the planner is now considering badly because cost estimates shifted."

The likely root cause: The release brought a stats-invalidating data shift (backfill, new tenant onboarded, big import) and the planner now misjudges selectivity for some parameter values; combined with cached plans reused across different parameter combinations, you get intermittent slowness.

The fix: - Step 1 — capture: enable `auto_explain`, gather plans for fast and slow runs of the same query. - Step 2 — compare plans: identify the operator that flipped (nested loop vs hash, index choice, join order, a new Sort or Filter). - Step 3 — fix the root cause: ANALYZE the affected tables; in SQL Server use `OPTION (RECOMPILE)` for worst-case queries or `OPTIMIZE FOR UNKNOWN`; in Postgres, consider `plan_cache_mode = force_custom_plan` so prepared statements re-plan each time with fresh parameter values. - Step 4 — verify with both fast and slow parameter values; confirm both pick the right plan. - Step 5 — guardrail: add a latency alert on `p99` so the next regression is caught in hours, not days.

What NOT to say: - "Roll back the release." — May not be possible; the release may not even be the cause (correlation with release time can be coincidental with a stats event), and you waste the diagnostic

opportunity. - *"Hint the plan and move on."* — Hides the root cause; the same shape will appear elsewhere on other queries with the same misestimate or sniffing pattern. - *"Restart Postgres — it will fix itself."* — Clears the plan cache so queries re-plan with current stats, but only treats the symptom — the regression returns next time stats drift.

Common follow-up: *"How would you set up a plan regression monitor going forward?"* — Postgres: `pg_stat_statements` + a daily diff on `mean_exec_time` per `queryid`, alerting on more than 3x regression with a 7-day baseline. SQL Server: Query Store alerts on the regressed-query report. GCC SRE teams typically wire these into Grafana with a Slack alert that includes the query text and EXPLAIN output.

Category 3, Part A — Design This Schema: OLTP Patterns

Design problems are open-ended — the interviewer says "design tables for X" and you walk them through entities, relationships, indexes, and edge cases. There is rarely one correct answer; what matters is the structure of your thinking. Part A is OLTP (transactional) schemas — payments, ledgers, multi-tenant data, audit trails, booking flows, messaging — the kind of write-heavy, concurrency-sensitive systems that power real Indian product companies. Part B (covered separately) shifts to analytics and warehousing. For each scenario below, you get the prompt, what to verbalize, a working schema, the rationale, edge cases, and the follow-up the interviewer is most likely to ask next.

Scenario161 · Double-Entry Payment Ledger

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Jupiter, Slice · Domain: Payments

The prompt:

"Design tables for a payments ledger that tracks money moving between user wallets, merchants, and our internal accounts. Walk me through your tables, relationships, indexes, and how you'd handle a failed partial settlement."

Thinking out loud (what to verbalize): 1. *"First, what are the entities and access patterns — accounts hold balances, transactions group entries, entries are the atomic debit and credit lines. Reads are mostly statements per account and reconciliation jobs."* 2. *"Next, the relationships — one transaction has at least two entries that must net to zero. Accounts and entries are many-to-one."* 3. *"For indexes I'd consider — entries by account_id and posted_at for statements, transactions by external_ref for idempotency."* 4. *"Edge cases to flag — partial failures, reversals, currency mismatch, and ensuring sum of entries per txn is zero via a constraint or trigger."*

The schema:

```

CREATE TABLE accounts (
    account_id BIGSERIAL PRIMARY KEY,
    owner_type VARCHAR(20) NOT NULL CHECK (owner_type IN ('user','merchant','internal')),
    owner_id BIGINT NOT NULL,
    currency CHAR(3) NOT NULL DEFAULT 'INR',
    account_kind VARCHAR(20) NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (owner_type, owner_id, account_kind, currency)
);

CREATE TABLE transactions (
    txn_id BIGSERIAL PRIMARY KEY,
    external_ref VARCHAR(80) UNIQUE,
    txn_type VARCHAR(30) NOT NULL,
    status VARCHAR(20) NOT NULL,
    posted_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    metadata JSONB
);

CREATE TABLE ledger_entries (
    entry_id BIGSERIAL PRIMARY KEY,
    txn_id BIGINT NOT NULL REFERENCES transactions(txn_id),
    account_id BIGINT NOT NULL REFERENCES accounts(account_id),
    direction CHAR(1) NOT NULL CHECK (direction IN ('D','C')),
    amount_paise BIGINT NOT NULL CHECK (amount_paise > 0),
    posted_at TIMESTAMP NOT NULL
);

CREATE INDEX idx_entries_account_time ON ledger_entries (account_id, posted_at DESC);

```

Why each choice: - Amounts stored as `BIGINT` paise — no floating-point drift, and INR easily fits. - `external_ref UNIQUE` makes retries idempotent — same upstream reference cannot post twice. - Entries are append-only — reversals create new compensating entries instead of mutating history.

Edge cases handled: - Partial settlement failure — transaction stays in `pending` status with no entries, retried as a fresh txn id, original `external_ref` blocks duplicates. - Currency mismatch — accounts carry currency; entries inherit via account, FX is its own transaction with FX accounts.

Alternatives considered + rejected: - Storing balance directly on the account row — rejected because it hides discrepancies; balance is derived `SUM(direction-signed amount)`, cached in a snapshot table for performance.

What NOT to say: - "I'll just update the wallet balance directly in a single UPDATE." That works for toys, not for any audit-grade system.

Common follow-up: "How would you reconcile this ledger against the bank statement at end of day, and where would mismatches surface?"

Scenario162 · Order Management with Line Items

Tags: Difficulty: Medium · Asked at: Flipkart, Meesho, Myntra, Nykaa · Domain: Commerce

The prompt:

"Design tables for an e-commerce order — header, line items, status transitions. Walk me through relationships, indexes, and how price changes mid-checkout get handled."

Thinking out loud (what to verbalize): 1. "First, what are the entities — orders are the header, order_items are the lines, and status_history captures every transition. Customers and products live elsewhere; I'll just reference them." 2. "Next, the relationships — order to items is one-to-many, items reference a product snapshot, not the live product, so a later price edit doesn't rewrite history." 3. "For indexes I'd consider — orders by customer_id and created_at, items by order_id, and a partial index on orders where status = 'pending' for the operations dashboard." 4. "Edge cases — price changes mid-checkout, partial cancellations of one line, and refunds that touch only some items."

The schema:

```
CREATE TABLE orders (
  order_id BIGSERIAL PRIMARY KEY,
  order_number VARCHAR(20) UNIQUE NOT NULL,
  customer_id BIGINT NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'pending',
  subtotal_paise BIGINT NOT NULL,
  tax_paise BIGINT NOT NULL,
  shipping_paise BIGINT NOT NULL,
  total_paise BIGINT NOT NULL,
  placed_at TIMESTAMP,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE order_items (
  order_item_id BIGSERIAL PRIMARY KEY,
  order_id BIGINT NOT NULL REFERENCES orders(order_id),
  product_id BIGINT NOT NULL,
  sku VARCHAR(60) NOT NULL,
  product_name_snapshot VARCHAR(255) NOT NULL,
  unit_price_paise BIGINT NOT NULL,
  quantity INTEGER NOT NULL CHECK (quantity > 0),
  line_total_paise BIGINT NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'pending'
);

CREATE INDEX idx_orders_customer_recent ON orders (customer_id, created_at DESC);
CREATE INDEX idx_orders_pending ON orders (created_at) WHERE status = 'pending';
CREATE INDEX idx_order_items_order ON order_items (order_id);
```

Why each choice: - Snapshot fields on `order_items` (`product_name_snapshot` , `unit_price_paise`) freeze the contract at order time. - `order_number` separate from `order_id` so the customer-facing identifier is shareable without exposing surrogate keys. - Per-item status enables partial cancel and partial fulfilment without splitting orders.

Edge cases handled: - Price change mid-checkout — checkout compares the customer's confirmed price against current product price; mismatch returns a "price changed, please reconfirm" flow. - Partial cancellation — one item flips to `cancelled` , order header stays `confirmed` and recomputes totals.

Alternatives considered + rejected: - Single denormalised `orders` table with a JSON of items — rejected because line-level status, returns, and aggregations across SKUs become painful.

What NOT to say: - "I'll join to the products table each time I show the order." That breaks the moment product details change.

Common follow-up: *"Now I need to support order splitting across warehouses — what would change in your schema?"*

Scenario163 · Order Status State Machine

Tags: Difficulty: Medium · Asked at: Flipkart, Swiggy, Zomato, Dunzo · Domain: Workflow

The prompt:

"Design tables to enforce a strict order state machine — only certain status transitions are legal. Walk me through how you'd prevent an illegal jump like 'delivered' to 'pending'."

Thinking out loud (what to verbalize): 1. *"First, the entities — the order itself, the legal-transitions catalogue, and the immutable status_history audit."* 2. *"Next, the access pattern — most reads are 'current status'; writes are 'attempt transition' and must validate atomically."* 3. *"For indexes — order_status_history by order_id and changed_at, and the legal-transitions table is tiny so a primary key on (from, to) is enough."* 4. *"Edge cases — concurrent transition attempts, system-initiated vs user-initiated transitions, and replaying history for support."*

The schema:


```

CREATE TABLE order_status_transitions (
    from_status VARCHAR(20) NOT NULL,
    to_status VARCHAR(20) NOT NULL,
    requires_role VARCHAR(20),
    PRIMARY KEY (from_status, to_status)
);

CREATE TABLE orders (
    order_id BIGSERIAL PRIMARY KEY,
    current_status VARCHAR(20) NOT NULL DEFAULT 'created',
    status_version INTEGER NOT NULL DEFAULT 0,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE order_status_history (
    history_id BIGSERIAL PRIMARY KEY,
    order_id BIGINT NOT NULL REFERENCES orders(order_id),
    from_status VARCHAR(20),
    to_status VARCHAR(20) NOT NULL,
    changed_by BIGINT,
    change_reason VARCHAR(255),
    changed_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_status_history_order ON order_status_history (order_id, changed_at);

```

Why each choice: - Transitions live in a table, not code, so the rules can be edited and audited like data.

- `status_version` on orders supports optimistic locking — concurrent transitions detect each other.
- History is append-only; the current status on orders is just a denormalised cache of "latest row".

Edge cases handled: - Concurrent transitions — `UPDATE orders SET status = ..., status_version = status_version + 1 WHERE order_id = ? AND status_version = ?` — one of two wins, the other retries. - System vs user transitions — `requires_role` column blocks customer-initiated `delivered` directly.

Alternatives considered + rejected: - Enforcing the state machine purely in application code — rejected because two services bypassing the check would corrupt state; the DB-level catalogue plus a transition stored procedure is harder to bypass.

What NOT to say: - "We'll just trust the application layer to validate." That falls apart with multiple services or background workers.

Common follow-up: "How would you support a soft 'on hold' state that any role can enter from any other state but only an admin can leave?"

Scenario164 · User Authentication with Sessions and Roles

Tags: Difficulty: Medium · Asked at: Freshworks, Zoho, Postman, BrowserStack · Domain: Identity

The prompt:

"Design auth tables — users, password credentials, sessions, roles, permissions. Walk me through relationships and how you'd handle multi-device logout."

Thinking out loud (what to verbalize): 1. *"First, the entities — users (identity), credentials (separate so we can rotate auth methods), sessions (one per device), roles and permissions (RBAC)."* 2. *"Next, relationships — user to sessions is one-to-many, user to roles many-to-many, role to permissions many-to-many."* 3. *"For indexes — sessions by user_id and by session_token_hash, expired-at for cleanup."* 4. *"Edge cases — revoke all sessions on password reset, soft-lock after failed attempts, role assignments that expire."*

The schema:

```
CREATE TABLE users (  
    user_id BIGSERIAL PRIMARY KEY,  
    email VARCHAR(255) UNIQUE NOT NULL,  
    phone VARCHAR(15) UNIQUE,  
    email_verified_at TIMESTAMP,  
    status VARCHAR(20) NOT NULL DEFAULT 'active',  
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE user_credentials (  
    credential_id BIGSERIAL PRIMARY KEY,  
    user_id BIGINT NOT NULL REFERENCES users(user_id),  
    password_hash VARCHAR(255) NOT NULL,  
    password_changed_at TIMESTAMP NOT NULL,  
    failed_attempts INTEGER NOT NULL DEFAULT 0,  
    locked_until TIMESTAMP  
);  
  
CREATE TABLE sessions (  
    session_id BIGSERIAL PRIMARY KEY,  
    user_id BIGINT NOT NULL REFERENCES users(user_id),  
    session_token_hash CHAR(64) NOT NULL UNIQUE,  
    device_fingerprint VARCHAR(100),  
    ip_address INET,  
    expires_at TIMESTAMP NOT NULL,  
    revoked_at TIMESTAMP  
);  
  
CREATE TABLE roles (  
    role_id SERIAL PRIMARY KEY,  
    role_name VARCHAR(50) UNIQUE NOT NULL  
);  
  
CREATE TABLE permissions (  
    permission_id SERIAL PRIMARY KEY,  
    permission_code VARCHAR(80) UNIQUE NOT NULL  
);  
  
CREATE TABLE user_roles (  
    user_id BIGINT NOT NULL REFERENCES users(user_id),  
    role_id INTEGER NOT NULL REFERENCES roles(role_id),  
    granted_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    expires_at TIMESTAMP,  
    PRIMARY KEY (user_id, role_id)  
);  
  
CREATE TABLE role_permissions (  
    role_id INTEGER NOT NULL REFERENCES roles(role_id),  
    permission_id INTEGER NOT NULL REFERENCES permissions(permission_id),  
    PRIMARY KEY (role_id, permission_id)  
);
```

```
CREATE INDEX idx_sessions_user_active ON sessions (user_id) WHERE revoked_at IS NULL;  
CREATE INDEX idx_sessions_expiry ON sessions (expires_at) WHERE revoked_at IS NULL;
```

Why each choice: - Credentials split from users so adding OAuth, OTP, or passkeys later is additive, not destructive. - We store `session_token_hash`, never the raw token — DB leak does not equal account takeover. - `expires_at` plus `revoked_at` allows both timeout and explicit logout.

Edge cases handled: - Multi-device logout — `UPDATE sessions SET revoked_at = NOW() WHERE user_id = ?` invalidates all devices. - Brute force — `failed_attempts` and `locked_until` give a soft lock without permanent damage.

Alternatives considered + rejected: - Stateless JWTs only — rejected because we cannot force-revoke a leaked token without a session table or a denylist anyway.

What NOT to say: - "We'll store passwords with a strong hash like MD5." Argon2id, scrypt, or bcrypt — that's the only acceptable answer.

Common follow-up: "Now add 2FA — what tables do you need, and how do you handle backup codes?"

Scenario165 · Multi-Tenant: Shared Schema with `tenant_id`

Tags: Difficulty: Medium · Asked at: Freshworks, Zoho, Chargebee, Postman · Domain: Multi-tenancy

The prompt:

"Design a multi-tenant SaaS schema using the shared-schema, `tenant_id` approach. Walk me through the trade-offs and how you'd guarantee one tenant never sees another's data."

Thinking out loud (what to verbalize): 1. "First, the rule — every business table carries a non-null `tenant_id`, and `tenant_id` is the first column of every meaningful index." 2. "Next, isolation — application enforces it but the DB also enforces via row-level security if available." 3. "For indexes — composite indexes always lead with `tenant_id` so query planner stays tenant-local." 4. "Edge cases — tenant deletion, noisy-neighbour tenants, cross-tenant analytics, and tenant migration to a dedicated cluster."

The schema:

```
CREATE TABLE tenants (  
    tenant_id BIGSERIAL PRIMARY KEY,  
    tenant_slug VARCHAR(60) UNIQUE NOT NULL,  
    plan VARCHAR(30) NOT NULL,  
    status VARCHAR(20) NOT NULL DEFAULT 'active',  
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE customers (  
    tenant_id BIGINT NOT NULL REFERENCES tenants(tenant_id),  
    customer_id BIGSERIAL NOT NULL,  
    email VARCHAR(255) NOT NULL,  
    name VARCHAR(150),  
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    PRIMARY KEY (tenant_id, customer_id),  
    UNIQUE (tenant_id, email)  
);  
  
CREATE TABLE invoices (  
    tenant_id BIGINT NOT NULL,  
    invoice_id BIGSERIAL NOT NULL,  
    customer_id BIGINT NOT NULL,  
    invoice_number VARCHAR(30) NOT NULL,  
    amount_paise BIGINT NOT NULL,  
    status VARCHAR(20) NOT NULL,  
    issued_at TIMESTAMP NOT NULL,  
    PRIMARY KEY (tenant_id, invoice_id),  
    FOREIGN KEY (tenant_id, customer_id) REFERENCES customers(tenant_id, customer_id),  
    UNIQUE (tenant_id, invoice_number)  
);  
  
CREATE INDEX idx_invoices_tenant_status ON invoices (tenant_id, status, issued_at DESC);
```

Why each choice: - Composite PK starting with `tenant_id` makes the table physically clustered by tenant — most queries hit one disk region. - Uniqueness constraints are tenant-scoped — two tenants can have customers with the same email. - A single shared schema scales operationally to thousands of tenants with one set of migrations.

Edge cases handled: - Tenant data wipe — `DELETE FROM customers WHERE tenant_id = ?` cascades; soft option flips `tenants.status` to `purged`. - Noisy neighbour — large tenant can be moved to its own DB; `tenant_id` becomes the shard key trivially.

Alternatives considered + rejected: - Schema-per-tenant — operationally heavy at 1000+ tenants, migration tooling becomes the bottleneck.

What NOT to say: - "The application middleware will always set `tenant_id` — we don't need DB enforcement." A single missed filter leaks data; defence-in-depth via row-level security is essential.

Common follow-up: "A tenant has 50 percent of the total rows in some tables. What's your plan?"

Scenario166 · Multi-Tenant: Schema-Per-Tenant

Tags: Difficulty: Hard · Asked at: Freshworks, Postman, ChargeBee, Salesforce-India · Domain: Multi-tenancy

The prompt:

"Design a schema-per-tenant multi-tenant approach. Walk me through the catalogue, migrations, and when you'd prefer this over shared-schema."

Thinking out loud (what to verbalize): 1. "First, the entities — a shared 'control plane' schema with the tenants registry, and one identical 'app' schema per tenant on the data plane." 2. "Next, routing — application looks up the tenant's schema name from the control plane on session start and issues `SET search_path` (or its equivalent)." 3. "For indexes — same indexes per schema; the control plane stays small and global." 4. "Edge cases — running migrations across hundreds of schemas, cross-tenant reporting, and backups per tenant."

The schema:

```

-- Control plane (shared)
CREATE SCHEMA control;

CREATE TABLE control.tenants (
    tenant_id BIGSERIAL PRIMARY KEY,
    tenant_slug VARCHAR(60) UNIQUE NOT NULL,
    schema_name VARCHAR(63) UNIQUE NOT NULL,
    db_node VARCHAR(80),
    plan VARCHAR(30) NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'active',
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE control.schema_migrations (
    tenant_id BIGINT NOT NULL REFERENCES control.tenants(tenant_id),
    migration_id VARCHAR(50) NOT NULL,
    applied_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (tenant_id, migration_id)
);

-- Per-tenant data plane (example for one tenant)
CREATE SCHEMA tenant_acme;

CREATE TABLE tenant_acme.customers (
    customer_id BIGSERIAL PRIMARY KEY,
    email VARCHAR(255) UNIQUE NOT NULL,
    name VARCHAR(150),
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE tenant_acme.invoices (
    invoice_id BIGSERIAL PRIMARY KEY,
    customer_id BIGINT NOT NULL REFERENCES tenant_acme.customers(customer_id),
    invoice_number VARCHAR(30) UNIQUE NOT NULL,
    amount_paise BIGINT NOT NULL,
    status VARCHAR(20) NOT NULL,
    issued_at TIMESTAMP NOT NULL
);

```

Why each choice: - Strong isolation — schema boundary means a buggy query simply cannot touch another tenant's tables. - Easy per-tenant backup, restore, or export — the schema is the natural unit. - `schema_migrations` per tenant tracks rollout progress, so a migration can be applied incrementally.

Edge cases handled: - Migration storm — apply in batches, ten schemas at a time, with a queue table tracking pending tenants. - Cross-tenant reporting — use a separate ETL into a shared analytics DB; never query 500 schemas live.

Alternatives considered + rejected: - Database-per-tenant — even stronger isolation but operational nightmare past 50 customers; usually reserved for compliance-driven enterprise tier.

What NOT to say: - "We'll just `UNION ALL` across all 200 schemas for the admin dashboard." That breaks at scale 10.

Common follow-up: *"How do you add a new column to all 800 tenant schemas without locking each one for five minutes?"*

Scenario167 · Audit Log Table

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Zerodha, Groww · Domain: Audit

The prompt:

"Design an audit log for a banking app — every state change on key tables must be captured. Walk me through whether you'd use a single audit table, per-table audit tables, or in-row history columns."

Thinking out loud (what to verbalize): 1. *"First, the access pattern — writes are heavy, reads are rare (only for support and compliance), and queries are usually 'show me history for record X'."* 2. *"Next, the trade-off — a single generic audit_log is simple but loses typed columns; per-table is type-safe but multiplies tables."* 3. *"For indexes — entity-type and entity-id together, plus actor-id for forensic queries."* 4. *"Edge cases — append-only enforcement, retention, GDPR-style redaction of personal fields, and high write volume."*

The schema:

```

CREATE TABLE audit_log (
  audit_id BIGSERIAL,
  entity_type VARCHAR(50) NOT NULL,
  entity_id BIGINT NOT NULL,
  action VARCHAR(20) NOT NULL,
  actor_type VARCHAR(20) NOT NULL,
  actor_id BIGINT,
  before_state JSONB,
  after_state JSONB,
  changed_fields TEXT[],
  request_id VARCHAR(60),
  ip_address INET,
  occurred_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (audit_id, occurred_at)
) PARTITION BY RANGE (occurred_at);

CREATE TABLE audit_log_2026_05 PARTITION OF audit_log
  FOR VALUES FROM ('2026-05-01') TO ('2026-06-01');

CREATE INDEX idx_audit_entity ON audit_log (entity_type, entity_id, occurred_at DESC);
CREATE INDEX idx_audit_actor ON audit_log (actor_type, actor_id, occurred_at DESC);
CREATE INDEX idx_audit_request ON audit_log (request_id);

```

Why each choice: - Single audit_log with JSONB before/after — one table to query, flexible across entity types. - Monthly partitioning — old months can be detached and archived cheaply. -

`changed_fields` array — fast filter "show me only password-change events" without parsing JSON.

Edge cases handled: - Append-only — revoke UPDATE and DELETE permissions for the application role; only a scheduled archival job can detach old partitions. - Right-to-redact — a separate redactions table flags audit ids; reads join and mask, raw rows stay immutable.

Alternatives considered + rejected: - In-row history (start_date / end_date columns on every business table) — rejected because most queries don't need history, and current-state reads pay the cost forever.

What NOT to say: - "We'll log audits to a file and parse them later." Files are not queryable, and reconciliation against the DB becomes painful.

Common follow-up: "At 500 writes per second to your audit_log, what becomes the bottleneck first?"

Scenario168 · Soft Delete Pattern

Tags: Difficulty: Easy · Asked at: Freshworks, Zoho, Razorpay, Postman · Domain: General OLTP

The prompt:

"Add soft-delete semantics across the schema — users can be 'deleted' but recoverable for 30 days. Walk me through the column design, query implications, and unique-constraint pitfalls."

Thinking out loud (what to verbalize): 1. "First, the marker — a nullable `deleted_at` timestamp on every business table, indexed via a partial index." 2. "Next, query hygiene — every read needs `WHERE deleted_at IS NULL`; we either centralise this in a view or in the data-access layer." 3. "For indexes — partial indexes on `WHERE deleted_at IS NULL` keep them small and fast." 4. "Edge cases — unique constraints that should ignore deleted rows, foreign keys to deleted parents, and hard-delete after 30 days."

The schema:

```
CREATE TABLE customers (
  customer_id BIGSERIAL PRIMARY KEY,
  email VARCHAR(255) NOT NULL,
  name VARCHAR(150) NOT NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  deleted_at TIMESTAMP,
  deleted_by BIGINT,
  deletion_reason VARCHAR(255)
);

-- Email must be unique among live customers, but reusable after soft delete
CREATE UNIQUE INDEX uq_customers_email_active
  ON customers (email) WHERE deleted_at IS NULL;

CREATE INDEX idx_customers_active ON customers (created_at DESC)
  WHERE deleted_at IS NULL;

CREATE INDEX idx_customers_purge_due ON customers (deleted_at)
  WHERE deleted_at IS NOT NULL;

CREATE VIEW v_customers AS
  SELECT * FROM customers WHERE deleted_at IS NULL;
```

Why each choice: - Partial unique index lets the same email re-register after the previous account is soft-deleted. - `v_customers` view gives accidental-safety — joining to the view never accidentally pulls deleted rows. - `idx_customers_purge_due` makes the nightly purge job a fast bounded scan.

Edge cases handled: - Deleted parent, live child — child FK has `ON DELETE RESTRICT`, so soft-deleting the parent does not orphan; deletion job must cascade soft-deletes. - Audit visibility — admin/support views read the base table directly; only customer-facing queries use the view.

Alternatives considered + rejected: - Moving deleted rows to a `customers_archive` table — rejected because reverting takes a re-insert with a new id; soft delete preserves identity.

What NOT to say: - "I'll just add `is_deleted BOOLEAN` ." A boolean loses the timestamp; without "when" you cannot drive the 30-day purge.

Common follow-up: "A user wants to be permanently deleted today, not in 30 days. What changes?"

Scenario169 · Append-Only Event Log

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Slice, Jupiter · Domain: Event sourcing

The prompt:

"Design an append-only event log for an event-sourced order service. Walk me through the table, ordering guarantees, and how readers rebuild state."

Thinking out loud (what to verbalize): 1. "First, the entities — events keyed by aggregate (`order_id`), versioned per aggregate, with a global monotonically-increasing sequence." 2. "Next, immutability — only `INSERT` is permitted; the application role does not have `UPDATE/DELETE`." 3. "For indexes — primary key by (`aggregate_id`, `version`), secondary on `global_seq` for projections." 4. "Edge cases — concurrent writes to the same aggregate, projection backfills, and snapshotting old aggregates."

The schema:

```

CREATE TABLE events (
  event_id BIGSERIAL PRIMARY KEY,
  aggregate_type VARCHAR(50) NOT NULL,
  aggregate_id BIGINT NOT NULL,
  aggregate_version INTEGER NOT NULL,
  event_type VARCHAR(80) NOT NULL,
  payload JSONB NOT NULL,
  metadata JSONB,
  occurred_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE (aggregate_type, aggregate_id, aggregate_version)
);

CREATE INDEX idx_events_aggregate ON events
  (aggregate_type, aggregate_id, aggregate_version);
CREATE INDEX idx_events_global ON events (event_id);
CREATE INDEX idx_events_type ON events (event_type, occurred_at);

CREATE TABLE snapshots (
  aggregate_type VARCHAR(50) NOT NULL,
  aggregate_id BIGINT NOT NULL,
  aggregate_version INTEGER NOT NULL,
  state JSONB NOT NULL,
  taken_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (aggregate_type, aggregate_id, aggregate_version)
);

```

Why each choice: - `UNIQUE (aggregate_type, aggregate_id, aggregate_version)` is the optimistic concurrency lock — two parallel writers cannot both insert version 7. - `event_id BIGSERIAL` gives a global ordering that downstream projections can checkpoint against. - Snapshots make replay bounded — rebuild reads "snapshot version N" then events after N.

Edge cases handled: - Concurrent writes — second writer fails the unique constraint, retries with re-read latest version. - Projection backfill — a new projection starts at `event_id = 0`, processes in order, checkpoints regularly.

Alternatives considered + rejected: - Mutable state table with an `events` audit on the side — rejected because it stops being the source of truth; events-as-the-truth makes rebuilds and time-travel trivial.

What NOT to say: - "I'll allow updates to old events to fix typos." That breaks every downstream consumer that already saw the old data.

Common follow-up: *"How would you garbage-collect events for an aggregate that the user has the right to be forgotten from?"*

Scenario170 · Outbox Pattern

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Postman, Chargebee · Domain: Reliable messaging

The prompt:

"Design an outbox table for reliably publishing events to a message queue from the same transaction as the business write. Walk me through how the relay works."

Thinking out loud (what to verbalize): 1. "First, the problem — we cannot atomically write to the DB and publish to Kafka. The outbox solves it by making the publish a DB write." 2. "Next, the relay — a separate process polls (or tails the WAL) and ships outbox rows to the queue, marking them dispatched." 3. "For indexes — partial index on undischatched rows so the relay scan is cheap." 4. "Edge cases — at-least-once delivery, ordering within an aggregate, and outbox table bloat."

The schema:

```
CREATE TABLE outbox (  
    outbox_id BIGSERIAL PRIMARY KEY,  
    aggregate_type VARCHAR(50) NOT NULL,  
    aggregate_id BIGINT NOT NULL,  
    event_type VARCHAR(80) NOT NULL,  
    payload JSONB NOT NULL,  
    target_topic VARCHAR(100) NOT NULL,  
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    dispatched_at TIMESTAMP,  
    dispatch_attempts INTEGER NOT NULL DEFAULT 0,  
    next_attempt_at TIMESTAMP  
);  
  
CREATE INDEX idx_outbox_pending ON outbox (created_at)  
    WHERE dispatched_at IS NULL;  
  
CREATE INDEX idx_outbox_retry ON outbox (next_attempt_at)  
    WHERE dispatched_at IS NULL AND dispatch_attempts > 0;
```

Why each choice: - Same transaction as the business write — atomicity is free. - `next_attempt_at` plus `dispatch_attempts` give exponential backoff. - Partial index keeps the relay query $O(\text{undispatched})$, not $O(\text{total})$.

Edge cases handled: - At-least-once — consumers must be idempotent; we send a stable `outbox_id` as the message key. - Ordering — relay processes rows in `created_at` order per aggregate, partitioning the queue by `aggregate_id`.

Alternatives considered + rejected: - Distributed transactions (XA) between DB and broker — rejected because they are operationally fragile and most modern brokers don't support them well.

What NOT to say: - "After publishing, the application calls `UPDATE outbox` ." If the app crashes after publish but before update, you'll re-publish — fine for at-least-once, but design the consumer for it.

Common follow-up: *"The outbox has 50 million rows because the cleanup job stopped. How do you recover without locking the table?"*

Scenario171 · Idempotency Key Tracking

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Stripe-India, PhonePe · Domain: Payments

The prompt:

"Design an idempotency-key table for a payments API — the same request retried with the same key must return the same response. Walk me through the table and the request-handling flow."

Thinking out loud (what to verbalize): 1. *"First, the entities — keyed by (merchant_id, idempotency_key) with stored request hash, response, and status."* 2. *"Next, the flow — on request: insert lock, on success: store response, on retry: return stored response if hash matches."* 3. *"For indexes — PK is the lookup; TTL index for cleanup after 24 hours."* 4. *"Edge cases — same key with different payload, in-flight requests, and key collisions across merchants."*

The schema:

```
CREATE TABLE idempotency_keys (  
    merchant_id BIGINT NOT NULL,  
    idempotency_key VARCHAR(80) NOT NULL,  
    request_hash CHAR(64) NOT NULL,  
    request_path VARCHAR(200) NOT NULL,  
    status VARCHAR(20) NOT NULL DEFAULT 'in_progress',  
    response_status INTEGER,  
    response_body JSONB,  
    locked_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    completed_at TIMESTAMP,  
    expires_at TIMESTAMP NOT NULL,  
    PRIMARY KEY (merchant_id, idempotency_key)  
);  
  
CREATE INDEX idx_idem_expiry ON idempotency_keys (expires_at);  
CREATE INDEX idx_idem_inflight ON idempotency_keys (locked_at)  
    WHERE status = 'in_progress';
```

Why each choice: - Composite PK scopes keys to the merchant — two merchants cannot collide. - `request_hash` lets us detect "same key, different body" and reject with 422. - `status = 'in_progress'` blocks concurrent retries from creating duplicate side effects.

Edge cases handled: - Same key, different payload — return 422 with

`idempotency_key_mismatch` instead of executing. - In-flight retry — second call sees `in_progress`, waits up to a timeout or returns 409.

Alternatives considered + rejected: - Caching idempotency in Redis only — rejected because we lose durability on Redis restart; DB-backed with optional Redis cache in front is safer.

What NOT to say: - "We'll just use the request hash as the dedup key." That breaks legitimately-different requests that happen to look alike, and prevents the client from retrying explicitly.

Common follow-up: *"A merchant sends one million requests per day. What's the storage and cleanup plan?"*

Scenario172 · Notification Preferences

Tags: Difficulty: Medium · Asked at: Swiggy, Zomato, Flipkart, Cred · Domain: Engagement

The prompt:

"Design notification preferences — users opt in or out per channel (email, SMS, push, WhatsApp) and per topic (order, marketing, security). Walk me through the schema and how a transactional 'critical security alert' overrides preferences."

Thinking out loud (what to verbalize): 1. *"First, the entities — users, channels, topics, and a preferences table holding the (user × channel × topic) opt state."* 2. *"Next, defaults — we don't insert a row per combination; absence means default-opt-state, which the topic catalogue dictates."* 3. *"For indexes — PK on (user, topic, channel); reads are 'should I send this'."* 4. *"Edge cases — critical overrides, quiet hours, channel-level kill switch, and unsubscribe-all."*

The schema:

```
CREATE TABLE topics (  
    topic_id SERIAL PRIMARY KEY,  
    topic_code VARCHAR(60) UNIQUE NOT NULL,  
    category VARCHAR(20) NOT NULL,  
    default_opt_in BOOLEAN NOT NULL DEFAULT TRUE,  
    is_critical BOOLEAN NOT NULL DEFAULT FALSE  
);  
  
CREATE TABLE channels (  
    channel_id SERIAL PRIMARY KEY,  
    channel_code VARCHAR(20) UNIQUE NOT NULL  
);  
  
CREATE TABLE user_notification_prefs (  
    user_id BIGINT NOT NULL,  
    topic_id INTEGER NOT NULL REFERENCES topics(topic_id),  
    channel_id INTEGER NOT NULL REFERENCES channels(channel_id),  
    opted_in BOOLEAN NOT NULL,  
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    PRIMARY KEY (user_id, topic_id, channel_id)  
);  
  
CREATE TABLE user_channel_kill_switch (  
    user_id BIGINT NOT NULL,  
    channel_id INTEGER NOT NULL REFERENCES channels(channel_id),  
    quiet_start TIME,  
    quiet_end TIME,  
    suppressed BOOLEAN NOT NULL DEFAULT FALSE,  
    PRIMARY KEY (user_id, channel_id)  
);
```

Why each choice: - We don't pre-populate every (user × topic × channel) row — sparse storage; missing row falls back to `topics.default_opt_in`. - Critical topics ignore prefs at send time — the catalogue is the source of truth, not millions of pref rows. - Channel kill switch is separate so a user can mute SMS at night without rewriting topic prefs.

Edge cases handled: - Critical override — sender checks `is_critical` and bypasses prefs and quiet hours. - Unsubscribe-all — a single update flips kill-switch for all channels in one row each.

Alternatives considered + rejected: - Storing prefs as a JSON blob on users — rejected because per-topic lookups and analytics on opt-out rates become impossible.

What NOT to say: - "We'll send marketing pushes even if the user opted out, because engagement metrics matter." Compliance and brand-trust suicide.

Common follow-up: *"How would you let a user 'snooze' a topic for 7 days without permanently changing their preference?"*

Scenario173 · Subscription / Recurring Billing

Tags: Difficulty: Hard · Asked at: Chargebee, Zoho, Freshworks, ChargeMod · Domain: Billing

The prompt:

"Design tables for a recurring-billing system — customers, subscriptions, plans, billing cycles, invoices. Walk me through how mid-cycle plan changes prorate."

Thinking out loud (what to verbalize): 1. "First, the entities — *plans* (catalogue), *subscriptions* (an instance with a customer and a plan), *billing_cycles* (each period), *invoices* (the bill for a cycle), and *subscription_events* (audit of changes)." 2. "Next, relationships — *subscription* has many *billing_cycles*, each cycle produces one invoice; plan changes create a new *subscription_event* row." 3. "For indexes — *subscription* by *next_renewal_at* to drive the billing job; *invoices* by *status* and *due_date*." 4. "Edge cases — proration on plan change, pause/resume, failed payments with retry, and tax changes."

The schema:


```
CREATE TABLE plans (  
    plan_id SERIAL PRIMARY KEY,  
    plan_code VARCHAR(50) UNIQUE NOT NULL,  
    billing_period VARCHAR(20) NOT NULL,  
    price_paise BIGINT NOT NULL,  
    is_active BOOLEAN NOT NULL DEFAULT TRUE  
);  
  
CREATE TABLE subscriptions (  
    subscription_id BIGSERIAL PRIMARY KEY,  
    customer_id BIGINT NOT NULL,  
    plan_id INTEGER NOT NULL REFERENCES plans(plan_id),  
    status VARCHAR(20) NOT NULL DEFAULT 'active',  
    started_at DATE NOT NULL,  
    current_period_start DATE NOT NULL,  
    current_period_end DATE NOT NULL,  
    next_renewal_at DATE NOT NULL,  
    cancelled_at TIMESTAMP  
);  
  
CREATE TABLE subscription_events (  
    event_id BIGSERIAL PRIMARY KEY,  
    subscription_id BIGINT NOT NULL REFERENCES subscriptions(subscription_id),  
    event_type VARCHAR(30) NOT NULL,  
    old_plan_id INTEGER,  
    new_plan_id INTEGER,  
    effective_at TIMESTAMP NOT NULL,  
    proration_amount_paise BIGINT,  
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE invoices (  
    invoice_id BIGSERIAL PRIMARY KEY,  
    subscription_id BIGINT NOT NULL REFERENCES subscriptions(subscription_id),  
    period_start DATE NOT NULL,  
    period_end DATE NOT NULL,  
    subtotal_paise BIGINT NOT NULL,  
    tax_paise BIGINT NOT NULL,  
    total_paise BIGINT NOT NULL,  
    status VARCHAR(20) NOT NULL,  
    issued_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    due_at TIMESTAMP NOT NULL,  
    paid_at TIMESTAMP  
);  
  
CREATE INDEX idx_subs_renewal ON subscriptions (next_renewal_at)  
    WHERE status = 'active';  
CREATE INDEX idx_invoices_due ON invoices (due_at)  
    WHERE status = 'unpaid';
```

Why each choice: - Plan as separate catalogue — price changes don't rewrite history; subscription rows pin the plan_id used in their period. - `subscription_events` is the audit and the proration ledger combined. - `next_renewal_at` indexed partial — the billing job is a tight scan.

Edge cases handled: - Plan change mid-cycle — insert `subscription_events` row with proration delta, current period ends today, new period begins on new plan. - Failed payment — invoice stays `unpaid`, dunning job retries with backoff, subscription flips to `past_due` after threshold.

Alternatives considered + rejected: - Storing current price on the subscription row directly — rejected because plan-catalogue edits should be auditable, and subscriptions can have grandfathered prices via an explicit override column or row.

What NOT to say: - "We'll just delete the subscription on cancel." Cancellation is a state change, not deletion; you need history for tax, reactivation, and metrics.

Common follow-up: *"How would you support a free trial with a credit card that gets charged only at trial end?"*

Scenario174 · Coupon / Discount Engine

Tags: Difficulty: Hard · Asked at: Flipkart, Myntra, Swiggy, Zomato · Domain: Commerce

The prompt:

"Design tables for a coupon engine — percent off, flat off, free shipping, first-time-user-only, capped usage. Walk me through eligibility checks and atomic redemption."

Thinking out loud (what to verbalize): 1. *"First, the entities — coupons (definition), coupon_rules (eligibility constraints), coupon_redemptions (audit + counter)."* 2. *"Next, eligibility — applied at checkout via a service that reads rules and runs them; some rules are SQL-checkable, some require app logic."* 3. *"For indexes — coupon_code for lookup, redemptions by coupon_id and user_id for cap checks."* 4. *"Edge cases — global usage cap race conditions, per-user cap, stacking rules, expiry, and fraud."*

The schema:

```

CREATE TABLE coupons (
    coupon_id BIGSERIAL PRIMARY KEY,
    coupon_code VARCHAR(40) UNIQUE NOT NULL,
    discount_type VARCHAR(20) NOT NULL,
    discount_value_paise BIGINT,
    discount_percent NUMERIC(5,2),
    max_discount_paise BIGINT,
    min_cart_paise BIGINT,
    starts_at TIMESTAMP NOT NULL,
    expires_at TIMESTAMP NOT NULL,
    total_usage_cap INTEGER,
    per_user_cap INTEGER,
    redemption_count INTEGER NOT NULL DEFAULT 0,
    is_stackable BOOLEAN NOT NULL DEFAULT FALSE,
    is_active BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE coupon_rules (
    rule_id BIGSERIAL PRIMARY KEY,
    coupon_id BIGINT NOT NULL REFERENCES coupons(coupon_id),
    rule_type VARCHAR(40) NOT NULL,
    rule_config JSONB NOT NULL
);

CREATE TABLE coupon_redemptions (
    redemption_id BIGSERIAL PRIMARY KEY,
    coupon_id BIGINT NOT NULL REFERENCES coupons(coupon_id),
    user_id BIGINT NOT NULL,
    order_id BIGINT NOT NULL,
    discount_applied_paise BIGINT NOT NULL,
    redeemed_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE UNIQUE INDEX uq_coupon_order ON coupon_redemptions (coupon_id, order_id);
CREATE INDEX idx_redemptions_user ON coupon_redemptions (coupon_id, user_id);
CREATE INDEX idx_coupons_active ON coupons (coupon_code)
    WHERE is_active = TRUE;

```

Why each choice: - `redemption_count` on coupons cached via atomic `UPDATE ... WHERE redemption_count < total_usage_cap` — race-safe. - Rules in a child table with JSONB config — adding a new rule type doesn't change the schema. - Unique `(coupon_id, order_id)` prevents double-redemption on the same order.

Edge cases handled: - Global cap race — `UPDATE coupons SET redemption_count = redemption_count + 1 WHERE coupon_id = ? AND (total_usage_cap IS NULL OR redemption_count < total_usage_cap) RETURNING redemption_count` — zero rows means cap exhausted. - Per-user cap — `SELECT COUNT(*) FROM coupon_redemptions WHERE coupon_id = ? AND user_id = ?` inside the same transaction as the order.

Alternatives considered + rejected: - Decrementing a separate `remaining_uses` column — same idea, just sign-flipped; not better.

What NOT to say: - "We'll check the cap then increment in two statements." That's a textbook race condition.

Common follow-up: *"How would you support a coupon that gives 20 percent off only on selected categories within the cart?"*

Scenario175 · Shopping Cart with Anonymous-to-Logged-In Merge

Tags: Difficulty: Medium · Asked at: Flipkart, Myntra, Nykaa, Meesho · Domain: Commerce

The prompt:

"Design a shopping cart that supports anonymous browsing and merges with the user's cart on login. Walk me through the schema and the merge algorithm."

Thinking out loud (what to verbalize): 1. *"First, the entities — carts keyed by either `user_id` or `anonymous_token`, and `cart_items` as lines."* 2. *"Next, the merge — on login, take the anonymous cart's items, dedupe by SKU into the user cart, sum quantities, delete the anonymous cart."* 3. *"For indexes — cart by `user_id` (unique) and by `anonymous_token` (unique), items by `cart_id`."* 4. *"Edge cases — both carts have the same SKU, stock validation at merge, price changes since add."*

The schema:

```
CREATE TABLE carts (  
    cart_id BIGSERIAL PRIMARY KEY,  
    user_id BIGINT UNIQUE,  
    anonymous_token CHAR(36) UNIQUE,  
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    last_active_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    CHECK ((user_id IS NOT NULL)::int + (anonymous_token IS NOT NULL)::int = 1)  
);  
  
CREATE TABLE cart_items (  
    cart_item_id BIGSERIAL PRIMARY KEY,  
    cart_id BIGINT NOT NULL REFERENCES carts(cart_id) ON DELETE CASCADE,  
    sku VARCHAR(60) NOT NULL,  
    product_id BIGINT NOT NULL,  
    quantity INTEGER NOT NULL CHECK (quantity > 0),  
    added_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    UNIQUE (cart_id, sku)  
);  
  
CREATE INDEX idx_carts_anonymous_stale ON carts (last_active_at)  
WHERE user_id IS NULL;
```

Why each choice: - CHECK constraint enforces "either anonymous or logged in, never both, never neither". - `UNIQUE (cart_id, sku)` makes "add again" a single UPSERT. - Cascade delete on cart cleanup means anonymous-cart purge is a single DELETE.

Edge cases handled: - Both carts have same SKU — UPSERT with quantity = anonymous.qty + user.qty, capped at per-customer max if any. - Stock validation — done at view time and again at checkout; cart row doesn't enforce stock.

Alternatives considered + rejected: - Keeping the anonymous cart in cookies/localStorage only — rejected because cross-device continuity and basket-abandonment campaigns need server-side state.

What NOT to say: - "On login, just delete the old cart and use the anonymous one." The user lost their saved-for-later items; merge, don't replace.

Common follow-up: *"How long do you keep anonymous carts before purging, and what's the storage trade-off?"*

Scenario176 · Wishlist with Collaborative Sharing

Tags: Difficulty: Medium · Asked at: Myntra, Nykaa, Flipkart, BookMyShow · Domain: Social commerce

The prompt:

"Design a wishlist that supports private lists, shareable read-only links, and collaborative editing by invited friends. Walk me through the access-control schema."

Thinking out loud (what to verbalize): 1. "First, the entities — wishlists (owner, name, visibility), wishlist_items, wishlist_collaborators (per-list ACL), wishlist_share_links." 2. "Next, visibility — three modes: private, link, public. Collaborator role is separate from visibility." 3. "For indexes — owner_id for 'my lists', share_token for resolving public links." 4. "Edge cases — revoking access, transferring ownership, deleted items, and link rotation."

The schema:

```
CREATE TABLE wishlists (  
    wishlist_id BIGSERIAL PRIMARY KEY,  
    owner_user_id BIGINT NOT NULL,  
    name VARCHAR(100) NOT NULL,  
    visibility VARCHAR(20) NOT NULL DEFAULT 'private',  
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE wishlist_items (  
    wishlist_item_id BIGSERIAL PRIMARY KEY,  
    wishlist_id BIGINT NOT NULL REFERENCES wishlists(wishlist_id) ON DELETE CASCADE,  
    product_id BIGINT NOT NULL,  
    added_by_user_id BIGINT NOT NULL,  
    note VARCHAR(255),  
    added_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    UNIQUE (wishlist_id, product_id)  
);  
  
CREATE TABLE wishlist_collaborators (  
    wishlist_id BIGINT NOT NULL REFERENCES wishlists(wishlist_id) ON DELETE CASCADE,  
    user_id BIGINT NOT NULL,  
    role VARCHAR(20) NOT NULL DEFAULT 'editor',  
    invited_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    PRIMARY KEY (wishlist_id, user_id)  
);  
  
CREATE TABLE wishlist_share_links (  
    share_token CHAR(22) PRIMARY KEY,  
    wishlist_id BIGINT NOT NULL REFERENCES wishlists(wishlist_id) ON DELETE CASCADE,  
    access_level VARCHAR(20) NOT NULL DEFAULT 'view',  
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    expires_at TIMESTAMP,  
    revoked_at TIMESTAMP  
);  
  
CREATE INDEX idx_wishlists_owner ON wishlists (owner_user_id);  
CREATE INDEX idx_share_links_active ON wishlist_share_links (wishlist_id)  
    WHERE revoked_at IS NULL;
```

Why each choice: - Visibility (anyone with link) is separate from collaborators (named users) — orthogonal access modes. - `added_by_user_id` on items shows attribution in shared lists. - Multiple share links per list with their own access levels allows view-only and editor links coexisting.

Edge cases handled: - Revocation — `UPDATE wishlist_share_links SET revoked_at = NOW()` invalidates a leaked link without disturbing collaborators. - Ownership transfer — single UPDATE on `wishlists.owner_user_id`; old owner can be added as collaborator if desired.

Alternatives considered + rejected: - One ACL row per (user × wishlist) including the owner — uniform but requires constant joins for the common "is this mine" check.

What NOT to say: - "We'll just append the user_id to a JSON array of editors." Bad for indexing, bad for revocation, bad for joins.

Common follow-up: "How do you show 'item added by Priya' on a 50-person collaborative list without $N+1$ joins?"

Scenario177 · Inventory + Reservation (Prevent Overselling)

Tags: Difficulty: Hard · Asked at: Flipkart, BigBasket, BookMyShow, Swiggy Instamart · Domain: Inventory

The prompt:

"Design inventory tables that prevent overselling — multiple customers try to buy the last unit at the same instant. Walk me through reservation, expiry, and confirmation."

Thinking out loud (what to verbalize): 1. "First, the entities — inventory (per SKU per warehouse with available and reserved counts), reservations (held units with TTL)." 2. "Next, the flow — add-to-cart reserves, expiry releases, order-confirm converts reservation to a stock movement." 3. "For indexes — inventory by (sku, warehouse) for the hot update; reservations by expires_at for the sweeper." 4. "Edge cases — concurrent reservations, partial reservation, expiry race with confirmation, warehouse failure."

The schema:

```

CREATE TABLE inventory (
    sku VARCHAR(60) NOT NULL,
    warehouse_id INTEGER NOT NULL,
    available_qty INTEGER NOT NULL CHECK (available_qty >= 0),
    reserved_qty INTEGER NOT NULL DEFAULT 0 CHECK (reserved_qty >= 0),
    version INTEGER NOT NULL DEFAULT 0,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (sku, warehouse_id)
);

CREATE TABLE reservations (
    reservation_id BIGSERIAL PRIMARY KEY,
    sku VARCHAR(60) NOT NULL,
    warehouse_id INTEGER NOT NULL,
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    user_id BIGINT NOT NULL,
    cart_id BIGINT,
    expires_at TIMESTAMP NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'held',
    FOREIGN KEY (sku, warehouse_id) REFERENCES inventory(sku, warehouse_id)
);

CREATE TABLE stock_movements (
    movement_id BIGSERIAL PRIMARY KEY,
    sku VARCHAR(60) NOT NULL,
    warehouse_id INTEGER NOT NULL,
    movement_type VARCHAR(30) NOT NULL,
    quantity_delta INTEGER NOT NULL,
    reference_id BIGINT,
    occurred_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_reservations_expiry ON reservations (expires_at)
    WHERE status = 'held';
CREATE INDEX idx_movements_sku ON stock_movements (sku, warehouse_id, occurred_at);

```

Why each choice: - Reserve via `UPDATE inventory SET available_qty = available_qty - ?, reserved_qty = reserved_qty + ? WHERE ... AND available_qty >= ?` — atomic; zero rows means insufficient. - Stock movements are the immutable truth; inventory row is a cached aggregate, reconcilable. - Reservations have TTL so abandoned carts don't lock inventory.

Edge cases handled: - Concurrent reservations — DB-level conditional UPDATE is the lock; no separate `SELECT FOR UPDATE` needed. - Expiry race — sweeper does `UPDATE reservations SET status = 'expired' WHERE expires_at < NOW() AND status = 'held'` and reverses the inventory update; confirmation does the opposite, checking status = 'held' first.

Alternatives considered + rejected: - Holding inventory only in Redis — fast but durability is uncertain; DB-backed with optional Redis cache is the standard.

What NOT to say: - "We'll fetch available, check it's $\geq$ requested, then update." Race condition; you'll oversell.

Common follow-up: *"How would you support buying across two warehouses when neither has enough alone?"*

Scenario178 · Booking System (Resource × Time-Slot)

Tags: Difficulty: Hard · Asked at: BookMyShow, MakeMyTrip, Cleartrip, Urban Company · Domain: Booking

The prompt:

"Design a booking system where customers book resources (a doctor, a meeting room, a salon chair) for time slots. Walk me through how you prevent double-booking the same slot."

Thinking out loud (what to verbalize): 1. *"First, the entities — resources, availability rules (recurring), bookings (the actual reservation), and optionally pre-generated slots."* 2. *"Next, the conflict model — exclude-style constraint on overlapping ranges, or generated discrete slots with a unique key."* 3. *"For indexes — bookings by (resource_id, start_at, end_at); availability by resource_id."* 4. *"Edge cases — timezones, overlapping reschedule, recurring bookings, blackout dates."*

The schema:

```

CREATE TABLE resources (
    resource_id BIGSERIAL PRIMARY KEY,
    resource_type VARCHAR(30) NOT NULL,
    name VARCHAR(150) NOT NULL,
    timezone VARCHAR(50) NOT NULL DEFAULT 'Asia/Kolkata',
    is_active BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE availability_rules (
    rule_id BIGSERIAL PRIMARY KEY,
    resource_id BIGINT NOT NULL REFERENCES resources(resource_id),
    day_of_week SMALLINT NOT NULL CHECK (day_of_week BETWEEN 0 AND 6),
    start_time TIME NOT NULL,
    end_time TIME NOT NULL,
    slot_minutes INTEGER NOT NULL
);

CREATE TABLE blackouts (
    blackout_id BIGSERIAL PRIMARY KEY,
    resource_id BIGINT NOT NULL REFERENCES resources(resource_id),
    starts_at TIMESTAMPTZ NOT NULL,
    ends_at TIMESTAMPTZ NOT NULL,
    reason VARCHAR(255)
);

CREATE TABLE bookings (
    booking_id BIGSERIAL PRIMARY KEY,
    resource_id BIGINT NOT NULL REFERENCES resources(resource_id),
    customer_id BIGINT NOT NULL,
    starts_at TIMESTAMPTZ NOT NULL,
    ends_at TIMESTAMPTZ NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'confirmed',
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    EXCLUDE USING GIST (
        resource_id WITH =,
        tstzrange(starts_at, ends_at) WITH &&
    ) WHERE (status = 'confirmed')
);

CREATE INDEX idx_bookings_resource_time ON bookings (resource_id, starts_at);

```

Why each choice: - GIST exclude constraint physically prevents two **confirmed** bookings from overlapping on the same resource — the DB rejects the second insert. - Timestamps stored in UTC (TIMESTAMPTZ); resource carries its display timezone. - Availability rules and blackouts are checked in the application before insert, but the exclusion constraint is the safety net.

Edge cases handled: - Concurrent double-booking — exclusion constraint causes the second transaction to fail; the app retries or shows "slot taken". - Reschedule — **UPDATE bookings SET starts_at = ?, ends_at = ?** re-evaluates the exclude; new conflict raises.

Alternatives considered + rejected: - Pre-generating one row per slot per day — works for fixed-grid scheduling like cinemas but doesn't fit flexible-duration bookings.

What NOT to say: - "We'll just check `WHERE NOT EXISTS overlapping booking` then insert."
Race condition without the exclude constraint.

Common follow-up: *"How would you support a booking that requires two resources simultaneously, like a doctor plus a room?"*

Scenario179 · Friend / Connection Graph (Mutual)

Tags: Difficulty: Medium · Asked at: Hike, ShareChat, Josh, Koo · Domain: Social

The prompt:

"Design a mutual friendship graph — connection is symmetric, requires both sides to accept.
Walk me through how you'd model it without storing each pair twice."

Thinking out loud (what to verbalize): 1. "First, the entities — `friend_requests` (directional, pending state) and `friendships` (mutual, confirmed)." 2. "Next, the trick — store the pair with `LEAST(a, b)` and `GREATEST(a, b)` to avoid duplicate (1,2) and (2,1) rows." 3. "For indexes — friendships need lookups from either side, so we add both `(user_low, user_high)` PK and a redundant index on `(user_high)` ." 4. "Edge cases — block, unfriend, simultaneous requests, and mutual-friend queries at scale."

The schema:

```
CREATE TABLE friend_requests (  
    request_id BIGSERIAL PRIMARY KEY,  
    requester_id BIGINT NOT NULL,  
    recipient_id BIGINT NOT NULL,  
    status VARCHAR(20) NOT NULL DEFAULT 'pending',  
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    responded_at TIMESTAMP,  
    UNIQUE (requester_id, recipient_id),  
    CHECK (requester_id <> recipient_id)  
);  
  
CREATE TABLE friendships (  
    user_low BIGINT NOT NULL,  
    user_high BIGINT NOT NULL,  
    became_friends_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    PRIMARY KEY (user_low, user_high),  
    CHECK (user_low < user_high)  
);  
  
CREATE INDEX idx_friendships_high ON friendships (user_high, user_low);  
  
CREATE TABLE blocks (  
    blocker_id BIGINT NOT NULL,  
    blocked_id BIGINT NOT NULL,  
    blocked_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    PRIMARY KEY (blocker_id, blocked_id)  
);
```

Why each choice: - Canonical ordering `user_low < user_high` halves storage and dedupes the relationship. - Index on `(user_high, user_low)` lets us look up friends from either direction with the same plan. - Block is directional and separate — A blocks B doesn't imply B blocks A.

Edge cases handled: - Simultaneous requests — the unique constraint catches the duplicate; the second one converts the first to `accepted` at the application layer. - Unfriend — single DELETE on friendships; friend_requests history stays for analytics.

Alternatives considered + rejected: - Storing both (A, B) and (B, A) rows — symmetric but 2× storage and update hazard if one row gets missed.

What NOT to say: - "We can ignore the ordering trick because storage is cheap." It's not just storage — it's the dedup invariant; without it you can have inconsistent state.

Common follow-up: "How would you compute 'mutual friends with X' for a user with 5000 friends?"

Scenario180 · Followers / Following (Asymmetric)

Tags: Difficulty: Medium · Asked at: ShareChat, Koo, Josh, MX TakaTak · Domain: Social

The prompt:

"Design follower/following relationships — asymmetric, like Instagram. Walk me through 'who follows me' and 'who I follow' queries at scale, and how a private account gates follow requests."

Thinking out loud (what to verbalize): 1. "First, the entities — follows (one row per A-follows-B), follow_requests for private accounts." 2. "Next, queries — 'followers of X' = scan by followee_id, 'following list of X' = scan by follower_id." 3. "For indexes — both (follower_id, followee_id) and (followee_id, follower_id) so both directions are O(1)." 4. "Edge cases — private accounts, mute (we follow but hide their posts), and fan-out on follow change."

The schema:

```
CREATE TABLE follows (
    follower_id BIGINT NOT NULL,
    followee_id BIGINT NOT NULL,
    followed_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    is_muted BOOLEAN NOT NULL DEFAULT FALSE,
    notify_on_post BOOLEAN NOT NULL DEFAULT FALSE,
    PRIMARY KEY (follower_id, followee_id),
    CHECK (follower_id <> followee_id)
);

CREATE INDEX idx_follows_followee ON follows (followee_id, followed_at DESC);

CREATE TABLE follow_requests (
    request_id BIGSERIAL PRIMARY KEY,
    requester_id BIGINT NOT NULL,
    target_id BIGINT NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'pending',
    requested_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    responded_at TIMESTAMP,
    UNIQUE (requester_id, target_id)
);

CREATE TABLE user_privacy (
    user_id BIGINT PRIMARY KEY,
    is_private BOOLEAN NOT NULL DEFAULT FALSE
);
```

Why each choice: - PK leads with follower_id (for "who I follow"); secondary index leads with followee_id (for "who follows me"). - Mute and notify_on_post on the relationship row — per-edge attributes. - Private flag on a separate `user_privacy` row to keep the hot user table thin.

Edge cases handled: - Private account — follow request first; on accept, app deletes the request row and inserts into follows. - Mute — keeps the follow row but the feed builder filters; muted user doesn't know.

Alternatives considered + rejected: - Counter columns (`follower_count`) maintained by triggers — fragile; we recompute or maintain via outbox events for eventual consistency.

What NOT to say: - "We'll just count rows on every profile view." Fine at small scale; at 10M+ profiles it kills your DB.

Common follow-up: *"A celebrity has 10 million followers. How does posting a new photo fan out to their feeds?"*

Scenario181 · Chat / Messaging (1-1 and Group)

Tags: Difficulty: Hard · Asked at: ShareChat, Hike, Koo, Slack-India · Domain: Messaging

The prompt:

"Design tables for 1-1 and group chat in one model. Walk me through conversations, participants, messages, and how you handle 'last read at' for unread counts."

Thinking out loud (what to verbalize): 1. *"First, unify by treating every chat (1-1 or group) as a 'conversation' with 2 or N participants — fewer code paths."* 2. *"Next, the entities — conversations, conversation_participants, messages, message_attachments."* 3. *"For indexes — messages by (conversation_id, sent_at DESC) for the timeline; participants by user_id for 'my chats'."* 4. *"Edge cases — large groups (10000 members), message edits, leave/rejoin, soft-delete for the sender vs everyone."*

The schema:

```

CREATE TABLE conversations (
    conversation_id BIGSERIAL PRIMARY KEY,
    conversation_type VARCHAR(20) NOT NULL,
    name VARCHAR(150),
    created_by_user_id BIGINT NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    last_message_at TIMESTAMP
);

CREATE TABLE conversation_participants (
    conversation_id BIGINT NOT NULL REFERENCES conversations(conversation_id),
    user_id BIGINT NOT NULL,
    joined_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    left_at TIMESTAMP,
    role VARCHAR(20) NOT NULL DEFAULT 'member',
    last_read_message_id BIGINT,
    last_read_at TIMESTAMP,
    PRIMARY KEY (conversation_id, user_id)
);

CREATE TABLE messages (
    message_id BIGSERIAL PRIMARY KEY,
    conversation_id BIGINT NOT NULL REFERENCES conversations(conversation_id),
    sender_user_id BIGINT NOT NULL,
    body TEXT,
    sent_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    edited_at TIMESTAMP,
    deleted_for_everyone_at TIMESTAMP
);

CREATE TABLE message_attachments (
    attachment_id BIGSERIAL PRIMARY KEY,
    message_id BIGINT NOT NULL REFERENCES messages(message_id),
    media_type VARCHAR(30) NOT NULL,
    media_url VARCHAR(500) NOT NULL,
    size_bytes BIGINT
);

CREATE INDEX idx_messages_conv_time ON messages (conversation_id, sent_at DESC);
CREATE INDEX idx_participants_user ON conversation_participants (user_id)
    WHERE left_at IS NULL;

```

Why each choice: - Same model for 1-1 and group — fewer joins, no special cases. -

- `last_read_message_id` on participants gives O(1) unread = count of messages with id > last_read.
 - `deleted_for_everyone_at` is soft — the tombstone is needed to suppress on every reader's device.

Edge cases handled: - Leave and rejoin — `left_at` set, new participant row not created; on rejoin, clear `left_at`, history visibility decided by group policy. - Very large groups — separate read-receipt table is optional; for 10000-member groups you don't track per-user-per-message reads.

Alternatives considered + rejected: - Different tables for direct and group messages — duplicates everything; merge filters and timeline logic become painful.

What NOT to say: - "We'll show 'seen' for every recipient on every message in a 5000-person group." Storage and write amplification explode; only viable for small chats.

Common follow-up: *"How would you support message replies and threads — same table or new?"*

Scenario182 · Read-Receipt Tracking

Tags: Difficulty: Medium · Asked at: Hike, ShareChat, Slack-India, Postman · Domain: Messaging

The prompt:

"Design read receipts for a group chat — show 'seen by N' under a message. Walk me through the storage trade-off and the per-message-per-user table."

Thinking out loud (what to verbalize): 1. "First, two strategies — per-message-per-user explicit rows, or per-conversation-per-user `last_read_message_id` aggregate." 2. "Next, the trade-off — explicit gives per-message granularity but explodes (1M messages × 100 members = 100M rows); aggregate is $O(\text{participants})$ but loses per-message read history." 3. "For indexes — explicit needs (`message_id`) for fan-in display, aggregate needs no extra." 4. "Edge cases — read on multiple devices, retroactive 'mark all read', and read-receipt opt-out."

The schema:


```
CREATE TABLE message_read_receipts (  
    message_id BIGINT NOT NULL,  
    reader_user_id BIGINT NOT NULL,  
    read_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    PRIMARY KEY (message_id, reader_user_id)  
);  
  
CREATE INDEX idx_receipts_reader ON message_read_receipts (reader_user_id, read_at);  
  
CREATE TABLE conversation_read_state (  
    conversation_id BIGINT NOT NULL,  
    user_id BIGINT NOT NULL,  
    last_read_message_id BIGINT NOT NULL,  
    last_read_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    PRIMARY KEY (conversation_id, user_id)  
);  
  
CREATE TABLE user_chat_settings (  
    user_id BIGINT PRIMARY KEY,  
    show_read_receipts BOOLEAN NOT NULL DEFAULT TRUE  
);
```

Why each choice: - We keep both — `conversation_read_state` drives unread badges (cheap), `message_read_receipts` drives "seen by" UI (only inserted when receipts are on). - Insert receipts in batch when the user opens the conversation; not one row per server round-trip. - Setting opt-out skips inserts entirely for that user; the count just shows lower.

Edge cases handled: - Multiple devices — read on phone, then on laptop — the second insert is a no-op via PK conflict, `last_read_message_id` moves forward via `GREATEST`. - Mark-all-read — single UPDATE on `conversation_read_state`; we don't backfill receipts unless required.

Alternatives considered + rejected: - Bitmap per message indexed by participant — clever but membership churn makes it brittle.

What NOT to say: - "We'll insert a receipt row for every member when each message arrives." Insanely expensive; receipts are inserted on read, not on send.

Common follow-up: *"In a 1000-member group, how do you compute 'seen by 873' efficiently?"*

Scenario183 · Order Returns + Refunds + RMAs

Tags: Difficulty: Hard · Asked at: Flipkart, Myntra, Nykaa, Meesho · Domain: Commerce

The prompt:

"Design tables for return-merchandise authorisation — customer initiates a return, we issue an RMA, items get picked up, then refunded. Walk me through partial returns and the financial reconciliation."

Thinking out loud (what to verbalize): 1. *"First, the entities — return_requests (header), return_items (line-level), rma_pickups (logistics), refunds (financial)."* 2. *"Next, the flow — request → approved → pickup → inspected → refund issued; state transitions logged."* 3. *"For indexes — return_requests by order_id and by customer; refunds by status for the payouts job."* 4. *"Edge cases — partial returns (return 1 of 3 items), item rejected on inspection, refund failure, and exchange rather than refund."*

The schema:

```

CREATE TABLE return_requests (
    rma_id BIGSERIAL PRIMARY KEY,
    rma_number VARCHAR(30) UNIQUE NOT NULL,
    order_id BIGINT NOT NULL,
    customer_id BIGINT NOT NULL,
    reason_code VARCHAR(50) NOT NULL,
    reason_notes TEXT,
    status VARCHAR(20) NOT NULL DEFAULT 'requested',
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE return_items (
    return_item_id BIGSERIAL PRIMARY KEY,
    rma_id BIGINT NOT NULL REFERENCES return_requests(rma_id),
    order_item_id BIGINT NOT NULL,
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    item_status VARCHAR(20) NOT NULL DEFAULT 'requested',
    inspection_result VARCHAR(20),
    inspection_notes TEXT,
    UNIQUE (rma_id, order_item_id)
);

CREATE TABLE rma_pickups (
    pickup_id BIGSERIAL PRIMARY KEY,
    rma_id BIGINT NOT NULL REFERENCES return_requests(rma_id),
    courier_partner VARCHAR(80),
    tracking_number VARCHAR(80),
    scheduled_at TIMESTAMP,
    picked_up_at TIMESTAMP,
    received_at TIMESTAMP
);

CREATE TABLE refunds (
    refund_id BIGSERIAL PRIMARY KEY,
    rma_id BIGINT NOT NULL REFERENCES return_requests(rma_id),
    amount_paise BIGINT NOT NULL,
    refund_mode VARCHAR(30) NOT NULL,
    txn_reference VARCHAR(100),
    status VARCHAR(20) NOT NULL DEFAULT 'initiated',
    initiated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    settled_at TIMESTAMP
);

CREATE INDEX idx_rma_order ON return_requests (order_id);
CREATE INDEX idx_refunds_pending ON refunds (initiated_at)
    WHERE status = 'initiated';

```

Why each choice: - Per-item **quantity** so a customer returning 2 of 3 units of one SKU is one return_item with quantity = 2. - Inspection result on item, not RMA, because parts of the return can be rejected (damaged customer-side). - Refund decoupled from RMA — multiple refunds possible (initial then a top-up after inspection adjustment).

Edge cases handled: - Item rejected on inspection — `item_status = 'rejected'`, refund recomputed excluding that line. - Refund failure — `refunds.status = 'failed'`, dunning retries, escalation after threshold to manual review.

Alternatives considered + rejected: - Reusing the orders schema for returns (negative-quantity rows) — clever but confusing for reporting and fraud checks.

What NOT to say: - "We'll refund the full order if any item is returned." Refund must match the inspected-and-accepted line items, no more, no less.

Common follow-up: *"How would you support an exchange — customer returns a size M, gets a size L of the same product?"*

Scenario184 · Wallet / Loyalty Points with Balance

Tags: Difficulty: Medium · Asked at: Paytm, PhonePe, Cred, Flipkart · Domain: Wallet

The prompt:

"Design a loyalty-points wallet — users earn points on orders, redeem on future orders, points expire after one year. Walk me through balance computation and expiry."

Thinking out loud (what to verbalize): 1. *"First, the entities — wallets (per user), wallet_transactions (every credit and debit), and a denormalised current_balance cached on the wallet."* 2. *"Next, expiry — credits have an `expires_at`; balance is sum of unexpired credits minus debits."* 3. *"For indexes — transactions by (wallet_id, posted_at) for statements; by expires_at for the expiry job."* 4. *"Edge cases — partial redemption against multiple credit lots (FIFO), expiry mid-redemption, and balance drift."*

The schema:

```
CREATE TABLE wallets (  
    wallet_id BIGSERIAL PRIMARY KEY,  
    user_id BIGINT UNIQUE NOT NULL,  
    wallet_type VARCHAR(20) NOT NULL DEFAULT 'loyalty',  
    current_balance INTEGER NOT NULL DEFAULT 0,  
    last_recomputed_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE wallet_transactions (  
    txn_id BIGSERIAL PRIMARY KEY,  
    wallet_id BIGINT NOT NULL REFERENCES wallets(wallet_id),  
    txn_type VARCHAR(20) NOT NULL,  
    points INTEGER NOT NULL CHECK (points > 0),  
    reference_type VARCHAR(30),  
    reference_id BIGINT,  
    expires_at TIMESTAMP,  
    posted_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE wallet_redemption_lots (  
    redemption_id BIGINT NOT NULL REFERENCES wallet_transactions(txn_id),  
    credit_txn_id BIGINT NOT NULL REFERENCES wallet_transactions(txn_id),  
    points_consumed INTEGER NOT NULL CHECK (points_consumed > 0),  
    PRIMARY KEY (redemption_id, credit_txn_id)  
);  
  
CREATE INDEX idx_wt_wallet_time ON wallet_transactions (wallet_id, posted_at DESC);  
CREATE INDEX idx_wt_expiry ON wallet_transactions (expires_at)  
    WHERE txn_type = 'credit' AND expires_at IS NOT NULL;
```

Why each choice: - Redemption lots table records which credits a debit consumed (FIFO) — auditable and supports partial reversals. - `current_balance` is a cache; a nightly job recomputes from transactions to catch drift. - Credits and debits are positive numbers with a `txn_type` discriminator — easier to read than signed math.

Edge cases handled: - Expiry — scheduled job inserts compensating `expiry` debit for unredeemed remainder of each expired credit; balance decreases atomically. - FIFO redemption — application picks oldest unexpired credits first; redemption_lots table records the allocation.

Alternatives considered + rejected: - Single balance column with no transaction history — fast but uncountable and unauditable.

What NOT to say: - "We'll keep the balance correct via a trigger on transactions." Triggers are easy to drift; explicit recompute job is more honest.

Common follow-up: "A user redeemed 500 points, then we discover the earning order was fraud. How do you unwind?"

Scenario185 · KYC Document Store (Versioned Attachments)

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Slice, Jupiter · Domain: KYC / Compliance

The prompt:

"Design a KYC document store — user uploads an ID document, reviewer verifies, user may re-upload after rejection. Walk me through versioning and current-version queries."

Thinking out loud (what to verbalize): 1. "First, the entities — *kyc_profiles* per user, *kyc_documents* (versioned upload), *kyc_reviews* (decision rows)." 2. "Next, current version — *is_current* flag on documents, or always pick *max(version)* per (profile, doc_type)." 3. "For indexes — documents by (profile, doc_type, version) for current pick; reviews by status for the queue." 4. "Edge cases — re-upload while review pending, redacted documents for privacy, retention period."

The schema:

```

CREATE TABLE kyc_profiles (
  profile_id BIGSERIAL PRIMARY KEY,
  user_id BIGINT UNIQUE NOT NULL,
  overall_status VARCHAR(20) NOT NULL DEFAULT 'pending',
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE kyc_documents (
  document_id BIGSERIAL PRIMARY KEY,
  profile_id BIGINT NOT NULL REFERENCES kyc_profiles(profile_id),
  document_type VARCHAR(40) NOT NULL,
  version INTEGER NOT NULL,
  storage_uri VARCHAR(500) NOT NULL,
  file_hash CHAR(64) NOT NULL,
  mime_type VARCHAR(60),
  uploaded_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  is_current BOOLEAN NOT NULL DEFAULT TRUE,
  UNIQUE (profile_id, document_type, version)
);

CREATE TABLE kyc_reviews (
  review_id BIGSERIAL PRIMARY KEY,
  document_id BIGINT NOT NULL REFERENCES kyc_documents(document_id),
  reviewer_id BIGINT,
  decision VARCHAR(20) NOT NULL,
  rejection_reason VARCHAR(255),
  reviewed_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE UNIQUE INDEX uq_kyc_current ON kyc_documents (profile_id, document_type)
  WHERE is_current = TRUE;
CREATE INDEX idx_reviews_pending ON kyc_reviews (reviewed_at)
  WHERE decision = 'pending';

```

Why each choice: - Partial unique index `WHERE is_current = TRUE` enforces exactly one current document per (profile, doc_type). - File stored externally (object store), DB holds the URI plus hash for tamper detection. - Reviews are append-only; a re-upload creates a new document row + a new review row.

Edge cases handled: - Re-upload during pending review — the new version flips `is_current = FALSE` on old, `TRUE` on new, in one transaction. - Retention — after closure, document rows stay with metadata; the object-store URI is purged after compliance window.

Alternatives considered + rejected: - Storing the file blob inside the DB — works for tiny attachments; KYC scans are too large and bloat backups.

What NOT to say: - "We'll overwrite the old version on re-upload." Loses history and breaks the auditor's reconstruction.

Common follow-up: "A user invokes their right to be forgotten. What survives in your tables and what doesn't?"

Scenario186 · Address Book with Primary + Multiple Addresses

Tags: Difficulty: Easy · Asked at: Flipkart, Swiggy, Zomato, Urban Company · Domain: Commerce

The prompt:

"Design an address book for users — multiple addresses, one default for shipping, one default for billing. Walk me through how you enforce 'exactly one default'."

Thinking out loud (what to verbalize): 1. "First, the entities — addresses with a `user_id`, and per-user defaults for shipping and billing." 2. "Next, the constraint — exactly one address marked default per (user, type); enforce via partial unique index." 3. "For indexes — addresses by `user_id` for 'my addresses', partial unique for default invariant." 4. "Edge cases — last default deleted, address tied to past orders, validation against PIN-code service."

The schema:

```
CREATE TABLE addresses (  
  address_id BIGSERIAL PRIMARY KEY,  
  user_id BIGINT NOT NULL,  
  contact_name VARCHAR(150) NOT NULL,  
  contact_phone VARCHAR(15) NOT NULL,  
  line1 VARCHAR(255) NOT NULL,  
  line2 VARCHAR(255),  
  city VARCHAR(80) NOT NULL,  
  state VARCHAR(80) NOT NULL,  
  pin_code CHAR(6) NOT NULL,  
  country CHAR(2) NOT NULL DEFAULT 'IN',  
  address_type VARCHAR(20) NOT NULL DEFAULT 'home',  
  is_default_shipping BOOLEAN NOT NULL DEFAULT FALSE,  
  is_default_billing BOOLEAN NOT NULL DEFAULT FALSE,  
  deleted_at TIMESTAMP,  
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE INDEX idx_addresses_user ON addresses (user_id) WHERE deleted_at IS NULL;  
  
CREATE UNIQUE INDEX uq_default_shipping ON addresses (user_id)  
  WHERE is_default_shipping = TRUE AND deleted_at IS NULL;  
  
CREATE UNIQUE INDEX uq_default_billing ON addresses (user_id)  
  WHERE is_default_billing = TRUE AND deleted_at IS NULL;
```


Why each choice: - Partial unique indexes enforce "at most one default" without trigger logic. - Soft delete for addresses so historical orders can still reference the address row. - Phone and name stored on the address row — recipient may differ from account owner.

Edge cases handled: - Marking a new default — transaction: clear old default, set new default, partial unique catches any race. - Last default deleted — application picks an arbitrary remaining address as default, or leaves none and prompts on next order.

Alternatives considered + rejected: - Separate `user_default_addresses` table — extra join on every checkout; cheaper to keep flags on the address row.

What NOT to say: - "I'll let two defaults exist and pick whichever has the lower id at checkout." Silent ambiguity hides bugs.

Common follow-up: *"A user moves house; how do you make sure their old orders still ship to the old address but new orders use the new one?"*

Scenario187 · Versioned Configuration / Feature Flags

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, BrowserStack · Domain: Config

The prompt:

"Design tables for feature flags with environment scoping, percentage rollouts, and per-user overrides. Walk me through evaluation and rollback."

Thinking out loud (what to verbalize): 1. *"First, the entities — flags (the catalogue), flag_rules (env-scoped rollout rules), flag_overrides (per-user)."* 2. *"Next, evaluation — check overrides first; else evaluate rules for current env in priority order."* 3. *"For indexes — flag_rules by (flag_id, environment), overrides by (flag_id, user_id)."* 4. *"Edge cases — atomic rollback (versioning), audit of who changed what, dependency between flags."*

The schema:

```

CREATE TABLE feature_flags (
  flag_id SERIAL PRIMARY KEY,
  flag_key VARCHAR(80) UNIQUE NOT NULL,
  description TEXT,
  flag_type VARCHAR(20) NOT NULL DEFAULT 'boolean',
  default_value JSONB NOT NULL,
  is_archived BOOLEAN NOT NULL DEFAULT FALSE
);

CREATE TABLE feature_flag_versions (
  version_id BIGSERIAL PRIMARY KEY,
  flag_id INTEGER NOT NULL REFERENCES feature_flags(flag_id),
  environment VARCHAR(20) NOT NULL,
  rules JSONB NOT NULL,
  rollout_percent NUMERIC(5,2) NOT NULL DEFAULT 0,
  is_current BOOLEAN NOT NULL DEFAULT TRUE,
  changed_by_user_id BIGINT,
  change_notes TEXT,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE feature_flag_overrides (
  flag_id INTEGER NOT NULL REFERENCES feature_flags(flag_id),
  environment VARCHAR(20) NOT NULL,
  user_id BIGINT NOT NULL,
  override_value JSONB NOT NULL,
  set_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  expires_at TIMESTAMP,
  PRIMARY KEY (flag_id, environment, user_id)
);

CREATE UNIQUE INDEX uq_flag_version_current
  ON feature_flag_versions (flag_id, environment)
  WHERE is_current = TRUE;

```

Why each choice: - New rule edit = new version row + flip `is_current`; rollback = flip current back to a prior version, one statement. - Rules in JSONB so adding new targeting types (country, device, app version) doesn't require schema changes. - Per-user overrides have TTL so QA toggles don't leak forever.

Edge cases handled: - Audit — every version has `changed_by_user_id` and `change_notes`. - Concurrent edits — both insert new versions; the partial unique on `is_current` makes one win.

Alternatives considered + rejected: - Storing all flags as a single JSON file in a single row — easy but no history, no scoping, no rollback granularity.

What NOT to say: - "Edits go straight to production." Always require staged rollout via the version row.

Common follow-up: "How does the SDK fetch flags without hammering the DB every request?"

Scenario188 · Webhook Delivery + Retry

Tags: Difficulty: Medium · Asked at: Razorpay, Cashfree, Cred, Postman · Domain: Webhooks

The prompt:

"Design tables for outgoing webhook delivery — register endpoints, dispatch events, retry on failure with exponential backoff. Walk me through the schema and dispatcher loop."

Thinking out loud (what to verbalize): 1. "First, the entities — *webhook_endpoints* (subscriber config), *webhook_events* (the event to deliver), *webhook_deliveries* (each attempt)." 2. "Next, the dispatcher — picks events with *next_attempt_at* $\leq$ now, posts HTTP, records response, computes next attempt." 3. "For indexes — partial index on events pending dispatch, on deliveries by event for audit." 4. "Edge cases — endpoint disabled mid-retry, large payloads, signing, and dead-letter after N attempts."

The schema:

```

CREATE TABLE webhook_endpoints (
    endpoint_id BIGSERIAL PRIMARY KEY,
    merchant_id BIGINT NOT NULL,
    target_url VARCHAR(500) NOT NULL,
    signing_secret_hash CHAR(64) NOT NULL,
    subscribed_events TEXT[] NOT NULL,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE webhook_events (
    event_id BIGSERIAL PRIMARY KEY,
    endpoint_id BIGINT NOT NULL REFERENCES webhook_endpoints(endpoint_id),
    event_type VARCHAR(80) NOT NULL,
    payload JSONB NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'pending',
    attempts INTEGER NOT NULL DEFAULT 0,
    next_attempt_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    last_response_code INTEGER,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    delivered_at TIMESTAMP,
    dead_lettered_at TIMESTAMP
);

CREATE TABLE webhook_deliveries (
    delivery_id BIGSERIAL PRIMARY KEY,
    event_id BIGINT NOT NULL REFERENCES webhook_events(event_id),
    attempted_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    response_code INTEGER,
    response_body TEXT,
    latency_ms INTEGER,
    error_message TEXT
);

CREATE INDEX idx_webhook_due ON webhook_events (next_attempt_at)
    WHERE status = 'pending';
CREATE INDEX idx_webhook_deliveries_event ON webhook_deliveries (event_id, attempted_at);

```

Why each choice: - Each attempt is a row in `webhook_deliveries` — full forensic audit of what the subscriber saw. - `next_attempt_at` indexed partial — dispatcher scan stays cheap even with millions of historical events. - Dead-letter via `dead_lettered_at` rather than deletion — investigation later is possible.

Edge cases handled: - Endpoint disabled mid-retry — dispatcher checks `endpoints.is_active` before each POST. - Backoff — `next_attempt_at = NOW() + interval based on attempts`; capped at, say, 24 hours; max 16 attempts then dead-letter.

Alternatives considered + rejected: - Pushing into a message queue and letting the queue handle retries — works, but the audit trail in queues is weaker than in a DB table.

What NOT to say: - "We'll retry forever until the subscriber accepts." Bug magnet; bound attempts and dead-letter.

Common follow-up: "A subscriber's endpoint is down for 6 hours. When it comes back, how do you avoid a thundering herd?"

Scenario189 · API Key Management with Rotation

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, BrowserStack · Domain: API security

The prompt:

"Design tables for API key management — merchants can have multiple keys with scopes, and they can rotate keys with a grace period. Walk me through how an incoming request authenticates."

Thinking out loud (what to verbalize): 1. "First, the entities — *api_keys* (per merchant, hashed), *api_key_scopes* (permissions on the key), and an audit of every key event." 2. "Next, lookup — we look up by *key_id_prefix* (first 8 chars stored cleartext) and then verify hash." 3. "For indexes — by *key_id_prefix* for the auth hot path; by merchant for the dashboard." 4. "Edge cases — leaked key revocation, IP allowlist per key, last-used tracking without write-amplification."

The schema:

```

CREATE TABLE api_keys (
  api_key_id BIGSERIAL PRIMARY KEY,
  merchant_id BIGINT NOT NULL,
  key_prefix CHAR(8) NOT NULL,
  key_hash CHAR(64) NOT NULL,
  label VARCHAR(80),
  status VARCHAR(20) NOT NULL DEFAULT 'active',
  last_used_at TIMESTAMP,
  expires_at TIMESTAMP,
  revoked_at TIMESTAMP,
  rotated_from_key_id BIGINT REFERENCES api_keys(api_key_id),
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE (key_prefix)
);

CREATE TABLE api_key_scopes (
  api_key_id BIGINT NOT NULL REFERENCES api_keys(api_key_id),
  scope_code VARCHAR(60) NOT NULL,
  PRIMARY KEY (api_key_id, scope_code)
);

CREATE TABLE api_key_ip_allowlist (
  api_key_id BIGINT NOT NULL REFERENCES api_keys(api_key_id),
  cidr_block CIDR NOT NULL,
  PRIMARY KEY (api_key_id, cidr_block)
);

CREATE TABLE api_key_events (
  event_id BIGSERIAL PRIMARY KEY,
  api_key_id BIGINT NOT NULL REFERENCES api_keys(api_key_id),
  event_type VARCHAR(30) NOT NULL,
  actor_user_id BIGINT,
  event_meta JSONB,
  occurred_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_keys_merchant ON api_keys (merchant_id) WHERE status = 'active';
CREATE INDEX idx_key_events_key ON api_key_events (api_key_id, occurred_at DESC);

```

Why each choice: - `key_prefix` is the lookup token; `key_hash` is constant-time-compared after — never store the raw key. - `rotated_from_key_id` lets the dashboard show "this key replaced that one" and supports a grace period where both work. - `last_used_at` updated in-memory and flushed periodically — direct UPDATE per request would hammer the row.

Edge cases handled: - Leaked key — single `UPDATE api_keys SET revoked_at = NOW(), status = 'revoked'` ; auth check fails immediately. - IP allowlist — auth path filters by `cidr_block` containing request IP; absence of rows means no restriction.

Alternatives considered + rejected: - Storing keys reversibly encrypted — rejected; one-way hash is non-negotiable.

What NOT to say: - "We'll log the actual key value when it's used so admins can investigate."
Catastrophic security mistake.

Common follow-up: *"How would you implement per-endpoint rate limiting tied to a specific API key?"*

Scenario190 · Rate Limit Counters

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Postman, BrowserStack · Domain: Throttling

The prompt:

"Design tables for API rate limiting — per API key, per endpoint, per time window. You can use Redis but pretend you must do it in the DB. Walk me through fixed-window vs sliding-window counters."

Thinking out loud (what to verbalize): 1. *"First, two algorithms — fixed window (counter per minute, simple, with edge-burst risk) and sliding window (more accurate, more storage)."* 2. *"Next, the entities — rate_limit_policies (rules), rate_limit_counters (the bucket state)."* 3. *"For indexes — counters keyed by (api_key, endpoint, window_start) with TTL via partial scan; policies looked up by (api_key, endpoint)."* 4. *"Edge cases — clock skew across servers, distributed counters, burst allowance, and counter reset."*

The schema:

```

CREATE TABLE rate_limit_policies (
    policy_id SERIAL PRIMARY KEY,
    api_key_id BIGINT NOT NULL,
    endpoint_pattern VARCHAR(200) NOT NULL,
    limit_count INTEGER NOT NULL,
    window_seconds INTEGER NOT NULL,
    burst_allowance INTEGER NOT NULL DEFAULT 0,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    UNIQUE (api_key_id, endpoint_pattern)
);

CREATE TABLE rate_limit_counters (
    api_key_id BIGINT NOT NULL,
    endpoint_pattern VARCHAR(200) NOT NULL,
    window_start TIMESTAMP NOT NULL,
    count INTEGER NOT NULL DEFAULT 0,
    last_request_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (api_key_id, endpoint_pattern, window_start)
);

CREATE INDEX idx_rl_counters_cleanup ON rate_limit_counters (window_start);

CREATE TABLE rate_limit_violations (
    violation_id BIGSERIAL PRIMARY KEY,
    api_key_id BIGINT NOT NULL,
    endpoint_pattern VARCHAR(200) NOT NULL,
    request_count INTEGER NOT NULL,
    occurred_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

```

Why each choice: - Fixed window keyed by truncated `window_start ( date_trunc('minute', NOW() ) )` — single UPSERT per request. - UPSERT with `count = count + 1 RETURNING count` checks the limit and increments in one round-trip. - Old window rows cleaned by a periodic DELETE; not held forever.

Edge cases handled: - Edge burst (twice the limit straddling the boundary) — to tame this, use a sliding-window of two adjacent fixed buckets, weight by overlap. - Distributed counters — DB row is the single source; for very high QPS, move to a per-shard counter and aggregate, or use Redis with the DB as fallback.

Alternatives considered + rejected: - Per-request audit row, counting on read — works at small scale; at high QPS the count query becomes a bottleneck.

What NOT to say: - "We'll just block when the count exceeds the limit by writing a log row." Logs aren't enforcement; the counter row is.

Common follow-up: "At 50000 requests per second across keys, the `rate_limit_counters` table is the hottest table in your DB. How do you scale it?"

Category 3, Part B — Design This Schema: Analytics & Warehousing Patterns

Part B covers the analytics half of the design round — star schema, slowly-changing dimensions, event tables, time-series, columnar warehouses, attribution models, and the layered modelling pattern (staging → intermediate → mart) that every modern data team uses. These show up heavily at product companies (Razorpay, Swiggy, Cred, Meesho, Flipkart, PhonePe), at data-role GCC panels (Walmart Labs, Target, Wells Fargo, Goldman, AmEx) and at every Snowflake / BigQuery / dbt-shaped opening. The interviewer is testing whether you can think in facts and dimensions, reason about late-arriving data, and pick the right grain — not whether you can recite Kimball verbatim. Each scenario below gives the prompt, the thinking, the schema sketch, the choices defended, the edge cases, the alternatives, the things to avoid saying, and the follow-up that usually lands.

Scenario191 · Star schema for sales

Tags: Difficulty: Medium · Asked at: Walmart Labs, Target, Flipkart, Meesho · Topic: Dimensional modelling

The interviewer's prompt: *"Design a star schema for an e-commerce sales reporting workload. Daily revenue by product, by city, by category — that kind of slicing."*

Thinking out loud: 1. *"First clarification — what's the grain of the fact? Per order line, or per order? I'll go per order line because that's the lowest grain we'll need to roll up from."* 2. *"Then dimensions: date, product, customer, store/channel, payment method. Keep dim keys as surrogate integers, never natural keys."* 3. *"Measures live in the fact — quantity, unit price, discount, tax, line total. Anything additive goes in the fact; anything descriptive goes in a dim."*

Schema sketch:

```

CREATE TABLE dim_date (
  date_key      INT PRIMARY KEY,          -- YYYYMMDD
  date_actual   DATE NOT NULL,
  day_of_week   SMALLINT, month SMALLINT, quarter SMALLINT, year SMALLINT,
  is_weekend    BOOLEAN, is_holiday BOOLEAN
);
CREATE TABLE dim_product (
  product_key   BIGSERIAL PRIMARY KEY,
  product_id    VARCHAR(50) NOT NULL,      -- natural key from source
  product_name  VARCHAR(255), category VARCHAR(100), brand VARCHAR(100)
);
CREATE TABLE dim_customer (
  customer_key  BIGSERIAL PRIMARY KEY,
  customer_id   VARCHAR(50), city VARCHAR(100), state VARCHAR(100), tier VARCHAR(20)
);
CREATE TABLE fact_sales (
  sale_key      BIGSERIAL PRIMARY KEY,
  date_key      INT REFERENCES dim_date(date_key),
  product_key   BIGINT REFERENCES dim_product(product_key),
  customer_key  BIGINT REFERENCES dim_customer(customer_key),
  order_id      VARCHAR(50),              -- degenerate dimension
  quantity      INT,
  unit_price    NUMERIC(12,2),
  discount_amt  NUMERIC(12,2),
  line_total    NUMERIC(12,2)
);
CREATE INDEX ON fact_sales(date_key);
CREATE INDEX ON fact_sales(product_key, date_key);

```

Why each choice: - Surrogate `date_key` as `YYYYMMDD INT` because BI tools and partition pruning both love it; cheaper to join than a date string. - Dims hold the natural key (`product_id`) so we can re-link if source IDs change. - Order ID stays in the fact as a degenerate dim — we will need it to count distinct orders, but it has no other attributes worth a table. - Measures are pre-computed at line grain so any roll-up is a `SUM` — no row-level math at query time.

Edge cases: - Refunds and cancellations — add a `transaction_type` or store negative line totals so net revenue stays a simple `SUM`. - Currency — if multi-country, store a `currency_code` and a `local_amount`, plus a converted `inr_amount` against `dim_date` FX rates. - Returns arriving days later — these are late-arriving facts (see Scenario20).

Alternatives: - Snowflake schema (next scenario) — normalised dims save space at the cost of more joins. - OBT — one big table denormalised, easier to query but storage-heavy.

What NOT to say: - "Just use the OLTP `orders` table for reporting" — kills production load, mixes write and read patterns, no historical preservation. - "Skip the date dimension, use the timestamp directly" — you lose easy holiday/weekend flags and partition keys.

Common follow-up: "How would you handle a product changing category — do you overwrite, version, or both?" (Leads into SCD discussion.)

Scenario192 · Snowflake schema — when normalised dims help

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan India, Wells Fargo · Topic: Dimensional modelling

The interviewer's prompt: "You designed a star schema. When would you normalise a dimension into a snowflake? Show me where."

Thinking out loud: 1. "Snowflaking means splitting a dim into multiple normalised tables — `dim_product` → `dim_product` + `dim_brand` + `dim_category` ." 2. "Worth it when a dim has very wide attribute branches, or when the same sub-dim is reused across many parent dims." 3. "Cost: more joins per query, harder for BI tools. Win: fewer updates if a brand attribute changes, smaller storage on huge dims."

Schema sketch:

```
CREATE TABLE dim_category (
  category_key INT PRIMARY KEY,
  category_name VARCHAR(100), parent_category VARCHAR(100)
);
CREATE TABLE dim_brand (
  brand_key INT PRIMARY KEY,
  brand_name VARCHAR(100), brand_owner_country VARCHAR(50)
);
CREATE TABLE dim_product (
  product_key BIGINT PRIMARY KEY,
  product_id VARCHAR(50),
  product_name VARCHAR(255),
  category_key INT REFERENCES dim_category(category_key),
  brand_key INT REFERENCES dim_brand(brand_key)
);
```

Why each choice: - Category and brand are reused across `dim_product` and potentially `dim_supplier` — normalising avoids duplicating brand details on every product row. - Brand attributes (owner country, holding company) update independently of products. One row to update in `dim_brand` versus thousands in `dim_product` . - Category hierarchy is recursive and stored once.

Edge cases: - Modern columnar warehouses (Snowflake, BigQuery, Redshift) compress repeated strings extremely well, so the storage saving is often tiny. Snowflaking can hurt query perf there. - BI tools that auto-generate joins (Tableau, Looker) may not pick up snowflake hops cleanly.

Alternatives: - Keep star schema; let column compression handle redundancy. - Hybrid — snowflake the genuinely shared sub-dims (geography, time-of-day) but star-flatten the rest.

What NOT to say: - "Always snowflake to follow third normal form" — modern analytics warehouses do not reward 3NF; they reward join elimination. - "Snowflake is always faster because tables are smaller" — opposite; extra joins are usually the bottleneck.

Common follow-up: "Same question, but the warehouse is BigQuery — does your answer change?"
(Yes — flatter is better on columnar; consider nested STRUCT for the sub-dim attributes.)

Scenario193 · SCD Type 1 — overwrite in place

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Wipro · Topic: Slowly changing dimensions

The interviewer's prompt: "A customer changes their phone number. The business does not care about the old number. How do you store this in the dimension?"

Thinking out loud: 1. "This is the classic SCD Type 1 — overwrite. We lose history, but we save storage and queries stay simple." 2. "Use Type 1 when the attribute is corrective (typo fix, phone update) or business-irrelevant for historical reporting." 3. "I will still record an `updated_at` so debugging is possible, even though we do not version the row."

Schema sketch:

```
CREATE TABLE dim_customer (  
  customer_key BIGSERIAL PRIMARY KEY,  
  customer_id  VARCHAR(50) UNIQUE NOT NULL,  
  full_name    VARCHAR(200),  
  email        VARCHAR(255),  
  phone_number VARCHAR(20),      -- Type 1 column — overwrite  
  city         VARCHAR(100),     -- Type 1 column — overwrite  
  signup_date  DATE,  
  updated_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
-- ETL pattern  
UPDATE dim_customer  
SET phone_number = :new_phone, updated_at = CURRENT_TIMESTAMP  
WHERE customer_id = :cust_id  
  AND phone_number IS DISTINCT FROM :new_phone;
```

Why each choice: - One row per customer — joins stay cheap, fact tables do not multiply. - `IS DISTINCT FROM` avoids no-op updates (matters for change-data-capture pipelines). - `updated_at` is a forensic field, not a historical one — you cannot answer "what was the phone number on 2024-03-15" with Type 1.

Edge cases: - If downstream models cache the old value (BI extracts, ML features) you need to invalidate them on overwrite. - Auditing — Type 1 leaves no audit trail; pair with a CDC log if compliance demands it.

Alternatives: - Type 2 (next scenario) if any future report might need history. - Type 3 if you only ever need the most recent previous value.

What NOT to say: - "Type 1 means just `INSERT` over the old row" — `UPDATE`, not `INSERT`. -
 "Type 1 is always wrong because we lose data" — for corrective fields it is the right answer; do not over-engineer.

Common follow-up: "Marketing now wants to know which city the customer was in when they placed a particular order — does your schema still work?" (No, because we overwrote city. Pivot to Type 2.)

Scenario194 · SCD Type 2 — full history rows

Tags: Difficulty: Medium · Asked at: Walmart Labs, Target, Wells Fargo, AmEx GCC · Topic: Slowly changing dimensions

The interviewer's prompt: "Same customer table, but now any change must be queryable historically. Design the schema and the ETL pattern."

Thinking out loud: 1. "Type 2 keeps history by versioning rows — every change inserts a new row, the old row is closed out." 2. "Need `valid_from`, `valid_to`, `is_current`. Surrogate key per version, natural key shared across versions." 3. "Facts must join on the surrogate key, not the natural key, so each fact row picks up the dim version that was active when the fact happened."

Schema sketch:

```
CREATE TABLE dim_customer (
  customer_key BIGSERIAL PRIMARY KEY,      -- changes with every version
  customer_id  VARCHAR(50) NOT NULL,       -- stable natural key
  full_name    VARCHAR(200),
  city         VARCHAR(100),
  tier          VARCHAR(20),
  valid_from   TIMESTAMP NOT NULL,
  valid_to     TIMESTAMP,                  -- NULL when current
  is_current   BOOLEAN NOT NULL DEFAULT TRUE,
  UNIQUE (customer_id, valid_from)
);
CREATE INDEX ON dim_customer (customer_id, is_current) WHERE is_current;
```

Why each choice: - Surrogate `customer_key` lets each fact row pin to the exact dim version at the time of the event. - `valid_to` NULL on the current row is the SQL idiom most warehouses recognise; pair with `is_current` for fast lookup. - Partial index on `is_current` keeps the "give me the live row" query cheap even on millions of versions.

Edge cases: - Concurrent updates — wrap the close-out and insert in a transaction so a customer never has two current rows. - Same-day multiple changes — decide whether you collapse them into one version (last-write-wins per day) or version each. - Storage explosion — bound by change frequency; for very hot dims, mix Type 1 (overwrite) for cheap fields with Type 2 (version) for important ones.

Alternatives: - Type 6 (Scenario6) — combine Type 2 history with a Type 1 "current value" column for convenience. - Bitemporal modelling — track `effective` and `recorded` timestamps separately.

What NOT to say: - "Just store the timestamp on the fact and look up the dim with `BETWEEN`" — works but kills join performance at scale. - "Use the natural key as the join column" — defeats the entire point of versioning.

Common follow-up: "How would you bulk-load a year of historical changes into this table from a system that never tracked them?" (Reconstruct from CDC log, or accept a single Type 2 row per year.)

Scenario195 · SCD Type 3 — limited-history columns

Tags: Difficulty: Easy · Asked at: Infosys, Cognizant, Capgemini · Topic: Slowly changing dimensions

The interviewer's prompt: "We only ever need to compare the customer's current tier with their previous one. Full history is overkill. Design for this."

Thinking out loud: 1. "SCD Type 3 — keep the current value and one previous value in the same row." 2. "One row per entity, two columns per tracked attribute: `current_value`, `previous_value`, `previous_changed_at`." 3. "Useful when business explicitly says 'I just want before/after' — for example, plan upgrades, tier changes."

Schema sketch:

```
CREATE TABLE dim_customer (  
  customer_id          VARCHAR(50) PRIMARY KEY,  
  full_name            VARCHAR(200),  
  city                 VARCHAR(100),  
  
  current_tier          VARCHAR(20),  
  previous_tier         VARCHAR(20),  
  tier_changed_at       TIMESTAMP,  
  
  updated_at           TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
-- ETL pattern when tier changes  
UPDATE dim_customer  
SET previous_tier = current_tier,  
    current_tier = :new_tier,  
    tier_changed_at = CURRENT_TIMESTAMP,  
    updated_at = CURRENT_TIMESTAMP  
WHERE customer_id = :cust_id  
    AND current_tier IS DISTINCT FROM :new_tier;
```

Why each choice: - One row per customer — joins remain trivial. - Only one prior value preserved — second-previous is lost forever, which is acceptable given the brief. - `tier_changed_at` lets reports filter by "tier changes this month".

Edge cases: - If two attributes (tier and city) both follow Type 3, you double the columns — pair with `..._changed_at` for each. - Reverting to the previous tier is silently lossy if you do not detect it before the `UPDATE`.

Alternatives: - Type 2 if depth of history matters. - Type 4 — mini-dimensions in a separate table for fast-changing attributes.

What NOT to say: - "Type 3 stores all history in one row" — it stores exactly one previous value. - "Type 3 is older and not used today" — still used heavily when business cares only about deltas.

Common follow-up: *"What happens if a customer changes tier four times in a week and the report runs at the end of the week?"* (Only the last change is recoverable; pivot to Type 2 if all four matter.)

Scenario196 · SCD Type 6 — hybrid 1+2+3

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan India, Target, Walmart Labs · Topic: Slowly changing dimensions

The interviewer's prompt: *"You want full Type 2 history plus easy current-value lookups without joining the latest row every time. How would you model it?"*

Thinking out loud: 1. "Type 6 = Type 1 (overwrite current value) + Type 2 (insert new version) + Type 3 (carry current alongside historical), all in one table." 2. "Each row has both `tier` (the value as of that version) and `current_tier` (overwritten on every change, same across all versions of the same customer)." 3. "Facts can either join on the version-specific column or the current column depending on the question."

Schema sketch:

```

CREATE TABLE dim_customer (
  customer_key      BIGSERIAL PRIMARY KEY,
  customer_id       VARCHAR(50) NOT NULL,
  tier               VARCHAR(20),           -- Type 2: as-of-version value
  current_tier       VARCHAR(20),           -- Type 1: overwritten on all versions
  city              VARCHAR(100),
  current_city       VARCHAR(100),
  valid_from         TIMESTAMP NOT NULL,
  valid_to           TIMESTAMP,
  is_current         BOOLEAN NOT NULL DEFAULT TRUE
);

-- ETL: when tier changes
BEGIN;
UPDATE dim_customer SET valid_to = CURRENT_TIMESTAMP, is_current = FALSE
  WHERE customer_id = :cust_id AND is_current;
INSERT INTO dim_customer (customer_id, tier, current_tier, valid_from, is_current)
  VALUES (:cust_id, :new_tier, :new_tier, CURRENT_TIMESTAMP, TRUE);
UPDATE dim_customer SET current_tier = :new_tier WHERE customer_id = :cust_id;
COMMIT;

```

Why each choice: - `tier` answers "what was their tier on 12 March" — Type 2 behaviour. - `current_tier` answers "what is their tier today" without joining the latest row — Type 1 behaviour, repeated across all versions. - Single table — no maintenance overhead of a separate "current" view.

Edge cases: - The bulk `UPDATE` of `current_tier` across all versions can be expensive on hot dims; alternative is a materialised view that joins to a `is_current` row. - The redundancy means writes touch more rows; queries gain from it.

Alternatives: - Type 2 only, plus a `dim_customer_current` view filtered to `is_current`. - Pure Type 2 with downstream models exposing both perspectives.

What NOT to say: - "Type 6 is just Type 2 with extra steps" — the convenience of current-value access without a self-join is the whole point at scale. - "Storage is the same as Type 2" — no, it costs more on writes; you trade write cost for read simplicity.

Common follow-up: "In your fact table, which column would you join to — `tier` or `current_tier` — and why?" (Depends on the report; "revenue by tier" historically means `tier`; "current MRR by tier" means `current_tier`.)

Scenario197 · Event tracking table — clickstream

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Meesho, PhonePe, Cred · Topic: Event modelling

The interviewer's prompt: "Design a table that captures every front-end event from our app — page views, button clicks, form submits — at scale of 100 million events a day."

Thinking out loud: 1. "This is a fact-like table but with high cardinality and schemaless properties. Wide rows are common." 2. "Core columns: `event_id`, `event_name`, `user_id`, `session_id`, `timestamp`, plus a JSON `properties` blob for variable payload." 3. "On a warehouse like Snowflake or BigQuery I would partition by event date and cluster by `event_name`, `user_id`."

Schema sketch:

```
CREATE TABLE events (
  event_id      UUID,
  event_name    VARCHAR(100) NOT NULL,      -- "page_view", "add_to_cart"
  event_ts      TIMESTAMP NOT NULL,
  event_date    DATE NOT NULL,              -- partition key
  user_id       VARCHAR(50),
  anonymous_id  VARCHAR(50),               -- pre-login tracker
  session_id    VARCHAR(50),
  device_type   VARCHAR(20),
  app_version   VARCHAR(20),
  properties    JSONB,                    -- variable payload
  context       JSONB                     -- IP, UA, referrer,
  PRIMARY KEY (event_id, event_date)
)
PARTITION BY RANGE (event_date);
-- Snowflake: CLUSTER BY (event_date, event_name, user_id)
-- BigQuery: PARTITION BY event_date CLUSTER BY event_name, user_id
```

Why each choice: - `event_date` partition prunes 99% of scans for the typical "last 7 days" query. - `properties` JSONB absorbs event-specific fields without a schema migration on every new event. - Both `user_id` and `anonymous_id` cover pre-login and post-login funnels.

Edge cases: - Schema drift in `properties` — invest in a contract / event registry early or analytics queries become a graveyard. - PII inside `properties` — strip or hash on ingestion; do not store raw phone or email here. - Out-of-order events from mobile clients with offline mode — server-receive time vs client-emit time matter; keep both.

Alternatives: - Per-event-type table (one table per `event_name`) — fewer JSON unpacks but a schema explosion. - Columnar format on a data lake (Parquet on S3) queried via Trino/Athena.

What NOT to say: - "Just dump it all in one massive table and add indexes" — indexes do not help columnar warehouses; partitioning and clustering do. - "Use TEXT instead of JSONB" — JSONB allows operator-level extraction (`properties->'item_id'`) without parsing every row.

Common follow-up: "How would you make funnel queries — `view` → `add-to-cart` → `purchase` — fast on this schema?" (Add `session_id` and order by `event_ts` ; consider a pre-aggregated funnel mart — see Scenario9.)

Scenario198 · Sessionisation-ready event schema

Tags: Difficulty: Medium · Asked at: Flipkart, Meesho, Swiggy, Walmart Labs · Topic: Sessionisation

The interviewer's prompt: "Your events have a `user_id` and a `timestamp`. Reporting wants 'sessions' — groups of activity separated by 30 minutes of inactivity. Design the schema so downstream queries are easy."

Thinking out loud: 1. "Two layers — raw events with no session, and a derived `session_id` assigned by a batch or stream job." 2. "Session boundary: gap of > 30 minutes between consecutive events for the same user." 3. "Store `session_id` back on the raw event row and also a `sessions` table for session-level aggregates."

Schema sketch:

```
CREATE TABLE events_with_session (
  event_id      UUID,
  user_id       VARCHAR(50) NOT NULL,
  event_ts      TIMESTAMP NOT NULL,
  event_date    DATE NOT NULL,
  event_name    VARCHAR(100),
  session_id    UUID NOT NULL,           -- assigned by sessioniser
  session_seq   INT NOT NULL,           -- 1-indexed within session
  properties    JSONB
)
PARTITION BY RANGE (event_date);

CREATE TABLE sessions (
  session_id    UUID PRIMARY KEY,
  user_id       VARCHAR(50) NOT NULL,
  session_start TIMESTAMP NOT NULL,
  session_end   TIMESTAMP NOT NULL,
  event_count   INT,
  page_view_count INT,
  device_type   VARCHAR(20),
  utm_source    VARCHAR(100),
  utm_campaign  VARCHAR(100)
);

-- Sessioniser query pattern (window function)
WITH gaps AS (
  SELECT *,
    LAG(event_ts) OVER (PARTITION BY user_id ORDER BY event_ts) AS prev_ts,
    CASE WHEN event_ts - LAG(event_ts) OVER (PARTITION BY user_id ORDER BY event_ts)
      > INTERVAL '30 minutes' THEN 1 ELSE 0 END AS is_new_session
  FROM events_raw
)
SELECT *, SUM(is_new_session) OVER (PARTITION BY user_id ORDER BY event_ts) AS session_num
FROM gaps;
```

Why each choice: - `session_id` on the event row eliminates expensive sessionisation at query time.
- `session_seq` lets you ask "first event in session" without another window function. - Separate `sessions` table pre-aggregates the per-session metrics most dashboards need.

Edge cases: - Anonymous → logged-in stitching mid-session — keep an `anonymous_id` column and stitch in the sessioniser. - Late-arriving events from mobile offline buffers can re-open closed sessions; decide a cut-off (e.g. ignore events older than 24h for sessionisation). - Cross-device sessions — typically not modelled at the session level; handled at the journey level.

Alternatives: - Compute sessions on the fly in every query — works for small data, dies at scale. - Streaming sessioniser (Flink / Kafka Streams) for near-real-time session tables.

What NOT to say: - "Just use a different table per session" — sessions are entities, not partitions. - "Use IP address as session_id" — fails with NAT, mobile networks, IP rotation.

Common follow-up: *"What changes if the inactivity threshold becomes configurable per product line?"*
(Move it to a config table; sessioniser reads at run time.)

Scenario199 · Funnel analysis schema

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Meesho, Flipkart · Topic: Funnel modelling

The interviewer's prompt: *"Design a schema that lets PMs build any funnel — view → add-to-cart → checkout → payment — without rewriting the SQL each time."*

Thinking out loud: 1. *"Two components — a generic event stream (Scenario7) plus a funnel-config table that defines step order and event matching."* 2. *"Either compute funnels on demand from events using window functions, or pre-compute a `funnel_progressions` mart for the top 5 funnels."* 3. *"Property-based matching matters — 'add_to_cart' is one event, but the PM may want to funnel only Electronics adds."*

Schema sketch:

```
CREATE TABLE funnel_definitions (
  funnel_id      UUID PRIMARY KEY,
  funnel_name    VARCHAR(100) NOT NULL,
  step_seq      INT NOT NULL,
  event_name     VARCHAR(100) NOT NULL,
  match_filter   JSONB,           -- e.g. {"category":"Electronics"}
  step_window_min INT,           -- max minutes from previous step
  UNIQUE (funnel_id, step_seq)
);

CREATE TABLE funnel_progressions (
  user_id        VARCHAR(50),
  funnel_id      UUID,
  entered_at     TIMESTAMP,
  step_1_at      TIMESTAMP,
  step_2_at      TIMESTAMP,
  step_3_at      TIMESTAMP,
  step_4_at      TIMESTAMP,
  completed      BOOLEAN,
  dropped_at_step INT,
  PRIMARY KEY (user_id, funnel_id, entered_at)
);
```

Why each choice: - A definitions table decouples PM-driven funnel changes from engineering changes.
 - A pre-aggregated `funnel_progressions` table answers "conversion by step" in milliseconds; nightly job rebuilds it. - `match_filter` as JSONB lets PMs add filters without schema changes.

Edge cases: - Window between steps — strict (must be in same session) vs loose (within 7 days). Both are valid; surface as config. - Cohort skew — funnel for users acquired in March vs April will differ; mart should include a `cohort_date` slice. - Repeat funnels — a user can enter the same funnel twice; the PK reflects this.

Alternatives: - Compute every funnel ad hoc with `MATCH_RECOGNIZE` (Snowflake) or row-pattern recognition. - Use a third-party tool (Mixpanel-style) and skip the mart.

What NOT to say: - "Hard-code one funnel table per funnel" — defeats configurability. - "Use a single event sequence column with comma-separated values" — analytics death; lose all filterability.

Common follow-up: "How would you answer 'what is the median time from step 2 to step 3'?"
 (`PERCENTILE_CONT(0.5)` on `step_3_at - step_2_at` for completed users.)

Scenario200 · A/B test exposure table

Tags: Difficulty: Medium · Asked at: Flipkart, Meesho, Cred, Swiggy · Topic: Experimentation

The interviewer's prompt: "Design a table that records which users saw which A/B test variant. We will join this to metrics later."

Thinking out loud: 1. "Exposures are immutable — once a user is assigned to a variant, that assignment is locked for the duration of the test." 2. "Need `experiment_id`, `variant_id`, `user_id`, `first-exposure timestamp`; PK should prevent multiple variants per user per experiment." 3. "Hash-based assignment so the same user always gets the same variant on retries."

Schema sketch:

```
CREATE TABLE experiments (
  experiment_id      UUID PRIMARY KEY,
  experiment_name    VARCHAR(200) NOT NULL,
  started_at        TIMESTAMP NOT NULL,
  ended_at          TIMESTAMP,
  hypothesis         TEXT,
  primary_metric     VARCHAR(100)
);

CREATE TABLE experiment_variants (
  experiment_id      UUID REFERENCES experiments(experiment_id),
  variant_id         VARCHAR(50),
  variant_name       VARCHAR(100),
  allocation_pct     NUMERIC(5,2),
  PRIMARY KEY (experiment_id, variant_id)
);

CREATE TABLE experiment_exposures (
  exposure_id        UUID PRIMARY KEY,
  experiment_id       UUID NOT NULL,
  variant_id         VARCHAR(50) NOT NULL,
  user_id            VARCHAR(50) NOT NULL,
  exposed_at         TIMESTAMP NOT NULL,
  exposed_date       DATE NOT NULL,
  device_type        VARCHAR(20),
  app_version        VARCHAR(20),
  UNIQUE (experiment_id, user_id)      -- one variant per user per test
)
PARTITION BY RANGE (exposed_date);
```

Why each choice: - `UNIQUE (experiment_id, user_id)` enforces single-variant-per-user — the most common data integrity bug in experimentation platforms. - Separate `experiments` and `variants` allows multi-arm tests cleanly. - `exposed_at` is the first exposure, not every exposure — analytics only needs the first.

Edge cases: - Reassignment mid-test — should be impossible; if it happens, treat as a contamination metric, not data. - Anonymous users — store `anonymous_id` and stitch on login. - Forced variants for QA — flag with a `is_qa` column so they are excluded from analysis.

Alternatives: - Emit an `exposure` event into the events table — works but harder to enforce uniqueness. - Store exposures in the experimentation tool only; ETL into warehouse for analysis.

What NOT to say: - "Just look at the events table to figure out who saw the variant" — flaky; you need an explicit exposure log. - "Allow multiple variants per user, take the last one" — destroys experimental validity.

Common follow-up: *"How would you compute lift on the primary metric — assume the metric is a count of completed orders per user?"* (Join exposures to orders, group by variant, run a test of proportions or means.)

Scenario201 · A/B test conversion tracking

Tags: Difficulty: Medium · Asked at: Flipkart, Meesho, Swiggy, Razorpay · Topic: Experimentation

The interviewer's prompt: *"We have exposures. Now design the conversion metrics table so the experimentation dashboard renders in under a second."*

Thinking out loud: 1. *"Conversions per user per experiment per metric — pre-aggregated, refreshed daily or hourly."* 2. *"Wide-table approach: one row per user per experiment with all metric values as columns; works at small scale."* 3. *"Long-table approach: one row per user per metric, scales better and supports unbounded metric list."*

Schema sketch:

```
CREATE TABLE experiment_metrics_long (  
  experiment_id      UUID NOT NULL,  
  variant_id         VARCHAR(50) NOT NULL,  
  user_id            VARCHAR(50) NOT NULL,  
  metric_name        VARCHAR(100) NOT NULL,  
  metric_value       NUMERIC(18,4),  
  window_start       DATE NOT NULL,  
  window_end         DATE NOT NULL,  
  computed_at        TIMESTAMP NOT NULL,  
  PRIMARY KEY (experiment_id, user_id, metric_name, window_end)  
);  
  
-- Dashboard query pattern  
SELECT variant_id, metric_name,  
       AVG(metric_value)      AS mean,  
       STDDEV(metric_value)   AS sd,  
       COUNT(DISTINCT user_id) AS n  
FROM experiment_metrics_long  
WHERE experiment_id = :exp_id AND window_end = CURRENT_DATE  
GROUP BY variant_id, metric_name;
```

Why each choice: - Long format absorbs new metrics without DDL. - `window_end` lets you compute trailing-7-day, trailing-14-day metrics from the same table. - Pre-aggregation per user means the dashboard scans tens of thousands of rows, not billions.

Edge cases: - Users with zero conversions still need a row (with value 0) for correct denominators. - Metric drift — definitions change; track `metric_definition_version` to avoid mixing apples and oranges. - Late-arriving conversions — re-compute the trailing window on a schedule (Scenario20).

Alternatives: - Wide table with one column per metric — simpler queries but every new metric is an ALTER. - Compute on demand from raw events — accurate but slow; only viable for small experiments.

What NOT to say: - "Use the raw events table for the dashboard" — kills the warehouse at peak. - "Recompute everything every page load" — pre-aggregate; recompute on a schedule.

Common follow-up: *"A metric definition changes mid-experiment. How do you avoid mixing the old and new metric values?"* (Version the metric; freeze in-flight experiments to the version they started on.)

Scenario202 · Time-series schema for sensor readings

Tags: Difficulty: Medium · Asked at: Walmart Labs, Target, Goldman, IoT GCCs · Topic: Time-series

The interviewer's prompt: *"Design a schema for sensor readings — 10,000 sensors emitting one reading per second. Queries are mostly 'last hour for sensor X' and 'aggregate per minute over last day'."*

Thinking out loud: 1. *"This is high-frequency append-only data. Partition by time, cluster by sensor."* 2. *"Pre-aggregate at minute, hour, and day grains because asking '7-day chart at second resolution' will melt the warehouse."* 3. *"On Postgres I would reach for TimescaleDB or partitioned tables; on BigQuery I would partition by hour and cluster by sensor_id."*

Schema sketch:

```

CREATE TABLE sensor_readings (
  sensor_id      INT NOT NULL,
  reading_ts     TIMESTAMP NOT NULL,
  reading_date   DATE NOT NULL,
  reading_value  NUMERIC(10,4),
  quality_flag   SMALLINT,
  PRIMARY KEY (sensor_id, reading_ts)
)
PARTITION BY RANGE (reading_date);

CREATE TABLE sensor_readings_1min (
  sensor_id      INT NOT NULL,
  bucket_ts      TIMESTAMP NOT NULL,          -- floor to minute
  avg_value      NUMERIC(10,4),
  min_value      NUMERIC(10,4),
  max_value      NUMERIC(10,4),
  sample_count   INT,
  PRIMARY KEY (sensor_id, bucket_ts)
);

CREATE TABLE sensor_readings_1hour (
  sensor_id      INT NOT NULL,
  bucket_ts      TIMESTAMP NOT NULL,
  avg_value      NUMERIC(10,4),
  min_value      NUMERIC(10,4),
  max_value      NUMERIC(10,4),
  sample_count   INT,
  PRIMARY KEY (sensor_id, bucket_ts)
);

```

Why each choice: - Composite PK (`sensor_id, reading_ts`) matches the dominant query pattern and gives a natural cluster. - Roll-ups at minute and hour grain cut scans by 60x and 3600x for charting use cases. - Partition by `reading_date` so old partitions can be dropped or tiered to cold storage cheaply.

Edge cases: - Gaps — missing readings are common; pre-aggregates need to know whether to fill with NULL, 0, or last value. - Timezone — store UTC, convert at presentation. - Sensor metadata changes — model in a `sensor_dim` table (Type 2 if calibration history matters).

Alternatives: - Dedicated time-series databases — InfluxDB, TimescaleDB, ClickHouse — purpose-built for this. - Long-tail data lake storage with Iceberg/Hudi for older readings.

What NOT to say: - "One giant table indexed on `reading_ts`" — index alone does not partition I/O. - "Use a row per sensor with array of readings" — fights every analytical query you will ever write.

Common follow-up: "How would you handle a sensor that emits a malformed reading once an hour?" (Quality flag in the raw table; exclude from roll-up via `WHERE quality_flag = 0`.)

Scenario203 · Retention cohort schema

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Meesho, Flipkart, Cred · Topic: Cohort analysis

The interviewer's prompt: "Design a schema that powers a retention curve — 'of users who signed up in March, what % were active in week 1, week 2, week 4, week 8'."

Thinking out loud: 1. "Need a cohort definition table (anchor date), a user-cohort table (which cohort each user belongs to), and an activity-per-period table." 2. "Pre-aggregate weekly because daily retention curves are noisy and rarely useful at this grain." 3. "Output table is a `cohort_retention_pct` mart that the dashboard pivots."

Schema sketch:

```
CREATE TABLE user_cohorts (
  user_id          VARCHAR(50) PRIMARY KEY,
  signup_date      DATE NOT NULL,
  cohort_week      DATE NOT NULL,          -- Monday of signup week
  cohort_month     DATE NOT NULL,          -- 1st of signup month
  acquisition_src  VARCHAR(50)
);

CREATE TABLE user_activity_weekly (
  user_id          VARCHAR(50),
  week_start       DATE,
  was_active       BOOLEAN,
  events_count     INT,
  PRIMARY KEY (user_id, week_start)
);

CREATE TABLE cohort_retention (
  cohort_week      DATE NOT NULL,
  weeks_since      INT NOT NULL,
  active_users     INT,
  cohort_size      INT,
  retention_pct    NUMERIC(5,2),
  PRIMARY KEY (cohort_week, weeks_since)
);
```

Why each choice: - `cohort_week` and `cohort_month` precomputed so dashboards do not re-bucket every load. - `user_activity_weekly` is a thin fact, joinable to cohorts to recompute retention any time the definition of "active" changes. - `cohort_retention` is the dashboard-ready mart, refreshed daily.

Edge cases: - "Active" definition matters — must be locked per report; surface as metadata. - Resurrection — a user dormant for months becoming active again skews curves; usually included. - Cohort size denominator stays fixed; numerator changes weekly.

Alternatives: - Compute the entire curve from raw events on demand (slow at scale). - Cohort matrix table — one row per cohort, columns for week_1, week_2... (rigid but fast).

What NOT to say: - "Group by signup month and event week in one query" — works but is the slow version; pre-aggregate. - "Use ISO week vs calendar week interchangeably" — pick one and document.

Common follow-up: "Now compute month-over-month retention curve overlay on the same chart." (Add `cohort_month` slice; mart already supports it.)

Scenario204 · DAU / MAU precomputed table

Tags: Difficulty: Easy · Asked at: Flipkart, Meesho, Cred, PhonePe, Swiggy · Topic: Activity metrics

The interviewer's prompt: "Design the table that backs the 'Daily Active Users' and 'Monthly Active Users' chart on the exec dashboard."

Thinking out loud: 1. "DAU is a `COUNT(DISTINCT user_id)` for a given day — expensive on raw events at scale." 2. "Pre-aggregate by day; MAU is then computed via a 30-day rolling distinct count or via HyperLogLog sketches if the warehouse supports them." 3. "Snowflake `HLL`, BigQuery `HLL_COUNT.MERGE`, DuckDB `approx_count_distinct` — all give MAU at constant cost."

Schema sketch:

```
CREATE TABLE dau_daily (
  activity_date DATE PRIMARY KEY,
  dau          INT NOT NULL,
  new_users    INT,
  returning_users INT,
  hll_sketch   BYTEA          -- serialized HyperLogLog of distinct users
);

-- BigQuery example
CREATE OR REPLACE TABLE dau_daily AS
SELECT
  DATE(event_ts)          AS activity_date,
  COUNT(DISTINCT user_id) AS dau,
  HLL_COUNT.INIT(user_id) AS hll_sketch
FROM events
WHERE event_ts >= CURRENT_DATE - INTERVAL 1 DAY
GROUP BY activity_date;

-- 30-day MAU from sketches (BigQuery)
SELECT HLL_COUNT.MERGE(hll_sketch) AS mau_30d
FROM dau_daily
WHERE activity_date BETWEEN CURRENT_DATE - 29 AND CURRENT_DATE;
```

Why each choice: - HLL sketches let MAU be computed in O(days) work, not O(events). - `new_users` vs `returning_users` precomputed because the dashboard always shows both. - Daily grain table is small (one row per day) — trivial to query.

Edge cases: - DAU spikes from bots — filter at the events layer, not the mart. - Timezone — "day" in IST vs UTC matters for an Indian product; document and apply consistently. - Sketches are approximate (~2% error); fine for DAU/MAU, not for billing.

Alternatives: - Exact distinct counts on raw events — accurate but slow. - Materialised view auto-refreshed by the warehouse.

What NOT to say: - "Just query raw events with COUNT(DISTINCT)" — kills the warehouse on a 10B row table. - "MAU is just 30x DAU" — wrong; distinct count is not additive.

Common follow-up: *"What changes if we want WAU (weekly) as well — can we reuse the sketches?"* (Yes, merge sketches over a 7-day window.)

Scenario205 · Subscriber churn cohort tables

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Flipkart, PhonePe, SaaS-shape GCCs · Topic: Cohort modelling

The interviewer's prompt: *"We sell paid subscriptions. Design the schema that lets finance answer 'what is the month-12 churn rate for the Jan 2024 cohort'."*

Thinking out loud: 1. "Need a subscriptions fact with start, end, plan, plus a cohort assignment per subscriber." 2. "Pre-aggregate a `cohort_churn` mart at month grain — same shape as Scenario13 but for subs." 3. "Edge case: pauses, plan downgrades, voluntary vs involuntary churn — model all."

Schema sketch:

```

CREATE TABLE subscriptions (
  subscription_id  UUID PRIMARY KEY,
  user_id          VARCHAR(50) NOT NULL,
  plan_id          VARCHAR(50) NOT NULL,
  started_at       TIMESTAMP NOT NULL,
  ended_at         TIMESTAMP,
  cohort_month     DATE NOT NULL,          -- 1st of start month
  churn_reason     VARCHAR(50),           -- 'voluntary','involuntary','downgrade'
  mrr_amount       NUMERIC(12,2)
);

CREATE TABLE subscription_lifecycle (
  subscription_id  UUID,
  event_ts         TIMESTAMP,
  event_type       VARCHAR(30),           -- 'activate','pause','resume','cancel'
  reason           VARCHAR(100),
  PRIMARY KEY (subscription_id, event_ts)
);

CREATE TABLE cohort_churn (
  cohort_month     DATE,
  months_since     INT,
  cohort_size      INT,
  active_subs      INT,
  churned_subs     INT,
  churn_pct        NUMERIC(5,2),
  PRIMARY KEY (cohort_month, months_since)
);

```

Why each choice: - `subscription_lifecycle` keeps the full event history so any future definition change can be reapplied. - `subscriptions` fact carries the cohort attribution. - `cohort_churn` mart is dashboard-ready; refreshed monthly.

Edge cases: - Pauses — exclude paused months from "active" count or include depending on definition. - Reactivations — same user signing up again creates a new subscription row, possibly a new cohort. - Involuntary churn (card fail) recovery — many fail-then-retry; freeze "churn" decision after a grace window.

Alternatives: - Wide table — one row per subscriber per month with active/inactive flag. - Survival analysis tooling in Python rather than SQL.

What NOT to say: - "Just count subs ended in month N" — misses paused, refunded, downgraded. - "Cohort starts when the user signs up, not when the sub starts" — only true if signup = activation; usually not.

Common follow-up: *"How would you compute net revenue retention (NRR) on this schema?"* (Sum of MRR for cohort at month N divided by MRR at month 0; include upgrades.)

Scenario206 · Revenue by product × month rollup

Tags: Difficulty: Easy · Asked at: Walmart Labs, Target, Flipkart, Meesho · Topic: Aggregate marts

The interviewer's prompt: "The CFO opens the dashboard and wants revenue per product per month in 1 second. Design the mart."

Thinking out loud: 1. "Fact already exists at line grain (Scenario1). Build a monthly roll-up keyed by product and month." 2. "Include gross, returns, net, units, distinct customers. Cover the common slices." 3. "Materialised view or scheduled mart, refresh nightly."

Schema sketch:

```
CREATE TABLE mart_product_monthly (
  product_key      BIGINT NOT NULL,
  month_start      DATE NOT NULL,
  gross_revenue    NUMERIC(18,2),
  refund_amount    NUMERIC(18,2),
  net_revenue      NUMERIC(18,2),
  units_sold       INT,
  unique_customers INT,
  refreshed_at     TIMESTAMP,
  PRIMARY KEY (product_key, month_start)
);
CREATE INDEX ON mart_product_monthly (month_start);

-- Refresh job (idempotent)
DELETE FROM mart_product_monthly WHERE month_start = :target_month;
INSERT INTO mart_product_monthly
SELECT
  product_key,
  DATE_TRUNC('month', date_actual) AS month_start,
  SUM(line_total) AS gross_revenue,
  SUM(CASE WHEN transaction_type='refund' THEN line_total ELSE 0 END) AS refund_amount,
  SUM(CASE WHEN transaction_type='sale' THEN line_total ELSE -line_total END) AS net_revenue,
  SUM(quantity) AS units_sold,
  COUNT(DISTINCT customer_key) AS unique_customers,
  CURRENT_TIMESTAMP AS refreshed_at
FROM fact_sales f JOIN dim_date d ON f.date_key = d.date_key
WHERE d.month_start = :target_month
GROUP BY product_key, month_start;
```

Why each choice: - Idempotent `DELETE+INSERT` per month makes back-fills safe. - `unique_customers` precomputed; otherwise dashboard does its own distinct count. - Refresh per month gives clean recompute windows.

Edge cases: - Late-arriving refunds — refresh the last 3 months on every run. - Currency conversion — apply with monthly average FX from a dim. - Time-zone — pin to financial close calendar, not naive UTC.

Alternatives: - Snowflake / BigQuery materialised view with auto-refresh. - Cube.dev or dbt incremental model.

What NOT to say: - "Just SELECT from `fact_sales` with `GROUP BY` " — works at small scale only.
- "INSERT-only mart, never delete" — back-fills become impossible without dedup logic.

Common follow-up: *"How would you let the CFO drill down to a single product's daily revenue from this mart?"* (Mart is monthly; click-through hits a daily mart or the fact directly, scoped by product key.)

Scenario207 · Multi-touch attribution model

Tags: Difficulty: Hard · Asked at: Razorpay, Meesho, Flipkart, Performance Marketing GCCs · Topic: Attribution

The interviewer's prompt: *"We run ads on Google, Meta, Affiliates. We want to attribute revenue across all touchpoints — first-touch, last-touch, linear, time-decay. Design the schema."*

Thinking out loud: 1. "Two layers — raw `marketing_touches` table with every click and impression, and an `attribution_results` mart with each touchpoint's credit per conversion." 2. "Attribution model is metadata, not a separate table per model — store all models' credits on the same touchpoint row." 3. "Conversion link via `user_id` (post-login) and `gclid/fbclid` (pre-login)."

Schema sketch:

```

CREATE TABLE marketing_touches (
  touch_id          UUID PRIMARY KEY,
  touch_ts          TIMESTAMP NOT NULL,
  touch_date        DATE NOT NULL,
  user_id           VARCHAR(50),
  anonymous_id       VARCHAR(50),
  channel           VARCHAR(50),          -- 'google_ads','meta','affiliate'
  campaign          VARCHAR(100),
  utm_source        VARCHAR(100),
  utm_medium        VARCHAR(100),
  click_id          VARCHAR(100),
  cost              NUMERIC(10,2)
)
PARTITION BY RANGE (touch_date);

CREATE TABLE conversions (
  conversion_id     UUID PRIMARY KEY,
  user_id           VARCHAR(50) NOT NULL,
  conversion_ts     TIMESTAMP NOT NULL,
  order_id          VARCHAR(50),
  revenue           NUMERIC(12,2)
);

CREATE TABLE attribution_credits (
  conversion_id     UUID NOT NULL,
  touch_id          UUID NOT NULL,
  model_name        VARCHAR(50) NOT NULL,  -- 'first','last','linear','time_decay'
  credit_pct        NUMERIC(6,4),
  credit_revenue    NUMERIC(12,2),
  PRIMARY KEY (conversion_id, touch_id, model_name)
);

```

Why each choice: - Touch and conversion are raw; attribution is a computed view over both, parameterised by model. - `attribution_credits` stores results per model so a dashboard can switch models without re-computation. - Partition by `touch_date` for cost-effective lookback window queries.

Edge cases: - Lookback window — usually 28 or 90 days; configurable. - Post-login stitching — must join via `user_id` and `click_id`; handle cases where user logs in mid-funnel. - View-through vs click-through — model as touch_type with `weight` adjustment.

Alternatives: - Use the ad platform's own attribution and just store reports. - Shapley / Markov chain attribution — store learned credits in the same table.

What NOT to say: - "Use last-touch only because it is simpler" — most India product teams want at least linear today. - "Recompute attribution every dashboard load" — too expensive; mart it.

Common follow-up: "How would you handle a user who clicks five Google ads, then sees a Meta retargeting ad, then converts?" (Linear: 17% each; first-touch: 100% to first Google; time-decay: heavier on later touches.)

Scenario208 · Customer 360 wide table

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Flipkart, Meesho, PhonePe · Topic: One-big-table

The interviewer's prompt: *"Build the 'Customer 360' table — one row per customer with everything marketing, finance, and product might want to slice by."*

Thinking out loud: 1. *"One row per customer with denormalised attributes — joins eliminated for downstream consumers."* 2. *"Refresh daily, computed from many upstream marts."* 3. *"Wide is fine on columnar warehouses; 100+ columns is normal."*

Schema sketch:

```
CREATE TABLE customer_360 (  
  customer_id          VARCHAR(50) PRIMARY KEY,  
  full_name            VARCHAR(200),  
  email                VARCHAR(255),  
  phone_hashed         VARCHAR(64),  
  
  signup_date          DATE,  
  signup_channel        VARCHAR(50),  
  acquisition_cost      NUMERIC(10,2),  
  
  city                 VARCHAR(100),  
  state                VARCHAR(100),  
  tier                  VARCHAR(20),  
  
  ltv_to_date          NUMERIC(12,2),  
  total_orders          INT,  
  first_order_date      DATE,  
  last_order_date       DATE,  
  avg_order_value       NUMERIC(10,2),  
  
  is_subscriber         BOOLEAN,  
  current_plan          VARCHAR(50),  
  mrr_amount            NUMERIC(10,2),  
  
  last_app_open_at      TIMESTAMP,  
  is_active_30d         BOOLEAN,  
  churn_risk_score      NUMERIC(4,3),      -- model output  
  
  refund_count          INT,  
  support_ticket_count  INT,  
  
  refreshed_at          TIMESTAMP NOT NULL  
);
```


Why each choice: - Single row per customer collapses what would otherwise be a 6-way join. - Includes ML score (`churn_risk_score`) so the marketing team does not re-derive it. - Refresh daily; the freshness is documented per column when SLAs differ.

Edge cases: - Columns from low-freshness sources can stale-bias decisions — document freshness per column. - PII columns (`email` , `phone`) — hash and gate access; do not surface to BI by default. - Schema bloat over time — review quarterly; deprecate unused columns with metadata.

Alternatives: - Multiple narrow marts joined at query time — slower but easier to evolve. - Feature store for ML features; keep the 360 table for analytics only.

What NOT to say: - "Wide tables are an antipattern" — not in modern columnar warehouses; they are the dominant pattern. - "Let downstream join everything themselves" — defeats the purpose; people will not.

Common follow-up: *"How would you keep this table consistent if `ltv_to_date` is updated daily but `churn_risk_score` is updated weekly?"* (Separate refresh windows per column group; store per-column `refreshed_at` if needed.)

Scenario209 · Slowly accumulating fact — order status history

Tags: Difficulty: Medium · Asked at: Swiggy, Flipkart, Meesho, Walmart Labs · Topic: Fact patterns

The interviewer's prompt: *"An order goes through created → packed → shipped → delivered → returned. Design the schema that powers 'how long did orders take to ship in March'."*

Thinking out loud: 1. *"Two valid shapes — accumulating snapshot (one row per order, columns for each milestone timestamp) or status_history (one row per status change)."* 2. *"Accumulating snapshot is fast for milestone-to-milestone duration queries; history is the audit log."* 3. *"Build both: history is source of truth, snapshot is the analytics-friendly form."*

Schema sketch:

```

CREATE TABLE order_status_history (
  history_id      BIGSERIAL PRIMARY KEY,
  order_id        VARCHAR(50) NOT NULL,
  status          VARCHAR(30) NOT NULL,
  changed_at      TIMESTAMP NOT NULL,
  changed_by      VARCHAR(50)
);
CREATE INDEX ON order_status_history (order_id, changed_at);

CREATE TABLE fact_order_lifecycle (
  order_id        VARCHAR(50) PRIMARY KEY,
  customer_key    BIGINT,
  created_at      TIMESTAMP,
  packed_at       TIMESTAMP,
  shipped_at       TIMESTAMP,
  delivered_at     TIMESTAMP,
  returned_at     TIMESTAMP,

  pack_duration_h NUMERIC(8,2),      -- packed_at - created_at
  ship_duration_h  NUMERIC(8,2),
  deliver_duration_h NUMERIC(8,2),

  is_returned     BOOLEAN,
  current_status   VARCHAR(30),
  refreshed_at    TIMESTAMP
);

```

Why each choice: - Accumulating snapshot is **UPDATE** -heavy as new milestones land — fine for one row per order. - Durations precomputed because every dashboard asks for them. - History table preserves the audit trail and supports re-derivation.

Edge cases: - Out-of-order events — the snapshot updater must check timestamps before overwriting. - Skipped milestones — **packed_at** NULL means it was never packed; treat as still-in-progress, not zero. - Returns reopen the lifecycle — model carefully (separate **returned_at** rather than a new row).

Alternatives: - Pure event-source via **order_status_history** ; compute lifecycle on the fly. - Stream-processed snapshot via Flink / Kafka.

What NOT to say: - "Just one row per order, no history" — first time the business asks 'what changed when', you are stuck. - "Use a single timestamp column with current status only" — loses every previous state.

Common follow-up: "How would you alert when an order has been in 'packed' for more than 24 hours?" (Query the snapshot for **packed_at < NOW() - 24h AND shipped_at IS NULL**.)

Scenario210 · Late-arriving facts

Tags: Difficulty: Hard · Asked at: Walmart Labs, Goldman, JP Morgan India, Target · Topic: Data quality

The interviewer's prompt: "Sales for 1 May arrived in your warehouse on 5 May because the source system was down. Your daily mart already ran for 1 May. How do you handle it?"

Thinking out loud: 1. "Late-arriving facts are normal. Design assuming they will happen rather than treating them as exceptions." 2. "Either (a) refresh a window of past days every run, or (b) use append-only fact with an `arrival_date` separate from `event_date`." 3. "For dashboards, the natural answer is option (a) — refresh the last 7 days nightly so anything that lagged catches up."

Schema sketch:

```
CREATE TABLE fact_sales (
  sale_id          BIGSERIAL PRIMARY KEY,
  event_date       DATE NOT NULL,          -- business day
  arrival_date     DATE NOT NULL,          -- when we received it
  product_key      BIGINT, customer_key BIGINT,
  line_total       NUMERIC(12,2)
)
PARTITION BY RANGE (event_date);

CREATE TABLE mart_sales_daily (
  event_date       DATE PRIMARY KEY,
  gross_revenue    NUMERIC(18,2),
  order_count      INT,
  refreshed_at     TIMESTAMP,
  is_finalised     BOOLEAN DEFAULT FALSE   -- true after 7d lookback closes
);

-- Refresh window pattern
DELETE FROM mart_sales_daily WHERE event_date BETWEEN :start AND :end;
INSERT INTO mart_sales_daily ...
WHERE event_date BETWEEN :start AND :end;
```

Why each choice: - `arrival_date` separate from `event_date` lets data engineering compute "arrival lag" SLAs. - 7-day rolling refresh absorbs most lateness automatically. - `is_finalised` flag tells downstream BI tools when numbers stop moving.

Edge cases: - Very late facts (months out) — finance often locks numbers; corrections go into a `prior_period_adjustments` table rather than mutating history. - Schema changes between old and new facts — store `schema_version` per row. - Reconciliation alerts — compute "arrival lag > 3 days" as a quality metric.

Alternatives: - Append-only `corrections` table layered over the original mart. - Snowflake `STREAM` + `TASK` for incremental, continuous catch-up.

What NOT to say: - "Pretend late data does not exist" — silently understates revenue. - "Re-run the entire history nightly" — wasteful at scale; window-refresh is the standard pattern.

Common follow-up: *"What if the late-arriving fact is for a closed financial period?"* (Finance freezes the period; corrections go to a separate adjustments table and reconciliation report.)

Scenario211 · Late-arriving dimensions

Tags: Difficulty: Hard · Asked at: Walmart Labs, Goldman, Target, Wells Fargo · Topic: Dimensional modelling

The interviewer's prompt: *"A fact references customer_id 12345, but the customer dimension does not have that row yet — it shows up an hour later. What does your ETL do?"*

Thinking out loud: 1. *"This is a late-arriving dimension. Three strategies — reject, defer, or insert a placeholder."* 2. *"Best practice in modern warehouses — insert a placeholder dim row with a known surrogate, fill in attributes when they arrive."* 3. *"Track these explicitly so they do not silently corrupt cohorts and segmentation."*

Schema sketch:

```
CREATE TABLE dim_customer (  
  customer_key      BIGSERIAL PRIMARY KEY,  
  customer_id       VARCHAR(50) UNIQUE NOT NULL,  
  full_name         VARCHAR(200),  
  city              VARCHAR(100),  
  tier               VARCHAR(20),  
  is_inferred       BOOLEAN NOT NULL DEFAULT FALSE,    -- TRUE when placeholder  
  inferred_at       TIMESTAMP,  
  attributes_arrived_at TIMESTAMP,  
  updated_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
-- ETL pseudo-flow  
-- 1. Fact references customer_id X  
-- 2. Lookup dim_customer; if missing, INSERT placeholder  
--    (customer_id, is_inferred=TRUE, inferred_at=NOW())  
-- 3. When customer dim batch arrives, UPDATE attributes,  
--    SET is_inferred=FALSE, attributes_arrived_at=NOW().
```

Why each choice: - Fact-table writes never block on dim arrival — keeps ingestion flowing. - Placeholder rows are visible in audits, not silently merged. - `is_inferred` lets analysts exclude these rows when working on customer segmentation if needed.

Edge cases: - Some attributes (like city) drive partitioning of downstream marts; recomputing marts after dim back-fill matters. - The customer may never arrive — treat ghosts as a data quality alert. - Duplicates — natural-key uniqueness must hold even for placeholders.

Alternatives: - Quarantine table — defer facts until the dim row arrives; sometimes required when fact joins must produce correct labels. - Backfill-and-replay — wait, then process the whole batch.

What NOT to say: - "Drop the fact row if the dim is missing" — silent data loss. - "Insert a random new customer row each time" — duplicates the customer dim and breaks reporting.

Common follow-up: *"What if 5% of facts trigger placeholders consistently — what does that suggest?"* (Source system has a different cadence than the customer feed; widen ingestion windows or fix the upstream contract.)

Scenario212 · Conformed dimensions across multiple facts

Tags: Difficulty: Medium · Asked at: Walmart Labs, Target, Goldman, JP Morgan India · Topic: Dimensional modelling

The interviewer's prompt: *"We have `fact_sales`, `fact_returns`, `fact_marketing`. All three reference `dim_customer` and `dim_date`. How do you guarantee they all use the same dim definitions?"*

Thinking out loud: 1. *"Conformed dims — exactly one canonical dim table per business entity, used by every fact."* 2. *"This is what makes 'drill across' work: a single chart can blend sales, returns, and marketing per customer."* 3. *"Govern via dbt models / governed datasets; never let a fact reach into a custom local dim."*

Schema sketch:

```
-- Shared dims
CREATE TABLE dim_customer (...);           -- one customer record per row
CREATE TABLE dim_date (...);
CREATE TABLE dim_product (...);

-- Multiple facts referencing them
CREATE TABLE fact_sales      (date_key, customer_key, product_key, ...);
CREATE TABLE fact_returns    (date_key, customer_key, product_key, ...);
CREATE TABLE fact_marketing  (date_key, customer_key, campaign_key, ...);

-- A "drill across" SQL pattern
SELECT c.tier,
       SUM(s.line_total) AS revenue,
       SUM(r.refund_amount) AS refunds,
       SUM(m.cost) AS marketing_cost
FROM dim_customer c
LEFT JOIN fact_sales      s ON s.customer_key = c.customer_key
LEFT JOIN fact_returns    r ON r.customer_key = c.customer_key
LEFT JOIN fact_marketing  m ON m.customer_key = c.customer_key
GROUP BY c.tier;
```

Why each choice: - Shared dims mean a single chart can blend three facts without a hand-rolled key map. - Surrogate `customer_key` is stable across facts because it points at the same dim version. - The dim becomes the contract — all facts conform to it, not the other way.

Edge cases: - Different facts at different grains — a sale is per order line, a marketing touch is per click; conformation works at the dim level, not the grain. - A fact-specific attribute (e.g. `seller_id` only meaningful for sales) belongs in the fact, not the customer dim. - Schema drift in the dim — version dim updates; downstream models test on the dim contract.

Alternatives: - Per-domain dims with bridge tables — heavier, only when business definitions truly diverge. - Federated query approach if dims live in different systems.

What NOT to say: - "Let each fact carry its own copy of customer attributes" — guarantees inconsistency. - "Conformation is a Kimball relic" — still the default in dbt-style marts today.

Common follow-up: *"What is the test you would write to ensure conformation does not break?"* (Assert that every `customer_key` in every fact exists in `dim_customer` ; dbt's `relationships` test does exactly this.)

Scenario213 · Junk dimension for low-cardinality booleans

Tags: Difficulty: Easy · Asked at: Goldman, JP Morgan India, Walmart Labs · Topic: Dimensional modelling

The interviewer's prompt: *"My fact has 6 boolean flags — `is_gift`, `is_prepaid`, `is_first_order`, `is_promo`, `is_cod_eligible`, `is_express`. Where do they go?"*

Thinking out loud: 1. *"Six booleans means 64 possible combinations — too small to be its own dim per flag, too inelegant to dump on the fact."* 2. *"Junk dimension — one row per combination, single key on the fact."* 3. *"Trades a tiny dim table for narrower facts and clean group-by semantics."*

Schema sketch:

```
CREATE TABLE dim_order_flags (
  order_flags_key  SMALLINT PRIMARY KEY,
  is_gift          BOOLEAN, is_prepaid BOOLEAN, is_first_order BOOLEAN,
  is_promo         BOOLEAN, is_cod_eligible BOOLEAN, is_express BOOLEAN,
  description      VARCHAR(255)          -- human-readable
);

CREATE TABLE fact_sales (
  sale_id          BIGSERIAL PRIMARY KEY,
  date_key         INT,
  customer_key     BIGINT,
  product_key      BIGINT,
  order_flags_key  SMALLINT REFERENCES dim_order_flags(order_flags_key),
  line_total       NUMERIC(12,2)
);
```

Why each choice: - Single small INT on the fact replaces 6 BOOLEANs — narrower row, better compression. - Dim is up to 64 rows; queries can group by combinations and slice with `WHERE is_gift = TRUE`. - Reports needing names (`description`) get them from the dim, not hard-coded.

Edge cases: - If flag set explodes (12 booleans → 4096 rows), revisit — junk dim is for small sets. - New flag added — backfill the junk dim and update flag combinations. - Sparse combinations — only create dim rows for combinations that actually appear.

Alternatives: - Store flags directly on the fact — fine at small scale; ugly when flags multiply. - Bitfield INT — compact but unreadable.

What NOT to say: - "One dim per boolean" — explosion of pointless dims. - "Just use a JSONB column for flags" — kills filter pushdown.

Common follow-up: "What if you have a non-boolean low-cardinality flag like 'priority' = HIGH/MED/LOW?" (Same junk dim, just more values per row.)

Scenario214 · Degenerate dimension

Tags: Difficulty: Easy · Asked at: Walmart Labs, Goldman, AmEx GCC · Topic: Dimensional modelling

The interviewer's prompt: "Where should `order_id` live in your star schema? It is unique per order, but the order has no real attributes worth a dim."

Thinking out loud: 1. "Classic degenerate dimension — an ID with no attributes of its own. Lives on the fact, no dim table needed." 2. "You still query by it ('distinct count of orders this month') so it has to be on the fact." 3. "Same pattern for `invoice_number`, `ticket_id`, anything ID-shaped with no descriptive attributes."

Schema sketch:

```
CREATE TABLE fact_sales (
  sale_id          BIGSERIAL PRIMARY KEY,    -- surrogate
  date_key         INT,
  customer_key     BIGINT,
  product_key      BIGINT,
  order_id         VARCHAR(50) NOT NULL,    -- degenerate dim
  invoice_number   VARCHAR(50),            -- degenerate dim
  line_total       NUMERIC(12,2)
);
CREATE INDEX ON fact_sales (order_id);

-- Distinct order count
SELECT date_key, COUNT(DISTINCT order_id) AS orders
FROM fact_sales
GROUP BY date_key;
```

Why each choice: - No table for `order_id` means no join cost to count orders. - Indexed so distinct counts are fast. - Same row holds the customer dim key — easy customer-per-order analytics.

Edge cases: - If `order_id` later acquires attributes (channel, source), promote to a real dim and remove from fact. - Be wary of treating a degenerate as a join key to other systems — it is meant for self-contained aggregations.

Alternatives: - Build an `order_dim` even if attributes are sparse — overhead without payoff. - Store both `order_id` and a surrogate `order_key` — redundant unless joining to an order-detail fact.

What NOT to say: - "Order ID belongs in a customer dim" — different grain, breaks the customer-per-row rule. - "Use `order_id` as the surrogate sale_key" — couples your warehouse to the source system's ID format.

Common follow-up: "When does a degenerate dim graduate into a full dim?" (When it acquires its own attributes — channel, source, fulfilment SLA — that need slicing.)

Scenario215 · Bridge table for many-to-many in star schema

Tags: Difficulty: Hard · Asked at: Walmart Labs, Target, JP Morgan India · Topic: Dimensional modelling

The interviewer's prompt: "A product can belong to multiple categories — Electronics and Gifts, say. How do you model this in a star schema?"

Thinking out loud: 1. "Many-to-many between fact and dim needs a bridge table. Bridge sits between the fact (or product dim) and the multi-valued dim (category)." 2. "Bridge carries a weight column so SUM aggregations are not double-counted." 3. "Same pattern for product-tag, customer-segment, account-owner."

Schema sketch:

```
CREATE TABLE dim_category (
  category_key      INT PRIMARY KEY,
  category_name     VARCHAR(100)
);

CREATE TABLE bridge_product_category (
  product_key       BIGINT REFERENCES dim_product(product_key),
  category_key      INT REFERENCES dim_category(category_key),
  weight            NUMERIC(5,4),          -- e.g. 0.5 if equal split
  PRIMARY KEY (product_key, category_key)
);

-- "Revenue by category" without double counting
SELECT cat.category_name,
       SUM(f.line_total * b.weight) AS attributed_revenue
FROM fact_sales f
JOIN bridge_product_category b ON f.product_key = b.product_key
JOIN dim_category cat ON cat.category_key = b.category_key
GROUP BY cat.category_name;
```

Why each choice: - Bridge encodes the many-to-many relationship without duplicating fact rows. - `weight` lets users choose between summed-across (full credit each side) and apportioned (weights sum to 1.0). - PK ensures no duplicate product-category pairs.

Edge cases: - Weights summing to less than 1 means unassigned credit — decide if that is OK. - Categories changing over time — bridge becomes Type 2 with `valid_from` / `valid_to`. - "Primary category" — add an `is_primary` flag for the case where you must pick one.

Alternatives: - Pre-join into a `fact_category` table that duplicates the fact row per category (simpler queries, larger storage). - Use ARRAY/STRUCT in BigQuery instead of a bridge table.

What NOT to say: - "Just denormalise: store categories as a comma-separated string" — kills filter and join. - "Pick the first category and ignore the rest" — destroys analytic completeness.

Common follow-up: *"What if the same product can move from one category to another over time?"* (Bridge becomes time-aware with `valid_from` / `valid_to` ; join requires temporal predicate.)

Scenario216 · Star schema vs One Big Table (OBT) tradeoff

Tags: Difficulty: Medium · Asked at: Razorpay, Meesho, Cred, Flipkart, modern data GCCs · Topic: Modelling tradeoffs

The interviewer's prompt: *"Modern thinking on Snowflake / BigQuery says 'just build one big table'. Why would you still use a star schema?"*

Thinking out loud: 1. "OBT denormalises everything into one wide table — no joins at query time." 2. "Star schema keeps facts narrow and joins to dims — less storage churn on dim updates, easier governance." 3. "In 2026 most modern teams use a hybrid — star at the warehouse layer, OBT at the BI consumption layer."

Schema sketch:

```
-- OBT shape
CREATE TABLE obt_sales (
  sale_id          BIGSERIAL PRIMARY KEY,
  event_date       DATE,
  event_year       INT, event_month INT, event_quarter INT,    -- date fields inlined
  product_id       VARCHAR(50), product_name VARCHAR(255),
  product_category VARCHAR(100), brand_name VARCHAR(100),
  customer_id      VARCHAR(50), customer_city VARCHAR(100),
  customer_tier    VARCHAR(20),
  line_total       NUMERIC(12,2)
);
-- Snowflake: CLUSTER BY (event_date, product_category)
-- BigQuery: PARTITION BY event_date CLUSTER BY product_category
```

Why each choice for OBT: - Zero joins for the most common queries — fast on columnar storage. - BI tools love flat tables — fewer wrong-join bugs. - Compression handles the redundancy well.

Why star is still a thing: - Updating a dim (e.g. category rename) updates one row, not millions. - Governance — dim ownership and lineage are clear. - Reuse — `dim_customer` powers sales, returns, marketing, churn marts.

Edge cases: - OBT on hot dims (SCD Type 2 every day) — storage and refresh cost explodes. - Star with too many small dims — endless joins; OBT-flatten the small ones into the fact. - Hybrid: star at the warehouse layer, OBT views or marts on top.

Alternatives: - Data vault — agnostic intermediate layer. - Activity schema — single events-shaped table for all activities.

What NOT to say: - "OBT is always the modern answer" — depends on dim volatility and dataset size. - "Star is always the modern answer" — modern columnar warehouses subvert old tradeoffs.

Common follow-up: "At what data scale or query pattern would you reverse this choice?" (When dims change daily, star wins; when BI users are non-technical, OBT wins.)

Scenario217 · Snowflake clustering key for an analytics fact

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Snowflake-shape GCCs · Topic: Snowflake performance

The interviewer's prompt: "You have a 5-billion-row fact table on Snowflake. Queries filter on `event_date` and `customer_id`. How would you cluster it?"

Thinking out loud: 1. "Snowflake stores data in micro-partitions; clustering keys colocate rows so partition pruning is effective." 2. "Most common predicate first — `event_date` because almost every query filters by time." 3. "Then `customer_id` for the secondary pattern. Avoid high-cardinality clustering on its own — Snowflake prefers ranges."

Schema sketch:

```
CREATE TABLE fact_events (  
  event_id          STRING,  
  event_date        DATE NOT NULL,  
  event_ts          TIMESTAMP NOT NULL,  
  customer_id       STRING NOT NULL,  
  product_id        STRING,  
  event_name        STRING,  
  properties        VARIANT  
)  
CLUSTER BY (event_date, customer_id);  
  
-- Inspect clustering health  
SELECT SYSTEM$CLUSTERING_INFORMATION('fact_events',  
  '(event_date, customer_id)');
```

Why each choice: - `event_date` first — the universal range predicate. - `customer_id` second — narrows within a day for per-customer queries. - Snowflake auto-reclusters in the background; you pay credits for the reclustering work.

Edge cases: - High clustering depth = bad — large delta between min/max micro-partition values for the cluster key. - Wide updates re-cluster everything; large batch deletes can fragment. - Sorting on load (ORDER BY in COPY INTO) reduces reclustering cost.

Alternatives: - Materialised views with their own clustering for sub-query patterns. - Search optimisation service on high-cardinality predicates.

What NOT to say: - "Add an index" — Snowflake has no traditional indexes; clustering is the mechanism. - "Cluster on everything" — diminishing returns; pick 2-3 columns at most.

Common follow-up: "How do you know your clustering is helping?"

(`SYSTEM$CLUSTERING_INFORMATION` ; look at depth and overlap; compare query elapsed before/after.)

Scenario218 · BigQuery partitioning + clustering for facts

Tags: Difficulty: Medium · Asked at: Flipkart, Meesho, Swiggy, BigQuery-shape GCCs · Topic: BigQuery performance

The interviewer's prompt: "Same problem on BigQuery — 5 billion rows, time and customer filters. How do you design the table?"

Thinking out loud: 1. "BigQuery has explicit partitioning (one date-shaped column) plus clustering (up to 4 columns)." 2. "Partition by ingestion date or event date — both prune scans massively." 3. "Cluster by `customer_id, product_id` — top filter columns."

Schema sketch:

```
CREATE TABLE analytics.fact_events (  
  event_id      STRING,  
  event_ts      TIMESTAMP,  
  event_date    DATE,  
  customer_id   STRING,  
  product_id    STRING,  
  event_name    STRING,  
  properties    JSON  
)  
PARTITION BY event_date  
CLUSTER BY customer_id, product_id  
OPTIONS (  
  partition_expiration_days = 730,      -- two-year retention  
  require_partition_filter = TRUE       -- forces queries to scope by partition  
);
```

Why each choice: - `PARTITION BY event_date` — partition pruning means a 1-day query scans 1/1000 of data. - `require_partition_filter = TRUE` — saves the team from accidental full-table scans (which BigQuery bills by bytes scanned). - Clustering by `customer_id, product_id` because both appear in most predicates.

Edge cases: - Daily partitions are the default; hourly partitions exist (BigQuery has time-unit partitioning) but cost more in metadata. - Number of partitions per table is capped (currently 4000) — design for retention vs grain. - Updates and merges within a partition are fine; cross-partition operations cost more.

Alternatives: - Integer-range partitioning if `event_date` is not natural. - Sharded tables (table_YYYYMMDD pattern) — older approach, partitioning is now preferred.

What NOT to say: - "Partition by both date and customer_id" — partition is one column only. - "Don't bother clustering if you partition" — clustering still meaningfully reduces bytes scanned within a partition.

Common follow-up: "What is the difference between `_PARTITIONTIME` and a column-based partition?" (`_PARTITIONTIME` is ingestion-time; column-based is event-time. Event-time matches business semantics for late-arriving data.)

Scenario219 · dbt-style staging → intermediate → mart layering

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Flipkart, Meesho, modern data GCCs · Topic: Modern data stack

The interviewer's prompt: "Walk me through the layered schema design we would use in dbt — staging, intermediate, mart. Why three layers?"

Thinking out loud: 1. "Staging — one-to-one with source tables; rename columns, cast types, no joins." 2. "Intermediate — business logic, joins between cleaned-up sources; reusable building blocks." 3. "Mart — dashboard-ready; star schemas, wide tables, exposed to BI."

Schema sketch:

```
-- 1. STAGING (stg_orders.sql)
WITH source AS (SELECT * FROM raw.shopify.orders)
SELECT
    id                AS order_id,
    customer_id,
    CAST(total_price AS NUMERIC(12,2)) AS total_amount,
    status,
    TIMESTAMP(created_at)              AS created_at,
    CURRENT_TIMESTAMP                  AS dbt_loaded_at
FROM source;

-- 2. INTERMEDIATE (int_orders_with_customer.sql)
SELECT
    o.order_id, o.total_amount, o.created_at,
    c.tier, c.city
FROM {{ ref('stg_orders') }} o
LEFT JOIN {{ ref('stg_customers') }} c USING (customer_id);

-- 3. MART (mart_sales_daily.sql)
SELECT
    DATE(created_at)              AS order_date,
    city, tier,
    COUNT(*)                      AS orders,
    SUM(total_amount)             AS revenue
FROM {{ ref('int_orders_with_customer') }}
GROUP BY 1, 2, 3;
```

Why each layer: - Staging isolates source changes — one rename here, no breakage downstream. - Intermediate prevents reimplementing the same joins in multiple marts. - Mart is the contract with the business — change only with explicit communication.

Edge cases: - Skipping intermediate is fine for tiny projects; mandatory for 100+ models. - Tests live at every layer — staging tests source quality, mart tests business invariants. - Snapshots (dbt's SCD Type 2 tool) live alongside staging.

Alternatives: - Data Vault layering — raw vault, business vault, marts. - Single mart layer for very small teams.

What NOT to say: - "Layers are overhead" — at 50+ models, no layering means unmaintainable. - "Marts can read from staging directly" — couples marts to source changes; defeats the layering.

Common follow-up: *"Where would you put a custom UDF or macro?"* (Macros in [macros/](#) ; reusable transforms applied across staging or intermediate.)

Scenario220 · Data lake + warehouse hybrid (Iceberg / Hudi pattern)

Tags: Difficulty: Hard · Asked at: Flipkart, Walmart Labs, Meesho, JP Morgan India, Goldman · Topic: Lakehouse architecture

The interviewer's prompt: *"We have terabytes of raw events landing in S3 daily. We want SQL on it via the warehouse but also keep the raw lake for ML. Design the layering."*

Thinking out loud: 1. *"Lakehouse pattern — Iceberg or Hudi tables on object storage, queried by Snowflake / BigQuery / Trino / Spark."* 2. *"Raw zone (immutable JSON/Avro), curated zone (Iceberg with schema), serving zone (warehouse marts)."* 3. *"Iceberg gives ACID, schema evolution, time travel — without leaving open storage."*

Schema sketch:

```
s3://datalake/
  raw/events/dt=2026-05-26/...avro
  curated/events_iceberg/                -- Iceberg table
    metadata/
    data/dt=2026-05-26/part-00001.parquet
  serving/                                -- mart layer
    mart_user_sessions/

-- Iceberg DDL (Spark / Trino syntax)
CREATE TABLE curated.events (
  event_id   STRING,
  event_ts   TIMESTAMP,
  user_id    STRING,
  event_name STRING,
  properties MAP<STRING, STRING>
)
USING iceberg
PARTITIONED BY (days(event_ts))
TBLPROPERTIES ('format-version' = '2');

-- Warehouse external table (Snowflake reads Iceberg)
CREATE OR REPLACE ICEBERG TABLE serving.events_iceberg
  EXTERNAL_VOLUME = 'datalake_vol'
  CATALOG = 'glue_catalog'
  CATALOG_TABLE_NAME = 'curated.events';
```

Why each choice: - Raw zone preserves source-of-truth bytes — required for replay and ML feature regeneration. - Iceberg in curated zone adds ACID and schema evolution while staying open-format. - Warehouse reads the same Iceberg files — no double-copy of data.

Edge cases: - Small files problem — frequent micro-batches create thousands of tiny Parquet files; periodic compaction is mandatory. - Schema evolution — additive only (add columns); breaking changes need versioned tables. - Cross-engine compatibility — confirm your engine's Iceberg version compatibility (V1 vs V2 manifests).

Alternatives: - Hudi for upsert-heavy workloads; Iceberg for analytic appends. - Delta Lake on Databricks ecosystem. - Direct warehouse-native ingestion (Snowpipe, BigQuery batch load) — simpler if open-format is not required.

What NOT to say: - "Just dump CSVs in S3 and query with Athena" — works at small scale; chokes on schema drift and updates. - "Lake and warehouse are obsolete; pick one" — the hybrid is the modern default for any team with both ML and BI needs.

Common follow-up: *"How would you handle a GDPR-style right-to-erasure request on Iceberg?"* (Iceberg row-level delete; Hudi delete API; or rewrite affected files and create a new snapshot.)

Category 4, Part A — What Will This Return: NULL, Type & Ordering

The classic SQL interview format goes like this — the interviewer slides a query across the table and asks, "What does this return? Walk me through your reasoning." They are not testing if you can run it in your head. They are testing whether you understand the silent rules the engine applies before producing output. Part A of this category focuses on the three areas that trip up the largest number of candidates: NULL semantics (the three-valued logic that breaks naive boolean thinking), implicit type coercion (where the engine quietly converts between integer, string and date and changes the answer), and result ordering (where two reasonable-looking queries return rows in different orders on different engines). Walk through each one out loud — even when the answer feels obvious, narrate the steps so the interviewer can follow your thinking.

Scenario221 · The NULL = NULL Trap

Tags: Difficulty: Easy · Asked at: Flipkart, Razorpay, Swiggy, Zoho · Dialect: vendor-agnostic

The setup:

```
-- Table data
CREATE TABLE customers (id INT, phone VARCHAR(15));
INSERT INTO customers VALUES
  (1, '9876543210'),
  (2, NULL),
  (3, NULL),
  (4, '9123456789');

-- The query
SELECT id FROM customers WHERE phone = NULL;
```

The question: "What does this query return? Walk me through your reasoning."

Thinking out loud: 1. "At first glance you might think rows 2 and 3 match because their phone is NULL." 2. "But SQL uses three-valued logic — NULL is not a value, it represents 'unknown'." 3. "Any comparison with NULL using `=` returns NULL, not TRUE. The WHERE clause keeps only rows where the predicate is TRUE." 4. "So even `NULL = NULL` evaluates to NULL, which is filtered out. The result is zero rows."

The answer (with output):

```
id
----
(0 rows)
```


Why this output: The expression `phone = NULL` evaluates to NULL for every row, and WHERE discards rows where the predicate is not TRUE. The correct form is `WHERE phone IS NULL`.

The trap: Candidates assume NULL behaves like a normal value and that equality works the same way as on integers or strings.

What NOT to say: - "It returns rows 2 and 3 because their phone column is NULL." - "It will throw an error because you cannot compare to NULL."

Common follow-up: "How would you rewrite this to actually return the rows with missing phone numbers?"

Scenario222 · The NULL Inequality

Tags: Difficulty: Easy · Asked at: Paytm, Zomato, Freshworks · Dialect: vendor-agnostic

The setup:

```
-- Table data
CREATE TABLE orders (id INT, status VARCHAR(20));
INSERT INTO orders VALUES
  (101, 'shipped'),
  (102, 'cancelled'),
  (103, NULL),
  (104, 'shipped');

-- The query
SELECT id FROM orders WHERE status <> 'cancelled';
```

The question: "What does this query return? Walk me through your reasoning."

Thinking out loud: 1. "Looks straightforward — give me every order that is not cancelled." 2. "Row 101 has status 'shipped', which is not equal to 'cancelled', so the predicate is TRUE." 3. "Row 102 is excluded. Row 104 is included by the same logic as 101." 4. "Now row 103 — status is NULL. `NULL <> 'cancelled'` evaluates to NULL, not TRUE, so it is filtered out."

The answer (with output):

```
id
----
101
104
```

Why this output: NULL is excluded from inequality predicates just as it is from equality predicates — both evaluate to NULL, which is not TRUE.

The trap: Candidates expect "not cancelled" to mean "everything except cancelled" and include the NULL row.

What NOT to say: - "It returns 101, 103, 104 because all three are not cancelled."

Common follow-up: *"What if the requirement is to include rows where status is missing — how would you write that?"*

Scenario223 · IS NULL vs Equals NULL

Tags: Difficulty: Easy · Asked at: Infosys, TCS, Wipro, Cognizant · Dialect: vendor-agnostic

The setup:

```
-- Table data
CREATE TABLE employees (emp_id INT, manager_id INT);
INSERT INTO employees VALUES
  (1, NULL),
  (2, 1),
  (3, 1),
  (4, NULL);

-- The query
SELECT emp_id FROM employees WHERE manager_id IS NULL;
```

The question: *"What does this query return, and how does it differ from WHERE manager_id = NULL ?"*

Thinking out loud: 1. "This uses `IS NULL`, which is the proper way to test for NULL — it returns TRUE or FALSE, never NULL." 2. "Rows 1 and 4 have NULL manager_id, so `IS NULL` returns TRUE for them." 3. "Rows 2 and 3 have a manager, so `IS NULL` returns FALSE — they are filtered out." 4. "The result is emp_id 1 and 4. With `= NULL` instead, the result would be zero rows."

The answer (with output):

```
emp_id
-----
1
4
```

Why this output: `IS NULL` is a special operator built for NULL detection — it always evaluates to a boolean. `= NULL` always evaluates to NULL.

The trap: Some candidates know `IS NULL` is right but cannot articulate why `= NULL` is wrong.

What NOT to say: - "Both forms work, but `IS NULL` is just the preferred style."

Common follow-up: "In SQL Server, what does `SET ANSI_NULLS OFF` do, and why should you never rely on it?"

Scenario224 · COALESCE Walks the Chain

Tags: Difficulty: Easy · Asked at: Razorpay, PhonePe, Cred · Dialect: vendor-agnostic

The setup:

```
-- Table data
CREATE TABLE contact (user_id INT, work_phone VARCHAR(15), home_phone VARCHAR(15), mobile VARCHAR(15))
INSERT INTO contact VALUES
  (1, NULL, NULL, '9876500001'),
  (2, '02212345678', NULL, '9876500002'),
  (3, NULL, '04412345678', NULL),
  (4, NULL, NULL, NULL);

-- The query
SELECT user_id, COALESCE(work_phone, home_phone, mobile, 'No contact') AS reach_at
FROM contact;
```

The question: "What does this query return for each row?"

Thinking out loud: 1. "COALESCE returns the first non-NULL argument from left to right." 2. "User 1: work is NULL, home is NULL, mobile is '9876500001' — that wins." 3. "User 2: work is '02212345678' — first non-NULL, that wins, the rest are not even checked." 4. "User 3: work NULL, home '04412345678' wins. User 4: all NULL, falls back to literal 'No contact'."

The answer (with output):

user_id	reach_at
1	9876500001
2	02212345678
3	04412345678
4	No contact

Why this output: COALESCE evaluates its arguments left to right and short-circuits at the first non-NULL value. A literal at the end provides a safe default.

The trap: Candidates sometimes think COALESCE returns the last non-NULL value, or returns NULL if any argument is NULL.

What NOT to say: - "It returns the last non-NULL value in the list."

Common follow-up: "What is the difference between COALESCE and ISNULL in SQL Server?"

Scenario225 · NULLIF Flips the Script

Tags: Difficulty: Medium · Asked at: Zoho, Freshworks, Postman · Dialect: vendor-agnostic

The setup:

```
-- Table data
CREATE TABLE survey (resp_id INT, rating INT);
INSERT INTO survey VALUES
  (1, 5),
  (2, 0),
  (3, 3),
  (4, 0),
  (5, 4);

-- The query
SELECT resp_id, NULLIF(rating, 0) AS valid_rating
FROM survey;
```

The question: "What does NULLIF do here, and what does the result look like?"

Thinking out loud: 1. "NULLIF(a, b) returns NULL when a = b, otherwise it returns a." 2. "It is the inverse of COALESCE in a sense — used to convert sentinel values like 0 or '' into proper NULLs." 3. "Row 1: rating 5, not equal to 0, returns 5. Rows 2 and 4: rating 0 equals 0, returns NULL." 4. "Rows 3 and 5: ratings 3 and 4, returned as-is."

The answer (with output):

resp_id	valid_rating
1	5
2	NULL
3	3
4	NULL
5	4

Why this output: NULLIF is the standard way to convert sentinel placeholders (0, -1, '') into proper NULLs so that downstream aggregations like AVG ignore them.

The trap: Candidates confuse NULLIF with COALESCE or think NULLIF returns 0 when the values match.

What NOT to say: - "NULLIF replaces NULLs with the second argument."

Common follow-up: "If you AVG over the valid_rating column above, what does the engine return?"

Scenario226 · IS DISTINCT FROM Saves the Day

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Setu · Dialect: Postgres

The setup:

```
-- Table data
CREATE TABLE audit_log (record_id INT, old_status VARCHAR(20), new_status VARCHAR(20));
INSERT INTO audit_log VALUES
  (1, 'pending', 'shipped'),
  (2, 'pending', 'pending'),
  (3, NULL, 'shipped'),
  (4, 'shipped', NULL),
  (5, NULL, NULL);

-- The query
SELECT record_id FROM audit_log WHERE old_status IS DISTINCT FROM new_status;
```

The question: "What does *IS DISTINCT FROM* return, and how is it different from `<>`?"

Thinking out loud: 1. "*IS DISTINCT FROM* treats two NULLs as equal — it returns *FALSE* if both sides are NULL." 2. "It returns *TRUE* if the values differ, including the case where one side is NULL and the other is not." 3. "Row 1: pending vs shipped — distinct, included. Row 2: pending vs pending — not distinct, excluded." 4. "Row 3: NULL vs shipped — distinct, included. Row 4: shipped vs NULL — distinct, included. Row 5: NULL vs NULL — not distinct, excluded."

The answer (with output):

```
record_id
-----
1
3
4
```

Why this output: *IS DISTINCT FROM* is null-safe equality — it gives the answer people usually intend when comparing two columns that may have NULLs.

The trap: With plain `<>`, rows 3, 4 and 5 would all be excluded because NULL comparisons yield NULL.

What NOT to say: - "*IS DISTINCT FROM* is the same as `<>` but newer."

Common follow-up: "How do you express the same logic in MySQL, which does not support *IS DISTINCT FROM*?"

Scenario227 · COUNT Star vs COUNT Column

Tags: Difficulty: Easy · Asked at: Flipkart, Myntra, Nykaa, Meesho · Dialect: vendor-agnostic

The setup:

```
-- Table data
CREATE TABLE deliveries (id INT, delivered_at TIMESTAMP);
INSERT INTO deliveries VALUES
  (1, '2026-05-01 10:00'),
  (2, NULL),
  (3, '2026-05-02 14:00'),
  (4, NULL),
  (5, '2026-05-03 09:00');

-- The query
SELECT COUNT(*) AS total, COUNT(delivered_at) AS delivered FROM deliveries;
```

The question: "What is the difference between these two counts, and what do they return?"

Thinking out loud: 1. "COUNT() counts rows, regardless of whether any column is NULL." 2.

"COUNT(column) counts only the rows where that specific column is non-NULL." 3. "Five rows total, so COUNT() is 5." 4. "Three rows have a non-NULL delivered_at, so COUNT(delivered_at) is 3."

The answer (with output):

```
total | delivered
-----+-----
5      | 3
```

Why this output: COUNT(*) operates on the row itself. COUNT(expr) operates on the expression and ignores NULLs — that is the only aggregate that treats * differently from a column name.

The trap: Candidates assume both produce the same answer, especially on tables they have not inspected for NULLs.

What NOT to say: - "They are the same — * is just shorthand."

Common follow-up: "What about COUNT(DISTINCT delivered_at) — what would that return here?"

Scenario228 · SUM Over All-NULL

Tags: Difficulty: Medium · Asked at: Paytm, Zomato, Swiggy · Dialect: vendor-agnostic

The setup:

```
-- Table data
CREATE TABLE refunds (order_id INT, refund_amount DECIMAL(10,2));
INSERT INTO refunds VALUES
  (1001, NULL),
  (1002, NULL),
  (1003, NULL);

-- The query
SELECT SUM(refund_amount) AS total_refund FROM refunds;
```

The question: "Three rows, all NULL refund_amount — what does SUM return?"

Thinking out loud: 1. "SUM ignores NULLs when adding values together." 2. "But here every value is NULL, so there is nothing to add." 3. "A common assumption is that SUM returns 0 when the input is empty or all-NULL — but the SQL standard says it returns NULL." 4. "So the answer is one row, with NULL in the total_refund column."

The answer (with output):

```
total_refund
-----
NULL
```

Why this output: SUM over an empty set or all-NULL input returns NULL, not 0. This is consistent across all major engines.

The trap: Candidates write `WHERE total_refund > 1000` expecting it to handle a NULL gracefully, then are surprised when downstream code breaks.

What NOT to say: - "It returns 0 because adding NULLs gives 0."

Common follow-up: "How would you guarantee that this returns 0 instead of NULL?"

Scenario229 · AVG With NULLs in the Mix

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Groww · Dialect: vendor-agnostic

The setup:

```
-- Table data
CREATE TABLE quiz_scores (student_id INT, score INT);
INSERT INTO quiz_scores VALUES
  (1, 80),
  (2, NULL),
  (3, 60),
  (4, NULL),
  (5, 100);

-- The query
SELECT AVG(score) AS avg_score FROM quiz_scores;
```

The question: "What does AVG return, and is the divisor 5 or 3?"

Thinking out loud: 1. "AVG is essentially SUM divided by COUNT — and crucially, both ignore NULLs." 2. "The non-NULL values are 80, 60 and 100. SUM is 240. COUNT is 3." 3. "240 divided by 3 is 80, not 240 divided by 5 which would be 48." 4. "So the average is 80."

The answer (with output):

```
avg_score
-----
80
```

Why this output: $AVG = \text{SUM}(\text{non-NULL}) / \text{COUNT}(\text{non-NULL})$. NULL rows do not contribute to either numerator or denominator.

The trap: Candidates assume NULLs are treated as 0 for averaging, dragging the average down.

What NOT to say: - "It is 48, because 240 divided by 5 students."

Common follow-up: "If the business wants to treat missing scores as 0, how would you compute that average?"

Scenario230 · CASE Without ELSE

Tags: Difficulty: Easy · Asked at: Infosys, Accenture, Capgemini · Dialect: vendor-agnostic

The setup:


```
-- Table data
CREATE TABLE products (id INT, price DECIMAL(10,2));
INSERT INTO products VALUES
  (1, 999),
  (2, 200),
  (3, 5000),
  (4, 50);

-- The query
SELECT id, price,
  CASE
    WHEN price >= 1000 THEN 'premium'
    WHEN price >= 500 THEN 'mid'
  END AS tier
FROM products;
```

The question: "What does the tier column show for each row?"

Thinking out loud: 1. "CASE evaluates branches top to bottom and returns the first one that matches." 2. "Row 1, price 999: not >= 1000, but >= 500, so 'mid'." 3. "Row 2, price 200: neither branch matches. Without an ELSE, CASE returns NULL." 4. "Row 3 is 'premium', row 4 is NULL for the same reason as row 2."

The answer (with output):

```
id | price | tier
---+-----+-----
1  | 999   | mid
2  | 200   | NULL
3  | 5000  | premium
4  | 50    | NULL
```

Why this output: A CASE expression with no ELSE clause returns NULL whenever no WHEN branch matches — there is an implicit `ELSE NULL`.

The trap: Candidates miss the silent NULL and assume an empty string or 0 will appear instead.

What NOT to say: - "Rows 2 and 4 return an empty string because no branch matched."

Common follow-up: "How would you label everything below 500 as 'budget' instead of NULL?"

Scenario231 · CASE And Three-Valued Logic

Tags: Difficulty: Medium · Asked at: Cred, Setu, Razorpay · Dialect: vendor-agnostic

The setup:

```
-- Table data
CREATE TABLE accounts (id INT, balance DECIMAL(10,2));
INSERT INTO accounts VALUES
  (1, 5000),
  (2, NULL),
  (3, 0),
  (4, -100);

-- The query
SELECT id, balance,
  CASE
    WHEN balance > 0 THEN 'positive'
    WHEN balance < 0 THEN 'negative'
    WHEN balance = 0 THEN 'zero'
    ELSE 'unknown'
  END AS status
FROM accounts;
```

The question: "What status does each row get? Pay attention to row 2."

Thinking out loud: 1. "Row 1: balance 5000, first branch matches, 'positive'." 2. "Row 2: balance NULL. `NULL > 0` is NULL, `NULL < 0` is NULL, `NULL = 0` is NULL — none are TRUE." 3. "Since no WHEN branch is TRUE, control falls to ELSE, returning 'unknown'." 4. "Row 3: 'zero'. Row 4: 'negative'."

The answer (with output):

id	balance	status
1	5000	positive
2	NULL	unknown
3	0	zero
4	-100	negative

Why this output: All comparisons against NULL evaluate to NULL — never TRUE — so a NULL value will fall through every WHEN branch and only the ELSE catches it.

The trap: Candidates assume `balance = 0` would catch NULL because both are "missing or zero".

What NOT to say: - "Row 2 returns 'zero' because NULL behaves like 0 in arithmetic."

Common follow-up: "How would you write a branch that specifically catches the NULL case?"

Scenario232 · ORDER BY With NULLs

Tags: Difficulty: Medium · Asked at: Flipkart, Myntra, Zomato · Dialect: Postgres / MySQL

The setup:

```
-- Table data
CREATE TABLE listings (id INT, sold_at TIMESTAMP);
INSERT INTO listings VALUES
  (1, '2026-05-01'),
  (2, NULL),
  (3, '2026-05-10'),
  (4, NULL),
  (5, '2026-05-05');

-- The query (Postgres)
SELECT id FROM listings ORDER BY sold_at ASC;
```

The question: "In what order do these rows come back in Postgres, and would MySQL behave the same way?"

Thinking out loud: 1. "ORDER BY puts the smallest values first by default. NULLs are not really comparable to dates, so the engine has to pick a position for them." 2. "Postgres default is **NULLS LAST** for ASC, **NULLS FIRST** for DESC." 3. "MySQL default is the opposite — **NULLS FIRST** for ASC, **NULLS LAST** for DESC." 4. "On Postgres, the order is row 1, 5, 3, then the two NULLs (2 and 4). On MySQL, the two NULLs would come first."

The answer (with output):

```
-- Postgres
id
----
1
5
3
2
4
```

Why this output: Each engine has a default position for NULLs in ORDER BY, and that default varies — Postgres puts them last on ASC, MySQL puts them first.

The trap: Candidates assume ORDER BY is fully deterministic across engines and write portable code that silently rearranges.

What NOT to say: - "NULLs always go to the bottom in every database."

Common follow-up: "How do you make ordering NULL-position explicit across engines?"

Scenario233 · ORDER BY DESC NULLS LAST

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman · Dialect: Postgres / Oracle

The setup:

```
-- Table data
CREATE TABLE leaderboard (player VARCHAR(20), score INT);
INSERT INTO leaderboard VALUES
  ('Aman', 850),
  ('Priya', NULL),
  ('Karthik', 920),
  ('Sneha', NULL),
  ('Rohit', 700);

-- The query
SELECT player, score FROM leaderboard ORDER BY score DESC NULLS LAST;
```

The question: "What is the row order?"

Thinking out loud: 1. "DESC sorts highest first. Karthik 920, Aman 850, Rohit 700 — those are the non-NULLs." 2. " **NULLS LAST** explicitly forces NULL rows to the bottom regardless of direction." 3. "So Priya and Sneha end up at the end. Their relative order is engine-defined unless we add a tiebreaker." 4. "Final order: Karthik, Aman, Rohit, then Priya and Sneha (in some order)."

The answer (with output):

```
player | score
-----+-----
Karthik | 920
Aman    | 850
Rohit   | 700
Priya   | NULL
Sneha   | NULL
```

Why this output: **NULLS LAST** overrides the engine default. On Postgres, DESC defaults to NULLS FIRST, so without this clause Priya and Sneha would appear at the top.

The trap: Candidates forget that MySQL versions before 8.0 do not support the **NULLS LAST** syntax at all.

What NOT to say: - "DESC always puts NULL at the bottom because NULL is less than every number."

Common follow-up: "How do you emulate **NULLS LAST** in MySQL or in older SQL Server versions?"

Scenario234 · Integer Compared to String

Tags: Difficulty: Medium · Asked at: Freshworks, Zoho, BrowserStack · Dialect: MySQL / SQL Server

The setup:

```
-- Table data
CREATE TABLE invoice (id INT, amount_str VARCHAR(20));
INSERT INTO invoice VALUES
  (1, '500'),
  (2, '500abc'),
  (3, 'abc500'),
  (4, '500.99');

-- The query (MySQL)
SELECT id FROM invoice WHERE amount_str = 500;
```

The question: "In MySQL, which rows match?"

Thinking out loud: 1. "The right side is an integer 500, the left side is a string column. MySQL will implicitly coerce the string to a number." 2. "MySQL string-to-number coercion reads digits from the left and stops at the first non-digit." 3. "'500' becomes 500 — matches. '500abc' also becomes 500 with a warning — matches. 'abc500' becomes 0 — does not match. '500.99' becomes 500.99 — does not equal 500." 4. "So rows 1 and 2 match."

The answer (with output):

```
id
----
1
2
```

Why this output: MySQL performs lossy string-to-number coercion that reads leading digits and silently discards the rest. Other engines like SQL Server may throw a conversion error or use stricter parsing.

The trap: Candidates expect only the exact text '500' to match, or expect an error.

What NOT to say: - "MySQL will throw a type mismatch error."

Common follow-up: "What does SQL Server do for the same query, and why is that arguably safer?"

Scenario235 · String Compared to Number

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Wipro · Dialect: SQL Server / Postgres

The setup:

```
-- Table data
CREATE TABLE products (id INT, sku VARCHAR(20));
INSERT INTO products VALUES
  (1, '100'),
  (2, '0100'),
  (3, '100 '),
  (4, '00100');

-- The query (SQL Server)
SELECT id FROM products WHERE sku = 100;
```

The question: "What is the behaviour in SQL Server?"

Thinking out loud: 1. "SQL Server has a precedence rule — varchar has lower precedence than int, so it will convert the varchar to int." 2. "'100' converts cleanly to 100 — matches." 3. "'0100' also parses as 100, trailing/leading zeros are dropped on numeric conversion — matches." 4. "'100 ' with trailing space — SQL Server tolerates trailing whitespace for numeric conversion, also matches. '00100' — same, matches."

The answer (with output):

```
id
----
1
2
3
4
```

Why this output: SQL Server converts the varchar to int because int has higher data type precedence. Numeric parsing collapses leading zeros and whitespace.

The trap: Postgres would raise an error here unless you cast explicitly — engine behaviour is not portable.

What NOT to say: - "Only the exact match '100' is returned because it is a string comparison."

Common follow-up: "How would Postgres handle this query, and how should you write a portable version?"

Scenario236 · CHAR vs VARCHAR Trailing Space

Tags: Difficulty: Hard · Asked at: Oracle India, Salesforce India, IBM India · Dialect: vendor-agnostic

The setup:

```

-- Table data
CREATE TABLE products_char (id INT, code CHAR(5));
CREATE TABLE products_varchar (id INT, code VARCHAR(5));

INSERT INTO products_char VALUES (1, 'ABC');
INSERT INTO products_varchar VALUES (1, 'ABC');

-- The query (run against both)
SELECT id FROM products_char WHERE code = 'ABC  ';      -- trailing spaces
SELECT id FROM products_varchar WHERE code = 'ABC  ';    -- trailing spaces

```

The question: "Both stored values are 'ABC'. Which queries return the row?"

Thinking out loud: 1. "CHAR is fixed length — 'ABC' is stored as 'ABC ' with two trailing spaces." 2. "The ANSI rule for CHAR comparison: the shorter string is padded with spaces before comparison. So 'ABC' vs 'ABC ' is treated as equal." 3. "VARCHAR stores 'ABC' as 'ABC' — no padding. But the comparison still pads the shorter side to match length in most engines." 4. "Different engines treat trailing spaces in VARCHAR comparison differently — Postgres preserves them strictly for VARCHAR, MySQL ignores them, SQL Server pads on comparison."

The answer (with output):

```

-- products_char (both engines):
id
----
1

-- products_varchar (Postgres):
id
----
(0 rows)

-- products_varchar (MySQL/SQL Server):
id
----
1

```

Why this output: CHAR is space-padded by storage and ANSI defines space-padded comparison. VARCHAR behaviour diverges — Postgres treats trailing spaces as significant, while MySQL and SQL Server typically pad on comparison.

The trap: Code that works on one engine silently breaks on migration, especially Postgres.

What NOT to say: - "All engines ignore trailing whitespace in string comparisons."

Common follow-up: "How do you defensively compare strings where trailing spaces should not matter, across engines?"

Scenario237 · Date String Parsing Surprise

Tags: Difficulty: Medium · Asked at: Flipkart, Zomato, Ola · Dialect: MySQL / SQL Server

The setup:

```
-- Table data
CREATE TABLE bookings (id INT, booking_date DATE);
INSERT INTO bookings VALUES
  (1, '2026-05-10'),
  (2, '2026-05-11'),
  (3, '2026-05-12');

-- The query (SQL Server, default region en-US)
SELECT id FROM bookings WHERE booking_date = '10/05/2026';
```

The question: "This is an Indian developer expecting `dd/mm/yyyy` — 10 May 2026. What does SQL Server actually return?"

Thinking out loud: 1. "In en-US default locale, SQL Server reads `mm/dd/yyyy` from a slash-separated literal." 2. "So '10/05/2026' is parsed as October 5, 2026 — not May 10, 2026." 3. "None of our stored dates are October 5, 2026, so the query returns zero rows." 4. "In MySQL, the engine is stricter — typically requires `yyyy-mm-dd` and would error or return zero rows."

The answer (with output):

```
id
----
(0 rows)
```

Why this output: String-to-date parsing depends on the engine and session locale. The ANSI-safe form is the ISO 8601 literal `'yyyy-mm-dd'`, which is unambiguous everywhere.

The trap: Indian developers default to `dd/mm/yyyy` mentally — many production bugs come from this exact confusion.

What NOT to say: - "It returns the May 10 row because `10/05` is obviously day 10, month 5."

Common follow-up: "What is the safest date literal format to use across engines and locales?"

Scenario238 · DATE vs TIMESTAMP

Tags: Difficulty: Medium · Asked at: Razorpay, Setu, Cred · Dialect: Postgres

The setup:

```
-- Table data
CREATE TABLE events (id INT, ts TIMESTAMP);
INSERT INTO events VALUES
  (1, '2026-05-10 00:00:00'),
  (2, '2026-05-10 09:30:00'),
  (3, '2026-05-10 23:59:59');

-- The query
SELECT id FROM events WHERE ts = DATE '2026-05-10';
```

The question: "Which rows match?"

Thinking out loud: 1. "The literal `DATE '2026-05-10'` is a date — it has no time component." 2. "When compared to a `TIMESTAMP`, the engine promotes the date to a timestamp at midnight: `2026-05-10 00:00:00`." 3. "So only rows with exactly midnight match — row 1." 4. "Rows 2 and 3 are on the same calendar day, but their time portion is non-zero, so they do not equal midnight."

The answer (with output):

```
id
----
1
```

Why this output: A `DATE` compared to a `TIMESTAMP` coerces to midnight. To match a full day, use a range predicate: `ts >= DATE '2026-05-10' AND ts < DATE '2026-05-11'`.

The trap: Candidates write `WHERE ts = current_date` and miss every row whose timestamp is not exactly midnight.

What NOT to say: - "All three rows match because they are on May 10."

Common follow-up: "How do you efficiently query all events from a specific day on a `TIMESTAMP` column that has an index?"

Scenario239 · Timezone Aware vs Naive

Tags: Difficulty: Hard · Asked at: Razorpay, Postman, Cred, Setu · Dialect: Postgres

The setup:

```
-- Session timezone
SET TIME ZONE 'Asia/Kolkata';

-- Table data
CREATE TABLE logins (id INT, login_at TIMESTAMPTZ);
INSERT INTO logins VALUES
  (1, '2026-05-10 09:00:00+05:30'),
  (2, '2026-05-10 03:30:00+00:00'),
  (3, '2026-05-10 23:30:00+05:30');

-- The query
SELECT id, login_at FROM logins
WHERE login_at = TIMESTAMPTZ '2026-05-10 09:00:00+05:30';
```

The question: "What does this return?"

Thinking out loud: 1. "TIMESTAMPTZ stores values as UTC internally. All three inserts get normalised to UTC." 2. "Row 1 is `2026-05-10 09:00 IST` which is `2026-05-10 03:30 UTC`." 3. "Row 2 is explicitly `2026-05-10 03:30 UTC` — the same point in time as row 1." 4. "The literal in the WHERE clause is also `2026-05-10 03:30 UTC` — so rows 1 and 2 both match. Row 3 is 18:00 UTC, no match."

The answer (with output):

```
id | login_at (displayed in Asia/Kolkata)
---+-----
1  | 2026-05-10 09:00:00+05:30
2  | 2026-05-10 09:00:00+05:30
```

Why this output: TIMESTAMPTZ compares the underlying UTC instant, regardless of how it was inserted or displayed. Two different-looking timestamps can represent the same point in time.

The trap: Candidates assume the textual form is what is stored and miss equality across timezones.

What NOT to say: - "Only row 1 matches because that is the exact text we inserted."

Common follow-up: "What is the difference between `TIMESTAMP` and `TIMESTAMPTZ`, and which should you use for audit columns?"

Scenario240 · BETWEEN Is Inclusive

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Accenture, HCL · Dialect: vendor-agnostic

The setup:

```
-- Table data
CREATE TABLE marks (student_id INT, percentage INT);
INSERT INTO marks VALUES
  (1, 60),
  (2, 75),
  (3, 80),
  (4, 90),
  (5, 100);

-- The query
SELECT student_id FROM marks WHERE percentage BETWEEN 75 AND 90;
```

The question: "Which rows match?"

Thinking out loud: 1. "BETWEEN is inclusive on both ends — equivalent to `percentage >= 75 AND percentage <= 90` ." 2. "Row 1 percentage 60 — no. Row 2 percentage 75 — yes, lower bound included." 3. "Row 3 percentage 80 — yes. Row 4 percentage 90 — yes, upper bound included." 4. "Row 5 percentage 100 — no, above the range."

The answer (with output):

```
student_id
-----
2
3
4
```

Why this output: BETWEEN x AND y is shorthand for `>= x AND <= y` — both endpoints are part of the matched set.

The trap: Candidates think one or both endpoints are exclusive, especially those coming from Python range semantics.

What NOT to say: - "Only row 3 matches because BETWEEN excludes the boundaries."

Common follow-up: "What would `WHERE percentage NOT BETWEEN 75 AND 90` return for the same table?"

Scenario241 · IN With NULL in the List

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Freshworks · Dialect: vendor-agnostic

The setup:

```
-- Table data
CREATE TABLE orders (id INT, status VARCHAR(20));
INSERT INTO orders VALUES
  (1, 'shipped'),
  (2, 'pending'),
  (3, 'cancelled'),
  (4, NULL);

-- The query
SELECT id FROM orders WHERE status IN ('shipped', 'pending', NULL);
```

The question: "Does row 4 (status NULL) match because NULL is in the list?"

Thinking out loud: 1. "`x IN (a, b, c)` is sugar for `x = a OR x = b OR x = c`." 2. "For row 4, status = NULL. That expands to `NULL = 'shipped' OR NULL = 'pending' OR NULL = NULL` — all three evaluate to NULL." 3. "NULL OR NULL OR NULL is NULL, not TRUE. The row is filtered out." 4. "Rows 1 and 2 match normally. Row 3 status 'cancelled' is not in the list."

The answer (with output):

```
id
----
1
2
```

Why this output: Adding NULL to an IN list does nothing — every comparison involving NULL yields NULL, never TRUE. Use `IS NULL` separately if you want to match NULL rows.

The trap: Candidates add NULL to the IN list expecting it to cover NULL rows.

What NOT to say: - "Row 4 matches because NULL is included in the list."

Common follow-up: "How would you correctly include NULL rows along with 'shipped' and 'pending'?"

Scenario242 · NOT IN With NULL

Tags: Difficulty: Hard · Asked at: Flipkart, Zomato, Swiggy, Paytm · Dialect: vendor-agnostic

The setup:

```
-- Table data
CREATE TABLE products (id INT);
INSERT INTO products VALUES (1), (2), (3), (4), (5);

CREATE TABLE blocked (product_id INT);
INSERT INTO blocked VALUES (2), (4), (NULL);

-- The query
SELECT id FROM products WHERE id NOT IN (SELECT product_id FROM blocked);
```

The question: "What does this return?"

Thinking out loud: 1. "`id NOT IN (2, 4, NULL)`" expands to "`id <> 2 AND id <> 4 AND id <> NULL`". 2. "That last predicate, `id <> NULL`, evaluates to `NULL` for every row." 3. "`TRUE AND TRUE AND NULL` evaluates to `NULL`, not `TRUE`." 4. "So no row produces a `TRUE` result — the query returns zero rows."

The answer (with output):

```
id
----
(0 rows)
```

Why this output: `NOT IN` with a `NULL` in the right-hand side is a notorious trap — the moment one `NULL` appears, the entire predicate becomes "not knowable" for every row.

The trap: This is the single most common SQL bug in production — a child table with a `NULL` silently makes the parent `NOT IN` return nothing.

What NOT to say: - "It returns 1, 3, 5 because those are the products not blocked."

Common follow-up: "How do you rewrite this safely so a `NULL` in the blocked table does not nuke the result?"

Scenario243 · EXISTS Always True

Tags: Difficulty: Medium · Asked at: Cred, Razorpay, Postman · Dialect: vendor-agnostic

The setup:

```
-- Table data
CREATE TABLE users (id INT, name VARCHAR(30));
INSERT INTO users VALUES (1, 'Aman'), (2, 'Priya'), (3, 'Karthik');

CREATE TABLE flags (flag VARCHAR(20));
INSERT INTO flags VALUES ('active');

-- The query
SELECT id FROM users WHERE EXISTS (SELECT 1 FROM flags);
```

The question: "What does *EXISTS* return for each user?"

Thinking out loud: 1. "*EXISTS* returns *TRUE* if the subquery produces at least one row, *FALSE* otherwise." 2. "The subquery is not correlated — it just selects from *flags*, which has one row." 3. "So *EXISTS* is *TRUE* for every outer row, regardless of the user." 4. "Result: all three users come back."

The answer (with output):

```
id
----
1
2
3
```

Why this output: *EXISTS* only cares whether the subquery yields rows. An uncorrelated *EXISTS* with non-empty input is effectively a constant *TRUE* filter.

The trap: Candidates assume *EXISTS* implies a correlation and try to find a join condition that is not there.

What NOT to say: - "It returns no rows because there is no join."

Common follow-up: "How would the result change if the *flags* table were empty?"

Scenario244 · DISTINCT Drops Ordering

Tags: Difficulty: Medium · Asked at: Flipkart, Myntra, Swiggy · Dialect: vendor-agnostic

The setup:

```
-- Table data
CREATE TABLE orders (city VARCHAR(20), amount INT);
INSERT INTO orders VALUES
  ('Mumbai', 500),
  ('Delhi', 300),
  ('Mumbai', 700),
  ('Bengaluru', 200),
  ('Delhi', 900);

-- The query
SELECT DISTINCT city FROM orders;
```

The question: *"In what order do the cities come back?"*

Thinking out loud: 1. *"DISTINCT removes duplicates — we get Mumbai, Delhi, Bengaluru once each."* 2. *"But the order is not guaranteed. DISTINCT does not imply ORDER BY."* 3. *"In Postgres or SQL Server, the order depends on the hash or sort plan the optimiser picks — could be anything."* 4. *"If a consistent order matters, add an explicit ORDER BY clause."*

The answer (with output):

```
city
-----
Bengaluru
Delhi
Mumbai
-- or any other order; engine-defined
```

Why this output: DISTINCT is a set operation, not an ordering operation. Many engines implement it via hashing or sorting, which incidentally orders results — but you cannot rely on it.

The trap: Candidates assume DISTINCT returns rows in insertion order or alphabetic order.

What NOT to say: - *"It always comes back sorted because DISTINCT sorts internally."*

Common follow-up: *"How do you guarantee an alphabetical order on a DISTINCT query?"*

Scenario245 · LIMIT Without ORDER BY

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Setu, Zomato · Dialect: vendor-agnostic

The setup:

```
-- Table data
CREATE TABLE transactions (id INT, amount INT);
INSERT INTO transactions VALUES
  (101, 500),
  (102, 700),
  (103, 200),
  (104, 900),
  (105, 100);

-- The query
SELECT id FROM transactions LIMIT 3;
```

The question: *"Which three IDs come back?"*

Thinking out loud: 1. *"LIMIT 3 returns three rows — but which three?"* 2. *"Without ORDER BY, the engine is free to return any three rows it likes — usually whichever ones it finds first based on storage layout, indexes or parallel scan order."* 3. *"On the same data, the same query can return different rows after a vacuum, an index rebuild, or a schema change."* 4. *"The honest answer: it returns three rows, but the choice is non-deterministic."*

The answer (with output):

```
id
----
-- any three of: 101, 102, 103, 104, 105
-- depending on storage and execution plan
```

Why this output: SQL does not define a row order unless you specify ORDER BY. LIMIT slices an unordered set, so it produces undefined-but-deterministic-for-this-run results.

The trap: Candidates assume insertion order, primary-key order, or visual order in their last SELECT.

What NOT to say: - *"It returns 101, 102, 103 because those were inserted first."*

Common follow-up: *"How do you write a deterministic 'first three' query?"*

Scenario246 · UNION vs UNION ALL

Tags: Difficulty: Easy · Asked at: Infosys, TCS, Cognizant, Wipro · Dialect: vendor-agnostic

The setup:


```
-- Table data
CREATE TABLE region_north (city VARCHAR(20));
INSERT INTO region_north VALUES ('Delhi'), ('Lucknow'), ('Jaipur');

CREATE TABLE region_west (city VARCHAR(20));
INSERT INTO region_west VALUES ('Mumbai'), ('Jaipur'), ('Pune');

-- The query
SELECT city FROM region_north UNION SELECT city FROM region_west;
-- vs
SELECT city FROM region_north UNION ALL SELECT city FROM region_west;
```

The question: "How many rows does each query return?"

Thinking out loud: 1. "UNION combines both sets and removes duplicates." 2. "North has Delhi, Lucknow, Jaipur. West has Mumbai, Jaipur, Pune. Six rows total." 3. "Jaipur is in both, so UNION dedupes to five rows." 4. "UNION ALL keeps duplicates — all six rows are returned."

The answer (with output):

```
-- UNION (5 rows)
city
-----
Delhi
Lucknow
Jaipur
Mumbai
Pune

-- UNION ALL (6 rows)
city
-----
Delhi
Lucknow
Jaipur
Mumbai
Jaipur
Pune
```

Why this output: UNION applies an implicit DISTINCT across the combined result set — more expensive at scale. UNION ALL concatenates without deduplication.

The trap: Candidates use UNION when they mean UNION ALL and incur a sort or hash cost they did not need.

What NOT to say: - "UNION and UNION ALL are equivalent except for syntax."

Common follow-up: "You have a query that runs slowly. Replacing UNION with UNION ALL — when is this safe?"

Scenario247 · INTERSECT With Duplicates

Tags: Difficulty: Medium · Asked at: Oracle India, Postman, Setu · Dialect: Postgres / SQL Server / Oracle

The setup:

```
-- Table data
CREATE TABLE list_a (val INT);
INSERT INTO list_a VALUES (1), (2), (2), (3);

CREATE TABLE list_b (val INT);
INSERT INTO list_b VALUES (2), (2), (2), (4);

-- The query
SELECT val FROM list_a INTERSECT SELECT val FROM list_b;
```

The question: *"How many 2s does the result contain?"*

Thinking out loud: 1. "INTERSECT returns rows present in both sets, with duplicates removed by default." 2. "Both sides have the value 2 — so 2 is in the intersection." 3. "But INTERSECT dedupes, so only one 2 comes back regardless of how many duplicates either side had." 4. "1 is only in A, 3 is only in A, 4 is only in B — none survive. Result: one row, value 2."

The answer (with output):

```
val
----
2
```

Why this output: INTERSECT, like UNION, applies set semantics — duplicates are removed. INTERSECT ALL (Postgres, Oracle) preserves duplicate counts and would return two 2s here (the minimum of A's count and B's count).

The trap: Candidates expect duplicate preservation and are surprised that the multiplicity disappears.

What NOT to say: - "It returns three 2s because B has three 2s."

Common follow-up: *"What does INTERSECT ALL return here, and which engines support it?"*

Scenario248 · EXCEPT Semantics

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Freshworks · Dialect: Postgres / SQL Server

The setup:

```
-- Table data
CREATE TABLE active_users (user_id INT);
INSERT INTO active_users VALUES (1), (2), (3), (4), (5);

CREATE TABLE blacklisted (user_id INT);
INSERT INTO blacklisted VALUES (2), (4), (4);

-- The query
SELECT user_id FROM active_users EXCEPT SELECT user_id FROM blacklisted;
```

The question: "What does *EXCEPT* return?"

Thinking out loud: 1. "EXCEPT returns rows in the first query that are not in the second." 2. "Active has 1, 2, 3, 4, 5. Blacklisted has 2 and 4." 3. "Remove 2 and 4 from active — left with 1, 3, 5." 4. "EXCEPT also dedupes by default, so even if active had duplicates, the result would be a set."

The answer (with output):

```
user_id
-----
1
3
5
```

Why this output: EXCEPT is set difference — rows of the left set whose values do not appear in the right set, deduplicated. MySQL did not support EXCEPT until version 8.0.31; many MySQL codebases emulate it with `LEFT JOIN ... WHERE right IS NULL`.

The trap: Candidates conflate EXCEPT with MINUS (Oracle's keyword) or write `NOT IN` and hit the NULL-in-list trap from Scenario 22.

What NOT to say: - "EXCEPT is exactly the same as NOT IN."

Common follow-up: "How would you write this with a LEFT JOIN, and why is that safer when the right side may contain NULLs?"

Scenario249 · Integer Division Surprise

Tags: Difficulty: Medium · Asked at: Oracle India, IBM India, Salesforce India · Dialect: Postgres / Oracle / SQL Server

The setup:

```
-- Table data
CREATE TABLE quiz (student_id INT, correct INT, total INT);
INSERT INTO quiz VALUES (1, 7, 10), (2, 3, 4), (3, 1, 3);

-- The query (Postgres or SQL Server)
SELECT student_id, correct / total AS ratio FROM quiz;
```

The question: "What is the ratio column?"

Thinking out loud: 1. "Both columns are INT. In Postgres and SQL Server, integer divided by integer returns integer — the fractional part is truncated." 2. "Row 1: $7 / 10 = 0$ (the 0.7 is truncated). Row 2: $3 / 4 = 0$. Row 3: $1 / 3 = 0$." 3. "All three return 0 — likely not what the writer wanted." 4. "To get the decimal, cast one operand to a numeric type: `correct * 1.0 / total` or `CAST(correct AS DECIMAL) / total`."

The answer (with output):

```
student_id | ratio
-----+-----
1          | 0
2          | 0
3          | 0
```

Why this output: INT / INT = INT in most engines. MySQL is the exception — it returns a decimal by default when at least one operand suggests a non-integer outcome.

The trap: Candidates expect 0.7 and write reports that silently show zero for everyone.

What NOT to say: - "It returns 0.7, 0.75 and 0.33."

Common follow-up: "How does MySQL behave for the same query, and how would you write a portable version?"

Scenario250 · Mixed Arithmetic With NULL

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Accenture · Dialect: vendor-agnostic

The setup:

```
-- Table data
CREATE TABLE order_charges (order_id INT, subtotal INT, tax INT, discount INT);
INSERT INTO order_charges VALUES
  (1, 1000, 180, 100),
  (2, 500, NULL, 50),
  (3, 2000, 360, NULL);

-- The query
SELECT order_id, subtotal + tax - discount AS net_total FROM order_charges;
```

The question: "What is the net_total for each row?"

Thinking out loud: 1. "Any arithmetic with NULL produces NULL — that is the propagation rule." 2. "Row 1: $1000 + 180 - 100 = 1080$. Clean." 3. "Row 2: $500 + \text{NULL} - 50$ — the first sub-expression is NULL, the whole thing is NULL." 4. "Row 3: $2000 + 360 - \text{NULL} = \text{NULL}$ by the same rule."

The answer (with output):

```
order_id | net_total
-----+-----
1         | 1080
2         | NULL
3         | NULL
```

Why this output: Arithmetic operators propagate NULL — there is no "skip" behaviour like there is in aggregates. Use COALESCE to substitute a default before the arithmetic: `subtotal + COALESCE(tax, 0) - COALESCE(discount, 0)`.

The trap: Candidates assume the engine treats NULL as zero in arithmetic, the same way it treats it in COUNT or SUM.

What NOT to say: - "Row 2 is 450 because NULL acts like 0."

Common follow-up: "Rewrite this so a missing tax or discount is treated as zero, but a missing subtotal still produces NULL."

Category 4, Part B — What Will This Return: Joins, Windows & Aggregation Edges

Part B continues the "trace through the output" format with harder cases — join cardinality, window frame edges, and aggregation traps that decide round-2 outcomes. Every scenario below is a realistic snippet pulled from the kind of code reviewer-style screen-share that mid and senior SQL interviews now lean on. You walk through table contents, query, your reasoning, the exact row-by-row output, the trap

that catches most candidates, and the follow-up the panel almost always fires back. Work through these in order — the cardinality scenarios early on are the muscle you will need for the window and aggregation work later.

Scenario251 · LEFT JOIN preserving outer rows

Tags: Difficulty: Easy-Medium · Asked at: TCS, Infosys, Cognizant, Wipro · Topic: Outer joins, NULL handling

Setup:

customers :

customer_id	name	city
1	Anita Rao	Bengaluru
2	Rohit Singh	Pune
3	Meera Iyer	Chennai

orders :

order_id	customer_id	amount
101	1	4500
102	1	1200
103	2	3300

```
SELECT c.customer_id, c.name, o.order_id, o.amount
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
ORDER BY c.customer_id, o.order_id;
```

Question: "What rows come out and what is the row count?"

Thinking out loud: 1. "LEFT JOIN keeps every row from the left side — customers — whether or not a match exists on the right." 2. "Anita has two orders, so she will appear twice. Rohit has one order. Meera has none." 3. "Meera will still appear once, but the order_id and amount columns will be NULL because no matching row exists in orders." 4. "Final count is two for Anita plus one for Rohit plus one for Meera as a NULL row — four rows total."

Answer:

customer_id	name	order_id	amount
1	Anita Rao	101	4500
1	Anita Rao	102	1200
2	Rohit Singh	103	3300
3	Meera Iyer	NULL	NULL

Why (the non-obvious thing): The whole point of LEFT JOIN is to preserve unmatched left rows by padding the right-side columns with NULL. If you forget that, you would predict three rows and miss Meera entirely.

The trap: Candidates count only the matched rows and answer three. That is the row count for INNER JOIN, not LEFT.

What NOT to say: - "Meera is dropped because she has no order" — that is the INNER JOIN answer - "Four rows but no NULLs anywhere" — the right side must be NULL for the unmatched outer row

Common follow-up: "How would you count customers who have never ordered?" — `WHERE o.order_id IS NULL` after the LEFT JOIN, then `COUNT(*)` .

Scenario252 · LEFT JOIN silently converted to INNER by WHERE clause

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Walmart Labs · Topic: Outer joins, predicate placement

Setup:`customers :`

customer_id	name
1	Anita Rao
2	Rohit Singh
3	Meera Iyer

`orders :`

order_id	customer_id	amount	status
101	1	4500	paid
102	2	3300	cancelled

```
SELECT c.customer_id, c.name, o.order_id, o.status
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.status = 'paid';
```

Question: "How many rows come back and why?"

Thinking out loud: 1. "LEFT JOIN would give three rows — Anita with order 101, Rohit with order 102, Meera with NULLs." 2. "Then the WHERE clause filters on `o.status = 'paid'`. For Meera the status column is NULL, and `NULL = 'paid'` evaluates to UNKNOWN, which the WHERE drops." 3. "Rohit's order has status cancelled, so he is also dropped." 4. "Only Anita's row survives. The LEFT JOIN has been silently converted into an INNER JOIN by the WHERE predicate."

Answer:

```
customer_id | name      | order_id | status
1           | Anita Rao | 101      | paid
```

Why (the non-obvious thing): Any non-null filter on a right-side column inside WHERE turns LEFT JOIN into INNER JOIN. To preserve outer rows the predicate must move into the ON clause.

The trap: You wrote LEFT JOIN expecting all three customers, but you lose Meera the moment you filter on a right-side column.

What NOT to say: - "Three rows because LEFT JOIN keeps everyone" — the WHERE collapses it - "It throws because Meera has NULL status" — NULL comparisons return UNKNOWN, not an error

Common follow-up: "How would you preserve Meera in the output?" — Move the predicate into ON: `LEFT JOIN orders o ON o.customer_id = c.customer_id AND o.status = 'paid'`.

Scenario253 · Self-join with `a.id < b.id` avoiding duplicates

Tags: Difficulty: Medium · Asked at: Flipkart, Zomato, JP Morgan · Topic: Self-join, pair generation

Setup:

`employees` :

emp_id	name	dept
1	Aditi	Sales
2	Bhavesht	Sales
3	Chitra	Sales
4	Devansh	Engg

```
SELECT a.name AS emp1, b.name AS emp2, a.dept
FROM employees a
JOIN employees b ON a.dept = b.dept AND a.emp_id < b.emp_id
ORDER BY a.dept, a.emp_id, b.emp_id;
```

Question: "How many pairs and what are they?"

Thinking out loud: 1. "Self-join on same department gives us every cross-product within a department, then the `a.emp_id < b.emp_id` predicate keeps only one ordering per pair." 2. "Sales has three people. Without the predicate we would get nine rows including self-pairs and both orderings. With `a.emp_id < b.emp_id`, that drops to $C(3,2) = 3$ pairs." 3. "Engineering has only Devansh, so no pair can be formed." 4. "Final answer is three rows, all from Sales — Aditi-Bhavesht, Aditi-Chitra, Bhavesht-Chitra."

Answer:

```
emp1 | emp2 | dept
Aditi | Bhavesht | Sales
Aditi | Chitra | Sales
Bhavesht | Chitra | Sales
```

Why (the non-obvious thing): The `<` predicate is the canonical trick for generating unordered pairs from a self-join. It eliminates both the self-match and the mirror duplicate in one stroke.

The trap: Forgetting that without the predicate you get N^2 rows including N self-pairs and $N(N-1)$ directional duplicates.

What NOT to say: - "Six pairs because Sales has three people" — that is the directed count, not unordered - "Engineering gives one row of Devansh with himself" — the `<` predicate explicitly excludes self-matches

Common follow-up: "What changes if you use `a.emp_id <> b.emp_id` instead?" — You get directional pairs both ways: six Sales rows instead of three.

Scenario254 · Self-join without the inequality — counting pairs twice**Tags:** Difficulty: Medium · Asked at: Paytm, Cred, Goldman Sachs · Topic: Self-join cardinality**Setup:**`employees` :

emp_id	name	dept
1	Aditi	Sales
2	Bhavesh	Sales
3	Chitra	Sales

```
SELECT a.name AS emp1, b.name AS emp2
FROM employees a
JOIN employees b ON a.dept = b.dept AND a.emp_id <> b.emp_id;
```

Question: "Row count?"

Thinking out loud: 1. "Three people in Sales. The join condition matches anyone in the same department who is not the same row." 2. "For each of the three rows on side a, two rows on side b qualify — anyone in Sales except itself." 3. "So $3 \times 2 = 6$ rows. Each unordered pair is counted twice: Aditi-Bhavesh and Bhavesh-Aditi are both present." 4. "Final answer is six rows."

Answer:

```
emp1 | emp2
Aditi | Bhavesh
Aditi | Chitra
Bhavesh | Aditi
Bhavesh | Chitra
Chitra | Aditi
Chitra | Bhavesh
```

Why (the non-obvious thing): `<>` excludes self-matches but not mirror duplicates. For N rows in a group you get $N(N-1)$ ordered pairs, not $N(N-1)/2$ unordered.

The trap: Reports built on this pattern double-count pair-based metrics like "how many co-worker pairs exist" and exaggerate by a factor of two.

What NOT to say: - "Three rows because there are three people" — that ignores the cross-product - "Nine rows because 3 times 3" — that includes self-pairs which `<>` excludes

Common follow-up: "How would you get the unordered count of three?" — Change `<>` to `<`, or wrap in `COUNT(*)/2`.

Scenario255 · CROSS JOIN cardinality

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Accenture · Topic: Cartesian product

Setup:

`sizes` :

size
S
M
L

`colors` :

color
Red
Blue
Green
Black

```
SELECT s.size, c.color
FROM sizes s
CROSS JOIN colors c;
```

Question: "How many rows and what is the shape of the output?"

Thinking out loud: 1. "CROSS JOIN has no ON clause — it pairs every row on the left with every row on the right." 2. "Three sizes times four colors equals twelve rows." 3. "Output is every size and every color combination, useful for generating a complete SKU grid." 4. "Order is not guaranteed unless we add ORDER BY, but typically engines stream left-then-right."

Answer:

```
size | color
S    | Red
S    | Blue
S    | Green
S    | Black
M    | Red
M    | Blue
M    | Green
M    | Black
L    | Red
L    | Blue
L    | Green
L    | Black
```

Why (the non-obvious thing): CROSS JOIN row count is the product of the two side counts. With 1,000 left rows and 1,000 right rows you get a million-row result — a classic accidental performance disaster when an ON clause is missing.

The trap: Writing `FROM a, b` without a WHERE clause is implicit CROSS JOIN. Junior developers do it without realising.

What NOT to say: - "Seven rows because 3 plus 4" — CROSS JOIN multiplies, not adds - "It throws because no ON clause" — CROSS JOIN is the one join that legally has no ON

Common follow-up: "*When would you intentionally use CROSS JOIN?*" — Generating calendar dimensions, SKU grids, time-of-day buckets paired with stores.

Scenario256 · Multi-table JOIN cardinality multiplication

Tags: Difficulty: Medium · Asked at: Swiggy, Razorpay, Walmart Labs · Topic: Join cardinality

Setup:

`users` :

user_id
1
2

`orders` :

order_id	user_id
101	1
102	1
103	2

order_items :

item_id	order_id
9001	101
9002	101
9003	102
9004	103
9005	103

```
SELECT u.user_id, o.order_id, i.item_id
FROM users u
JOIN orders o ON o.user_id = u.user_id
JOIN order_items i ON i.order_id = o.order_id;
```

Question: "How many rows in the final result?"

Thinking out loud: 1. "Start with users — two rows. Join to orders gives three rows because user 1 has two orders and user 2 has one." 2. "Now join order_items. Order 101 has two items, order 102 has one item, order 103 has two items." 3. "So 101 contributes 2 rows, 102 contributes 1 row, 103 contributes 2 rows — total of five rows in the final result." 4. "Each row in the final result is a (user, order, item) triple."

Answer:

```
user_id | order_id | item_id
1       | 101     | 9001
1       | 101     | 9002
1       | 102     | 9003
2       | 103     | 9004
2       | 103     | 9005
```

Why (the non-obvious thing): Each join expands rows by the multiplicity of matches on the next table. The final row count is the sum of items per user, not the product of total table sizes.

The trap: Candidates multiply $2 \times 3 \times 5$ and get 30, forgetting that ON clauses constrain the cross-product.

What NOT to say: - "Thirty rows because two times three times five" — that would be three independent CROSS JOINS - "Ten rows" — that is some other ad-hoc count, not what the joins produce

Common follow-up: "If you *SUM(amount)* at this level, what happens to user 1's revenue?" — It double-counts order 101 because of its two items.

Scenario257 · One-to-many JOIN inflating row count for aggregation

Tags: Difficulty: Medium · Asked at: Flipkart, PhonePe, Cred · Topic: Aggregation pitfalls

Setup:

`orders` :

order_id	amount
101	1000
102	500

`order_items` :

item_id	order_id	qty
9001	101	1
9002	101	2
9003	102	1

```
SELECT o.order_id, SUM(o.amount) AS total
FROM orders o
JOIN order_items i ON i.order_id = o.order_id
GROUP BY o.order_id;
```

Question: "What does the total column say for each order?"

Thinking out loud: 1. "Join inflates order 101 to two rows because it has two items. Order 102 stays at one row." 2. "After grouping by order_id, the SUM(o.amount) is summing the amount once for every joined row." 3. "Order 101: amount 1000 summed twice equals 2000. Order 102: 500 summed once equals 500." 4. "So the reported totals are inflated for any order with more than one item."

Answer:

```
order_id | total
101      | 2000
102      | 500
```

Why (the non-obvious thing): Aggregating a parent-side column after joining to a child table produces inflated sums because the parent row is duplicated for every child row.

The trap: This is one of the most common analytics bugs in production. Revenue dashboards mysteriously double or triple after a "harmless" join.

What NOT to say: - "1000 and 500 because amount is on the orders table" — the join still duplicates the parent row - "It throws a duplicate key error" — GROUP BY collapses cleanly, the bug is silent

Common follow-up: "How would you avoid this?" — Aggregate items first in a subquery, then join. Or use SELECT DISTINCT on (order_id, amount) before summing.

Scenario258 · Many-to-many JOIN explosion

Tags: Difficulty: Medium-Hard · Asked at: Zomato, Walmart Labs, JP Morgan · Topic: Join cardinality

Setup:

students :

student_id	name
1	Ananya
2	Bilal

enrollments :

student_id	course_id
1	C1
1	C2
2	C1
2	C3

grades :

student_id	course_id	score
1	C1	85
1	C1	90
1	C2	75
2	C1	60

```
SELECT s.name, e.course_id, g.score
FROM students s
JOIN enrollments e ON e.student_id = s.student_id
JOIN grades g ON g.student_id = s.student_id AND g.course_id = e.course_id;
```

Question: "Predict the row count."

Thinking out loud: 1. "Ananya is enrolled in C1 and C2. For C1 she has two grade rows; for C2 she has one. That contributes $2 + 1 = 3$ rows." 2. "Bilal is enrolled in C1 and C3. For C1 he has one grade; for C3 he has zero. That contributes $1 + 0 = 1$ row." 3. "Total is 4 rows." 4. "The C1 doubling for Ananya is the explosion — two grade records for the same enrollment pair."

Answer:

name	course_id	score
Ananya	C1	85
Ananya	C1	90
Ananya	C2	75
Bilal	C1	60

Why (the non-obvious thing): When both joined tables have multiple rows per matching key, the join multiplies them. The explosion is bounded by the per-key multiplicity, not by the table size.

The trap: Reports built on this produce inflated counts that look "almost right" — easy to ship and hard to detect.

What NOT to say: - "Four rows because enrollments has four rows" — only because the multiplicity happened to land here, not by rule - "Eight rows because every enrollment cross-products with every grade" — that ignores the course_id predicate

Common follow-up: *"How would you collapse the duplicate C1 grades for Ananya to one row?"* — Pre-aggregate grades by (student_id, course_id) with AVG, MAX, or whatever the business wants.

Scenario259 · Inequality range join

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman Sachs, Walmart Labs · Topic: Range joins

Setup:

payments :

payment_id	amount
1	250
2	800
3	1500
4	4500

fee_tiers :

min_amt	max_amt	fee_pct
0	999	2.0
1000	4999	1.5
5000	99999	1.0

```
SELECT p.payment_id, p.amount, f.fee_pct
FROM payments p
JOIN fee_tiers f ON p.amount BETWEEN f.min_amt AND f.max_amt;
```

Question: *"What rows come out?"*

Thinking out loud: 1. "BETWEEN is inclusive on both ends. We need to walk each payment and find its tier." 2. "Payment 1 has amount 250 — falls into tier 1 (0-999), fee 2.0." 3. "Payment 2 has amount 800 — also tier 1, fee 2.0." 4. "Payment 3 has amount 1500 — tier 2 (1000-4999), fee 1.5. Payment 4 has amount 4500 — also tier 2, fee 1.5. Nothing hits tier 3."

Answer:

payment_id	amount	fee_pct
1	250	2.0
2	800	2.0
3	1500	1.5
4	4500	1.5

Why (the non-obvious thing): Range joins use inequality predicates instead of equality. They are correct, but the optimiser cannot use a normal hash join — it falls back to nested loop or merge join, which gets expensive on large inputs.

The trap: If your tier ranges overlap by even one unit, a payment can land in two tiers and the row count doubles for boundary values.

What NOT to say: - "It throws because BETWEEN needs equality" — BETWEEN works fine in joins - "Four rows in tier 1 because that is the first row" — every payment is checked against every tier

Common follow-up: "What if amount is exactly 999?" — Lands in tier 1. If you change to `>= AND <`, the boundary handling changes.

Scenario260 · FULL OUTER JOIN with two partially-overlapping tables

Tags: Difficulty: Medium-Hard · Asked at: Walmart Labs, Goldman Sachs, JP Morgan · Topic: Outer joins

Setup:

`payroll` :

emp_id	salary
1	50000
2	60000
3	70000

`benefits` :

emp_id	plan
2	Premium
3	Standard
4	Premium

```
SELECT p.emp_id AS payroll_id, b.emp_id AS benefits_id, p.salary, b.plan
FROM payroll p
FULL OUTER JOIN benefits b ON p.emp_id = b.emp_id
ORDER BY COALESCE(p.emp_id, b.emp_id);
```

Question: "What rows come out?"

Thinking out loud: 1. "FULL OUTER JOIN keeps every row from both sides. Matches join up; unmatched rows on either side appear with NULLs on the other side." 2. "Emp 1 only in payroll — appears with benefits_id and plan NULL." 3. "Emps 2 and 3 are in both — clean join." 4. "Emp 4 only in benefits — appears with payroll_id and salary NULL. Total of four rows."

Answer:

payroll_id	benefits_id	salary	plan
1	NULL	50000	NULL
2	2	60000	Premium
3	3	70000	Standard
NULL	4	NULL	Premium

Why (the non-obvious thing): FULL OUTER JOIN is the union of LEFT and RIGHT outer joins. The unmatched rows on each side carry NULLs in the other side's columns — including the join key.

The trap: Selecting `p.emp_id` alone loses emp 4. You must use COALESCE to get a unified employee identifier.

What NOT to say: - "Three rows because there are three payroll entries" — that ignores emp 4 on the benefits side - "MySQL handles this with no syntax change" — MySQL did not support FULL OUTER JOIN natively for years; needs UNION emulation

Common follow-up: "How would you find rows that exist in only one table?" — `WHERE p.emp_id IS NULL OR b.emp_id IS NULL`.

Scenario261 · ROW_NUMBER vs RANK on tied values

Tags: Difficulty: Medium · Asked at: Flipkart, Swiggy, PhonePe · Topic: Window functions, ranking

Setup:

`exam_scores` :

student	marks
Asha	95
Bilal	90
Chitra	90
Dhruv	85

```
SELECT student,
       marks,
       ROW_NUMBER() OVER (ORDER BY marks DESC) AS rn,
       RANK()      OVER (ORDER BY marks DESC) AS rk
FROM exam_scores;
```

Question: "What do *rn* and *rk* look like row by row?"

Thinking out loud: 1. "Both functions order by marks descending. The tie between Bilal and Chitra at 90 is where the two functions diverge." 2. "ROW_NUMBER always assigns a unique sequence: 1, 2, 3, 4. Tie-breaking is implementation-defined unless we add a secondary ORDER BY." 3. "RANK gives both tied rows the same rank — both get 2 — then skips to 4 for the next distinct value." 4. "So Asha gets 1/1, Bilal gets 2/2, Chitra gets 3/2, Dhruv gets 4/4."

Answer:

student	marks	rn	rk
Asha	95	1	1
Bilal	90	2	2
Chitra	90	3	2
Dhruv	85	4	4

Why (the non-obvious thing): ROW_NUMBER cares about order; RANK cares about value. RANK leaves "holes" in the sequence after ties — that is why Dhruv jumps from 2 to 4.

The trap: Using ROW_NUMBER to assign "rank" produces non-deterministic results when ties exist, because the tie-break ordering is engine-dependent.

What NOT to say: - "Both produce 1, 2, 3, 4" — only ROW_NUMBER does that - "RANK gives 1, 2, 2, 3" — that is DENSE_RANK behaviour, not RANK

Common follow-up: "What is the difference between RANK and DENSE_RANK?" — DENSE_RANK does not leave gaps after ties; Dhruv would get 3, not 4.

Scenario262 · DENSE_RANK on tied values

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman Sachs · Topic: Window functions, ranking

Setup:

exam_scores :

student	marks
Asha	95
Bilal	90
Chitra	90
Dhruv	85
Eshan	85
Farah	80

```
SELECT student,
       marks,
       DENSE_RANK() OVER (ORDER BY marks DESC) AS drk
FROM exam_scores;
```

Question: "Show me the drk column."

Thinking out loud: 1. "DENSE_RANK assigns the same rank to ties but does not leave gaps." 2. "Distinct marks in descending order are 95, 90, 85, 80 — ranks 1, 2, 3, 4." 3. "Every row gets the rank of its mark group: Asha 1, Bilal and Chitra both 2, Dhruv and Eshan both 3, Farah 4." 4. "Compare with RANK, where Dhruv and Eshan would be 4 and Farah would be 6."

Answer:

student	marks	drk
Asha	95	1
Bilal	90	2
Chitra	90	2
Dhruv	85	3
Eshan	85	3
Farah	80	4

Why (the non-obvious thing): DENSE_RANK is the right choice when you want "ordinal position among distinct values" — top 3 distinct scores is a DENSE_RANK problem, not a RANK problem.

The trap: Using RANK when you mean DENSE_RANK and your top-N query mysteriously returns fewer rows than expected because ties consume rank slots.

What NOT to say: - "DENSE_RANK leaves a gap of 1 after ties" — no, it never leaves a gap - "It is the same as ROW_NUMBER" — only when there are no ties

Common follow-up: "Top 3 distinct scores — RANK or DENSE_RANK?" — DENSE_RANK <= 3 gives you the top three score groups; RANK <= 3 may give you fewer rows depending on ties at the top.

Scenario263 · LAG with no previous row returns NULL

Tags: Difficulty: Medium · Asked at: Swiggy, Zomato, Paytm · Topic: Window functions, LAG

Setup:

daily_revenue :

day	amount
1	1000
2	1200
3	900
4	1500

```
SELECT day,
       amount,
       LAG(amount) OVER (ORDER BY day) AS prev_amount,
       amount - LAG(amount) OVER (ORDER BY day) AS delta
FROM daily_revenue;
```

Question: "What is delta for day 1 and what is delta for day 2?"

Thinking out loud: 1. "LAG looks one row back in the ordered window. Day 1 has no previous row, so LAG returns NULL." 2. " `amount - NULL` is NULL — any arithmetic on NULL propagates." 3. "Day 1: delta is NULL. Day 2: prev is 1000, delta is $1200 - 1000 = 200$." 4. "Day 3: delta is $900 - 1200 = -300$. Day 4: delta is $1500 - 900 = 600$."

Answer:

day	amount	prev_amount	delta
1	1000	NULL	NULL
2	1200	1000	200
3	900	1200	-300
4	1500	900	600

Why (the non-obvious thing): LAG returns NULL when the offset goes outside the partition. Any subsequent arithmetic produces NULL, which often surprises candidates who expected zero.

The trap: Reports that look fine in the middle of the series but the first row shows NULL — the dashboard then breaks or shows a blank.

What NOT to say: - "Day 1 delta is 0 because there is nothing before" — LAG returns NULL, not 0 - "Day 1 delta is 1000 because that is the amount" — LAG does not fall back to the current row

Common follow-up: "How would you make day 1 delta show 0?" — `LAG(amount, 1, 0)` — the third argument is the default for out-of-partition rows.

Scenario264 · LEAD with offset past partition end

Tags: Difficulty: Medium · Asked at: Razorpay, Walmart Labs, Cred · Topic: Window functions, LEAD

Setup:

`monthly_users` :

month_no	active
1	100
2	130
3	150

```
SELECT month_no,  
       active,  
       LEAD(active, 2) OVER (ORDER BY month_no) AS active_in_2_months  
FROM monthly_users;
```

Question: "Show me the third column."

Thinking out loud: 1. "LEAD(active, 2) looks two rows ahead." 2. "Month 1 plus 2 is month 3 — value 150." 3. "Month 2 plus 2 would be month 4, which does not exist — NULL." 4. "Month 3 plus 2 is month 5 — also NULL."

Answer:

month_no	active	active_in_2_months
1	100	150
2	130	NULL
3	150	NULL

Why (the non-obvious thing): Any LEAD or LAG that points beyond the partition boundary returns NULL unless you supply a default as the third argument.

The trap: Forecast queries that compare current vs future-month values silently turn NULL near the end of the series.

What NOT to say: - "It wraps around to the start" — windows do not wrap - "It returns 0" — that requires the default argument

Common follow-up: "How would you make missing future values fall back to 0?" —
`LEAD(active, 2, 0)`.

Scenario265 · FIRST_VALUE vs LAST_VALUE default frame trap

Tags: Difficulty: Hard · Asked at: Goldman Sachs, JP Morgan, Walmart Labs · Topic: Window frames

Setup:

`stock_price`:

day	price
1	100
2	110
3	105
4	120

```
SELECT day,
       price,
       FIRST_VALUE(price) OVER (ORDER BY day) AS first_p,
       LAST_VALUE(price) OVER (ORDER BY day) AS last_p
FROM stock_price;
```

Question: "What does last_p look like in each row?"

Thinking out loud: 1. "The default frame for a window with ORDER BY is RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW. That bites LAST_VALUE." 2. "FIRST_VALUE always sees the first row of the frame — and since the frame starts at UNBOUNDED PRECEDING, it is always row 1's price = 100. Consistent across all rows." 3. "LAST_VALUE sees the last row of the frame, which is CURRENT ROW. So it just returns the current row's price every time." 4. "To get the actual last price across the whole partition you need to widen the frame to UNBOUNDED FOLLOWING."

Answer:

```
day | price | first_p | last_p
1   | 100   | 100     | 100
2   | 110   | 100     | 110
3   | 105   | 100     | 105
4   | 120   | 100     | 120
```

Why (the non-obvious thing): LAST_VALUE without an explicit frame is almost always a bug. Everyone assumes it means "last in partition" but the default frame stops at the current row, so it just echoes the current row's value.

The trap: Candidates expect last_p to be 120 in every row. Instead it equals price every row.

What NOT to say: - "Both columns equal 100 in every row" — that is the FIRST_VALUE column only - "LAST_VALUE returns the maximum" — no, it returns the value at the end of the frame, not the max

Common follow-up: "How do you fix it to return 120 in every row?" — Add explicit frame: `OVER (ORDER BY day ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)` .

Scenario266 · Running total SUM() OVER (ORDER BY ...)

Tags: Difficulty: Easy-Medium · Asked at: Flipkart, Swiggy, TCS Digital · Topic: Window functions, cumulative sum

Setup:

`daily_sales` :

day	amount
1	200
2	300
3	150
4	400

```
SELECT day,
       amount,
       SUM(amount) OVER (ORDER BY day) AS running_total
FROM daily_sales;
```

Question: "What is running_total at each row?"

Thinking out loud: 1. "SUM with ORDER BY and no frame clause defaults to RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW — a cumulative sum." 2. "Day 1: just 200. Day 2: 200 + 300 = 500. Day 3: 500 + 150 = 650. Day 4: 650 + 400 = 1050." 3. "The window grows by one row at every step." 4. "This is the canonical running total pattern."

Answer:

```
day | amount | running_total
1   | 200    | 200
2   | 300    | 500
3   | 150    | 650
4   | 400    | 1050
```

Why (the non-obvious thing): SUM with an ORDER BY clause silently becomes cumulative because of the default frame. Drop the ORDER BY and SUM becomes a constant total across the whole partition.

The trap: Mixing up "running" vs "total" because you forgot to add or omit ORDER BY.

What NOT to say: - "It returns 1050 in every row" — only if you remove ORDER BY - "It returns 200, 300, 150, 400 because windows do not aggregate" — they do; that is the whole point

Common follow-up: "What if I want a 3-day rolling sum?" — `OVER (ORDER BY day ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)` .

Scenario267 · Multiple windows in one query

Tags: Difficulty: Medium · Asked at: Razorpay, Walmart Labs, Goldman Sachs · Topic: Window functions

Setup:

`employee_salary` :

emp_id	dept	salary
1	Sales	50000
2	Sales	60000
3	Engg	80000
4	Engg	75000
5	Engg	90000

```
SELECT emp_id,
       dept,
       salary,
       AVG(salary) OVER ()           AS overall_avg,
       AVG(salary) OVER (PARTITION BY dept) AS dept_avg,
       RANK() OVER (PARTITION BY dept ORDER BY salary DESC) AS dept_rank
FROM employee_salary;
```

Question: "For emp_id 3 in Engg, what are the three window column values?"

Thinking out loud: 1. "Overall avg is over the whole table — $(50000+60000+80000+75000+90000)/5 = 355000/5 = 71000$." 2. "Dept avg for Engg is $(80000+75000+90000)/3 = 245000/3 \approx 81666.67$." 3. "Dept rank by salary descending in Engg: 90000 is rank 1, 80000 is rank 2, 75000 is rank 3. Emp 3 is rank 2." 4. "All three windows compute independently and are stitched onto the same row — no GROUP BY needed."

Answer (for emp 3 only):

```
emp_id | dept | salary | overall_avg | dept_avg | dept_rank
3      | Engg | 80000  | 71000       | 81666.67 | 2
```

Why (the non-obvious thing): Each window function has its own OVER clause and computes independently. You can mix global, partition-scoped, and order-scoped windows in a single SELECT.

The trap: Believing that PARTITION BY in one OVER clause affects the others. It does not — windows are isolated.

What NOT to say: - "Dept avg equals overall avg" — only true if every dept has identical mean - "RANK uses the global ordering" — PARTITION BY restricts ranking to within the dept

Common follow-up: "How would you find the highest-paid employee per department?" — Wrap in a CTE and filter `WHERE dept_rank = 1`.

Scenario268 · Window function with PARTITION BY combined with GROUP BY

Tags: Difficulty: Hard · Asked at: JP Morgan, Walmart Labs, Goldman Sachs · Topic: Aggregation + windows

Setup:

`sales` :

region	rep	amount
North	Aman	1000
North	Aman	500
North	Beena	800
South	Chetan	2000
South	Chetan	300

```
SELECT region,
       rep,
       SUM(amount) AS rep_total,
       SUM(SUM(amount)) OVER (PARTITION BY region) AS region_total
FROM sales
GROUP BY region, rep;
```

Question: "What does each row look like?"

Thinking out loud: 1. "GROUP BY collapses rows to one per (region, rep). Aman: 1500, Beena: 800, Chetan: 2300." 2. "The outer SUM(SUM(amount)) is a window function evaluated after grouping. It sums the rep_totals within each region." 3. "North region: Aman 1500 + Beena 800 = 2300. South region: just Chetan 2300." 4. "So every Aman or Beena row shows region_total 2300, and Chetan's row shows region_total 2300."

Answer:

region	rep	rep_total	region_total
North	Aman	1500	2300
North	Beena	800	2300
South	Chetan	2300	2300

Why (the non-obvious thing): Window functions run after GROUP BY. The inner aggregate (SUM(amount)) produces the grouped total per row; the outer window aggregate then operates on those grouped values within a partition.

The trap: Candidates write SUM(amount) inside the window thinking it aggregates raw amounts, but post-GROUP-BY the raw rows no longer exist — only the grouped rep totals.

What NOT to say: - "Region_total is the sum of raw amounts" — it is the sum of the already-grouped rep_totals - "This throws because you cannot nest aggregates" — nesting an aggregate inside a window aggregate is legal

Common follow-up: "What does each rep's percentage of region revenue look like?" — `rep_total * 1.0 / SUM(SUM(amount)) OVER (PARTITION BY region)`.

Scenario269 · NTILE when row count is not divisible by N

Tags: Difficulty: Medium-Hard · Asked at: Cred, PhonePe, Walmart Labs · Topic: Window functions, bucketing

Setup:

`customer_spend` :

customer	spend
C1	100
C2	200
C3	300
C4	400
C5	500
C6	600
C7	700

```
SELECT customer,
       spend,
       NTILE(3) OVER (ORDER BY spend) AS tier
FROM customer_spend;
```

Question: "Seven rows split into three tiers — which customer is in which tier?"

Thinking out loud: 1. "Seven divided by three is 2 remainder 1. NTILE distributes the remainder one row at a time, starting from the first bucket." 2. "So bucket 1 gets $\text{ceil}(7/3) = 3$ rows, bucket 2 gets 2 rows, bucket 3 gets 2 rows." 3. "In ORDER BY spend ascending: C1, C2, C3 → tier 1. C4, C5 → tier 2. C6, C7 → tier 3." 4. "The extra row always goes to the lower-numbered buckets first."

Answer:

```
customer | spend | tier
C1       | 100   | 1
C2       | 200   | 1
C3       | 300   | 1
C4       | 400   | 2
C5       | 500   | 2
C6       | 600   | 3
C7       | 700   | 3
```

Why (the non-obvious thing): NTILE never produces equal-sized buckets when the row count is not divisible by N. The remainder is front-loaded into the earliest buckets, one row per bucket.

The trap: Reports describing "top quartile of customers" can include a slightly larger or smaller group than expected, biasing percentile calculations.

What NOT to say: - "Every bucket has the same row count" — only when divisible - "The remainder goes into bucket N" — it goes into bucket 1, then 2, and so on

Common follow-up: "How would you ensure equal-sized groups regardless of remainder?" — You cannot guarantee it with NTILE; use modulo logic or PERCENT_RANK with explicit cutoffs.

Scenario270 · GROUP BY with COUNT

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Accenture · Topic: Aggregation basics

Setup:

`orders` :

order_id	status
1	paid
2	paid
3	pending
4	paid
5	cancelled
6	pending

```
SELECT status, COUNT(*) AS cnt
FROM orders
GROUP BY status
ORDER BY cnt DESC;
```

Question: "What is the output?"

Thinking out loud: 1. "GROUP BY status collapses rows that share the same status into one row each. COUNT() counts rows per group." 2. "Paid appears 3 times, pending 2 times, cancelled 1 time." 3. "ORDER BY cnt DESC means paid comes first, then pending, then cancelled." 4. "Three rows in the output."

Answer:

```
status | cnt
paid   | 3
pending | 2
cancelled | 1
```

Why (the non-obvious thing): COUNT(*) counts every row in the group, including those with NULL in any column. It is the safe default for "how many rows per group".

The trap: Using COUNT(status) when status is sometimes NULL — those rows drop silently from the count.

What NOT to say: - "It returns one row with cnt 6" — that would need no GROUP BY - "It throws because there are duplicate statuses" — duplicates are exactly what GROUP BY exists for

Common follow-up: "What is the difference between COUNT() and COUNT(status)?" — COUNT() counts rows; COUNT(status) counts non-NULL values of status.

Scenario271 · GROUP BY with HAVING filter

Tags: Difficulty: Easy-Medium · Asked at: Razorpay, Swiggy, Cognizant · Topic: HAVING vs WHERE

Setup:

product_sales :

product	qty
A	5
B	1
A	3
C	10
B	2
C	4
D	1


```
SELECT product, SUM(qty) AS total
FROM product_sales
GROUP BY product
HAVING SUM(qty) >= 5
ORDER BY product;
```

Question: "Which products survive HAVING?"

Thinking out loud: 1. "Group totals: A = 8, B = 3, C = 14, D = 1." 2. "HAVING filters group-level results, after aggregation. We keep groups where SUM(qty) >= 5." 3. "A passes (8), C passes (14). B (3) and D (1) are dropped." 4. "Two rows in output, ordered by product alphabetically: A then C."

Answer:

product	total
A	8
C	14

Why (the non-obvious thing): HAVING operates on the grouped output of the query, not the raw rows. You can use aggregate functions in HAVING but not in WHERE (unless you nest with subqueries).

The trap: Trying to filter aggregates in WHERE — `WHERE SUM(qty) >= 5` — and getting "aggregate function not allowed in WHERE" errors.

What NOT to say: - "HAVING and WHERE are interchangeable" — they run at different points in the query lifecycle - "HAVING runs before GROUP BY" — it runs after

Common follow-up: "When would you put a predicate in WHERE vs HAVING?" — WHERE for row-level filters (faster, reduces rows before grouping); HAVING for filters that reference aggregates.

Scenario272 · GROUP BY with ROLLUP — extra subtotal row

Tags: Difficulty: Medium · Asked at: Walmart Labs, JP Morgan, Goldman Sachs · Topic: ROLLUP

Setup:

`regional_sales` :

region	amount
North	1000
North	500
South	800
South	700
East	300

```
SELECT region, SUM(amount) AS total
FROM regional_sales
GROUP BY ROLLUP (region)
ORDER BY region NULLS LAST;
```

Question: "How many rows and what does the extra one mean?"

Thinking out loud: 1. "Plain GROUP BY region would give three rows: North 1500, South 1500, East 300." 2. "ROLLUP adds an extra row representing the grand total across all groups, with NULL in the region column." 3. "Grand total is 1500 + 1500 + 300 = 3300." 4. "Four rows in total — three group rows plus one grand-total row with NULL region."

Answer:

```
region | total
East   | 300
North  | 1500
South  | 1500
NULL   | 3300
```

Why (the non-obvious thing): ROLLUP produces hierarchical subtotals. With one group column you get one extra row — the grand total. With two columns you get subtotals at every level plus the grand total.

The trap: Downstream code or dashboards that do not handle the NULL row crash or display "Unknown region: 3300" as a real category.

What NOT to say: - "ROLLUP changes the totals per group" — no, those are unchanged; it just adds rows - "The NULL row is a bug" — it is a feature, denoting "all"

Common follow-up: "How would you label the grand total row instead of NULL?" —

`COALESCE(region, 'TOTAL')` in the SELECT.

Scenario273 · GROUP BY GROUPING SETS — multiple group levels

Tags: Difficulty: Medium-Hard · Asked at: Walmart Labs, JP Morgan, Goldman Sachs · Topic: GROUPING SETS

Setup:

`sales_by_dim` :

region	category	amount
North	Books	100
North	Music	200
South	Books	300
South	Music	400

```
SELECT region, category, SUM(amount) AS total
FROM sales_by_dim
GROUP BY GROUPING SETS ((region), (category), ())
ORDER BY region NULLS LAST, category NULLS LAST;
```

Question: "How many rows and what does each represent?"

Thinking out loud: 1. "GROUPING SETS is a union of group-by clauses in one query. We asked for three sets: by region, by category, and overall." 2. "By region: North 300, South 700 — category is NULL in those rows." 3. "By category: Books 400, Music 600 — region is NULL in those rows." 4. "Overall: grand total 1000 with both region and category NULL. Total of 5 rows."

Answer:

```
region | category | total
North  | NULL     | 300
South  | NULL     | 700
NULL   | Books    | 400
NULL   | Music    | 600
NULL   | NULL     | 1000
```

Why (the non-obvious thing): GROUPING SETS lets you compute multiple group-bys in one pass over the data instead of running three separate queries and UNIONing them. The empty tuple `()` means "no grouping" — the grand total.

The trap: Forgetting that ROLLUP(region, category) is the hierarchical superset; GROUPING SETS is the explicit list. They differ in what subtotals appear.

What NOT to say: - "It produces a Cartesian product" — no, just the union of group results - "You cannot use empty parentheses in GROUPING SETS" — `()` is exactly the grand-total marker

Common follow-up: "How would you distinguish 'NULL because rolled up' from 'NULL because data is missing'?" — Use the `GROUPING()` function (next scenario).

Scenario274 · GROUPING() function output for rolled-up rows

Tags: Difficulty: Hard · Asked at: JP Morgan, Goldman Sachs, Walmart Labs · Topic: GROUPING function

Setup:

`sales_by_dim` :

region	category	amount
North	Books	100
NULL	Music	200
South	Books	300

```
SELECT region,
       category,
       SUM(amount) AS total,
       GROUPING(region) AS g_region,
       GROUPING(category) AS g_category
FROM sales_by_dim
GROUP BY ROLLUP (region, category)
ORDER BY g_region, g_category, region NULLS FIRST, category NULLS FIRST;
```

Question: "What is the `GROUPING()` value for the rolled-up total row vs the row with a genuinely NULL region?"

Thinking out loud: 1. "`GROUPING(col)` returns 1 if the column was rolled up in this row (i.e., aggregated away), 0 if the value is a real (possibly NULL) value from the data." 2. "For the row where `region=NULL` because the data is genuinely NULL — `GROUPING(region)` returns 0. That tells the consumer: this NULL is real." 3. "For the ROLLUP grand-total row — `region NULL`, `category NULL` — `GROUPING(region)=1` and `GROUPING(category)=1`. That tells the consumer: these NULLs mean 'all'." 4. "`ROLLUP(region, category)` produces detail rows, region-subtotal rows (`category NULL` with `GROUPING(category)=1`), and one grand-total row (both `GROUPING=1`)."

Answer (the key rows):

region	category	total	g_region	g_category	
North	Books	100	0	0	
NULL	Music	200	0	0	<- real NULL in data
South	Books	300	0	0	
North	NULL	100	0	1	<- region subtotal
NULL	NULL	200	0	1	<- "NULL region" subtotal (real)
South	NULL	300	0	1	<- region subtotal
NULL	NULL	600	1	1	<- grand total

Why (the non-obvious thing): Without GROUPING() you cannot distinguish a "subtotal NULL" from a "real data NULL". GROUPING() is the one safe way to label rollup rows in dashboards.

The trap: Consumers downstream that filter `WHERE region IS NULL` to find the grand total accidentally drag in real NULL region rows too.

What NOT to say: - "GROUPING(col) returns the NULL count" — no, it is a 0/1 flag - "You can use GROUPING without ROLLUP" — it is only meaningful when ROLLUP, CUBE, or GROUPING SETS produces aggregated NULLs

Common follow-up: "How would you label rows for a dashboard?" — `CASE WHEN GROUPING(region) = 1 THEN 'All Regions' ELSE COALESCE(region, 'Unknown') END`.

Scenario275 · DISTINCT after JOIN dedupes the cartesian result

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, PhonePe · Topic: DISTINCT, joins

Setup:

`customers` :

customer_id	name
1	Anita
2	Bilal

`orders` :

order_id	customer_id
101	1
102	1
103	2

```
SELECT DISTINCT c.customer_id, c.name
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id;
```

Question: "How many rows come out?"

Thinking out loud: 1. "Without *DISTINCT*, the join produces three rows because Anita has two orders. We would see Anita twice, Bilal once." 2. "*DISTINCT* collapses identical row tuples. Both Anita rows are (1, 'Anita') — same tuple — so they collapse to one." 3. "Bilal stays at one row." 4. "Final answer: two rows."

Answer:

```
customer_id | name
1           | Anita
2           | Bilal
```

Why (the non-obvious thing): *DISTINCT* deduplicates on the selected columns only. If you *SELECT* more columns that differ between joined rows (like *order_id*), *DISTINCT* cannot collapse anything.

The trap: Using *DISTINCT* as a "patch" for accidental joins is a code smell. The right fix is usually to use *EXISTS* or to aggregate before joining.

What NOT to say: - "DISTINCT removes duplicates within a column" — it works on the whole row tuple
- "DISTINCT changes the join cardinality" — no, the join still runs; *DISTINCT* runs after

Common follow-up: "What is the more efficient pattern for finding customers with at least one order?"
— `WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)`.

Scenario276 · DISTINCT inside vs outside subquery

Tags: Difficulty: Medium-Hard · Asked at: Goldman Sachs, JP Morgan, Walmart Labs · Topic: *DISTINCT* placement

Setup:

sales :

customer	amount
A	100
A	100
B	200
B	300
C	100

```
-- Query 1
SELECT SUM(amount) FROM (SELECT DISTINCT customer, amount FROM sales) t;

-- Query 2
SELECT DISTINCT SUM(amount) FROM sales;
```

Question: "What does each query return and why are they different?"

Thinking out loud: 1. "Query 1: inner *DISTINCT* removes the duplicate (A, 100) tuple. Remaining rows: (A, 100), (B, 200), (B, 300), (C, 100). Outer *SUM* = 700." 2. "Query 2: *SUM* is computed first — $100+100+200+300+100 = 800$. Then *DISTINCT* on a single one-row result is a no-op." 3. "So Query 1 returns 700, Query 2 returns 800." 4. "The lesson: *DISTINCT* is a row-level operator, not a value-level one. Its placement decides what gets deduplicated."

Answer:

```
Query 1: 700
Query 2: 800
```

Why (the non-obvious thing): *DISTINCT* inside the subquery deduplicates the raw input rows before aggregation; *DISTINCT* outside an aggregate is almost always meaningless (you only have one row to deduplicate).

The trap: Using `SELECT DISTINCT SUM(x)` thinking it sums distinct values. It does not — for that you need `SUM(DISTINCT x)`.

What NOT to say: - "Both return the same value" — they differ by 100 - "Query 2 throws an error" — *DISTINCT* after a single-row aggregate is legal, just useless

Common follow-up: "What does `SUM(DISTINCT amount)` give you?" — Sums each unique amount once: $100 + 200 + 300 = 600$.

Scenario277 · Window function in WHERE causes engine error

Tags: Difficulty: Medium-Hard · Asked at: Cred, Razorpay, Walmart Labs · Topic: Query execution order

Setup:

employee_salary :

emp_id	dept	salary
1	Sales	50000
2	Sales	60000
3	Engg	80000
4	Engg	75000

```
-- Attempt 1 (broken)
SELECT emp_id, dept, salary
FROM employee_salary
WHERE RANK() OVER (PARTITION BY dept ORDER BY salary DESC) = 1;
```

Question: "What happens when you run this?"

Thinking out loud: 1. "Window functions run after WHERE in the logical query execution order — WHERE applies before windows." 2. "So putting a window function in WHERE is a syntax error in every major dialect. The planner cannot evaluate something that does not exist yet." 3. "The fix is a subquery or CTE: compute the rank first, then filter on it in the outer query." 4. "Postgres reports 'window functions are not allowed in WHERE'; SQL Server reports similar; MySQL 8 reports 'invalid use of group function'."

Answer:

```
ERROR: window functions are not allowed in WHERE
HINT: Use a subquery or CTE to filter on a window result.
```

Working fix:


```
WITH ranked AS (  
  SELECT emp_id, dept, salary,  
         RANK() OVER (PARTITION BY dept ORDER BY salary DESC) AS r  
  FROM employee_salary  
)  
SELECT emp_id, dept, salary FROM ranked WHERE r = 1;
```

Why (the non-obvious thing): The SQL logical order is FROM → WHERE → GROUP BY → HAVING → SELECT → window → ORDER BY → LIMIT. WHERE cannot reference anything computed in SELECT or window phases.

The trap: This is a daily mistake. Candidates write the natural-sounding query and are surprised the engine refuses.

What NOT to say: - "Returns the top salary per dept" — no, it errors before returning anything -
"Returns all rows because WHERE always passes" — the parser refuses to compile

Common follow-up: "Can you put a window function in HAVING?" — No, same restriction. Always wrap in a subquery or CTE.

Scenario278 · Window inside CASE WHEN

Tags: Difficulty: Medium-Hard · Asked at: Razorpay, Goldman Sachs, JP Morgan · Topic: Window functions, conditional logic

Setup:

employee_salary :

emp_id	dept	salary
1	Sales	50000
2	Sales	60000
3	Engg	80000
4	Engg	75000

```
SELECT emp_id, dept, salary,  
       CASE WHEN RANK() OVER (PARTITION BY dept ORDER BY salary DESC) = 1  
            THEN 'top'  
            ELSE 'rest'  
       END AS band  
FROM employee_salary;
```

Question: "Does this work, and what does it return?"

Thinking out loud: 1. "CASE WHEN sits inside SELECT, and window functions ARE legal inside SELECT. So this compiles." 2. "For each row, the window evaluates the rank; the CASE labels rank-1 rows as 'top' and others as 'rest'." 3. "Sales: emp 2 (60000) is top, emp 1 (50000) is rest. Engg: emp 3 (80000) is top, emp 4 (75000) is rest." 4. "Four rows, all original columns plus the band label."

Answer:

emp_id	dept	salary	band
1	Sales	50000	rest
2	Sales	60000	top
3	Engg	80000	top
4	Engg	75000	rest

Why (the non-obvious thing): Windows are legal anywhere a regular expression is legal in SELECT, including inside CASE expressions and arithmetic. The restriction is only WHERE and HAVING.

The trap: Believing windows are forbidden inside CASE. They are not. The earlier WHERE restriction is sometimes overgeneralised.

What NOT to say: - "This fails because window in CASE" — it does not - "All four rows get 'top'" — only the top-ranked row per partition does

Common follow-up: "How would you assign the label 'second' to rank 2?" — Extend the CASE with `WHEN RANK() OVER (...) = 2 THEN 'second'`. Yes, the window can be repeated.

Scenario279 · Multiple aggregates with different DISTINCT

Tags: Difficulty: Medium · Asked at: PhonePe, Swiggy, Walmart Labs · Topic: COUNT DISTINCT

Setup:

`page_views` :

user_id	page
1	/a
1	/a
1	/b
2	/a
2	/b
3	/a

```
SELECT COUNT(*) AS total_views,  
       COUNT(DISTINCT user_id) AS unique_users,  
       COUNT(DISTINCT page) AS unique_pages  
FROM page_views;
```

Question: "What does each column show?"

Thinking out loud: 1. "COUNT() counts every row — 6 rows." 2. "COUNT(DISTINCT user_id) counts unique user values: 1, 2, 3 — 3." 3. "COUNT(DISTINCT page) counts unique page values: /a and /b — 2." 4. "Three columns, one row."*

Answer:

```
total_views | unique_users | unique_pages  
6           | 3            | 2
```

Why (the non-obvious thing): Each DISTINCT inside an aggregate creates an independent dedup pass — on large data this can be expensive, since the engine must materialise distinct sets for each metric.

The trap: Five COUNT(DISTINCT ...) columns in one query can balloon memory and runtime. Consider HyperLogLog approximations (Snowflake, BigQuery) for high-cardinality counts.

What NOT to say: - "Unique_pages is 6 because COUNT() is 6" — *DISTINCT does not refer to COUNT()* - "COUNT(DISTINCT user_id, page) counts unique pairs" — only valid in some engines; not part of this query

Common follow-up: "How would you count unique (user_id, page) pairs in standard SQL?" — `COUNT(DISTINCT user_id || ' ' || page)` or via a derived table that GROUP BYs both columns.

Scenario280 · COUNT inside CASE inside SUM (conditional aggregation)

Tags: Difficulty: Medium-Hard · Asked at: Razorpay, Goldman Sachs, Walmart Labs · Topic: Conditional aggregation

Setup:

`orders` :

order_id	status	amount
1	paid	1000
2	paid	500
3	pending	800
4	cancelled	300
5	paid	1200
6	pending	400

```
SELECT SUM(CASE WHEN status = 'paid'      THEN 1 ELSE 0 END) AS paid_cnt,  
       SUM(CASE WHEN status = 'pending'   THEN amount ELSE 0 END) AS pending_amt,  
       COUNT(CASE WHEN status = 'cancelled' THEN 1 END)          AS cancelled_cnt  
FROM orders;
```

Question: "Three numbers, what are they?"

Thinking out loud: 1. "First column: SUM of 1-when-paid-else-0 gives the count of paid rows. Three paid orders → 3." 2. "Second column: SUM of amount-when-pending-else-0 gives the pending revenue. 800 + 400 = 1200." 3. "Third column: COUNT of CASE returning 1-when-cancelled-else-NULL. COUNT skips NULLs, so it counts only cancelled rows. One cancelled order → 1." 4. "Note the trick in column 3: omitting ELSE makes the result NULL for non-matching rows, and COUNT ignores NULLs. That is the canonical conditional COUNT idiom."

Answer:

```
paid_cnt | pending_amt | cancelled_cnt  
3        | 1200        | 1
```

Why (the non-obvious thing): Conditional aggregation is the pre-window-functions workhorse for pivoting data in a single pass. The COUNT-without-ELSE form is the cleanest way to count conditionally because of how COUNT handles NULL.

The trap: Writing `COUNT(CASE WHEN status = 'cancelled' THEN 1 ELSE 0 END)` accidentally counts every row — because COUNT sees both 0 and 1 as non-NULL values.

What NOT to say: - "All three return 6" — only COUNT(*) would; these are filtered - "cancelled_cnt counts all rows because of the ELSE" — there is no ELSE; that is the point

Common follow-up: "How would you do this with FILTER syntax instead?" — Postgres and SQL standard support `COUNT(*) FILTER (WHERE status = 'paid')`, which is cleaner than CASE.

Category 5, Part A — Index Design Decisions: Given This Workload

Part A is the proactive design conversation. The interviewer hands you a table definition, a list of queries with their relative frequency, and the write volume — then asks which indexes you would create and why. The trick is that almost any column can be indexed, but every index costs you write throughput, RAM in the buffer pool, and storage. The right answer is never "index everything that appears in a WHERE clause"; it is a defensible set of indexes justified by cardinality, selectivity, query frequency, and the write-amplification budget. This part trains the reasoning: walking through composite column order, recognising when a partial or expression index pays off, knowing when to skip an index because the optimiser will pick a scan anyway, and explaining the trade-off out loud so the interviewer sees the thought process — not just the DDL.

Scenario281 · Read-Heavy Lookup by Aadhaar-Linked Mobile

Tags: Difficulty: Easy · Asked at: PhonePe, Paytm, Razorpay, BharatPe · Dialect: vendor-agnostic

The setup:

```
CREATE TABLE customers (  
  customer_id BIGINT PRIMARY KEY,  
  mobile      VARCHAR(13) NOT NULL,    -- ~98% unique, +91 prefixed  
  email       VARCHAR(120),           -- ~80% populated  
  full_name   VARCHAR(120) NOT NULL,  
  kyc_status  VARCHAR(20) NOT NULL,    -- 4 distinct values  
  created_at  TIMESTAMP NOT NULL  
);  
-- 18 crore rows, growing 5 lakh/day
```

The workload: - Query 1 (92% of reads): `SELECT * FROM customers WHERE mobile = ?` — every login, ~8000 qps - Query 2 (6%): `SELECT * FROM customers WHERE email = ?` —

password reset, ~80 qps - Query 3 (2%): admin search by `full_name LIKE 'Rohit%'` - Writes: 6 inserts/sec, almost no updates after creation

The question: "What indexes would you create? Walk me through your reasoning."

Thinking out loud: 1. "Mobile is 92 percent of traffic and almost unique — selectivity is one row per lookup. This is the first index, no debate." 2. "Email at 6 percent and 80 qps still justifies its own index; the alternative is a full scan of eighteen crore rows on every password reset." 3. "Full name prefix search at 2 percent is borderline. A B-tree supports `LIKE 'Rohit%'` natively because the left-anchored prefix is a range scan. Cheap to add." 4. "Write volume is six per second — even three secondary indexes cost almost nothing on inserts. No write-amplification worry here."

The recommended indexes:

```
CREATE UNIQUE INDEX idx_customers_mobile ON customers (mobile);
CREATE INDEX idx_customers_email ON customers (email) WHERE email IS NOT NULL;
CREATE INDEX idx_customers_name ON customers (full_name);
```

Why each one: - `idx_customers_mobile` UNIQUE because mobile is the de-facto login key; uniqueness gives both lookup speed and a constraint guarantee. - `idx_customers_email` is partial (in engines that support it) because 20 percent of rows have NULL email — skipping them shrinks the index by a fifth. - `idx_customers_name` supports left-anchored LIKE; ordinary B-tree, no special trigram needed yet.

What NOT to index (and why): - `kyc_status` — only four distinct values, 4.5 crore rows per value. The optimiser will choose a scan over a 25-percent-selectivity index. - `created_at` alone — no query filters by it; an unused index is pure write tax.

Common follow-up: "Now you also need to count active customers per state — would you add a state column index?"

Scenario282 · Order Listing With Filter Plus Sort

Tags: Difficulty: Medium · Asked at: Flipkart, Myntra, Meesho, Nykaa · Dialect: Postgres

The setup:

```
CREATE TABLE orders (
  order_id      BIGSERIAL PRIMARY KEY,
  customer_id   BIGINT NOT NULL,      -- ~3 crore customers
  status        VARCHAR(20) NOT NULL, -- 8 statuses, skewed
  placed_at     TIMESTAMP NOT NULL,
  total_amount  NUMERIC(10,2) NOT NULL
);
-- 80 crore rows
```

The workload: - Query 1 (85%): "my orders" page — `WHERE customer_id = ? ORDER BY placed_at DESC LIMIT 20` - Query 2 (10%): seller dashboard — `WHERE status = 'SHIPPED' ORDER BY placed_at DESC LIMIT 100` - Writes: 4000 inserts/sec at sale peaks, status updates frequent

The question: "What indexes would you create? Walk me through your reasoning."

Thinking out loud: 1. "Eighty-five percent of traffic filters by customer and sorts by placed_at desc. A composite on (customer_id, placed_at DESC) lets the engine seek to the customer and walk the index in the desired order — no sort step." 2. "For the seller dashboard, status is low-cardinality but the SHIPPED slice is bounded. A composite (status, placed_at DESC) gives the same seek-then-walk benefit." 3. "I am intentionally not adding a single-column index on placed_at — no query uses it alone, and it would just bloat the write path." 4. "Four thousand inserts per second is meaningful. Two composites is the ceiling — a third would start hurting peak throughput."

The recommended indexes:

```
CREATE INDEX idx_orders_cust_placed
ON orders (customer_id, placed_at DESC);

CREATE INDEX idx_orders_status_placed
ON orders (status, placed_at DESC);
```

Why each one: - `idx_orders_cust_placed` — column order matters: equality column first, sort column second, DESC to match the query so the engine reads forward, not backward. - `idx_orders_status_placed` — same logic; the status slice is large but the LIMIT 100 caps the cost.

What NOT to index (and why): - `total_amount` — no query filters on it. Indexing every NUMERIC column is a junior reflex. - A separate `(placed_at)` index — redundant; the composite already covers any query that filters by customer.

Common follow-up: "Sale day hits 12000 inserts per second — do you keep both indexes?"

Scenario283 · Transaction Date Range With Account Filter

Tags: Difficulty: Medium · Asked at: HDFC, ICICI, Axis, Yes Bank · Dialect: SQL Server

The setup:

```
CREATE TABLE txn_ledger (
  txn_id          BIGINT IDENTITY PRIMARY KEY,
  account_no      VARCHAR(18) NOT NULL,
  txn_date        DATE NOT NULL,
  amount          DECIMAL(15,2) NOT NULL,
  txn_type        CHAR(2) NOT NULL,      -- DR / CR
  branch_code     CHAR(5) NOT NULL
);
-- 240 crore rows, 5 years of history
```

The workload: - Query 1 (78%): passbook —

`WHERE account_no = ? AND txn_date BETWEEN ? AND ?` - Query 2 (12%): RBI report —
`WHERE txn_date BETWEEN ? AND ? AND branch_code = ?` - Writes: 1500 inserts/sec, append-only

The question: "What indexes would you create? Walk me through your reasoning."

Thinking out loud: 1. "Passbook is the hot query. The right composite is (account_no, txn_date) — equality first, range second. A range column anywhere except the last position of a composite blocks the index from being used for filters past it." 2. "The RBI report flips the priority — branch_code is the equality, txn_date is the range. Different composite needed: (branch_code, txn_date)." 3. "I considered one index (txn_date, account_no) to serve both, but the date-first ordering forces a wide range scan even for the passbook query." 4. "Append-only writes are cheap to index — date columns produce sequential leaf inserts, no page splits."

The recommended indexes:

```
CREATE INDEX IX_txn_account_date ON txn_ledger (account_no, txn_date);
CREATE INDEX IX_txn_branch_date  ON txn_ledger (branch_code, txn_date);
```

Why each one: - `IX_txn_account_date` — equality on account, range on date. The classic "constants first, ranges last" rule. - `IX_txn_branch_date` — same pattern for the report; cannot be folded into the first index because no query filters by account AND branch.

What NOT to index (and why): - `amount` — analysts aggregate it, never filter by it. The aggregate scans the table anyway because the report covers a date range. - `txn_type` — two values; the optimiser will scan.

Common follow-up: "Can you make the passbook query *INCLUDE* amount and txn_type to make it a covering index?"

Scenario284 · Multi-Tenant SaaS Query

Tags: Difficulty: Medium · Asked at: Freshworks, Zoho, Chargebee, Postman · Dialect: Postgres

The setup:

```
CREATE TABLE tickets (  
  ticket_id    BIGSERIAL PRIMARY KEY,  
  tenant_id    INT NOT NULL,           -- 12000 tenants, Zipf distribution  
  status       VARCHAR(20) NOT NULL,  
  priority     VARCHAR(10) NOT NULL,  
  assigned_to  BIGINT,  
  created_at   TIMESTAMP NOT NULL  
);  
-- 50 crore rows
```

The workload: - Query 1 (60%): tenant dashboard — `WHERE tenant_id = ? AND status = 'OPEN' ORDER BY created_at DESC LIMIT 50` - Query 2 (25%): agent inbox — `WHERE tenant_id = ? AND assigned_to = ?` - Query 3 (15%): priority view — `WHERE tenant_id = ? AND priority = 'HIGH'` - Writes: 600 inserts/sec, status flips often

The question: "What indexes would you create? Walk me through your reasoning."

Thinking out loud: 1. "Multi-tenant golden rule: `tenant_id` is always the leading column. It is the cheapest, most selective filter, and it gives row-level tenant isolation as a side effect." 2. "Three different second-position columns — `status`, `assigned_to`, `priority`. Each query justifies its own composite." 3. "For the dashboard, I'd add `created_at` as a third column to support the sort without a separate sort step." 4. "Six hundred writes per second times three indexes is manageable on a tuned Postgres box."

The recommended indexes:

```
CREATE INDEX idx_tickets_tenant_status_created  
  ON tickets (tenant_id, status, created_at DESC);  
  
CREATE INDEX idx_tickets_tenant_assignee  
  ON tickets (tenant_id, assigned_to);  
  
CREATE INDEX idx_tickets_tenant_priority  
  ON tickets (tenant_id, priority);
```

Why each one: - The dashboard index is three columns — (`tenant`, `status`, `created_at DESC`) — so the `LIMIT 50` stops the index scan after fifty leaf entries. - The agent inbox composite serves the lookup directly. - The priority composite is small enough to live in the buffer pool.

What NOT to index (and why): - A single-column (`tenant_id`) index — every composite above already has `tenant_id` as the leading column. - `ticket_id` alone — already the PK; the implicit B-tree is enough.

Common follow-up: "One tenant has 40 percent of all rows — does your strategy still hold?"

Scenario285 · Active-Subset Filter Tempting a Partial Index

Tags: Difficulty: Medium · Asked at: Swiggy, Zomato, Dunzo · Dialect: Postgres

The setup:

```
CREATE TABLE delivery_jobs (  
  job_id          BIGSERIAL PRIMARY KEY,  
  rider_id        BIGINT,  
  status          VARCHAR(15) NOT NULL,    -- 6 values  
  is_active        BOOLEAN NOT NULL,       -- only ~2% are TRUE  
  city_id         INT NOT NULL,  
  assigned_at     TIMESTAMP  
);  
-- 12 crore rows
```

The workload: - Query 1 (95%): dispatcher map —

`WHERE is_active = TRUE AND city_id = ?` - Query 2 (5%): historical lookup — `WHERE job_id = ?` - Writes: 800 status flips/sec; `is_active` goes TRUE on assignment, FALSE on delivery

The question: "What indexes would you create? Walk me through your reasoning."

Thinking out loud: 1. "`is_active` is a textbook partial-index candidate. Two percent of twelve crore is twenty-four lakh rows — a partial index on just the TRUE slice is fifty times smaller than a full index." 2. "Combining the partial with `city_id` makes the dispatcher map query a tiny scan on a tiny index." 3. "The 5-percent historical lookup hits the PK directly — no extra index needed." 4. "Write amplification is bounded: only the TRUE-to-FALSE flip removes from the partial index; updates inside the partial slice are normal."

The recommended indexes:

```
CREATE INDEX idx_jobs_active_city  
ON delivery_jobs (city_id)  
WHERE is_active = TRUE;
```

Why each one: - The partial WHERE clause makes the index store only twenty-four lakh entries instead of twelve crore. The B-tree fits entirely in `shared_buffers` for any reasonable box. - `city_id` leads because the query always equality-filters on it.

What NOT to index (and why): - A full index on `(is_active, city_id)` — wastes space on the 98 percent of rows the dispatcher never looks at. - `assigned_at` — no query uses it as a predicate today.

Common follow-up: "Postgres `VACUUM` runs every hour. Does the partial index need `REINDEX` more often?"

Scenario286 · Case-Insensitive Name Search

Tags: Difficulty: Medium · Asked at: Naukri, Shine, Foundit, Apna · Dialect: Postgres / MySQL

The setup:

```
CREATE TABLE candidates (  
  candidate_id BIGSERIAL PRIMARY KEY,  
  full_name    VARCHAR(120) NOT NULL,  
  email        VARCHAR(120) NOT NULL,  
  skill_tags   TEXT  
);  
-- 4 crore rows
```

The workload: - Query 1 (70%): recruiter search — `WHERE LOWER(full_name) = LOWER(?)` -
Query 2 (30%): `WHERE email = ?` - Writes: 300 inserts/sec, profile updates rare

The question: "What indexes would you create? Walk me through your reasoning."

Thinking out loud: 1. "A plain index on `full_name` will never be used because the query wraps it in `LOWER()`. The expression on the column side blocks index usage — this is the classic SARGability trap." 2. "Two options: store a lower-cased shadow column, or use an expression index. Postgres supports expression indexes natively; for MySQL 8 we'd use a generated column." 3. "Email is straightforward — a unique B-tree." 4. "Three hundred writes per second is trivial."

The recommended indexes (Postgres):

```
CREATE INDEX idx_candidates_name_lower  
  ON candidates (LOWER(full_name));  
  
CREATE UNIQUE INDEX idx_candidates_email  
  ON candidates (email);
```

For MySQL 8+:

```
ALTER TABLE candidates  
  ADD COLUMN full_name_lower VARCHAR(120) AS (LOWER(full_name)) STORED,  
  ADD INDEX idx_candidates_name_lower (full_name_lower);
```

Why each one: - The expression index stores the lower-cased form so the optimiser matches `WHERE LOWER(col) = ?` directly. - The MySQL variant uses a STORED generated column because pre-8.0.13 versions cannot index expressions inline.

What NOT to index (and why): - A plain `(full_name)` — unused. The application is committed to case-insensitive search. - `skill_tags` — long free-form TEXT; needs full-text or trigram, not a B-tree (covered later).

Common follow-up: "What if the query was `LOWER(full_name) LIKE 'rohit%'` instead?"

Scenario287 · Foreign Key Backing for Join and Cascade

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Mindtree · Dialect: vendor-agnostic

The setup:

```
CREATE TABLE invoices (  
    invoice_id BIGSERIAL PRIMARY KEY,  
    customer_id BIGINT NOT NULL REFERENCES customers(customer_id) ON DELETE CASCADE,  
    amount NUMERIC(12,2) NOT NULL,  
    invoice_dt DATE NOT NULL  
);  
-- 9 crore invoices, 40 lakh customers
```

The workload: - Query 1 (50%): `SELECT * FROM invoices WHERE customer_id = ?` - Query 2 (40%): join — `customers c JOIN invoices i ON i.customer_id = c.customer_id WHERE c.email = ?` - Query 3 (10%): GDPR delete — `DELETE FROM customers WHERE customer_id = ?` (cascades) - Writes: 2000 invoice inserts/sec

The question: "What indexes would you create? Walk me through your reasoning."

Thinking out loud: 1. "MySQL InnoDB auto-creates an index on every FK column, but Postgres and SQL Server do not. The interviewer expects me to know this gap and fix it explicitly." 2. "Without an FK index, the ON DELETE CASCADE has to scan invoices for every customer delete — a full nine-crore scan in the worst case." 3. "The same index serves Query 1 and Query 2 — the planner uses it as the inner side of the hash or nested-loop join." 4. "Two thousand writes per second tolerates one extra index easily."

The recommended indexes:

```
CREATE INDEX idx_invoices_customer ON invoices (customer_id);
```

Why each one: - `idx_invoices_customer` — supports lookup, join, and cascade delete with one structure.

What NOT to index (and why): - `invoice_dt` alone — no query filters by date yet. Premature. - `amount` — analyst aggregates only; no equality filter.

Common follow-up: "What's the cost of forgetting an FK index in Postgres when you run a DELETE CASCADE on five thousand parent rows?"

Scenario288 · Covering Index for the Hot Product Card Query

Tags: Difficulty: Hard · Asked at: Flipkart, Amazon India, Tata CLiQ, Reliance Digital · Dialect: SQL Server / Postgres

The setup:

```
CREATE TABLE products (  
  product_id BIGINT PRIMARY KEY,  
  sku        VARCHAR(40) NOT NULL,  
  title      VARCHAR(255) NOT NULL,  
  price      NUMERIC(10,2) NOT NULL,  
  mrp        NUMERIC(10,2) NOT NULL,  
  category_id INT NOT NULL,  
  is_listed  BOOLEAN NOT NULL,  
  rating     NUMERIC(2,1)  
);  
-- 6 crore rows
```

The workload: - Query 1 (88%): product card — `SELECT product_id, title, price, mrp, rating FROM products WHERE category_id = ? AND is_listed = TRUE ORDER BY rating DESC LIMIT 24` - Query 2 (12%): admin edit — full row by product_id - Writes: 150 updates/sec to price

The question: "What indexes would you create? Walk me through your reasoning."

Thinking out loud: 1. "Product card is eighty-eight percent of reads and demands a fast scan. A composite (category_id, is_listed, rating DESC) supports the filter and the sort — but the engine still has to fetch title/price/mrp from the heap. That's one random I/O per result row times twenty-four." 2. "A covering index includes the projected columns so the query is satisfied entirely from the index. SQL Server: INCLUDE clause. Postgres 11+: INCLUDE clause." 3. "Price updates are the cost — every price change writes both the heap and the covering index. One-fifty per second is fine." 4. "is_listed could be folded into a partial index instead of a column."

The recommended indexes (SQL Server):

```
CREATE INDEX IX_products_category_listed_rating  
ON products (category_id, rating DESC)  
INCLUDE (title, price, mrp)  
WHERE is_listed = 1;
```

Postgres equivalent:

```
CREATE INDEX idx_products_category_rating
ON products (category_id, rating DESC)
INCLUDE (title, price, mrp)
WHERE is_listed = TRUE;
```

Why each one: - The composite serves the filter and the sort; the INCLUDE columns make it an index-only scan; the partial WHERE keeps the index lean.

What NOT to index (and why): - A separate index on `sku` — not in the hot query path; add only if there's a sku lookup endpoint. - `mrp` as a key column — it appears in projection, not predicate.

Common follow-up: "Price updates tripled to 450 per second — would you still keep price in INCLUDE?"

Scenario289 · Keyset Pagination Replacing OFFSET

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Groww, Zerodha · Dialect: Postgres

The setup:

```
CREATE TABLE wallet_txns (
  txn_id      BIGSERIAL PRIMARY KEY,
  user_id     BIGINT NOT NULL,
  amount      NUMERIC(12,2) NOT NULL,
  created_at  TIMESTAMP NOT NULL
);
-- 40 crore rows
```

The workload: - Query 1 (100%): wallet history with infinite scroll — `WHERE user_id = ? AND (created_at, txn_id) < (?, ?) ORDER BY created_at DESC, txn_id DESC LIMIT 30` - Writes: 1200 inserts/sec, append-only

The question: "What indexes would you create? Walk me through your reasoning."

Thinking out loud: 1. "Keyset pagination uses a composite cursor — `(created_at, txn_id)` — to avoid OFFSET. The index must match the sort order column-for-column." 2. "Composite: `(user_id, created_at DESC, txn_id DESC)`. User equality first, then the cursor columns in the exact sort order." 3. "Both sort columns are DESC. Engines that store ascending B-trees read them backward, but the explicit DESC declaration ensures forward-only scans which are cache-friendly." 4. "Twelve hundred sequential inserts per second is the easy end of the spectrum — `created_at` increases monotonically, so leaf inserts are append-friendly."

The recommended indexes:

```
CREATE INDEX idx_wallet_user_cursor
ON wallet_txns (user_id, created_at DESC, txn_id DESC);
```

Why each one: - Three-column composite, exactly matching the WHERE-plus-ORDER-BY shape. Each page is a thirty-row index range scan — constant time regardless of how deep the user scrolls.

What NOT to index (and why): - A separate `(created_at)` — global `created_at` queries do not exist in this workload. - An OFFSET-friendly index — keyset is the goal; OFFSET-based pagination is what we're replacing.

Common follow-up: *"User asks for the millionth page — how does keyset compare to OFFSET in execution time?"*

Scenario290 · Top-N Per Group Using Lateral Join

Tags: Difficulty: Hard · Asked at: Hotstar, JioCinema, Sony LIV, ZEE5 · Dialect: Postgres

The setup:

```
CREATE TABLE video_views (
  view_id      BIGSERIAL PRIMARY KEY,
  user_id      BIGINT NOT NULL,
  video_id     BIGINT NOT NULL,
  watched_at   TIMESTAMP NOT NULL,
  duration_s   INT NOT NULL
);
-- 220 crore rows
```

The workload: - Query 1 (100%): "continue watching" — for one user, return the latest watch per video, top 10 videos - Writes: 9000 view inserts/sec at prime time

The question: *"What indexes would you create? Walk me through your reasoning."*

Thinking out loud: 1. "Top-N per group is solved with a lateral join: for each video the user has watched, fetch the latest row. The index that makes this cheap is `(user_id, video_id, watched_at DESC)`." 2. "The lateral subquery does a one-row LIMIT — the engine seeks the composite, lands on the latest row for `(user, video)`, returns." 3. "Composite column order: user equality (very selective), then video equality, then `watched_at DESC` for the limit-1 seek." 4. "Nine thousand writes per second is heavy, but one index on the hot path is acceptable — sub-second insert latency holds."

The recommended indexes:

```
CREATE INDEX idx_views_user_video_watched
ON video_views (user_id, video_id, watched_at DESC);
```

Why each one: - The three-column composite supports the lateral join with a single index seek per video. Without watched_at in the index, the engine has to scan all rows per (user, video) and sort.

What NOT to index (and why): - A (video_id, watched_at) index — would serve "trending video" queries, but that's a different workload not on this list. - duration_s — analytics column, no predicate use.

Common follow-up: "Now you also need 'top 100 videos this hour overall' — what changes?"

Scenario291 · Most Recent N Per User Without DISTINCT

Tags: Difficulty: Hard · Asked at: Truecaller, Inshorts, DailyHunt, Moj · Dialect: Postgres

The setup:

```
CREATE TABLE notifications (
  notif_id BIGSERIAL PRIMARY KEY,
  user_id BIGINT NOT NULL,
  channel VARCHAR(20) NOT NULL,    -- push/email/sms/in_app
  sent_at  TIMESTAMP NOT NULL,
  payload  JSONB
);
-- 90 crore rows
```

The workload: - Query 1 (100%): notification bell — last 50 notifs per user, ordered by sent_at DESC - Writes: 5000 inserts/sec, no updates

The question: "What indexes would you create? Walk me through your reasoning."

Thinking out loud: 1. "The query is WHERE user_id = ? ORDER BY sent_at DESC LIMIT 50 — composite (user_id, sent_at DESC) handles it." 2. "No need for a window function or DISTINCT — the index is the answer to the access pattern." 3. "Column order is rigid: user_id equality leads, sent_at DESC supports the sort and LIMIT." 4. "Five thousand inserts per second on one composite is fine; sent_at is monotonic so leaf inserts are at the right edge."

The recommended indexes:

```
CREATE INDEX idx_notif_user_sent
ON notifications (user_id, sent_at DESC);
```


Why each one: - Equality-first, sort-second, DESC matched to the query. Reading fifty index entries returns fifty rows; no sort step, no extra scan.

What NOT to index (and why): - A `(channel)` index — channel never appears in WHERE. - A GIN on payload — until the JSONB is queried, indexing it is wasted write cost.

Common follow-up: "Add a filter `WHERE channel = 'push'` — does the existing index still work?"

Scenario292 · Geographic Range — Lat-Long Search

Tags: Difficulty: Hard · Asked at: Ola, Uber India, Rapido, Yulu · Dialect: Postgres + PostGIS

The setup:

```
CREATE TABLE driver_pings (  
  ping_id      BIGSERIAL PRIMARY KEY,  
  driver_id    BIGINT NOT NULL,  
  lat          DOUBLE PRECISION NOT NULL,  
  lon          DOUBLE PRECISION NOT NULL,  
  is_online    BOOLEAN NOT NULL,  
  pinged_at    TIMESTAMP NOT NULL  
);  
-- 14 crore rows, rolling 24h window
```

The workload: - Query 1 (100%): "find drivers within 3 km of (lat, lon) who are online" — 2000 qps -
Writes: 25000 ping inserts/sec

The question: "What indexes would you create? Walk me through your reasoning."

Thinking out loud: 1. "A B-tree on lat or lon cannot answer a bounding-box query. We need a spatial index — GiST in Postgres with PostGIS, or SP-GiST for point-only workloads." 2. "Add a generated `geom POINT` column and a GiST index on it. The `ST_DWithin` function uses the spatial index for the bounding-box pre-filter." 3. "is_online is a 50/50 split — a partial GiST index on the TRUE slice cuts the search space in half." 4. "Write rate is high. GiST inserts are more expensive than B-tree, so I'd keep the index narrow: only geom, with the partial WHERE."

The recommended indexes:

```
ALTER TABLE driver_pings  
  ADD COLUMN geom geometry(Point, 4326)  
  GENERATED ALWAYS AS (ST_SetSRID(ST_MakePoint(lon, lat), 4326)) STORED;  
  
CREATE INDEX idx_pings_geom_online  
  ON driver_pings USING GIST (geom)  
  WHERE is_online = TRUE;
```

Why each one: - GiST is the right access method for 2D range queries — it implements the bounding-box decomposition. - The partial WHERE keeps the spatial index focused on the queryable slice.

What NOT to index (and why): - B-tree on lat or lon individually — useless for radius search. - driver_id — no lookup-by-driver query in this workload.

Common follow-up: "Why GiST and not SP-GiST or BRIN for this workload?"

Scenario293 · Full-Text Search Over Course Catalogue

Tags: Difficulty: Medium · Asked at: BYJU's, Vedantu, Unacademy, PhysicsWallah · Dialect: Postgres

The setup:

```
CREATE TABLE courses (  
  course_id BIGSERIAL PRIMARY KEY,  
  title VARCHAR(255) NOT NULL,  
  description TEXT,  
  category VARCHAR(50) NOT NULL,  
  language VARCHAR(20) NOT NULL  
);  
-- 4 lakh rows, mostly Hindi/English
```

The workload: - Query 1 (80%): search bar —

`WHERE to_tsvector('simple', title || ' ' || description) @@ plainto_tsquery(?)` - Query 2 (20%): `WHERE category = ? AND language = ?` - Writes: 100 updates/day

The question: "What indexes would you create? Walk me through your reasoning."

Thinking out loud: 1. "Full-text search needs an inverted index. Postgres ships GIN for this — fast lookups, slower writes, ideal for low-write tables like a catalogue." 2. "The tsvector should be a stored generated column so the GIN doesn't recompute it on every query." 3. "For category-plus-language browse, a composite B-tree is enough — both columns are low cardinality but together they're selective." 4. "Write volume is nothing — one hundred updates per day. GIN tax is invisible."

The recommended indexes:

```
ALTER TABLE courses  
  ADD COLUMN search_doc tsvector  
  GENERATED ALWAYS AS (  
    to_tsvector('simple', coalesce(title, '') || ' ' || coalesce(description, ''))  
  ) STORED;  
  
CREATE INDEX idx_courses_fts ON courses USING GIN (search_doc);  
CREATE INDEX idx_courses_cat_lang ON courses (category, language);
```

Why each one: - GIN over a stored tsvector handles the search-bar query without recomputation. - The composite serves the filter-only browse path.

What NOT to index (and why): - `title` as a B-tree — left-anchored LIKE is a fraction of the workload, not the main pattern. - Separate single-column indexes on category and language — the composite covers both.

Common follow-up: *"Hindi search returns nothing — what configuration knob did you forget?"*

Scenario294 · JSONB Path Frequently Queried

Tags: Difficulty: Hard · Asked at: Cred, Razorpay, Setu, Decentro · Dialect: Postgres

The setup:

```
CREATE TABLE webhook_events (  
  event_id BIGSERIAL PRIMARY KEY,  
  source   VARCHAR(40) NOT NULL,  
  payload  JSONB NOT NULL,  
  received_at TIMESTAMP NOT NULL  
);  
-- 30 crore rows
```

The workload: - Query 1 (60%): debugging — `WHERE payload ->> 'merchant_id' = ?` - Query 2 (40%): replay — `WHERE source = ? AND received_at > ?` - Writes: 3500 events/sec

The question: *"What indexes would you create? Walk me through your reasoning."*

Thinking out loud: 1. "Two ways to index a JSON path: an expression B-tree on `(payload->>'merchant_id')` for one specific path, or a GIN on the whole document for any key." 2. "Sixty percent of traffic uses one specific key. Expression B-tree is leaner — single key, B-tree speed, partial supported." 3. "For replay, a composite (source, received_at) is the standard answer — equality leads, range trails." 4. "Thirty-five hundred writes per second is heavy. Two indexes — one expression, one composite — is the right ceiling."

The recommended indexes:

```
CREATE INDEX idx_webhook_merchant  
  ON webhook_events ((payload ->> 'merchant_id'))  
  WHERE payload ? 'merchant_id';  
  
CREATE INDEX idx_webhook_source_time  
  ON webhook_events (source, received_at);
```

Why each one: - Expression B-tree is smaller and faster than GIN for one known key. - The partial WHERE skips rows that don't have the key at all. - Composite for replay follows the equality-first rule.

What NOT to index (and why): - A blanket `GIN (payload)` — useful only if the access pattern is unknown; here it isn't. - `event_id` separately — already the PK.

Common follow-up: "Now five different keys are queried regularly — switch to GIN or add five expression indexes?"

Scenario295 · Append-Only Large Table — BRIN Candidate

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan, Wells Fargo, AmEx GBS · Dialect: Postgres

The setup:

```
CREATE TABLE access_logs (  
  log_id      BIGSERIAL PRIMARY KEY,  
  user_id     BIGINT NOT NULL,  
  endpoint    VARCHAR(120) NOT NULL,  
  status_code SMALLINT NOT NULL,  
  logged_at   TIMESTAMP NOT NULL  
);  
-- 1200 crore rows, 18 months
```

The workload: - Query 1 (95%): "show last 24 hours of errors" — `WHERE logged_at BETWEEN ? AND ? AND status_code >= 500` - Query 2 (5%): rare user audit — `WHERE user_id = ? AND logged_at > ?` - Writes: 40000 inserts/sec, append-only, never updated

The question: "What indexes would you create? Walk me through your reasoning."

Thinking out loud: 1. "Twelve hundred crore rows with append-only inserts and time-correlated access is a textbook BRIN scenario. BRIN stores min/max per block range — a one-thousandth the size of a B-tree on the same column." 2. "BRIN is read-efficient only when the column is physically correlated with insert order — `logged_at` is, because we never backfill." 3. "For the user audit path, a B-tree on (`user_id`, `logged_at`) is small enough — five percent of traffic but unbounded scan otherwise." 4. "Forty thousand inserts per second is exactly where B-tree maintenance starts hurting — BRIN's near-zero insert cost is the lever."

The recommended indexes:

```
CREATE INDEX idx_access_logged_at_brin  
  ON access_logs USING BRIN (logged_at) WITH (pages_per_range = 32);  
  
CREATE INDEX idx_access_user_time  
  ON access_logs (user_id, logged_at);
```

Why each one: - BRIN is right when the data is naturally ordered on disk by the indexed column. - pages_per_range = 32 tightens the granularity; the default 128 leaves too many false positives on the hot 24-hour window. - The user composite handles the rare audit path without bloating the write path.

What NOT to index (and why): - `status_code` standalone — twenty distinct codes, the optimiser scans inside the BRIN-narrowed block range. - `endpoint` — high cardinality but no predicate use.

Common follow-up: "Now you're asked to find all 500s in the past hour for a specific endpoint — does BRIN still hold?"

Scenario296 · Bulk Load Plus Reporting — When to Index

Tags: Difficulty: Medium · Asked at: Walmart Labs, Target, AmEx, Wells Fargo · Dialect: Postgres / SQL Server

The setup:

```
CREATE TABLE sales_load_staging (  
  txn_id      BIGINT,  
  store_id    INT,  
  sku         VARCHAR(40),  
  qty         INT,  
  amount      NUMERIC(10,2),  
  sale_date   DATE  
);  
-- Nightly load: 8 crore rows in 90 minutes
```

The workload: - Load (00:00-01:30): bulk COPY/BULK INSERT - Reports (08:00-23:59): `WHERE store_id = ? AND sale_date = ?`, plus `WHERE sku = ?` aggregations - After 7 days: data moves to the warehouse

The question: "What indexes would you create? Walk me through your reasoning."

Thinking out loud: 1. "Indexes during a bulk load slow inserts dramatically — sometimes by 10x. The standard trick: load with no indexes, then create them after the load completes." 2. "For reports, two composites: (store_id, sale_date) and (sku, sale_date)." 3. "This table lives for seven days. Index maintenance over that window pays for itself many times in report speed." 4. "Postgres CREATE INDEX takes a strong lock; CREATE INDEX CONCURRENTLY is reader-safe but takes longer — for a load window, the plain CREATE INDEX is fine."

The recommended indexes (created after the load):

```
CREATE INDEX idx_staging_store_date ON sales_load_staging (store_id, sale_date);  
CREATE INDEX idx_staging_sku_date   ON sales_load_staging (sku, sale_date);
```

Why each one: - Both composites follow the equality-first rule for the reporting predicates. - Building after the load amortises the index-construction sort across all rows once, not per-row.

What NOT to index (and why): - `amount` , `qty` — projection only, no predicates. - `txn_id` — no point lookup by txn in reporting.

Common follow-up: "Reports started at 06:00 instead of 08:00 — would you still drop and recreate?"

Scenario297 · Mixed OLTP and Reporting on One Table

Tags: Difficulty: Hard · Asked at: Cred, Razorpay, Groww, Zerodha · Dialect: Postgres

The setup:

```
CREATE TABLE trades (  
  trade_id    BIGSERIAL PRIMARY KEY,  
  user_id     BIGINT NOT NULL,  
  symbol      VARCHAR(20) NOT NULL,  
  qty         INT NOT NULL,  
  price       NUMERIC(12,2) NOT NULL,  
  traded_at   TIMESTAMP NOT NULL  
);  
-- 35 crore rows
```

The workload: - OLTP (75%): `WHERE user_id = ? AND traded_at > ?` — portfolio screen - OLTP (15%): `WHERE trade_id = ?` — receipt view - Reporting (10%): nightly — `SELECT symbol, SUM(qty * price) FROM trades WHERE traded_at >= ? GROUP BY symbol` - Writes: 6000 trades/sec at market hours

The question: "What indexes would you create? Walk me through your reasoning."

Thinking out loud: 1. "OLTP portfolio screen is the hot path — composite (`user_id`, `traded_at DESC`)." 2. "Receipt view hits the PK directly. Already covered." 3. "Reporting at ten percent and once a night does not justify a dedicated index. A sequential scan over the date range is acceptable for a batch job." 4. "Six thousand inserts per second is moderately heavy — one OLTP-tuned composite is the right ceiling."

The recommended indexes:

```
CREATE INDEX idx_trades_user_time  
ON trades (user_id, traded_at DESC);
```

Why each one: - Single composite serves the hot OLTP path. - Report runs at night when load is low; a parallel sequential scan over the relevant partition is fine.

What NOT to index (and why): - `symbol` alone — only the report uses it, and it groups not filters by it.
- A separate `traded_at` — the composite covers any user-bounded date filter.

Common follow-up: "What if reporting moved to real-time intraday — would you add a symbol index?"

Scenario298 · Sharded Table — Local vs Global Indexes

Tags: Difficulty: Hard · Asked at: Flipkart, Hotstar, Swiggy, Zomato · Dialect: vendor-agnostic / Citus

The setup:

```
-- Sharded by user_id across 32 nodes
CREATE TABLE messages (
  message_id BIGINT PRIMARY KEY,      -- snowflake
  user_id    BIGINT NOT NULL,         -- shard key
  conversation_id BIGINT NOT NULL,
  body       TEXT,
  sent_at    TIMESTAMP NOT NULL
);
-- 200 crore rows total
```

The workload: - Query 1 (80%): `WHERE user_id = ? AND sent_at > ?` — single-shard - Query 2 (15%): `WHERE message_id = ?` — by snowflake; cross-shard - Query 3 (5%): `WHERE conversation_id = ?` — usually crosses 2 shards - Writes: 30000 messages/sec

The question: "What indexes would you create? Walk me through your reasoning."

Thinking out loud: 1. "With `user_id` as the shard key, the 80-percent query is one-shard — a local composite (`user_id, sent_at DESC`) on each shard works." 2. "`message_id` lookups have to scatter-gather without a global index. Two options: pay the scatter, or add a routing table (`message_id` → shard). Most teams do the latter." 3. "`conversation_id` queries cross shards by design. A local index on each shard, queried in parallel by the coordinator, is the pragmatic answer." 4. "True global indexes in distributed Postgres (Citus) come with two-phase commit cost on every insert — usually a hard no for 30k writes per second."

The recommended indexes (per shard):

```
CREATE INDEX idx_messages_user_sent ON messages (user_id, sent_at DESC);
CREATE INDEX idx_messages_conv     ON messages (conversation_id);
-- message_id: rely on PK + routing table; no shard-level secondary
```

Why each one: - The local composite makes the hot 80-percent query a single-shard B-tree seek. - The conversation index lives on every shard; coordinator fans out and merges.

What NOT to index (and why): - A global secondary on message_id — write amplification across 32 shards per insert. - `sent_at` standalone — same composite already covers it.

Common follow-up: "How does the routing table get kept in sync with inserts?"

Scenario299 · Time-Series With Rollup Tables

Tags: Difficulty: Hard · Asked at: Hotstar, Hopscotch, Apna, Inshorts · Dialect: Postgres / Timescale

The setup:

```
CREATE TABLE metrics_raw (  
  metric_id BIGSERIAL PRIMARY KEY,  
  service   VARCHAR(40) NOT NULL,  
  metric_name VARCHAR(60) NOT NULL,  
  value     DOUBLE PRECISION NOT NULL,  
  ts        TIMESTAMP NOT NULL  
);  
-- 800 crore rows  
  
CREATE TABLE metrics_hourly (  
  service   VARCHAR(40),  
  metric_name VARCHAR(60),  
  hour_bucket TIMESTAMP,  
  avg_value DOUBLE PRECISION,  
  p99_value DOUBLE PRECISION,  
  PRIMARY KEY (service, metric_name, hour_bucket)  
);
```

The workload: - Query 1 (90%): dashboard — `WHERE service = ? AND metric_name = ? AND hour_bucket > ?` against hourly - Query 2 (10%): drill-down — same predicates against raw, for last 1 hour only - Writes: 50000 raw inserts/sec; hourly rollup runs once an hour

The question: "What indexes would you create? Walk me through your reasoning."

Thinking out loud: 1. "Raw at 50k inserts/sec — every secondary index is expensive. Skip secondaries; use BRIN on ts and rely on the rollup for the hot path." 2. "Hourly is read-heavy; a B-tree composite on (service, metric_name, hour_bucket) — already the PK, so no extra work." 3. "Drill-down on raw is one-hour windowed — BRIN on ts narrows to a few blocks, then a residual filter on service plus metric_name is cheap." 4. "The architectural answer is: indexes on the cold/aggregated table, BRIN on the hot/raw table."

The recommended indexes:

```
CREATE INDEX idx_raw_ts_brin ON metrics_raw USING BRIN (ts);  
-- metrics_hourly already has a (service, metric_name, hour_bucket) PK
```


Why each one: - BRIN on the raw table keeps insert cost flat at 50k/sec. - The hourly PK is the dashboard's index — no extra structure needed.

What NOT to index (and why): - (service, metric_name) on raw — would 4x the insert cost and is only used for a tiny drill-down slice. - Any B-tree on `value` — analysts aggregate, never filter.

Common follow-up: "What if drill-down expands to one day instead of one hour?"

Scenario300 · B-tree vs Hash vs GIN — Picking the Access Method

Tags: Difficulty: Medium · Asked at: TCS Digital, Infosys Lex, Wipro Topcoder, Capgemini Engineering · Dialect: Postgres

The setup:

```
CREATE TABLE pincode_meta (  
  pincode      CHAR(6) PRIMARY KEY,  
  city         VARCHAR(60),  
  state        VARCHAR(40),  
  served_by    TEXT[],           -- array of courier names  
  metadata     JSONB  
);  
-- 1.5 lakh rows
```

The workload: - Query 1: `WHERE pincode = ?` — exact equality, 95% - Query 2: `WHERE 'Delhivery' = ANY(served_by)` — array membership, 4% - Query 3: `WHERE metadata @> '{"hub":"BLR"}'` — JSON containment, 1% - Writes: 50/day

The question: "What indexes would you create? Walk me through your reasoning."

Thinking out loud: 1. "Pincode is the PK; the default B-tree handles exact equality fine. No need for a hash index — hash B-trees in Postgres are now WAL-logged but only marginally faster on exact equality and useless for ranges." 2. "Array membership and JSON containment are GIN territory. GIN inverts the composite into key→row mappings, which makes ANY/@@/@> queries fast." 3. "Two GIN indexes on a 1.5-lakh row low-write table is essentially free." 4. "Avoid the rookie answer of 'hash index for equality, B-tree for range' — modern B-trees handle both and are the default for good reason."

The recommended indexes:

```
CREATE INDEX idx_pincode_served ON pincode_meta USING GIN (served_by);  
CREATE INDEX idx_pincode_meta_gin ON pincode_meta USING GIN (metadata jsonb_path_ops);
```

Why each one: - GIN over the text array supports ANY/@> semantics directly. - GIN with jsonb_path_ops is the smaller, faster variant for @> containment lookups specifically.

What NOT to index (and why): - Hash on `pincode` — PK B-tree already serves; hash buys nothing. - B-tree on `metadata` — JSON columns are inherently non-orderable for predicate purposes.

Common follow-up: *"Why jsonb_path_ops and not the default jsonb_ops?"*

Scenario301 · Existing Index Inventory — Which to Drop

Tags: Difficulty: Hard · Asked at: Wells Fargo, JP Morgan, Goldman, AmEx GBS · Dialect: Postgres

The setup:

```
-- Existing indexes on a 6 crore row 'employees' table:
-- ix1: (department_id)
-- ix2: (department_id, designation)
-- ix3: (department_id, designation, location)
-- ix4: (designation)
-- ix5: (joining_date)
-- ix6: (manager_id) — used by FK cascade
-- ix7: (LOWER(email)) — used by login
```

The workload audit shows: - ix1: 0 scans in 30 days - ix2: 12 scans - ix3: 4 lakh scans - ix4: 80000 scans - ix5: 0 scans - ix6: 200 scans (cascade) - ix7: 9 lakh scans

The question: *"What indexes would you create? Walk me through your reasoning."*

Thinking out loud: 1. *"ix1 is a strict prefix of ix3 and unused — drop it. The optimiser can use ix3 for any query that would have used ix1."* 2. *"ix2 has only 12 scans and is also a prefix of ix3 — drop ix2 unless the 12 scans are a critical low-frequency-but-high-value query."* 3. *"ix5 is zero scans — drop. Six crore rows of B-tree maintenance for nothing."* 4. *"ix6 is FK-cascade backing — must stay even with low scan count, otherwise DELETE on parent becomes a sequential scan."* 5. *"ix4 is independent — different column, used; keep. ix7 is the login expression index, critical; keep."*

The recommended changes:

```
DROP INDEX ix1;
DROP INDEX ix2;
DROP INDEX ix5;
-- ix3, ix4, ix6, ix7 retained
```

Why each one: - Dropping prefix-redundant unused indexes is the cheapest write-throughput win. - Keep FK-backing indexes even if scan counts are low — they protect cascade performance.

What NOT to drop (and why): - ix6 — looks low-use, but cascade deletes without it can lock the table for minutes. - ix7 — login query; latency-critical.

Common follow-up: "How do you verify an index is truly unused before dropping in prod?"

Scenario302 · Same Column, Five Access Patterns — One Composite

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, PhonePe, Setu · Dialect: Postgres

The setup:

```
CREATE TABLE merchant_settlements (  
    settlement_id BIGSERIAL PRIMARY KEY,  
    merchant_id   BIGINT NOT NULL,  
    status        VARCHAR(15) NOT NULL,  
    settled_at    TIMESTAMP,  
    amount        NUMERIC(12,2) NOT NULL,  
    currency      CHAR(3) NOT NULL  
);  
-- 50 crore rows
```

Five access patterns, all merchant-scoped: - (A) `WHERE merchant_id = ?` (full listing) - (B) `WHERE merchant_id = ? AND status = ?` - (C) `WHERE merchant_id = ? AND status = ? AND settled_at BETWEEN ? AND ?` - (D) `WHERE merchant_id = ? AND settled_at > ?` - (E) `WHERE merchant_id = ? AND currency = ?`

The question: "What indexes would you create? Walk me through your reasoning."

Thinking out loud: 1. "Left-prefix rule: a composite (merchant_id, status, settled_at) serves A, B, C. The optimiser can use the leftmost columns even if the rightmost is absent." 2. "D — (merchant_id, settled_at) — is NOT served by the above composite because status sits in between as a non-filtered column." 3. "E — (merchant_id, currency) — needs its own composite." 4. "So three composites total. Cannot collapse further without giving up on D or E."

The recommended indexes:

```
CREATE INDEX idx_sett_merchant_status_time  
ON merchant_settlements (merchant_id, status, settled_at);  
  
CREATE INDEX idx_sett_merchant_time  
ON merchant_settlements (merchant_id, settled_at);  
  
CREATE INDEX idx_sett_merchant_currency  
ON merchant_settlements (merchant_id, currency);
```

Why each one: - The three composites cover all five patterns without forcing the optimiser to skip-scan or sort. - Every composite leads with merchant_id because every query filters on it.

What NOT to index (and why): - A single-column `(merchant_id)` — redundant with any of the composites above. - A four-column mega-composite `(merchant_id, status, currency, settled_at)` — would only marginally help and bloats the index.

Common follow-up: "Pattern F arrives — `WHERE merchant_id = ? AND amount > ?`. Add an index or live with the scan?"

Scenario303 · High-Write, Low-Read Table — Minimal Index Strategy

Tags: Difficulty: Medium · Asked at: Truecaller, Inshorts, Moj, Sharechat · Dialect: Postgres

The setup:

```
CREATE TABLE click_events (  
  event_id    BIGSERIAL PRIMARY KEY,  
  user_id     BIGINT NOT NULL,  
  ad_id       BIGINT NOT NULL,  
  clicked_at  TIMESTAMP NOT NULL,  
  metadata    JSONB  
);  
-- 60000 inserts/sec, near-zero reads (queries hit a downstream warehouse)
```

The workload: - Reads: only ad-hoc debug — `WHERE event_id = ?` — a few times a day - Writes: 60000 inserts/sec, append-only

The question: "What indexes would you create? Walk me through your reasoning."

Thinking out loud: 1. "This is a write-firehose table. Every secondary index costs ~10-20% of insert throughput. The discipline here is index minimalism." 2. "The PK already exists for event_id lookups — the debug pattern is covered." 3. "No analytical query runs against this table — those queries hit the warehouse downstream after CDC." 4. "Resist the urge to add user_id or ad_id indexes 'just in case'."

The recommended indexes:

```
-- PRIMARY KEY (event_id) is the only index. No secondaries.
```

Why each one: - The PK B-tree handles the rare debug lookup. No other access pattern justifies the write tax.

What NOT to index (and why): - `user_id` — every secondary index at 60k writes/sec costs CPU and WAL traffic. - `ad_id` — same. - A GIN on `metadata` — Postgres GIN's pending-list flush at 60k/sec is a real outage risk.

Common follow-up: "Three months later, an analyst wants real-time queries on this table — what's your migration plan?"

Scenario304 · Star Schema Fact Table — Surrogate Keys, Dimension Joins

Tags: Difficulty: Medium · Asked at: Walmart Labs, Target, Tata Steel BI, AmEx Analytics · Dialect: SQL Server / Snowflake / Postgres

The setup:

```
CREATE TABLE fact_sales (  
    sale_id      BIGINT IDENTITY PRIMARY KEY,  
    date_key     INT NOT NULL,           -- FK to dim_date  
    store_key    INT NOT NULL,           -- FK to dim_store  
    product_key  INT NOT NULL,           -- FK to dim_product  
    customer_key INT NOT NULL,           -- FK to dim_customer  
    qty          INT NOT NULL,  
    revenue      DECIMAL(12,2) NOT NULL  
);  
-- 80 crore rows
```

The workload: - Query 1 (60%): "revenue by week and store" — joins fact + dim_date + dim_store, groups by week/store_name - Query 2 (30%): "category sales for last quarter" — joins fact + dim_product + dim_date - Query 3 (10%): customer-level rollup — joins all four dims - Writes: nightly load of 8 lakh rows

The question: "What indexes would you create? Walk me through your reasoning."

Thinking out loud: 1. "Star-schema fact tables are read-mostly. The right answer is one B-tree per surrogate FK, plus a columnstore on the wide table if the engine supports it." 2. "For SQL Server: a clustered columnstore on the fact is the standard pattern — 10x compression and order-of-magnitude faster scans." 3. "For Postgres without columnstore: B-tree on each surrogate key supports nested-loop joins; for the heavy scans, BRIN on date_key works if loaded in date order." 4. "Nightly load — drop nothing, rebuild only if the columnstore needs reorg after the batch."

The recommended indexes (SQL Server):

```
CREATE CLUSTERED COLUMNSTORE INDEX cci_fact_sales ON fact_sales;  
CREATE INDEX IX_fact_date  ON fact_sales (date_key);  
CREATE INDEX IX_fact_store ON fact_sales (store_key);  
CREATE INDEX IX_fact_prod  ON fact_sales (product_key);  
CREATE INDEX IX_fact_cust  ON fact_sales (customer_key);
```

Postgres variant:

```
CREATE INDEX idx_fact_date_brin ON fact_sales USING BRIN (date_key);
CREATE INDEX idx_fact_store ON fact_sales (store_key);
CREATE INDEX idx_fact_prod ON fact_sales (product_key);
CREATE INDEX idx_fact_cust ON fact_sales (customer_key);
```

Why each one: - Columnstore is the answer for wide scans across millions of rows. - B-trees on dimension keys speed up the dimension joins.

What NOT to index (and why): - `qty` or `revenue` — pure measures; never appear in WHERE. - `sale_id` separately — already the PK.

Common follow-up: "How do you keep the columnstore healthy after seven nightly loads?"

Scenario305 · Leading-Wildcard LIKE — Trigram Index

Tags: Difficulty: Hard · Asked at: Naukri, Apna, JustDial, Sulekha · Dialect: Postgres

The setup:

```
CREATE TABLE listings (
  listing_id BIGSERIAL PRIMARY KEY,
  title      VARCHAR(255) NOT NULL,
  body      TEXT,
  city      VARCHAR(60)
);
-- 2 crore rows
```

The workload: - Query 1 (70%): substring search — `WHERE title ILIKE '%plumber%'` - Query 2 (20%): exact city — `WHERE city = ?` - Query 3 (10%): `listing_id` by PK - Writes: 500/day

The question: "What indexes would you create? Walk me through your reasoning."

Thinking out loud: 1. "`ILIKE '%plumber%'` is the classic leading-wildcard case. Plain B-tree cannot help; `LIKE 'foo%'` could use B-tree, but the leading percent kills it." 2. "Postgres `pg_trgm` GIN index breaks strings into 3-character grams and inverts them — `%plumber%` becomes a set of grams the GIN can look up." 3. "City is a low-cardinality B-tree." 4. "Five hundred writes a day means GIN insert overhead is invisible."

The recommended indexes:

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX idx_listings_title_trgm
  ON listings USING GIN (title gin_trgm_ops);
CREATE INDEX idx_listings_city ON listings (city);
```

Why each one: - pg_trgm GIN is the standard answer for `%substring%` patterns. - B-tree on city for the exact-equality filter.

What NOT to index (and why): - B-tree on `title` — useless for the leading-wildcard pattern. - Full-text on `title` — overkill for short titles; trigram is a better fit when users type partial words.

Common follow-up: *"Write rate rises to 50000 per day — does trigram still fit?"*

Scenario306 · Sparse Column — Filtered Index

Tags: Difficulty: Medium · Asked at: HSBC, Standard Chartered, Wells Fargo, AmEx · Dialect: SQL Server / Postgres

The setup:

```
CREATE TABLE loan_applications (  
  app_id          BIGINT IDENTITY PRIMARY KEY,  
  customer_id     BIGINT NOT NULL,  
  status          VARCHAR(20) NOT NULL,  
  rejection_code  VARCHAR(10),          -- NULL for 96% of rows  
  applied_at      DATE NOT NULL  
);  
-- 4 crore rows
```

The workload: - Query 1 (60%): RBI compliance — `WHERE rejection_code = 'NPA-01'` — handful of distinct codes - Query 2 (40%): customer lookup — `WHERE customer_id = ?` - Writes: 5000 inserts/day

The question: *"What indexes would you create? Walk me through your reasoning."*

Thinking out loud: 1. *"rejection_code is NULL 96 percent of the time — a filtered index `WHERE rejection_code IS NOT NULL` covers four percent of rows, sixteen lakh entries — fits in cache trivially."* 2. *"customer_id is the standard B-tree."* 3. *"Filtered indexes (SQL Server) and partial indexes (Postgres) are the right tool for sparse predicates."* 4. *"Five thousand writes a day is nothing — write amplification not in question."*

The recommended indexes (SQL Server):

```
CREATE INDEX IX_loan_rejection  
  ON loan_applications (rejection_code)  
  WHERE rejection_code IS NOT NULL;  
  
CREATE INDEX IX_loan_customer  
  ON loan_applications (customer_id);
```

Postgres equivalent:

```
CREATE INDEX idx_loan_rejection
  ON loan_applications (rejection_code)
  WHERE rejection_code IS NOT NULL;

CREATE INDEX idx_loan_customer
  ON loan_applications (customer_id);
```

Why each one: - Filtered index keeps the rejection-code structure tiny and predictable. - Customer index serves the lookup path.

What NOT to index (and why): - A full index on `rejection_code` — 96 percent NULL entries are wasted leaf space and have to be stored even if they're never queried (depending on engine). - `status` — five values, queries don't filter by it.

Common follow-up: "What if the application also queries `WHERE rejection_code IS NULL` for pending applications?"

Scenario307 · Sort by created_at DESC LIMIT — Handling Descending

Tags: Difficulty: Medium · Asked at: Flipkart, Cred, Razorpay, BharatPe · Dialect: Postgres / MySQL

The setup:

```
CREATE TABLE chat_messages (
  message_id BIGSERIAL PRIMARY KEY,
  room_id    BIGINT NOT NULL,
  sender_id  BIGINT NOT NULL,
  body       TEXT,
  created_at TIMESTAMP NOT NULL
);
-- 80 crore rows
```

The workload: - Query 1 (100%): chat scroll —

`WHERE room_id = ? ORDER BY created_at DESC LIMIT 100` - Writes: 12000 messages/sec

The question: "What indexes would you create? Walk me through your reasoning."

Thinking out loud: 1. "Postgres B-trees are ordered ascending by default. The engine can read them backward for a DESC sort with comparable performance — modern Postgres handles backward scans efficiently." 2. "For MySQL InnoDB, backward index scans are slower because of how the leaf pages are linked. Explicitly declaring `created_at DESC` in the composite asks InnoDB to physically order leaves accordingly." 3. "The composite is (room_id, created_at DESC). Room equality leads, sort column follows." 4. "Twelve thousand inserts per second is heavy but manageable on one composite index."

The recommended indexes:

```
CREATE INDEX idx_chat_room_created
ON chat_messages (room_id, created_at DESC);
```

Why each one: - Equality-first, DESC sort column second. On MySQL the explicit DESC matters; on Postgres it's still a best-practice signal even if the engine could read backward.

What NOT to index (and why): - A separate `(created_at)` — no global created_at query, only room-scoped. - `sender_id` — no predicate use today.

Common follow-up: "On MySQL 5.7, does DESC in CREATE INDEX actually do anything?"

Scenario308 · Composite for IN(...) Plus Range

Tags: Difficulty: Hard · Asked at: Swiggy, Zomato, Dunzo, Blinkit · Dialect: Postgres

The setup:

```
CREATE TABLE restaurant_orders (
  order_id      BIGSERIAL PRIMARY KEY,
  city_id       INT NOT NULL,
  restaurant_id BIGINT NOT NULL,
  ordered_at    TIMESTAMP NOT NULL,
  status        VARCHAR(20) NOT NULL
);
-- 40 crore rows
```

The workload: - Query 1 (100%): ops dashboard — `WHERE city_id IN (1, 5, 12, 18, 27) AND ordered_at BETWEEN ? AND ?` - Writes: 4500 inserts/sec

The question: "What indexes would you create? Walk me through your reasoning."

Thinking out loud: 1. "IN(...) is treated as equality by the optimiser — each value triggers an index seek. So city_id can still lead a composite." 2. "The composite (city_id, ordered_at) works: for each IN value, the engine does an index range scan on ordered_at." 3. "Putting ordered_at first would force a date range scan and then filter — much worse." 4. "4500 inserts per second on one composite is acceptable."

The recommended indexes:

```
CREATE INDEX idx_orders_city_time
ON restaurant_orders (city_id, ordered_at);
```

Why each one: - `city_id` leads despite the IN — the optimiser treats each IN value as a separate seek prefix. - `ordered_at` second for the range.

What NOT to index (and why): - `(ordered_at, city_id)` — range column first kills index usability for the city filter. - `restaurant_id` — not in this workload's predicates.

Common follow-up: "What if the IN list grows to 100 cities — does the plan still use the index?"

Scenario309 · Multi-Column UNIQUE Constraint and Its Backing Index

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, HCL · Dialect: vendor-agnostic

The setup:

```
CREATE TABLE enrollments (
  enrollment_id BIGSERIAL PRIMARY KEY,
  student_id    BIGINT NOT NULL,
  course_id     BIGINT NOT NULL,
  term          CHAR(6) NOT NULL,      -- e.g. '2026SP'
  enrolled_at   TIMESTAMP NOT NULL
);
-- 50 lakh rows
```

The business rule: a student can enrol in a given course only once per term.

The workload: - Query 1 (50%): "is this student in this course this term?" — `WHERE student_id = ? AND course_id = ? AND term = ?` - Query 2 (40%): student timetable — `WHERE student_id = ? AND term = ?` - Query 3 (10%): course roster — `WHERE course_id = ? AND term = ?` - Writes: 30000 enrollments/day, batchy

The question: "What indexes would you create? Walk me through your reasoning."

Thinking out loud: 1. "The business rule maps directly to `UNIQUE (student_id, course_id, term)`. The engine creates a backing index for the constraint — that index also serves Query 1." 2. "Query 2 needs `(student_id, term)` — leftmost prefix of the UNIQUE is `(student_id, course_id)`, so Query 2 cannot use the UNIQUE without a skip-scan. Add `(student_id, term)`." 3. "Query 3 needs `(course_id, term)` — same skip-scan issue from the unique. Add `(course_id, term)`." 4. "Three indexes total. Write rate is moderate; cost is acceptable."

The recommended indexes:

```
ALTER TABLE enrollments
  ADD CONSTRAINT uq_enrol_stud_course_term
  UNIQUE (student_id, course_id, term);

CREATE INDEX idx_enrol_student_term ON enrollments (student_id, term);
CREATE INDEX idx_enrol_course_term ON enrollments (course_id, term);
```

Why each one: - UNIQUE constraint enforces the business rule AND serves Query 1. - Two additional composites handle the other access patterns where the UNIQUE's column order doesn't help.

What NOT to index (and why): - A single `(term)` — terms have 50/50 cardinality at most; not selective enough alone. - `enrolled_at` — no query uses it.

Common follow-up: "Could you reorder the UNIQUE columns to also serve Query 2?"

Scenario310 · Temporary / Scratch Table — When to Index

Tags: Difficulty: Medium · Asked at: Walmart Labs, Target, Wells Fargo, Goldman · Dialect: Postgres / SQL Server

The setup:

```
-- Created per-session by a reporting workflow
CREATE TEMP TABLE tmp_eligible_orders (
  order_id      BIGINT,
  customer_id   BIGINT,
  fulfilled_at  TIMESTAMP
);
-- Populated with 12 lakh rows from a CTE
```

The workload (within one session): - Step A: INSERT 12 lakh rows - Step B: join `tmp` to `customers` ON `customer_id` — once - Step C: aggregate by date — once - Step D: DROP TABLE at session end

The question: "What indexes would you create? Walk me through your reasoning."

Thinking out loud: 1. "Indexes on temp tables only pay off if the rows are read multiple times or via a selective predicate. Here each step is one full pass — sequential scan with hash aggregate beats index seek." 2. "Twelve lakh rows is small enough to live in work_mem; the optimiser will hash-join and hash-aggregate happily." 3. "For Postgres specifically, `ANALYZE tmp_eligible_orders` after population is more important than indexing — without statistics the planner assumes 1000 rows and chooses bad plans." 4. "If we ever moved from one-pass to nested-loop join driven by tmp, then a B-tree on `customer_id` would earn its keep."

The recommended indexes:

```
-- No indexes on this temp table.  
ANALYZE tmp_eligible_orders; -- Postgres  
-- UPDATE STATISTICS tmp_eligible_orders; -- SQL Server
```

Why each one: - ANALYZE/UPDATE STATISTICS gives the planner accurate row counts to pick hash vs nested loop intelligently. - No B-tree is justified for one-pass workloads.

What NOT to index (and why): - `customer_id` — single hash join doesn't benefit from a B-tree. - `fulfilled_at` — single aggregation pass; sequential scan is optimal.

Common follow-up: "What if the temp table is reused across five reports in the same session — does your answer change?"

Category 5, Part B — Why Is My Index Not Being Used? Plus Maintenance

Part B is the reactive side of index design — the index already exists, but the query is still slow and the plan keeps choosing a sequential scan. Each scenario below walks through a real production diagnosis: read the `EXPLAIN` carefully, work out why the planner ignored the index you built, and fix either the query, the statistics, the index definition, or the maintenance cadence. The last block covers the maintenance and sizing decisions — `REINDEX`, `VACUUM`, online builds, bloat, oversized index footprints — that come up in every SDE-3 and staff round in Indian product companies (Razorpay, Cred, PhonePe, Swiggy, Zerodha, Flipkart) and senior GCC rounds (Walmart Labs, Goldman, JP Morgan India, AmEx, Target). Planner behaviour varies by engine, so each scenario calls out the dialect difference where it matters.

Scenario311 · Function Wrapping the Indexed Column

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Wells Fargo · Topic: Sargability · Engines: All

Setup:

```
CREATE TABLE customers (  
  customer_id  BIGSERIAL PRIMARY KEY,  
  email        VARCHAR(255) NOT NULL,  
  city         VARCHAR(80),  
  created_at   TIMESTAMPTZ DEFAULT now()  
);  
CREATE INDEX idx_customers_email ON customers (email);  
-- 12 million rows; the slow query  
SELECT customer_id FROM customers WHERE UPPER(email) = 'PRIYA@F00.IN';
```

Question: "This query takes 9 seconds and `EXPLAIN` shows a Seq Scan even though `email` is indexed. Why, and how do you fix it?"

Thinking out loud: 1. "The B-tree is sorted on the raw `email` value, not on `UPPER(email)`. So `UPPER(email) = 'PRIYA@F00.IN'` cannot be answered by a range scan — the planner would have to compute the function for every row first, which is exactly what a Seq Scan does." 2. "The predicate is non-sargable — Search-ARGument-ABLE means the planner can isolate the column on one side of the operator. Wrapping the column in any function breaks that." 3. "Two clean fixes: rewrite the predicate to keep the column bare (`email = 'priya@foo.in'` if data is already lowercase, or store a lowercase shadow column), or create an expression index on `UPPER(email)` so the function call becomes part of the index key." 4. "In production I'd usually go with the expression index because it doesn't require changing application code that may have `UPPER` scattered across multiple queries."

Diagnosis: Predicate `UPPER(email) = ...` is non-sargable against an index on `email`. Planner falls back to Seq Scan.

Fix:

```
-- Option A: expression index (Postgres, Oracle)
CREATE INDEX idx_customers_email_upper ON customers (UPPER(email));
-- Option B: store normalized column and index it
ALTER TABLE customers ADD COLUMN email_lc VARCHAR(255)
    GENERATED ALWAYS AS (LOWER(email)) STORED;
CREATE INDEX idx_customers_email_lc ON customers (email_lc);
```

After the expression index, re-run `ANALYZE` and confirm Index Scan in `EXPLAIN`.

Why: B-trees index the literal column expression. Any function on the column changes the value space, so the planner cannot use the existing index to satisfy the predicate.

NOT to say: "Indexes never work with functions" — they do, via expression indexes or generated columns. "The planner should just figure it out" — it cannot reverse arbitrary functions.

Follow-up: "Would `LOWER(email) = 'priya@foo.in'` be sargable if the column were `CITEXT`?" — Yes, because `CITEXT` comparisons are case-insensitive at the operator level, so you do not need `LOWER()` at all.

Scenario312 · Implicit Cast Killing the Index

Tags: Difficulty: Easy · Asked at: Infosys, MySQL shops, AmEx · Topic: Sargability · Engines: MySQL especially

Setup:

```
CREATE TABLE orders (
  order_id      BIGINT PRIMARY KEY,
  customer_id   BIGINT NOT NULL,
  amount_paise BIGINT,
  status        VARCHAR(20)
);
CREATE INDEX idx_orders_customer ON orders (customer_id);
-- ORM is sending customer_id as a string
SELECT * FROM orders WHERE customer_id = '7724901';
```

Question: "The index on `customer_id` exists but `EXPLAIN` shows `type: ALL`. The ORM serialises IDs as strings. What's the root cause?"

Thinking out loud: 1. "`customer_id` is `BIGINT`, the literal is a string. MySQL has to implicitly cast one side — and per SQL standard ordering, string to number conversion happens on the column side, which means MySQL effectively does `CAST(customer_id AS CHAR) = '7724901'`." 2. "That cast turns the predicate non-sargable — same situation as wrapping the column in a function. The B-tree on the integer value cannot be range-scanned for a string comparison." 3. "On Postgres the rules are different — Postgres prefers to cast the literal, so `'7724901'::text` becomes `7724901::bigint` and the index is used. MySQL and SQL Server are the usual offenders." 4. "The cleanest fix is to parameterise correctly — make the driver bind the parameter as an integer. If the schema is wrong (the column should have been a string all along, like a phone number stored as `BIGINT`), fix the schema."

Diagnosis: Implicit cast direction in MySQL converts the column, not the literal, making the predicate non-sargable.

Fix:

```
-- Application side: bind as integer, not string
PreparedStatement ps = c.prepareStatement("SELECT * FROM orders WHERE customer_id = ?");
ps.setLong(1, 7724901L); // not setString
```

Or normalize the column type if it was wrong from the start.

Why: When operand types differ, the engine inserts a cast. MySQL's cast precedence rules often cast the column side for string-vs-number comparisons, which destroys sargability.

NOT to say: "String and int comparison just works in SQL" — it returns the right result, but at the cost of a full scan. "Postgres has the same problem" — Postgres typically casts the literal, preserving the index.

Follow-up: "What if the column is `VARCHAR` and the literal is a number — say `WHERE phone = 9876543210`?" — MySQL casts the column from string to number; loses leading zeros; both wrong result and full scan. Always quote string literals as strings.

Scenario313 · Leading Wildcard in LIKE

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Walmart Labs · Topic: Sargability · Engines: All

Setup:

```
CREATE TABLE products (  
    sku          VARCHAR(40) PRIMARY KEY,  
    product_name VARCHAR(200),  
    brand        VARCHAR(80)  
);  
CREATE INDEX idx_products_name ON products (product_name);  
-- Customer searches "soap" anywhere in the name  
SELECT sku, product_name FROM products WHERE product_name LIKE '%soap%';
```

Question: "This query does a full scan even though `product_name` is indexed. Why does `LIKE 'soap%'` use the index but `LIKE '%soap%'` doesn't?"

Thinking out loud: 1. "A B-tree is sorted lexicographically. `LIKE 'soap%'` is equivalent to the range `product_name >= 'soap' AND product_name < 'soat'` — a contiguous range the index can satisfy." 2. " `LIKE '%soap%'` has no contiguous range — `soap` could appear at any position, so the index entries are not co-located. The planner correctly chooses Seq Scan." 3. "For real substring search, you need a different data structure. Postgres has `pg_trgm` GIN indexes; MySQL has `FULLTEXT` indexes; SQL Server has Full-Text Search. These index n-grams or tokens, not the full string." 4. "If the column is small enough and lookup happens often, a `reverse(product_name)` expression index can also handle trailing-wildcard searches like `LIKE '%abc'`."

Diagnosis: Leading wildcard makes the predicate non-sargable against a B-tree.

Fix:

```
-- Postgres trigram index for substring search  
CREATE EXTENSION IF NOT EXISTS pg_trgm;  
CREATE INDEX idx_products_name_trgm ON products USING gin (product_name gin_trgm_ops);  
SELECT sku, product_name FROM products WHERE product_name ILIKE '%soap%';  
  
-- MySQL fulltext  
ALTER TABLE products ADD FULLTEXT INDEX ft_name (product_name);  
SELECT sku, product_name FROM products WHERE MATCH(product_name) AGAINST('soap');
```

Why: B-trees support prefix ranges, not arbitrary substrings. Substring search needs token-based indexing (trigrams, fulltext).

NOT to say: "LIKE never uses an index" — `LIKE 'prefix%'` does. "Just add more memory" — the issue is algorithmic, not RAM.

Follow-up: "Why does `pg_trgm` need GIN and not B-tree?" — Trigram indexes store many entries per row (one per overlapping 3-gram), so GIN's posting-list structure is the right fit. B-tree assumes one key per row.

Scenario314 · Low Cardinality — Planner Picks Seq Scan Anyway

Tags: Difficulty: Medium · Asked at: Cred, Flipkart, Goldman · Topic: Selectivity · Engines: All

Setup:

```
CREATE TABLE shipments (  
  shipment_id  BIGSERIAL PRIMARY KEY,  
  order_id     BIGINT,  
  status       VARCHAR(20), -- only 5 distinct values  
  created_at   TIMESTAMPTZ  
);  
CREATE INDEX idx_shipments_status ON shipments (status);  
-- 80 million rows, status='delivered' is 78 million of them  
SELECT shipment_id FROM shipments WHERE status = 'delivered';
```

Question: "I created an index on `status` but Postgres still does a Seq Scan. Why is it ignoring my index?"

Thinking out loud: 1. "`status` has 5 distinct values and `delivered` covers 97% of rows. Reading the index, then doing 78 million heap fetches (random I/O), is dramatically more expensive than one sequential read of the table." 2. "The planner is correct — it estimated rows to fetch $\approx 78M$, multiplied by `random_page_cost`, came out higher than `seq_page_cost` $\times$ `total_pages`. Seq Scan wins." 3. "Indexes earn their keep on selective predicates — roughly $< 5\text{-}10\%$ of rows. Above that, the random I/O penalty plus the index traversal cost makes a scan cheaper." 4. "Real fix depends on workload: if `delivered` is genuinely most of the data and we rarely query it, drop the index for that value. If we frequently query the rare values (`returned`, `lost`), use a partial index that only covers those."

Diagnosis: Predicate matches majority of rows; planner correctly prefers Seq Scan.

Fix:

```
-- Partial index: only the rare, queryable values  
DROP INDEX idx_shipments_status;  
CREATE INDEX idx_shipments_status_active  
  ON shipments (status)  
  WHERE status IN ('pending', 'in_transit', 'returned', 'lost');
```

Tiny index, fast lookups for the values that matter; no overhead on the dominant `delivered` writes.

Why: Index access is only cheaper than Seq Scan when selectivity is high (few rows match). On low-cardinality columns where one value dominates, the index is the wrong tool.

NOT to say: *"The planner is broken — force it to use the index"* — forcing usually makes it slower. *"Just add more indexes"* — won't help; selectivity is the problem.

Follow-up: *"What if I really need `WHERE status = 'delivered'` to be fast?"* — That's a date-range or pagination problem, not a status problem. Index on `(created_at)` or a composite `(status, created_at)` and add a time bound.

Scenario315 · Composite Index Column-Order Mismatch

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Walmart Labs · Topic: Composite indexes · Engines: All

Setup:

```
CREATE TABLE bookings (
  booking_id BIGSERIAL PRIMARY KEY,
  city_id INT,
  movie_id INT,
  show_time TIMESTAMPTZ,
  total_paise BIGINT
);
CREATE INDEX idx_bookings_city_movie ON bookings (city_id, movie_id);
-- Query only filters on movie_id
SELECT COUNT(*) FROM bookings WHERE movie_id = 4421;
```

Question: *"I have a composite index on `(city_id, movie_id)` but my `WHERE movie_id = 4421` query doesn't use it. Why?"*

Thinking out loud: 1. "B-tree composite indexes are sorted left-to-right. `(city_id, movie_id)` is sorted primarily by `city_id`, then by `movie_id` within each `city_id`. Without a `city_id` predicate, the `movie_id` values are scattered across the whole index." 2. "This is the leftmost-prefix rule. The planner can use the index for queries on `(city_id)`, `(city_id, movie_id)`, or with a range like `(city_id = ? AND movie_id > ?)`. It cannot efficiently use it for `(movie_id)` alone." 3. "Postgres can sometimes do an Index Skip Scan (loose index scan, Postgres 17+) if `city_id` has very few distinct values, but the default planner avoids it. MySQL 8 added skip scan in limited cases. SQL Server has Index Skip Scan in some plans." 4. "Two options: add a second single-column index on `movie_id`, or flip the composite to `(movie_id, city_id)` if queries on `movie_id` alone are common and `city_id`-only queries are rare."

Diagnosis: Query violates leftmost-prefix rule on composite index.

Fix:

```
-- Add the missing index
CREATE INDEX idx_bookings_movie ON bookings (movie_id);
-- Or, if movie-only is more common, reorder the composite
CREATE INDEX idx_bookings_movie_city ON bookings (movie_id, city_id);
DROP INDEX idx_bookings_city_movie;
```

Why: Composite index keys are concatenated; without a prefix predicate the index is sorted on the wrong dimension for the lookup.

NOT to say: "The planner should use any column in the index" — only the leftmost prefix is efficient. "Column order doesn't matter" — it dictates which queries the index can serve.

Follow-up: "How do I decide column order in a composite?" — Equality columns first, then range columns, then ORDER BY columns. Within equality, put the higher-cardinality column first if both are equally common in WHERE.

Scenario316 · OR Across Different Columns

Tags: Difficulty: Medium · Asked at: Swiggy, Zomato, AmEx · Topic: Predicate fragmentation · Engines: All

Setup:

```
CREATE TABLE users (
  user_id      BIGSERIAL PRIMARY KEY,
  phone        VARCHAR(15),
  email        VARCHAR(120)
);
CREATE INDEX idx_users_phone ON users (phone);
CREATE INDEX idx_users_email ON users (email);
-- Login flow that accepts either
SELECT user_id FROM users WHERE phone = '9876543210' OR email = 'priya@foo.in';
```

Question: "Both columns have indexes but this OR query still does a Seq Scan. Why?"

Thinking out loud: 1. "With **AND**, the planner can pick the most selective index and filter the rest in memory. With **OR**, every row matching either condition needs to be returned — there is no single index that covers both columns." 2. "Postgres handles this with a BitmapOr — scan **idx_users_phone** for matches, scan **idx_users_email** for matches, OR the bitmaps, then heap-fetch. MySQL has **index_merge_union** for the same idea. SQL Server may pick a similar plan." 3. "The reason it sometimes still does Seq Scan: if either branch is non-selective, the bitmap union becomes huge and a Seq Scan wins. Or if the planner's statistics are off, it underestimates and picks the wrong shape." 4. "Cleanest pattern in application code: rewrite as **UNION** of two single-predicate queries. Each can use its own index cleanly; **UNION ALL** plus **DISTINCT** if duplicates are a concern."

Diagnosis: `OR` across columns needs a `BitmapOr` / `index_merge_union` plan, which the planner skips if statistics suggest a `Seq Scan` is cheaper.

Fix:

```
-- Rewrite as UNION
SELECT user_id FROM users WHERE phone = '9876543210'
UNION
SELECT user_id FROM users WHERE email = 'priya@foo.in';
```

Each branch uses its own index; planner has no ambiguity.

Why: A single index cannot satisfy a disjunction over different columns; the planner either combines bitmaps or scans. `UNION` decomposes into two clean index lookups.

NOT to say: "*OR is always slow*" — `OR` on the same column with a few values is fine. "*Add a multi-column index*" — composite indexes do not help with `OR` across different columns.

Follow-up: "*What if it's `WHERE phone IN ('A','B') OR email IN ('C','D')` ?*" — Same pattern: rewrite as `UNION` of two `IN` predicates, each using its own index.

Scenario317 · NOT IN With NULLs

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan, Cred · Topic: NULL semantics · Engines: All

Setup:

```
CREATE TABLE invoices (
  invoice_id  BIGSERIAL PRIMARY KEY,
  customer_id BIGINT,
  refund_id   BIGINT -- nullable
);
CREATE INDEX idx_invoices_refund ON invoices (refund_id);

CREATE TABLE refunds (
  refund_id  BIGINT PRIMARY KEY,
  reason     VARCHAR(80)
);
-- "Show invoices not in the refunds table"
SELECT invoice_id FROM invoices
WHERE refund_id NOT IN (SELECT refund_id FROM refunds);
```

Question: "*This query is doing a full scan plus a hash anti-join and returning wrong rows when there's even one NULL in `refunds.refund_id` . Walk me through both issues.*"

Thinking out loud: 1. " `NOT IN` with a subquery that can return NULL has SQL-standard three-valued logic semantics: `x NOT IN (1, 2, NULL)` is UNKNOWN, not TRUE — so no rows ever match if any NULL appears. The planner cannot push down index lookups because it has to materialise the full subquery to check for NULLs." 2. "This forces a hash anti-join with extra NULL-handling logic. The index on `refund_id` is bypassed because the engine has to scan everything in `refunds` anyway." 3. "The fix is to use `NOT EXISTS` instead, which has clean two-valued semantics — NULL in the subquery does not contaminate the result. `NOT EXISTS` correlates row-by-row and the planner can use an anti-semi-join with the index." 4. "Alternatively `LEFT JOIN ... WHERE r.refund_id IS NULL` is equivalent. Either pattern uses the index cleanly and gives correct results."

Diagnosis: `NOT IN` against a nullable subquery has UNKNOWN semantics on NULL, blocking index use and producing wrong results.

Fix:

```
SELECT i.invoice_id
FROM invoices i
WHERE NOT EXISTS (
  SELECT 1 FROM refunds r WHERE r.refund_id = i.refund_id
);
```

Postgres, MySQL 8, and SQL Server all plan this as an Anti Semi Join using the index on `refunds.refund_id`.

Why: SQL's three-valued logic makes `NOT IN` with NULLs return UNKNOWN. The planner cannot prove safety, so it scans. `NOT EXISTS` is safe and indexable.

NOT to say: "NOT IN and NOT EXISTS are the same" — they differ on NULL. "Just add an index on `refunds.refund_id`" — the index is bypassed by the NULL semantics.

Follow-up: "If I add `WHERE refund_id IS NOT NULL` to the subquery, does it help?" — Yes, semantically it makes `NOT IN` safe and the planner can then use the index, but `NOT EXISTS` is the cleaner idiom.

Scenario318 · Stale Statistics, Wrong Plan

Tags: Difficulty: Medium · Asked at: Razorpay, Walmart Labs, Target · Topic: Statistics · Engines: All

Setup:

```
CREATE TABLE events (
  event_id      BIGSERIAL PRIMARY KEY,
  user_id       BIGINT,
  event_type    VARCHAR(40),
  created_at    TIMESTAMPTZ
);
CREATE INDEX idx_events_user ON events (user_id);
CREATE INDEX idx_events_created ON events (created_at);
-- Bulk-loaded 200M rows yesterday; stats still show table at 50M
SELECT event_id FROM events WHERE user_id = 88012;
```

Question: "Yesterday this query used the index and ran in 20ms. After last night's bulk load, it does a Seq Scan and takes 6 seconds. No index was changed. What happened?"

Thinking out loud: 1. "Bulk loads (COPY, LOAD DATA INFILE, INSERT ... SELECT) on Postgres don't automatically trigger `ANALYZE`. The planner is still using statistics that say the table has 50M rows and `user_id` has 5M distinct values. Estimated rows per `user_id` ≈ 10 ." 2. "After the load, real table is 200M rows but `user_id` distinct count didn't grow proportionally — maybe only 8M distinct now. Estimated 10 rows per user is fine, but the planner's view of total work is wrong and it picks Seq Scan because it thinks the table is only 50M rows and the random I/O cost vs sequential changed." 3. "In Postgres, autovacuum should ANALYZE after a threshold of changes, but bulk loads can outrun it. In MySQL, `innodb_stats_persistent` and `innodb_stats_auto_recalc` govern this. SQL Server has Auto Update Statistics." 4. "Fix: run `ANALYZE events;` manually right after the bulk load. Long-term, make sure autovacuum/auto-update-stats is tuned for the largest tables. For very large changes, increase `default_statistics_target` per column to get better histograms."

Diagnosis: Statistics not updated after bulk load; planner's cost model uses stale row counts and distributions.

Fix:

```
-- Postgres
ANALYZE events;
-- Or in the bulk-load script
ANALYZE events (user_id, created_at);

-- MySQL
ANALYZE TABLE events;

-- SQL Server
UPDATE STATISTICS dbo.events WITH FULLSCAN;
```

Why: The optimiser is a cost-based planner; its cost estimates depend on row count, distinct values, histograms, and correlation. Stale stats mean wrong costs mean wrong plans.

NOT to say: "Just rebuild the index" — REINDEX doesn't update planner stats. "The planner randomly changed" — it deterministically followed the costs it was given.

Follow-up: "How often should I run ANALYZE on a 1TB OLTP table?" — Let autovacuum/auto-update-stats handle it, but tune thresholds (`autovacuum_analyze_scale_factor` to 0.02 or lower for large tables). After every bulk load, run it explicitly.

Scenario319 · Index Too Small To Bother With

Tags: Difficulty: Medium · Asked at: Cred, Zerodha, Goldman · Topic: Cost model · Engines: All

Setup:

```
CREATE TABLE pin_codes (  
  pin      INT PRIMARY KEY,  
  city     VARCHAR(80),  
  state    VARCHAR(80)  
);  
CREATE INDEX idx_pin_codes_state ON pin_codes (state);  
-- Only 19,000 rows, fits entirely in 1 MB of buffer cache  
SELECT pin FROM pin_codes WHERE state = 'Maharashtra';
```

Question: "I created an index on `state` and EXPLAIN still shows Seq Scan. Why won't a 19,000-row table use the index?"

Thinking out loud: 1. "The whole table fits in maybe 2-3 buffer pages. A Seq Scan is one page read after another sequentially — basically free once cached. An Index Scan would have to traverse the B-tree, then random-fetch from the heap for each match." 2. "For tables this small, the planner often picks Seq Scan because the total I/O cost is lower than even the index traversal startup cost. This is the correct decision." 3. "It only matters for very small tables, but candidates panic and try to force the index with hints. Don't — `EXPLAIN ANALYZE` will show the Seq Scan is faster end to end." 4. "Real lesson: indexes have a fixed startup cost (root + intermediate pages). On a table where the entire heap fits in cache and the scan is sub-millisecond, that startup cost dominates."

Diagnosis: Table is small enough that Seq Scan beats Index Scan; planner is correct.

Fix: Drop the unnecessary index, or accept the Seq Scan.

```
-- If the index is never used and only adds write overhead, drop it  
DROP INDEX idx_pin_codes_state;
```

Why: Index Scan has fixed traversal cost. Below a threshold table size, Seq Scan is cheaper even for selective predicates.

NOT to say: "Force the index with a hint" — almost always slower in this scenario. "The planner is wrong" — measure with `EXPLAIN ANALYZE` first.

Follow-up: "How small is too small for an index to ever earn its keep?" — Rule of thumb: less than 1000 rows, indexes rarely help unique-lookup workloads. For lookup tables under 10,000 rows, only index columns you actually filter on in mixed-join contexts.

Scenario320 · Index Bloat — REINDEX Required

Tags: Difficulty: Hard · Asked at: Cred, Razorpay, Walmart Labs · Topic: Bloat · Engines: Postgres especially

Setup:

```
CREATE TABLE wallet_transactions (
  txn_id      BIGSERIAL PRIMARY KEY,
  wallet_id   BIGINT,
  amount_paise BIGINT,
  status      VARCHAR(20),
  created_at  TIMESTAMPTZ
);
CREATE INDEX idx_wallet_txn_wallet ON wallet_transactions (wallet_id);
-- Heavy update workload, status changes pending -> success -> reconciled
-- pgstattuple shows idx_wallet_txn_wallet is 8GB; table is 3GB
```

Question: "The index is more than twice the size of the table. Queries on `wallet_id` got 4x slower over the last quarter. What's happening and how do you fix it without downtime?"

Thinking out loud: 1. "In Postgres MVCC, an UPDATE creates a new heap tuple and a new index entry pointing to it. The old entries are not removed immediately; they become dead and are only cleaned up by VACUUM. Heavy update workloads leave the index full of dead entries — bloat." 2. "Bloat in B-trees inflates leaf pages, increases tree height, and slows lookups because more pages must be fetched. Auto VACUUM marks dead tuples reusable in the heap but does not aggressively reclaim space inside index pages." 3. " `pgstattuple` and `pg_stat_user_indexes` (or the `pgstattuple_approx` extension) tell you how much of an index is dead. Anything over 30% bloat is worth attention." 4. "Fix is REINDEX, but plain REINDEX takes an AccessExclusiveLock — bad for production. `REINDEX INDEX CONCURRENTLY` (Postgres 12+) does it online: builds a fresh index alongside, swaps it in, drops the old. Slightly slower build, no downtime."

Diagnosis: B-tree index bloat from heavy updates; dead index entries accumulate faster than VACUUM reclaims them.

Fix:

```
-- Online REINDEX (Postgres 12+), no AccessExclusiveLock
REINDEX INDEX CONCURRENTLY idx_wallet_txn_wallet;

-- For very bloated tables, REINDEX TABLE CONCURRENTLY does all indexes
REINDEX TABLE CONCURRENTLY wallet_transactions;
```

After, autovacuum cadence should be tuned to prevent recurrence.

Why: MVCC update workloads bloat B-tree leaf pages. VACUUM marks dead entries but doesn't physically rebalance. REINDEX compacts.

NOT to say: "Just run VACUUM FULL" — it works but rewrites the whole table with AccessExclusiveLock; far worse for uptime than CONCURRENT REINDEX. "Bloat is a Postgres myth" — it is measurable and real.

Follow-up: "What's the difference between table bloat and index bloat?" — Table bloat is dead tuples in the heap; reclaimed by autovacuum and (slowly) reused. Index bloat is dead entries in B-tree leaves; never reclaimed in-place at scale, requires REINDEX.

Scenario321 · Wrong Join Algorithm Picked

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan, Flipkart · Topic: Join planning · Engines: All

Setup:

```
CREATE TABLE orders (
  order_id      BIGINT PRIMARY KEY,
  customer_id   BIGINT,
  created_at    TIMESTAMPTZ
);
CREATE INDEX idx_orders_customer ON orders (customer_id);

CREATE TABLE customers (
  customer_id   BIGINT PRIMARY KEY,
  tier          VARCHAR(20)
);
-- 200M orders, 5M customers, 50K of which are tier='platinum'
SELECT o.order_id
FROM orders o JOIN customers c ON o.customer_id = c.customer_id
WHERE c.tier = 'platinum';
```

Question: "This used to be fast as a Nested Loop with index lookups. After last week's stats update it's now a Hash Join scanning all 200M orders. P95 is 4x worse. What went wrong?"

Thinking out loud: 1. "Nested Loop with index lookup costs roughly $\text{outer_rows} * \text{index_lookup_cost}$ ". For 50K platinum customers and a B-tree index lookup ~ 4 page reads each,

that's manageable." 2. "Hash Join builds an in-memory hash on the smaller side then scans the larger. Costs `build + scan_larger`. Here it's building a 50K-row hash and scanning 200M orders — sequentially, no index lookups. If the cost model thinks index lookups are expensive or the row estimate for platinum is too high, Hash Join may win on paper but lose in reality due to cache effects." 3. "Most common trigger: `random_page_cost` set too high (default 4.0 was tuned for spinning disks; SSDs warrant 1.1). With SSDs, random index fetches are nearly as cheap as sequential, so Nested Loop with index lookups should win. Stale or off cost parameters tip the balance toward Hash." 4. "Other cause: row estimate for `tier='platinum'` is wrong — if stats say 5M instead of 50K, planner correctly avoids Nested Loop. Refresh stats with `CREATE STATISTICS` or higher `default_statistics_target` for the column."

Diagnosis: Cost parameters or stats steered the planner toward Hash Join when index-driven Nested Loop is actually cheaper on SSD.

Fix:

```
-- Postgres: tune for SSDs
ALTER SYSTEM SET random_page_cost = 1.1;
SELECT pg_reload_conf();

-- Improve cardinality estimate
ANALYZE customers;
ALTER TABLE customers ALTER COLUMN tier SET STATISTICS 500;
ANALYZE customers;
```

Why: Join algorithm choice is cost-driven. If `random_page_cost` overstates SSD random I/O or row estimates are off, the planner picks the wrong algorithm.

NOT to say: "Force the join with a hint" — fix the root cause first. "Hash Join is always slower" — for two big tables with no selective predicate, Hash Join wins.

Follow-up: "When is Merge Join actually the best choice?" — When both inputs are already sorted on the join key (an index already in that order, or a prior sort step). Common in OLAP and after a sorted GROUP BY.

Scenario322 · Cross Join Forced by Non-Sargable Join Condition

Tags: Difficulty: Hard · Asked at: Walmart Labs, Target, Goldman · Topic: Join sargability · Engines: All

Setup:

```

CREATE TABLE auctions (
  auction_id  BIGSERIAL PRIMARY KEY,
  start_time  TIMESTAMPTZ,
  end_time    TIMESTAMPTZ
);
CREATE INDEX idx_auctions_start ON auctions (start_time);
CREATE INDEX idx_auctions_end   ON auctions (end_time);

CREATE TABLE bids (
  bid_id      BIGSERIAL PRIMARY KEY,
  auction_id  BIGINT,
  placed_at   TIMESTAMPTZ
);
-- "Which bids landed inside their auction's window"
SELECT b.bid_id
FROM bids b JOIN auctions a
  ON b.placed_at BETWEEN a.start_time AND a.end_time;

```

Question: "This is doing a Nested Loop with a Seq Scan on auctions for every bid — effectively a cross product check. Why isn't it using any index?"

Thinking out loud: 1. "The join condition is a *BETWEEN* range. Neither column on the inner side has a single-value lookup — the planner cannot use `idx_auctions_start` to seek both sides of the range simultaneously." 2. "For interval queries like this, B-tree indexes on start/end individually don't help. The correct data structure is a range type with a GiST index in Postgres, or a spatial index in some engines." 3. "Postgres `tstzrange` with a GiST index lets the planner do an Index Scan on `range @> timestamp` containment lookups, turning the cross-product into an indexed lookup per bid." 4. "In engines without range types (MySQL pre-8, SQL Server), the pragmatic fix is to add `auction_id` as a foreign key to bids and join on equality first, then filter on the range — the equality join uses the index, the range filter happens on fetched rows."

Diagnosis: BETWEEN join is non-sargable on B-tree indexes; planner falls back to nested loop without index seek.

Fix:

```

-- Postgres: range type plus GiST
ALTER TABLE auctions ADD COLUMN window tstzrange
  GENERATED ALWAYS AS (tstzrange(start_time, end_time, '[]')) STORED;
CREATE INDEX idx_auctions_window ON auctions USING gist (window);

SELECT b.bid_id
FROM bids b JOIN auctions a ON a.window @> b.placed_at;

```

Why: Range-containment joins need range indexes (GiST/SP-GiST in Postgres, spatial in others) — B-tree indexes a point, not an interval.

NOT to say: "Add an index on `placed_at`" — doesn't help the join shape. "This is fundamentally $O(nm)$ " — not with the right index.

Follow-up: "What if I just need 'most recent auction before this bid'?" — That's a lateral join: `LATERAL (SELECT ... FROM auctions WHERE start_time <= b.placed_at ORDER BY start_time DESC LIMIT 1)`. Indexable.

Scenario323 · Parameter Sniffing in SQL Server

Tags: Difficulty: Hard · Asked at: AmEx, Goldman, Walmart Labs · Topic: Plan cache · Engines: SQL Server primarily

Setup:

```
CREATE TABLE rides (
  ride_id      BIGINT PRIMARY KEY,
  driver_id    INT,
  city_id      INT,
  created_at   DATETIME2
);
CREATE INDEX idx_rides_city_created ON rides (city_id, created_at);
-- The proc
CREATE PROCEDURE GetRidesByCity @city_id INT, @from DATETIME2 AS
  SELECT ride_id FROM rides
  WHERE city_id = @city_id AND created_at >= @from;
GO
```

Question: "This proc runs in 30ms most of the time but every Monday morning it takes 8 seconds for the first caller, then it's fast again. What's the pattern?"

Thinking out loud: 1. "SQL Server caches a plan based on the parameter values from the first execution. If the first caller on Monday passed `@city_id = 1` (Mumbai, 80% of rows), the planner picked a Seq Scan plan because the predicate is non-selective. That plan stays cached." 2. "Subsequent callers asking for `@city_id = 47` (Pune, 0.5% of rows) reuse the cached Seq Scan plan, which is catastrophic for selective lookups." 3. "This is parameter sniffing. Postgres has the same issue with prepared statements once `plan_cache_mode` allows generic plans. Oracle had bind variable peeking, fixed by adaptive cursor sharing." 4. "Fixes: `OPTION (RECOMPILE)` per query (recompiles every time, no caching), `OPTIMIZE FOR` hint (always plan for a representative value), local variables (the planner uses average density), or split into two procs for skewed values."

Diagnosis: Plan cache pinned to a plan that suits the first-seen parameter values, not the current ones.

Fix:

```
-- Option A: recompile every time
CREATE OR ALTER PROCEDURE GetRidesByCity @city_id INT, @from DATETIME2 AS
    SELECT ride_id FROM rides
    WHERE city_id = @city_id AND created_at >= @from
    OPTION (RECOMPILE);

-- Option B: optimize for a representative value
... OPTION (OPTIMIZE FOR (@city_id = UNKNOWN));
```

Why: Plan caching trades recompile cost for stable plans, but a stable plan is wrong when parameter selectivity varies wildly.

NOT to say: "Disable plan caching globally" — kills throughput. "It's a one-off blip" — it'll recur every plan eviction.

Follow-up: "What's the trade-off of `OPTION (RECOMPILE)`?" — Always-fresh plan, but compile cost on every call. Fine for low-frequency procs; not for hot OLTP paths.

Scenario324 · Plan Cache Pollution

Tags: Difficulty: Hard · Asked at: Goldman, Walmart Labs, JP Morgan · Topic: Plan cache · Engines: SQL Server, Oracle, Postgres

Setup:

```
-- Application string-concatenates the user_id rather than binding
SELECT order_id FROM orders WHERE user_id = 7724901;
SELECT order_id FROM orders WHERE user_id = 7724902;
SELECT order_id FROM orders WHERE user_id = 7724903;
-- ... thousands of variants
```

Question: "After a deploy that switched from prepared statements to raw string SQL, the plan cache grew to 8GB and our buffer cache hit ratio crashed. Explain the link."

Thinking out loud: 1. "Each unique query text gets its own plan cache entry. With bound parameters, all `user_id = ?` queries share one plan. With concatenation, each `user_id = 7724901`, `user_id = 7724902` is a different text, so each gets its own cached plan." 2. "On a busy service with millions of distinct values, you flood the plan cache with single-use plans. Memory that should hold data pages is consumed by plan metadata. Buffer cache shrinks; disk I/O rises; latency suffers." 3. "SQL Server has `optimize for ad hoc workloads` to stash a small stub on first execution and the full plan on second use, which helps with truly one-off queries. Postgres uses `plan_cache_mode` and prepared statement reuse. Oracle has cursor sharing modes." 4. "Root fix is at the application layer: use parameterised queries (`PreparedStatement`, `?` placeholders). Database-side workarounds (forced parameterisation, `optimize for ad hoc`) are mitigation, not solution."

Diagnosis: Non-parameterised SQL fragments the plan cache into millions of single-use entries, evicting data pages.

Fix:

```
// Application: bind, don't concatenate
PreparedStatement ps = c.prepareStatement(
    "SELECT order_id FROM orders WHERE user_id = ?");
ps.setLong(1, userId);
```

Plus enable `optimize for ad hoc workloads` (SQL Server) or `FORCED` cursor sharing (Oracle) as defence in depth.

Why: Plan cache is keyed on query text. Unique text per call defeats caching and steals memory from the buffer cache.

NOT to say: *"More RAM fixes plan cache pollution"* — only delays it. *"The cache will self-evict"* — yes, but at the cost of constant recompiles and cache churn.

Follow-up: *"Why is recompiling so expensive?"* — Optimisation walks join orders, picks plans across many indexes, may run statistics queries; can be 10-100x the actual execution cost for a short query.

Scenario325 · Histograms Missing or Stale (MySQL 8)

Tags: Difficulty: Medium · Asked at: Flipkart, Swiggy, Razorpay · Topic: Statistics · Engines: MySQL 8

Setup:

```
CREATE TABLE listings (
    listing_id    BIGINT PRIMARY KEY,
    category_id  INT,
    price_paise  BIGINT,
    created_at   DATETIME
);
CREATE INDEX idx_listings_category ON listings (category_id);
CREATE INDEX idx_listings_price    ON listings (price_paise);
-- Skewed: category 1 has 100K rows, category 2 has 8M, category 3 has 4K
SELECT listing_id FROM listings WHERE category_id = 3 AND price_paise < 50000;
```

Question: *"On MySQL 8, this picks the wrong index — uses `idx_listings_price` when it should use `idx_listings_category`. InnoDB index dives sample a few pages and get the skew wrong. What's the modern fix?"*

Thinking out loud: 1. *"MySQL 8 introduced explicit histograms via `ANALYZE TABLE ... UPDATE HISTOGRAM`. Without histograms, InnoDB's cardinality estimates come from index dives (sampling a*

small number of pages per index), which is fine for uniform data but bad for skew." 2. "For `category_id = 3` (4K rows), index dives might estimate 2M rows because they hit pages dominated by category 2. The planner thinks `idx_listings_category` returns too many rows and picks `idx_listings_price` instead, then filters." 3. "Histograms record actual value distribution buckets. With an equi-height histogram on `category_id`, the planner sees that value 3 is only 4K rows — and picks the right index." 4. "Postgres has had this since forever via `pg_statistic` histograms; SQL Server has step histograms; MySQL 8 is catching up. Per-column setting, refreshed by `ANALYZE TABLE ... UPDATE HISTOGRAM ON category_id WITH 100 BUCKETS`."

Diagnosis: Skewed column without a histogram; planner uses index dives that miss the skew.

Fix:

```
-- MySQL 8 explicit histogram
ANALYZE TABLE listings UPDATE HISTOGRAM ON category_id WITH 100 BUCKETS;
-- Re-EXPLAIN; planner now sees true selectivity
```

Why: Index dives sample pages, not values. They cannot detect skew if the rare value sits on rarely-sampled pages.

NOT to say: "MySQL doesn't support statistics" — it does as of 8.0. "InnoDB stats are always wrong" — only on skewed columns without histograms.

Follow-up: "How many buckets should I use?" — Default 100 is fine; up to 1024 if you have many distinct values and tight selectivity matters. More buckets = more storage, slower ANALYZE.

Scenario326 · Extended Statistics for Correlated Columns

Tags: Difficulty: Hard · Asked at: Goldman, Walmart Labs, Cred · Topic: Statistics · Engines: Postgres

Setup:

```
CREATE TABLE addresses (
  address_id  BIGSERIAL PRIMARY KEY,
  city        VARCHAR(80),
  state       VARCHAR(80),
  pincode     INT
);
CREATE INDEX idx_addresses_city_state ON addresses (city, state);
-- Mumbai is in Maharashtra, Bengaluru in Karnataka, etc — perfect functional dependency
SELECT * FROM addresses WHERE city = 'Mumbai' AND state = 'Maharashtra';
```

Question: "Postgres estimates 200 rows for this; reality is 80,000. It picks a Nested Loop on the outer side which blows up. Why is the estimate so wrong?"

Thinking out loud: 1. "Postgres assumes independence between columns by default: $P(\text{city} = \text{'Mumbai'} \text{ AND } \text{state} = \text{'Maharashtra'}) = P(\text{city} = \text{'Mumbai'}) * P(\text{state} = \text{'Maharashtra'})$. With 400 cities and 30 states, that's roughly $1/400 * 1/30 \approx 8$ in 100,000 — but every Mumbai row IS Maharashtra, so the true joint probability is just $1/400$." 2. "The independence assumption fails for correlated columns (city implies state). The estimate is 100-1000x too low, and downstream join orders get picked based on wrong cardinalities." 3. " `CREATE STATISTICS` (Postgres 10+) lets you declare multi-column statistics: `ndistinct` for distinct-value counts, `dependencies` for functional dependencies, `mcv` for most-common-value lists across columns." 4. "After creating and `ANALYZE` -ing the extended statistics, the planner uses the joint distribution and gets the row estimate right. The plan flips to the right shape — often Bitmap Heap Scan or Hash Join instead of Nested Loop."

Diagnosis: Postgres assumes column independence; correlated columns yield wildly wrong joint estimates.

Fix:

```
CREATE STATISTICS stats_addr_city_state (dependencies, mcv)
  ON city, state FROM addresses;
ANALYZE addresses;
-- EXPLAIN now shows accurate joint row estimate
```

Why: Cost-based planners assume independence between predicates unless told otherwise. Functional dependencies break that.

NOT to say: "Just rewrite the query" — won't fix the cardinality model. "Postgres can't handle correlated columns" — it can, with extended statistics.

Follow-up: "What other engines have something equivalent?" — SQL Server has multi-column statistics auto-created on indexed combinations. Oracle has extended statistics via `DBMS_STATS.CREATE_EXTENDED_STATS`.

Scenario327 · Leftmost-Prefix Misuse on a Wide Composite

Tags: Difficulty: Medium · Asked at: PhonePe, Razorpay, Walmart Labs · Topic: Composite indexes · Engines: All

Setup:

```
CREATE TABLE app_events (
  event_id BIGSERIAL PRIMARY KEY,
  user_id BIGINT,
  session_id UUID,
  event_type VARCHAR(40),
  ts TIMESTAMPTZ
);
CREATE INDEX idx_app_events_uset ON app_events (user_id, session_id, event_type, ts);
-- Query filters on user_id and event_type, skips session_id
SELECT * FROM app_events
WHERE user_id = 7724901 AND event_type = 'click'
ORDER BY ts DESC LIMIT 50;
```

Question: "The composite index covers all the right columns but only the `user_id` part of the prefix gets used; `event_type` filter happens after. Why?"

Thinking out loud: 1. "Composite index prefix is `(user_id, session_id, event_type, ts)`. With `user_id` equality, the index seeks to that user's range. Within it, entries are sorted by `session_id` then `event_type` — but we have no `session_id` predicate." 2. "Without the second column predicate, the planner can use the prefix on `user_id` only; then it must scan all entries for that user and filter on `event_type` post-fetch. The `ts` ordering for ORDER BY can't be cleanly used either, because entries are sorted by `session_id` first within the user." 3. "Reorder the index: `(user_id, event_type, ts)`. Now equality on the first two, range/ordering on the third — classic equality-equality-range pattern. Drop `session_id` from the index unless it's needed for other queries." 4. "If `session_id` is needed for a different query, create a separate index. Don't try to make one super-wide index serve every shape — it ends up serving none well."

Diagnosis: Wide composite with the wrong column placed second blocks the leftmost-prefix path past `user_id`.

Fix:

```
CREATE INDEX idx_app_events_user_type_ts
ON app_events (user_id, event_type, ts DESC);
DROP INDEX idx_app_events_uset;
```

Optionally `INCLUDE (event_id)` for an index-only scan covering the SELECT list.

Why: B-tree composite indexes serve queries only along the leftmost prefix. A "skipped" middle column means everything past it is scan-and-filter.

NOT to say: "Just add all columns and the planner figures it out" — column order matters more than column count. "More columns in the index is always better" — wider indexes are slower to maintain and bigger to traverse.

Follow-up: "Do I always want equality columns first?" — Yes, then range columns, then ORDER BY columns. That's the standard equality-equality-range rule.

Scenario328 · Index-Only Scan Blocked by Missing INCLUDE

Tags: Difficulty: Hard · Asked at: Cred, Razorpay, Goldman · Topic: Covering indexes · Engines: Postgres 11+, SQL Server, MySQL 8

Setup:

```
CREATE TABLE invoice_lines (  
  line_id      BIGSERIAL PRIMARY KEY,  
  invoice_id   BIGINT,  
  sku          VARCHAR(40),  
  qty          INT,  
  price_paise BIGINT  
);  
CREATE INDEX idx_invoice_lines_invoice ON invoice_lines (invoice_id);  
SELECT sku, qty FROM invoice_lines WHERE invoice_id = 88012;
```

Question: "EXPLAIN shows Index Scan + Heap Fetch, not Index Only Scan. The index has `invoice_id`. Why isn't it index-only?"

Thinking out loud: 1. "An Index Only Scan requires every column referenced in the query to be in the index — the predicate columns AND the SELECT list columns. Here `sku` and `qty` are not in the index, so the engine has to go to the heap to read them." 2. "Postgres supports `INCLUDE` columns since version 11 — they're stored in the leaf pages but not part of the B-tree key, so they don't slow comparisons but make the index covering for that query." 3. "SQL Server has had `INCLUDE` since 2005; MySQL 8's secondary indexes always include the PK at the leaf, so wider secondary indexes naturally cover more. Postgres `INCLUDE` is the right primitive." 4. "Add `INCLUDE (sku, qty)` and re-`EXPLAIN`. Should flip to Index Only Scan with `Heap Fetches: 0` (assuming visibility map is up to date — see next scenario)."

Diagnosis: Index is not covering — SELECT list references columns not in the index leaves.

Fix:

```
CREATE INDEX idx_invoice_lines_invoice_cover  
  ON invoice_lines (invoice_id) INCLUDE (sku, qty);  
DROP INDEX idx_invoice_lines_invoice;
```

Why: Index Only Scan avoids the heap altogether, but only if all needed columns live in the index leaves.

NOT to say: "Add the include columns as part of the key" — works, but slows down comparisons and bloats the tree. **INCLUDE** is the cleaner primitive.

Follow-up: "What's the trade-off of a covering index?" — Bigger on disk, slower writes (every UPDATE to an included column rewrites the index entry), but eliminates heap fetches on hot read paths.

Scenario329 · Visibility Map Stale, Index-Only Falls Back

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Walmart Labs · Topic: MVCC, Index-only scan · Engines: Postgres

Setup:

```
CREATE TABLE ledger_entries (  
    entry_id      BIGSERIAL PRIMARY KEY,  
    account_id    BIGINT,  
    amount_paise  BIGINT,  
    posted_at     TIMESTAMPTZ  
);  
CREATE INDEX idx_ledger_account_cover  
    ON ledger_entries (account_id) INCLUDE (amount_paise, posted_at);  
-- Recently bulk-inserted 50M rows  
SELECT amount_paise, posted_at FROM ledger_entries WHERE account_id = 88012;
```

Question: "My covering index just got built. The query has all columns covered. But **EXPLAIN ANALYZE** shows **Heap Fetches: 4982** — why is it still going to the heap?"

Thinking out loud: 1. "Postgres Index Only Scan needs to confirm that each tuple version is visible to the current transaction. It uses the visibility map (VM) — one bit per heap page saying 'all tuples on this page are visible to all transactions.' If the VM bit is set, no heap fetch is needed." 2. "The VM is updated by VACUUM, not by INSERT. Freshly-loaded heap pages have the VM bit unset. So even though the index has the data, the planner can't trust the visibility info without going to the heap." 3. "Fix: run **VACUUM** (not **VACUUM FULL**) on the table. That sets the all-visible bits on stable pages. After VACUUM, Index Only Scan can return data without any heap fetches." 4. "Autovacuum eventually does this, but on a freshly-loaded table you can force it. For appliance-style ETL pipelines, run VACUUM ANALYZE as the final step of the load."

Diagnosis: Visibility map bits not set on newly-loaded heap pages; Index Only Scan falls back to heap fetches.

Fix:

```
VACUUM ANALYZE ledger_entries;  
-- Re-EXPLAIN ANALYZE; Heap Fetches should drop to 0
```

Why: Postgres MVCC needs per-page visibility info to skip heap fetches. The VM is maintained by VACUUM, not by writes.

NOT to say: "Index Only Scan doesn't work in Postgres" — it does, just needs an up-to-date VM. "Rebuild the index" — index is fine; the heap visibility map is the issue.

Follow-up: "Why doesn't VACUUM run more aggressively?" — Cost-based throttling (`vacuum_cost_limit`). On heavy-write tables, you tune `autovacuum_vacuum_scale_factor` lower (default 0.2 means 20% dead tuples before triggering).

Scenario330 · Index on Column With High NULL Ratio

Tags: Difficulty: Medium · Asked at: Swiggy, Zomato, Cred · Topic: Sparse data · Engines: All

Setup:

```
CREATE TABLE delivery_attempts (  
  attempt_id BIGSERIAL PRIMARY KEY,  
  order_id BIGINT,  
  driver_id BIGINT,  
  failure_reason VARCHAR(120) -- NULL for successful attempts (99% of rows)  
);  
CREATE INDEX idx_delivery_failure_reason ON delivery_attempts (failure_reason);  
-- Only failed attempts have a reason; we query those  
SELECT order_id FROM delivery_attempts WHERE failure_reason = 'address_not_found';
```

Question: "99% of rows have NULL in `failure_reason` and my B-tree index is huge. Should NULLs even be in the index? Why is this slow?"

Thinking out loud: 1. "By default, Postgres B-tree indexes include NULL entries. With 99% NULL, the index is mostly useless padding — most pages hold NULL entries that we never query." 2. "This bloats the index, slows scans, and increases write cost (every INSERT writes an index entry even for NULLs we'll never query)." 3. "Partial index on the predicate side is the clean fix:

`WHERE failure_reason IS NOT NULL`. Index becomes 1% of original size, fits in cache, lookups are fast." 4. "Oracle excludes NULLs from B-tree indexes by default (which is why composite indexes with all-NULL rows aren't indexed there) — different default, same lesson. SQL Server lets you create filtered indexes for the same purpose."

Diagnosis: Index dominated by NULL entries that the query never uses; mostly wasted space and write cost.

Fix:

```
-- Postgres partial index
DROP INDEX idx_delivery_failure_reason;
CREATE INDEX idx_delivery_failure_reason_nn
  ON delivery_attempts (failure_reason)
  WHERE failure_reason IS NOT NULL;

-- SQL Server filtered index
CREATE INDEX idx_delivery_failure_reason
  ON delivery_attempts (failure_reason)
  WHERE failure_reason IS NOT NULL;
```

Why: Sparse data benefits from a partial/filtered index that excludes the dominant NULL value.

NOT to say: *"Just drop the index"* — you still need it for failure lookups. *"NULLs don't take space in indexes"* — they do, except in Oracle.

Follow-up: *"What's the difference between a partial index and a filtered index?"* — Naming only. Postgres calls it partial, SQL Server calls it filtered, same idea.

Scenario331 · Index Unused After TRUNCATE

Tags: Difficulty: Medium · Asked at: Razorpay, AmEx, Walmart Labs · Topic: Statistics reset · Engines: All

Setup:

```
-- Nightly ETL truncates and reloads
TRUNCATE TABLE staging_orders;
COPY staging_orders FROM '/data/orders_2026_05_25.csv' CSV;
-- 80M rows loaded
CREATE INDEX idx_staging_orders_cust ON staging_orders (customer_id);
SELECT order_id FROM staging_orders WHERE customer_id = 7724901;
```

Question: *"After the nightly TRUNCATE + COPY, the index exists but queries do Seq Scans for the first hour. Then magically start using the index. What's going on?"*

Thinking out loud: 1. *"TRUNCATE resets table statistics — Postgres considers the table empty (reltuples = 0) until ANALYZE runs. COPY adds rows but doesn't update stats."* 2. *"For the first hour, autovacuum hasn't analysed the table yet. Planner sees a 0-row table and figures Seq Scan is cheaper than even one index lookup. So it scans the (now huge) heap."* 3. *"After autovacuum runs ANALYZE (triggered by `autovacuum_analyze_scale_factor` of changes — but on a 'zero-row' table, even one row is infinite percent change, so it usually fires quickly), stats reflect reality and the planner switches to Index Scan."* 4. *"Don't wait. Always run `ANALYZE` immediately after a TRUNCATE + reload. Or wrap the load in a script that ends with `ANALYZE staging_orders;`."*

Diagnosis: TRUNCATE reset stats to zero; planner uses stale 0-row estimate until ANALYZE catches up.

Fix:

```
TRUNCATE TABLE staging_orders;
COPY staging_orders FROM '/data/orders_2026_05_25.csv' CSV;
ANALYZE staging_orders; -- run explicitly
```

Why: TRUNCATE is metadata-only fast, but it nukes the row count and stats. Planner cannot make good decisions on empty-table stats.

NOT to say: "TRUNCATE is the same as DELETE for stats" — DELETE keeps stats but marks rows dead; TRUNCATE resets to zero. "Just rebuild the index" — index is fine; stats are stale.

Follow-up: "Why does autovacuum take an hour to catch up here?" —

`autovacuum_analyze_threshold` (50 rows default) plus `scale_factor`; on tables with frequent zero-to-huge cycles, lower the `scale_factor` or run ANALYZE explicitly.

Scenario332 · Partial Index Predicate Doesn't Match Query

Tags: Difficulty: Medium · Asked at: Cred, Razorpay, Flipkart · Topic: Partial indexes · Engines: Postgres, SQL Server

Setup:

```
CREATE TABLE payouts (
  payout_id    BIGSERIAL PRIMARY KEY,
  vendor_id    BIGINT,
  status       VARCHAR(20),
  amount_paise BIGINT,
  created_at   TIMESTAMPTZ
);
CREATE INDEX idx_payouts_pending
  ON payouts (vendor_id, created_at)
  WHERE status = 'pending';
-- Query
SELECT payout_id FROM payouts
WHERE vendor_id = 88012 AND status IN ('pending', 'failed');
```

Question: "My partial index on `status = 'pending'` doesn't get used when the query asks for `status IN ('pending', 'failed')`. The index would still help for the pending portion. Why doesn't the planner use it?"

Thinking out loud: 1. "For a partial index to be used, the planner must prove the query's predicate implies the index's WHERE clause. `status IN ('pending', 'failed')` is a superset of `status = 'pending'` — it does NOT imply `status = 'pending'`, it could match `failed` too. So the planner cannot use the partial index alone — it would miss the `failed` rows." 2. "Implication direction

matters: query predicate => index predicate. `status = 'pending'` => `status = 'pending'` (yes). `status IN ('pending','failed')` => `status = 'pending'` (no). `status = 'pending' AND amount > 1000` => `status = 'pending'` (yes)." 3. "Options: (a) split into UNION of two queries, one with `status = 'pending'` (uses partial) and one with `status = 'failed'` (needs different index); (b) create a broader partial index `WHERE status IN ('pending','failed')`; (c) drop the partial in favour of a full index on `(vendor_id, status, created_at)` ." 4. "Real call depends on volumes — if 'pending' is 0.1% of payouts and 'failed' is 0.05%, two narrow partial indexes plus UNION is the cleanest. If both states together are still selective enough, one broader partial works."

Diagnosis: Query predicate does not imply the partial index predicate; planner cannot use it.

Fix:

```
-- Option A: broaden the partial
DROP INDEX idx_payouts_pending;
CREATE INDEX idx_payouts_active
  ON payouts (vendor_id, created_at)
  WHERE status IN ('pending', 'failed');

-- Option B: UNION rewrite
SELECT payout_id FROM payouts WHERE vendor_id = 88012 AND status = 'pending'
UNION ALL
SELECT payout_id FROM payouts WHERE vendor_id = 88012 AND status = 'failed';
```

Why: Partial-index applicability is governed by predicate implication, not predicate overlap. Planner needs proof.

NOT to say: "The planner is too strict" — using a partial index for a non-implied query would return wrong results. "Just add a hint" — hints can't override correctness.

Follow-up: "How do I check whether a partial index will be used?" — `EXPLAIN` the query and look for the index name. Postgres can also do `EXPLAIN (BUFFERS)` to confirm pages read.

Scenario333 · Expression Index Not Matched Literally

Tags: Difficulty: Hard · Asked at: Cred, Goldman, Walmart Labs · Topic: Expression indexes · Engines: Postgres, Oracle

Setup:

```
CREATE TABLE customers (
  customer_id BIGSERIAL PRIMARY KEY,
  full_name VARCHAR(200),
  email VARCHAR(120)
);
CREATE INDEX idx_customers_name_lower ON customers (LOWER(full_name));
-- Query
SELECT customer_id FROM customers WHERE LOWER(full_name) = 'priya sharma';
SELECT customer_id FROM customers WHERE LOWER(full_name) ILIKE 'priya sharma';
SELECT customer_id FROM customers WHERE full_name ILIKE 'priya sharma'; -- doesn't use it
```

Question: "The expression index on `LOWER(full_name)` works for `WHERE LOWER(full_name) = ...` but doesn't get used for `WHERE full_name ILIKE 'priya sharma'`. Both seem case-insensitive. Why?"

Thinking out loud: 1. "Expression indexes are matched textually — the predicate expression must syntactically match the indexed expression (up to a few simplifications). `LOWER(full_name) = 'priya sharma'` matches. `full_name ILIKE 'priya sharma'` is a different operator entirely (`~*`) and the planner cannot see the equivalence." 2. "Even `LOWER(full_name) = LOWER('PRIYA SHARMA')` may not match if the planner doesn't constant-fold the RHS. Best practice: keep both sides clean — `LOWER(col) = 'lowercase_literal'`." 3. "For `ILIKE` patterns, you'd either need to keep using `LOWER(col) LIKE 'priya%'` (which uses the expression index for prefix patterns), or use the `citext` extension which makes the column case-insensitive at the type level." 4. "Or in Postgres, the `pg_trgm` extension supports `ILIKE` patterns directly with a GIN index — different tool for a different shape."

Diagnosis: Predicate doesn't syntactically match indexed expression; planner cannot use the expression index.

Fix:

```
-- Option A: rewrite to match
SELECT customer_id FROM customers
WHERE LOWER(full_name) LIKE 'priya sharma';

-- Option B: citext column
ALTER TABLE customers ALTER COLUMN full_name TYPE citext;
CREATE INDEX idx_customers_name ON customers (full_name);
-- Now any case-insensitive comparison uses the index
```

Why: Expression index matching is syntactic. The planner does limited normalisation; not arbitrary equivalence proofs.

NOT to say: "The planner should figure out equivalent expressions" — it can't, in general. "Just use `ILIKE` everywhere" — it bypasses your carefully-built index.

Follow-up: "Why doesn't `LOWER(full_name) = LOWER('PRIYA SHARMA')` always match?" — Some planner versions constant-fold the RHS, others don't. Always test with `EXPLAIN`.

Scenario334 · Foreign Key Without Index — Slow Cascading DELETE

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Goldman · Topic: FK indexes · Engines: All

Setup:

```
CREATE TABLE customers (  
  customer_id  BIGSERIAL PRIMARY KEY,  
  name         VARCHAR(120)  
);  
CREATE TABLE orders (  
  order_id     BIGSERIAL PRIMARY KEY,  
  customer_id  BIGINT REFERENCES customers(customer_id) ON DELETE CASCADE  
  -- no index on customer_id!  
);  
CREATE TABLE order_items (  
  item_id      BIGSERIAL PRIMARY KEY,  
  order_id     BIGINT REFERENCES orders(order_id) ON DELETE CASCADE  
  -- no index on order_id either  
);  
  
DELETE FROM customers WHERE customer_id = 7724901; -- takes 8 minutes
```

Question: "Deleting one customer takes 8 minutes. Cascading deletes should be quick. What's wrong?"

Thinking out loud: 1. "When you `DELETE` a customer, Postgres has to find all referencing rows in `orders` and delete them, then for each deleted order find all `order_items` and delete those. If there's no index on the referencing column, each lookup is a full table scan." 2. "Postgres does NOT auto-create indexes on FK columns (only on the referenced PK side). MySQL InnoDB does auto-create one. SQL Server doesn't. Oracle doesn't. So in most engines, you must remember to add it." 3. "For 50M orders and 200M order_items, the cascade walks both tables linearly per customer. Add indexes on the FK columns and the cascade becomes $O(\text{rows_to_delete})$ instead of $O(\text{rows_in_table})$." 4. "Same issue with `UPDATE CASCADE` on a PK change (rare) and with the FK validity check on `INSERT` (less impactful because PK is indexed)."

Diagnosis: Foreign key columns not indexed; cascade walks each child table fully.

Fix:

```
CREATE INDEX idx_orders_customer  ON orders (customer_id);  
CREATE INDEX idx_order_items_order ON order_items (order_id);  
-- Cascade DELETE now uses index lookups; sub-second
```


Why: ON DELETE CASCADE issues a DELETE against child tables filtered by the parent key. Without an index on the FK column, that DELETE does a full scan.

NOT to say: "FKs auto-create indexes" — only MySQL InnoDB does. "Cascade is always slow" — only without indexes.

Follow-up: "How do I find unindexed FKs in Postgres?" — Query `pg_constraint` joined with `pg_index` to find FKs without a matching leftmost-prefix index. A common DBA audit query.

Scenario335 · ORDER BY Causes Sort Despite Index

Tags: Difficulty: Medium · Asked at: Cred, Flipkart, Walmart Labs · Topic: Sort avoidance · Engines: All

Setup:

```
CREATE TABLE notifications (  
  notif_id      BIGSERIAL PRIMARY KEY,  
  user_id       BIGINT,  
  created_at    TIMESTAMPTZ,  
  read_status   VARCHAR(10)  
);  
CREATE INDEX idx_notif_user_created ON notifications (user_id, created_at);  
-- Want: latest 20 for a user  
SELECT notif_id FROM notifications  
WHERE user_id = 88012  
ORDER BY created_at DESC LIMIT 20;
```

Question: "EXPLAIN shows the index gets used for the WHERE but a Sort step appears after. The index has `created_at` — why does the planner still sort?"

Thinking out loud: 1. "The index is ascending on `(user_id, created_at)`. Within `user_id = 88012`, entries are sorted by `created_at ASC`. The ORDER BY wants `created_at DESC`." 2. "Postgres can read the B-tree backward — `Index Scan Backward` — and avoid the sort. But the planner only does this if the ORDER BY exactly matches the index's reverse order. Sometimes it picks Sort anyway if cost estimates differ." 3. "Cleanest fix: define the index in the order the query wants. `CREATE INDEX ... ON notifications (user_id, created_at DESC)`. Now forward scan satisfies the LIMIT directly." 4. "This also matters for composite ORDER BY with mixed directions — `ORDER BY user_id ASC, created_at DESC`. Backward scan only works if all columns flip together. Mixed-direction ORDER BY needs an index with matching mixed directions, or a Sort step."

Diagnosis: Index sort order doesn't match ORDER BY direction; planner adds a Sort step.

Fix:

```
CREATE INDEX idx_notif_user_created_desc
  ON notifications (user_id, created_at DESC);
DROP INDEX idx_notif_user_created;
```

Now `LIMIT 20` returns after fetching 20 index entries; no sort.

Why: B-tree indexes have a fixed order. Reading backward works for unanimous reverse, but mixed-direction `ORDER BY` needs a matching mixed-direction index.

NOT to say: *"The planner should always read backward"* — it does when it can; the issue is mixed direction or cost estimates. *"ORDER BY is always sortable for free"* — only when the index provides the order.

Follow-up: *"What about `ORDER BY user_id DESC, created_at ASC`?"* — Needs `(user_id DESC, created_at ASC)` index, or accept a Sort. Postgres won't read part of an index in one direction and part in the other.

Scenario336 · When to REINDEX

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Walmart Labs · Topic: Maintenance · Engines: Postgres especially

Setup:

```
-- Production observability over 6 months
-- pg_stat_user_indexes shows idx_orders_customer fragmentation; bloat 45%
-- Lookups slowed from 2ms p99 to 18ms p99
```

Question: *"When do you decide it's time to REINDEX a hot production index, and how do you do it without taking the service down?"*

Thinking out loud: 1. *"Indicators: index size growing faster than table size, bloat ratio (via `pgstattuple`) over 30%, latency on indexed lookups creeping up, `pg_stat_user_indexes.idx_scan` healthy but per-lookup time rising."* 2. *"Don't REINDEX on a hunch. Measure bloat first — `pgstattuple_approx(indexrelid)` for cheap estimates on huge indexes, `pgstattuple` for accurate (but slower) numbers."* 3. *"Use `REINDEX INDEX CONCURRENTLY` (Postgres 12+). It builds the new index in parallel, takes only short locks at swap time. Takes 1.5-2x longer than plain REINDEX but no exclusive lock."* 4. *"Schedule during low-traffic windows even with `CONCURRENTLY` — the rebuild generates I/O and WAL. For very large indexes (100GB+), consider partitioning the table so each partition's index is smaller and rebuildable independently."*

Diagnosis: Heavy-write indexes accumulate bloat; periodic REINDEX restores compactness.

Fix:

```
-- Measure first
SELECT pg_size_pretty(pg_relation_size('idx_orders_customer'));
SELECT * FROM pgstattuple_approx('idx_orders_customer');

-- Rebuild online
REINDEX INDEX CONCURRENTLY idx_orders_customer;

-- Confirm size shrank
SELECT pg_size_pretty(pg_relation_size('idx_orders_customer'));
```

Why: VACUUM cleans dead tuples in the heap and marks dead index entries reusable, but doesn't physically compact B-tree pages. REINDEX rebuilds the structure densely.

NOT to say: "REINDEX weekly on every index" — wasteful for low-write tables. "REINDEX always takes the table down" — CONCURRENTLY exists.

Follow-up: "What if CONCURRENTLY fails halfway?" — It leaves a **INVALID** index behind (marked unusable). Drop it manually and retry. **pg_stat_user_indexes** flags invalid indexes.

Scenario337 · VACUUM ANALYZE Cadence

Tags: Difficulty: Medium · Asked at: Cred, Razorpay, Goldman · Topic: Maintenance · Engines: Postgres

Setup:

```
-- 2TB OLTP table, 5K writes per second
-- Default autovacuum_vacuum_scale_factor = 0.2 means autovacuum runs after
-- 20% of table changed = 400M rows; takes hours; floods I/O
```

Question: "On a 2TB Postgres OLTP table with heavy writes, default autovacuum thresholds are wrong. How do you tune VACUUM and ANALYZE cadence?"

Thinking out loud: 1. "The default **autovacuum_vacuum_scale_factor** is 0.2 (vacuum when 20% of rows changed). On a 2TB table that's 400M dead tuples before autovacuum fires — bloat climbs, queries slow, then a huge VACUUM hits and floods I/O." 2. "For large tables, tune per-table: **ALTER TABLE orders SET (autovacuum_vacuum_scale_factor = 0.02);** so vacuum kicks in after 2% changed instead of 20%. Smaller, more frequent VACUUMs are friendlier than infrequent huge ones." 3. "Same for ANALYZE — **autovacuum_analyze_scale_factor = 0.01** on large tables keeps stats fresh. Stats drift causes plan regressions; cheap to keep them current." 4. "Watch **pg_stat_user_tables.last_autovacuum** and **n_dead_tup**. If autovacuum is falling behind, raise **autovacuum_max_workers** and **autovacuum_vacuum_cost_limit** (default 200 throttles aggressively)."

Diagnosis: Default autovacuum thresholds are tuned for medium tables; large OLTP tables need per-table overrides.

Fix:

```
ALTER TABLE orders SET (  
    autovacuum_vacuum_scale_factor = 0.02,  
    autovacuum_analyze_scale_factor = 0.01,  
    autovacuum_vacuum_cost_limit   = 2000  
);
```

Why: Autovacuum is throttled and proportional to table size. Defaults assume mid-sized tables; large tables need tighter percentages.

NOT to say: *"Disable autovacuum and run nightly VACUUM"* — guarantees bloat during the day and a thundering herd at night. *"VACUUM FULL is the proper fix"* — it locks the whole table; only for offline maintenance.

Follow-up: *"What's the right thing to monitor?"* — `n_dead_tup`, `last_autovacuum`, `n_mod_since_analyze`, and bloat ratios from `pgstattuple_approx`. Alert when any of these drift past thresholds.

Scenario338 · Indexes Bigger Than the Table

Tags: Difficulty: Medium · Asked at: Flipkart, Walmart Labs, Cred · Topic: Index sizing · Engines: All

Setup:

```
CREATE TABLE products (  
    product_id  BIGSERIAL PRIMARY KEY,  
    sku         VARCHAR(40),  
    name        VARCHAR(200),  
    brand       VARCHAR(80),  
    category_id INT,  
    price_paise BIGINT,  
    in_stock    BOOLEAN  
);  
-- Over time, 9 indexes added: sku, name, brand, category_id, price_paise,  
-- in_stock, (brand, category_id), (category_id, price_paise), (sku, in_stock)  
-- Table: 4 GB. Sum of indexes: 11 GB.
```

Question: *"Indexes total 11GB on a 4GB table. Write throughput dropped to 60% of last quarter. What's the assessment and what do you cut?"*

Thinking out loud: 1. *"Every INSERT updates every applicable index. Every UPDATE to an indexed column updates that index. Nine indexes mean nine extra writes per row insert. That's the throughput*

hit." 2. "Audit what's actually used: `pg_stat_user_indexes.idx_scan` shows lookup counts per index. Anything with `idx_scan = 0` over 30 days is dead weight — drop it." 3. "Then dedupe: `(brand, category_id)` and `(category_id, price_paise)` overlap nothing usable; a single broader composite often serves both query shapes if column orders align. Check actual query patterns first." 4. "Low-cardinality column indexes like `in_stock` (2 values) are almost always wasteful — partial index `WHERE in_stock = false` if the rare value is queried; else drop."

Diagnosis: Index sprawl from years of one-off additions; write amplification crushes throughput.

Fix:

```
-- Find unused indexes
SELECT schemaname, relname, indexrelname, idx_scan, pg_size_pretty(pg_relation_size(indexrelid))
FROM pg_stat_user_indexes
WHERE idx_scan = 0
ORDER BY pg_relation_size(indexrelid) DESC;

-- Drop the dead weight
DROP INDEX idx_products_in_stock;
DROP INDEX idx_products_brand;      -- subsumed by (brand, category_id)
DROP INDEX idx_products_category;  -- subsumed by (category_id, price_paise)
```

Why: Indexes are write-amplifying. The mantra: every index has a cost; only keep ones whose lookups pay for that cost.

NOT to say: "More indexes never hurts" — they do, on writes and on cache. "Just add storage" — doesn't help throughput.

Follow-up: "How long should I observe `idx_scan` before dropping?" — At least one full business cycle: 30 days minimum, 90 days for quarter-end report indexes.

Scenario339 · Bloat From Frequent Updates on Indexed Columns

Tags: Difficulty: Hard · Asked at: Cred, Razorpay, Walmart Labs · Topic: HOT updates · Engines: Postgres

Setup:

```
CREATE TABLE wallet_balances (
  wallet_id      BIGINT PRIMARY KEY,
  balance_paise BIGINT,
  last_txn_at    TIMESTAMPTZ
);
CREATE INDEX idx_wallet_last_txn ON wallet_balances (last_txn_at);
-- Every transaction updates balance_paise and last_txn_at
-- Index on last_txn_at bloats 10x in a month
```

Question: " `last_txn_at` updates every transaction. The index on it bloats badly. Postgres has HOT updates that should help but they're not kicking in. Why, and what do you do?"

Thinking out loud: 1. "HOT (Heap-Only Tuple) updates in Postgres avoid index updates when (a) no indexed column changes, and (b) the new tuple fits on the same page. Here `last_txn_at` IS indexed, so HOT is disabled for these updates — every update writes a new heap tuple AND a new index entry." 2. "Each new index entry pushes the dead one toward eventual VACUUM. Heavy update rate outruns VACUUM; index grows." 3. "Two strategic options. (a) Drop the index on `last_txn_at` if the lookup it supports is rare — restoring HOT eligibility. (b) Tune `fillfactor` lower (e.g., 70) so each page has more free space, increasing the chance a HOT-tuple fits." 4. "For the wallet pattern specifically, a separate `wallet_activity` table that's append-only often beats indexing the in-place balance column. Splits the read pattern from the write hot-spot."

Diagnosis: Indexed column is being updated frequently, blocking HOT updates and bloating the index.

Fix:

```
-- Lower fillfactor to give more in-page room for HOT
ALTER TABLE wallet_balances SET (fillfactor = 70);
-- Reload existing data (CLUSTER or pg_repack) to apply
CLUSTER wallet_balances USING wallet_balances_pkey;

-- Or remove the bloat-causing index if it's not earning its keep
DROP INDEX idx_wallet_last_txn;
```

Why: HOT requires no indexed column to change. Index on a frequently-updated column blocks HOT, causing index bloat and write amplification.

NOT to say: "Just REINDEX nightly" — masks the cause. "HOT is automatic in all cases" — only when no indexed column changes and the new tuple fits.

Follow-up: "How do I check HOT update ratio?" — `pg_stat_user_tables.n_tup_hot_upd / n_tup_upd`. Healthy systems are 90%+. If yours is below 50%, an indexed column is being updated heavily.

Scenario340 · Online Index Creation Without Downtime

Tags: Difficulty: Hard · Asked at: Razorpay, PhonePe, Flipkart, Walmart Labs · Topic: Online DDL · Engines: All

Setup:

```
-- 800 GB orders table, p99 5ms; need to add an index on (vendor_id, created_at)
-- Cannot take downtime; cannot block writes for more than a few seconds
```

Question: "You need to add a multi-column index on an 800GB hot OLTP table without taking the application down. Walk me through how you'd do it on Postgres, MySQL, and SQL Server."

Thinking out loud: 1. "Postgres: `CREATE INDEX CONCURRENTLY`. Takes only `ShareUpdateExclusive` lock (not `AccessExclusive`); reads and writes proceed. Build takes 2-3x longer than blocking `CREATE INDEX` because it does two table scans. Watch for `INVALID` index state on failure." 2. "MySQL InnoDB: `ALTER TABLE ... ADD INDEX ALGORITHM=INPLACE, LOCK=NONE` for most index types. Blocks only briefly at the start and end. For huge tables and limits InnoDB can't handle, `pt-online-schema-change` (Percona) or `gh-ost` (GitHub) build a shadow table and swap." 3. "SQL Server: `CREATE INDEX ... WITH (ONLINE = ON)`. Enterprise Edition only. Takes a brief schema-mod lock; otherwise concurrent reads and writes work. Resumable Indexes (2017+) let you pause and resume long builds." 4. "Across all engines: pick a low-traffic window, monitor replication lag (the WAL/binlog spike from the build can stress replicas), set statement timeouts so a failed build doesn't hang forever, and verify the new index name is unique before starting."

Diagnosis: Need to add an index online without blocking writes; each engine has its own primitive.

Fix:

```
-- Postgres
CREATE INDEX CONCURRENTLY idx_orders_vendor_created
  ON orders (vendor_id, created_at);
-- Check for INVALID state
SELECT indexrelid::regclass FROM pg_index WHERE indisvalid = false;

-- MySQL 8 InnoDB
ALTER TABLE orders ADD INDEX idx_orders_vendor_created (vendor_id, created_at),
  ALGORITHM=INPLACE, LOCK=NONE;

-- SQL Server (Enterprise)
CREATE INDEX idx_orders_vendor_created
  ON dbo.orders (vendor_id, created_at)
  WITH (ONLINE = ON, RESUMABLE = ON);
```

Why: All major engines have online index build paths because production schema changes are mandatory. Each uses a different mechanism (concurrent build, in-place ALTER, online index option) but the goal is the same — no read/write downtime.

NOT to say: "Just take downtime" — unacceptable for OLTP services. "Use a maintenance window for everything" — these primitives exist precisely so you don't have to.

Follow-up: "What if `CREATE INDEX CONCURRENTLY` fails halfway?" — Postgres leaves an `INVALID` index. You must `DROP INDEX CONCURRENTLY` it (still without blocking writes) before retrying. Always script the retry path; never assume first-try success on multi-hour builds.

Category 6, Part A — Transaction Conflict: Deadlocks & Lock Ordering

Two transactions enter, both want the same rows in different order, both block, the engine kills one — that is the entire shape of every deadlock you will ever debug. Part A walks through how real-world deadlocks form (transfers, hot rows, foreign keys, gap locks, lock escalation), how the engine detects them (wait-for graph cycles), and the fixes you actually ship in production: deterministic lock ordering, application-level retry with backoff, design changes that remove the conflict altogether (sharded counters, atomic upserts, optimistic versioning), and the harder calls — when to reach for a saga instead of a distributed transaction. Every scenario gives you a runnable two-transaction trace, the engine's verbatim output, the diagnosis, and three fix options with trade-offs. Part B will pick up isolation-level anomalies (lost updates, write skew, phantom reads) — read this part first; you cannot reason about isolation bugs until the deadlock vocabulary is muscle memory.

Scenario341 · Two-Account Transfer Deadlock — The Classic

Tags: Difficulty: Easy · Asked at: Razorpay, PhonePe, Cred, JP Morgan · Dialect: vendor-agnostic

The scenario:

Two users initiate transfers at the same instant. Transaction T1 moves two thousand rupees from account 101 to account 202. Transaction T2 moves five hundred rupees from account 202 to account 101. Both transactions update the source account first, then the destination. Within milliseconds the engine kills one of them with a deadlock error.

T1 (transaction 1):

```
BEGIN;
UPDATE accounts SET balance = balance - 2000 WHERE id = 101;
-- pauses here briefly while application logs the debit
UPDATE accounts SET balance = balance + 2000 WHERE id = 202;
COMMIT;
```

T2 (transaction 2):

```
BEGIN;
UPDATE accounts SET balance = balance - 500 WHERE id = 202;
-- pauses here briefly
UPDATE accounts SET balance = balance + 500 WHERE id = 101;
COMMIT;
```


What happens:

```
ERROR: deadlock detected
DETAIL: Process 18421 waits for ShareLock on transaction 990012; blocked by process 18430.
        Process 18430 waits for ShareLock on transaction 990013; blocked by process 18421.
HINT:   See server log for query details.
```

Thinking out loud: 1. "T1 has held a row-exclusive lock on row 101 since its first UPDATE and now wants 202." 2. "T2 has held a row-exclusive lock on row 202 and now wants 101 — that is a perfect cycle." 3. "The wait-for graph contains an edge $T1 \rightarrow T2$ and $T2 \rightarrow T1$, so the deadlock detector picks one victim and kills it. The other one finishes." 4. "Fix is deterministic lock ordering — lock the lower account id first regardless of which direction the money moves."

The diagnosis: Each transaction has acquired an exclusive lock on one row and is waiting for the other. The wait-for graph has a two-node cycle, which is the textbook deadlock. The engine's detector runs every second or so, walks the graph, finds the cycle, and aborts the cheaper-to-rollback transaction.

The fix: - **Option A — Lock ordering:** Sort the two account ids before touching them. Always update the smaller id first, then the larger. Both transactions now want 101 first, then 202; T2 waits cleanly for T1 and no cycle forms. - **Option B — Application retry:** Catch the deadlock SQLSTATE (40P01 in Postgres, 1213 in MySQL), wait fifty to two hundred milliseconds with jitter, then re-run the whole transaction. Works only if the transaction is idempotent or you stamp it with a request id. - **Option C — Atomic ledger row:** Insert a single row into a `transfers` table inside one statement, then let a trigger or a nightly batch reconcile balances. Removes the two-row write entirely.

What NOT to say: - Do not claim "use SERIALIZABLE to fix it" — serializable makes it worse because the engine takes more conservative locks and you get more deadlocks, not fewer.

Common follow-up: "How do you decide the lock order when the rows aren't numeric ids — say, two UUIDs?"

Scenario342 · INSERT vs UPDATE Conflict on a Hot Row

Tags: Difficulty: Medium · Asked at: Swiggy, Zomato, Flipkart, Walmart Labs · Dialect: Postgres

The scenario:

A restaurant's `daily_stats` table has one row per (restaurant_id, date). T1 inserts a new row for today during a midnight rollover job. T2, started a few milliseconds earlier, updates today's row to bump the order counter. T1 sits and waits, then deadlocks.

T1 (rollover job):

```
BEGIN;
INSERT INTO daily_stats (restaurant_id, date, orders)
VALUES (4501, CURRENT_DATE, 0)
ON CONFLICT (restaurant_id, date) DO NOTHING;
UPDATE restaurants SET last_rollover_at = NOW() WHERE id = 4501;
COMMIT;
```

T2 (order ingestion):

```
BEGIN;
UPDATE restaurants SET total_orders = total_orders + 1 WHERE id = 4501;
INSERT INTO daily_stats (restaurant_id, date, orders) VALUES (4501, CURRENT_DATE, 1)
ON CONFLICT (restaurant_id, date) DO UPDATE SET orders = daily_stats.orders + 1;
COMMIT;
```

What happens: T1 holds the index lock on the new `daily_stats` row and wants the `restaurants` row 4501. T2 holds row 4501 in `restaurants` and is now waiting for the unique-index slot on `daily_stats`. Postgres detects the cycle and aborts one.

Thinking out loud: 1. "Both transactions touch two tables but in opposite order — restaurant first vs stats first." 2. "ON CONFLICT still acquires an index-level lock on the unique key while it figures out whether to insert or update." 3. "Engine sees the cycle and kills the lower-priority transaction (Postgres uses `deadlock_timeout`, default 1s)." 4. "Fix is to standardise the table order: every transaction touching both must touch `restaurants` before `daily_stats`, or vice versa, but never mixed."

The diagnosis: This is a multi-table version of the classic deadlock. The cycle forms across tables, not within one row. The trigger is that two transactions touched the same pair of resources in opposite order.

The fix: - Option A — Table-order convention: Document in the codebase: "Always update `restaurants` before `daily_stats`." Enforce in a code review checklist or a lint rule. - **Option B —**

Single statement: Push the counter bump into a single `INSERT ... ON CONFLICT DO UPDATE` so the rollover job becomes a no-op when ingestion has already created the row. - **Option C —**

Background rollover: Drop the explicit rollover transaction entirely; let lazy creation via `ON CONFLICT` in the ingestion path handle it. Run a cleanup job only for restaurants that had no orders.

What NOT to say: - Do not say "ON CONFLICT is atomic so it can't deadlock" — it is atomic at the row level, but the surrounding transaction can still deadlock with other transactions touching different rows or tables.

Common follow-up: "What if the unique index is on `(restaurant_id, date)` but you also have a partial index on `date` where `orders > 0` — does that change anything?"

Scenario343 · Foreign Key Deadlock — Child INSERT Blocked by Parent UPDATE

Tags: Difficulty: Medium · Asked at: PhonePe, Cred, Goldman, Wells Fargo · Dialect: MySQL InnoDB

The scenario:

An `orders` table has a foreign key to `customers`. T1 updates a customer's address. T2 inserts a new order for the same customer. The INSERT needs to verify the customer still exists, which takes a shared lock on the parent row. T1 needs an exclusive lock on the parent, T2 needs a shared lock. Then T1 also tries to insert an audit row — and the cycle closes.

T1 (customer update + audit):

```
BEGIN;
UPDATE customers SET address = 'new address' WHERE id = 7001;
INSERT INTO customer_audit (customer_id, change) VALUES (7001, 'address');
COMMIT;
```

T2 (new order):

```
BEGIN;
INSERT INTO orders (customer_id, amount) VALUES (7001, 1499);
UPDATE customers SET last_order_at = NOW() WHERE id = 7001;
COMMIT;
```

What happens:

```
ERROR 1213 (40001): Deadlock found when trying to get lock; try restarting transaction
```

Thinking out loud: 1. "T1 has an exclusive lock on `customers(7001)`. T2's INSERT into `orders` needs a shared lock on the parent for FK validation — it waits." 2. "T1 inserts into `customer_audit`, which has its own FK back to `customers` — depending on InnoDB's lock cascade, it may need to revalidate." 3. "Meanwhile, T2's second statement tries to take an X lock on `customers(7001)` — and now there's a cycle: T1 holds X on customers, T2 waits for X, but T2 also holds the index slot on orders." 4. "InnoDB detects the wait-for cycle and aborts the smaller transaction (lowest weight, typically fewest locks held)."

The diagnosis: Foreign-key checks acquire shared locks on parent rows. When a transaction updates the parent and another inserts a child concurrently, the locks compete. The cycle forms when each transaction needs both rows.

The fix: - Option A — Skip FK locks: In MySQL, set `foreign_key_checks = 0` for bulk inserts (only when you can vouch for the data) — removes the shared lock acquisition on the parent. - **Option B — Defer the parent update:** T2's second UPDATE on `customers.last_order_at` is the trigger. Move it to an asynchronous worker that batches updates every few seconds. - **Option C — Lock the parent explicitly first:** Both transactions issue `SELECT ... FOR UPDATE` on the parent row at the start. Everyone serializes on that lock, no cycle.

What NOT to say: - Do not advise "drop the foreign key" — it removes the symptom but loses the integrity guarantee, and reviewers will catch it instantly.

Common follow-up: *"InnoDB takes shared locks for FK checks. What about Postgres — does it behave the same way?"*

Scenario344 · Composite Key Lock Acquisition Order

Tags: Difficulty: Medium · Asked at: Flipkart, Myntra, Target, AmEx · Dialect: vendor-agnostic

The scenario:

An `inventory` table is keyed on `(warehouse_id, sku)`. T1 decrements stock for SKU A in warehouse 1, then SKU B in warehouse 2. T2 decrements SKU B in warehouse 2, then SKU A in warehouse 1. Same two rows, opposite order, instant deadlock.

T1:

```
BEGIN;
UPDATE inventory SET qty = qty - 1 WHERE warehouse_id = 1 AND sku = 'A';
UPDATE inventory SET qty = qty - 1 WHERE warehouse_id = 2 AND sku = 'B';
COMMIT;
```

T2:

```
BEGIN;
UPDATE inventory SET qty = qty - 1 WHERE warehouse_id = 2 AND sku = 'B';
UPDATE inventory SET qty = qty - 1 WHERE warehouse_id = 1 AND sku = 'A';
COMMIT;
```

What happens: The engine kills one transaction with a deadlock error.

Thinking out loud: 1. "Composite keys do not help here — what matters is the order of physical row locks, not the index structure." 2. "T1 holds (1, A), waits for (2, B). T2 holds (2, B), waits for (1, A). Cycle."

3. "The fix is the same as single-column: sort the row identifiers and acquire locks in a global order." 4. "For composite keys, sort by warehouse_id first, then sku — lexicographic order on the tuple."

The diagnosis: Composite keys do not change the deadlock arithmetic. The engine still holds row locks indexed by the heap or clustered-index position. A consistent ordering rule, applied at the application layer, eliminates the cycle.

The fix: - **Option A — Lexicographic lock order:** Sort the rows you intend to touch by (warehouse_id, sku) before the first UPDATE. Both transactions converge to the same order. - **Option B — Pre-flight SELECT FOR UPDATE in order:** Issue `SELECT ... FOR UPDATE` for all rows in sorted order at the start of the transaction, then do the writes. - **Option C — Batch operation:** Replace N individual UPDATES with a single multi-row UPDATE statement; the engine handles locking internally and is far less likely to deadlock.

What NOT to say: - Do not say "MySQL takes the lock on the primary key, so order doesn't matter" — order matters at the row level, regardless of how the index is structured.

Common follow-up: "If the rows you need to lock are determined by a JOIN at runtime, how do you sort them without first reading the data?"

Scenario345 · Index Gap Lock Deadlock — MySQL Repeatable Read

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman, JP Morgan · Dialect: MySQL InnoDB

The scenario:

InnoDB at Repeatable Read takes gap locks on the index to prevent phantoms. T1 inserts an order with id 5005. T2 inserts id 5007. Both inserts fall into the same index gap, and both transactions get blocked on each other's gap lock — classic phantom-prevention deadlock.

T1:

```
BEGIN;
SELECT * FROM orders WHERE id BETWEEN 5000 AND 5010 FOR UPDATE;
INSERT INTO orders (id, customer_id, amount) VALUES (5005, 7001, 1499);
COMMIT;
```

T2:

```
BEGIN;  
SELECT * FROM orders WHERE id BETWEEN 5000 AND 5010 FOR UPDATE;  
INSERT INTO orders (id, customer_id, amount) VALUES (5007, 7002, 999);  
COMMIT;
```

What happens:

```
ERROR 1213 (40001): Deadlock found when trying to get lock; try restarting transaction
```

Thinking out loud: 1. "Both *SELECT*s at Repeatable Read take gap locks on the range (5000, 5010) on the PK index." 2. "Gap locks are shared with other gap locks but block inserts into the gap." 3. "T1's *INSERT* of 5005 needs an insert-intention lock that conflicts with T2's gap lock. T2's *INSERT* of 5007 conflicts with T1's gap lock. Cycle." 4. "Fix options: drop to *READ COMMITTED*, or use *SELECT* without *FOR UPDATE*, or partition the id space so the two transactions never share a gap."

The diagnosis: MySQL InnoDB at REPEATABLE READ takes next-key locks (record + gap) on indexes scanned by `SELECT ... FOR UPDATE`. Two transactions scanning the same range hold compatible gap locks but their subsequent *INSERT*s request insert-intention locks that conflict with the other's gap lock — instant cycle.

The fix: - **Option A — READ COMMITTED:** Switch the session or transaction to *READ COMMITTED*, which disables gap locks. You give up phantom prevention but eliminate this entire class of deadlock. - **Option B — Avoid the scan:** Replace `SELECT ... FOR UPDATE` over a range with a lookup on the exact id. Gap locks only appear on range scans. - **Option C — Auto-assigned ids:** Let the database assign ids via auto-increment so each transaction inserts into a different physical position; the conflicting gap never exists.

What NOT to say: - Do not say "REPEATABLE READ is always safer than *READ COMMITTED*" — in MySQL it adds gap locks, which produce deadlocks that *READ COMMITTED* simply does not have.

Common follow-up: "Does Postgres have gap locks?"

Scenario346 · Both Transactions Wait on the Same Row

Tags: Difficulty: Easy · Asked at: Infosys, TCS, Wipro, Wells Fargo · Dialect: vendor-agnostic

The scenario:

T1 updates a row, doesn't commit. T2 updates the same row, waits. T3 arrives and also wants the row, waits behind T2. Eventually T1's session times out or the application crashes — and both T2 and T3 are still waiting. No deadlock, just a stalled queue.

T1:

```
BEGIN;
UPDATE invoices SET status = 'paid' WHERE id = 9001;
-- application hangs, never commits
```

T2:

```
BEGIN;
UPDATE invoices SET status = 'cancelled' WHERE id = 9001;
-- waiting for T1
```

What happens: T2 and T3 block indefinitely. The engine does not flag a deadlock because there is no cycle — just a chain of waiters.

Thinking out loud: 1. "There is no cycle in the wait-for graph. The detector finds nothing. This is starvation, not deadlock." 2. "The problem is T1's open transaction holding the lock. Engine has no way to know T1 will never commit." 3. "Fix is at the connection layer: set `lock_timeout` or `statement_timeout` so waiters give up after, say, three seconds." 4. "Also set `idle_in_transaction_session_timeout` on Postgres so a hung T1 gets reaped automatically."

The diagnosis: A long-held lock is not a deadlock — it is a queue. The engine cannot help unless you tell it how long you are willing to wait or how long a session may sit idle inside a transaction.

The fix: - **Option A — Lock timeout:** Set `SET lock_timeout = '3s'` per transaction so the waiter aborts rather than blocking forever. - **Option B — Idle-in-transaction timeout:** Postgres has `idle_in_transaction_session_timeout`. Set it to ten seconds globally so a leaked connection is killed. - **Option C — Application-side circuit breaker:** Wrap the UPDATE in a timer; if it doesn't return in N seconds, cancel and surface the error to the user.

What NOT to say: - Do not call this a deadlock — it is a lock-wait timeout scenario, and interviewers test whether you can distinguish them.

Common follow-up: "What's the default lock-wait timeout in MySQL? In Postgres?"

Scenario347 · SELECT FOR UPDATE Order Mismatch

Tags: Difficulty: Medium · Asked at: Razorpay, Stripe, PhonePe, Goldman · Dialect: vendor-agnostic

The scenario:

A booking service locks the seat row and the user row before issuing a ticket. T1 locks seat 42 then user 7001. T2, processing a different booking for the same user, locks user 7001 first then seat 42. Both block, engine fires the deadlock detector.

T1:

```
BEGIN;
SELECT * FROM seats WHERE id = 42 FOR UPDATE;
SELECT * FROM users WHERE id = 7001 FOR UPDATE;
INSERT INTO bookings (seat_id, user_id) VALUES (42, 7001);
COMMIT;
```

T2:

```
BEGIN;
SELECT * FROM users WHERE id = 7001 FOR UPDATE;
SELECT * FROM seats WHERE id = 42 FOR UPDATE;
INSERT INTO bookings (seat_id, user_id) VALUES (42, 7001);
COMMIT;
```

What happens: Deadlock; one transaction is rolled back with the appropriate SQLSTATE.

Thinking out loud: 1. "FOR UPDATE takes explicit row-level exclusive locks. Same wait-for cycle as any UPDATE-only deadlock." 2. "The bug is that the two code paths were written by different teams who chose different lock orders." 3. "The fix is policy: every code path that locks multiple rows must do so in a single documented order." 4. "Enforce with a helper function `lock_for_booking(seat_id, user_id)` that internally always locks seats before users."

The diagnosis: Explicit row locks via `FOR UPDATE` deadlock exactly like UPDATE-induced locks. The acquisition order is set by the application, so the fix lives in the application.

The fix: - **Option A — Wrapper function:** Centralise the lock acquisition in a single helper. No caller chooses the order; the helper does, consistently. - **Option B — Sort by table name:** Adopt the rule "always lock tables alphabetically by table name". Mechanical and easy to audit. - **Option C — NOWAIT:** Use `SELECT ... FOR UPDATE NOWAIT` so the second waiter fails immediately and the application can either retry in the canonical order or surface an error.

What NOT to say: - Do not say "FOR UPDATE is safer than UPDATE because the lock is explicit" — the lock is the same row-level exclusive lock; the only difference is intent.

Common follow-up: "What does FOR UPDATE SKIP LOCKED do, and when would you use it for a queue worker?"

Scenario348 · Auto-Increment Lock Contention in MySQL

Tags: Difficulty: Medium · Asked at: Flipkart, Myntra, Walmart Labs, Goldman · Dialect: MySQL InnoDB

The scenario:

A high-throughput logging table uses `id BIGINT AUTO_INCREMENT`. Under heavy concurrent INSERTs, throughput plateaus and you see waits on the table-level AUTO-INC lock. Occasionally a multi-row INSERT deadlocks with a single-row INSERT.

T1 (multi-row insert):

```
BEGIN;
INSERT INTO event_log (event_type, payload) VALUES
  ('view', '...'), ('click', '...'), ('scroll', '...');
COMMIT;
```

T2 (single-row insert):

```
BEGIN;
INSERT INTO event_log (event_type, payload) VALUES ('purchase', '...');
COMMIT;
```

What happens: With `innodb_autoinc_lock_mode = 1` (the traditional default), the multi-row INSERT holds a special AUTO-INC table lock until it has reserved its block of ids. Other inserters wait. Under specific orderings, this interacts with row locks elsewhere to form a deadlock.

Thinking out loud: 1. "AUTO-INC lock is a special table-level lock, not a row lock." 2. "Mode 0 (traditional) holds it for the whole INSERT — full serialization. Mode 1 (consecutive) holds it only until the count is known. Mode 2 (interleaved) doesn't hold a lock but breaks statement-based replication." 3. "For pure throughput on row-based replication, switch to mode 2." 4. "To avoid contention entirely, generate ids in the application — UUID v7 or a Snowflake-style id."

The diagnosis: The MySQL AUTO-INC lock is a coarse-grained lock that limits concurrent INSERT throughput. The mode controls how aggressively the lock is held; the deadlock arises only when the AUTO-INC waiter is also waiting for a row lock held by a third transaction.

The fix: - **Option A — Mode 2 (interleaved):** Set `innodb_autoinc_lock_mode = 2`. Inserts no longer wait on the AUTO-INC lock. Requires row-based binlog replication. - **Option B — Application-generated ids:** Switch to UUIDs (preferably UUID v7 for index locality) or a Twitter-Snowflake style id. The AUTO-INC lock disappears. - **Option C — Batch the inserts:** One multi-row INSERT replaces many single-row ones. Fewer lock acquisitions per row inserted.

What NOT to say: - Do not say "MySQL doesn't have a table-level lock for INSERT" — the AUTO-INC lock is exactly that, and interviewers love when candidates know its name.

Common follow-up: "Why does UUID v4 hurt InnoDB performance, and what does UUID v7 fix?"

Scenario349 · Bulk UPDATE vs Many Single-Row UPDATES

Tags: Difficulty: Medium · Asked at: Swiggy, Zomato, Target, AmEx · Dialect: vendor-agnostic

The scenario:

A nightly job runs `UPDATE orders SET status = 'archived' WHERE created_at < NOW() - INTERVAL '90 days'`. During the same window, a user-facing transaction updates a single order in that range. The bulk job holds a wide range of row locks; the single-row update deadlocks with it.

T1 (bulk archive):

```
BEGIN;
UPDATE orders SET status = 'archived'
WHERE created_at < NOW() - INTERVAL '90 days';
COMMIT;
```

T2 (user-facing):

```
BEGIN;
UPDATE orders SET shipping_status = 'returned' WHERE id = 8042;
UPDATE customers SET return_count = return_count + 1 WHERE id = 7001;
COMMIT;
```

What happens: T1 holds a row lock on order 8042 (among thousands of others). T2 waits on order 8042 but also already holds the customer lock. T1 then needs the customer for an FK validation chain — deadlock.

Thinking out loud: 1. "Bulk updates take a row lock per matched row. With thousands of rows, the lock footprint is huge." 2. "The cycle forms when the bulk job needs any other resource — even an FK shared lock — that the small transaction already holds." 3. "Fix: chunk the bulk update into batches of, say, one thousand rows with a sleep or commit between chunks." 4. "Better: run the bulk job in a maintenance window with a `LOCK TABLE ... IN ROW EXCLUSIVE MODE` that locks out the OLTP path cleanly."

The diagnosis: Bulk DML holds locks for the duration of the statement. With concurrent OLTP traffic, the bulk job becomes a deadlock magnet because every other transaction is one row away from sharing a resource with it.

The fix: - **Option A — Chunked bulk update:** Loop in batches with explicit commits between, so each batch holds only N row locks for a short time. - **Option B — Maintenance window:** Schedule the bulk job for low-traffic hours and accept the trade-off. - **Option C — Move to a separate archive table:** Use partition switching (Postgres declarative partitioning or SQL Server partition swap) — flips an entire partition into the archive table without row locks.

What NOT to say: - Do not say "use `WITH (NOLOCK)`" — that is SQL Server dirty-read hint, doesn't apply to UPDATES, and is the wrong answer to a write conflict anyway.

Common follow-up: *"How would you chunk a multi-million row archive job without scanning the same table over and over?"*

Scenario350 · Lock Escalation Cascade in SQL Server

Tags: Difficulty: Hard · Asked at: AmEx, JP Morgan, Goldman, Walmart Labs · Dialect: SQL Server

The scenario:

A transaction holds row locks on five thousand rows. SQL Server's lock-escalation threshold (around five thousand locks on a single table) trips, and the row locks are escalated to a table-level intent-exclusive lock. Another transaction holding a small number of row locks elsewhere on the same table now blocks; if its locks are needed by the escalating transaction, you get a deadlock.

T1 (large update):

```
BEGIN TRAN;
UPDATE orders SET region = 'south'
WHERE customer_state IN ('TN', 'KL', 'KA', 'AP', 'TS');
-- ~5000 rows; lock escalation to table lock triggers
```

T2 (small concurrent update):

```
BEGIN TRAN;
UPDATE orders SET status = 'cancelled' WHERE id = 99001;
-- already holds row lock on 99001
COMMIT TRAN;
```

What happens: T1 attempts to escalate to a table-level X lock. T2 holds a row lock that conflicts with the escalation. SQL Server fails the escalation, retries; if T2 then needs another row T1 has just locked, you get a deadlock.

Thinking out loud: 1. "SQL Server escalates to table or partition lock when row-lock count crosses the threshold." 2. "Escalation is opportunistic — it tries, fails, retries — but combined with a concurrent transaction it can cycle." 3. "Disable escalation per table with `ALTER TABLE ... SET (LOCK_ESCALATION = DISABLE)` if memory allows." 4. "Or partition the table so escalation only affects one partition, not the whole table."

The diagnosis: Lock escalation reduces lock-manager memory pressure by exchanging many fine-grained locks for one coarse lock. Under concurrent load this can cause deadlocks because the coarse lock conflicts with row locks held by other sessions.

The fix: - **Option A — Disable escalation on hot tables:** `ALTER TABLE orders SET (LOCK_ESCALATION = DISABLE)` . Trades memory for fewer deadlocks. - **Option B — Use partition-level escalation:** `LOCK_ESCALATION = AUTO` with a partitioned table; escalation stops at the partition. - **Option C — Chunk the large update:** Keep the row-lock count under the threshold; no escalation ever fires.

What NOT to say: - Do not say "use `WITH (ROWLOCK)` " hint — it is a hint, not a contract, and the engine will still escalate under memory pressure.

Common follow-up: "What is the default lock-escalation threshold in SQL Server, and how does it interact with `trace flag 1224` ?"

Scenario351 · Hot Row in an Invoicing Counter

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, PhonePe, Stripe · Dialect: vendor-agnostic

The scenario:

Every invoice needs a sequential invoice number. The application reads the `next_number` from a `counters` table, increments it, writes it back, and uses it. Under concurrent load, every transaction serializes on that one row; throughput collapses to single-threaded.

T1:

```
BEGIN;
SELECT next_number FROM counters WHERE name = 'invoice' FOR UPDATE;
-- application reads, increments to N+1
UPDATE counters SET next_number = next_number + 1 WHERE name = 'invoice';
INSERT INTO invoices (number, ...) VALUES (N, ...);
COMMIT;
```

T2 (identical):

```
BEGIN;
SELECT next_number FROM counters WHERE name = 'invoice' FOR UPDATE;
-- blocks until T1 commits
...
```

What happens: No deadlock — just serialization. Throughput is one transaction per round-trip. The counter row is the bottleneck.

Thinking out loud: 1. "This is contention, not a deadlock. There is no cycle, just a queue on one row." 2. "Fix is to either remove the SELECT (use atomic UPDATE ... RETURNING) or shard the counter across many rows." 3. "Atomic UPDATE shortens the lock window from two round-trips to one." 4. "Sharded counter scales horizontally — N counter rows give roughly N times the throughput."

The diagnosis: Single-row counters are a known anti-pattern under load. Every transaction serialises on the row's exclusive lock. The fix is to either shrink the critical section or spread the contention.

The fix: - **Option A — Atomic UPDATE ... RETURNING:** `UPDATE counters SET next_number = next_number + 1 WHERE name = 'invoice' RETURNING next_number`. One statement; lock held only for the UPDATE's duration. - **Option B — Sharded counter:** Keep N rows for the same counter; each transaction picks one at random and increments it. Reassemble with a SUM at read time. Trade-off: ids are no longer strictly sequential. - **Option C — Sequence object:** Postgres has SEQUENCE, SQL Server has SEQUENCE, MySQL 8 has AUTO_INCREMENT or a generator function. These are designed exactly for this.

What NOT to say: - Do not say "use a UUID instead" — invoice numbers usually have a regulatory or human-readable requirement; UUIDs do not satisfy it.

Common follow-up: "If invoice numbers must be strictly sequential with no gaps, can you still use a sequence?"

Scenario352 · Sharded Counter Pattern as the Fix

Tags: Difficulty: Medium · Asked at: Cred, Flipkart, Walmart Labs, Google · Dialect: vendor-agnostic

The scenario:

Following Scenario 11, you migrate the invoice counter to a sharded design with sixteen counter rows. Throughput improves twenty-fold. But now a reporting query that needs the current total scans all sixteen rows and occasionally sees an inconsistent total.

Schema:

```
CREATE TABLE counters_sharded (  
  name TEXT,  
  shard SMALLINT,  
  value BIGINT,  
  PRIMARY KEY (name, shard)  
);
```

Writer:

```
UPDATE counters_sharded  
SET value = value + 1  
WHERE name = 'invoice' AND shard = MOD(ABS(HASHTEXT(SESSION_ID())), 16);
```

Reader:

```
SELECT SUM(value) FROM counters_sharded WHERE name = 'invoice';
```

What happens: The SUM is correct on consistent snapshots (Postgres MVCC) but may double-count or miss under READ COMMITTED if writes happen between the per-row reads in a non-snapshot engine.

Thinking out loud: 1. "Sharding kills the deadlock and the queue but changes the read pattern." 2.

"Reader needs a consistent snapshot across all shards — Postgres REPEATABLE READ or SERIALIZABLE gives it." 3. "For MySQL InnoDB, REPEATABLE READ also gives a consistent snapshot for the read." 4. "For an approximate counter, READ COMMITTED is fine and faster."

The diagnosis: Sharded counters trade write contention for read complexity. The reader needs snapshot isolation to see a consistent sum.

The fix: - **Option A — Snapshot read:** Run the SUM in a REPEATABLE READ or SERIALIZABLE transaction. - **Option B — Approximate counter:** Accept that the SUM is a recent-but-not-instant value. Cache it for thirty seconds. - **Option C — Two-table pattern:** Increment a sharded counter for speed; periodically materialise the SUM into a `counter_totals` row for cheap reads.

What NOT to say: - Do not say "use a single counter for reads" — that reintroduces the bottleneck you just engineered around.

Common follow-up: "How would you decide between 16 shards and 64 shards?"

Scenario353 · Atomic UPDATE col = col + 1 vs SELECT-then-UPDATE

Tags: Difficulty: Easy · Asked at: Razorpay, PhonePe, Wells Fargo, Walmart · Dialect: vendor-agnostic

The scenario:

A like counter on a social post is incremented as `SELECT likes FROM posts WHERE id = X; ... UPDATE posts SET likes = N + 1 WHERE id = X`. Under concurrent likes, the final count is wrong — N appears when two users liked at the same time, not 2N.

Bad pattern:

```
SELECT likes FROM posts WHERE id = 9001;
-- application reads likes = 10
UPDATE posts SET likes = 11 WHERE id = 9001;
```

Correct pattern:

```
UPDATE posts SET likes = likes + 1 WHERE id = 9001 RETURNING likes;
```

What happens: With the read-modify-write pattern at READ COMMITTED, two concurrent transactions both read 10, both write 11, and one increment is lost.

Thinking out loud: 1. "This is the lost-update anomaly in its simplest form." 2. "Atomic UPDATE removes the race because the read-and-write happen inside one row lock." 3. "Alternative is optimistic locking — add a version column, only update if version matches." 4. "Or upgrade isolation to REPEATABLE READ, which detects the lost update and aborts the second transaction."

The diagnosis: Read-modify-write across a transaction boundary leaks the intermediate value. The engine has no idea your UPDATE depends on what you SELECTed. Atomic UPDATE compresses the operation into a single locked statement.

The fix: - **Option A — Atomic UPDATE:** `UPDATE posts SET likes = likes + 1 WHERE id = X`. Always correct under any isolation level. - **Option B — Optimistic version:** `UPDATE posts SET likes = N+1, version = version+1 WHERE id = X AND version = V`. Retry on zero rows affected. - **Option C — SELECT FOR UPDATE:** Lock the row at read time so the second transaction waits. Higher latency, simpler logic.

What NOT to say: - Do not say "this is fine because the write is fast" — speed has nothing to do with it; the race exists at any latency.

Common follow-up: *"What if you need to increment two columns and the second one depends on the first?"*

Scenario354 · Distributed Deadlock Across Two Databases

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan, Walmart Labs, Stripe · Dialect: vendor-agnostic + distributed

The scenario:

A booking service writes to a `bookings` database in Mumbai. A payments service writes to a `payments` database in Singapore. T1 holds a booking row in Mumbai and waits for a payment row in Singapore. T2 holds the payment row in Singapore and waits for the booking row in Mumbai. Neither database can see the cycle.

T1 (booking service):

```
-- Mumbai DB
BEGIN;
UPDATE bookings SET status = 'pending_payment' WHERE id = 5001;
-- now calls payments service which opens transaction in Singapore
```

T2 (payments service):

```
-- Singapore DB
BEGIN;
UPDATE payments SET status = 'reconciling' WHERE booking_id = 5001;
-- now calls booking service which tries to open transaction in Mumbai
```

What happens: Each database holds its own lock and waits on the other. Neither has a global wait-for graph, so neither can detect the cycle. Both transactions hang until the application timeout fires.

Thinking out loud: 1. "No single database has the global view, so neither detector triggers." 2. "Two-phase commit with a transaction manager could detect it, but XA is operationally painful and rare in modern Indian stacks." 3. "The pragmatic fix is the saga pattern — break the work into local transactions with compensating actions." 4. "Alternative is to centralise all writes for one entity in one database — single source of truth."

The diagnosis: Distributed deadlock has no automatic detector outside of a coordinated transaction manager. You either prevent it via design (saga, single source of truth) or detect it via timeouts at the application layer.

The fix: - **Option A — Saga pattern:** Local transactions chained with explicit compensations. No distributed lock state to deadlock. - **Option B — XA / two-phase commit:** A transaction coordinator manages locks across both DBs and can detect cycles. Heavyweight and slow. - **Option C — Strict ordering:** Always lock the booking before the payment, in every code path. Prevents the cycle by convention.

What NOT to say: - Do not say "Postgres can detect distributed deadlocks" — it cannot; it only sees its own wait-for graph.

Common follow-up: *"Draw the saga for a booking-then-payment workflow including the compensation transactions."*

Scenario355 · Long-Running Transaction Holding Locks

Tags: Difficulty: Easy · Asked at: TCS, Wipro, Wells Fargo, JP Morgan · Dialect: Postgres + MySQL

The scenario:

A reporting query runs inside a transaction with `SELECT ... FOR UPDATE` for "consistency". It takes thirty minutes. For thirty minutes, every OLTP write blocks behind it. The team blames the database; the database is fine.

T1:

```
BEGIN;
SELECT * FROM orders WHERE created_at > NOW() - INTERVAL '7 days' FOR UPDATE;
-- 30-minute report generation
COMMIT;
```

T2:

```
UPDATE orders SET status = 'shipped' WHERE id = 9001;
-- waits 30 minutes
```

What happens: No deadlock — just one transaction starving every other writer that touches the locked rows.

Thinking out loud: 1. "FOR UPDATE on a reporting query is almost always a bug. Reports should be read-only." 2. "Remove FOR UPDATE entirely and use REPEATABLE READ or SERIALIZABLE for snapshot consistency." 3. "Or move the report to a read replica." 4. "If you really must lock, lock only the rows you intend to modify."

The diagnosis: Long-running transactions hold locks for their entire duration. The bug is using `FOR UPDATE` on read-only work.

The fix: - **Option A — Drop FOR UPDATE:** Use plain SELECT inside REPEATABLE READ for snapshot consistency without writer-blocking locks. - **Option B — Read replica:** Send reports to a replica; OLTP traffic is unaffected. - **Option C — Statement timeout:** Set `statement_timeout = '5min'` so a runaway report dies before causing damage.

What NOT to say: - Do not say "FOR UPDATE prevents the data from changing during the report" — it does, but at the cost of blocking writes; that is rarely the trade-off you want.

Common follow-up: *"How would you build a 'consistent point-in-time snapshot' read pattern without FOR UPDATE?"*

Scenario356 · Idle-in-Transaction Warning in Postgres

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Google · Dialect: Postgres

The scenario:

A background worker opens a transaction, does one INSERT, and then waits for a Kafka message before committing. The Kafka consumer hangs. Postgres now has a connection in `idle in transaction` state for hours, holding locks and bloating MVCC.

Symptom:

```
$ select state, query from pg_stat_activity where state like 'idle in%';
state          | query
idle in transaction | INSERT INTO outbox (event_type, payload) VALUES ($1, $2)
```

What happens: The transaction holds row locks and prevents vacuum on the table. Other transactions either block on the locks or, more insidiously, accumulate dead tuples that vacuum cannot reclaim. Performance degrades over hours.

Thinking out loud: 1. "Idle-in-transaction is the silent killer in Postgres. No alert by default; just slow degradation." 2. "Set `idle_in_transaction_session_timeout = '60s'` globally so a stuck connection is killed automatically." 3. "Don't hold a transaction across an external network call. Commit, send to Kafka, then start a new transaction if you need a follow-up write." 4. "Use the outbox pattern: write the event row inside the original transaction, let a separate worker pick it up and publish."

The diagnosis: Holding a database transaction across an external call is a structural bug. The fix is the outbox pattern or an immediate commit.

The fix: - **Option A** — `idle_in_transaction_session_timeout` : Postgres kills any session sitting idle in a transaction past the limit. Belt-and-braces protection. - **Option B** — **Outbox pattern**: Write the message to an `outbox` table inside the transaction, commit, then a separate worker reads outbox and publishes to Kafka. - **Option C** — **Commit before the network call**: If correctness allows, commit the DB write first, then do the network call. Accept that the network call may fail, requiring a retry.

What NOT to say: - Do not say "Postgres handles this automatically" — it does not, unless you configure `idle_in_transaction_session_timeout` .

Common follow-up: "What's the difference between `idle in transaction` and `idle in transaction (aborted)` ?"

Scenario357 · Retry with Exponential Backoff

Tags: Difficulty: Medium · Asked at: Razorpay, Stripe, PhonePe, Goldman · Dialect: vendor-agnostic

The scenario:

The application is hitting deadlocks under load. The fix is to retry the failed transaction. A naive `retry 3 times` makes the problem worse — all victims retry simultaneously and re-collide. Production needs exponential backoff with jitter.

Pattern (pseudocode):

```
for attempt in 1..max_attempts:
    try:
        BEGIN;
        ...work...
        COMMIT;
        break
    except DeadlockError:
        sleep(min(2^attempt * base_ms, cap_ms) + random_jitter_ms)
        continue
```

SQL hooks:

```
-- Postgres: SQLSTATE 40P01 (deadlock_detected) or 40001 (serialization_failure)
-- MySQL: error 1213 (deadlock) or 1205 (lock wait timeout)
-- SQL Server: error number 1205
```

Thinking out loud: 1. "The naive retry stampedes. Jitter spreads the retries across a time window so they don't re-collide." 2. "Exponential backoff caps the worst-case wait but starts cheap." 3. "Always cap the

attempt count — three to five — to avoid retrying through a real outage." 4. *"Distinguish deadlock from lock timeout; both are retrieable but other errors (constraint violation) are not."*

The diagnosis: Deadlock is a transient condition. The right response is to retry, but the retry policy needs jitter and caps to avoid making the problem worse.

The fix: - **Option A — Exponential backoff with jitter:** `delay = base * 2^attempt + uniform(0, jitter)` . Industry standard. - **Option B — Library-level retry:** Use a library (Spring Retry, Polly, Tenacity) so policy is consistent across the codebase. - **Option C — Decorrelated jitter (AWS-recommended):** `delay = uniform(base, prev_delay * 3)` . Spreads retries even better than full jitter.

What NOT to say: - Do not say "retry forever" — eventually the policy makes the deadlock worse and you mask real bugs.

Common follow-up: *"How do you make the transaction safe to retry — what's the idempotency story?"*

Scenario358 · Idempotency Before Retry

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Stripe, JP Morgan · Dialect: vendor-agnostic

The scenario:

The retry policy in Scenario 17 fires on a payment-creation transaction. Without idempotency, the retry may double-charge the customer because the first attempt may have committed before the deadlock error fired — though usually it didn't.

Schema:

```
CREATE TABLE payments (  
  id BIGSERIAL PRIMARY KEY,  
  request_id UUID NOT NULL UNIQUE,  
  amount NUMERIC NOT NULL,  
  ...  
);
```

Transaction:

```
BEGIN;  
INSERT INTO payments (request_id, amount) VALUES ($1, $2)  
ON CONFLICT (request_id) DO NOTHING  
RETURNING id;  
COMMIT;
```

Thinking out loud: 1. "The first deadlocked attempt aborts cleanly — but in distributed systems we cannot always be sure." 2. "Idempotency key on a unique constraint converts a retry into a no-op." 3. "Generate the request_id at the API boundary (or on the client), pass it through every retry." 4. " **ON CONFLICT DO NOTHING** plus RETURNING gives you 'created or already exists' semantics in one round-trip."

The diagnosis: Retry is only safe if the operation is idempotent. The simplest construction is a unique idempotency key plus an UPSERT.

The fix: - **Option A — Idempotency key with UNIQUE constraint:** As above. Engine enforces uniqueness. - **Option B — Application-side dedup table:**

INSERT INTO idempotency_keys (key) VALUES (\$1) first; on duplicate, return cached response. - **Option C — Natural unique key:** If the operation has a natural unique attribute (e.g., transaction reference number), use it instead of a synthetic key.

What NOT to say: - Do not say "we'll handle duplicates downstream" — by the time downstream sees the duplicate, money has moved.

Common follow-up: "How long do you keep idempotency keys around, and where do you store them — same DB or a cache?"

Scenario359 · Statement Retry vs Transaction Retry

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan, Walmart Labs, Razorpay · Dialect: vendor-agnostic

The scenario:

Your ORM has a built-in retry that re-runs the failed statement on deadlock. The transaction continues with subsequent statements. This is silently wrong: a deadlock aborts the entire transaction in every major engine, and the connection is now in an error state.

Bad pattern:

```
# pseudo-Python with broken statement-level retry
with conn.begin():
    conn.execute("UPDATE accounts SET balance = balance - 100 WHERE id = 1")
    for attempt in range(3):
        try:
            conn.execute("UPDATE accounts SET balance = balance + 100 WHERE id = 2")
            break
        except DeadlockError:
            continue # WRONG: transaction is already aborted
```

Correct pattern:

```
for attempt in range(3):
    try:
        with conn.begin():
            conn.execute("UPDATE accounts SET balance = balance - 100 WHERE id = 1")
            conn.execute("UPDATE accounts SET balance = balance + 100 WHERE id = 2")
        break
    except DeadlockError:
        sleep(backoff(attempt))
```

Thinking out loud: 1. "A deadlock aborts the whole transaction. The session must issue `ROLLBACK` and start a new `BEGIN`." 2. "Retry must wrap the entire transaction — every statement — not just the one that failed." 3. "Postgres explicitly errors out on subsequent statements with 'current transaction is aborted'." 4. "This is one of the most common ORM-related bugs in production Indian fintech codebases."

The diagnosis: The retry boundary must match the transaction boundary. Statement-level retry inside an aborted transaction is invalid; the engine rejects every subsequent statement until `ROLLBACK`.

The fix: - **Option A — Wrap the transaction:** Retry loop outside the `BEGIN ... COMMIT` block. -

Option B — Library-level transaction retry: Use a library that understands transaction boundaries (e.g., Spring's `@Retryable` on a `@Transactional` method). - **Option C — Stateful retry helper:** A custom helper that owns the connection, knows when it's safe to retry, and recreates the transaction context.

What NOT to say: - Do not say "Postgres will let you retry the failed statement" — it will not; the next statement returns `SQLSTATE 25P02`.

Common follow-up: "What is `SQLSTATE 25P02` and when do you see it?"

Scenario360 · Read-Modify-Write Race Even at Repeatable Read

Tags: Difficulty: Hard · Asked at: Cred, Stripe, Goldman, Google · Dialect: Postgres + MySQL

The scenario:

A transaction at `REPEATABLE READ` reads a row, computes a new value, writes it back. Surely the snapshot protects it? In MySQL InnoDB, yes — `REPEATABLE READ` takes locks that prevent lost updates. In Postgres, no — `REPEATABLE READ` uses snapshot isolation, which detects lost updates and aborts one transaction (the engine raises a serialization failure).

T1:

```
-- Postgres, REPEATABLE READ
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT balance FROM accounts WHERE id = 101; -- reads 5000
UPDATE accounts SET balance = 5500 WHERE id = 101;
COMMIT;
```

T2 (concurrent):

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT balance FROM accounts WHERE id = 101; -- reads 5000
UPDATE accounts SET balance = 5300 WHERE id = 101;
COMMIT;
```

What happens (Postgres):

```
ERROR:  could not serialize access due to concurrent update
HINT:   The transaction might succeed if retried.
```

Thinking out loud: 1. "Postgres REPEATABLE READ is snapshot isolation. It detects the lost update and aborts the second transaction." 2. "MySQL REPEATABLE READ is different — it uses next-key locks; the second SELECT would block until T1 commits." 3. "This is the most surprising cross-engine difference in isolation behavior." 4. "Fix is the standard read-modify-write fix: atomic UPDATE, optimistic version, or SELECT FOR UPDATE."

The diagnosis: REPEATABLE READ in Postgres protects you against the lost update by aborting the second transaction with a serialization failure. You still need to retry.

The fix: - **Option A — Retry with backoff:** Catch SQLSTATE 40001 and retry the entire transaction. - **Option B — Atomic UPDATE:** Replace the read-modify-write with `UPDATE ... SET col = expression`. No snapshot, no failure. - **Option C — Application-level optimistic:** Add a version column; UPDATE WHERE version = old_version. Retry on zero rows affected.

What NOT to say: - Do not assume "Postgres and MySQL REPEATABLE READ behave the same" — they do not.

Common follow-up: "What does SERIALIZABLE add on top of Postgres REPEATABLE READ, and when is the extra cost worth it?"

Scenario361 · INSERT ... ON CONFLICT Atomic Upsert

Tags: Difficulty: Easy · Asked at: Razorpay, Stripe, Swiggy, Walmart · Dialect: Postgres + MySQL

The scenario:

A device-registration endpoint inserts a row keyed on (user_id, device_id). If the row exists, update last_seen. The naive pattern of SELECT-then-INSERT-or-UPDATE races: two concurrent requests both SELECT, see nothing, both INSERT, one fails with a unique-key violation. Worse, in MySQL with auto-increment, you have already burned an id.

Bad pattern:

```
SELECT id FROM devices WHERE user_id = 7001 AND device_id = 'abc';
-- nothing returned
INSERT INTO devices (user_id, device_id, last_seen) VALUES (7001, 'abc', NOW());
-- ERROR: duplicate key
```

Correct pattern (Postgres):

```
INSERT INTO devices (user_id, device_id, last_seen)
VALUES (7001, 'abc', NOW())
ON CONFLICT (user_id, device_id)
DO UPDATE SET last_seen = EXCLUDED.last_seen;
```

Correct pattern (MySQL):

```
INSERT INTO devices (user_id, device_id, last_seen)
VALUES (7001, 'abc', NOW())
ON DUPLICATE KEY UPDATE last_seen = VALUES(last_seen);
```

Thinking out loud: 1. "Upsert is the right primitive for 'register or refresh'." 2. "Both engines have atomic upserts; use them instead of building one out of SELECT + INSERT/UPDATE." 3. "In MySQL, ON DUPLICATE KEY can burn auto-increment ids; partial mitigation is `INSERT IGNORE` for the no-update case." 4. "In Postgres, ON CONFLICT requires a unique or exclusion constraint to target."

The diagnosis: SELECT-then-act is a race waiting to happen. Atomic upsert compresses the decision into a single statement under a single lock.

The fix: - **Option A — Native upsert:** `INSERT ... ON CONFLICT` (Postgres) / `INSERT ... ON DUPLICATE KEY UPDATE` (MySQL). - **Option B — INSERT then UPDATE on conflict:** Try INSERT, catch duplicate-key, run UPDATE. Works but doubles round-trips on conflict. - **Option C — MERGE statement:** SQL Server and Postgres 15+ have MERGE — see Scenario 22 for its concurrency caveats.

What NOT to say: - Do not say "wrap SELECT-then-INSERT in a transaction and it's safe" — it is not, unless you also SELECT FOR UPDATE the key, and even then you may need to handle phantoms.

Common follow-up: "What's `EXCLUDED.column` in Postgres `ON CONFLICT`, and how does it differ from `OLD.column` in MySQL?"

Scenario362 · MERGE INTO Concurrency Bug in SQL Server

Tags: Difficulty: Hard · Asked at: Walmart Labs, AmEx, JP Morgan, Goldman · Dialect: SQL Server (and Postgres 15+)

The scenario:

A `MERGE INTO` statement is used as an upsert. Under concurrent load, you start seeing primary-key violations on the `INSERT` branch even though you "checked" via `MERGE`. SQL Server's `MERGE` has a well-known concurrency hole: between the `WHEN MATCHED` check and the `INSERT`, another transaction can insert the row.

Vulnerable pattern:

```
MERGE devices AS target
USING (VALUES (7001, 'abc')) AS src(user_id, device_id)
ON target.user_id = src.user_id AND target.device_id = src.device_id
WHEN MATCHED THEN UPDATE SET last_seen = SYSUTCDATETIME()
WHEN NOT MATCHED THEN INSERT (user_id, device_id, last_seen)
    VALUES (src.user_id, src.device_id, SYSUTCDATETIME());
```

Hardened pattern:

```
MERGE devices WITH (HOLDLOCK) AS target
USING (VALUES (7001, 'abc')) AS src(user_id, device_id)
ON target.user_id = src.user_id AND target.device_id = src.device_id
WHEN MATCHED THEN UPDATE SET last_seen = SYSUTCDATETIME()
WHEN NOT MATCHED THEN INSERT (user_id, device_id, last_seen)
    VALUES (src.user_id, src.device_id, SYSUTCDATETIME());
```

Thinking out loud: 1. "`MERGE` without `HOLDLOCK` takes shared locks that other transactions can also take. Two `MERGE`s can both decide 'NOT MATCHED' and both try to `INSERT`." 2. "`HOLDLOCK` upgrades the lock to a key-range lock that prevents phantom inserts." 3. "Postgres 15 added `MERGE`; check its concurrency semantics carefully — `INSERT ... ON CONFLICT` is still simpler." 4. "For high-concurrency upserts in SQL Server, many teams prefer `IF EXISTS`-style explicit branches with `HOLDLOCK` over `MERGE`."

The diagnosis: `MERGE` without explicit lock hints in SQL Server is not atomic across concurrent sessions. The fix is `HOLDLOCK` or switching to `INSERT-then-UPDATE`.

The fix: - **Option A** — **WITH (HOLDLOCK)** : Forces a key-range lock that prevents phantom inserts. - **Option B** — **Use INSERT-then-UPDATE pattern**: Try INSERT inside TRY/CATCH; on duplicate-key, UPDATE. - **Option C** — **Avoid MERGE for upserts**: Many SQL Server experts (Aaron Bertrand among them) recommend never using MERGE for upserts due to its bug history.

What NOT to say: - Do not say "MERGE is atomic by definition" — the statement is, but its semantics regarding concurrent inserts are surprising.

Common follow-up: *"Why has MERGE been called 'Microsoft's gift to people who hate their DBAs'?"*

Scenario363 · Range Lock from REPEATABLE READ Scan

Tags: Difficulty: Hard · Asked at: PhonePe, Cred, JP Morgan, Goldman · Dialect: MySQL InnoDB

The scenario:

A reporting transaction at REPEATABLE READ scans `SELECT SUM(amount) FROM payments WHERE created_at >= '2026-05-01'`. InnoDB takes next-key locks on the index, blocking concurrent INSERTs into that date range. A morning standup explains why "the database is slow" — it isn't; it's locked.

T1 (report):

```
BEGIN;  
SELECT SUM(amount) FROM payments WHERE created_at >= '2026-05-01' FOR UPDATE;  
-- runs for 90 seconds
```

T2 (writer):

```
INSERT INTO payments (amount, created_at) VALUES (499, '2026-05-26 11:00:00');  
-- blocks for 90 seconds
```

Thinking out loud: 1. "Even without FOR UPDATE, REPEATABLE READ in MySQL takes next-key locks if the engine thinks it needs to prevent phantoms." 2. "With FOR UPDATE the locks are explicit and definitely block writers." 3. "Fix: drop FOR UPDATE; drop to READ COMMITTED for the report; or send the report to a read replica." 4. "Or use a covering index that lets the engine use a non-locking snapshot read path."

The diagnosis: REPEATABLE READ in MySQL InnoDB uses next-key locks on range scans, which block inserts into the range. The combination of REPEATABLE READ + FOR UPDATE + long-running query is a writer-starvation pattern.

The fix: - **Option A — READ COMMITTED:** Switch session to READ COMMITTED; uses single-row locks, no gap locks. - **Option B — Drop FOR UPDATE:** Report queries are read-only; FOR UPDATE is a bug. - **Option C — Read replica:** Reports run on the replica with zero impact on the OLTP primary.

What NOT to say: - Do not say "the engine should release locks faster" — locks are held until COMMIT by design.

Common follow-up: *"Postgres MVCC doesn't take read locks at REPEATABLE READ — why does MySQL?"*

Scenario364 · Application Lock vs Database Lock — Trade-offs

Tags: Difficulty: Medium · Asked at: Razorpay, Stripe, Cred, Goldman · Dialect: vendor-agnostic

The scenario:

A leader-election routine for a background job needs to ensure only one instance runs at a time. Option A is `SELECT pg_advisory_lock(42)`. Option B is a Redis SET NX EX. Option C is a database row with an exclusive lock and a heartbeat. Which one and why?

Postgres advisory lock:

```
SELECT pg_advisory_lock(42);
-- holder runs the job
SELECT pg_advisory_unlock(42);
```

Redis lock:

```
SET job_lock $instance_id NX EX 30
-- holder renews every 10s
DEL job_lock when done
```

Database row lock:

```
UPDATE job_leadership
SET leader = 'instance-A', last_heartbeat = NOW()
WHERE job = 'archive'
  AND (last_heartbeat < NOW() - INTERVAL '30 sec' OR leader = 'instance-A')
RETURNING leader;
```

Thinking out loud: 1. "Advisory locks are cheap, but tied to the connection — if the connection dies, the lock releases (good for liveness, bad for orphan-detection)." 2. "Redis is fast but introduces an extra dependency and is famously hard to make correct under partition (see Martin Kleppmann on Redlock)." 3. "Database row with heartbeat is most portable; you control the timeout." 4. "For multi-process leader election within one DB, advisory locks are the sweet spot."

The diagnosis: "Lock" is overloaded. Pick the right kind for the use case: in-process for in-memory state, advisory for inter-process coordination, row-level for data, distributed for cross-service.

The fix (option = recommendation per use case): - **Advisory lock (Postgres `pg_advisory_lock` , MySQL `GET_LOCK`)**: Best for leader election within one database, low overhead. - **Row lock with heartbeat**: Best when you need explicit ownership tracking, take-over after crash, audit log. - **Redis (or Etcd, ZooKeeper)**: Best when coordination spans services that don't share a database.

What NOT to say: - Do not say "Redlock is the standard for distributed locks" — its correctness is hotly debated, and many teams have moved to Etcd or ZooKeeper for hard locks.

Common follow-up: "Walk through one scenario where Redlock loses the lock guarantee."

Scenario365 · Deadlock Timeout vs Lock Timeout

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan, Wells Fargo, AmEx · Dialect: Postgres + MySQL + SQL Server

The scenario:

Engineering wants to "lower the deadlock timeout to fix slow transactions". But deadlock timeout and lock-wait timeout are different settings with different effects. Confusing them is a common interview trap.

Postgres: - `deadlock_timeout` (default 1s): how long to wait before running the deadlock detector. Lower means faster detection, higher CPU. - `lock_timeout` (default 0 = wait forever): how long a statement waits for a lock before aborting with an error.

MySQL InnoDB: - `innodb_deadlock_detect` (ON/OFF): whether to run the detector at all. - `innodb_lock_wait_timeout` (default 50s): how long to wait for a lock.

SQL Server: - Deadlocks detected by a continuously running monitor; no timeout setting. - `SET LOCK_TIMEOUT 5000` (ms): per-session lock wait timeout.

Thinking out loud: 1. "Deadlock timeout in Postgres is detector frequency, not a wait-for-deadlock timeout." 2. "Lock timeout is the right knob for 'don't wait forever'." 3. "For an OLTP app, lock_timeout of two to five seconds is sane." 4. "For batch jobs, longer is fine, but document the choice."

The diagnosis: The two timeouts solve different problems. Use `lock_timeout` to bound waiter latency; leave `deadlock_timeout` alone unless you have specific evidence.

The fix (no single fix — choice depends on goal): - **For latency-sensitive OLTP:** `SET lock_timeout = '3s'` per transaction. - **For Postgres deadlock detection latency:** Lower `deadlock_timeout` to 500ms only if your workload is highly contended. - **For MySQL:** Tune `innodb_lock_wait_timeout` down from 50s — typical default is too long.

What NOT to say: - Do not say "lowering `deadlock_timeout` fixes deadlocks faster" in Postgres — it makes the detector run more often, but the deadlock itself is not resolved any sooner than the next detection cycle.

Common follow-up: *"Postgres runs the deadlock detector lazily. What's the worst-case latency from cycle formation to deadlock-error?"*

Scenario366 · Deferred Constraints and Commit-Time Failure

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman, Stripe · Dialect: Postgres (primarily)

The scenario:

A transaction inserts rows in an order that temporarily violates an FK, expecting to fix it later in the same transaction. With normal constraints, the first insert fails immediately. With DEFERRABLE constraints (Postgres), the check is held until COMMIT — and if it fails then, you get an error at commit, after all your work.

Schema:

```
CREATE TABLE orders (  
  id BIGSERIAL PRIMARY KEY,  
  customer_id BIGINT REFERENCES customers(id) DEFERRABLE INITIALLY DEFERRED  
);
```

T1:

```
BEGIN;  
INSERT INTO orders (customer_id) VALUES (7001); -- customer doesn't exist YET  
-- ...other work...  
INSERT INTO customers (id) VALUES (7001); -- creates the parent  
COMMIT; -- FK is checked here; succeeds because parent now exists
```

What happens: If the parent insert is forgotten, the COMMIT fails:

```
ERROR: insert or update on table "orders" violates foreign key constraint
DETAIL: Key (customer_id)=(7001) is not present in table "customers".
```

Thinking out loud: 1. "Deferrable constraints are useful for bulk loads and circular FK structures." 2. "The cost is that errors surface at COMMIT, not at the offending statement — debugging is harder." 3. "Use DEFERRABLE INITIALLY IMMEDIATE to allow per-transaction `SET CONSTRAINTS ALL DEFERRED` when needed." 4. "Don't make every FK deferrable by default — production debugging suffers."

The diagnosis: Deferred constraints trade earlier failure for circular-reference flexibility. Be deliberate.

The fix: - **Option A — Use DEFERRABLE selectively:** Only on tables where circular FKs or bulk loads require it. - **Option B — DEFERRABLE INITIALLY IMMEDIATE:** Default to immediate check; opt into deferred behavior per transaction with `SET CONSTRAINTS`. - **Option C — Restructure the schema:** A truly circular FK is often a smell. Consider whether the relationship is really bidirectional.

What NOT to say: - Do not say "deferred constraints are a free win" — they make debugging harder when errors fire at COMMIT.

Common follow-up: "MySQL doesn't support DEFERRABLE constraints. How do you load circular FK data in MySQL?"

Scenario367 · Saga Pattern When Distributed Transactions Are Infeasible

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Goldman, Stripe · Dialect: distributed

The scenario:

A booking flow spans three services: bookings, payments, notifications. Each owns its own database. A two-phase commit across all three is operationally painful. The team adopts the saga pattern: each step is a local transaction with a compensating action defined for failure.

Forward saga: 1. Create booking (T1: INSERT into bookings.bookings) 2. Charge payment (T2: INSERT into payments.charges) 3. Send confirmation (T3: INSERT into notifications.outbox)

Compensations (reverse order on failure): 1. If T3 fails: refund payment, cancel booking 2. If T2 fails: cancel booking

Sketch:

```
-- T1 (bookings DB)
BEGIN;
INSERT INTO bookings (id, status, idempotency_key) VALUES (...);
INSERT INTO outbox (event_type, payload) VALUES ('BookingCreated', ...);
COMMIT;
-- worker picks up outbox, calls payments service
```

Thinking out loud: 1. "Sagas convert distributed locking into local transactions plus compensation logic." 2. "Each step is its own local ACID transaction; the engine handles deadlock recovery within each." 3. "Compensations must be idempotent because retries are inevitable." 4. "Outbox pattern publishes the event reliably without holding cross-service locks."

The diagnosis: Distributed transactions cost more than they pay. Saga + outbox is the pragmatic Indian-fintech standard for cross-service consistency.

The fix: - **Option A — Orchestration saga:** Central coordinator (a state machine) drives each step. Easier to reason about, single point of state. - **Option B — Choreography saga:** Each service publishes events; others react. More decoupled, harder to debug. - **Option C — Outbox pattern as a building block:** Pair with either orchestration or choreography for reliable event delivery.

What NOT to say: - Do not say "use 2PC, it's safer" — XA in production is rare in Indian-stack microservices for good reason; performance and operational complexity kill it.

Common follow-up: "What's the difference between orchestration and choreography sagas, and which one would you pick for a payment flow?"

Scenario368 · Two-Phase Commit Reality Check

Tags: Difficulty: Hard · Asked at: JP Morgan, Goldman, Wells Fargo, Walmart Labs · Dialect: distributed

The scenario:

A senior engineer suggests 2PC for the cross-DB booking + payment transaction. The new engineer is excited — atomic across two DBs! Then they read the failure modes: blocked transactions on coordinator failure, in-doubt transactions requiring manual resolution, dramatic throughput drop.

2PC outline: 1. Coordinator sends PREPARE to all participants. 2. Each participant writes prepared state to its log and replies PREPARED or ABORT. 3. If all PREPARED, coordinator writes COMMIT, then sends COMMIT to all. 4. If any ABORT or timeout, coordinator sends ROLLBACK.

Failure modes: - Coordinator crashes after PREPARE but before COMMIT — participants are blocked indefinitely with locks held. - Network partition between coordinator and one participant — same. -

Recovery requires the coordinator's log to be intact and the DBAs to manually resolve in-doubt transactions.

Thinking out loud: 1. *"2PC trades availability for atomicity. Under failure, it picks blocked-and-correct over available-and-possibly-wrong."* 2. *"It's not theoretically broken — it's operationally heavy."* 3. *"Postgres supports it (PREPARE TRANSACTION / COMMIT PREPARED) but most ops teams avoid it."* 4. *"In Indian fintech, only certain bank-to-bank integrations use 2PC; everything else is saga."*

The diagnosis: 2PC is correct but operationally painful. The blocking problem is fundamental — see Three-Phase Commit, which trades safety for availability and is also rarely used.

The fix (when 2PC is genuinely required): - **Option A — Use a battle-tested transaction manager:** Java's JTA + Atomikos or Narayana; .NET's MSDTC. - **Option B — Saga:** Replace the 2PC with a saga + compensations. - **Option C — Single source of truth:** Restructure so the cross-DB write isn't needed.

What NOT to say: - Do not say "2PC is unsafe" — it is safe but blocking; the trade-off is documented and clear.

Common follow-up: *"What's the FLP impossibility result, and how does it relate to 2PC's blocking property?"*

Scenario369 · Optimistic Locking with a Version Column

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Walmart Labs, Goldman · Dialect: vendor-agnostic

The scenario:

A product-catalog editor allows multiple admins to edit the same product. Pessimistic locking (FOR UPDATE) would block one admin while another is editing. Optimistic locking lets both work; the second to save sees "stale version, please reload".

Schema:

```
CREATE TABLE products (  
  id BIGINT PRIMARY KEY,  
  name TEXT,  
  price NUMERIC,  
  version INT NOT NULL DEFAULT 0  
);
```

Update:


```
UPDATE products
SET name = $name, price = $price, version = version + 1
WHERE id = $id AND version = $expected_version;
-- check affected_rows: 1 = success, 0 = stale
```

Thinking out loud: 1. "Optimistic locking is wait-free in the success path." 2. "Loser gets `affected_rows = 0`; application surfaces a reload prompt." 3. "Works at any isolation level — the `WHERE` clause is the synchronisation." 4. "Trade-off: under heavy contention, lots of retries; pessimistic locking may be cheaper then."

The diagnosis: Optimistic locking pushes conflict detection into the `WHERE` clause. Cheap when conflicts are rare, costly when they're common.

The fix (here, the pattern *is* the fix): - **Option A — Version column:** Simplest; increment on every `UPDATE`. - **Option B — Timestamp column:** Use `last_modified_at` instead of version. Slight risk of clock skew. - **Option C — Hash of row contents:** ETag-style; only update if the current hash matches. Useful for HTTP-driven editors.

What NOT to say: - Do not say "optimistic locking is always better than pessimistic" — under heavy write contention, retries cost more than waits.

Common follow-up: "How would you handle merging two admins' edits instead of just rejecting one?"

Scenario370 · Pessimistic SELECT FOR UPDATE Pattern

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Wells Fargo, Goldman · Dialect: vendor-agnostic

The scenario:

A money-transfer service uses `SELECT FOR UPDATE` to lock both source and destination accounts before computing the new balances. It's the textbook pessimistic-locking pattern for fintech, and it works — provided you remember to lock in a deterministic order (Scenario 1) and to use `FOR UPDATE`, not `FOR SHARE`.

Pattern:

```

BEGIN;
-- Sort by id to avoid deadlock with the reverse transfer
SELECT id, balance FROM accounts WHERE id IN (101, 202)
ORDER BY id
FOR UPDATE;

-- Application reads both balances, computes new ones
UPDATE accounts SET balance = balance - 2000 WHERE id = 101;
UPDATE accounts SET balance = balance + 2000 WHERE id = 202;
INSERT INTO transfers (from_id, to_id, amount) VALUES (101, 202, 2000);
COMMIT;

```

Thinking out loud: 1. "FOR UPDATE acquires row-level exclusive locks. Other readers can proceed (in Postgres MVCC); other writers wait." 2. "FOR SHARE is wrong here — it allows another reader to also take a share lock and read stale data." 3. "Always sort the ids before locking — deterministic order prevents deadlock." 4. " **FOR UPDATE NOWAIT** fails immediately if another transaction holds the lock; **SKIP LOCKED** skips locked rows (useful for queue workers)."

The diagnosis: Pessimistic locking is the default for high-stakes financial workflows. Combine FOR UPDATE with deterministic ordering and idempotent retries for production-grade correctness.

The fix (this is the recommended pattern): - **Option A — FOR UPDATE + sorted ids:** As shown above. Battle-tested. - **Option B — FOR UPDATE NOWAIT** : Fail fast for user-facing flows where waiting is unacceptable. - **Option C — FOR UPDATE SKIP LOCKED** : For queue workers — pick the next unlocked job.

What NOT to say: - Do not say "FOR SHARE prevents writes" — it prevents writes from acquiring exclusive locks but allows concurrent shared readers, which is rarely what you want for read-modify-write.

Common follow-up: "How does SKIP LOCKED interact with isolation levels — does it give you the same correctness as a full FOR UPDATE?"

Category 6, Part B — Isolation Level Fixes & Anomaly Scenarios

Part B is about diagnosing concurrency anomalies (dirty read, non-repeatable read, phantom, write skew, lost update) and picking the right isolation level to fix them — without over-paying with Serializable when Snapshot would suffice. Every scenario below interleaves two transactions and asks you to spot what slipped through, then reason about the cheapest correctness fix. The trap is symmetric: candidates either reach for Serializable on every problem (kills throughput) or stay on Read Committed and learn the hard way that not all anomalies are visible at that level. Indian payment, inventory, and booking systems run very hot on Read Committed by default, so these scenarios mirror the production landmines that get filed as "ghost transactions", "double booking", or "stock went negative". For each scenario: read the interleaving, name the anomaly, then prescribe the minimal fix.

Scenario371 · Dirty Read — Reporting Picks Up Uncommitted Salary

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro · Dialect: SQL Server (READ UNCOMMITTED), Postgres (does not exist) · Topic: Dirty Read

The scenario:

Finance runs a payroll summary report. Around month-end, the totals it prints sometimes do not match the totals when the same query is rerun ten seconds later. No rows were deleted in between. The report runs at isolation `READ UNCOMMITTED` to "avoid blocking the OLTP system."

```
-- Txn A (payroll admin, accidentally still open)
BEGIN;
UPDATE salaries SET amount = 200000 WHERE emp_id = 42; -- typo, meant 20000
-- ...long pause while admin notices typo...

-- Txn B (report, READ UNCOMMITTED)
SELECT SUM(amount) FROM salaries; -- includes 200000

-- Txn A
ROLLBACK;
```

What happens: Txn B reads Txn A's uncommitted update and the rollback erases it. Report is wrong by 180000 and there is no row that ever existed which would explain it.

Thinking out loud: 1. "The numbers shift without any DML happening between runs — that's the classic dirty-read smell." 2. "Report is on `READ UNCOMMITTED`, so it sees uncommitted writes from other sessions." 3. "If the other session rolls back, the report's value never reflects any committed state." 4. "This is the only anomaly that can produce a value the database never agreed to."

Diagnosis: Dirty read. `READ UNCOMMITTED` on the reporting side allows visibility of uncommitted writes. A rollback then leaves the report holding a phantom number.

Fix options: - Move the report to `READ COMMITTED` (default on SQL Server and Postgres). Cost: small reader-writer blocking on SQL Server unless you enable `READ COMMITTED SNAPSHOT`. - On Postgres, dirty reads do not occur even if you ask for `READ UNCOMMITTED` — the engine silently treats it as `READ COMMITTED`. So the bug only reproduces on engines that honour the request (SQL Server, older MySQL configurations). - Design fix: keep reports out of OLTP and run them off a replica or a snapshot table; do not use isolation level to paper over architecture choices.

What NOT to say: - "Just retry the report" — it can land on a different rollback next time. - "Wrap the admin's `UPDATE` in a stored procedure" — does not address the reader's isolation contract.

Common follow-up: "On Postgres, does *READ UNCOMMITTED* ever produce dirty reads in practice?"

Scenario372 · Non-Repeatable Read — Audit Sees Two Balances

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, ICICI · Dialect: vendor-agnostic (default *READ COMMITTED*) · Topic: Non-Repeatable Read

The scenario:

An audit job under *READ COMMITTED* reads a customer's balance, performs a series of compliance checks, then re-reads the balance to confirm it has not changed during the check. The job sometimes fails the consistency assertion even when no fraud is occurring.

```
-- Txn Audit (READ COMMITTED)
BEGIN;
SELECT balance FROM accounts WHERE acct_id = 9001; -- returns 50000
-- ...runs sanctions checks, KYC lookups...

-- Txn UPI (commits between the two reads)
BEGIN;
UPDATE accounts SET balance = balance + 12000 WHERE acct_id = 9001;
COMMIT;

-- Txn Audit
SELECT balance FROM accounts WHERE acct_id = 9001; -- now returns 62000
COMMIT;
```

What happens: The same row read twice in the same transaction returns two different committed values. The audit assertion `read1 == read2` fails.

Thinking out loud: 1. "Two reads of the same row in one transaction, two different values — that's non-repeatable read by definition." 2. "*READ COMMITTED* only guarantees you never see uncommitted data — it says nothing about stability across reads." 3. "Each statement under *READ COMMITTED* takes its own snapshot." 4. "Either I need a stable snapshot for the whole transaction or I need to lock the row."

Diagnosis: Non-repeatable read. *READ COMMITTED* allows committed writes from other transactions to appear between two statements of the same reader.

Fix options: - Raise audit to *REPEATABLE READ* (MySQL InnoDB, Postgres) or *SNAPSHOT* (SQL Server). The whole transaction now sees one consistent view. - Use `SELECT ... FOR UPDATE` on the row at the start of the audit — locks out concurrent writers until commit. - Design fix: copy the balance into a local variable at the start; the assertion `var == reread()` is the wrong test in any case — write the audit against an immutable snapshot.

What NOT to say: - "READ COMMITTED is broken" — it gives exactly what it promises, no more. - "Use SERIALIZABLE" — works but is overkill for a single-row stability requirement.

Common follow-up: *"How does Postgres's REPEATABLE READ differ from the SQL standard's REPEATABLE READ?"*

Scenario373 · Phantom Read — Loan Cap Bypassed By Concurrent Insert

Tags: Difficulty: Medium · Asked at: HDFC, Bajaj Finserv, IDFC First · Dialect: SQL Server (REPEATABLE READ), Oracle (REPEATABLE READ) · Topic: Phantom Read

The scenario:

A loan-approval flow checks how many active loans a customer already holds, and refuses if they have three or more. Under REPEATABLE READ on SQL Server, two concurrent applications occasionally both succeed even though their combined effect breaks the cap.

```
-- Txn A (REPEATABLE READ)
BEGIN;
SELECT COUNT(*) FROM loans WHERE cust_id = 71 AND status = 'active'; -- returns 2
-- decides 2 < 3, will insert

-- Txn B (REPEATABLE READ, concurrent)
BEGIN;
SELECT COUNT(*) FROM loans WHERE cust_id = 71 AND status = 'active'; -- returns 2
INSERT INTO loans(cust_id, status, principal) VALUES (71, 'active', 50000);
COMMIT;

-- Txn A
INSERT INTO loans(cust_id, status, principal) VALUES (71, 'active', 80000);
COMMIT;
-- Customer now has 4 active loans
```

What happens: Both transactions saw count of two, both decided to insert, both committed. The cap of three was breached.

Thinking out loud: 1. "Each transaction's individual rows did not change between its reads — REPEATABLE READ held them stable." 2. "But a new row appeared in the same predicate range — that's a phantom, not a non-repeatable read." 3. "REPEATABLE READ in non-MVCC engines locks the rows it read, not the rows it would have read." 4. "The only standard fix is SERIALIZABLE, or a range lock equivalent."

Diagnosis: Phantom read. REPEATABLE READ keeps existing rows stable but does not prevent new rows from entering the predicate.

Fix options: - **SERIALIZABLE** — SQL Server takes range locks, Oracle uses serialization, Postgres uses SSI. - MySQL InnoDB's **REPEATABLE READ** is unusual: it uses gap locks and next-key locks, so phantoms are already prevented at **REPEATABLE READ** (a deliberate strengthening above the SQL standard). - Design fix: enforce the cap with a **CHECK** against a maintained counter column protected by **SELECT ... FOR UPDATE** on the customer row, so concurrent inserts serialise on that single lock.

What NOT to say: - "Just use **REPEATABLE READ** harder" — there is no "harder", you must climb a level on this engine. - "Add a unique constraint on (cust_id, status)" — kills the data model, customers legitimately have multiple active loans.

Common follow-up: *"Why does MySQL InnoDB not need **SERIALIZABLE** to prevent this phantom?"*

Scenario374 · Lost Update — Wallet Top-Up Vanishes

Tags: Difficulty: Easy · Asked at: Paytm, Mobikwik, PhonePe · Dialect: vendor-agnostic · Topic: Lost Update

The scenario:

A wallet top-up service runs at **READ COMMITTED** without row locking. Two simultaneous top-ups for the same user — one for 500, one for 800 — sometimes leave the wallet showing only one of the two amounts added, not both.

```
-- Txn A (top-up 500, READ COMMITTED)
BEGIN;
SELECT balance FROM wallets WHERE user_id = 555; -- reads 1000
-- application computes 1000 + 500 = 1500

-- Txn B (top-up 800, READ COMMITTED, concurrent)
BEGIN;
SELECT balance FROM wallets WHERE user_id = 555; -- reads 1000
UPDATE wallets SET balance = 1800 WHERE user_id = 555;
COMMIT;

-- Txn A
UPDATE wallets SET balance = 1500 WHERE user_id = 555;
COMMIT;
-- Final balance: 1500. The 800 top-up was overwritten.
```

What happens: Txn A read 1000, computed 1500 in app memory, and wrote that value back — clobbering Txn B's committed 1800. The 800 top-up is gone but the customer has the SMS receipt.

Thinking out loud: 1. *"Both reads saw 1000 because they happened before either commit."* 2.

"Application-side arithmetic means the database has no idea each transaction depended on the value

being 1000." 3. "Whichever transaction commits last wins, the other's effect disappears — that's a lost update."

Diagnosis: Lost update. Read-modify-write done in application memory under READ COMMITTED, with no locking or version check.

Fix options: - Use SQL arithmetic so the database does the read-modify-write atomically: `UPDATE wallets SET balance = balance + 500 WHERE user_id = 555;` — the row lock during UPDATE forces serial execution. - Pessimistic lock: `SELECT balance FROM wallets WHERE user_id = 555 FOR UPDATE;` before the application read. - Optimistic concurrency: add a `version` column; `UPDATE ... SET balance = ?, version = version + 1 WHERE user_id = ? AND version = ?;` — zero rows updated means retry. - Raise to REPEATABLE READ on Postgres/MySQL — these engines abort one of the conflicting transactions automatically (Postgres raises "could not serialize access due to concurrent update").

What NOT to say: - "It's a race condition, add a sleep" — please no. - "Use SERIALIZABLE for everything" — REPEATABLE READ already catches this on MVCC engines.

Common follow-up: "Which of these fixes survives the application crashing between read and write?"

Scenario375 · Write Skew — Two Doctors Both Go Off-Call

Tags: Difficulty: Hard · Asked at: Practo, 1mg, PharmEasy · Dialect: Postgres (SSI), MySQL (does not catch) · Topic: Write Skew

The scenario:

A hospital roster rule says at least one doctor must remain on call at all times. Two doctors, both currently on call, simultaneously request to go off duty. Under REPEATABLE READ, both requests succeed and the ward is left uncovered.

```

-- Txn A (Dr Iyer, REPEATABLE READ)
BEGIN;
SELECT COUNT(*) FROM oncall_doctors WHERE shift_id = 17 AND on_call = TRUE; -- returns 2
-- 2 > 1, safe to go off

-- Txn B (Dr Reddy, REPEATABLE READ, concurrent)
BEGIN;
SELECT COUNT(*) FROM oncall_doctors WHERE shift_id = 17 AND on_call = TRUE; -- returns 2
UPDATE oncall_doctors SET on_call = FALSE WHERE doctor_id = 'reddy';
COMMIT;

-- Txn A
UPDATE oncall_doctors SET on_call = FALSE WHERE doctor_id = 'iyer';
COMMIT;
-- 0 doctors on call

```

What happens: Each transaction read a snapshot showing two on-call doctors. Each made a decision based on that snapshot. Each wrote a different row. No write-write conflict; no row was read and then modified by the same transaction. Yet the invariant is broken.

Thinking out loud: 1. "Neither transaction overwrote the other's write — they touched different rows." 2. "Both made a decision off a snapshot that was true when they read but false by the time they committed." 3. "That's write skew — the classic invariant-violation anomaly." 4. "MVCC at REPEATABLE READ does not detect this because no row both wrote."

Diagnosis: Write skew. Two transactions read overlapping data, made disjoint writes that together violate an invariant. Only SERIALIZABLE (or explicit locking) catches it.

Fix options: - Postgres SERIALIZABLE (Serializable Snapshot Isolation, SSI) — detects the read-write dependency cycle at commit and aborts one transaction with serialization failure. - Materialise the conflict: have each transaction `SELECT ... FOR UPDATE` on every row that is currently `on_call = TRUE`, so the rows being read are locked and a write skew becomes a write-write conflict. - Maintain a counter row protected by a row lock — both transactions update the same counter, the second blocks. - MySQL InnoDB at SERIALIZABLE acquires shared locks on read, which prevents the skew but at significant concurrency cost.

What NOT to say: - "REPEATABLE READ should prevent that, the database is buggy" — REPEATABLE READ never promised invariant preservation across disjoint writes. - "Just use a unique constraint" — there is nothing unique to constrain.

Common follow-up: "What is the simplest invariant that survives write skew at REPEATABLE READ?"

Scenario376 · Read Skew — Inconsistent View Across Two Tables

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Zerodha · Dialect: vendor-agnostic · Topic: Read Skew

The scenario:

An end-of-day reconciliation reads the total of `accounts.balance` for a customer, then reads the running total of `ledger_entries.amount` for the same customer. The two should be equal. Under READ COMMITTED they are sometimes off by a single recent transfer.

```
-- Txn Recon (READ COMMITTED)
BEGIN;
SELECT SUM(balance) FROM accounts WHERE cust_id = 12; -- snapshot 1: 30000

-- Txn Transfer (commits between the two reads)
BEGIN;
UPDATE accounts SET balance = balance - 5000 WHERE acct_id = 100;
UPDATE accounts SET balance = balance + 5000 WHERE acct_id = 101;
INSERT INTO ledger_entries(cust_id, acct_id, amount) VALUES (12, 100, -5000), (12, 101, 5000);
COMMIT;

-- Txn Recon
SELECT SUM(amount) FROM ledger_entries WHERE cust_id = 12; -- snapshot 2: includes the new ±5000
COMMIT;
```

What happens: The recon job's two reads landed on either side of the transfer commit. Account sum is the "before" state, ledger sum is the "after" state. Recon flags a spurious mismatch.

Thinking out loud: 1. "Each statement under READ COMMITTED takes a fresh snapshot." 2. "A consistent cross-table read needs both statements to see the same instant of the database." 3. "That's read skew — closely related to non-repeatable read but spread across multiple objects."

Diagnosis: Read skew. Each statement saw a consistent state by itself, but together they did not see a consistent state.

Fix options: - REPEATABLE READ on Postgres or SNAPSHOT on SQL Server pins the whole transaction to one snapshot. - `SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;` at the start of the recon transaction is enough. - Design fix: precompute the ledger sum and account sum into a single materialised view refreshed atomically.

What NOT to say: - "Add a delay between the two reads" — makes it worse. - "Run recon at SERIALIZABLE" — works but no need to pay the abort cost.

Common follow-up: "Why is REPEATABLE READ enough here when SERIALIZABLE was needed for write skew?"

Scenario377 · Concurrent Inserts Both Trying To Reserve The Same Username

Tags: Difficulty: Easy · Asked at: Flipkart, Meesho, Nykaa · Dialect: vendor-agnostic · Topic: Uniqueness Race

The scenario:

A signup flow first checks if a username exists, then inserts a new user. Under load, two concurrent signups for the same username both pass the existence check, and one of the inserts later fails with a duplicate key violation — but only after expensive downstream work (welcome email, KYC start) was already triggered.

```
-- Txn A
SELECT 1 FROM users WHERE username = 'arnav99'; -- 0 rows
-- triggers welcome email pipeline
INSERT INTO users(username) VALUES ('arnav99'); -- succeeds

-- Txn B (concurrent)
SELECT 1 FROM users WHERE username = 'arnav99'; -- 0 rows (before A committed)
-- triggers welcome email pipeline
INSERT INTO users(username) VALUES ('arnav99'); -- fails with unique violation
```

What happens: Check-then-act on a uniqueness predicate is racy. Both transactions found nothing, both did work, one failed at insert time.

Thinking out loud: 1. "The existence check is informational, not enforcing — anything can change between *SELECT* and *INSERT*." 2. "The actual gatekeeper is the unique index on username — that always wins." 3. "I should let the database be the source of truth, not pre-validate in app code."

Diagnosis: TOCTOU on a uniqueness predicate. The *SELECT* and *INSERT* are not atomic.

Fix options: - Drop the *SELECT*, attempt the *INSERT*, catch the duplicate-key error and return "username taken" to the user. Cheaper and correct. - `INSERT ... ON CONFLICT DO NOTHING RETURNING id;` (Postgres) — if *RETURNING* is empty, the username was taken. - `INSERT IGNORE` (MySQL) returns row count zero for taken usernames. - If the downstream work is expensive, defer it until after the *INSERT* succeeds — the welcome email pipeline should be triggered from the row, not from the check.

What NOT to say: - "Add a transaction wrapper around *SELECT* and *INSERT*" — doesn't help unless you lock the table or use *SERIALIZABLE*. - "Use a distributed lock on the username string" — over-engineering, the unique index already serialises this.

Common follow-up: "Is the unique constraint enough if usernames can differ by case?"

Scenario378 · Increment-Counter Race Despite "Atomic" Single Statement

Tags: Difficulty: Easy · Asked at: Razorpay, Cred · Dialect: vendor-agnostic · Topic: Counter Race

The scenario:

A `daily_metrics` table tracks API call counts per endpoint per day. The increment is done with `count = count + 1`, but only after a `SELECT` establishes the row exists — otherwise an `INSERT` is issued. Under load, two concurrent first-call-of-the-day requests sometimes both insert, producing duplicate key errors or duplicate rows.

```
-- Txn A
SELECT count FROM daily_metrics WHERE endpoint = '/login' AND day = '2026-05-26'; -- 0 rows
INSERT INTO daily_metrics(endpoint, day, count) VALUES ('/login', '2026-05-26', 1);

-- Txn B (concurrent)
SELECT count FROM daily_metrics WHERE endpoint = '/login' AND day = '2026-05-26'; -- 0 rows
INSERT INTO daily_metrics(endpoint, day, count) VALUES ('/login', '2026-05-26', 1); -- duplicate k
```

What happens: Two transactions both saw no row, both inserted, the second failed at the unique index. The increment for that call is lost.

Thinking out loud: 1. "The `SELECT-then-INSERT-or-UPDATE` pattern is a textbook race." 2. "I should be using upsert — let the database decide insert vs update." 3. " `UPDATE count = count + 1` on a single row is atomic because of the row lock during `UPDATE`."

Diagnosis: Upsert race. `SELECT-then-INSERT-or-UPDATE` has a window where both transactions decide to insert.

Fix options: - Postgres: `INSERT INTO daily_metrics(endpoint, day, count) VALUES ('/login', '2026-05-26', 1) ON CONFLICT (endpoint, day) DO UPDATE SET count = daily_metrics.count + 1;` - MySQL: `INSERT INTO daily_metrics(endpoint, day, count) VALUES (...) ON DUPLICATE KEY UPDATE count = count + 1;` - SQL Server: `MERGE` or `IF EXISTS (...) UPDATE ELSE INSERT` inside an explicit transaction with `HOLDLOCK` hint. - Design fix: ensure the row exists for all endpoints at the start of day (a cron seeds the rows) so only `UPDATE` is ever needed.

What NOT to say: - "Use `SERIALIZABLE`" — sledgehammer for a tack. - "Catch the duplicate and retry the `UPDATE`" — works but the upsert idiom is one statement, not two.

Common follow-up: "What happens to `ON CONFLICT` under high contention on the same key?"

Scenario379 · Balance Check + Debit Race (TOCTOU)

Tags: Difficulty: Medium · Asked at: HDFC, Axis, RBL · Dialect: vendor-agnostic · Topic: TOCTOU

The scenario:

A debit flow checks the account balance is sufficient before debiting. Under READ COMMITTED, two concurrent debits of 600 each, against a balance of 1000, sometimes both succeed — leaving a negative balance of -200.

```
-- Txn A
BEGIN;
SELECT balance FROM accounts WHERE acct_id = 70; -- 1000
-- app: 1000 >= 600, proceed

-- Txn B (concurrent)
BEGIN;
SELECT balance FROM accounts WHERE acct_id = 70; -- 1000
-- app: 1000 >= 600, proceed
UPDATE accounts SET balance = balance - 600 WHERE acct_id = 70;
COMMIT;

-- Txn A
UPDATE accounts SET balance = balance - 600 WHERE acct_id = 70;
COMMIT;
-- final balance: -200
```

What happens: Each transaction made the sufficient-balance decision on a stale read. Both UPDATES used `balance - 600`, so both subtractions applied — leaving a negative balance.

Thinking out loud: 1. "The UPDATE arithmetic is atomic — the row lock means each `- 600` is applied serially. But the decision to debit was made in app code on a stale read." 2. "That's classic TOCTOU: the value that gates the decision is no longer the value when the action executes." 3. "I need to put the check inside the database, where it sees the locked, current value."

Diagnosis: TOCTOU on balance check. Application checked, database executed — the gap allowed overspend.

Fix options: - Pessimistic lock: `SELECT balance FROM accounts WHERE acct_id = 70 FOR UPDATE;` — blocks the second transaction until the first commits, the second then reads the updated 400 and refuses. - Push the check into the UPDATE: `UPDATE accounts SET balance = balance - 600 WHERE acct_id = 70 AND balance >= 600;` — zero rows updated means refuse. App must check `rowcount`. - Add a CHECK constraint: `CHECK (balance >= 0)` — both UPDATES run, the second errors out, transaction rolls back automatically. - Combine: the CHECK is the belt, the in-statement WHERE is the braces.

What NOT to say: - "Add `SELECT FOR UPDATE NOWAIT`" — non-blocking lock acquisition will fail immediately, blocks user experience. - "Use `SERIALIZABLE`" — works, but the targeted fixes are cheaper.

Common follow-up: "Why is `UPDATE ... WHERE balance >= 600` safe even at `READ COMMITTED`?"

Scenario380 · Inventory Deduction Race Going Negative

Tags: Difficulty: Medium · Asked at: Flipkart, Myntra, BigBasket · Dialect: vendor-agnostic · Topic: Inventory Race

The scenario:

Diwali sale, single SKU has 3 units left. Twenty checkout flows arrive within 200 ms. The deduction code reads stock, validates `>= 1`, then issues `UPDATE stock = stock - 1`. The dashboard ends up showing stock at -4 and the warehouse has to call seven customers to apologise.

```
-- Each of 20 concurrent Txns:
SELECT stock FROM inventory WHERE sku = 'TSHIRT-RED-M'; -- many see 3
-- app: 3 >= 1, proceed
UPDATE inventory SET stock = stock - 1 WHERE sku = 'TSHIRT-RED-M';
COMMIT;
```

What happens: Twenty reads saw 3. Twenty decisions proceeded. Twenty `UPDATE`s ran serially (each waiting for the row lock), each decrementing by 1. Final stock: -17 if no guard, capped at -4 if a partial guard intercepts.

Thinking out loud: 1. "Reads were not serialised, decisions were made stale." 2. "UPDATEs serialised on the row lock but did not re-check the guard." 3. "Either re-check inside the `UPDATE`, or lock the row before the read."

Diagnosis: Same pattern as the balance-debit. TOCTOU on inventory level.

Fix options: - `UPDATE inventory SET stock = stock - 1 WHERE sku = 'TSHIRT-RED-M' AND stock > 0;` — atomic check-and-decrement. App checks `rowcount`; if zero, refund or refuse. - CHECK constraint `CHECK (stock >= 0)` — backstop. - `SELECT ... FOR UPDATE` row lock before decision — works but reduces throughput, every checkout queues on the same row. - Reservation pattern: deduct into a `reserved` column at cart-time, real deduct at payment success — decouples checkout flow from the contention point. - For very hot SKUs, partition the stock counter into

N rows ("shards") and pick one at random per checkout; reconcile periodically. Accepts complexity in exchange for throughput.

What NOT to say: - "Use a queue and process one at a time" — kills checkout latency. - "Just oversell, refund later" — bad UX during sale spikes.

Common follow-up: "How would you handle a flash-sale with one million concurrent requests for one SKU?"

Scenario381 · "Add To Cart" Creates Duplicate Cart Rows

Tags: Difficulty: Easy · Asked at: Meesho, Snapdeal, Zepto · Dialect: vendor-agnostic · Topic: Duplicate Row

The scenario:

A user double-taps "Add to cart" on a slow connection. The handler does `SELECT cart_id FROM carts WHERE user_id = ?` and creates one if absent. Sometimes the user ends up with two cart rows and items split across both — abandoned-cart emails go to the wrong cart.

```
-- Txn A (first tap)
SELECT cart_id FROM carts WHERE user_id = 88; -- 0 rows
INSERT INTO carts(user_id) VALUES (88); -- inserts cart 501

-- Txn B (second tap, concurrent)
SELECT cart_id FROM carts WHERE user_id = 88; -- 0 rows (A not yet committed)
INSERT INTO carts(user_id) VALUES (88); -- inserts cart 502
```

What happens: Two carts for one user. The "items" table now has rows pointing at two different cart IDs for the same user.

Thinking out loud: 1. "There's no unique constraint on `carts.user_id`, so the database has nothing to push back with." 2. "The check-then-insert is racy even at `REPEATABLE READ` — phantoms slip through unless I lock the absence." 3. "The right fix is to make the unique constraint exist and use `upsert`."

Diagnosis: Missing unique constraint plus check-then-insert race. The schema invariant ("one cart per user") was implicit in app code, not enforced by the database.

Fix options: - Add `UNIQUE (user_id)` on the carts table. - Replace the pattern with `INSERT ... ON CONFLICT (user_id) DO UPDATE SET updated_at = now() RETURNING cart_id;` — guarantees one row per user, returns the canonical `cart_id` whether it was just created or already existed.

- Make the `cart_id` deterministic from `user_id` at the application layer (e.g. `cart_id = "cart-" + user_id`) so two flows compute the same key and the unique index catches the second attempt.

What NOT to say: - "Tell the frontend not to double-tap" — never trust the client. - "Add a Redis lock around add-to-cart" — additional infra, fragile, doesn't survive a node crash mid-lock.

Common follow-up: "How would you migrate to the unique constraint when existing rows already violate it?"

Scenario382 · Idempotent INSERT Via UNIQUE Constraint

Tags: Difficulty: Medium · Asked at: Razorpay, BillDesk, Juspay · Dialect: vendor-agnostic · Topic: Idempotency

The scenario:

A payment webhook from PSP retries on any non-200 response. The handler inserts into `payment_events`. After a network glitch, the same `psp_event_id` is sometimes received twice and reconciliation finds duplicate event rows.

```
-- First delivery
INSERT INTO payment_events(psp_event_id, status, amount)
VALUES ('evt_98aa3f', 'success', 4999);
-- response timed out, PSP retries

-- Retry delivery
INSERT INTO payment_events(psp_event_id, status, amount)
VALUES ('evt_98aa3f', 'success', 4999);
-- inserts again; reconciliation now double-counts
```

What happens: Without a unique constraint on `psp_event_id`, the second insert silently succeeds.

Thinking out loud: 1. "At-least-once delivery is the norm for webhooks. The receiver must enforce idempotency." 2. "The cheapest idempotency mechanism is a unique key on a natural deduplication identifier." 3. "Database does the dedup, the app code just catches the unique violation and returns 200."

Diagnosis: Missing idempotency enforcement. Webhook is at-least-once but receiver is not idempotent.

Fix options: - Add `UNIQUE (psp_event_id)` to `payment_events`. The retried insert raises a unique-violation error, handler catches it, returns 200. - Better:

`INSERT ... ON CONFLICT (psp_event_id) DO NOTHING RETURNING event_id;` — if `RETURNING` is empty, the event was already processed, return 200. No exception flow needed. - Pair the insert with the side-effect in the same transaction (e.g. updating order status). If the insert was a

duplicate, the side-effect is skipped automatically. - Keep a dedicated

`processed_events(psp_event_id)` audit table even if you do not retain the full payload — the unique key there is your dedup gate.

What NOT to say: - "Have the PSP not retry" — PSPs always retry, and they should. - "Check existence before insert" — back to the same TOCTOU class.

Common follow-up: *"What is the retention policy for the dedup keys, and what happens after they expire?"*

Scenario383 · INSERT ON CONFLICT For Atomic Upsert

Tags: Difficulty: Medium · Asked at: Cred, Niyo, INDmoney · Dialect: Postgres (ON CONFLICT), MySQL (ON DUPLICATE KEY), SQL Server (MERGE) · Topic: Upsert

The scenario:

A user profile sync from a third party pushes the same row periodically — sometimes the row exists already (update fields), sometimes not (insert it). The naive "try INSERT, catch error, do UPDATE" pattern has weird race-condition reports.

```
-- Attempt 1
INSERT INTO user_profiles(user_id, kyc_status, last_synced)
VALUES (101, 'verified', now());
-- if duplicate-key error, the app does:
UPDATE user_profiles SET kyc_status = 'verified', last_synced = now()
WHERE user_id = 101;
```

What happens: Between the INSERT failure and the UPDATE, another transaction can DELETE the row. The UPDATE then affects zero rows and the profile is silently missing.

Thinking out loud: 1. "Two-statement upsert has a window where state changes between the statements." 2. "The right pattern is one atomic statement that either inserts or updates."

Diagnosis: Non-atomic upsert. The retry-on-error idiom has a race window.

Fix options: - Postgres:

`INSERT INTO user_profiles(user_id, kyc_status, last_synced) VALUES (101, 'verified', now()) ON CONFLICT (user_id) DO UPDATE SET kyc_status = EXCLUDED.kyc_status, last_synced = EXCLUDED.last_synced;` - MySQL: `INSERT INTO user_profiles(...) VALUES (...) ON DUPLICATE KEY UPDATE kyc_status = VALUES(kyc_status), last_synced = VALUES(last_synced);` - SQL Server: `MERGE` (note: MERGE has historical bugs in SQL Server, use cautiously and serialize with HOLDLOCK if concurrent). -

Oracle: `MERGE INTO ... USING ... ON ... WHEN MATCHED THEN UPDATE WHEN NOT MATCHED THEN INSERT`.

What NOT to say: - "Try-INSERT-then-UPDATE is fine if we wrap it in a transaction" — wrapping does not close the race; another transaction can still mutate the row between your statements. - "Use SERIALIZABLE so the race is impossible" — overkill for a sync job.

Common follow-up: "What does `DO NOTHING` mean in `ON CONFLICT`, and when would you use it instead of `DO UPDATE`?"

Scenario384 · Optimistic Locking With Version Column

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Zerodha · Dialect: vendor-agnostic · Topic: Optimistic Concurrency

The scenario:

A trading app lets two devices edit the same order draft. Without locking, the slower device's save can clobber the faster device's edits — but the user has no idea their changes were overwritten.

```
-- Edit pattern with version column
SELECT order_id, qty, price, version FROM draft_orders WHERE order_id = 'ord_77';
-- returns (qty=10, price=500, version=3)
-- user edits qty to 12

UPDATE draft_orders
  SET qty = 12, version = version + 1
 WHERE order_id = 'ord_77' AND version = 3;
-- check rowcount: if 0, another writer won, retry or surface conflict
```

What happens: If a concurrent writer already bumped version to 4, the UPDATE affects zero rows. The slower device gets a "this draft was edited elsewhere" message and can choose to reload or merge.

Thinking out loud: 1. "Optimistic concurrency assumes most writers don't conflict. When they do, the loser is detected, not blocked." 2. "The version column is the conflict-detection token. Every read includes it, every write asserts it." 3. "No locks held; no waits; the conflict is detected by zero-rows-updated."

Diagnosis: Without a version column, simultaneous writers cause lost updates (back to Scenario 4). The version column converts the lost-update silent corruption into a detectable conflict.

Fix options: - Add `version INT NOT NULL DEFAULT 0` and update with `WHERE id = ? AND version = ?`. App must check `rowcount`. - Equivalent using `updated_at TIMESTAMP` — but timestamps can collide at sub-millisecond resolution, version integer is safer. - ORM support: Hibernate `@Version`, JPA `@Version`, Django `select_for_update` (different pattern), Rails `lock_version`. - Hybrid: optimistic by default, fall back to pessimistic `SELECT FOR UPDATE` on the second retry after a detected conflict.

What NOT to say: - "We don't need this if we have transactions" — transactions don't prevent lost updates at READ COMMITTED. - "Check `updated_at` is newer in the WHERE" — race conditions on clock skew, prefer monotonic integer.

Common follow-up: "How do you surface a conflict to the user without being annoying?"

Scenario385 · Pessimistic SELECT FOR UPDATE

Tags: Difficulty: Medium · Asked at: HDFC, Axis, Yes Bank · Dialect: vendor-agnostic · Topic: Pessimistic Locking

The scenario:

An inter-account transfer must read both source and destination balances, validate the source has enough, then update both. Under load, multiple transfers between the same pair sometimes interleave badly.

```
BEGIN;
SELECT balance FROM accounts WHERE acct_id IN (101, 102) FOR UPDATE;
-- both rows locked exclusively
-- ... app does validation ...
UPDATE accounts SET balance = balance - 1000 WHERE acct_id = 101;
UPDATE accounts SET balance = balance + 1000 WHERE acct_id = 102;
COMMIT;
```

What happens: The FOR UPDATE lock blocks concurrent transactions trying to lock the same rows. They queue, run serially, no anomalies.

Thinking out loud: 1. "Pessimistic locking assumes conflict is likely and pre-empts it by blocking the second writer." 2. "FOR UPDATE acquires exclusive row locks, held until commit or rollback." 3. "Deadlock risk: always lock in a consistent order (e.g. by `acct_id` ascending)."

Diagnosis: Without FOR UPDATE, the read is non-locking — concurrent transactions can read the same balances and base decisions on stale values.

Fix options: - `SELECT ... FOR UPDATE` — exclusive, blocks both readers and writers (in InnoDB) or just writers (in Postgres, where readers can still read). - `SELECT ... FOR NO KEY UPDATE` (Postgres) — slightly weaker, allows concurrent reads that don't touch key columns. - `SELECT ... FOR SHARE` (Postgres) or `LOCK IN SHARE MODE` (MySQL) — shared lock, blocks writers but not other readers. Useful for read-validate-then-write patterns where many txns can read concurrently. - Deadlock prevention: order locks by primary key, set `lock_timeout` so a deadlocked transaction errors out rather than waiting indefinitely.

What NOT to say: - "Lock the entire table" — kills throughput. - "FOR UPDATE prevents all anomalies" — only the ones whose rows you actually locked; phantoms still need SERIALIZABLE on most engines.

Common follow-up: *"How do you avoid deadlock when two transfers go in opposite directions on the same pair of accounts?"*

Scenario386 · SELECT FOR UPDATE SKIP LOCKED For Queue Processing

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Dunzo · Dialect: Postgres, MySQL 8+ · Topic: Queue Pattern

The scenario:

A job-queue table holds pending jobs. Multiple worker processes need to pick "the next available job" without two workers grabbing the same job. The naive `SELECT ... LIMIT 1 FOR UPDATE` causes workers to block each other and serialise — throughput drops to one worker effectively active at a time.

```
-- Worker pattern
BEGIN;
SELECT job_id, payload FROM jobs
  WHERE status = 'pending'
 ORDER BY created_at
  LIMIT 1
  FOR UPDATE SKIP LOCKED;
-- process the job
UPDATE jobs SET status = 'done' WHERE job_id = ?;
COMMIT;
```

What happens: With SKIP LOCKED, each worker grabs a different row — if the first row is locked by worker A, worker B silently picks the next unlocked row instead of waiting. Throughput scales with worker count.

Thinking out loud: 1. "Plain `FOR UPDATE` makes workers queue on the same row — exactly what I'm trying to avoid." 2. "`SKIP LOCKED` says: don't wait on rows already locked, pretend they aren't there." 3. "Combined with `LIMIT 1`, each worker gets the next row no other worker is currently working on."

Diagnosis: Without `SKIP LOCKED`, `FOR UPDATE` serialises workers on the first row of the queue. `SKIP LOCKED` parallelises them.

Fix options: - Postgres: `FOR UPDATE SKIP LOCKED` — available since 9.5. - MySQL: `FOR UPDATE SKIP LOCKED` — available since 8.0. - Oracle: `FOR UPDATE SKIP LOCKED` — original implementer of the pattern. - SQL Server: `READPAST` hint with `UPDLOCK` — equivalent. - Pair with `LIMIT` (Postgres/MySQL) or `FETCH FIRST 1 ROW ONLY` (Oracle/SQL Server) so each worker takes a small batch. - Design fix: dedicated queue systems (RabbitMQ, SQS) handle this natively; database-as-queue works for low scale and avoids extra infra.

What NOT to say: - "Use `NOLOCK` and just hope" — `NOLOCK` is dirty read, will let two workers pick the same job. - "Have workers pick by `job_id MOD worker_count = my_id`" — works but loses ordering and breaks if worker count changes.

Common follow-up: "What happens if a worker crashes after locking a job but before updating its status?"

Scenario387 · READ COMMITTED SNAPSHOT vs READ COMMITTED on SQL Server

Tags: Difficulty: Hard · Asked at: TCS GCC, Infosys GCC, Capgemini GCC (BFSI clients) · Dialect: SQL Server · Topic: SQL Server Isolation

The scenario:

A SQL Server OLTP system shows reads blocking writes and writes blocking reads under load. Enabling `READ COMMITTED SNAPSHOT` (RCSI) eliminates the blocking, but a senior engineer flags that this changes semantic guarantees.

Default behaviour (READ COMMITTED, locking): - Readers acquire shared locks, writers acquire exclusive locks. Each blocks the other. - A reader sees the latest committed data but blocks any in-flight writer until its statement completes. - Statement-level consistency only — two statements in one transaction see different snapshots.

With READ COMMITTED SNAPSHOT enabled: - Readers do not take shared locks. They read the last committed version of each row from the tempdb version store. - Writers do not block readers; readers do not block writers. - Semantic difference: a reader sees a snapshot of each row as of the start of the statement, even if a writer subsequently commits during the read.

Thinking out loud: 1. "Default SQL Server isolation is lock-based — concurrent readers and writers contend." 2. "RCSI is MVCC-flavoured READ COMMITTED — readers see versioned snapshots, writers don't block them." 3. "Same statement-level consistency contract, but reached via versions instead of locks." 4. "The trade is tempdb pressure for reduced blocking — usually worth it on OLTP."

Diagnosis: The locking READ COMMITTED is the source of the reader-writer contention. RCSI removes it but moves the cost to tempdb version-store maintenance.

Fix options: - `ALTER DATABASE my_db SET READ_COMMITTED_SNAPSHOT ON;` — flips all READ COMMITTED transactions to use snapshots. - Use `ALLOW_SNAPSHOT_ISOLATION ON` separately to enable explicit SNAPSHOT isolation level (full-transaction snapshot, equivalent to Postgres REPEATABLE READ). - Tradeoff: tempdb size grows with version-store volume; long-running transactions cause version-store bloat.

What NOT to say: - "RCSI is the same as Postgres MVCC" — Postgres versions live in the table, SQL Server's live in tempdb — operational behaviour differs. - "RCSI removes all anomalies" — it removes the locking artefacts; non-repeatable reads and phantoms still apply at the READ COMMITTED level.

Common follow-up: "What happens to tempdb if a long-running report runs while RCSI is on?"

Scenario388 · Postgres SSI Catching Write Skew

Tags: Difficulty: Hard · Asked at: Razorpay, Cred · Dialect: Postgres · Topic: Serializable Snapshot Isolation

The scenario:

A two-doctor-on-call scenario (like Scenario 5) is implemented in Postgres at SERIALizable. Both transactions appear to commit successfully — until the second one is suddenly aborted with `ERROR: could not serialize access due to read/write dependencies among transactions`.

Postgres SSI behaviour: - SERIALizable in Postgres is implemented as Serializable Snapshot Isolation (SSI). - Each transaction runs at REPEATABLE READ semantics PLUS tracks read-write dependencies between concurrent transactions. - At commit, Postgres checks for a dangerous dependency cycle; if one exists, one transaction is aborted with `serialization_failure` (SQLSTATE 40001). - The application's responsibility is to retry the aborted transaction.

Thinking out loud: 1. "Postgres SSI gives serializable semantics without taking shared read locks — it tracks dependencies instead." 2. "The cost is paid at commit time as a serialization failure on the loser transaction." 3. "Applications must be retry-aware — every SERIALizable transaction needs a retry wrapper."

Diagnosis: SSI is the only practical way Postgres prevents write skew without app-level locking. The retry loop is part of the contract.

Fix options: - Wrap SERIALizable transactions in a retry loop: catch `serialization_failure`, reset state, retry up to N times with exponential backoff. - Keep transactions short — long transactions increase SSI's tracking overhead and the chance of detected cycles. - Use SERIALizable only on the transactions that actually need write-skew protection; keep the rest at READ COMMITTED. - Don't mix SERIALizable and lower-level transactions casually — SSI's guarantees apply to interactions between SERIALizable transactions; a concurrent READ COMMITTED writer can still cause anomalies for a SERIALizable reader.

What NOT to say: - "SERIALizable in Postgres is the same as in SQL Server" — SQL Server uses range locks, Postgres uses SSI. Different cost models. - "Just retry forever on failure" — bound the retries and surface persistent conflicts to the user.

Common follow-up: *"What is the difference between SSI's serialization failure and a deadlock abort?"*

Scenario389 · MySQL InnoDB Gap Locks Preventing Phantoms

Tags: Difficulty: Hard · Asked at: Flipkart, Myntra, MakeMyTrip · Dialect: MySQL · Topic: Gap Locking

The scenario:

A booking system at MySQL InnoDB's default REPEATABLE READ never appears to suffer phantoms — but the same code on SQL Server's REPEATABLE READ does. The team wonders if MySQL is silently using SERIALizable.

InnoDB REPEATABLE READ behaviour: - InnoDB uses next-key locks: a row lock plus a gap lock on the gap before the row. - A range scan like `SELECT ... WHERE x BETWEEN 5 AND 10 FOR UPDATE` locks the rows in that range AND the gaps between them, preventing inserts. - This is stricter than the SQL standard's REPEATABLE READ, which only requires existing rows be stable.

Thinking out loud: 1. *"InnoDB intentionally over-delivers on REPEATABLE READ — gap locks prevent phantoms even though the standard doesn't require it."* 2. *"The price: gap locks reduce insert concurrency in ranges being read."* 3. *"This is why MySQL apps often don't need SERIALizable for invariant safety."*

Diagnosis: InnoDB's REPEATABLE READ is strictly stronger than the standard. Gap locks close the phantom window that other engines leave open.

Fix options: - Stay at REPEATABLE READ on MySQL — phantoms already prevented for locking reads (FOR UPDATE / LOCK IN SHARE MODE). - For non-locking reads, MySQL takes a snapshot at the first

read and reuses it for the transaction — also prevents phantoms via MVCC. - To get insert concurrency back on a range, switch to READ COMMITTED — disables gap locks but reintroduces phantom risk.

What NOT to say: - "MySQL REPEATABLE READ is broken because it doesn't match the SQL standard" — it's a deliberate, documented strengthening. - "Disable gap locks for performance" — they exist for correctness; replace with explicit locking strategy first.

Common follow-up: *"How do gap locks contribute to InnoDB deadlocks?"*

Scenario390 · Choosing Snapshot Over Serializable For Performance

Tags: Difficulty: Medium · Asked at: Cred, Razorpay, Zerodha · Dialect: vendor-agnostic · Topic: Isolation Trade-off

The scenario:

A long-running analytics query needs to scan millions of rows for a consistent view. The team initially set it to SERIALIZABLE for safety, but it conflicts with concurrent writers and frequently aborts with serialization failures. The right answer is usually Snapshot, not Serializable.

The trade-off: - SERIALIZABLE (Postgres SSI, SQL Server with range locks): protects against write skew and other dependency-cycle anomalies; conflicts manifest as aborts or blocking. - SNAPSHOT / REPEATABLE READ on MVCC engines: gives a consistent transaction-long snapshot; no read-write blocking; vulnerable to write skew but read-only queries cannot cause write skew by definition.

Thinking out loud: 1. "A read-only analytics query cannot suffer write skew — write skew requires both transactions to write." 2. "All I need is a consistent snapshot — REPEATABLE READ delivers that without the SSI overhead." 3. "Reserve SERIALIZABLE for transactions that read-and-write invariants."

Diagnosis: SERIALIZABLE was over-specified. The query is read-only and only needs snapshot consistency.

Fix options: - Postgres: `SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;` or `SET TRANSACTION ISOLATION LEVEL REPEATABLE READ READ ONLY;` — read-only hint allows additional optimisations. - SQL Server: `SET TRANSACTION ISOLATION LEVEL SNAPSHOT;` — explicit snapshot, full-transaction consistency. - Decide per-transaction, not per-database — SET TRANSACTION applies only to the current transaction. - For very long analytics, consider a snapshot copy on a read replica to avoid impact on OLTP.

What NOT to say: - "SNAPSHOT and SERIALIZABLE are the same thing" — Snapshot does not protect against write skew; Serializable does. - "Just SERIALIZABLE everything for safety" — over-paying for correctness you don't need.

Common follow-up: *"Name a workload where SNAPSHOT is wrong and SERIALIZABLE is required."*

Scenario391 · REPEATABLE READ Inside A Long Batch — Bloat Consequence

Tags: Difficulty: Hard · Asked at: Razorpay, BillDesk, Juspay · Dialect: Postgres · Topic: MVCC Bloat

The scenario:

A nightly batch processes millions of orders inside a single REPEATABLE READ transaction. The next morning, the database is sluggish — query plans are unchanged but everything is slow. A DBA notices the table has 10x its expected on-disk size.

What happens: Postgres MVCC keeps old row versions alive as long as any open transaction's snapshot could see them. A long REPEATABLE READ batch holds a snapshot for hours; VACUUM cannot reclaim any row updated or deleted during the batch. The table bloats.

Thinking out loud: 1. "REPEATABLE READ pins a snapshot for the whole transaction." 2. "VACUUM can only free row versions older than the oldest snapshot." 3. "Hours-long batch transactions hold snapshots that block VACUUM hours behind."

Diagnosis: Long-running REPEATABLE READ transaction caused MVCC version retention, blocking VACUUM and producing table bloat.

Fix options: - Break the batch into smaller transactions — each commit releases its snapshot and allows VACUUM to make progress. - Use a server-side cursor with `WITH HOLD` if you genuinely need to iterate a single result across many transactions. - Schedule batches during low-OLTP windows so VACUUM has slack to recover afterwards. - Run `pg_stat_activity` checks for `xact_start` older than N minutes and alert; long transactions are operational smells.

What NOT to say: - "Run VACUUM FULL nightly" — locks the table, OLTP downtime. - "REPEATABLE READ is fundamentally broken for batches" — the isolation level is fine; the transaction scope is wrong.

Common follow-up: "How does autovacuum decide which tables to vacuum, and what tuning helps in this case?"

Scenario392 · Postgres VACUUM Behaviour With Long Transactions

Tags: Difficulty: Hard · Asked at: Razorpay, Cred · Dialect: Postgres · Topic: MVCC / Vacuum

The scenario:

A monitoring dashboard shows `pg_stat_user_tables.n_dead_tup` climbing steadily on a hot table. Autovacuum runs frequently but the dead tuple count keeps growing. A `pg_stat_activity` query shows a user's reporting session open for the past four hours.

What happens: A reporting transaction holding a snapshot prevents VACUUM from removing row versions that other transactions deleted or superseded. Even though VACUUM runs, it cannot reclaim anything older than what the snapshot still needs.

Thinking out loud: 1. "Dead tuples are stale versions awaiting reclamation by VACUUM." 2. "VACUUM uses the oldest open transaction's xmin as the horizon for reclamation." 3. "A long-open transaction shifts that horizon backwards — VACUUM works, but reclaims nothing."

Diagnosis: An idle-in-transaction or long-running transaction held the xmin horizon and prevented VACUUM from doing its job.

Fix options: - `idle_in_transaction_session_timeout` — auto-terminates sessions idle inside a transaction beyond a threshold (e.g. 10 minutes). - `old_snapshot_threshold` — lets VACUUM reclaim rows older than N minutes even if a snapshot needed them; subsequent queries on those tables get "snapshot too old" errors. - Monitor `pg_stat_activity` for `state = 'idle in transaction'` and alert. - Educate developers: open transactions only when needed, commit promptly, never hold a transaction across user input.

What NOT to say: - "Just run VACUUM more often" — frequency doesn't help if the horizon hasn't moved. - "Restart Postgres to clear it" — works in panic, but the underlying habit will repeat.

Common follow-up: "What does `idle in transaction (aborted)` mean in `pg_stat_activity`?"

Scenario393 · Connection Pool + Transaction Scope (Leaking Transactions)

Tags: Difficulty: Hard · Asked at: Razorpay, PhonePe, Zerodha · Dialect: vendor-agnostic · Topic: App Pool Discipline

The scenario:

An application using HikariCP randomly experiences "transaction state from another request" symptoms — a query that should see a freshly inserted row sometimes does not, and rows from a previous user's session occasionally appear in the current user's request. The bug is intermittent and hard to reproduce.

What happens: A connection was returned to the pool with an open transaction. The next checkout-and-use got the connection back with the previous transaction still active. Reads see the previous transaction's snapshot; writes commit under it.

Thinking out loud: 1. *"Connection pools recycle physical connections — they don't reset transaction state unless asked to."* 2. *"If a request flow leaks an open transaction, the next user of that connection inherits it."* 3. *"That's a serious cross-request data isolation breach."*

Diagnosis: Transaction scope was not aligned with request scope. Connection returned to pool dirty.

Fix options: - Use the pool's "test-on-return" or "reset-on-return" hook to issue `ROLLBACK` (or check `autocommit`) before returning a connection to the pool. - Discipline: every code path that opens a transaction must close it (commit or rollback) in a finally / try-with-resources block. - Spring `@Transactional`, JTA, or equivalent transaction managers — let the framework own start/commit/rollback. Don't mix manual `connection.setAutoCommit(false)` with framework transactions. - Add observability: log a warning when a connection is checked out with `getAutoCommit() == false` at the start of a request.

What NOT to say: - "Increase the pool size" — does not address state leakage. - "Always create a new connection per request" — defeats the purpose of pooling; you need the cleanup, not the disposal.

Common follow-up: *"How does Spring's @Transactional decide when to commit vs roll back?"*

Scenario394 · Read Replica + Transaction Visibility Lag

Tags: Difficulty: Hard · Asked at: Razorpay, PhonePe, Flipkart · Dialect: vendor-agnostic · Topic: Replication

The scenario:

A user updates their profile on the primary, gets a success response, navigates to the profile page — which is served from a read replica — and sees the OLD profile. After a few seconds it shows the new one. Customer support is buried with "your app is broken" tickets.

What happens: The primary has the new value; the replica is N ms behind. The read for the page rendering went to the replica before the replication stream had applied the change.

Thinking out loud: 1. *"Replication is asynchronous (in most setups) — committed-on-primary does not mean visible-on-replica."* 2. *"Read-your-writes consistency does not hold across primary/replica unless explicitly arranged."* 3. *"The simplest fix is: after a write, route the user's subsequent reads to the primary for a short window."*

Diagnosis: Read-your-writes anomaly due to replica lag. The transaction committed on primary but the user's next read was served from a stale replica.

Fix options: - Sticky-to-primary for N seconds after a write — application records "last write at" timestamp; reads within the staleness window go to primary. - Synchronous replication mode for critical writes — primary waits for ack from at least one replica before returning success. Trades latency for consistency. - Read from primary for read-after-write paths, replicas for analytics and cold reads. - Postgres: `synchronous_commit = on` + `synchronous_standby_names` ensures replicas are caught up before commit returns. - MySQL: semi-sync replication offers similar guarantees.

What NOT to say: - "Just refresh the page" — pushes the bug to users. - "Add a 2-second delay before reading" — fragile and slow.

Common follow-up: "What does session consistency mean in DynamoDB terms?"

Scenario395 · Multi-Statement Transaction Holding Read View Too Long

Tags: Difficulty: Medium · Asked at: TCS, Wipro, Capgemini · Dialect: vendor-agnostic · Topic: Transaction Scope

The scenario:

A user-facing checkout transaction includes a call to an external SMS provider while a row is locked. The lock is held for the duration of the SMS API call (which can take 4 seconds). Concurrent transactions trying to read or write the same row queue up, request timeouts cascade.

```
BEGIN;
SELECT ... FROM orders WHERE order_id = 99 FOR UPDATE;
-- ... call SMS API (4 seconds) ...
UPDATE orders SET status = 'confirmed' WHERE order_id = 99;
COMMIT;
```

What happens: The row lock acquired by FOR UPDATE is held until commit. The external call extends the lock's lifetime to seconds. Concurrent transactions block, downstream timeouts ensue.

Thinking out loud: 1. "Never hold a database transaction open across an external network call." 2. "The lock is the contention point; everything queues on it." 3. "Move the external call out of the transaction or use compensating actions if it fails."

Diagnosis: Transaction scope extends across an external API call, holding locks for far too long.

Fix options: - Restructure: do the database work (commit), then do the SMS call. If the SMS fails, log it and retry asynchronously — most flows can tolerate eventual SMS delivery. - If the SMS must succeed before commit, use the outbox pattern (see Scenario 29) — insert a row to send the SMS, commit, then a background worker reads the outbox and sends. - Set `lock_timeout` so a transaction holding a lock

too long is aborted (treats the disease, not the cause). - Monitor longest-held lock duration as a database SLI.

What NOT to say: - "Make the SMS call async with a thread pool inside the transaction" — the thread returns but the transaction still holds the lock. - "Increase the lock timeout" — makes the problem worse.

Common follow-up: "How do you architect a transaction that needs to atomically commit a database write and an external action?"

Scenario396 · Application Retry Pattern After Serialization Failure

Tags: Difficulty: Medium · Asked at: Razorpay, Cred · Dialect: Postgres SERIALIZABLE · Topic: Retry Loop

The scenario:

A team enables SERIALIZABLE for a critical transfer flow on Postgres. Some transactions now fail with `serialization_failure` (SQLSTATE 40001). The team's "fix" is to log the error and surface a 500 to the user. The right pattern is to retry transparently.

```
-- Pseudocode
for attempt in 1..MAX_RETRIES:
    try:
        BEGIN;
        -- ... transaction body ...
        COMMIT;
        return success
    catch SerializationFailure:
        ROLLBACK;
        sleep(backoff * 2^attempt)
    catch DeadlockError:
        ROLLBACK;
        sleep(backoff * 2^attempt)
raise FailureAfterMaxRetries
```

Thinking out loud: 1. "Serialization failures are a normal outcome of SERIALIZABLE under contention — they're a signal to retry, not an error to surface." 2. "Deadlocks at any isolation level are similarly retry-able." 3. "Exponential backoff and a retry cap prevent thundering-herd retries."

Diagnosis: Treating `serialization_failure` as a terminal error misuses the abstraction. SSI's contract is "either succeed or abort and retry."

Fix options: - Centralise the retry logic in a transactional helper. Every SERIALIZABLE transaction is wrapped automatically. - Distinguish error classes: serialization failures and deadlocks are retry-able;

constraint violations are not. - Cap retries (e.g. 3) and surface a user-facing "please try again" only after exhausting them. - Metric: count retries per endpoint; persistent high counts indicate hot conflicts to redesign. - Make the transaction body idempotent so retries are safe — no externally visible side effects until after commit.

What NOT to say: - "Retry forever" — unbounded retries cascade into capacity issues. - "Disable SERIALIZABLE to avoid the failures" — gives up the correctness guarantee that motivated SERIALIZABLE.

Common follow-up: *"What does the SQL state 40001 mean across Postgres, MySQL, and SQL Server?"*

Scenario397 · Idempotency Key + At-Least-Once Delivery

Tags: Difficulty: Medium · Asked at: Razorpay, Stripe, Cashfree · Dialect: vendor-agnostic · Topic: Idempotency

The scenario:

A payment-create endpoint can be retried by the client on network errors. Without an idempotency key, two retries result in two charges on the customer's card.

```
-- Client provides idempotency_key in request header
CREATE TABLE idempotency_records (
  idempotency_key TEXT PRIMARY KEY,
  response_body JSON,
  status_code INT,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- Handler flow
BEGIN;
INSERT INTO idempotency_records(idempotency_key, response_body, status_code)
VALUES (:key, NULL, NULL)
ON CONFLICT (idempotency_key) DO NOTHING
RETURNING idempotency_key;
-- if RETURNING is empty: the key was used before; SELECT the stored response and return it
-- if RETURNING is non-empty: proceed with the actual work, then UPDATE the record with the response
COMMIT;
```

What happens: First attempt inserts the key, processes the payment, stores the response. Retries hit the conflict path, find the stored response, return it unchanged. The customer sees one charge, the integrator's retries are absorbed safely.

Thinking out loud: 1. "At-least-once is the only realistic delivery contract for network APIs." 2. "The server must enforce idempotency — clients cannot." 3. "An idempotency key is the contract: same key, same outcome, regardless of how many times you call."

Diagnosis: Endpoint is not idempotent. Retries cause duplicate side effects.

Fix options: - Idempotency key per request, indexed in a dedicated table with the response cached. - Store the response so retries return the original response — clients see exactly the same data as the first call. - TTL the keys (e.g. 24 hours) to bound storage growth. - Pair with a unique constraint on the underlying domain object where possible (e.g. payment unique on (order_id, attempt_no)).

What NOT to say: - "Tell clients not to retry" — they will, and they should. - "Use a UUID as the idempotency key on the server side" — defeats the purpose; the client must supply the key for retries to share it.

Common follow-up: "How long should an idempotency key be retained, and what happens to retries that arrive after expiry?"

Scenario398 · Outbox Pattern To Prevent Dual-Write Inconsistency

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Flipkart · Dialect: vendor-agnostic · Topic: Dual-Write

The scenario:

An order-placement handler inserts into the orders table, then publishes an event to a message broker. Sometimes the database insert commits but the broker publish fails (network blip). Downstream services that listen to the event never know the order exists.

```
-- Broken pattern: dual write
BEGIN;
INSERT INTO orders(...) VALUES (...);
COMMIT;
-- now publish to Kafka
broker.publish("order_placed", payload); -- may fail; we have no way to retry consistently
```

What happens: The database commit and the broker publish are two separate writes to two systems. There is no shared atomic boundary, so any partial failure leaves the systems inconsistent.

Thinking out loud: 1. "Two-phase commit between databases and brokers is rarely available and rarely fast enough." 2. "Outbox pattern: write the event to an **outbox** table in the same transaction as the order. A relay polls the outbox and publishes." 3. "Single transaction means either both the order and the outbox row commit, or neither does."

Diagnosis: Dual-write inconsistency. No atomicity across database and broker.

Fix options: - Outbox table: `CREATE TABLE outbox (id BIGSERIAL, aggregate_id TEXT, event_type TEXT, payload JSONB, published BOOLEAN DEFAULT FALSE);` - Insert into outbox inside the same transaction as the domain change. Atomic. - A separate relay process polls outbox `WHERE published = FALSE FOR UPDATE SKIP LOCKED`, publishes, marks as `published = TRUE`. - At-least-once delivery is acceptable because downstream is built to be idempotent. - Alternative: Change Data Capture (Debezium) reads the WAL and emits events — no application-side outbox needed.

What NOT to say: - "Use a distributed transaction (XA)" — operationally painful, slow, often unavailable for brokers. - "Retry the publish in a finally block" — does not survive process crash between commit and publish.

Common follow-up: *"How do you guarantee event ordering when multiple relay workers publish from the outbox in parallel?"*

Scenario399 · Saga Compensation Pattern

Tags: Difficulty: Hard · Asked at: Swiggy, Flipkart, Razorpay · Dialect: vendor-agnostic · Topic: Distributed Transaction

The scenario:

A travel booking spans three services: flight, hotel, payment. If the payment fails after the flight and hotel are booked, both bookings need to be reversed. There is no shared database transaction that spans the three services.

The pattern: - Each step is its own local transaction in its own service. - For each step, a corresponding compensating transaction is defined. - If any step fails, the orchestrator invokes the compensating actions for all completed prior steps in reverse order.

```
forward:    flight.book → hotel.book → payment.charge
compensate: payment.refund ← hotel.cancel ← flight.cancel
```

Thinking out loud: 1. *"Distributed atomicity is hard; the saga gives up atomic rollback for explicit compensation."* 2. *"Each service is responsible for its own consistency; the orchestrator coordinates the choreography."* 3. *"Compensation is not always perfectly inverse — refund may charge a fee, cancellation may leave audit traces. Domain decides."*

Diagnosis: No global transaction is possible across services. Sagas decompose the workflow into local transactions plus compensations.

Fix options: - Orchestrator-based saga: a central coordinator (e.g. AWS Step Functions, Camunda, Temporal) drives the steps and compensations. - Choreography-based saga: each service listens for events and reacts. No central coordinator. Harder to reason about, easier to scale. - Each step must be idempotent (retries are inevitable). - Compensations must also be idempotent — retrying a refund must not refund twice. - Persist saga state so a crash mid-workflow can resume.

What NOT to say: - "Use a distributed transaction (XA) across the services" — usually impractical with HTTP services and brokers. - "Just wrap everything in a try-catch" — does not address process crashes or partial network failures.

Common follow-up: *"How does a saga handle the case where a compensating action itself fails?"*

Scenario400 · Eventual Consistency vs Strong Consistency Choice

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Flipkart · Dialect: vendor-agnostic · Topic: Consistency Model

The scenario:

A product manager asks whether to use eventual consistency for a "follower count" feature on a social product, vs strong consistency for the "wallet balance" feature on a payments product. The engineering team needs to articulate the trade-off and decide.

Strong consistency: - After a write completes, all subsequent reads everywhere see the new value. - Achieved via single-leader synchronous replication, consensus protocols, or distributed locks. - Cost: higher latency, lower availability during partitions (CAP theorem trade).

Eventual consistency: - Reads may see stale values for a bounded period after a write. - All replicas converge given time and no further writes. - Cost: application must tolerate staleness, handle reordering, design for read-your-writes via session affinity.

Thinking out loud: 1. *"Follower count being off by 30 seconds is fine — UX impact is invisible."* 2. *"Wallet balance being off by 30 seconds enables double-spend — UX impact is a refund queue and possibly fraud."* 3. *"Strong consistency is paid for in latency and availability; pay only where the business actually needs it."*

Diagnosis: Consistency model is a per-domain choice, not a global database setting. Pick strong where invariants are financial or safety-critical; pick eventual where staleness is invisible to users.

Fix options: - For wallet: single-region single-writer Postgres / Spanner. Synchronous replication. SERIALIZABLE for the critical writes. - For follower count: write to primary, propagate asynchronously to replicas; serve reads from replicas. Tolerate brief staleness. - Per-feature decision: same database can

host both, with isolation level chosen per transaction. - Document the consistency contract in the API spec so consumers know what to expect.

What NOT to say: - "Always strong consistency for safety" — gives up scale and latency unnecessarily.
- "Always eventual for performance" — leads to data corruption in financial or safety-critical paths.

Common follow-up: "For an Indian UPI payment, which consistency model does NPCI require, and why?"

Category 7, Part A — Data Quality: Duplicate Investigations

There is one question that every product company, analytics team and data role in India eventually asks you — and it is rarely written down in a job description. It goes something like this: "*I see duplicates in this report. Find the cause.*" No schema is shared upfront. The query may or may not be shown. Sometimes you only get the output and a Slack screenshot from a panicking founder. The interviewer is not testing whether you know window functions; they are testing whether you can think like a data engineer — form a hypothesis, narrow the search space, isolate the offending row, and propose a fix that survives the next deployment. The 25 scenarios below cover the most common duplicate investigations you will face: join multipliers, migration leftovers, at-least-once delivery, case sensitivity, encoding mismatches, ETL race conditions, materialised view drift, and a few sneaky ones that have ruined many a Monday standup. Work through them out loud — the interview is as much about your narration as it is about the final SQL.

Scenario401 · Daily revenue report doubled overnight

Tags: Difficulty: Easy · Asked at: Razorpay, Meesho, CRED · Dialect: vendor-agnostic

The scenario:

You run a daily revenue report for a D2C brand. Yesterday it showed roughly ₹12 lakh, today it shows ₹24 lakh — but the founder swears nothing changed on the marketing side. The report query is below.

```
SELECT DATE(o.created_at) AS order_date,
       SUM(o.total_amount) AS revenue
FROM orders o
JOIN order_items oi ON oi.order_id = o.id
WHERE DATE(o.created_at) = CURRENT_DATE - INTERVAL '1 day'
GROUP BY DATE(o.created_at);
```

What's wrong: Revenue is roughly doubled compared to the previous day, yet no surge in traffic, ad spend or AOV is visible elsewhere.

Thinking out loud: 1. "Anything that suddenly doubles points to a join multiplier, not a real revenue jump." 2. " `orders` is being joined to `order_items`, but the SUM is on `o.total_amount` — an order-level column. Each order with two line items will count twice." 3. "Let me check the average number of line items per order — if it averages ~2 today, that explains the 2× number perfectly." 4. "Confirmed: most orders today have 2 SKUs. The join is fan-out, the SUM is at the wrong grain."

The diagnosis: Classic one-to-many join multiplier. `orders.total_amount` was summed at the line-item grain, so every additional line item duplicated the order total.

The fix:

```
SELECT DATE(created_at) AS order_date,  
       SUM(total_amount) AS revenue  
FROM orders  
WHERE DATE(created_at) = CURRENT_DATE - INTERVAL '1 day'  
GROUP BY DATE(created_at);
```

If the join was genuinely needed (to filter by SKU, say), aggregate first:

```
SELECT DATE(o.created_at), SUM(o.total_amount)  
FROM orders o  
WHERE o.id IN (SELECT DISTINCT order_id FROM order_items WHERE sku_category = 'apparel')  
AND DATE(o.created_at) = CURRENT_DATE - INTERVAL '1 day'  
GROUP BY DATE(o.created_at);
```

Prevention going forward: - Always check the grain of each column before SUM. A simple comment above the query: `-- grain: one row per order_item` saves hours. - Add a CI check that flags any query summing an order-level column over a join to a child table.

What NOT to say: - "Maybe yesterday's number was wrong." Anchoring on the wrong day is the most common mistake. The doubling pattern is the signal.

Common follow-up: "What if some orders had 1 item and some had 3 — would the revenue still be exactly doubled?"

Scenario402 · Duplicate users after a migration script

Tags: Difficulty: Easy · Asked at: Cult.fit, Urban Company, Nykaa · Dialect: Postgres

The scenario:

A growth analyst pings you: "Our `users` table count went from 4.2 million to 4.8 million overnight, but signup events only show 3,000 new users yesterday." A migration was run last night to backfill phone numbers from a legacy CRM into the main `users` table.

```
SELECT COUNT(*) FROM users;           -- 4,800,000
SELECT COUNT(DISTINCT email) FROM users; -- 4,203,000
```

What's wrong: 600K extra rows that aren't accounted for by new signups. The distinct email count matches the pre-migration count almost exactly.

Thinking out loud: 1. "The `DISTINCT` email count is unchanged, but row count jumped. That means the migration inserted instead of updated." 2. "Let me find a sample email with multiple rows." 3. "Yes — most duplicates have the same email but different `created_at` (one row from the original signup, one from last night's migration)." 4. "The migration script must have been written as `INSERT` when it should have been `UPDATE ... WHERE email = ...` or an `INSERT ... ON CONFLICT DO UPDATE`."

The diagnosis: The backfill script used `INSERT INTO users (...)` with no conflict handling, on a table where `email` had no unique constraint.

The fix: First find the dupes:

```
SELECT email, COUNT(*) AS row_count
FROM users
GROUP BY email
HAVING COUNT(*) > 1
ORDER BY row_count DESC
LIMIT 100;
```

Then dedupe, keeping the oldest row (the real signup) and merging the phone number forward:

```
WITH ranked AS (
  SELECT id, email,
         ROW_NUMBER() OVER (PARTITION BY email ORDER BY created_at ASC) AS rn
  FROM users
)
DELETE FROM users WHERE id IN (SELECT id FROM ranked WHERE rn > 1);
```

Prevention going forward: - Add `UNIQUE (email)` constraint so a buggy script fails loudly instead of silently doubling the table. - Migration scripts must be run in a transaction with a pre-check on row count delta.

What NOT to say: - "Just truncate and re-import." You will lose user IDs that downstream tables reference.

Common follow-up: "What if a real user has two genuinely different accounts on the same email — how do you decide which to keep?"

Scenario403 · Order count 2x from joining line items

Tags: Difficulty: Easy · Asked at: Flipkart, BigBasket, Zepto · Dialect: vendor-agnostic

The scenario:

Your weekly business review deck shows "12,400 orders this week" but the ops team's internal dashboard says 6,200. Both teams pull from the same warehouse. Your query:

```
SELECT COUNT(*) AS order_count
FROM orders o
JOIN order_items oi ON oi.order_id = o.id
WHERE o.created_at >= CURRENT_DATE - INTERVAL '7 days';
```

What's wrong: You're counting rows after a one-to-many join, so each order contributes once per line item.

Thinking out loud: 1. " `COUNT(*)` after a join to a child table will always inflate." 2. "Ops is right — the true count is at the order grain." 3. "The fix is either `COUNT(DISTINCT o.id)` or remove the join entirely." 4. "If I needed the join for a filter (say, orders containing at least one apparel SKU), I'd still need `DISTINCT`."

The diagnosis: Counting at the wrong grain after a one-to-many join.

The fix:

```
SELECT COUNT(*) AS order_count
FROM orders
WHERE created_at >= CURRENT_DATE - INTERVAL '7 days';

-- OR if the join is necessary:
SELECT COUNT(DISTINCT o.id) AS order_count
FROM orders o
JOIN order_items oi ON oi.order_id = o.id
WHERE oi.category = 'apparel'
AND o.created_at >= CURRENT_DATE - INTERVAL '7 days';
```

Prevention going forward: - Establish a team convention: any `COUNT(*)` over a multi-table join must justify why `DISTINCT` is not needed in a comment.

What NOT to say: - "Both numbers are valid views." They are not — one is wrong. Do not validate a bug.

Common follow-up: "What's the performance cost of `COUNT(DISTINCT)` vs restructuring the query?"

Scenario404 · Duplicate webhook deliveries inflating GMV

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Juspay · Dialect: Postgres

The scenario:

Your payment service receives webhooks from a PSP. The GMV dashboard shows ₹47L for yesterday; finance reconciliation shows ₹42L. The webhook handler simply inserts every payload into `payment_events`.

```
SELECT SUM(amount) AS gmv
FROM payment_events
WHERE event_type = 'payment.captured'
      AND DATE(received_at) = CURRENT_DATE - INTERVAL '1 day';
```

What's wrong: Webhook providers offer at-least-once delivery, not exactly-once. Some payments captured once will arrive 2–5 times.

Thinking out loud: 1. "Webhooks are at-least-once. If the network burped or our 200 was slow, the PSP retries." 2. "Every payment carries a unique `payment_id` from the PSP — that's our idempotency key." 3. "Let me count distinct payment IDs vs rows." 4. "Yes — 18,200 rows, 16,400 distinct `payment_id`. About 11 percent duplicates."

The diagnosis: At-least-once webhook delivery with no dedupe on the consumer side.

The fix:

```
SELECT SUM(amount) AS gmv
FROM (
  SELECT DISTINCT ON (payment_id) payment_id, amount, received_at
  FROM payment_events
  WHERE event_type = 'payment.captured'
        AND DATE(received_at) = CURRENT_DATE - INTERVAL '1 day'
  ORDER BY payment_id, received_at ASC
) deduped;
```

Prevention going forward: - `UNIQUE (payment_id, event_type)` on `payment_events` so re-deliveries are rejected at write time. - Use `INSERT ... ON CONFLICT DO NOTHING` in the consumer.

What NOT to say: - "We should ask the PSP to stop retrying." The retry behaviour is correct; your handler must be idempotent.

Common follow-up: "What if the PSP changes the `payment_id` format mid-month — how does your dedupe survive?"

Scenario405 · Race condition on INSERT — missing UNIQUE constraint

Tags: Difficulty: Medium · Asked at: Swiggy, Zomato, Dunzo · Dialect: Postgres

The scenario:

Your signup endpoint does a `SELECT ... WHERE email = ?` then `INSERT` if not found. Most days it is fine. On a flash sale day, you find 84 rows where two users have the same email registered within 200ms of each other.

```
SELECT email, COUNT(*) FROM users
WHERE DATE(created_at) = '2026-05-10'
GROUP BY email HAVING COUNT(*) > 1;
```

What's wrong: Two concurrent requests both ran the `SELECT`, both got "not found", both inserted.

Thinking out loud: 1. "This is a classic check-then-act race. `SELECT` and `INSERT` are not atomic." 2. "Under low traffic the window is small enough to never trigger; under flash-sale traffic it bites." 3. "The application logic cannot fix this reliably — the database has to enforce uniqueness." 4. "Add a `UNIQUE` constraint, switch the `INSERT` to use `ON CONFLICT`, and the race becomes a no-op."

The diagnosis: TOCTOU (time-of-check vs time-of-use) race; uniqueness enforced in app code, not in the schema.

The fix:

```
ALTER TABLE users ADD CONSTRAINT users_email_unique UNIQUE (email);

INSERT INTO users (email, created_at, source)
VALUES ($1, NOW(), $2)
ON CONFLICT (email) DO NOTHING
RETURNING id;
```

Dedupe existing rows first using a ROW_NUMBER cleanup like Scenario2.

Prevention going forward: - All "logical primary keys" (email, phone, GSTIN, PAN) must have a UNIQUE constraint at schema level. - Application code never decides uniqueness.

What NOT to say: - "We'll add a Redis lock." Locks across services are notoriously hard to get right; database constraints are bulletproof.

Common follow-up: *"What if email is nullable — does UNIQUE still work?"*

Scenario406 · Same email registered twice — case sensitivity

Tags: Difficulty: Easy · Asked at: Cult.fit, Unacademy, BYJU'S · Dialect: MySQL

The scenario:

A support agent flags that "Rahul.Sharma@gmail.com" and "rahul.sharma@gmail.com" exist as two separate accounts. Your `users` table is on MySQL with the default case-insensitive collation, but the duplicates exist anyway.

```
SELECT id, email FROM users WHERE LOWER(email) = LOWER('rahul.sharma@gmail.com');  
-- returns 2 rows
```

What's wrong: Email is case-insensitive by RFC for the domain part and effectively for the local part with most providers. Treating it as case-sensitive lets duplicates in.

Thinking out loud: 1. "MySQL's default collation `utf8mb4_unicode_ci` is case-insensitive, but if the column was created with `utf8mb4_bin` collation, it becomes case-sensitive." 2. "Let me check the column collation." 3. "Confirmed — someone created the table with `_bin` collation, presumably copy-pasted from another schema." 4. "Fix is twofold: lower-case all emails on write, and change the collation."

The diagnosis: Column collation was binary, allowing two casings of the same email to coexist.

The fix:

```
-- Postgres equivalent: use CITEXT or LOWER() at write time.  
ALTER TABLE users  
  MODIFY COLUMN email VARCHAR(255) COLLATE utf8mb4_unicode_ci;  
  
-- Then normalise:  
UPDATE users SET email = LOWER(email);  
  
-- Then dedupe.
```

Prevention going forward: - Lower-case emails at the application layer before insert. - Add a CHECK constraint: `CHECK (email = LOWER(email))`.

What NOT to say: - "Gmail ignores dots and casing." It does, but you are not Gmail — you should not silently merge accounts on the back of that.

Common follow-up: *"How would you handle the dedupe — which account keeps the order history?"*

Scenario407 · Trailing whitespace creating "different" duplicates

Tags: Difficulty: Easy · Asked at: Meesho, Snapdeal, Paytm Mall · Dialect: vendor-agnostic

The scenario:

Your seller onboarding system shows two `seller_name = 'Mehta Traders'` rows but a `GROUP BY seller_name` shows three groups: `Mehta Traders`, `Mehta Traders` (trailing space), and `Mehta Traders` (two spaces).

```
SELECT seller_name, COUNT(*) FROM sellers
GROUP BY seller_name;
```

What's wrong: What looks like the same merchant is three different keys to the database. Reports group three ways.

Thinking out loud: 1. *"This usually comes from CSV uploads or an Excel paste where trailing spaces are invisible to the human eye."* 2. *"Let me confirm with LENGTH."* 3. *"Yes — lengths 13, 14, 15. Spaces."* 4. *"Trim on write, and backfill existing rows."*

The diagnosis: Source data had trailing whitespace that was preserved on insert.

The fix:

```
-- Find them:
SELECT seller_name, LENGTH(seller_name), COUNT(*)
FROM sellers
WHERE seller_name <> TRIM(seller_name);

-- Normalise:
UPDATE sellers SET seller_name = TRIM(seller_name);

-- Then dedupe with ROW_NUMBER.
```


Prevention going forward: - Application layer trims all free-text fields. - Database constraint: `CHECK (seller_name = TRIM(seller_name))`.

What NOT to say: - "It's just cosmetic." It is not — every downstream join on `seller_name` fails to match.

Common follow-up: "What if there's a non-breaking space (U+00A0) — does `TRIM` catch it?"

Scenario408 · Encoding difference — utf8 vs utf8mb4

Tags: Difficulty: Hard · Asked at: Razorpay, Khatabook, Vedantu · Dialect: MySQL

The scenario:

Two rows for the user `Pooja 🤔` exist in your `customers` table. One has the emoji rendered, one has `?` or `????`. They are treated as distinct in joins to `orders`, splitting the order history across two ghosts.

```
SELECT id, name FROM customers WHERE name LIKE 'Pooja%';
```

What's wrong: The `customers.name` column is `utf8` (3-byte), which silently mangles 4-byte characters like emoji into `?`. A later migration to `utf8mb4` preserved the emoji for new rows. Same user, different bytes, two rows after a merge.

Thinking out loud: 1. "`utf8` in MySQL is a historical misnomer — only handles up to 3 bytes. Emoji and many Asian scripts need 4." 2. "The two rows have visually different `name` values. One is `Pooja ????` , the other `Pooja 🤔`." 3. "Dedupe needs a canonical form — strip non-ASCII for matching, but keep the emoji version on the surviving row." 4. "Migrate the column to `utf8mb4` for the future."

The diagnosis: Column encoding mismatch destroyed identity. The dedupe key was the corrupted `name`.

The fix:

```
ALTER TABLE customers MODIFY name VARCHAR(255)
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

-- Identify dupes by phone (assuming phone is reliable):
SELECT phone, GROUP_CONCAT(id) AS ids, GROUP_CONCAT(name) AS names
FROM customers
GROUP BY phone HAVING COUNT(*) > 1;

-- Merge order history to the surviving id, then delete the ghost.
```

Prevention going forward: - All text columns are `utf8mb4` from day one. No exceptions. - Connection charset is `utf8mb4` end-to-end.

What NOT to say: - "Just strip emoji from input." You are not allowed to mutate user-supplied identifiers like display names.

Common follow-up: "What's the storage cost difference between `utf8` and `utf8mb4`?"

Scenario409 · Soft-deleted rows reappearing in count

Tags: Difficulty: Medium · Asked at: Freshworks, Zoho, Chargebee · Dialect: vendor-agnostic

The scenario:

The active user count from your `users` table doubled overnight. No backfill, no migration. A new junior dev shipped a refactor of the user-list query yesterday.

```
SELECT COUNT(*) FROM users WHERE is_active = TRUE;
```

What's wrong: Soft-deleted users had `is_active = TRUE` and `deleted_at IS NOT NULL`. The previous query also checked `deleted_at IS NULL`; the refactor dropped that filter.

Thinking out loud: 1. "A doubled count with no data change is almost always a missing filter." 2. "`users` uses soft delete — `deleted_at` timestamp. The flag `is_active` was a separate signal that did not get cleared on deletion." 3. "Check the diff: previous query had `AND deleted_at IS NULL`, current doesn't." 4. "Restore the filter; also align `is_active` with `deleted_at` so the two cannot diverge."

The diagnosis: Two competing notions of "active" — `is_active` flag and `deleted_at IS NULL` — only the latter was being maintained.

The fix:

```
SELECT COUNT(*) FROM users
WHERE is_active = TRUE AND deleted_at IS NULL;

-- Long-term: backfill is_active to reflect deletion state:
UPDATE users SET is_active = FALSE WHERE deleted_at IS NOT NULL;
```

Prevention going forward: - Create a view `active_users` that encapsulates the canonical filter. - Add a trigger or application-layer hook so soft delete also flips `is_active`.

What NOT to say: - "Just remove the `is_active` column." That breaks every downstream report that depends on it without a migration plan.

Common follow-up: *"What if a user re-activates their account after soft delete — how do you handle the resurrection?"*

Scenario410 · Duples from CROSS JOIN when ON condition forgotten

Tags: Difficulty: Easy · Asked at: Flipkart, Amazon India, Myntra · Dialect: vendor-agnostic

The scenario:

An analyst built a report joining `orders` and `users`. The output has 4 billion rows on a 200K-order day. The query crashed BigQuery.

```
SELECT o.id, u.name
FROM orders o, users u
WHERE o.created_at >= CURRENT_DATE - INTERVAL '1 day';
```

What's wrong: The old comma syntax with no `WHERE o.user_id = u.id` produces a Cartesian product. 200K orders × 20K users = 4 billion rows.

Thinking out loud: 1. "The comma syntax is implicit CROSS JOIN. Without a join predicate it is a full cartesian product." 2. "That explains the obscene row count." 3. "Modern syntax `JOIN ON` forces you to write the predicate, so this class of bug is much rarer." 4. "Fix is trivial — add the join condition. Long term, ban comma-join syntax in lints."

The diagnosis: Missing join predicate produced a Cartesian product.

The fix:

```
SELECT o.id, u.name
FROM orders o
JOIN users u ON u.id = o.user_id
WHERE o.created_at >= CURRENT_DATE - INTERVAL '1 day';
```

Prevention going forward: - Lint rule: ban `FROM a, b` style joins. - Code review: every JOIN must have an ON clause that connects to the previous tables.

What NOT to say: - "Let me add `LIMIT 100` ." That hides the bug; the report would silently show wrong sample data.

Common follow-up: *"How would BigQuery / Snowflake's query planner have warned you about this?"*

Scenario411 · Recursive CTE looping and inflating rows

Tags: Difficulty: Hard · Asked at: Microsoft India, Oracle, SAP Labs · Dialect: Postgres

The scenario:

An org-chart query that walks a `manager_id` hierarchy returns 47,000 rows for a company of 1,200 employees. The query has been adding rows on every refresh.

```
WITH RECURSIVE org AS (
  SELECT id, name, manager_id, 1 AS depth FROM employees WHERE manager_id IS NULL
  UNION ALL
  SELECT e.id, e.name, e.manager_id, o.depth + 1
  FROM employees e JOIN org o ON e.manager_id = o.id
)
SELECT * FROM org;
```

What's wrong: Someone introduced a circular `manager_id` (A reports to B, B reports to A) by mistake during an org restructure. The recursive CTE walks the cycle until the recursion depth limit, then stops — but the row count balloons.

Thinking out loud: 1. "Recursive CTEs blow up if the graph has cycles. Postgres has no cycle detection unless you ask for it." 2. "Use `CYCLE` clause (Postgres 14+) or maintain a visited set manually." 3. "First, find the cycle. Then break it. Then fix the query." 4. "Add a constraint that prevents a cycle going forward, ideally via a check or a trigger."

The diagnosis: Cycle in the hierarchy + recursive CTE with no cycle detection.

The fix:

```
-- Find cycles:
WITH RECURSIVE org AS (
  SELECT id, name, manager_id, ARRAY[id] AS path FROM employees WHERE manager_id IS NULL
  UNION ALL
  SELECT e.id, e.name, e.manager_id, o.path || e.id
  FROM employees e JOIN org o ON e.manager_id = o.id
  WHERE e.id <> ALL(o.path) -- cycle guard
)
SELECT * FROM org;

-- Fix the offending row:
UPDATE employees SET manager_id = NULL WHERE id = <cycle_root>;
```

Prevention going forward: - Trigger that rejects updates which would create a cycle. - All recursive CTEs in the codebase carry an explicit cycle guard.

What NOT to say: - "Set `max_recursive_depth` lower." That hides the bug.

Common follow-up: "How would you detect cycles in a graph with 10 million edges efficiently?"

Scenario412 · UNION silently deduping when UNION ALL was needed

Tags: Difficulty: Medium · Asked at: TCS, Infosys BPM, Wipro · Dialect: vendor-agnostic

The scenario:

Finance needs a list of every transaction (deposits + withdrawals) for a customer. The report shows 412 rows but the source tables together hold 487 rows. Some transactions are vanishing.

```
SELECT customer_id, amount FROM deposits WHERE customer_id = 42
UNION
SELECT customer_id, amount FROM withdrawals WHERE customer_id = 42;
```

What's wrong: `UNION` removes duplicate rows. Two deposits of the same amount on the same customer become one row.

Thinking out loud: 1. "UNION dedupes, UNION ALL doesn't. Finance needs every transaction." 2. "Two ₹1,000 deposits look identical in this projection, so UNION collapses them." 3. "The fix is UNION ALL, plus including a key that disambiguates." 4. "This is a recurring source of bugs because UNION is what most beginners reach for first."

The diagnosis: `UNION` deduplicated rows that finance considered distinct transactions.

The fix:

```
SELECT id, customer_id, amount, 'deposit' AS type, created_at FROM deposits WHERE customer_id = 42
UNION ALL
SELECT id, customer_id, amount, 'withdrawal' AS type, created_at FROM withdrawals WHERE customer_id = 42
ORDER BY created_at;
```

Prevention going forward: - Team convention: default to `UNION ALL`. Only use `UNION` when dedupe is explicitly required and documented.

What NOT to say: - "UNION is more correct." There is no "more correct" — it depends on intent. Default to preserving rows.

Common follow-up: *"What's the performance cost of UNION vs UNION ALL?"*

Scenario413 · Duples from re-running an upsert without conflict handling

Tags: Difficulty: Medium · Asked at: Razorpay, Khatabook, Pine Labs · Dialect: Postgres

The scenario:

A nightly ETL inserts daily aggregates into `daily_kpis`. The pipeline failed at 3 a.m. and was re-run at 9 a.m. Now the dashboard shows yesterday's revenue doubled.

```
INSERT INTO daily_kpis (kpi_date, metric, value)
SELECT CURRENT_DATE - 1, 'gross_revenue', SUM(amount)
FROM orders WHERE DATE(created_at) = CURRENT_DATE - 1;
```

What's wrong: The script inserts unconditionally. A re-run produces a second row for the same date and metric.

Thinking out loud: 1. "This is an idempotency bug. Re-running the script should be a no-op or an overwrite, not an insert." 2. "Add a `UNIQUE` constraint on `(kpi_date, metric)` so the second insert fails or overwrites." 3. "Use `ON CONFLICT DO UPDATE` for the overwrite semantic." 4. "Clean up the existing duples first."

The diagnosis: ETL script was not idempotent — no upsert logic.

The fix:

```
ALTER TABLE daily_kpis ADD CONSTRAINT daily_kpis_uk UNIQUE (kpi_date, metric);

INSERT INTO daily_kpis (kpi_date, metric, value)
SELECT CURRENT_DATE - 1, 'gross_revenue', SUM(amount)
FROM orders WHERE DATE(created_at) = CURRENT_DATE - 1
ON CONFLICT (kpi_date, metric) DO UPDATE SET value = EXCLUDED.value;
```

Prevention going forward: - Every nightly aggregate write goes through an upsert or a DELETE-then-INSERT in a transaction. - ETL framework enforces idempotency via primary key.

What NOT to say: - "We'll just remember not to re-run." Operational rules that depend on memory always fail.

Common follow-up: "What if the underlying `orders` row count changed between runs — which value wins?"

Scenario414 · Customer 360 view double-counting orders + refunds

Tags: Difficulty: Medium · Asked at: Nykaa, Lenskart, Pepperfry · Dialect: vendor-agnostic

The scenario:

The "lifetime value" column in your CRM shows ₹84,000 for a customer who has actually spent ₹52,000. The LTV query joins `orders` to `refunds` to compute net revenue.

```
SELECT o.customer_id,
       SUM(o.total_amount) - SUM(r.refund_amount) AS ltv
FROM orders o
LEFT JOIN refunds r ON r.order_id = o.id
GROUP BY o.customer_id;
```

What's wrong: If an order has 2 partial refunds, the join produces 2 rows for that order, doubling `o.total_amount` in the SUM.

Thinking out loud: 1. "Same fan-out problem as Scenario1, but with the twist of two child tables being aggregated together." 2. "Solution: pre-aggregate each side before joining." 3. "This pattern shows up everywhere — order vs payments, order vs shipments, order vs returns."

The diagnosis: Fan-out join inflated `total_amount`.

The fix:

```
SELECT o.customer_id,  
       COALESCE(SUM(o.total_amount), 0) - COALESCE(SUM(r.refund_total), 0) AS ltv  
FROM orders o  
LEFT JOIN (  
  SELECT order_id, SUM(refund_amount) AS refund_total  
  FROM refunds  
  GROUP BY order_id  
) r ON r.order_id = o.id  
GROUP BY o.customer_id;
```

Prevention going forward: - Pre-aggregate every child table to the parent grain before joining. - Use a "one-grain-per-CTE" convention.

What NOT to say: - "Use DISTINCT inside SUM" — `SUM(DISTINCT o.total_amount)` does not do what people think; two orders with the same amount would be collapsed.

Common follow-up: "What if refunds happen weeks after the order — how do you snapshot LTV?"

Scenario415 · Funnel dupes from client-side event firing twice

Tags: Difficulty: Medium · Asked at: Swiggy, Zomato, Cure.fit · Dialect: vendor-agnostic

The scenario:

The conversion funnel shows checkout-start to checkout-complete at 84 percent — implausibly high. Product manager suspects something is off. The funnel query counts distinct user events.

```
SELECT  
  COUNT(DISTINCT CASE WHEN event = 'checkout_start' THEN user_id END) AS started,  
  COUNT(DISTINCT CASE WHEN event = 'checkout_complete' THEN user_id END) AS completed  
FROM events  
WHERE event_date = CURRENT_DATE - 1;
```

What's wrong: The web frontend was shipped two weeks ago with a `useEffect` that fires `checkout_start` on every re-render. So the same user fires it 5–20 times. Looks like more starts; but actually the count is bloated identically on both sides? No — the bug here is the other direction: `checkout_complete` fires once, `checkout_start` fires repeatedly, so the ratio looks good. The inverse bug exists too.

Wait — re-reading: `COUNT(DISTINCT user_id)` collapses the dupes back to one per user. So a re-render bug actually does not inflate this query. The real problem: a `checkout_complete` event also fires from the "thank-you" page on every reload, but you are counting distinct users — so the actual

issue is users reloading after success and being counted only once, masking the bug entirely. Conversion is 84 percent of users who started, but most "starts" are accidental.

Thinking out loud: 1. "The numerator looks too high. Either the denominator is undercounted or the numerator is over." 2. "Inspect raw event counts before `DISTINCT` — `checkout_start` has 1.4M rows for 280K users. That's 5 fires per user." 3. "`DISTINCT` on `user_id` hides the re-fire issue but reveals a different one: starts include accidental fires from re-renders that never had real intent." 4. "Either filter to the first fire per user per session, or fix the client."

The diagnosis: Client-side event was firing on every re-render; `DISTINCT user_id` papered over the duplication but made the funnel look unrealistically efficient because trivial "starts" were counted as intent.

The fix:

```
-- Take first event per user per session:
WITH first_events AS (
  SELECT user_id, session_id, event,
         ROW_NUMBER() OVER (PARTITION BY user_id, session_id, event ORDER BY ts) AS rn
  FROM events WHERE event_date = CURRENT_DATE - 1
)
SELECT
  COUNT(DISTINCT CASE WHEN event = 'checkout_start' AND rn = 1 THEN user_id END),
  COUNT(DISTINCT CASE WHEN event = 'checkout_complete' AND rn = 1 THEN user_id END)
FROM first_events;
```

Prevention going forward: - Client SDK debounces events. Every event has a client-generated UUID for dedupe. - Analytics warehouse dedupes on (`user_id`, `session_id`, `event`, `client_uuid`).

What NOT to say: - "Just remove the `DISTINCT`." That makes the number even more wrong.

Common follow-up: "How would you instrument the client to distinguish true intent from re-render fires?"

Scenario416 · Daily snapshot table written twice for same day

Tags: Difficulty: Easy · Asked at: TCS, Capgemini, Cognizant · Dialect: vendor-agnostic

The scenario:

Your `daily_inventory_snapshot` table has 2,840,000 rows for May 14 and 1,420,000 rows for every other day in the month. The schema has no unique constraint on (`sku`, `snapshot_date`) .

```
SELECT snapshot_date, COUNT(*) FROM daily_inventory_snapshot
GROUP BY snapshot_date ORDER BY snapshot_date;
```

What's wrong: The snapshot job ran twice on May 14 — probably someone re-ran it manually after a 'failure' that had already succeeded.

Thinking out loud: 1. "Row count for one day is exactly double. The snapshot job ran twice." 2. "Each snapshot row should be unique per (sku, snapshot_date)." 3. "Delete one set of duplicates and add the constraint."

The diagnosis: Snapshot job re-run without idempotency.

The fix:

```
WITH ranked AS (
  SELECT ctid, sku, snapshot_date,
         ROW_NUMBER() OVER (PARTITION BY sku, snapshot_date ORDER BY ingested_at) AS rn
  FROM daily_inventory_snapshot WHERE snapshot_date = '2026-05-14'
)
DELETE FROM daily_inventory_snapshot
WHERE ctid IN (SELECT ctid FROM ranked WHERE rn > 1);

ALTER TABLE daily_inventory_snapshot ADD CONSTRAINT inv_snap_uk UNIQUE (sku, snapshot_date);
```

(`ctid` is Postgres; on MySQL use the primary key column.)

Prevention going forward: - Snapshot job uses `DELETE WHERE snapshot_date = ? THEN INSERT` inside a transaction. - UNIQUE constraint blocks the second insert.

What NOT to say: - "We can leave it — downstream reports take the first value." Downstream reports likely SUM, so the day is doubled.

Common follow-up: "How would you write the snapshot job so a partial failure can be safely retried?"

Scenario417 · Cron job ran twice in parallel

Tags: Difficulty: Medium · Asked at: Razorpay, Juspay, Pine Labs · Dialect: vendor-agnostic

The scenario:

The settlement report for May 18 contains every transaction twice. Cron logs show the job started at 02:00 and again at 02:01 because the previous day's run took 25 hours to finish.

```
SELECT settlement_id, COUNT(*) FROM settlements
WHERE settlement_date = '2026-05-18'
GROUP BY settlement_id HAVING COUNT(*) > 1;
```

What's wrong: Two instances of the same cron ran concurrently. Both finished, both wrote.

Thinking out loud: 1. *"This is a 'cron is not a job runner' problem. cron just runs the command; if a run overlaps with the next, both proceed."* 2. *"Use a job lock — distributed lock via Redis, advisory lock in Postgres, or a flag table."* 3. *"Long term, move to a real scheduler like Airflow / Temporal that knows about overlap."*

The diagnosis: Cron overlap; no mutual exclusion.

The fix: At the start of the job:

```
SELECT pg_try_advisory_lock(hashtext('settlement_job'));
-- if false, exit early
-- ... do work ...
SELECT pg_advisory_unlock(hashtext('settlement_job'));
```

Or use `flock` on a file at the OS level.

Prevention going forward: - All long-running batch jobs acquire an advisory lock first. - Migrate to Airflow / Temporal for first-class scheduling.

What NOT to say: - "Just run cron every 25 hours." That introduces drift.

Common follow-up: *"What if the job crashes after acquiring the lock — how do you ensure it gets released?"*

Scenario418 · ETL idempotency missing — partial reruns inflate

Tags: Difficulty: Hard · Asked at: Walmart Labs, Target India, Lowe's · Dialect: vendor-agnostic

The scenario:

Your ETL processes hourly shards from a Kafka source into a warehouse fact table. A shard failed midway — 60 percent ingested, then the process died. Operator re-ran the shard. Now 60 percent of that hour is doubled.

```
SELECT event_uuid, COUNT(*) FROM fact_events
WHERE event_hour = '2026-05-19 14:00:00'
GROUP BY event_uuid HAVING COUNT(*) > 1;
```

What's wrong: The ETL appends rows by `event_uuid`, but does not check whether the uuid is already present. A partial rerun doubles the partial slice.

Thinking out loud: 1. "Idempotency requires either (a) deterministic deletion + reinsert per shard, or (b) merge-on-uuid." 2. "For high-throughput inserts, the staging-then-merge pattern is standard." 3. "Fix the existing dupes; switch the load to merge on UUID."

The diagnosis: ETL was not idempotent. Partial failure plus retry inflated.

The fix:

```
-- Clean up:
WITH ranked AS (
  SELECT id, event_uuid,
         ROW_NUMBER() OVER (PARTITION BY event_uuid ORDER BY ingested_at) AS rn
  FROM fact_events WHERE event_hour = '2026-05-19 14:00:00'
)
DELETE FROM fact_events WHERE id IN (SELECT id FROM ranked WHERE rn > 1);

-- Going forward: stage into a temp table, then MERGE.
MERGE INTO fact_events AS target
USING staging AS src
  ON target.event_uuid = src.event_uuid
 WHEN NOT MATCHED THEN INSERT (...) VALUES (...);
```

Prevention going forward: - Every fact table has a natural deduplication key (`event_uuid`). - Every ETL load is `MERGE` or `INSERT ... ON CONFLICT DO NOTHING`, never plain `INSERT`. - Shard boundaries are documented so retries are bounded.

What NOT to say: - "We'll add a `LIMIT 1` somewhere." That has no relevance to idempotency.

Common follow-up: "How do you handle the case where the same event_uuid arrives with two different payloads?"

Scenario419 · Materialised view + source table both queried by report

Tags: Difficulty: Hard · Asked at: Microsoft India, Adobe, Salesforce India · Dialect: Postgres

The scenario:

A dashboard joins `mv_daily_sales` (a materialised view aggregating from `orders`) with `orders` itself to "enrich" with recent orders. Today's total is wildly inflated because some recent orders appear in both.

```
SELECT region, SUM(revenue) AS total
FROM (
  SELECT region, total_amount AS revenue FROM orders WHERE created_at >= CURRENT_DATE
  UNION ALL
  SELECT region, gross AS revenue FROM mv_daily_sales WHERE sales_date = CURRENT_DATE
) x GROUP BY region;
```

What's wrong: The mat view is refreshed every hour. Orders placed in the last hour appear once in `orders` and (after refresh) again in `mv_daily_sales`.

Thinking out loud: 1. "Two sources of truth for the same window: live table + materialised aggregate. Overlap is inevitable." 2. "Standard fix: live-tail from `orders` for the time window after the last MV refresh, historical from the MV." 3. "Track the MV's freshness timestamp explicitly."

The diagnosis: Time-window overlap between materialised view and live source.

The fix:

```
WITH mv_max AS (
  SELECT MAX(refreshed_at) AS as_of FROM mv_refresh_log WHERE mv_name = 'mv_daily_sales'
)
SELECT region, SUM(revenue) AS total FROM (
  SELECT region, total_amount AS revenue FROM orders, mv_max
  WHERE created_at > mv_max.as_of
  UNION ALL
  SELECT region, gross AS revenue FROM mv_daily_sales WHERE sales_date = CURRENT_DATE
) x GROUP BY region;
```

Prevention going forward: - Maintain a refresh log table. Every report that mixes MV + live uses it. - Or refresh more frequently and accept the slight latency, querying only the MV.

What NOT to say: - "Drop the materialised view." MVs exist for a reason; the problem is how the report mixes sources.

Common follow-up: "What if the MV refresh itself takes 20 minutes — how do you reason about the overlap window?"

Scenario420 · Backfill script overlap with live ingest

Tags: Difficulty: Hard · Asked at: PhonePe, Razorpay, Cred · Dialect: Postgres

The scenario:

Engineering ran a backfill to reingest two weeks of payment events from a CSV export, because a downstream consumer needed them. During the same window, live webhook ingest was still running. Now there are dupes for every payment that landed during the backfill.

```
SELECT payment_id, COUNT(*) FROM payment_events
WHERE event_date BETWEEN '2026-05-05' AND '2026-05-19'
GROUP BY payment_id HAVING COUNT(*) > 1;
```

What's wrong: The backfill assumed it was the sole writer; live ingest was racing it.

Thinking out loud: 1. "Backfills must either (a) pause live ingest, (b) write to a separate staging schema and switch over, or (c) use upsert keyed by payment_id so the second write is a no-op." 2. "None of those was done; both writers raced, and there's no unique constraint."

The diagnosis: Backfill + live ingest racing on the same table without unique key or coordination.

The fix:

```
-- Clean up:
WITH ranked AS (
  SELECT ctid, payment_id,
         ROW_NUMBER() OVER (PARTITION BY payment_id ORDER BY received_at) AS rn
  FROM payment_events
  WHERE event_date BETWEEN '2026-05-05' AND '2026-05-19'
)
DELETE FROM payment_events
WHERE ctid IN (SELECT ctid FROM ranked WHERE rn > 1);

-- Future:
ALTER TABLE payment_events ADD CONSTRAINT payment_uk UNIQUE (payment_id);
```

Prevention going forward: - Every event table has a UNIQUE constraint on the natural key. - Backfills run with `ON CONFLICT DO NOTHING`. - Runbook for backfills requires either a pause-and-resume of live ingest, or a documented overlap handling step.

What NOT to say: - "It's a one-off, we'll dedupe and move on." If it happened once, the conditions for it to happen again still exist.

Common follow-up: "How would you verify the backfill produced the right data even after dedupe?"

Scenario421 · Bridge table missing a primary key

Tags: Difficulty: Medium · Asked at: Infosys, Wipro, TCS · Dialect: vendor-agnostic

The scenario:

A `user_roles` bridge table maps users to roles. Rohit Shetty shows three "admin" rows. Permissions audit fails because the role count is off.

```
SELECT user_id, role_id, COUNT(*)
FROM user_roles
GROUP BY user_id, role_id HAVING COUNT(*) > 1;
```

What's wrong: The bridge table has no primary key, just two FK columns. Re-running the role-assignment endpoint inserts a second row.

Thinking out loud: 1. "Bridge tables must have a composite PK on the two FK columns. Otherwise duplicates are inevitable." 2. "Clean up, then add the constraint."

The diagnosis: Missing composite primary key on bridge table.

The fix:

```
WITH ranked AS (
  SELECT ctid, user_id, role_id,
         ROW_NUMBER() OVER (PARTITION BY user_id, role_id ORDER BY assigned_at) AS rn
  FROM user_roles
)
DELETE FROM user_roles WHERE ctid IN (SELECT ctid FROM ranked WHERE rn > 1);

ALTER TABLE user_roles ADD PRIMARY KEY (user_id, role_id);
```

Prevention going forward: - All bridge tables in the schema audit must have composite PKs. - Migration linter rejects new bridge tables without one.

What NOT to say: - "Add a surrogate `id` PK." That allows duplicates on (user_id, role_id) — the original bug.

Common follow-up: "What if `user_roles` also has an `assigned_at` and you genuinely want to track multiple assignments — how do you model that?"

Scenario422 · Many-to-many join inflation in a wide table

Tags: Difficulty: Hard · Asked at: Flipkart, Myntra, Ajo · Dialect: vendor-agnostic

The scenario:

A "customer summary" view joins `customers` → `addresses` → `orders` → `tags`. Each customer has multiple addresses, multiple orders, and multiple tags. The view returns 3.4 million rows for 12,000 customers.

```
SELECT c.id, c.name, a.city, o.total_amount, t.tag
FROM customers c
LEFT JOIN addresses a ON a.customer_id = c.id
LEFT JOIN orders o ON o.customer_id = c.id
LEFT JOIN tags t ON t.customer_id = c.id;
```

What's wrong: A customer with 3 addresses × 12 orders × 4 tags produces 144 rows. The wide join multiplies all child tables together.

Thinking out loud: 1. "Three independent child tables fan out the Cartesian product." 2. "You almost never want this. Either aggregate each child before joining, or use lateral subqueries to get a single 'most recent' value." 3. "Or model it as JSON aggregation in the SELECT."

The diagnosis: Multiple independent child joins produced a multiplicative explosion.

The fix:

```
SELECT c.id, c.name,
       (SELECT array_agg(city) FROM addresses WHERE customer_id = c.id) AS cities,
       (SELECT SUM(total_amount) FROM orders WHERE customer_id = c.id) AS lifetime_value,
       (SELECT array_agg(tag) FROM tags WHERE customer_id = c.id) AS tags
FROM customers c;
```

Or aggregate-then-join each side.

Prevention going forward: - "Wide" customer views are never built with raw multi-joins. Use CTE-per-child-aggregate or JSON aggregation.

What NOT to say: - "Add DISTINCT to the SELECT." `DISTINCT` over a wide projection is wildly expensive and still wrong if any column is an aggregate.

Common follow-up: "How would you build this same view in BigQuery using `STRUCT` and `ARRAY_AGG`?"

Scenario423 · A/B test exposure counted in control + treatment via cookie collision

Tags: Difficulty: Hard · Asked at: Swiggy, Zomato, Meesho · Dialect: vendor-agnostic

The scenario:

Your A/B test results show 12 percent overlap — users appearing in both control and treatment. Your assignment is supposed to be deterministic via cookie hash, but the overlap is real.

```
SELECT user_id, COUNT(DISTINCT variant) AS variants
FROM ab_exposures
WHERE experiment_id = 'checkout_v3'
GROUP BY user_id HAVING COUNT(DISTINCT variant) > 1;
```

What's wrong: Users hit the experiment from web and from app — different cookie / device ID — and got bucketed independently on each.

Thinking out loud: 1. "A/B assignment must be keyed on a stable identity (logged-in user_id), not on a per-device cookie." 2. "For logged-out users, you accept some bucket drift; for logged-in users it's a bug." 3. "Mitigation: drop those users from analysis (they pollute both arms), and fix the assignment logic to key on user_id when known."

The diagnosis: Bucketing keyed on cookie rather than on stable user identity.

The fix:

```
-- Exclude polluted users from the analysis:
WITH polluted AS (
  SELECT user_id FROM ab_exposures
  WHERE experiment_id = 'checkout_v3'
  GROUP BY user_id HAVING COUNT(DISTINCT variant) > 1
)
SELECT variant, COUNT(DISTINCT user_id) AS exposed, SUM(converted::int) AS converted
FROM ab_exposures
WHERE experiment_id = 'checkout_v3'
  AND user_id NOT IN (SELECT user_id FROM polluted)
GROUP BY variant;
```

Prevention going forward: - A/B assignment service prefers `user_id` over cookie when available; falls back to cookie only for logged-out traffic. - Exposure events are deduplicated per (experiment_id, user_id, variant) at the warehouse layer.

What NOT to say: - "Just keep both buckets and weight." That biases the test in unpredictable ways.

Common follow-up: "How would you have caught this before launching the experiment?"

Scenario424 · Affiliate click dupes from refresh and back-button

Tags: Difficulty: Medium · Asked at: Cuelinks, INRDeals, EarnKaro · Dialect: vendor-agnostic

The scenario:

Affiliate payouts are calculated per unique click. Your top affiliate is showing 4.2x the clicks of last month with no campaign change. Browsing patterns reveal users hitting refresh on the redirect page.

```
SELECT affiliate_id, COUNT(*) AS clicks
FROM affiliate_clicks
WHERE click_date = CURRENT_DATE - 1
GROUP BY affiliate_id
ORDER BY clicks DESC;
```

What's wrong: The redirect endpoint logs a click on every hit. Browser back-button + refresh on the destination produces phantom clicks.

Thinking out loud: 1. "Affiliate clicks need a session-level dedupe, not raw-hit counting." 2. "The dedupe key is usually (affiliate_id, visitor_id, day) — at most one click per visitor per affiliate per day."

The diagnosis: Raw hit counting without dedupe on visitor + day.

The fix:

```
SELECT affiliate_id, COUNT(DISTINCT visitor_id) AS unique_clicks
FROM affiliate_clicks
WHERE click_date = CURRENT_DATE - 1
GROUP BY affiliate_id;
```

Better: enforce at write time using `INSERT ... ON CONFLICT DO NOTHING` with a UNIQUE on (affiliate_id, visitor_id, click_date) .

Prevention going forward: - Add the UNIQUE constraint. - Define the click-counting rule formally in the affiliate T&C so payouts are unambiguous.

What NOT to say: - "Pay them anyway, it's good for the relationship." You will be subsidising bot-like behaviour and the next affiliate will exploit it.

Common follow-up: "How do you handle a legitimate user who clicks twice on two different days?"

Scenario425 · Same address marked primary for multiple users

Tags: Difficulty: Medium · Asked at: BigBasket, Grofers, Zepto · Dialect: vendor-agnostic

The scenario:

A "household" feature shows that 47 customers share the same primary address. The team thinks it is a data quality smell — but is it noise, families, or shared sellers?

```
SELECT address_hash, COUNT(DISTINCT user_id) AS users
FROM user_addresses
WHERE is_primary = TRUE
GROUP BY address_hash
HAVING COUNT(DISTINCT user_id) > 5
ORDER BY users DESC;
```

What's wrong: Hard to tell without context. The query is correct — but the *interpretation* requires you to think like a data engineer.

Thinking out loud: 1. "Five users on one address could be a joint family (legitimate) or a hostel (legitimate) or fraud rings (not legitimate) or normalisation noise (street address parsed two ways)." 2. "First, check whether the addresses are actually the same string or just the same hash — bug in the hash function would collapse different addresses." 3. "Then segment by household size distribution: top-1 percent might be fraud; the middle bulk is families."

The diagnosis: This is not a single bug — it is a data quality investigation. The "duplicate" is real but the meaning depends on segment. Most production data has at least 3 distinct sources of address dupes: fraud, families, and parsing noise.

The fix:

```
SELECT address_hash,
       address_line1,
       pincode,
       COUNT(DISTINCT user_id) AS users,
       MIN(created_at) AS first_user_seen,
       MAX(created_at) AS last_user_seen
FROM user_addresses
WHERE is_primary = TRUE
GROUP BY address_hash, address_line1, pincode
HAVING COUNT(DISTINCT user_id) > 5
ORDER BY users DESC;

-- Then sample 20 rows and eyeball.
```

Prevention going forward: - Address normalisation library (split, lower, strip, geocode) before hashing. - Periodic audit query that flags "high-occupancy" addresses for review.

What NOT to say: - "It's a bug, dedupe them." You may delete a legitimate joint-family address book.

Common follow-up: *"How would you design an address-deduplication system that distinguishes 'same building' from 'same household'?"*

Category 7, Part B — Data Quality: Missing Rows & Counts Mismatch

"The dashboard says 9,87,432 orders, the app says 9,91,108, and finance says 9,89,200 — which one is right and who do I have to call?" That is the kind of message that lands in your DM on a Monday morning when nothing technically broke but every system has a slightly different version of reality. Counts not matching across systems and "where did these rows go" investigations are the most common, most underrated kind of data work. They look like trivia. They are actually how a data person earns their salt — because the answer is almost never "the database is wrong"; it is almost always a join, a filter, a timezone, a partition, a replication lag, or a definition drift that someone treated as obvious. The 25 scenarios below walk through the detective playbooks for these — the questions to ask, the diagnostic queries to run, the prevention to put in place, and the things not to say in the meeting before you have evidence.

Scenario426 · DB Count vs GA4 Count Mismatch — 12% Gap

Tags: `intermediate` `dq-investigation` `analytics-tools` `attribution` `vendor-agnostic`

Business context: You run an Indian D2C skincare brand on Shopify. The marketing lead pings you on Slack — "GA4 is showing 8,742 purchases yesterday, but our DB is showing 9,931. Which one is correct? We are about to scale ad spend based on GA4 ROAS." The 12% gap is too big to ignore.

SQL:

```
SELECT COUNT(*) AS db_orders, SUM(total_amount) AS db_revenue
FROM orders
WHERE order_status = 'paid'
      AND created_at >= '2026-05-25' AND created_at < '2026-05-26';
```

Suspicious output: `db_orders = 9931` , `db_revenue = 47,82,650` . GA4 reports `purchases = 8742` . Discrepancy of 1,189 orders (~12%).

Thinking out loud: 1. First, are we comparing the same definition? GA4 counts **purchase** events fired client-side; DB counts paid orders server-side. These are structurally different sources, so some gap is expected — but 12% is high. 2. Common reasons GA4 under-counts: ad-blockers (10-15% in India is normal), users who close the browser before the thank-you page fires, single-page checkout glitches, consent-banner rejections, and Safari ITP. 3. Common reasons DB over-counts vs GA4: server-to-server reorders (COD confirmations, retry payments), backend imports, manual orders entered by the support team, B2B/wholesale orders. 4. The right diagnostic is to bucket the gap — how many orders had no GA4 client_id, came from COD, came from manual channel, came from mobile-app (which fires different events).

Diagnosis: Of the 1,189 missing orders — 612 were COD (GA4 purchase fires only on thank-you page, which COD users often skip), 287 had ad-blocker (no client_id captured), 198 came from native mobile app (separate GA4 property not aggregated), 92 were manual orders by support.

Fix or diagnostic query:

```
SELECT
  CASE WHEN payment_method = 'cod' THEN 'cod'
        WHEN ga4_client_id IS NULL THEN 'no_client_id'
        WHEN source_channel = 'mobile_app' THEN 'mobile_app'
        WHEN created_by_user_id IS NOT NULL THEN 'manual' ELSE 'web' END AS bucket,
  COUNT(*) AS order_count
FROM orders
WHERE order_status = 'paid' AND created_at::date = '2026-05-25'
GROUP BY 1;
```

Prevention: Send server-side **purchase** events via Measurement Protocol with the order_id as deduplication key. Stamp **ga4_client_id** on the order record at checkout. Treat DB as source of truth for revenue, GA4 only for attribution.

NOT to say: "GA4 is wrong, ignore it." Both are right for what they measure — GA4 measures client-side behaviour, DB measures money received.

Follow-up: "How would you bucket the gap by acquisition channel to see if one channel has a worse data loss rate than another?"

Scenario427 · App Reports 1,000 Orders, DB Shows 950 — Find the 50

Tags: **intermediate** **dq-investigation** **pending-states** **vendor-agnostic**

Business context: A food-delivery startup in Bengaluru — the operations dashboard inside their internal app says 1,000 orders placed today, but the analytics DB count says 950. The CEO asks in standup: "Where are the 50 orders?"

SQL:

```
SELECT COUNT(*) FROM analytics_orders WHERE order_date = CURRENT_DATE;
-- returns 950
SELECT COUNT(*) FROM ops.orders_live WHERE order_date = CURRENT_DATE;
-- returns 1000
```

Suspicious output: Difference of exactly 50 orders. The fact that the gap is round-ish is suspicious — possibly a single batch or a single failure.

Thinking out loud: 1. The ops app is reading from the live OLTP system; the analytics DB is downstream after an ETL. So the gap could be (a) ETL hasn't finished yet, (b) ETL filter is excluding some orders, (c) some orders are in a state that ETL skips. 2. The first thing I'd check is the ETL last-run timestamp. If it ran at 22:00 yesterday and we're querying at 14:00 today, the day isn't "done" yet. 3. Second, what status are those 50? Pending, cancelled, failed? Many ETLs intentionally skip non-terminal states. 4. Third, do they share a common feature — same restaurant, same payment method, same city? A correlated cluster usually points to a single bug.

Diagnosis: ETL filter was `WHERE order_status IN ('placed','delivered','cancelled')`. The 50 missing orders were in `order_status = 'pending_payment'` — a state added to the source schema last week but never added to the ETL filter. The ETL is technically running fine, just blind to the new state.

Fix or diagnostic query:

```
SELECT order_status, COUNT(*)
FROM ops.orders_live
WHERE order_date = CURRENT_DATE
      AND order_id NOT IN (SELECT order_id FROM analytics_orders WHERE order_date = CURRENT_DATE)
GROUP BY order_status;
```

Prevention: ETLs should ingest all rows and let the BI layer filter — never apply business filters at ingestion. Add a CI check that fails if a new status appears in source but isn't in the analytics dimension table.

NOT to say: "It's probably a sync delay, let's check tomorrow." That defers the diagnosis; the gap might be persistent and not delay-related.

Follow-up: "How would you build a reconciliation report that runs hourly and alerts when the gap exceeds 1%?"

Scenario428 · Timezone Mismatch Between Source and Report

Tags: `intermediate` `dq-investigation` `timezone` `vendor-agnostic`

Business context: A Mumbai-based SaaS company runs on AWS Mumbai region but their data warehouse is in us-east-1 for cost reasons. The finance team's daily revenue report (run at 9 AM IST) keeps showing different numbers than the operations dashboard. The CFO is annoyed.

SQL:

```
-- Finance report (run from warehouse in UTC)
SELECT DATE(created_at) AS d, SUM(amount)
FROM payments
WHERE created_at >= '2026-05-25' AND created_at < '2026-05-26'
GROUP BY 1;

-- Ops dashboard (uses IST)
SELECT DATE(created_at AT TIME ZONE 'Asia/Kolkata') AS d, SUM(amount)
FROM payments
WHERE created_at AT TIME ZONE 'Asia/Kolkata' >= '2026-05-25'
  AND created_at AT TIME ZONE 'Asia/Kolkata' < '2026-05-26'
GROUP BY 1;
```

Suspicious output: Finance shows ₹42,18,300 for 25-May. Ops shows ₹44,02,150 for 25-May. Gap of ₹1,83,850.

Thinking out loud: 1. The classic five-and-a-half-hour problem. IST is UTC+5:30, so payments between 00:00 IST and 05:30 IST belong to the previous day in UTC. 2. The finance query is filtering on UTC dates but the user probably means IST dates. Roughly 5.5 hours of evening-IST payments leak into "tomorrow" in UTC. 3. Conversely, payments from 00:00 to 05:30 IST on 26-May would actually count as 25-May in UTC. 4. The fix is to always specify the timezone in date filters explicitly, and to standardise on a single business timezone (IST for an Indian business).

Diagnosis: `created_at` is stored as `TIMESTAMP WITH TIME ZONE` in UTC. The finance query is implicitly treating UTC dates as business dates, which shifts the day boundary by 5.5 hours.

Fix or diagnostic query:

```
SELECT
  DATE(created_at AT TIME ZONE 'Asia/Kolkata') AS ist_date,
  SUM(amount) AS revenue
FROM payments
WHERE created_at >= TIMESTAMP '2026-05-25 00:00:00' AT TIME ZONE 'Asia/Kolkata'
  AND created_at <  TIMESTAMP '2026-05-26 00:00:00' AT TIME ZONE 'Asia/Kolkata'
GROUP BY 1;
```

(In MySQL: use `CONVERT_TZ(created_at, '+00:00', '+05:30')` ; in SQL Server: `AT TIME ZONE 'India Standard Time' .`)

Prevention: Create a `dim_date_ist` table and a single helper view `payments_ist` that pre-converts. Any new report must use this view, not the raw table.

NOT to say: "Let's just store everything in IST." That breaks if the company ever expands to another timezone.

Follow-up: "What if some users are in the US — how would you let them see their own local-day numbers without breaking finance?"

Scenario429 · Cron Job Cuts Off at Midnight, Loses 30 Minutes of Rows

Tags: `intermediate` `dq-investigation` `cron` `vendor-agnostic`

Business context: A logistics company runs a nightly ETL that aggregates the day's shipments. The ETL runs at 23:30 IST and is supposed to capture "today." But every morning, the reconciliation report shows ~80 shipments missing compared to the OLTP system.

SQL:

```
-- ETL filter (runs at 23:30 IST on 25-May)
INSERT INTO daily_shipments_summary
SELECT shipment_id, status, created_at
FROM shipments
WHERE created_at >= '2026-05-25 00:00:00'
      AND created_at < '2026-05-26 00:00:00';
```

Suspicious output: Summary table has 4,217 shipments for 25-May. Source `shipments` table for the same date range has 4,298. Gap of 81.

Thinking out loud: 1. The ETL runs at 23:30, but the day isn't over yet — there are still 30 minutes left in the day, plus all the late-arriving rows after the cron starts. 2. The cron is filtering for the entire day, but it can only see rows up to the moment it runs. The 80-shipment gap matches the typical 23:30-00:00 traffic. 3. Two clean fixes: (a) run the cron after midnight (e.g., 01:30 next day) for "yesterday", or (b) keep the 23:30 schedule but re-run a "catch-up" job at 01:00 for any late rows. 4. There's also a subtle one — rows created at 23:30:15 while the cron is running may or may not be captured depending on transaction isolation level.

Diagnosis: Cron starts at 23:30 IST, scans rows up to its query start time. The 30 minutes between cron start and midnight produce ~80 shipments that never get into the summary.

Fix or diagnostic query:

```
-- Either shift the cron schedule:
-- crontab: 30 1 * * * (run at 01:30 the NEXT day for "yesterday")
-- And filter:
WHERE created_at >= CURRENT_DATE - INTERVAL '1 day'
      AND created_at < CURRENT_DATE;
```


Prevention: Never run a "today" aggregation while today is still happening. Always aggregate "yesterday" the next morning. Add a sentinel check: `assert row_count_in_summary = row_count_in_source` before marking the run successful.

NOT to say: "It's just 80 rows, ignore it." Eighty rows today is the same logic that becomes 8,000 rows next quarter.

Follow-up: "Suppose the business needs a real-time 'today so far' dashboard — how would you architect that without the cron-cutoff problem?"

Scenario430 · Soft-Deleted Rows Counted in One Report, Excluded in Another

Tags: `intermediate` `dq-investigation` `soft-delete` `vendor-agnostic`

Business context: An ed-tech platform tracks user enrolments. The product team's dashboard says "12,400 active enrolments this week." The finance team's revenue report says "11,950 enrolments paid this week." The gap of 450 keeps recurring.

SQL:

```
-- Product dashboard
SELECT COUNT(*) FROM enrolments
WHERE enrolled_at >= '2026-05-19' AND enrolled_at < '2026-05-26';
-- Finance report
SELECT COUNT(*) FROM enrolments
WHERE enrolled_at >= '2026-05-19' AND enrolled_at < '2026-05-26'
  AND deleted_at IS NULL;
```

Suspicious output: Product = 12,400. Finance = 11,950. Gap = 450 — and those 450 all have `deleted_at` populated.

Thinking out loud: 1. The schema uses soft-deletes (`deleted_at` set when a row is logically removed but kept for audit). The two reports disagree on whether to count soft-deleted rows. 2. Neither query is wrong per se — they're answering different business questions. Product wants "enrolments ever made" (including those that later got refunded/cancelled). Finance wants "currently valid enrolments." 3. The real bug is that both teams think they're answering the same question and don't realise the definition differs. 4. The cleanest fix is to define organisation-wide what "enrolment count" means, and codify it in a single view that both teams use.

Diagnosis: Definition drift. The 450 are refunds/cancellations that happened between enrolment and reporting date. Product counts them (they happened), finance excludes them (they're no longer valid revenue).

Fix or diagnostic query:

```
CREATE VIEW v_enrolments_finance AS
SELECT * FROM enrolments WHERE deleted_at IS NULL;
CREATE VIEW v_enrolments_product AS
SELECT * FROM enrolments;
-- Document the difference. Force both teams to use the view that matches their question.
```

Prevention: For every soft-delete column, document the policy: "this report includes/excludes deleted rows." Add a `deleted_count` column to summary tables so the gap is visible, not hidden.

NOT to say: "Finance's number is correct, product is double-counting." Both reflect a valid business view — the gap itself is the insight (it equals refunds).

Follow-up: "How would you redesign the table so the definition is unambiguous regardless of which team queries it?"

Scenario431 · Status Casing Bug — 'Completed' vs 'completed'

Tags: `intermediate` `dq-investigation` `casing` `vendor-agnostic`

Business context: A B2B invoicing platform reports 1,820 "completed" payments for the week. The CFO does a spot-check by exporting the raw table and finds 1,847 rows with completed status. Gap of 27.

SQL:

```
SELECT COUNT(*) FROM payments
WHERE status = 'completed' AND week_of = '2026-05-19';
-- returns 1820

SELECT COUNT(*) FROM payments
WHERE LOWER(status) = 'completed' AND week_of = '2026-05-19';
-- returns 1847
```

Suspicious output: 27 rows have `status = 'Completed'` (capital C) instead of `'completed'`. The original query is case-sensitive and misses them.

Thinking out loud: 1. In PostgreSQL and most databases with case-sensitive string equality, `'completed' != 'Completed'`. In MySQL with default collation, they'd be equal — so the behaviour depends on the engine. 2. Where would inconsistent casing come from? Multiple ingestion sources writing to the same column — perhaps a legacy app writes 'Completed' while the new API writes 'completed'. 3. The fix is twofold: short-term, use `LOWER()` in the filter; long-term, normalise the casing at write time and add a CHECK constraint. 4. Worth checking what other statuses are affected — could be 'Refunded' vs 'refunded', 'Cancelled' vs 'cancelled', etc.

Diagnosis: 23 of the 27 were written by an older mobile-app version that didn't standardise casing. 4 were written by a third-party integration that uses TitleCase by convention.

Fix or diagnostic query:

```
SELECT status, COUNT(*) FROM payments
WHERE week_of = '2026-05-19'
GROUP BY status ORDER BY 2 DESC;
-- See all variants. Then:
UPDATE payments SET status = LOWER(status) WHERE status != LOWER(status);
ALTER TABLE payments ADD CONSTRAINT status_lowercase CHECK (status = LOWER(status));
```

Prevention: Use a CHECK constraint or trigger to enforce lowercase. Better — use an enum type if your database supports it. Even better — make the column a foreign key to a `payment_statuses` lookup table.

NOT to say: "Just always use LOWER() in queries." Defensive but doesn't fix the root cause; the next engineer will write a non-LOWER query and the bug returns.

Follow-up: "What if the column also has leading/trailing whitespace — how would you find and fix that?"

Scenario432 · INNER JOIN Silently Drops Rows

Tags: `intermediate` `dq-investigation` `joins` `vendor-agnostic`

Business context: A travel booking startup. The marketing team asks for "all bookings with the user's signup source." The data analyst joins `bookings` to `users` and reports 4,200 bookings for the month. The product team insists there were 4,580.

SQL:

```
SELECT COUNT(*)
FROM bookings b
INNER JOIN users u ON b.user_id = u.user_id
WHERE b.booked_at >= '2026-05-01';
-- returns 4200
```

Suspicious output: Bookings table standalone count is 4,580. Joined count is 4,200. Gap of 380.

Thinking out loud: 1. INNER JOIN drops bookings whose `user_id` doesn't match any row in `users`. So 380 bookings have a `user_id` that doesn't exist in users. 2. Possible reasons: (a) guest checkouts where `user_id` is null, (b) users who were hard-deleted from the users table but their bookings remain, (c) FK isn't enforced and there's data corruption, (d) `user_id` in bookings table points to a different table (e.g., `admin_users`). 3. The fix is to use LEFT JOIN and add a flag column so the gap is

visible, not silent. 4. Also need to investigate why those 380 are orphaned — could be a real bug or could be expected (guest bookings).

Diagnosis: 290 are guest checkouts (`user_id IS NULL` in bookings), 87 are users who deleted their account (cascade-deleted from users but bookings remained), 3 are corrupted (user_id points to nothing).

Fix or diagnostic query:

```
SELECT
  COUNT(*) AS total_bookings,
  COUNT(u.user_id) AS bookings_with_user,
  COUNT(*) - COUNT(u.user_id) AS bookings_missing_user
FROM bookings b
LEFT JOIN users u ON b.user_id = u.user_id
WHERE b.booked_at >= '2026-05-01';
```

Prevention: Default to LEFT JOIN when one side is "facts." Only use INNER JOIN when you explicitly want to exclude unmatched rows, and document why. Add a data-quality alert for orphaned bookings.

NOT to say: "INNER JOIN is faster, so I used it." Correctness > performance, especially for reports.

Follow-up: "What if you wanted to count bookings whose user signed up in the last 30 days — would INNER JOIN be appropriate there?"

Scenario433 · WHERE Clause Excludes NULL Silently

Tags: `intermediate` `dq-investigation` `null-handling` `vendor-agnostic`

Business context: A health-tech app reports the number of appointments where the doctor is NOT a cardiologist. The number looks low. On review, the gap is around 600 appointments.

SQL:

```
SELECT COUNT(*) FROM appointments
WHERE doctor_specialty != 'cardiology'
  AND appointment_date = '2026-05-25';
-- returns 1240
SELECT COUNT(*) FROM appointments
WHERE appointment_date = '2026-05-25';
-- returns 2100
SELECT COUNT(*) FROM appointments
WHERE doctor_specialty = 'cardiology' AND appointment_date = '2026-05-25';
-- returns 260
-- 1240 + 260 = 1500, not 2100 - 600 missing
```

Suspicious output: Total 2,100 appointments. Cardiology = 260. Non-cardiology = 1,240. Sum = 1,500. 600 are unaccounted.

Thinking out loud: 1. Classic SQL three-valued-logic bug. `NULL != 'cardiology'` evaluates to NULL, not TRUE. So rows with `doctor_specialty IS NULL` are excluded from both queries. 2. The 600 missing rows are appointments where the doctor's specialty was never recorded — possibly walk-ins, telemedicine, or general physicians. 3. The fix is to either: (a) treat NULL as "non-cardiology" by using `IS DISTINCT FROM` (Postgres) or `doctor_specialty != 'cardiology' OR doctor_specialty IS NULL`, or (b) coalesce to a sentinel value. 4. The deeper question is — should NULL be allowed at all in this column? If every appointment must have a specialty, make it NOT NULL.

Diagnosis: 600 rows have `doctor_specialty IS NULL`. The `!=` filter silently excluded them.

Fix or diagnostic query:

```
-- Postgres
SELECT COUNT(*) FROM appointments
WHERE doctor_specialty IS DISTINCT FROM 'cardiology'
  AND appointment_date = '2026-05-25';

-- Vendor-agnostic
SELECT COUNT(*) FROM appointments
WHERE (doctor_specialty != 'cardiology' OR doctor_specialty IS NULL)
  AND appointment_date = '2026-05-25';
```

Prevention: When writing exclusion filters, always think "what about NULL?" Add a unit test that asserts `cardiology_count + non_cardiology_count = total_count`.

NOT to say: "I'll just add `IS NOT NULL` to fix it." That excludes NULLs entirely, which may not be the intent.

Follow-up: "What if the column is `doctor_specialty[]` (an array) and you want appointments where cardiology is NOT in the array — how does the NULL logic change?"

Scenario434 · Source Stored in Local Time, Report Assumes UTC

Tags: `intermediate` `dq-investigation` `timezone` `vendor-agnostic`

Business context: A pan-Asia logistics SaaS. Some warehouses are in Singapore, some in Bengaluru, some in Dubai. The OLTP stores `event_time` as `TIMESTAMP` (no timezone) in each warehouse's local time. The central warehouse aggregates and assumes UTC. Daily counts are wrong for every warehouse.

SQL:

```
SELECT warehouse_id, COUNT(*) AS scans_today
FROM scans
WHERE event_time >= '2026-05-25 00:00:00' AND event_time < '2026-05-26 00:00:00';
```

Suspicious output: Bengaluru warehouse usually has ~12,000 scans/day. Today it shows 9,800. Singapore shows 14,200 instead of typical 13,000.

Thinking out loud: 1. The query is in UTC. Bengaluru scans happen in IST (UTC+5:30). A scan at 02:00 IST is at 20:30 UTC the previous day. So a UTC-day filter chops off the early-morning IST hours and adds late-evening hours from the previous IST day. 2. Singapore is UTC+8. Same problem in the opposite direction — the UTC day includes some of yesterday's Singapore evening and misses today's Singapore morning. 3. This is worse than it sounds because each warehouse is shifted by a different amount. There is no single timezone you can convert to without splitting per-warehouse. 4. The fix is to store the warehouse timezone explicitly and convert when querying.

Diagnosis: `event_time` is a naive timestamp stored in each warehouse's local time. The central query treated all timestamps as UTC, causing per-warehouse day shifts.

Fix or diagnostic query:

```
SELECT
    s.warehouse_id,
    COUNT(*) AS scans_today
FROM scans s
JOIN warehouses w ON s.warehouse_id = w.warehouse_id
WHERE s.event_time >= TIMESTAMP '2026-05-25 00:00:00' AT TIME ZONE w.timezone
    AND s.event_time <  TIMESTAMP '2026-05-26 00:00:00' AT TIME ZONE w.timezone
GROUP BY 1;
```

Prevention: Always store timestamps in UTC (`TIMESTAMP WITH TIME ZONE`). Store the entity's timezone separately. Never use naive `TIMESTAMP` for events from multiple regions.

NOT to say: "Let's just convert all warehouses to IST." Doesn't help — the underlying data is in mixed timezones; conversion needs the warehouse timezone, not a single uniform one.

Follow-up: "What if the warehouse moves to a new timezone (DST or a regulatory change) — how would you handle historical data?"

Scenario435 · Daylight Saving Time Drops Rows

Tags: `advanced` `dq-investigation` `dst` `vendor-agnostic`

Business context: A US-India hybrid SaaS — Indian users are unaffected, but US users in spring see their daily activity reports drop one hour's worth of data. The product manager in Boston complains every March.

SQL:

```
SELECT
  generate_series(
    TIMESTAMP '2026-03-08 00:00:00' AT TIME ZONE 'America/New_York',
    TIMESTAMP '2026-03-09 00:00:00' AT TIME ZONE 'America/New_York',
    INTERVAL '1 hour'
  ) AS hr;
```

Suspicious output: The series skips the 02:00-03:00 ET hour (because DST jumped forward at 02:00). When aggregating hourly buckets, that hour has zero rows — but the user-facing report shows it as "no activity," misleading the PM.

Thinking out loud: 1. DST spring-forward means 02:00 doesn't exist on that day in US time zones. Fall-back means 01:00-02:00 happens twice. 2. If the underlying data is stored in UTC (which it should be), no rows are actually lost — it's the bucketing that's wrong. 3. If the underlying data is stored in local time as a naive timestamp, then the spring-forward hour literally has no rows because those moments don't exist locally — but also some rows are ambiguous in fall-back. 4. The diagnostic is to query in UTC and confirm the row counts are continuous, then fix the local-time presentation.

Diagnosis: Data is stored in UTC, but the hourly bucketing is converting to America/New_York and then bucketing — which produces a gap on spring-forward and a duplicate on fall-back.

Fix or diagnostic query:

```
-- Don't bucket on local time. Bucket on UTC, then convert the bucket label.
SELECT
  DATE_TRUNC('hour', event_time) AS utc_hour,
  DATE_TRUNC('hour', event_time AT TIME ZONE 'America/New_York') AS local_hour_label,
  COUNT(*) AS events
FROM user_events
WHERE event_time >= '2026-03-08' AND event_time < '2026-03-09'
GROUP BY 1, 2;
```

Prevention: For continuous time-series, always bucket in UTC. Only convert for human-facing labels. Document DST handling in the schema notes.

NOT to say: "Indian users are fine, ignore the US one." DST bugs compound — fall-back duplicates also break sum-of-hourly equals daily-total.

Follow-up: "What if your data warehouse uses `DATE_TRUNC` in local time and you can't change it — how do you patch the report?"

Scenario436 · Late-Arriving Facts Not Yet Ingested

Tags: `intermediate` `dq-investigation` `late-data` `vendor-agnostic`

Business context: A fintech reconciles bank settlements daily. The 9 AM IST report consistently shows yesterday's settlement count as ~3% lower than what the bank statement (received by email at noon) eventually shows.

SQL:

```
SELECT settlement_date, COUNT(*) AS settled_txns
FROM bank_settlements
WHERE settlement_date = CURRENT_DATE - 1
GROUP BY 1;
```

Suspicious output: 9 AM count = 9,700. By 2 PM, same query = 9,985. Difference of 285 transactions that "appeared" between the two runs.

Thinking out loud: 1. The bank's settlement feed is delivered in batches — typically 80% by 6 AM IST, the rest trickling in until noon. The 9 AM report only sees what's arrived by 9 AM. 2. This is not a bug; it's the nature of late-arriving data. The fix is process, not query. 3. Options: (a) delay the report until noon, (b) flag the report as "preliminary" until end-of-day, (c) keep a `data_complete_as_of` timestamp on the summary table. 4. The deeper question is whether downstream consumers can tolerate preliminary data. Finance usually cannot — they need certified.

Diagnosis: The bank feed has a known SLA of "complete by noon IST." Running the report at 9 AM means inherent under-counting.

Fix or diagnostic query:

```
-- Mark report freshness explicitly:
SELECT
    settlement_date,
    COUNT(*) AS settled_txns,
    MAX(ingested_at) AS data_freshness,
    CASE WHEN MAX(ingested_at) < CURRENT_DATE + TIME '12:00:00'
        THEN 'PRELIMINARY' ELSE 'FINAL' END AS data_state
FROM bank_settlements
WHERE settlement_date = CURRENT_DATE - 1
GROUP BY 1;
```

Prevention: Define an "as-of" cutoff for every report. Track ingestion completeness via a watermarking table. Distinguish between preliminary and final reports in the UI.

NOT to say: "There's a bug — 285 transactions are missing." There's no bug; the data hasn't arrived yet. Saying "bug" when it's a process issue erodes trust in engineering.

Follow-up: "How would you build an alert if late-arriving data exceeds the historical norm (e.g., 5% by noon instead of the usual 3%)?"

Scenario437 · Pipeline Failure Leaves Partial Day

Tags: `intermediate` `dq-investigation` `pipeline-failure` `vendor-agnostic`

Business context: A retail company's daily sales summary table is populated by an Airflow DAG that runs at 02:00 IST. On 23-May the DAG failed silently after processing 6 of 12 hourly partitions. The summary table has only half the day, but the report doesn't show it.

SQL:

```
SELECT sale_date, SUM(amount) AS daily_sales
FROM daily_sales_summary
WHERE sale_date = '2026-05-23'
GROUP BY 1;
-- returns 27,40,000 (should be ~55,00,000)
```

Suspicious output: 23-May shows ₹27,40,000. Adjacent days show ~₹54-56 lakh. The number is half the typical — suspiciously round.

Thinking out loud: 1. The fact that it's almost exactly half is a tell — likely a process that processed half the partitions before dying. 2. Next check: the source table (`sales` OLTP). What's the actual sum for 23-May? 3. Also check Airflow logs for that DAG run. A failure should have alerted; if it didn't, that's a second bug. 4. Even before fixing, we should check whether any downstream reports already consumed the bad data and need re-running.

Diagnosis: The Airflow DAG processed partitions for 00:00-12:00 IST and then a task failed. No alert was configured. The summary table has half the rows.

Fix or diagnostic query:

```
-- Compare source vs summary
SELECT
  (SELECT SUM(amount) FROM sales WHERE sale_date = '2026-05-23') AS source_total,
  (SELECT daily_sales FROM daily_sales_summary WHERE sale_date = '2026-05-23') AS summary_total;
-- Then backfill:
DELETE FROM daily_sales_summary WHERE sale_date = '2026-05-23';
INSERT INTO daily_sales_summary SELECT sale_date, SUM(amount) FROM sales WHERE sale_date = '2026-05-23';
```

Prevention: Every aggregation job must record `row_count_processed` in a metadata table; another job must verify against source. Failed DAG runs must page someone. Add a daily anomaly check: alert if today's number deviates >30% from 7-day average.

NOT to say: "Probably a slow sales day." Round-half is a process signature, not a business signal.

Follow-up: "How would you design an idempotent backfill so re-running the DAG for 23-May doesn't double-count?"

Scenario438 · Missing Partition Filter Loses a Day

Tags: `intermediate` `dq-investigation` `partitioning` `vendor-agnostic`

Business context: A telecom analytics team queries a partitioned `cdr_records` table (Call Detail Records) — partitioned by `event_date`. The daily report's count is consistently 1/30th lower than the operational count.

SQL:

```
SELECT COUNT(*) FROM cdr_records WHERE event_month = '2026-05';  
-- returns 28.5 crore (should be ~29.5 crore)
```

Suspicious output: ~3% lower than expected. The shortfall is roughly one day's worth in a 30-day month.

Thinking out loud: 1. The table is partitioned by `event_date` (daily partitions). The query filters on `event_month`, which is a derived column. 2. If the partition pruning logic isn't using `event_date` directly, it might miss one partition — e.g., the last day if it was created at midnight UTC vs local time. 3. Could also be that one daily partition exists but wasn't attached to the parent table — a common mistake with manually-managed partitions. 4. Check `pg_partitions` (Postgres) or `INFORMATION_SCHEMA.PARTITIONS` (MySQL) to see which partitions exist.

Diagnosis: One daily partition (`cdr_records_2026_05_25`) was created but never attached to the parent table. The query plan never scans it.

Fix or diagnostic query:

```
-- Postgres: list partitions  
SELECT child.relname AS partition_name  
FROM pg_inherits  
JOIN pg_class parent ON pg_inherits.inhparent = parent.oid  
JOIN pg_class child ON pg_inherits.inhrelid = child.oid  
WHERE parent.relname = 'cdr_records'  
ORDER BY child.relname;  
-- Attach the missing one:  
ALTER TABLE cdr_records ATTACH PARTITION cdr_records_2026_05_25  
FOR VALUES FROM ('2026-05-25') TO ('2026-05-26');
```

Prevention: Automate partition creation and attachment via `pg_partman` or a daily DAG. Add a check: number of partitions in the last 30 days should be exactly 30. Always filter on the partition column directly when possible.

NOT to say: "Query is fine, just a sampling discrepancy." 3% on a one-month query is too systematic to be sampling.

Follow-up: "How would you write a daily check that asserts every expected partition exists and contains data?"

Scenario439 · Read Replica Lag Hides Recent Rows

Tags: `intermediate` `dq-investigation` `replication` `vendor-agnostic`

Business context: A B2C app. Customer support runs a "recent orders" query against a read replica. A customer placed an order 30 seconds ago and is now on the phone — but the support agent's query returns nothing. They tell the customer "we don't see your order," and the customer escalates.

SQL:

```
SELECT * FROM orders
WHERE customer_email = 'priya@example.com'
ORDER BY created_at DESC LIMIT 5;
-- returns nothing (recent order not yet replicated)
```

Suspicious output: The order exists on the primary but not on the replica. Replica lag is currently ~45 seconds during peak hours.

Thinking out loud: 1. Read replicas are async. Any query against a replica can be slightly stale. 2. For most analytics use cases this is fine; for customer-support real-time queries it's a disaster. 3. The fix is to route certain queries (recent orders, recent payments, account status) to the primary, or to use a "recent reads from primary" policy. 4. Could also use a read-your-own-writes pattern — check replica first, fall back to primary if not found.

Diagnosis: Read replica lag of 45s. Recent orders haven't replicated yet. Support tool was configured to use replica for cost reasons.

Fix or diagnostic query:

```
-- Monitor lag (Postgres)
SELECT
  client_addr,
  state,
  pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn) AS lag_bytes,
  EXTRACT(EPOCH FROM (now() - replay_time)) AS lag_seconds
FROM pg_stat_replication;
```

Prevention: Route latency-sensitive reads to primary. For replica reads, always show "data as of X seconds ago" in UI. Alert if replica lag exceeds 30 seconds.

NOT to say: "The order doesn't exist." It does; the replica just hasn't caught up. Phrase customer-facing responses to allow for replication.

Follow-up: "How would you implement a 'read your own writes' pattern that uses replica by default but primary for the user's own recent writes?"

Scenario440 · Outbox Events Stuck, Downstream Missing Rows

Tags: `advanced` `dq-investigation` `outbox` `vendor-agnostic`

Business context: An e-commerce platform uses the outbox pattern — orders are written to an `outbox` table, then a worker publishes them to Kafka, then a downstream warehouse ingests. Warehouse shows 12,400 orders. App DB shows 12,580. Gap of 180.

SQL:

```
SELECT COUNT(*) FROM app.orders WHERE order_date = CURRENT_DATE;
-- 12580
SELECT COUNT(*) FROM warehouse.fact_orders WHERE order_date = CURRENT_DATE;
-- 12400
SELECT COUNT(*) FROM app.outbox WHERE event_type='order_created' AND published_at IS NULL;
-- 180
```

Suspicious output: Exactly 180 unpublished events in the outbox. The downstream is missing exactly those 180.

Thinking out loud: 1. The outbox pattern is precisely what's broken — events written to the outbox, never published. The publisher worker has fallen behind or died. 2. Why? Could be (a) worker crashed and didn't restart, (b) Kafka topic at capacity / unhealthy, (c) the worker's poll loop is stuck on a poison message, (d) the worker is running but its filter is wrong. 3. Check the worker process status, the Kafka topic lag, and the most recent successful publish time. 4. After fixing the worker, the events need to be re-driven — but ensure idempotency on the consumer side.

Diagnosis: Publisher worker crashed at 14:30 due to OOM. Auto-restart wasn't configured. 180 events accumulated unpublished.

Fix or diagnostic query:

```
-- Find oldest unpublished event
SELECT MIN(created_at), MAX(created_at), COUNT(*)
FROM app.outbox
WHERE published_at IS NULL;
-- Drive the publisher manually or wait for it to catch up after restart
```

Prevention: Health check on publisher worker (alert if no publish in 60 seconds). Limit outbox backlog with an alert. Make publisher idempotent so safe re-drive is possible. Monitor Kafka consumer lag.

NOT to say: "Maybe warehouse ETL is slow." It's a downstream symptom; the root cause is the outbox publisher, which has a clear signature (NULL `published_at`).

Follow-up: "How would you ensure exactly-once delivery from outbox to Kafka to warehouse, given that any of those steps can retry?"

Scenario441 · ETL Filter Mismatch Between Staging and Mart

Tags: `intermediate` `dq-investigation` `etl` `vendor-agnostic`

Business context: A B2B SaaS. The data engineer reports the staging table (`stg_subscriptions`) has 8,450 rows for the month. The mart table (`fact_subscriptions`) has 8,320. Both are supposed to be 1:1 from source.

SQL:

```
SELECT COUNT(*) FROM stg_subscriptions WHERE month = '2026-05';
-- 8450
SELECT COUNT(*) FROM fact_subscriptions WHERE month = '2026-05';
-- 8320
```

Suspicious output: 130 rows in staging, missing from mart. No errors in pipeline logs.

Thinking out loud: 1. Each pipeline step (source → staging → mart) is supposed to be lossless unless filtering is explicit. A 130-row gap means a filter exists somewhere. 2. Check the dbt model (or whatever does staging → mart) for any `WHERE` clauses. Could be `WHERE is_test_user = false`, `WHERE country != 'XX'`, or `WHERE amount > 0`. 3. Sample the 130 missing rows: what's their pattern? If they're all test users, the filter is intentional. If they're regular customers, it's a bug. 4. Compare schema — maybe an INNER JOIN to a dimension table drops rows whose dimension isn't loaded.

Diagnosis: The mart model joins to `dim_plans` via INNER JOIN. 130 subscriptions are on a new plan code that hasn't been added to `dim_plans` yet. Those subscriptions silently disappear.

Fix or diagnostic query:

```
SELECT s.plan_code, COUNT(*)
FROM stg_subscriptions s
LEFT JOIN dim_plans p ON s.plan_code = p.plan_code
WHERE s.month = '2026-05' AND p.plan_code IS NULL
GROUP BY 1;
```

Prevention: Use LEFT JOIN with a "missing dimension" sentinel ('UNKNOWN'). Add row-count tests in dbt (`dbt-utils` `equal_rowcount`) between staging and mart. Alert when new plan codes appear in source.

NOT to say: "Mart and staging will eventually align." They won't if the join is silent.

Follow-up: "How would you handle a scenario where a new plan code legitimately appears every week — should the dimension auto-extend?"

Scenario442 · Currency Mismatch — Base vs Reporting

Tags: `intermediate` `dq-investigation` `currency` `vendor-agnostic`

Business context: An Indian SaaS sells to US, India, and Singapore. Revenue is stored in customer's local currency in `payments`. The finance dashboard reports ₹4.2 crore for the month, but the CFO's own spreadsheet says ₹4.8 crore.

SQL:

```
SELECT SUM(amount) AS total_revenue_inr
FROM payments
WHERE payment_date >= '2026-05-01' AND currency = 'INR';
-- 4.2 crore
```

Suspicious output: The query only sums INR rows. USD and SGD payments are excluded — but the finance team thinks "total revenue" includes all currencies converted to INR.

Thinking out loud: 1. The query filters `WHERE currency = 'INR'` and sums amounts as-is. USD and SGD revenue is missing. 2. To get a proper total, sum across currencies after converting via a daily exchange rate. 3. Need a `dim_exchange_rates` table with daily rates per currency pair. 4. Also need to define which rate to use — payment date, end-of-month, average — and document it.

Diagnosis: Currency conversion is missing. USD payments (~₹50 lakh equivalent) and SGD payments (~₹10 lakh) are excluded.

Fix or diagnostic query:

```
SELECT SUM(p.amount * er.rate_to_inr) AS total_revenue_inr
FROM payments p
JOIN dim_exchange_rates er
  ON er.from_currency = p.currency
  AND er.to_currency = 'INR'
  AND er.rate_date = p.payment_date::date
WHERE p.payment_date >= '2026-05-01';
```

Prevention: Always normalise to a single reporting currency at the warehouse layer. Store both `amount_local` and `amount_inr` columns. Document the exchange-rate convention (payment date vs settlement date).

NOT to say: "The dashboard is for INR revenue, that's why USD is excluded." If the label says "total revenue," it should be total in one currency.

Follow-up: "What if a customer refunds in their local currency three months after payment — which exchange rate do you apply to the refund?"

Scenario443 · Aggregation Granularity Mismatch

Tags: `intermediate` `dq-investigation` `granularity` `vendor-agnostic`

Business context: A logistics startup. The source `shipments` table is event-level. The mart `daily_shipments` is daily-aggregated. The hourly dashboard query reads from daily mart and shows zero shipments for every hour — but the daily totals are correct.

SQL:

```
SELECT
  DATE_TRUNC('hour', shipment_date) AS hr,
  SUM(shipments_count)
FROM daily_shipments
WHERE shipment_date = '2026-05-25'
GROUP BY 1;
-- returns one row at 00:00 with the entire day's count
```

Suspicious output: Daily total is correct (4,200) but it all shows up in the 00:00 hour bucket — every other hour is empty.

Thinking out loud: 1. The mart is already aggregated at the day level. There's no hourly information to recover from a daily aggregate. 2. The dashboard query is trying to extract a finer grain than the data supports — like trying to listen to individual notes from a song mixed down to mono. 3. The fix is to either build an hourly mart from the source, or have the hourly dashboard read directly from source. 4.

This is a common bug when business users ask for "hourly view" of a daily-aggregated KPI and the engineer doesn't notice the granularity gap.

Diagnosis: The mart's grain is day. The dashboard needs hour. The dashboard can't synthesise the finer grain from coarser data.

Fix or diagnostic query:

```
-- Read from source for hourly:
SELECT
  DATE_TRUNC('hour', shipped_at) AS hr,
  COUNT(*) AS shipments
FROM shipments
WHERE shipped_at >= '2026-05-25' AND shipped_at < '2026-05-26'
GROUP BY 1
ORDER BY 1;
```

Prevention: Document the grain of every mart (`grain: day` , `grain: hour`). Reject downstream queries that mismatch grain. Build separate hourly and daily marts when both are needed.

NOT to say: "There's no data for those hours." There is data; the mart just isn't at the right grain.

Follow-up: "How would you build a single mart that supports both daily and hourly queries efficiently?"

Scenario444 · Cohort Definition Mismatch

Tags: `intermediate` `dq-investigation` `cohort` `vendor-agnostic`

Business context: An ed-tech platform tracks "Q1 2026 user cohort retention." The growth team reports 42% retention. The product team reports 38%. Same data, two numbers. Both are reading the same warehouse.

SQL:

```
-- Growth team
SELECT user_id FROM users
WHERE first_signup_at >= '2026-01-01' AND first_signup_at < '2026-04-01';
-- Product team
SELECT user_id FROM users
WHERE first_purchase_at >= '2026-01-01' AND first_purchase_at < '2026-04-01';
```

Suspicious output: Different cohort sizes (growth: 12,400 users; product: 9,800 users). Because the cohorts are different, the retention rates differ.

Thinking out loud: 1. Both teams said "Q1 cohort" but they meant different things. Growth meant "signed up in Q1." Product meant "first purchase in Q1." 2. Neither is wrong; both are valid cohort

definitions. The bug is that the term "Q1 cohort" is ambiguous across teams. 3. A user who signed up in December but first purchased in January is in product's cohort but not growth's. A user who signed up in March but never purchased is in growth's but not product's. 4. The fix is to define cohorts explicitly and consistently across the org.

Diagnosis: Definition drift. "Cohort" was never formalised — signup-cohort vs purchase-cohort vs first-active-cohort all coexist.

Fix or diagnostic query:

```
-- Create explicit, named cohort dimensions:
CREATE VIEW v_signup_cohort_q1_2026 AS
SELECT user_id FROM users WHERE first_signup_at >= '2026-01-01' AND first_signup_at < '2026-04-01'

CREATE VIEW v_purchase_cohort_q1_2026 AS
SELECT user_id FROM users WHERE first_purchase_at >= '2026-01-01' AND first_purchase_at < '2026-04-01'
```

Prevention: Maintain a `metrics_catalog` (or LookML, dbt semantic layer) that defines every cohort term canonically. Add a meeting hygiene rule: "always say signup-cohort or purchase-cohort, never just cohort."

NOT to say: "One of the queries is wrong." Both are technically correct; the bug is at the definition layer.

Follow-up: "How would you build a single retention dashboard that lets users toggle between signup-cohort and purchase-cohort definitions?"

Scenario445 · 'Active Users' Defined Differently Across Teams

Tags: `intermediate` `dq-investigation` `definition-drift` `vendor-agnostic`

Business context: A consumer app. Product says MAU is 4.2 lakh. Marketing says MAU is 3.8 lakh. Finance reports MAU at 3.5 lakh in the board deck. All three pull from the same `user_events` table.

SQL:

```
-- Product: opened the app
SELECT COUNT(DISTINCT user_id) FROM user_events WHERE event_name='app_open' AND event_date >= CURRENT_DATE - 30;

-- Marketing: any tracked event
SELECT COUNT(DISTINCT user_id) FROM user_events WHERE event_date >= CURRENT_DATE - 30;

-- Finance: completed a paid transaction
SELECT COUNT(DISTINCT user_id) FROM user_events WHERE event_name='purchase_completed' AND event_date >= CURRENT_DATE - 30;
```

Suspicious output: Three different definitions, three different numbers. Each team thinks the others are wrong.

Thinking out loud: 1. "Active" is one of the most overloaded words in data. Every team uses it for their own purpose. 2. The right fix isn't to pick a winner — it's to name each definition (MAU_app_opener, MAU_engaged, MAU_paid) and use the named version in every report. 3. The board deck should explicitly say which definition it uses (typically MAU_engaged or MAU_paid). 4. This is a leadership/ comms problem as much as a SQL problem.

Diagnosis: Three valid definitions, but the same label "MAU" was used for all three.

Fix or diagnostic query:

```
-- Create a metrics catalog table or dbt model:
CREATE VIEW v_mau AS
SELECT
  CURRENT_DATE AS as_of,
  COUNT(DISTINCT CASE WHEN event_name='app_open' THEN user_id END) AS mau_app_open,
  COUNT(DISTINCT user_id) AS mau_any_event,
  COUNT(DISTINCT CASE WHEN event_name='purchase_completed' THEN user_id END) AS mau_purchase
FROM user_events
WHERE event_date >= CURRENT_DATE - 30;
```

Prevention: Adopt a semantic layer (dbt metrics, LookML, Cube.js). Force every dashboard to reference a named metric, not a hand-written query. Quarterly metric audit.

NOT to say: "Marketing is over-counting." Marketing isn't wrong; they have a broader definition.

Follow-up: "How would you migrate three teams from their hand-written MAU queries to a shared semantic layer without breaking their dashboards?"

Scenario446 · Attribution Model — Last-Click vs First-Click

Tags: [advanced](#) [dq-investigation](#) [attribution](#) [vendor-agnostic](#)

Business context: A D2C brand asks "which channel drove the most conversions in May?" The performance marketing team reports Google Ads at top. The brand team reports Instagram at top. Both pull from the same [conversions](#) table.

SQL:

```
-- Performance team: last-click
SELECT last_click_channel, COUNT(*) FROM conversions
WHERE converted_at >= '2026-05-01' GROUP BY 1 ORDER BY 2 DESC;
-- Brand team: first-click
SELECT first_click_channel, COUNT(*) FROM conversions
WHERE converted_at >= '2026-05-01' GROUP BY 1 ORDER BY 2 DESC;
```

Suspicious output: Last-click puts Google Ads first; first-click puts Instagram first. Both totals equal the same overall conversion count, so the sum is consistent — but the per-channel attribution differs.

Thinking out loud: 1. This is structurally correct — different attribution models produce different per-channel numbers. The bug is treating one as definitive. 2. Real attribution should be multi-touch (linear, time-decay, U-shaped). The choice depends on business need. 3. For a true picture, build a single attribution view that supports multiple models and show all. 4. The CFO should never see a single attribution number without the model name attached.

Diagnosis: Attribution model drift. Both queries are correct but they answer different questions.

Fix or diagnostic query:

```
SELECT
  channel,
  COUNT(*) FILTER (WHERE first_click_channel = channel) AS first_click_attributions,
  COUNT(*) FILTER (WHERE last_click_channel = channel) AS last_click_attributions,
  SUM(linear_weight) AS linear_attributions
FROM conversions c
JOIN UNNEST(c.touchpoints) t(channel, linear_weight) ON TRUE
WHERE c.converted_at >= '2026-05-01'
GROUP BY channel;
```

Prevention: Always label attribution numbers with the model used. Build a single attribution dashboard with a model toggle. Document the org-default model (usually multi-touch for budget allocation).

NOT to say: "Last-click is the right model." It depends on the business; brand vs performance teams legitimately need different models.

Follow-up: "If a conversion has 7 touchpoints across 4 channels over 14 days, how does a time-decay model weight each?"

Scenario447 · Funnel Step Counted Differently Downstream

Tags: `intermediate` `dq-investigation` `funnel` `vendor-agnostic`

Business context: An ed-tech platform measures signup-to-payment funnel. The "added to cart" step shows 14,200 users in the product analytics tool but 13,800 in the data warehouse. Same event.

SQL:

```
-- Product analytics: distinct users who fired add_to_cart event
SELECT COUNT(DISTINCT user_id) FROM events WHERE event_name='add_to_cart' AND event_date='2026-05-25';
-- Warehouse funnel mart: users with at least one cart row created
SELECT COUNT(DISTINCT user_id) FROM cart_items WHERE created_at::date='2026-05-25';
```

Suspicious output: Events table: 14,200. Cart_items table: 13,800. Gap of 400.

Thinking out loud: 1. The events table records the user firing the `add_to_cart` event (client-side tracking). The cart_items table records the server-side cart row creation. 2. The gap of 400 likely represents events that fired but never made it to the server (network failure, ad-blocker, app crash before request). 3. Could also be deduplication — if a user adds 3 items, that's 3 events but 3 cart_items. So counting distinct users should match. Unless one is counting events, the other items. 4. Worth confirming both queries use `DISTINCT user_id`, not `COUNT(*)`.

Diagnosis: Both queries use distinct users. The 400 are client-side events without corresponding server rows. Half are ad-blocker/network failures, half are app crashes mid-action.

Fix or diagnostic query:

```
SELECT e.user_id
FROM events e
LEFT JOIN cart_items c ON e.user_id = c.user_id
    AND c.created_at::date = e.event_date
WHERE e.event_name = 'add_to_cart'
    AND e.event_date = '2026-05-25'
    AND c.user_id IS NULL;
```

Prevention: Reconcile client-side events with server-side actions daily. Treat server-side as source of truth for funnel metrics. Use client-side only for cohort discovery.

NOT to say: "Product analytics is wrong." Both are recording real things; the question is which is the truth source for the funnel.

Follow-up: "How would you build a daily reconciliation report between client-side events and server-side actions and alert when divergence exceeds a threshold?"

Scenario448 · Materialised View Refreshed at Wrong Time

Tags: `intermediate` `dq-investigation` `materialized-view` `vendor-agnostic`

Business context: A retail dashboard reads from a materialised view `mv_daily_sales`. The view is supposed to refresh every hour. The morning report shows yesterday's sales as ₹38 lakh; the underlying table shows ₹42 lakh. Gap of ₹4 lakh.

SQL:

```
SELECT * FROM mv_daily_sales WHERE sale_date = CURRENT_DATE - 1;
-- 38 lakh
SELECT SUM(amount) FROM sales WHERE sale_date = CURRENT_DATE - 1;
-- 42 lakh
```

Suspicious output: The MV is stale. Last refresh was 18 hours ago.

Thinking out loud: 1. Check when the MV was last refreshed. `pg_stat_user_tables` or `last_refresh` metadata. 2. If the refresh job ran at 23:00 but never finished, the MV is from before yesterday's full day was captured. 3. Could also be that the refresh job runs but quietly fails on a lock or memory issue. 4. The fix is to monitor MV freshness explicitly and alert when it exceeds the SLA.

Diagnosis: Cron triggered refresh at 02:00, but the job hung on a lock and never completed. No alert was set up.

Fix or diagnostic query:

```
-- Postgres
SELECT
    schemaname, matviewname,
    pg_size_pretty(pg_total_relation_size(quote_ident(schemaname) || '.' || quote_ident(matviewname)
    (SELECT MAX(check_time) FROM mv_refresh_log WHERE mv_name = matviewname) AS last_refresh
FROM pg_matviews;

REFRESH MATERIALIZED VIEW CONCURRENTLY mv_daily_sales;
```

Prevention: Log MV refresh start/end to a metadata table. Alert if refresh takes longer than 2x the historical average. Use `REFRESH ... CONCURRENTLY` to avoid blocking readers.

NOT to say: "The MV must have a bug in its definition." The definition is fine; the refresh schedule is the problem.

Follow-up: "How would you decide between a materialised view that refreshes hourly vs a regular view that runs the query each time?"

Scenario449 · Schema Migration Changed Column Meaning Mid-Week

Tags: `advanced` `dq-investigation` `migration` `vendor-agnostic`

Business context: A B2B platform changed the meaning of `status = 'inactive'` mid-week — before Wednesday it meant "user paused"; from Wednesday onwards it means "user deleted." The weekly active users report shows a sudden 18% drop on Wednesday.

SQL:

```
SELECT week_starting, COUNT(*) FROM users
WHERE status != 'inactive' AND last_seen_at >= week_starting
GROUP BY 1;
-- Mon-Tue: ~24,000/day. Wed onwards: ~19,700/day.
```

Suspicious output: A clean step-change on Wednesday — not a gradual decline, which would indicate a real business issue.

Thinking out loud: 1. A sharp step-change in a metric, dated exactly to a deployment or migration, is almost always a definition change rather than a business event. 2. Check the migration log / git history for Wednesday. Look for ALTER TABLE, model changes, or enum value changes. 3. The old report's `!= 'inactive'` filter now excludes a different set of users than before. 4. Need to either backfill the historical meaning, or document the break and start a new metric series.

Diagnosis: Wednesday's migration repurposed the 'inactive' enum value. Old data has 'inactive' meaning paused; new data has 'inactive' meaning deleted. The filter now excludes both, causing the drop.

Fix or diagnostic query:

```
-- For historical data, distinguish using migration date:
SELECT week_starting, COUNT(*) FROM users
WHERE last_seen_at >= week_starting
  AND CASE
    WHEN last_seen_at < '2026-05-22' THEN status NOT IN ('inactive', 'deleted')
    ELSE status NOT IN ('deleted')
  END
GROUP BY 1;
```

Prevention: Never repurpose enum values. Add new enum values (e.g., 'paused' and 'deleted') instead of changing existing ones. Migration plans must include a downstream-impact review.

NOT to say: "Users dropped 18% on Wednesday." The drop is an artefact of the migration, not a business signal. Saying it out loud causes panic.

Follow-up: "How would you design the migration so the downstream report breaks loudly (errors out) instead of silently shifting?"

Scenario450 · Federated Query — One Source Had Data Quality Issue

Tags: `advanced` `dq-investigation` `federation` `vendor-agnostic`

Business context: A holding company runs a federated query across its three subsidiaries' data warehouses (BigQuery, Snowflake, Redshift). The combined "total customers" report shows 4.2 lakh. Each subsidiary's local count sums to 4.5 lakh. Gap of 30,000.

SQL:

```
-- Federated query (simplified)
SELECT 'sub_a' AS source, COUNT(*) FROM ext.sub_a_customers
UNION ALL
SELECT 'sub_b', COUNT(*) FROM ext.sub_b_customers
UNION ALL
SELECT 'sub_c', COUNT(*) FROM ext.sub_c_customers;
```

Suspicious output: Federated total = 4.2 lakh. Sub-A local = 1.5 lakh. Sub-B local = 1.7 lakh. Sub-C local = 1.3 lakh. Sum of locals = 4.5 lakh.

Thinking out loud: 1. Each subsidiary should match the federated read of itself. So compare each federated count to its local count. 2. Likely one subsidiary's federated view filters differently than its local view — perhaps the federated view excludes some statuses or has a time-window filter. 3. Could also be that the federated read uses a snapshot that's older than the live data. 4. Could also be schema drift — a column name change in one subsidiary broke the federated query but didn't error out, instead silently returning a subset.

Diagnosis: Sub-B's federated view definition excludes `customer_type = 'enterprise'` — a filter added 6 months ago for compliance and forgotten. Sub-B local shows 1.7 lakh including enterprise; federated shows 1.4 lakh excluding them. Difference = 30,000.

Fix or diagnostic query:

```
-- Reconcile per subsidiary
SELECT
  'sub_b_federated' AS source, COUNT(*) FROM ext.sub_b_customers
UNION ALL
SELECT 'sub_b_local', COUNT(*) FROM sub_b.customers;
-- Then inspect the federated view definition to find the filter
```

Prevention: Document every federated view's filter logic. Add a daily reconciliation job that compares federated vs source counts and alerts on divergence > 0.5%. Treat federated views as code with version control.

NOT to say: "Cross-warehouse federation is unreliable." It's reliable when the definitions are consistent; the problem is process, not tech.

Follow-up: "How would you build a single 'customer 360' across three warehouses where each defines 'customer' slightly differently?"

Category 7, Part C — Data Quality: Drift & Audit Trail Gaps

Part C is about diagnosing data that LOOKS right but is wrong over time — the silent failures that survive every assertion you wrote on day one. The query still parses. The dashboard still renders. The pipeline still runs green. And yet revenue has been undercounted for six weeks, the cohort retention chart has

been quietly redefined, the audit trail you swore was bulletproof is missing every bulk insert, and your trended KPI has been creeping up because a vendor changed an upstream encoding nobody told you about. The senior data role interview signal here is unambiguous: have you been burned by these in production, do you instinctively reach for run-over-run reconciliation, schema-evolution tooling, and audit completeness checks before you trust a number — and when you ship a change, do you leave behind enough breadcrumbs that the next on-call can tell what you altered and when? Twenty-five scenarios below, each written the way a senior interviewer expects you to think out loud: symptom first, hypotheses in order, the diagnosis, the fix, what the over-eager junior would have said instead, and the inevitable follow-up. Indian-context tables throughout, vendor-agnostic SQL with dialect call-outs where the syntax forks.

Scenario451 · Revenue Line Went Flat — Upstream Pipeline Silently Broken

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Flipkart · Dialect: vendor-agnostic

The scenario:

The CFO's daily revenue dashboard shows a flat line for the last nine days. Bookings are happening — support is fielding tickets, customers are receiving order confirmations — yet `fact_revenue_daily` has not advanced. The query that powers the chart returns rows for every date, the bar just plots at the same height because the underlying total has not moved.

```
SELECT booking_date, SUM(amount_inr) AS revenue_inr
FROM fact_revenue_daily
WHERE booking_date >= CURRENT_DATE - INTERVAL '30' DAY
GROUP BY booking_date
ORDER BY booking_date;
```

Expected: Daily revenue that tracks reality — visible day-over-day swings. **Actual:** Flat line; the most recent nine `booking_date` values all sum to a value identical to the value nine days ago.

Thinking out loud: 1. "Flat means either (a) no new rows landing for those dates, or (b) the rows that exist are stale duplicates of an older snapshot." 2. "Let me count rows per `booking_date` — if it is zero for the last nine days, the load stopped. If row counts are normal but amounts are identical, the source has frozen but the loader kept copying the same snapshot." 3. "Either way the dashboard cannot tell — it just renders what is in the warehouse."

The diagnosis: Row count per recent date is zero; `MAX(load_ts)` on the table is nine days old. The upstream Airflow DAG has been red since a credential rotation but the alerting rule was set up against a different DAG name after a recent rename, so nobody got paged. The chart silently re-rendered the last good day at the right edge of the window because the y-axis auto-scaled.

The fix:

```
-- Daily freshness guard – runs on a schedule, raises if breached
SELECT
  MAX(booking_date)           AS latest_business_date,
  CURRENT_DATE - MAX(booking_date) AS days_behind,
  MAX(load_ts)                AS latest_load_ts,
  CURRENT_TIMESTAMP - MAX(load_ts) AS hours_since_last_load
FROM fact_revenue_daily;

-- And a row-count anomaly check
SELECT booking_date, COUNT(*) AS row_count
FROM fact_revenue_daily
WHERE booking_date >= CURRENT_DATE - INTERVAL '14' DAY
GROUP BY booking_date
HAVING COUNT(*) = 0 OR COUNT(*) < 0.5 * AVG(COUNT(*)) OVER ();
```

Why this works: Freshness checks treat the absence of data as a first-class signal rather than an absence of evidence. The HAVING-with-window pattern catches both zero-row days and days that landed but landed light.

What NOT to say: - "Refresh the dashboard cache" — the chart is faithfully showing what is in the warehouse. - "The pipeline must be fine, the dashboard renders" — green render ≠ green data.

Common follow-up: *"How would you instrument this check so a similar rename does not silently disable it?"*

Scenario452 · Metric Trending Up — Definition Changed Last Week, No One Announced

Tags: Difficulty: Medium · Asked at: Swiggy, Zomato, Meesho · Dialect: vendor-agnostic

The scenario:

Weekly Active Users (WAU) jumped 22 percent week-on-week with no corresponding marketing push, no app release, no product change in the funnel. The CMO wants to know what worked. You suspect the metric definition was edited; you need to prove it before the credit gets misattributed.

```
SELECT week_start, wau
FROM mart_engagement_weekly
WHERE week_start >= CURRENT_DATE - INTERVAL '90' DAY
ORDER BY week_start;
```

Expected: A trend you can explain through known levers. **Actual:** Step-change in the most recent week with no upstream reason.

Thinking out loud: 1. "Step-changes that line up with a Monday usually mean someone changed something on the weekend." 2. "Let me check the dbt model history — `git log` on `mart_engagement_weekly.sql` for the last fortnight." 3. "Also let me back-fill the prior definition and overlay the two — if the old definition is still flat, it is a definition change, not a real lift."

The diagnosis: A teammate widened the activity definition from "logged a session" to "logged a session OR opened a push notification." The change was merged on Sunday, the model rebuilt overnight, the dashboard read the new numbers Monday morning. No CHANGELOG entry, no announcement on the analytics channel.

The fix:

```
-- Reconcile old vs new definition side by side
WITH new_def AS (
  SELECT week_start, COUNT(DISTINCT user_id) AS wau_new
  FROM fact_user_activity
  WHERE event_type IN ('session_start', 'push_open')
  GROUP BY week_start
),
old_def AS (
  SELECT week_start, COUNT(DISTINCT user_id) AS wau_old
  FROM fact_user_activity
  WHERE event_type = 'session_start'
  GROUP BY week_start
)
SELECT n.week_start, wau_old, wau_new, wau_new - wau_old AS lift_from_def_change
FROM new_def n
JOIN old_def o USING (week_start)
ORDER BY week_start;
```

Why this works: The reconciliation isolates definitional lift from real lift. A senior analyst always shows both lines on the same axis when defending a metric change.

What NOT to say: - "Push must be driving real engagement" — you have not proven that. - "Roll back the change" — you have not yet established whether the new definition is wrong, only that the change was unannounced.

Common follow-up: "How do you enforce that definition changes are versioned and announced?"

Scenario453 · New Product Line Not Appearing in Report — Filter Hardcoded to Old Product List

Tags: Difficulty: Easy · Asked at: Flipkart, Myntra, Nykaa · Dialect: vendor-agnostic

The scenario:

The new "Home Appliances" category went live three weeks ago. Marketing is pushing it heavily, GMV is real, the operations team sees order flow. The category-mix dashboard still shows only Fashion, Electronics, and Grocery. Home Appliances is invisible.

```
SELECT category, SUM(gmv_inr) AS gmv
FROM fact_orders_enriched
WHERE category IN ('Fashion','Electronics','Grocery')
  AND order_date >= CURRENT_DATE - INTERVAL '30' DAY
GROUP BY category;
```

Expected: All active categories represented proportionally. **Actual:** New categories silently dropped.

Thinking out loud: 1. "That `IN` list is a frozen snapshot from whenever this report was first written." 2. "Anything not in the list is filtered out at the `WHERE` stage." 3. "This is the most common drift bug in BI — taxonomies grow, filters don't."

The diagnosis: A hardcoded `IN` clause inside a downstream view enumerated the categories that existed at report-creation time. New rows in `dim_category` never propagated into the report because the filter is by string literal, not by a reference to the dimension table.

The fix:

```
-- Drive the filter from the dimension table, not a literal list
SELECT c.category, SUM(f.gmv_inr) AS gmv
FROM fact_orders_enriched f
JOIN dim_category c ON c.category_id = f.category_id
WHERE c.is_active = TRUE
  AND f.order_date >= CURRENT_DATE - INTERVAL '30' DAY
GROUP BY c.category
ORDER BY gmv DESC;
```

Why this works: The filter now follows whatever the dimension says is currently active, so new categories appear automatically when added upstream.

What NOT to say: - "Just add 'Home Appliances' to the IN list" — that fixes today and rots tomorrow. - "Why didn't anyone tell me about the new category" — that is the bug, the report should not have required telling.

Common follow-up: "How would you audit the warehouse for other hardcoded category lists?"

Scenario454 · Cohort Retention Metric Off — Definition Tweaked in dbt Model

Tags: Difficulty: Hard · Asked at: PhonePe, Cred, Razorpay · Dialect: vendor-agnostic, dbt-flavoured

The scenario:

The retention chart for the January signup cohort shifted overnight. Week-4 retention now reads 38 percent; yesterday's print of the same chart said 31 percent. No new data has landed in the underlying fact table. Someone edited the model.

```
SELECT cohort_month, weeks_since_signup, retention_pct
FROM mart_cohort_retention
WHERE cohort_month = '2026-01-01';
```

Expected: Stable historical numbers, only the rightmost cell moves as new data arrives. **Actual:** All cells shifted up.

Thinking out loud: 1. "Historical cells should be immutable unless the definition changed." 2. "Let me diff the dbt model between yesterday and today." 3. "I will also look for a `dbt run --full-refresh` in the deploy logs — that would have rebuilt history."

The diagnosis: The retention definition was loosened from "had a purchase in the week" to "had a purchase OR a meaningful in-app event in the week." The model was full-refreshed during deploy, so prior weeks rebuilt against the new definition. No DECLARE-and-pin pattern, no snapshot table.

The fix: Pin historical retention as a snapshot, not a recomputation:

```
-- Snapshot pattern: write the cohort table once per period and never recompute
INSERT INTO mart_cohort_retention_snapshot
(cohort_month, weeks_since_signup, retention_pct, definition_version, snapshot_ts)
SELECT cohort_month,
       weeks_since_signup,
       retention_pct,
       'v2_includes_inapp_events',
       CURRENT_TIMESTAMP
FROM mart_cohort_retention
WHERE NOT EXISTS (
  SELECT 1 FROM mart_cohort_retention_snapshot s
  WHERE s.cohort_month = mart_cohort_retention.cohort_month
        AND s.weeks_since_signup = mart_cohort_retention.weeks_since_signup
);
```

Why this works: A snapshot table with a `definition_version` column preserves what the number was at the time the number was reported, so historical comparability survives definition changes.

What NOT to say: - "It is fine, the new definition is better" — better is not the issue; comparability is the issue. - "Just roll back the dbt model" — that loses the work, instead snapshot and continue.

Common follow-up: "How would you communicate a definition change to executive stakeholders?"

Scenario455 · Funnel Step Disappeared — Event Renamed Upstream

Tags: Difficulty: Medium · Asked at: Swiggy, Zomato, Dunzo · Dialect: vendor-agnostic

The scenario:

The checkout funnel chart shows a 100 percent drop-off between the "address selected" and "payment attempted" steps. Either the entire user base abandoned simultaneously eight days ago, or the event has been renamed and the funnel definition still references the old name.

```
SELECT event_name, COUNT(*) AS n
FROM fact_events
WHERE event_date = CURRENT_DATE - INTERVAL '1' DAY
      AND event_name IN ('address_selected', 'payment_attempted', 'payment_success')
GROUP BY event_name;
```

Expected: All three steps populated. **Actual:** `payment_attempted` returns zero; the other two are healthy.

Thinking out loud: 1. "Either nobody attempts payment any more — implausible given confirmed orders — or the event name changed." 2. "Let me query the raw event names that look similar." 3. " `LIKE 'payment%'` will surface whatever the new spelling is."

The diagnosis: The mobile team renamed `payment_attempted` to `payment_initiated` in app version 4.18, released eight days ago. The release notes mentioned it; the analytics consumer was not on the distribution list.

The fix:

```

-- Discover all current variants
SELECT event_name, MIN(event_ts) AS first_seen, MAX(event_ts) AS last_seen, COUNT(*) AS r
FROM fact_events
WHERE event_name LIKE 'payment%'
GROUP BY event_name
ORDER BY first_seen;

-- Funnel definition that tolerates the rename via an event-name dimension
SELECT funnel_step, COUNT(DISTINCT user_id) AS users
FROM fact_events f
JOIN dim_event_alias a ON a.raw_event_name = f.event_name
WHERE a.funnel_step IN ('address', 'payment_attempt', 'payment_success')
    AND f.event_date = CURRENT_DATE - INTERVAL '1' DAY
GROUP BY funnel_step;

```

Why this works: Funnels driven by an aliasing dimension survive instrumentation renames — you update `dim_event_alias` once and every funnel using it inherits the fix.

What NOT to say: - "Users must have stopped paying" — six hours of confirmed order revenue says otherwise. - "Add an `OR event_name = 'payment_initiated'`" — that re-creates the same brittleness for the next rename.

Common follow-up: *"How would you detect a rename automatically rather than waiting for someone to notice?"*

Scenario456 · Schema Migration Added NOT NULL With Default 0, Skewing Past Data

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Zerodha · Dialect: Postgres, MySQL

The scenario:

The `transactions` table got a new column `discount_inr` with `NOT NULL DEFAULT 0`. Marketing's "discount uplift" report now shows zero discount applied for every transaction predating the migration. The reality is that pre-migration discounts existed but were stored elsewhere and not back-filled.

```

SELECT DATE_TRUNC('month', txn_ts) AS month,
       SUM(discount_inr) AS total_discount
FROM transactions
WHERE txn_ts >= CURRENT_DATE - INTERVAL '12' MONTH
GROUP BY 1
ORDER BY 1;

```

Expected: A consistent discount series across the full year. **Actual:** Hockey-stick at the migration date — zero before, normal after.

Thinking out loud: 1. " `DEFAULT 0` on a `NOT NULL` add does not mean 'we know it was zero', it means 'we filled in zero for rows that did not have a value'." 2. "The pre-migration discount data lives somewhere else — promo redemptions, voucher ledger — and was never joined back." 3. "The report is faithfully summing zeros and confidently lying."

The diagnosis: A backfill step was supposed to populate `discount_inr` from the legacy `voucher_redemptions` table at migration time and was skipped under deadline pressure. The default zero masked the missing data and silently entered every downstream aggregation.

The fix: Backfill from the source of truth and add a sentinel for "we genuinely do not know":

```
-- Treat unknown-pre-migration as NULL, not 0
ALTER TABLE transactions ALTER COLUMN discount_inr DROP NOT NULL;

UPDATE transactions t
SET discount_inr = v.amount_inr
FROM voucher_redemptions v
WHERE v.txn_id = t.txn_id
      AND t.txn_ts < '2026-03-01'
      AND t.discount_inr = 0;

-- Anything still 0 and pre-migration is genuinely unknown; mark as NULL
UPDATE transactions
SET discount_inr = NULL
WHERE txn_ts < '2026-03-01' AND discount_inr = 0;
```

Why this works: Distinguishing "zero" from "unknown" stops the report from silently averaging in fabricated zeros. Reports that need a number can `COALESCE(discount_inr, 0)` explicitly with their eyes open.

What NOT to say: - "It is fine, defaults are normal" — defaults on backfills change the meaning of historical numbers. - "We can just exclude pre-migration data" — that loses a year of context unnecessarily.

Common follow-up: "Write a check that flags any column where post-migration values differ meaningfully from pre-migration values."

Scenario457 · ALTER TABLE Changed Column Type — Implicit Cast Broke Downstream

Tags: Difficulty: Medium · Asked at: Flipkart, PhonePe, Paytm · Dialect: Postgres, MySQL, SQL Server

The scenario:

`order_id` was widened from INT to BIGINT to accommodate growth. The transactional database accepted the change cleanly. A downstream Python ETL into the warehouse now fails every night with a silent type mismatch — rows are loaded with the high bits truncated, producing collisions in the warehouse `fact_orders` table.

Expected: One-to-one row mapping from source to warehouse. **Actual:** Warehouse row count diverges from source row count; two different source `order_id` values now collide on the same warehouse `order_id`.

Thinking out loud: 1. "*BIGINT → INT lossy cast on the loader side. The source widened, the target did not.*" 2. "*Implicit casts in pandas, JDBC, and SQLAlchemy sometimes truncate silently rather than raise.*" 3. "*Reconciliation count will reveal it.*"

The diagnosis: The warehouse `fact_orders.order_id` was still INT; the loader cast BIGINT to INT and silently wrapped values above 2,147,483,647. The first such value collided with a small-numbered legacy order, overwriting on upsert.

The fix:

```
-- 1. Widen the warehouse column
ALTER TABLE fact_orders ALTER COLUMN order_id TYPE BIGINT;
-- (MySQL): ALTER TABLE fact_orders MODIFY order_id BIGINT;
-- (SQL Server): ALTER TABLE fact_orders ALTER COLUMN order_id BIGINT NOT NULL;

-- 2. Reconciliation query – should always return 0
SELECT s.source_row_count, w.warehouse_row_count,
       s.source_row_count - w.warehouse_row_count AS gap
FROM (SELECT COUNT(*) AS source_row_count FROM source.orders) s
CROSS JOIN (SELECT COUNT(*) AS warehouse_row_count FROM fact_orders) w;

-- 3. Find collided rows (multiple source PKs mapping to one warehouse PK)
SELECT order_id, COUNT(DISTINCT source_natural_key) AS collisions
FROM fact_orders
GROUP BY order_id
HAVING COUNT(DISTINCT source_natural_key) > 1;
```

Why this works: Widening the warehouse type halts further damage. The reconciliation guard catches future width drift before it corrupts a quarter of data.

What NOT to say: - "Cast back to INT in the source" — that re-introduces the original ceiling. - "Run the loader again, it will sort itself out" — collisions are already in place.

Common follow-up: "*How do you detect schema drift between source and warehouse before it causes a load failure?*"

Scenario458 · Time-Zone Migration Across Systems — Historical Data Labelled Wrong

Tags: Difficulty: Hard · Asked at: Swiggy, Zomato, Ola · Dialect: vendor-agnostic

The scenario:

A platform was migrated from server-local time (Asia/Kolkata) to UTC storage. All new timestamps come in as UTC. Historical rows were not relabelled. Every report that filters by `event_ts >= '2026-01-01'` is now off by five and a half hours for the historical portion and aligned for the post-migration portion.

```
SELECT DATE(event_ts) AS event_date, COUNT(*)
FROM fact_events
WHERE event_ts >= '2026-01-01'
GROUP BY 1
ORDER BY 1;
```

Expected: Sensible per-day buckets. **Actual:** A 5.5-hour smear at the cutover; days before the cutover are pulled forward by 5.5 hours, hiding events that happened in the early morning IST window.

Thinking out loud: 1. "The column type is `TIMESTAMP WITHOUT TIME ZONE` so it carries no zone metadata." 2. "The historical values are 'wall-clock IST' and the new values are 'wall-clock UTC'." 3. "I need to relabel the historical values or apply a conditional shift."

The diagnosis: The migration changed the meaning of the column without changing the column. Pre-cutover rows are IST wall-clock stored as naive timestamps; post-cutover rows are UTC wall-clock stored as naive timestamps. The boundary is the deploy time.

The fix: Mark the boundary explicitly and normalise to UTC across the board:

```
-- Add a UTC-correct column
ALTER TABLE fact_events ADD COLUMN event_ts_utc TIMESTAMP;

UPDATE fact_events
SET event_ts_utc = CASE
    WHEN event_ts < '2026-03-15 00:00:00'
    THEN event_ts - INTERVAL '5 hours 30 minutes' -- was IST wall-clock, shift to UTC
    ELSE event_ts -- already UTC
END;

-- For reports, derive IST on read
SELECT DATE(event_ts_utc AT TIME ZONE 'Asia/Kolkata') AS event_date_ist, COUNT(*)
FROM fact_events
GROUP BY 1 ORDER BY 1;
```

Why this works: Storing in a single canonical zone (UTC) and converting at read time prevents the boundary discontinuity. The CASE handles the historical relabelling once.

What NOT to say: - "Just subtract 5:30 from everything" — that breaks the post-cutover rows that are already UTC. - "It is only 5.5 hours, the daily totals are close enough" — they are not, the boundary day is materially wrong.

Common follow-up: *"How do you handle the IST half-hour offset in tools that assume integer-hour zones?"*

Scenario459 · ID Space Exhausted — Auto-Increment Hit INT Max, Wrapped or Failed

Tags: Difficulty: Hard · Asked at: PhonePe, Razorpay, Flipkart · Dialect: MySQL, Postgres

The scenario:

A fast-growing payments service began rejecting new transactions with a duplicate-key error. The application logs blame the database; the database is healthy. The `txn_id INT AUTO_INCREMENT` column has reached 2,147,483,647 (signed INT max). MySQL refuses to insert; in older configurations or unsigned-INT, the counter would wrap to a low value and collide.

Expected: Continuous inserts. **Actual:** Duplicate-key errors at the writer; the most recent successful insert has `txn_id` at or near the signed-INT ceiling.

Thinking out loud: 1. "`SELECT MAX(txn_id) FROM transactions` will tell me where we are." 2. "If it is near 2.1B, this is the INT exhaustion that every team thinks 'will not happen for years'." 3. "Migration to BIGINT on a 200M-row hot table is a multi-hour DDL on InnoDB unless I use `pt-online-schema-change` or `gh-ost`."

The diagnosis: The table outgrew the type. Signed-INT can address ~2.1 billion values; an OLTP payments table that inserts a few hundred million per year hits that ceiling in single-digit years. Unsigned-INT doubles the runway but only delays the same problem.

The fix: Plan and execute the migration online; do not lock the table:

```
-- Postgres: ALTER TYPE is metadata-only for INT → BIGINT in newer versions,
-- but the sequence behind it needs widening too
ALTER TABLE transactions ALTER COLUMN txn_id TYPE BIGINT;
ALTER SEQUENCE transactions_txn_id_seq AS BIGINT MAXVALUE 9223372036854775807;

-- MySQL: use online schema change tooling for hot tables
-- pt-online-schema-change --alter "MODIFY txn_id BIGINT NOT NULL AUTO_INCREMENT" \
-- D=payments,t=transactions --execute

-- Monitor headroom going forward
SELECT MAX(txn_id) AS current_high,
       2147483647 AS signed_int_max,
       9223372036854775807 AS bigint_max,
       ROUND(100.0 * MAX(txn_id) / 2147483647, 2) AS pct_of_int
FROM transactions;
```

Why this works: BIGINT buys nine quintillion values — effectively forever for any realistic transaction rate. The monitoring query catches the next column on the same trajectory before it bites.

What NOT to say: - "Reset the sequence to 1" — that causes collisions with existing rows. - "Delete old rows to free up IDs" — auto-increment is monotonic, deletes do not free values.

Common follow-up: "How would you audit every INT primary key in the database for headroom?"

Scenario460 · UUID Format Inconsistency Between Two Services

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy · Dialect: Postgres, MySQL

The scenario:

Service A writes UUIDs as `b1f2c3d4-e5f6-7890-abcd-ef1234567890` (lowercase, hyphenated). Service B writes UUIDs as `B1F2C3D4E5F67890ABCDEF1234567890` (uppercase, no hyphens). The join between their tables returns no rows; the two systems are talking past each other.

```
SELECT *
FROM service_a_orders a
JOIN service_b_shipments b ON b.order_uuid = a.order_uuid;
```

Expected: A row for every order with a corresponding shipment. **Actual:** Empty.

Thinking out loud: 1. "The columns might be `VARCHAR` instead of native `UUID`, in which case the join is a string comparison and string comparison is case-sensitive in most collations." 2. "Even if one

side is `UUID` and the other `VARCHAR`, casting on either side normalises only one direction." 3. "I will canonicalise both sides on read."

The diagnosis: Service A column is `UUID`, Service B column is `VARCHAR(32)` with no hyphens and uppercase letters. The implicit cast on the join either fails outright or silently produces zero matches depending on the dialect.

The fix:

```
-- Normalise both sides to canonical UUID
SELECT *
FROM service_a_orders a
JOIN service_b_shipments b
  ON a.order_uuid::TEXT = LOWER(
    REGEXP_REPLACE(b.order_uuid, '^[0-9a-fA-F]{8})([0-9a-fA-F]{4})([0-9a-fA-F]{4})([0-9a-fA-F]{4})$',
    '\1-\2-\3-\4-\5');

-- Long-term fix: convert service_b_shipments.order_uuid to native UUID type
ALTER TABLE service_b_shipments
  ALTER COLUMN order_uuid TYPE UUID
  USING LOWER(REGEXP_REPLACE(order_uuid, '^[0-9a-fA-F]{8})([0-9a-fA-F]{4})([0-9a-fA-F]{4})([0-9a-fA-F]{4})$',
    '\1-\2-\3-\4-\5'));
```

Why this works: Converting the rogue column to native `UUID` makes the format mismatch impossible going forward — the database enforces canonical representation on insert.

What NOT to say: - "Just `LOWER()` both sides" — that does not insert the hyphens. - "Add another column for the canonical form" — possible, but the type fix is cleaner.

Common follow-up: "How would you detect this drift across all `UUID` columns in the schema?"

Scenario461 · Audit Table Missing Rows — TRIGGER Not Firing in Bulk Path

Tags: Difficulty: Hard · Asked at: Razorpay, HDFC, ICICI Lombard · Dialect: Postgres, MySQL

The scenario:

Compliance reviews show that `customers_audit` has rows for every change made through the application, but is missing entries for the monthly bulk update job that imports 50,000 KYC refreshes. The trigger is `FOR EACH ROW` and looks correct.

```
CREATE TRIGGER customers_audit_trg
AFTER UPDATE ON customers
FOR EACH ROW
EXECUTE FUNCTION customers_audit_fn();
```

Expected: One audit row per updated customer. **Actual:** Audit rows for application traffic only; bulk-import path leaves no trail.

Thinking out loud: 1. "FOR EACH ROW should fire 50,000 times, not zero." 2. "Unless the bulk path uses COPY (Postgres) or LOAD DATA INFILE (MySQL) which bypass triggers, or TRUNCATE + INSERT instead of UPDATE, or the trigger is AFTER UPDATE and the bulk path is INSERT ... ON CONFLICT doing an UPSERT." 3. "Most likely the bulk path does DELETE + INSERT, missing the UPDATE trigger entirely."

The diagnosis: The monthly KYC refresh does DELETE WHERE last_kyc_at < threshold; INSERT ... for changed rows. The AFTER UPDATE trigger never fires because no UPDATE happens — the bulk path is effectively a replace. Compliance has been missing 50,000 events per month for who knows how long.

The fix: Cover all DML paths and consider statement-level triggers for bulk:

```
-- Fire on INSERT, UPDATE, and DELETE
CREATE TRIGGER customers_audit_ins_trg
AFTER INSERT ON customers
FOR EACH ROW EXECUTE FUNCTION customers_audit_fn();

CREATE TRIGGER customers_audit_del_trg
AFTER DELETE ON customers
FOR EACH ROW EXECUTE FUNCTION customers_audit_fn();

-- For Postgres COPY, application code must call the audit function explicitly
-- because COPY bypasses row-level triggers by design in some configurations.

-- Verification: bulk-load row count vs audit row count for the same window
SELECT
  (SELECT COUNT(*) FROM customers WHERE last_modified_ts BETWEEN $1 AND $2) AS data_changes,
  (SELECT COUNT(*) FROM customers_audit WHERE audit_ts BETWEEN $1 AND $2) AS audit_rows;
```

Why this works: Triggers on every DML verb close the obvious gap; the reconciliation query is the safety net that detects future bypasses (a new bulk path that skips triggers shows up immediately).

What NOT to say: - "The trigger looks correct, must be a permissions issue" — the trigger is correct for UPDATE, the bug is that the bulk path is not UPDATE. - "Just add the audit rows manually for the missing months" — fix the leak first, then back-fill from logs if possible.

Common follow-up: "How do you design an audit pattern that survives bulk replaces?"

Scenario462 · Audit Log Truncated by Retention Policy

Tags: Difficulty: Medium · Asked at: HDFC, Axis, Razorpay · Dialect: vendor-agnostic

The scenario:

A compliance request asks for who edited customer KYC records in the last 18 months. The audit table has rows only for the last 90 days. A retention job has been silently pruning the rest.

```
SELECT MIN(audit_ts), MAX(audit_ts), COUNT(*)  
FROM customers_audit;
```

Expected: 18+ months of history. **Actual:** 90 days; oldest row exactly 90 days back, suggesting an active retention job.

Thinking out loud: 1. "Audit data is regulated — retention is usually defined externally." 2. "Some team set retention based on transactional-table cost without realising audit had a separate, longer requirement." 3. "I need to recover what I can, change retention, and split audit storage from transactional storage."

The diagnosis: A nightly DELETE job runs `DELETE FROM customers_audit WHERE audit_ts < CURRENT_DATE - INTERVAL '90' DAY`. The job was created when audit data lived on the hot OLTP node and disk was tight. The compliance retention requirement is 7 years for KYC, 18 months for everything else. Nobody objected when the 90-day rule was introduced because nobody owned it.

The fix:

```
-- Stop the deletion immediately  
-- (Disable the cron / unschedule the Airflow DAG / drop the EVENT in MySQL)  
  
-- Move audit to a cold-storage table partitioned by month  
CREATE TABLE customers_audit_archive (LIKE customers_audit INCLUDING ALL)  
PARTITION BY RANGE (audit_ts);  
  
-- Restore from backup the 90-day to 18-month window, then attach as historical partitions  
  
-- Going forward: hot table holds 90 days, archive holds the regulated tail  
CREATE OR REPLACE VIEW customers_audit_full AS  
SELECT * FROM customers_audit  
UNION ALL  
SELECT * FROM customers_audit_archive  
WHERE audit_ts < (SELECT MIN(audit_ts) FROM customers_audit);
```

Why this works: Splitting hot and cold storage keeps the OLTP cost down while preserving the audit tail for the regulated retention period. The view gives consumers one query surface.

What NOT to say: - "Increase retention to 7 years on the hot table" — punishes OLTP performance. - "Run the deleted rows back from logs" — only works if you have WAL/binlog for that period.

Common follow-up: *"How do you ensure retention policies are owned and reviewed by the right team?"*

Scenario463 · Soft Delete Pattern but Trigger Does Not Capture

Tags: Difficulty: Medium · Asked at: Flipkart, Myntra, Nykaa · Dialect: Postgres, MySQL

The scenario:

The application implements soft deletes by setting `is_deleted = TRUE`. The audit trigger is `AFTER DELETE` and captures hard deletes only. Soft deletes leave no audit trail because, technically, no row is deleted.

```
CREATE TRIGGER orders_audit_del_trg
AFTER DELETE ON orders
FOR EACH ROW EXECUTE FUNCTION orders_audit_fn();
```

Expected: Audit trail for every effective deletion. **Actual:** Audit captures only hard deletes; soft deletes are invisible.

Thinking out loud: 1. "A soft delete is technically an `UPDATE` — set `is_deleted = TRUE`." 2. "The `AFTER DELETE` trigger does not fire on `UPDATE`." 3. "I need an `UPDATE` trigger with a `WHEN (NEW.is_deleted IS DISTINCT FROM OLD.is_deleted)` clause."

The diagnosis: Audit pattern was designed when deletes were hard. When the team moved to soft-delete for recoverability, the trigger was not updated.

The fix:

```
-- Postgres: WHEN clause restricts to actual deletion transitions
CREATE TRIGGER orders_audit_softdel_trg
AFTER UPDATE ON orders
FOR EACH ROW
WHEN (OLD.is_deleted IS DISTINCT FROM NEW.is_deleted)
EXECUTE FUNCTION orders_audit_fn();

-- MySQL: no WHEN, gate inside the trigger body
DELIMITER //
CREATE TRIGGER orders_audit_softdel_trg
AFTER UPDATE ON orders
FOR EACH ROW
BEGIN
    IF NOT (OLD.is_deleted <=> NEW.is_deleted) THEN
        INSERT INTO orders_audit (...) VALUES (...);
    END IF;
END //
DELIMITER ;
```

Why this works: The trigger now fires precisely when the soft-delete flag flips, capturing every effective deletion regardless of physical DML verb.

What NOT to say: - "Convert soft deletes back to hard deletes" — defeats the recoverability requirement. - "Audit every UPDATE" — wasteful, the WHEN clause keeps the audit table tight.

Common follow-up: *"How do you audit a soft-restore (un-delete) as well?"*

Scenario464 · Audit Table Grew Too Large — Partition or Archive

Tags: Difficulty: Medium · Asked at: PhonePe, Cred, Razorpay · Dialect: Postgres, MySQL

The scenario:

`transactions_audit` is 2.1 TB. Index rebuilds take 14 hours. Every query against it is slow. The audit data is regulatorily required for 7 years so simple deletion is not an option.

Expected: Manageable storage and acceptable query latency on recent rows. **Actual:** Monolithic table; OLTP write latency now dominated by index maintenance on audit indexes.

Thinking out loud: 1. "A 2 TB monolithic audit table on the OLTP cluster is wrong on every axis." 2. "Partition by month; drop or detach old partitions; archive detached partitions to cold storage." 3. "Move audit off the OLTP cluster if regulator permits."

The diagnosis: Audit was placed on the transactional cluster for transactional consistency with the underlying tables. That was reasonable when audit was 10 GB; at 2 TB it is the cluster's main cost driver.

The fix:

```
-- Convert to partitioned table by audit_ts (declarative partitioning, Postgres 11+)
-- (One-time migration using pg_partman or manual: CREATE PARTITIONED copy, INSERT, swap.)

CREATE TABLE transactions_audit_p (
  audit_id BIGSERIAL,
  audit_ts TIMESTAMPTZ NOT NULL,
  /* ... other columns ... */
) PARTITION BY RANGE (audit_ts);

-- Create monthly partitions
CREATE TABLE transactions_audit_p_2026_05
  PARTITION OF transactions_audit_p
  FOR VALUES FROM ('2026-05-01') TO ('2026-06-01');

-- Old partitions can be detached and exported to Parquet on S3 / Azure Blob
ALTER TABLE transactions_audit_p DETACH PARTITION transactions_audit_p_2024_01;

-- MySQL: PARTITION BY RANGE (TO_DAYS(audit_ts))
```

Why this works: Partition pruning makes recent-window queries fast. Detached partitions can live cheaply in object storage with a Parquet export, queried via Athena / Trino / Postgres FDW when needed for compliance lookups.

What NOT to say: - "Add more indexes" — already the bottleneck. - "Delete older rows" — regulator says no.

Common follow-up: *"What is the cost / latency trade-off of keeping cold partitions on the cluster vs S3?"*

Scenario465 · Data Lineage Broken — Column Renamed Mid-Stream

Tags: Difficulty: Medium · Asked at: Swiggy, Zomato, Meesho · Dialect: vendor-agnostic, dbt-flavoured

The scenario:

A dbt model halfway down the DAG renamed `customer_id` to `cust_id` for "consistency." Three downstream models still reference `customer_id`. The whole branch is failing every night with "column does not exist" — and the BI tool is reading the last successful materialisation, which is now a week old and silently stale.

Expected: A clean DAG run; downstream models reflect today's data. **Actual:** Failing branch; BI shows week-old data with no obvious staleness indicator.

Thinking out loud: 1. "Renames without dependency-aware refactoring break the DAG." 2.

"`dbt deps` and `dbt ls --select +my_model` would have shown the impact pre-merge." 3.

"The BI tool reading the last good materialisation is the silent-failure mode — the chart looks fresh."

The diagnosis: The rename was merged without checking downstream references. The CI did not catch it because the downstream models were marked `optional` or excluded from the test selection. The materialised tables in the warehouse from before the rename are still present and queryable, so the BI tool happily renders them.

The fix:

```
-- Quick fix: alias in the renaming model so downstream still resolves the old name
SELECT
  cust_id,
  cust_id AS customer_id,  -- transitional alias
  ...
FROM ...

-- Real fix: refactor downstream models, deprecate the alias with a fixed sunset date
-- (Track the deprecation as a project-level TODO with an owner.)

-- Add a freshness assertion that the BI tool can read
SELECT MAX(load_ts) AS last_loaded
FROM downstream_model;
-- If last_loaded < CURRENT_DATE - INTERVAL '2' DAY, raise.
```

Why this works: The transitional alias re-greens the DAG without rushed downstream edits. The freshness assertion ensures the BI tool's staleness becomes visible the next time it happens.

What NOT to say: - "Rename it back" — undoes the consistency improvement. - "Just rebuild from scratch" — disruptive and unnecessary.

Common follow-up: "How would you enforce rename impact analysis in CI for the dbt project?"

Scenario466 · Source Dataset Replaced With New Vendor — Values Shifted

Tags: Difficulty: Hard · Asked at: Flipkart, Nykaa, Myntra · Dialect: vendor-agnostic

The scenario:

The pin-code-to-region mapping vendor was switched from Vendor A to Vendor B. The new mapping uses different region definitions — Vendor A's "North 1" is now split between Vendor B's "North-East" and "North-Central." Region-level KPIs shifted overnight with no marketing change.

Expected: Stable region KPIs across the cutover. **Actual:** Visible discontinuity at the cutover date; some regions appear smaller, others larger.

Thinking out loud: 1. "Reference data swaps change the meaning of every downstream group-by." 2. "I need both vendors' mappings for the overlap period to reconcile." 3. "Long-term, I need a canonical region definition that survives vendor swaps."

The diagnosis: The new vendor was wired in directly, replacing the old `dim_region` overnight. Historical fact rows joined to the new dim now bucket into the new regions, but the chart's prior weeks still show old buckets because they were materialised. The mixed display reads as a real shift.

The fix:

```
-- Step 1: keep both vendor mappings, mark them
CREATE TABLE dim_pincode_region (
  pincode VARCHAR(6),
  vendor VARCHAR(20),
  region VARCHAR(50),
  effective_from DATE,
  effective_to DATE
);

-- Step 2: define a canonical region as the intersection / agreed mapping
CREATE TABLE dim_pincode_region_canonical (
  pincode VARCHAR(6),
  canonical_region VARCHAR(50)
);

-- Step 3: join facts to canonical for stable historical reporting
SELECT cr.canonical_region, SUM(f.gmv_inr)
FROM fact_orders f
JOIN dim_pincode_region_canonical cr ON cr.pincode = f.delivery_pincode
GROUP BY cr.canonical_region;
```

Why this works: A vendor-neutral canonical layer absorbs vendor swaps without disturbing historical comparability. Vendor-specific tables remain available for any vendor-specific drill-down.

What NOT to say: - "Tell business the new regions are better" — they may be, but the comparison break is the immediate problem. - "Re-run last year's reports with the new mapping" — that retroactively rewrites history that was reported.

Common follow-up: "How do you negotiate canonical reference data with a vendor that wants to define its own?"

Scenario467 · Customer ID Space Collision After Migration

Tags: Difficulty: Hard · Asked at: Razorpay, PhonePe, Cred · Dialect: vendor-agnostic

The scenario:

Two regional databases were consolidated into a single customer master. Both used sequential integer IDs starting at 1. After the merge, `customer_id = 12345` could mean one customer from the South region or a completely different customer from the North region. Downstream reports double-count.

Expected: A unique ID per customer. **Actual:** Collisions on natural-key joins.

Thinking out loud: 1. "Two ID spaces that both start at 1 will collide unless re-keyed during merge." 2. "The fix is a surrogate key in the unified table, with the legacy region-qualified key kept as an attribute." 3. "All downstream joins must move to the surrogate."

The diagnosis: The consolidation copied both tables into a single `customer_master` preserving the original `customer_id`. The migration team thought "we will dedupe later." Later never came. Reports now show inflated counts because joins on `customer_id` match across both populations.

The fix:

```
-- Add a surrogate primary key
ALTER TABLE customer_master ADD COLUMN customer_uid UUID DEFAULT gen_random_uuid();
ALTER TABLE customer_master ADD COLUMN source_region VARCHAR(10);
ALTER TABLE customer_master ADD CONSTRAINT customer_master_legacy_uk UNIQUE (source_region, customer_id);

-- Backfill source_region from the merge metadata
UPDATE customer_master SET source_region = 'S' WHERE migrated_from = 'south_db';
UPDATE customer_master SET source_region = 'N' WHERE migrated_from = 'north_db';

-- Downstream tables need a customer_uid column joined via (source_region, customer_id)
ALTER TABLE fact_orders ADD COLUMN customer_uid UUID;
UPDATE fact_orders f
SET customer_uid = cm.customer_uid
FROM customer_master cm
WHERE cm.source_region = f.source_region
AND cm.customer_id = f.customer_id;
```

Why this works: Surrogate keys are immune to source-system collisions. The composite uniqueness on `(source_region, customer_id)` prevents future regression.

What NOT to say: - "Renumber one region's customer_ids" — breaks every external reference to those IDs. - "Add a region prefix to customer_id" — changes the type and breaks integrations.

Common follow-up: "How would you detect this collision without explicit knowledge of the merge?"

Scenario468 · Currency Conversion Table Stale

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Nykaa · Dialect: vendor-agnostic

The scenario:

A cross-border revenue dashboard shows USD revenue translated to INR at last quarter's exchange rate. The FX table is updated quarterly via a manual upload that someone forgot to do this quarter. The INR figure is materially understated.

```
SELECT order_date, SUM(amount_usd * fx.usd_to_inr) AS revenue_inr
FROM fact_orders o
JOIN dim_fx_rates fx ON fx.fx_date = (SELECT MAX(fx_date) FROM dim_fx_rates)
GROUP BY order_date;
```

Expected: Each order translated at the rate effective on its `order_date`. **Actual:** Every order translated at one stale rate.

Thinking out loud: 1. "`MAX(fx_date)`" returns whatever the latest available rate is — that is a single rate, not a per-day rate." 2. "Even if the table had daily rates, a manual upload is fragile." 3. "I need an effective-date join and an automated source."

The diagnosis: Two bugs in one place. First, the join collapses all dates to one rate. Second, the rate that one date returns is months old. Either one alone would be wrong; together they produce confidently incorrect output.

The fix:

```
-- Effective-dated join
SELECT o.order_date,
       SUM(o.amount_usd * fx.usd_to_inr) AS revenue_inr
FROM fact_orders o
JOIN dim_fx_rates fx
  ON fx.fx_date = (
    SELECT MAX(fx_date) FROM dim_fx_rates WHERE fx_date <= o.order_date
  )
GROUP BY o.order_date;

-- Automate the load from RBI reference rates daily, and assert freshness
SELECT MAX(fx_date) AS latest_fx_date,
       CURRENT_DATE - MAX(fx_date) AS days_behind
FROM dim_fx_rates;
-- raise if days_behind > 1
```

Why this works: Each order is translated at the rate effective on its own date, not a single point-in-time rate. The freshness check catches the next stale-load incident on day two, not day ninety.

What NOT to say: - "FX moves slowly enough that this does not matter" — not after multi-percent monthly moves. - "Lock in the quarterly rate as policy" — a finance decision, not a data-quality decision.

Common follow-up: *"How would you handle weekends and holidays when there is no fresh rate published?"*

Scenario469 · Exchange Rate Cron Failed Silently for 3 Days

Tags: Difficulty: Medium · Asked at: PhonePe, Razorpay, Cred · Dialect: vendor-agnostic

The scenario:

The daily FX-rate refresh cron has been failing for three days due to a vendor API endpoint change. The cron is configured to exit 0 on HTTP errors so as not to spam the on-call channel. The `dim_fx_rates` table now silently uses three-day-old rates with no alert.

Expected: Daily rate, daily alert if missing. **Actual:** Three-day-old rate, zero alerts.

Thinking out loud: 1. "`exit 0 on failure` is a footgun pattern — it converts loud failures into silent ones." 2. "The signal needs to come from the data, not the cron exit code." 3. "A freshness SLA on the table itself, monitored by a separate scheduled check, is the right pattern."

The diagnosis: The cron design assumed the alerting layer would notice failed runs. It does not, because the cron suppresses non-zero exits. The vendor changed `/api/v1/latest` to `/api/v2/latest` without a deprecation period; the cron has been receiving 404s for three days.

The fix:

```
-- Data-side freshness monitor (runs every hour, independent of the cron)
SELECT
  MAX(fx_date) AS latest_fx_date,
  CURRENT_DATE - MAX(fx_date) AS days_behind,
  CASE WHEN CURRENT_DATE - MAX(fx_date) > 1 THEN 'BREACHED' ELSE 'OK' END AS sla_status
FROM dim_fx_rates;

-- Cron-side fix: stop swallowing errors, page on failure
-- (Shell): set -e ; curl --fail https://vendor/api/v2/latest ...
```

Why this works: A data-side SLA monitor is independent of cron health. If the data is fresh, the cron is by definition working; if the data is stale, something is wrong regardless of cron exit codes.

What NOT to say: - "Make the on-call read the cron logs daily" — humans do not scale, automation does. - "Add a try/except in the cron" — does not help if it already swallows errors.

Common follow-up: *"How would you handle multiple data sources where some are allowed to lag and others are not?"*

Scenario470 · Holiday Calendar Drift (Multiple Regions, Multiple Calendars)

Tags: Difficulty: Hard · Asked at: Swiggy, Zomato, Ola · Dialect: vendor-agnostic

The scenario:

A business-day calculator uses a single holiday calendar. Operations in Kerala observe Onam, in Maharashtra Ganesh Chaturthi, in West Bengal Durga Puja, and in Delhi the national holidays only. SLA calculations on shipments are wrong in every state because the calendar is one-size-fits-all.

```
SELECT shipment_id,  
       business_days_between(dispatch_date, delivery_date) AS bd_to_deliver  
FROM fact_shipments;
```

Expected: A regionally aware business-day count. **Actual:** Single national calendar misses state-specific holidays; SLAs look better or worse than reality.

Thinking out loud: 1. *"Regional holiday observance varies meaningfully — Kerala alone has ~20 public holidays vs Delhi's ~17 with very different dates."* 2. *"A composite calendar dimension keyed by (region, date) is the only correct structure."* 3. *"The function needs to take a region argument."*

The diagnosis: The calendar table is keyed only by date. Functions consuming it cannot distinguish a Kerala-only holiday from a national one. SLA breaches in Kerala under-count when Onam week is treated as business days; in Delhi, over-count when Onam is mistakenly excluded.

The fix:

```

CREATE TABLE dim_holiday_calendar (
  region_code VARCHAR(10) NOT NULL,
  holiday_date DATE NOT NULL,
  holiday_name VARCHAR(100),
  PRIMARY KEY (region_code, holiday_date)
);

-- Region-aware business-day calculation
CREATE OR REPLACE FUNCTION business_days_between(p_from DATE, p_to DATE, p_region VARCHAR)
RETURNS INT AS $$
  SELECT COUNT(*)::INT
  FROM generate_series(p_from, p_to - INTERVAL '1' DAY, INTERVAL '1' DAY) AS d(dt)
  WHERE EXTRACT(DOW FROM dt) NOT IN (0,6) -- exclude Sun, Sat
  AND NOT EXISTS (
    SELECT 1 FROM dim_holiday_calendar h
    WHERE h.region_code IN (p_region, 'IN') -- region-specific OR national
    AND h.holiday_date = dt::DATE
  );
$$ LANGUAGE SQL;

```

Why this works: A composite key (region, date) lets the same calendar table serve every region. National holidays are stored once with `region_code = 'IN'`; state holidays add a row per state that observes them.

What NOT to say: - "Use a third-party API for holidays" — viable, but the data model is still the problem.
- "Just always use the national calendar" — that is precisely the bug.

Common follow-up: "How do you handle moveable feasts like Eid where the date varies year to year?"

Scenario471 · Geo Coordinates Changed Precision (Lossy Conversion)

Tags: Difficulty: Medium · Asked at: Swiggy, Zomato, Dunzo · Dialect: Postgres, MySQL

The scenario:

A geospatial fact table stored latitude and longitude as `DECIMAL(9,6)` (six decimal places, ~10 cm precision). A migration cast them to `DECIMAL(8,4)` (four decimal places, ~11 m precision) to save space. Delivery-distance KPIs shifted upward because pickup and drop points are now snapped onto a coarser grid.

Expected: Stable distance distributions. **Actual:** Average distance per delivery shifted by 2–3 percent without a corresponding operational change.

Thinking out loud: 1. "Loss of three decimal places means ~111 m of imprecision at the equator." 2. "Distance between two coarsely-snapped points is biased — sometimes longer, sometimes shorter, but

the distribution shifts." 3. "This is a one-way migration; original precision cannot be reconstructed without the source."

The diagnosis: The DECIMAL narrowing rounded every coordinate. The migration was approved on storage grounds without modelling its impact on distance calculations. Last week's storage saving cost this week's KPI integrity.

The fix:

```
-- Restore the wider type
ALTER TABLE locations ALTER COLUMN lat TYPE DECIMAL(9,6);
ALTER TABLE locations ALTER COLUMN lng TYPE DECIMAL(9,6);

-- Reload historical coordinates from the upstream source (e.g., the GPS event log)
UPDATE locations l
SET lat = src.lat, lng = src.lng
FROM gps_events_raw src
WHERE src.location_id = l.location_id;

-- Consider native geospatial types for new design
-- Postgres: GEOGRAPHY(POINT, 4326)
-- MySQL 8+: POINT SRID 4326
```

Why this works: Native geospatial types use double-precision floats internally and avoid the DECIMAL-narrowing trap. Reload from the source recovers true precision.

What NOT to say: - "Just multiply by 100 and round" — that does not recover lost bits. - "DECIMAL(8,4) is enough for our use case" — it visibly is not, the KPI moved.

Common follow-up: "How would you validate a precision change before merging it?"

Scenario472 · Sensor Data Drift (Calibration Change)

Tags: Difficulty: Hard · Asked at: IoT-heavy fintechs, logistics, manufacturing platforms · Dialect: vendor-agnostic

The scenario:

A warehouse temperature sensor was re-calibrated last month after vendor recall. Pre-recall readings ran 1.8 °C high; post-recall readings are accurate. The cold-chain compliance report flags pre-recall periods as "all in spec" when they were not, and post-recall periods as "occasionally out of spec" when they are tightly controlled.

```
SELECT sensor_id, reading_ts, temp_c
FROM fact_sensor_readings
WHERE sensor_id = 'WH-BLR-12';
```

Expected: A consistent measurement scale across time. **Actual:** A 1.8 °C step at the calibration date; downstream "in spec" determinations are wrong on both sides.

Thinking out loud: 1. "Calibration is not just a future-facing change — it implies an offset to past readings if we want a single comparable series." 2. "I need a `dim_sensor_calibration` table that records calibration epochs and offsets." 3. "All historical reads must be normalised by joining to this table."

The diagnosis: The sensor recall came with a calibration certificate documenting a +1.8 °C bias. The certificate was filed; no one fed it into the warehouse. The fact table contains uncorrected raw readings.

The fix:

```
CREATE TABLE dim_sensor_calibration (
  sensor_id VARCHAR(20) NOT NULL,
  effective_from TIMESTAMPTZ NOT NULL,
  effective_to TIMESTAMPTZ NOT NULL,
  offset_c DECIMAL(5,2) NOT NULL,
  PRIMARY KEY (sensor_id, effective_from)
);

INSERT INTO dim_sensor_calibration VALUES
  ('WH-BLR-12', '2025-01-01', '2026-04-15', -1.80, /* pre-recall, subtract 1.8 */),
  ('WH-BLR-12', '2026-04-15', '9999-12-31', 0.00 /* post-recall, no offset */);

-- Corrected view
CREATE VIEW fact_sensor_readings_corrected AS
SELECT s.sensor_id, s.reading_ts, s.temp_c + c.offset_c AS temp_c_corrected, s.temp_c AS temp_c_raw
FROM fact_sensor_readings s
JOIN dim_sensor_calibration c
  ON c.sensor_id = s.sensor_id
  AND s.reading_ts >= c.effective_from
  AND s.reading_ts < c.effective_to;
```

Why this works: Raw readings are preserved (forensics), corrected readings are derived (reporting). Future calibration events add rows without rewriting history.

What NOT to say: - "Just subtract 1.8 from all old readings" — destroys the raw value. - "Throw out pre-recall data" — over a year of cold-chain history would be lost.

Common follow-up: "How would you back-test the corrected series against ground-truth thermometers if any are available?"

Scenario473 · Reference Data Table Updated Without Version Tracking

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Cred · Dialect: vendor-agnostic

The scenario:

The `dim_merchant_category_code` table is updated in place via direct UPDATE statements when classifications change. A merchant moved from `5411 – Grocery` to `5812 – Restaurant` last month. All historical transactions for that merchant now look like restaurant transactions. Year-over-year analyses are corrupted.

```
SELECT mcc, SUM(amount_inr)
FROM fact_transactions t
JOIN dim_merchant_category_code mcc_dim ON mcc_dim.merchant_id = t.merchant_id
WHERE txn_date BETWEEN '2025-01-01' AND '2026-04-30'
GROUP BY mcc;
```

Expected: Each transaction grouped by the MCC that was effective when it happened. **Actual:** Every transaction grouped by today's MCC.

Thinking out loud: 1. "This is the textbook reason for Slowly Changing Dimensions Type 2." 2. "In-place UPDATE on a dimension destroys point-in-time accuracy for every fact that references it." 3. "I need an effective-dated dimension."

The diagnosis: The team treated the dimension as a lookup, not as a time-series. There is no `effective_from / effective_to` on the dimension and no surrogate key. The fact join resolves to whatever the dimension says now.

The fix:

```
-- Convert to SCD Type 2
CREATE TABLE dim_merchant_category_code_scd2 (
  mcc_surrogate BIGSERIAL PRIMARY KEY,
  merchant_id INT NOT NULL,
  mcc CHAR(4) NOT NULL,
  effective_from DATE NOT NULL,
  effective_to DATE NOT NULL DEFAULT '9999-12-31',
  is_current BOOLEAN NOT NULL DEFAULT TRUE,
  UNIQUE (merchant_id, effective_from)
);

-- Fact joins on the effective window
SELECT d.mcc, SUM(t.amount_inr)
FROM fact_transactions t
JOIN dim_merchant_category_code_scd2 d
  ON d.merchant_id = t.merchant_id
  AND t.txn_date >= d.effective_from
  AND t.txn_date < d.effective_to
WHERE t.txn_date BETWEEN '2025-01-01' AND '2026-04-30'
GROUP BY d.mcc;
```

Why this works: Each historical transaction joins to the MCC that was in force on its own date. Updating a merchant's MCC creates a new row, never overwriting the old one.

What NOT to say: - "Add a snapshot table" — possible, but SCD2 is the canonical answer to this problem. - "Store the MCC on the fact" — denormalises but works; SCD2 is more flexible.

Common follow-up: *"What is the difference between SCD Type 1, Type 2, and Type 6, and when do you reach for each?"*

Scenario474 · Slowly Changing Dimension Type 1 Overwrote History Needed

Tags: Difficulty: Hard · Asked at: Flipkart, Myntra, Nykaa · Dialect: vendor-agnostic

The scenario:

Customer address is stored as SCD Type 1 — in-place overwrite. A logistics post-mortem asks "what address did we ship to on March 12?" but the warehouse only knows today's address. The customer moved on March 20.

Expected: The address effective at the time of shipment. **Actual:** Today's address; historical shipping context is unrecoverable.

Thinking out loud: 1. *"Type 1 dimensions overwrite history. They are correct for attributes that have no temporal meaning (e.g., a corrected typo)."* 2. *"For anything where 'what was it on date X' is a question*

someone will ask, Type 2 is the answer." 3. "I can recover some history from the shipment table if shipping address was denormalised onto the shipment row."

The diagnosis: The original modelling team chose Type 1 to keep the dimension small. Address was treated as a "current state" attribute. Operations now needs historical accuracy for dispute resolution.

The fix:

```
-- Recover what is recoverable from the shipment row itself (denorm address)
SELECT s.shipment_id, s.shipped_to_address, s.ship_date
FROM fact_shipments s
WHERE s.customer_id = 12345 AND s.ship_date = '2026-03-12';

-- Going forward, convert dim_customer_address to SCD2
CREATE TABLE dim_customer_address_scd2 (
  address_sk BIGSERIAL PRIMARY KEY,
  customer_id INT NOT NULL,
  address_line1 VARCHAR(200),
  city VARCHAR(80),
  pincode VARCHAR(6),
  effective_from DATE NOT NULL,
  effective_to DATE NOT NULL DEFAULT '9999-12-31'
);

-- Capture address-at-time-of-event on every transactional fact
ALTER TABLE fact_shipments ADD COLUMN ship_address_sk BIGINT
REFERENCES dim_customer_address_scd2 (address_sk);
```

Why this works: Two defences: SCD2 going forward so address history survives, plus a denormalised foreign key on the fact so the answer to "what address on day X" is a single column read.

What NOT to say: - "Address never matters historically" — it does, for disputes, refunds, fraud investigations. - "Convert all dimensions to SCD2" — overkill; choose based on temporal relevance.

Common follow-up: "Which attributes on dim_customer warrant Type 2 vs Type 1?"

Scenario475 · Late-Arriving Dimension Confused Fact-Table Joins

Tags: Difficulty: Hard · Asked at: Swiggy, Zomato, Razorpay · Dialect: vendor-agnostic

The scenario:

A new merchant signs up on March 1 and processes 200 transactions in the first hour. The dimension load runs at midnight, so the merchant does not exist in `dim_merchant` until March

2. Every transaction from March 1 joins to NULL on the merchant side; the dashboard shows 200 transactions for "Unknown Merchant."

```
SELECT m.business_name, COUNT(*)
FROM fact_transactions t
JOIN dim_merchant m ON m.merchant_id = t.merchant_id
WHERE t.txn_date = '2026-03-01'
GROUP BY m.business_name;
```

Expected: Every transaction attributed to its merchant. **Actual:** New-merchant transactions appear under NULL / are dropped by INNER JOIN.

Thinking out loud: 1. "Late-arriving dimensions are a classic data-warehouse problem." 2. "Two patterns: (a) insert a placeholder row with `merchant_id` only and back-fill attributes when they arrive, or (b) outer-join and tolerate NULLs in the report." 3. "Pattern (a) preserves fact integrity; pattern (b) admits the gap on every read."

The diagnosis: The fact-table load runs hourly; the dimension load runs nightly. Transactions for merchants registered today arrive at the warehouse 24 hours before their dimension row does. The INNER JOIN drops them; the LEFT JOIN preserves them but with NULL attributes.

The fix:

```
-- Pattern A: create a placeholder dim row at fact-load time
INSERT INTO dim_merchant (merchant_id, business_name, load_status)
SELECT DISTINCT t.merchant_id, '(pending dimension load)', 'pending'
FROM fact_transactions t
LEFT JOIN dim_merchant m ON m.merchant_id = t.merchant_id
WHERE m.merchant_id IS NULL
      AND t.load_ts >= NOW() - INTERVAL '1' HOUR;

-- The nightly dim load then upserts real attributes onto the placeholder rows
MERGE INTO dim_merchant d
USING staging_merchant s ON d.merchant_id = s.merchant_id
WHEN MATCHED THEN UPDATE SET business_name = s.business_name,
                             load_status = 'loaded',
                             ...
WHEN NOT MATCHED THEN INSERT (merchant_id, business_name, load_status, ...) VALUES (...);

-- Sanity check: anything still 'pending' after 48h is investigated
SELECT merchant_id, load_status, first_seen_ts
FROM dim_merchant
WHERE load_status = 'pending' AND first_seen_ts < NOW() - INTERVAL '48' HOUR;
```

Why this works: Placeholder rows keep the join working on day one. The MERGE on the nightly load fills in the real attributes. The watchdog query catches stragglers.

What NOT to say: - "Run the dimension load every minute" — fixes the race for the cost of more upstream load. - "Drop new merchants from the report until tomorrow" — silently under-counts revenue.

Common follow-up: *"How would you handle a late-arriving SCD Type 2 dimension where the historical window changes after the fact?"*

Category 7, Part D — Data Quality: Timezone, Encoding, Locale & NULL Issues

The last bucket of DQ — the boring-but-painful issues that cost real money in production. Timezones split orders across days and make Monday's revenue spill into Sunday's report. Encoding silently breaks string equality so "Mumbai" and "Mumbai" stop joining when one row came in as Latin1 and the other as UTF-8. NULL semantics double-count or miss rows depending on which clause they hide in, and downstream Excel readers turn them into zeroes without anyone noticing. The interviewer asks these because you'll hit them in week one — a cron at 23:59 IST will silently process "today" as yesterday's UTC date, a payments file with a trailing space won't reconcile, a `COUNT(col)` will silently change the denominator of a key metric, and the CFO will ask you why the dashboard moved 4%. The 25 scenarios below walk through the symptom, the diagnosis, and the fix you'd narrate on a screen-share — including the dialect quirks that make these issues so dialect-dependent.

Scenario476 · Daily Order Count Off By One Day For Some Users

Tags: Difficulty: Medium · Asked at: Flipkart, Swiggy, Zomato · Dialect: vendor-agnostic, with PostgreSQL/MySQL call-outs

The scenario:

The product team runs a daily report grouping `orders.created_at` by date. For most users the numbers tie out, but a subset of orders consistently appear on the wrong calendar day — yesterday's order shows up in today's bucket. The pattern: it's always orders placed late at night IST.

```
SELECT DATE(created_at) AS order_date, COUNT(*) AS orders
FROM orders
WHERE DATE(created_at) = CURRENT_DATE
GROUP BY DATE(created_at);
```

Expected: Today's IST orders grouped under today's date. **Actual:** Orders placed between 00:00 and 05:29 IST land in yesterday's bucket.

Thinking out loud: 1. "The orders table almost certainly stores `created_at` in UTC — that's the AWS-RDS default." 2. " `DATE(created_at)` extracts the UTC date, not the IST date." 3. "An order placed at 02:00 IST is 20:30 UTC the previous day — so `DATE()` returns yesterday." 4. "The fix is to convert to IST before extracting the date."

The diagnosis: The column is stored as UTC (either `timestamp without time zone` interpreted as UTC, or `timestampz`). Calling `DATE()` on it extracts the UTC calendar date. India is UTC+5:30, so orders placed in the first 5.5 hours of each IST day are still on the previous UTC date. The grouping then bucketing them yesterday is correct from the database's point of view but wrong from the business's.

The fix (PostgreSQL with `timestampz`):

```
SELECT (created_at AT TIME ZONE 'Asia/Kolkata')::date AS order_date,
       COUNT(*) AS orders
FROM orders
WHERE (created_at AT TIME ZONE 'Asia/Kolkata')::date = CURRENT_DATE
GROUP BY (created_at AT TIME ZONE 'Asia/Kolkata')::date;
```

In MySQL: `DATE(CONVERT_TZ(created_at, '+00:00', '+05:30'))`.

Why this works: `AT TIME ZONE` shifts the instant onto the IST clock before the date is extracted, so an IST-evening order stays on its IST date. Make sure `CURRENT_DATE` on the right side is also IST-based — either set the session timezone or wrap it the same way.

What NOT to say: - "Just add 5.5 hours to `created_at`" — works until DST in a multi-region app and lies if you ever onboard a non-IST tenant. - "Store `created_at` as IST going forward" — fixes new rows, breaks historical comparisons and breaks the first time you serve a Singapore customer.

Common follow-up: "How would you avoid hand-writing `AT TIME ZONE 'Asia/Kolkata'` in every query in the codebase?"

Scenario477 · Cron At 23:59 IST Processing "Today" But DB Is UTC

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Paytm · Dialect: vendor-agnostic

The scenario:

A reconciliation cron runs at 23:59 IST every night. It computes "today's" settlements by filtering `WHERE settlement_date = CURRENT_DATE`. The job is occasionally missing settlements

that the operations team can see in the UI, and the mismatch always happens on the last 30 minutes of the day.

```
SELECT merchant_id, SUM(amount) AS settled_today
FROM settlements
WHERE settlement_date = CURRENT_DATE
GROUP BY merchant_id;
```

Expected: All settlements that show "today" in the UI. **Actual:** Settlements created in the last ~30 minutes of the IST day are missing.

Thinking out loud: 1. "23:59 IST is 18:29 UTC — `CURRENT_DATE` in a UTC session is still yesterday's UTC date." 2. "But `settlement_date` was stamped using the application's IST clock, so it's today's IST date." 3. "Filter is comparing a UTC date to an IST date. Mismatch by one for the IST evening." 4. "And the run is starting at 23:59 — if a settlement lands at 23:59:45, the run probably already started, double-trouble."

The diagnosis: Two bugs stacked. First, `CURRENT_DATE` resolves against the session timezone (UTC on most managed databases), while `settlement_date` was written using IST. Second, a cron that runs at the boundary is racing the data it's trying to summarise.

The fix:

```
SELECT merchant_id, SUM(amount) AS settled_today
FROM settlements
WHERE settlement_date = (CURRENT_TIMESTAMP AT TIME ZONE 'Asia/Kolkata')::date
GROUP BY merchant_id;
```

And schedule the cron at 00:15 the *next* day, processing `today - 1`, not at 23:59 of the current day.

Why this works: The filter and the data now agree on what "today" means. Running the job after the day has fully closed removes the race.

What NOT to say: - "We'll subtract a day in the WHERE clause." — fragile, and now you have to remember which side has the offset. - "Just run the cron more often." — masks the boundary bug rather than fixing it.

Common follow-up: "Could you have caught this in CI? What would the test look like?"

Scenario478 · DST Transition Losing An Hour Of Events

Tags: Difficulty: Hard · Asked at: Multinationals operating in US/EU · Dialect: vendor-agnostic, PostgreSQL examples

The scenario:

Your event pipeline ingests clickstream from a US-based partner. Every March, the partner's daily counts dip noticeably for a 24-hour window. You're tasked with explaining the dip before the QBR. The data is real, not corrupted.

```
SELECT date_trunc('hour', event_time) AS hr, COUNT(*)
FROM events
WHERE event_time >= '2026-03-08'
      AND event_time < '2026-03-09'
GROUP BY 1
ORDER BY 1;
```

Expected: 24 hourly buckets for that day. **Actual:** 23 buckets — the 2 AM bucket is missing for that one day.

Thinking out loud: 1. "March is when US DST springs forward — clocks jump from 02:00 to 03:00." 2. "If `event_time` is local US time, there is no 02:00 row that day — the hour doesn't exist." 3. "My hourly bucketing didn't fail, it correctly produced a non-existent hour with zero rows."

The diagnosis: The events are timestamped in the partner's local civil time (US/Eastern). On the spring-forward Sunday, 02:00–02:59 local does not exist. There's no data to be lost — there is no such hour — but reports comparing year-over-year see "23 hours" of activity.

The fix: Always store and reason in UTC. If display in local time matters, convert at query time and document the discontinuity:

```
SELECT date_trunc('hour', event_time_utc AT TIME ZONE 'America/New_York') AS hr_local,
       COUNT(*)
FROM events
WHERE event_time_utc >= '2026-03-08 05:00:00+00'   -- 00:00 EST
      AND event_time_utc < '2026-03-09 05:00:00+00'   -- 00:00 EDT next day
GROUP BY 1
ORDER BY 1;
```

Why this works: UTC has no DST. Year-over-year comparisons stay honest; the missing local hour is a presentation artefact, not a data gap.

What NOT to say: - "Linearly interpolate the missing hour." — invents data. - "We lost an hour of data, file a partner ticket." — there is no data to recover.

Common follow-up: "What about the fall-back, when 01:00–01:59 happens twice in the same local day?"

Scenario479 · Multi-Region App, Single DB Timezone, Regional Reports Wrong

Tags: Difficulty: Medium · Asked at: Freshworks, Zoho, Postman · Dialect: vendor-agnostic

The scenario:

A SaaS product serves customers in India, the US, and Germany. The database is in UTC. Every regional sales lead is complaining that "today" on their dashboard doesn't match their local "today".

```
SELECT COUNT(*) AS leads_today
FROM crm_leads
WHERE created_at::date = CURRENT_DATE
      AND tenant_country = :country;
```

Expected: Each lead sees the count for their local "today". **Actual:** Everyone sees the UTC count, which is wrong for everyone.

Thinking out loud: 1. "Single DB timezone, three audiences. The query is ignoring the audience entirely." 2. "I need a per-tenant timezone, applied at query time." 3. "And `CURRENT_DATE` on the right side has to be in the same timezone."

The diagnosis: A single global filter cannot serve three different "todays". The query needs to know the tenant's timezone (stored on `tenants`) and apply it both to the timestamp column and to the boundary expression.

The fix:

```
SELECT COUNT(*) AS leads_today
FROM crm_leads l
JOIN tenants t ON t.tenant_id = l.tenant_id
WHERE (l.created_at AT TIME ZONE t.timezone)::date
      = (CURRENT_TIMESTAMP AT TIME ZONE t.timezone)::date
      AND t.country = :country;
```

Why this works: Each tenant's "today" is computed against its own timezone, so the comparison is apples to apples for that tenant. Storing the IANA name (`Asia/Kolkata` , not `IST`) on `tenants` keeps DST correct for the regions that have it.

What NOT to say: - "Run three separate dashboards in three separate timezones." — works but doesn't scale to 50 tenants. - "Store everything in local time." — sounds tidy, breaks the moment a customer moves regions or expands.

Common follow-up: "What if a single user belongs to multiple tenants in different timezones?"

Scenario480 · "Yesterday's" Report Different Depending On User's Locale

Tags: Difficulty: Medium · Asked at: Razorpay, Stripe-India, Zoho · Dialect: vendor-agnostic

The scenario:

A daily email goes out at 09:00 IST with "yesterday's" revenue. Customer-success reps in different cities see different totals when they open the dashboard at the same moment — Bangalore and Delhi agree, but the Singapore office (UTC+8) disagrees.

```
SELECT SUM(amount) AS revenue
FROM transactions
WHERE created_at::date = CURRENT_DATE - INTERVAL '1 day';
```

Expected: One "yesterday" for the whole org. **Actual:** Each session timezone produces a different "yesterday".

Thinking out loud: 1. " `CURRENT_DATE` follows session timezone — a session in SGT and one in IST disagree." 2. "There's no single answer to 'yesterday' across timezones without a reference point." 3. "The business wants 'yesterday in IST' for India ops — make that explicit, not implicit."

The diagnosis: "Yesterday" is timezone-dependent. The query, the data, and the reporter must agree on a single canonical zone. Right now each session implicitly imposes its own answer.

The fix: Pin "yesterday" to the business calendar (IST for an India ops report):

```
WITH ist_today AS (
  SELECT (CURRENT_TIMESTAMP AT TIME ZONE 'Asia/Kolkata')::date AS d
)
SELECT SUM(amount) AS revenue
FROM transactions t, ist_today
WHERE (t.created_at AT TIME ZONE 'Asia/Kolkata')::date = ist_today.d - 1;
```

Why this works: Whoever runs the query, wherever they sit, gets the same "IST yesterday." The output is deterministic.

What NOT to say: - "Set the session timezone in the BI tool." — works for that one tool, fragile across users. - "Send three different emails." — solves the symptom, not the inconsistency.

Common follow-up: "What if the head of sales in Singapore wants SGT-based 'yesterday' on the same email?"

Scenario481 · Same String Fails JOIN — Collation Mismatch

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Wipro — also product cos. inheriting legacy MS-SQL · Dialect: SQL Server / MySQL specific

The scenario:

You join `orders.customer_email` to `users.email` and you get zero rows for some legitimate matches. The strings are visibly identical. You copy-paste them into a comparison and get `True`. But the join still drops them.

```
SELECT u.user_id, o.order_id
FROM orders o
JOIN users u ON o.customer_email = u.email;
```

Expected: Match on every shared email. **Actual:** A subset of perfectly-equal-looking emails don't match.

Thinking out loud: 1. "Visually identical strings, JOIN fails. That smells like collation." 2. "One column is `utf8_bin`, the other `utf8_general_ci` — `bin` treats case as significant; `ci` doesn't." 3. "When two collations meet in a comparison and they disagree on case sensitivity, the engine sometimes refuses or silently picks the stricter one."

The diagnosis: The two columns have different collations. In MySQL the engine may either error out or coerce in a way that changes equality behaviour; in SQL Server you get an "Cannot resolve the collation conflict" error or, worse, a silent mismatch when explicit `COLLATE` clauses are added without thought.

The fix (vendor-agnostic — pick one):

```
SELECT u.user_id, o.order_id
FROM orders o
JOIN users u
  ON LOWER(TRIM(o.customer_email)) = LOWER(TRIM(u.email));
```

Or pin the collation explicitly:

```
-- MySQL
ON o.customer_email COLLATE utf8mb4_general_ci = u.email COLLATE utf8mb4_general_ci
```

Why this works: Both sides are now normalised by the same function (or the same collation), so equality means the same thing on both sides.

What NOT to say: - "Just ALTER one of the columns to match." — okay long-term, terrible for hotfixing a join. - "Convert one to BINARY." — opposite of what you want; makes it case-sensitive.

Common follow-up: "What's the performance cost of LOWER() in the JOIN, and how would you keep the index usable?"

Scenario482 · "Mumbai" vs "mumbai" vs "MUMBAI" As Different Cities

Tags: Difficulty: Easy · Asked at: Swiggy, Zomato, Dunzo · Dialect: vendor-agnostic

The scenario:

A logistics dashboard groups deliveries by city. The city dropdown shows "Mumbai", "mumbai", "MUMBAI", "Mumbai ", and "mumbai-india" as five distinct cities. The product manager is upset.

```
SELECT city, COUNT(*) AS deliveries
FROM deliveries
GROUP BY city
ORDER BY deliveries DESC;
```

Expected: One row for "Mumbai". **Actual:** Five rows, three of which are casing/whitespace variants.

Thinking out loud: 1. "The column is free-text and was never normalised on insert." 2. "GROUP BY treats each variant as distinct because the bytes are distinct." 3. "Either fix on write (preferred) or normalise on read for the report."

The diagnosis: City strings came in from multiple ingestion paths (mobile app, partner API, manual data entry) with no normalisation contract. The grouping is correct given the data; the data is the bug.

The fix (read-side, quick win):

```
SELECT INITCAP(LOWER(TRIM(city))) AS city_clean, COUNT(*) AS deliveries
FROM deliveries
GROUP BY INITCAP(LOWER(TRIM(city)))
ORDER BY deliveries DESC;
```

Write-side (the actual fix): a normalised `city_id` foreign key into a curated `cities` table, with the free-text field still retained for audit.

Why this works: Read-side collapses variants for the report; write-side prevents the next variant from appearing.

What NOT to say: - "We'll add a CHECK constraint listing every valid city." — unmaintainable for 4,000 Indian cities. - "Let the BI tool clean it up." — happens once, then breaks the next week.

Common follow-up: "How would you migrate the historical free-text column to the normalised foreign key?"

Scenario483 · Latin1 vs UTF-8 Silent Corruption On Insert

Tags: Difficulty: Hard · Asked at: Older enterprise stacks, legacy MySQL shops · Dialect: MySQL-heavy

The scenario:

A name field stores Indian names with diacritics ("Pooja Khannã"). Some rows render correctly, others display as "Pooja Khannã" or "????". The corruption is consistent per-source: the mobile app's rows are fine, the partner CSV import's rows are mangled.

```
SELECT user_id, full_name FROM users WHERE user_id IN (101, 202);
-- 101: 'Pooja Khanna' (correct)
-- 202: 'Pooja Khannã' (mojibake)
```

Expected: Both render correctly. **Actual:** Bytes from one path were interpreted as Latin1 when they were UTF-8 (or vice versa) and were silently re-encoded on the way in.

Thinking out loud: 1. "Mojibake is the calling card of double-encoding or wrong-encoding-on-write." 2. "Different sources, different corruption — the CSV import declared the wrong encoding." 3. "At read time the bytes are wrong, no SQL fix can recover them perfectly."

The diagnosis: The CSV import opened the file as Latin1 but the file was UTF-8. Each multi-byte UTF-8 character got expanded into two Latin1 characters, which were then re-encoded as UTF-8 for storage. The result is mojibake bytes that don't decode back cleanly.

The fix: - Future imports: declare `SET NAMES utf8mb4; LOAD DATA ... CHARACTER SET utf8mb4` . - Historical: attempt a best-effort repair: `sql UPDATE users SET full_name = CONVERT(CAST(CONVERT(full_name USING latin1) AS BINARY) USING utf8mb4) WHERE full_name LIKE '%Ã%' OR full_name LIKE '%â%';` And spot-check before committing.

Why this works: The repair undoes the wrong decode-then-encode chain. The cure is fragile, so the real fix is to set encoding correctly at every ingress.

What NOT to say: - "I'll just delete the bad rows." — loses data, doesn't fix the source. - "Strip non-ASCII." — discards every Indian name with a diacritic.

Common follow-up: "What's your check-list before turning on a new ingestion pipeline?"

Scenario484 · Emoji Breaks MySQL utf8 (Need utf8mb4)

Tags: Difficulty: Easy · Asked at: Consumer apps with chat features · Dialect: MySQL specific

The scenario:

A chat feature throws "Incorrect string value: '\xF0\x9F\x98\x80' for column 'message'" on certain inserts. The user is typing an emoji. The schema declares `utf8`.

```
INSERT INTO chat_messages (room_id, body) VALUES (10, 'great call 🤔');  
-- ERROR 1366 (HY000): Incorrect string value
```

Expected: Insert succeeds, emoji round-trips. **Actual:** Error or silent truncation at the emoji.

Thinking out loud: 1. "MySQL's `utf8` is historically a 3-byte alias — it can't store 4-byte code points." 2. "Emoji is in the supplementary planes, needs 4 bytes — that's `utf8mb4`." 3. "The fix is column charset, connection charset, and server-side default — all three."

The diagnosis: MySQL's `utf8` is the legacy 3-byte variant; full Unicode requires `utf8mb4`. Emoji, several historical Indic combining sequences, and many uncommon symbols need the 4-byte range.

The fix:

```
ALTER TABLE chat_messages  
  CONVERT TO CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;  
  
-- Also pin the connection:  
SET NAMES utf8mb4;  
  
-- And the server / database defaults:  
ALTER DATABASE myapp CHARACTER SET = utf8mb4 COLLATE = utf8mb4_0900_ai_ci;
```

Why this works: All three layers — server, connection, column — now speak the same 4-byte-capable charset, so emoji and rare characters survive end to end.

What NOT to say: - "Strip emojis at the app layer." — product hostile and breaks user content. - "Use BLOB." — bypasses the charset entirely; loses the ability to text-search, sort, or collate.

Common follow-up: "How would you migrate a 200 GB messages table to utf8mb4 with minimal downtime?"

Scenario485 · Trailing Whitespace Defeating Equality

Tags: Difficulty: Easy · Asked at: Most data engineering interviews · Dialect: vendor-agnostic, with SQL Server / Oracle call-outs

The scenario:

A reconciliation job matches `payments.reference_id` to `bank_settlements.reference_id`. About 2% of payments fail to match. When you eyeball them in a query result, they look identical. The CFO has been asking about the 2% for a week.

```
SELECT p.payment_id
FROM payments p
LEFT JOIN bank_settlements s ON s.reference_id = p.reference_id
WHERE s.reference_id IS NULL;
```

Expected: Zero unmatched. **Actual:** 2% unmatched, all visually identical to a matched row.

Thinking out loud: 1. "Invisible characters — trailing space, leading space, NBSP, zero-width-space." 2. "`LENGTH(reference_id)` will tell me — if it differs from `LENGTH(TRIM(reference_id))`, gotcha." 3. "In CHAR columns SQL Server pads; in VARCHAR it doesn't. Either could bite."

The diagnosis: The bank settlement file has trailing spaces on the reference. `=` in standard SQL is case- and whitespace-sensitive. SQL Server's `=` actually treats trailing spaces as equal for VARCHAR, which complicates the picture — but on a join from a true `CHAR(20)` to a `VARCHAR(20)` with explicit collation overrides, the behaviour is unreliable.

The fix:

```
SELECT p.payment_id
FROM payments p
LEFT JOIN bank_settlements s
  ON TRIM(s.reference_id) = TRIM(p.reference_id)
WHERE s.reference_id IS NULL;
```

And add a write-time normaliser so the file's trailing spaces never enter the table.

Why this works: `TRIM` strips all leading/trailing whitespace on both sides, making the comparison purely on content.

What NOT to say: - "Set ANSI_PADDING off." — fixes one corner of one dialect, makes future joins surprise everyone. - "Compare with LIKE." — slower and still wrong if the trailing chars include `_` or `%`.

Common follow-up: "What about NBSP (U+00A0) — does TRIM catch it?"

Scenario486 · CHAR vs VARCHAR Padding Inflating Storage

Tags: Difficulty: Medium · Asked at: Database-focused interviews, Oracle/DB2 shops · Dialect: Oracle/DB2 examples, MySQL/Postgres comparison

The scenario:

A `status_code CHAR(20)` column was used to store values like 'OK' or 'FAILED'. The table is 4x larger than the size predicted by row count × column count of meaningful data. Storage budget overrun.

```
SELECT status_code, LENGTH(status_code)
FROM transactions
LIMIT 5;
-- 'OK                ' 20
-- 'FAILED             ' 20
```

Expected: `LENGTH('OK') = 2` . **Actual:** `CHAR(20)` pads to 20 on every row; LENGTH returns the padded value in some dialects.

Thinking out loud: 1. " `CHAR(N)` is fixed-width — every row is stored as N bytes regardless of content." 2. "For short enum-ish values that's huge waste." 3. "And comparing `CHAR` to `VARCHAR` invites the trailing-space ambiguity from the previous scenario."

The diagnosis: `CHAR(20)` allocates 20 bytes per value (or 20 character-widths after charset, often more). For a status code, this is a 10x storage and IO penalty. The choice of `CHAR` was historical — likely copied from an old COBOL-era schema.

The fix: Migrate to `VARCHAR` (or `TEXT` in Postgres):

```
ALTER TABLE transactions
  ALTER COLUMN status_code TYPE VARCHAR(20)
  USING TRIM(status_code);
```

For a true enum, use a lookup table or your dialect's `ENUM` type if applicable.

Why this works: `VARCHAR` stores only the actual length plus a small length prefix, eliminating the padding waste. The `TRIM` during alter cleans up the existing padding.

What NOT to say: - "It doesn't matter, disk is cheap." — compressed backups, IO bandwidth, and cache hit ratios still suffer. - "Just shrink CHAR to CHAR(6)." — still pads to 6, still wastes on 2-char values.

Common follow-up: *"In Postgres, does VARCHAR(20) actually save space versus TEXT?"*

Scenario487 · Currency: ₹ vs INR vs Rs As Three Representations

Tags: Difficulty: Easy · Asked at: Razorpay, PhonePe, Cashfree · Dialect: vendor-agnostic

The scenario:

The amount field is numeric but a sibling `display_amount` field is free-text. Some rows show "₹500", others "INR 500", others "Rs. 500", others "500/-". A search-by-amount feature returns wildly different results depending on which prefix the user typed.

```
SELECT * FROM invoices
WHERE display_amount LIKE '%500%';
-- Returns rows for 500, 1500, 2500, 50000...
```

Expected: A single normalised representation, or no free-text at all. **Actual:** Four representations of the same value, plus a LIKE-search bug on top.

Thinking out loud: 1. "Currency should never live in display text in a database — it's a derived value." 2. "Store amount + currency code separately, render at the edge." 3. "The search bug is also LIKE-based; substring matching on numbers is dangerous (500 matches 5000)."

The diagnosis: Two conflated problems. The first is that human-readable currency strings should never be the source of truth — they should be derived from `amount_minor` (integer paise) and `currency_code` ('INR') at presentation time. The second is that LIKE on a number-as-string is a substring search, which matches all "500"-containing values.

The fix:

```
ALTER TABLE invoices
  ADD COLUMN amount_minor BIGINT NOT NULL DEFAULT 0,
  ADD COLUMN currency_code CHAR(3) NOT NULL DEFAULT 'INR';

-- Drop display_amount once views are migrated to render from the canonical columns.

SELECT * FROM invoices
WHERE amount_minor = 50000 -- ₹500 stored as paise
  AND currency_code = 'INR';
```

Why this works: Numeric equality on integers is unambiguous and fast. The display string is computed at the edge (`"₹" || (amount_minor / 100.0)`) and never enters the WHERE clause.

What NOT to say: - "Strip the symbol with regex." — works until someone types "RS.500" with no space. - "Store as DECIMAL with the symbol." — DECIMAL can't hold a symbol; the symbol then leaks back into a text column.

Common follow-up: "Why minor units (paise) instead of `DECIMAL(10,2)` ?"

Scenario488 · Number Formatting Locale: "1,000.50" vs "1.000,50"

Tags: Difficulty: Medium · Asked at: Multinational SaaS, fintech with EU customers · Dialect: vendor-agnostic

The scenario:

A CSV upload accepts amounts. Indian users type "1,00,000" (lakh comma grouping) and German users paste "1.000,50". The pipeline parses both with the same logic and produces garbage for one of them.

```
INSERT INTO refunds(amount) VALUES (CAST('1.000,50' AS DECIMAL(10,2)));
-- ERROR or unexpected 1.00050
```

Expected: Parsed correctly per locale. **Actual:** One locale parses fine, the other corrupts.

Thinking out loud: 1. "The database doesn't know about locale — it expects `1234.56` plain." 2. "Locale-specific parsing must happen at the app/import layer, before the cast." 3. "Storage is always locale-free; display is locale-aware."

The diagnosis: SQL `CAST(string AS DECIMAL)` uses the database's invariant numeric format (`.` as decimal, no thousands separator). Locale formatting must be stripped or interpreted *before* the SQL boundary.

The fix: Normalise at ingestion (pseudocode for the import layer):

```
amount = locale_parse(raw, user_locale) # returns 1000.50 as float/Decimal
```

Then insert the plain number:

```
INSERT INTO refunds(amount) VALUES (1000.50);
```

Inside SQL, when stuck with locale-formatted strings, normalise explicitly:

```
-- Indian: remove commas
SELECT CAST(REPLACE('1,00,000', ',', '' ) AS DECIMAL(12,2));

-- German: replace dots with empty, comma with dot
SELECT CAST(REPLACE(REPLACE('1.000,50', '.', ''), ',', '' ) AS DECIMAL(12,2));
```

Why this works: The database sees a single canonical numeric format, eliminating ambiguity. Locale-handling lives where locale is known: at the user's edge.

What NOT to say: - "Force everyone to use Indian formatting." — alienates users. - "Try both parses and pick the bigger one." — silent corruption waiting to happen.

Common follow-up: *"How would you detect a locale mismatch in incoming data?"*

Scenario489 · Phone Number Stored With/Without Country Code Mixed

Tags: Difficulty: Easy · Asked at: Swiggy, Zomato, Ola, Uber-India · Dialect: vendor-agnostic

The scenario:

The user table has `phone` stored as a string. Some are `9876543210`, some `+919876543210`, some `91 9876543210`, some `09876543210`. Lookup by phone, OTP routing, and dedup all break in inconsistent ways.

```
SELECT COUNT(*) FROM users WHERE phone = '9876543210';
-- Returns 1, but the same person also has rows for '+919876543210' and '919876543210'.
```

Expected: One canonical phone format per user. **Actual:** Five representations of the same number co-exist.

Thinking out loud: 1. "Phone numbers should be E.164 — `+91XXXXXXXXXX` — at storage." 2. "Display can strip the `+91` for Indian audiences if the product wants that." 3. "Until normalised, lookups must canonicalise both sides."

The diagnosis: No write-time normalisation contract. Every signup path (web, mobile, partner) injects its own format.

The fix:

```
-- Read-side immediate workaround
SELECT *
FROM users
WHERE regexp_replace(phone, '^[0-9]', '', 'g')
      IN ('919876543210', '9876543210', '09876543210');

-- Write-side proper fix: a stored canonical column
ALTER TABLE users ADD COLUMN phone_e164 VARCHAR(16);
UPDATE users
SET phone_e164 = '+91' || RIGHT(regexp_replace(phone, '^[0-9]', '', 'g'), 10);

CREATE UNIQUE INDEX ux_users_phone_e164 ON users(phone_e164);
```

Why this works: The canonical column is the single source of truth; lookups, dedup, and OTP routing all go through it.

What NOT to say: - "Validate phone format with a regex on the way in." — necessary but not sufficient; historical data still wrong. - "Store every variant in an array." — increases write complexity, doesn't fix lookups.

Common follow-up: "How would you handle a 9-digit Indian landline vs a 10-digit mobile?"

Scenario490 · Email Case Sensitivity — RFC vs Practice

Tags: Difficulty: Medium · Asked at: Stripe-India, Razorpay, most SaaS · Dialect: vendor-agnostic

The scenario:

A user signs up with `Abhi@example.com`. Tries to log in next week with `abhi@example.com`. System says "no account found." Support escalates after the third such ticket of the day.

```
SELECT * FROM users WHERE email = 'abhi@example.com';
-- 0 rows — but the row stored 'Abhi@example.com'.
```

Expected: Login works regardless of capitalisation. **Actual:** Case-sensitive comparison; user is locked out of their own account.

Thinking out loud: 1. "RFC 5321 says the local-part may be case-sensitive; the domain is not." 2. "In practice, virtually no mail server treats it as case-sensitive — Gmail, Outlook, AWS, all case-fold." 3. "So storing case-sensitive emails creates a class of self-inflicted bugs with no upside."

The diagnosis: The application stored email as-typed and compares case-sensitively. The RFC technically permits this on the local part, but real-world mail providers fold case. Users assume case-insensitive equality.

The fix:

```
-- Add a canonical lower-cased column with a unique index
ALTER TABLE users ADD COLUMN email_canonical VARCHAR(254) GENERATED ALWAYS AS (LOWER(email)) STORED;
CREATE UNIQUE INDEX ux_users_email_canonical ON users(email_canonical);

-- Lookups
SELECT * FROM users WHERE email_canonical = LOWER('abhi@example.com');
```

Why this works: Equality is now defined on the canonical form. Indexing the generated column keeps lookups fast.

What NOT to say: - "We're respecting the RFC." — technically true, practically harmful. - "Force users to remember the original case." — UX-hostile, doesn't reflect reality of how email works.

Common follow-up: "What about `dots-and-plus-signs` aliasing — `abhi+work@gmail.com` vs `abhi@gmail.com`?"

Scenario491 · Address Normalisation Issues

Tags: Difficulty: Hard · Asked at: Logistics/delivery apps, Swiggy, Dunzo, Porter · Dialect: vendor-agnostic

The scenario:

The address table is free-text. Repeat-customer detection thinks the same household is three different households because their address is typed slightly differently each delivery. Marketing's "first-time customer" coupon is being abused.

```
-- Three rows that are the same place
'12, MG Road, Bangalore - 560001'
'12 M.G. Road, Bengaluru, 560001'
'No. 12, Mahatma Gandhi Road, Bangalore 560001'
```

Expected: Same household recognised. **Actual:** Three "households" in the system.

Thinking out loud: 1. "Address normalisation is a non-trivial NLP problem — but you can get 80% with PIN + door number + structured fields." 2. "The schema should split into structured fields: door_no, street, locality, city, state, pincode." 3. "Same-household match can then key on (door_no, pincode) or (geocoded_lat_lng rounded)."

The diagnosis: Free-text addresses cannot be reliably joined or deduped. You need structured fields at minimum, and ideally a geocode reference at the row level.

The fix:

```
ALTER TABLE addresses
  ADD COLUMN door_no VARCHAR(20),
  ADD COLUMN street VARCHAR(120),
  ADD COLUMN locality VARCHAR(120),
  ADD COLUMN city VARCHAR(80),
  ADD COLUMN state CHAR(2),
  ADD COLUMN pincode CHAR(6),
  ADD COLUMN geo_lat DECIMAL(9,6),
  ADD COLUMN geo_lng DECIMAL(9,6);

-- Household match: door + pin OR geocode within 25 m
SELECT a1.user_id, a2.user_id
FROM addresses a1
JOIN addresses a2
  ON a1.user_id <> a2.user_id
 AND a1.pincode = a2.pincode
 AND LOWER(TRIM(a1.door_no)) = LOWER(TRIM(a2.door_no));
```

Why this works: Structured fields make equality meaningful. Geocoding is the gold standard for "same place" matching; PIN + door is a pragmatic fallback.

What NOT to say: - "Compare with full-text search and a similarity threshold." — works for ~70% but false-positives on common addresses. - "Trust the user's claim that it's a new household." — bypasses the coupon-abuse problem.

Common follow-up: "How would you build the migration from free-text to structured at scale?"

Scenario492 · NULL Counted As Zero In A Downstream System

Tags: Difficulty: Medium · Asked at: BI-heavy interviews · Dialect: vendor-agnostic

The scenario:

SQL says `AVG(rating) = 4.6` . An Excel export of the same data says `AVG = 3.2` . The finance team has been making decisions off the Excel number.

```
SELECT AVG(rating) FROM reviews;
-- 4.6 in SQL - NULL ratings excluded from the average.
-- 3.2 in Excel - NULLs read as 0 by the BI exporter, pulling the mean down.
```

Expected: Same number in both systems. **Actual:** Excel coerced NULLs to 0; the average is wrong by a lot.

Thinking out loud: 1. "SQL `AVG(col)` ignores NULL — denominator and numerator both skip NULL rows." 2. "Many BI exporters / Excel readers treat empty cells as 0, not NULL." 3. "So the downstream system computes `SUM/N` where N includes the NULL-as-zero rows."

The diagnosis: Semantic mismatch at the SQL-to-Excel boundary. SQL's NULL means "unknown"; Excel's empty cell often gets coerced to 0 in numeric formulas. The averaging behaviour diverges accordingly.

The fix: At the export boundary, decide explicitly: - If "no rating" means "not rated yet", export NULL as text "N/A" so Excel doesn't coerce. - If business says "no rating = 0", make that explicit in SQL: `AVG(COALESCE(rating, 0))` — and accept the consequence.

Best: produce an exported column for the user with the explicit denominator: `AVG_rated` , `COUNT_rated` , `COUNT_unrated` .

Why this works: The downstream consumer can't silently misinterpret what they're shown when the columns force the intent.

What NOT to say: - "Fix Excel." — not yours to fix. - "Filter out NULLs in the export and don't tell anyone." — hides the problem and shifts the mean differently.

Common follow-up: "What if some reviewers rated 0 deliberately? How do you tell that apart from 'not rated'?"

Scenario493 · NULL Excluded From COUNT(col) Silently Changing The Denominator

Tags: Difficulty: Medium · Asked at: Most data interviews · Dialect: vendor-agnostic

The scenario:

A funnel report shows "conversion rate = paid / signed_up". Conversion has been climbing steadily and the team is celebrating. You spot a new column being added to the signed_up count's denominator that's silently shrinking it.

```
SELECT COUNT(email)          AS signed_up,    -- ignores NULL emails
       COUNT(*) FILTER (WHERE plan = 'paid') AS paid
FROM users;
```

Expected: `signed_up` = total users. **Actual:** `signed_up` excludes users with NULL email — the denominator dropped after a recent schema change that made `email` optional.

Thinking out loud: 1. "`COUNT(col)` counts non-NULL values of `col` . `COUNT(*)` counts rows." 2. "If `email` became nullable last sprint, `COUNT(email)` quietly drops those users." 3. "Conversion rate `paid / signed_up` goes up because the denominator shrunk, not because more people paid."

The diagnosis: The denominator definition was tied to a specific column's non-NULL count. When that column's semantics changed, the metric changed without any code change to the metric itself.

The fix:

```
SELECT COUNT(*)                AS signed_up,
       COUNT(*) FILTER (WHERE plan = 'paid') AS paid,
       COUNT(*) FILTER (WHERE plan = 'paid')::float
       / NULLIF(COUNT(*), 0)    AS conversion_rate
FROM users;
```

And document: "conversion = paid users / all rows in users". Any change to user definition triggers a metric review.

Why this works: `COUNT(*)` is decoupled from any column nullability change. The metric definition is now stable against schema drift.

What NOT to say: - "Add `WHERE email IS NOT NULL` ." — same bug, written more explicitly. - "Lock the schema." — doesn't address the underlying brittleness.

Common follow-up: "What's the difference between `COUNT(*)` , `COUNT(1)` , and `COUNT(col)` in execution and semantics?"

Scenario494 · NULL In A Foreign Key — Unknown vs Not-Applicable

Tags: Difficulty: Hard · Asked at: Schema-design interviews · Dialect: vendor-agnostic

The scenario:

The `orders` table has `coupon_id` as a nullable FK to `coupons`. Reporting can't agree: some queries treat NULL as "no coupon used", others treat it as "coupon was used but is missing in the table." Two different interpretations produce two different revenue impact numbers.

```
-- "Orders that used a coupon"
SELECT COUNT(*) FROM orders WHERE coupon_id IS NOT NULL;
-- Some teams say: that's orders with known coupon.
-- Others say: NULL coupon_id means we lost the coupon link.
```

Expected: One unambiguous meaning of NULL. **Actual:** NULL is being asked to carry two semantic meanings.

Thinking out loud: 1. "NULL in a nullable FK can mean either 'not applicable' (no coupon used) or 'unknown' (we don't know which coupon, but one was used)." 2. "You can't tell them apart from the value alone." 3. "Schema must make the distinction explicit — separate boolean or sentinel value."

The diagnosis: A single NULL value is being asked to encode two different business concepts. Without explicit modelling, every query has to guess which one was meant.

The fix:

```
ALTER TABLE orders
  ADD COLUMN used_coupon BOOLEAN NOT NULL DEFAULT FALSE;

-- Backfill: assume the historical NULLs = "no coupon used"
UPDATE orders SET used_coupon = (coupon_id IS NOT NULL);

-- Now queries are unambiguous
SELECT COUNT(*) FROM orders WHERE used_coupon = TRUE;
-- Orders with a coupon, regardless of whether coupon_id is present.
```

Why this works: The boolean carries intent; the FK carries identity. Their NULL states are independently meaningful.

What NOT to say: - "Use a sentinel `coupon_id = 0` for 'no coupon'." — pollutes the coupons table and breaks FK semantics. - "Document the convention." — humans forget, queries don't.

Common follow-up: "How would you handle the same problem in a star schema where dimension keys can't be NULL?"

Scenario495 · NULL Handling In AVG — Denominator Changes

Tags: Difficulty: Medium · Asked at: Analytics interviews · Dialect: vendor-agnostic

The scenario:

Two queries that look identical produce different averages. One uses `AVG(col)`, the other uses `SUM(col) / COUNT(*)`. The numbers diverge whenever `col` has NULLs.

```
SELECT AVG(discount_amount)           AS avg_v1,  
       SUM(discount_amount) / COUNT(*) AS avg_v2,  
       SUM(discount_amount) / COUNT(discount_amount) AS avg_v3  
FROM orders;
```

Expected: All three equal. **Actual:** `v1 = v3` (NULLs excluded), `v2` lower (NULLs in denominator, not numerator).

Thinking out loud: 1. "`AVG(col) = SUM(col) / COUNT(col)` — NULLs excluded from both." 2. "`SUM(col) / COUNT(*)` includes NULL rows in the denominator but not the numerator — pulls the average toward zero." 3. "Both are valid for different questions; the bug is using one as if it were the other."

The diagnosis: SQL aggregates skip NULL inputs. `COUNT(*)` counts rows; `COUNT(col)` counts non-NULL values. Dividing the SUM by the wrong denominator changes what the metric *means*.

The fix: Be explicit about intent:

```
-- "Average discount among orders that had a discount"  
SELECT AVG(discount_amount) FROM orders WHERE discount_amount IS NOT NULL;  
  
-- "Average discount per order (treating NULLs as zero discount)"  
SELECT AVG(COALESCE(discount_amount, 0)) FROM orders;
```

Name the columns in the output to match the intent — `avg_discount_when_applied` vs `avg_discount_per_order`.

Why this works: The query name and the formula now agree on what's being asked. The reader can't accidentally misread.

What NOT to say: - "AVG is broken; let me reimplement it." — AVG is correct; the metric definition was unclear. - "Just always COALESCE to 0." — silently turns "no discount available" into "zero discount applied".

Common follow-up: "What does AVG return if every row is NULL?"

Scenario496 · COALESCE(col, 0) Hiding Real NULLs In Metrics

Tags: Difficulty: Medium · Asked at: Razorpay, Stripe-India · Dialect: vendor-agnostic

The scenario:

A KPI "average transaction fee" started displaying `0.00` for a chunk of recent days. Investigation shows the underlying transaction loader is failing for some integrations, leaving `fee_amount` NULL — but the metric definition uses `COALESCE(fee_amount, 0)`, which masks the loader failure as healthy zero-fee data.

```
SELECT AVG(COALESCE(fee_amount, 0)) AS avg_fee
FROM transactions
WHERE date_trunc('day', created_at) = CURRENT_DATE - 1;
```

Expected: Metric reflects actual fees or alerts on missing data. **Actual:** Metric reads "0.00" with no alarm, hiding a data pipeline failure.

Thinking out loud: 1. "COALESCE turns NULL into 0 — that's correct only if NULL means zero." 2. "Here, NULL means 'we don't have the data', which is a separate failure mode." 3. "The fix is to surface the NULL rate as its own metric, not paper over it."

The diagnosis: `COALESCE` was added without distinguishing between "no fee charged" (truly zero) and "fee unknown" (data missing). Both collapse to the same value, hiding the data quality issue.

The fix:

```
SELECT AVG(fee_amount)                AS avg_fee_known,
       COUNT(*) FILTER (WHERE fee_amount IS NULL)
       / NULLIF(COUNT(*), 0)::float    AS null_rate,
       COUNT(*) FILTER (WHERE fee_amount = 0)    AS zero_fee_count
FROM transactions
WHERE date_trunc('day', created_at) = CURRENT_DATE - 1;
```

Alert when `null_rate` crosses 1%.

Why this works: Each business concept gets its own metric. The dashboard shows when data is missing rather than treating missing as zero.

What NOT to say: - "COALESCE is fine, the alert is missing." — half-right, but the metric should still be honest first. - "Set DEFAULT 0 on the column." — locks the wrong semantics into the schema.

Common follow-up: *"Where else in the codebase might COALESCE be hiding data quality issues?"*

Scenario497 · ZERO vs NULL — When Each Is Correct

Tags: Difficulty: Medium · Asked at: Schema-design interviews · Dialect: vendor-agnostic

The scenario:

A nutrition app stores `calories_burned` per workout session. Some sessions show 0, others NULL. The product team can't tell what either one means and the in-app charts have unexplained gaps.

```
SELECT session_id, calories_burned FROM workouts;
-- 1, 250
-- 2, 0
-- 3, NULL
```

Expected: A clear convention. **Actual:** Three values in use with no clear semantic contract.

Thinking out loud: 1. "Zero = 'we tracked the workout, the user burned zero (e.g., a 30-second pause)' — measured zero." 2. "NULL = 'we didn't track, sensor didn't connect' — unknown." 3. "They're different concepts; mixing them creates ambiguity."

The diagnosis: The team conflated "measured zero" with "not measured". The right answer depends on the metric: - For *sums*: NULL is fine (excluded); zero would be misleading if it meant "unknown". - For *averages*: NULL excludes from denominator (probably right); zero pulls the mean down. - For *charts*: NULL = gap in the line; zero = data point at zero.

The fix: Define and enforce: - "Zero means measured zero (you sat still during a tracked workout)." - "NULL means unmeasured (no sensor / app crash / sync failure)."

Add a NOT NULL constraint to the column once historical data is cleaned, OR add a `measurement_status` column ('tracked', 'untracked', 'error') and require it always.

Why this works: Future queries can reason about each case explicitly: `WHERE measurement_status = 'tracked' AND calories_burned > 0`.

What NOT to say: - "Coalesce everything to zero, problem solved." — overstates the data, fabricates measurements. - "Coalesce everything to NULL." — loses measured zeros.

Common follow-up: *"How would you make this distinction visible to chart consumers without changing the schema?"*

Scenario498 · Empty String vs NULL — Pick One And Enforce

Tags: Difficulty: Easy · Asked at: Most data interviews · Dialect: vendor-agnostic, Oracle being the famous exception

The scenario:

A `middle_name` column has rows with `NULL`, rows with `' '`, and rows with `' '` (whitespace). All three mean "no middle name." Reports filtering "has middle name" return different counts depending on which check they use.

```
SELECT COUNT(*) FROM users WHERE middle_name IS NULL;           -- 4,000
SELECT COUNT(*) FROM users WHERE middle_name = '';             -- 2,500
SELECT COUNT(*) FROM users WHERE TRIM(middle_name) = '';       -- 2,700
SELECT COUNT(*) FROM users WHERE COALESCE(NULLIF(TRIM(middle_name), ' '), NULL) IS NULL; -- 6,700
```

Expected: A single canonical "no middle name" representation. **Actual:** Three representations, four different counts.

Thinking out loud: 1. "Empty string and NULL are technically different in standard SQL — Oracle is the exception, treating them as the same." 2. "The right answer for most apps: pick one (usually NULL) and enforce." 3. "Whitespace-only is a third bug — normalise on write."

The diagnosis: The write path has no normalisation contract. Forms submit `' '` for blank fields, the mobile app submits `NULL`, and an old import path submits `' '`.

The fix:

```
-- Normalise existing data
UPDATE users
SET middle_name = NULL
WHERE TRIM(COALESCE(middle_name, ' ')) = ' ';

-- Enforce going forward
ALTER TABLE users
ADD CONSTRAINT chk_middle_name_no_empty
CHECK (middle_name IS NULL OR LENGTH(TRIM(middle_name)) > 0);
```

Why this works: "No middle name" now has a single representation. Queries become predictable.

What NOT to say: - "It's fine, just handle all three in every query." — burden travels with every future query. - "Oracle does it right, let's emulate." — fine viewpoint, but most stacks aren't Oracle.

Common follow-up: "What does Oracle do differently here, and what's the implication for portable SQL?"

Scenario499 · ANSI vs MySQL Behaviour On NULL In `WHERE col != value`

Tags: Difficulty: Medium · Asked at: Dialect-aware interviews · Dialect: vendor-agnostic, MySQL/Postgres/Oracle differ on edge cases

The scenario:

A query `WHERE status != 'cancelled'` excludes cancelled orders but also silently excludes orders whose status is NULL. The product manager expected NULL-status orders to be included because "they're not cancelled."

```
SELECT COUNT(*) FROM orders WHERE status != 'cancelled';  
-- Excludes NULL-status rows along with 'cancelled' rows.
```

Expected: Anything not literally 'cancelled' — including NULL-status. **Actual:** NULL-status orders are dropped because `NULL != 'cancelled'` evaluates to NULL, not TRUE.

Thinking out loud: 1. "Three-valued logic: `NULL != 'x'` is `UNKNOWN`, not `TRUE`." 2. "WHERE keeps rows where the predicate is TRUE — UNKNOWN is treated as FALSE for filtering." 3. "To include NULLs, you need explicit `OR status IS NULL` or `IS DISTINCT FROM`."

The diagnosis: Standard SQL applies three-valued logic. `WHERE NULL = anything` and `WHERE NULL != anything` are both UNKNOWN, both filtered out. The intuition "NULL is not cancelled, so include it" doesn't match the language semantics.

The fix:

```
SELECT COUNT(*)  
FROM orders  
WHERE status IS DISTINCT FROM 'cancelled'; -- Postgres  
-- Or vendor-agnostic:  
WHERE (status IS NULL OR status != 'cancelled');
```

MySQL has the null-safe equality operator `<=>`, so `NOT (status <=> 'cancelled')` also works.

Why this works: `IS DISTINCT FROM` (or the explicit `OR`) treats `NULL` as a distinct value, so the predicate evaluates to `TRUE` for `NULL`-status rows.

What NOT to say: - "Just `COALESCE` the status to a sentinel string." — invents data and pollutes downstream queries. - "Set status `NOT NULL` and backfill 'unknown'." — sometimes right, but a schema change for a query bug is overkill.

Common follow-up: *"What does the same query do in Oracle, where empty string == NULL?"*

Scenario500 · `IS DISTINCT FROM` As The `NULL`-Safe Equality Fix

Tags: Difficulty: Medium · Asked at: Senior-level, dialect-aware · Dialect: PostgreSQL/SQL Standard with MySQL equivalents

The scenario:

A change-data-capture pipeline diffs `old` and `new` row versions to detect updates. It misses updates where a column went from `'A'` to `NULL` or from `NULL` to `'B'`, because the diff uses `old.col != new.col`. Customer-success has been chasing ghost bugs for a week.

```
SELECT *
FROM old_snapshot o
JOIN new_snapshot n USING (id)
WHERE o.email != n.email
      OR o.phone != n.phone;
-- Misses cases where one side is NULL.
```

Expected: Detect any change, including to/from `NULL`. **Actual:** Changes involving `NULL` slip through the diff.

Thinking out loud: 1. "`a != b` returns `NULL` if either side is `NULL` — three-valued logic strikes again." 2. "For change detection I need `NULL`-safe equality." 3. "That's exactly what `IS DISTINCT FROM` was designed for."

The diagnosis: The diff predicate uses standard `!=`, which is `NULL`-unsafe. `NULL`-to-value and value-to-`NULL` transitions evaluate to `UNKNOWN` and are filtered out, masking real changes.

The fix:

```
SELECT *
FROM old_snapshot o
JOIN new_snapshot n USING (id)
WHERE o.email IS DISTINCT FROM n.email
      OR o.phone IS DISTINCT FROM n.phone;
```

MySQL equivalent using the `<=>` operator:

```
WHERE NOT (o.email <=> n.email)
      OR NOT (o.phone <=> n.phone);
```

For wide tables, generate a row hash on both sides and compare hashes (still treating NULL explicitly).

Why this works: `IS DISTINCT FROM` returns FALSE when both sides are NULL, TRUE when one side is NULL and the other isn't, and otherwise behaves like `!=`. It is total — never returns UNKNOWN.

What NOT to say: - "Wrap every column in COALESCE." — works but invents a sentinel that might collide with real data. - "Use a hash and compare." — fine pattern, but the hash function still has to handle NULL consistently.

Common follow-up: *"How would you efficiently diff two snapshots of a 200-column table?"*
